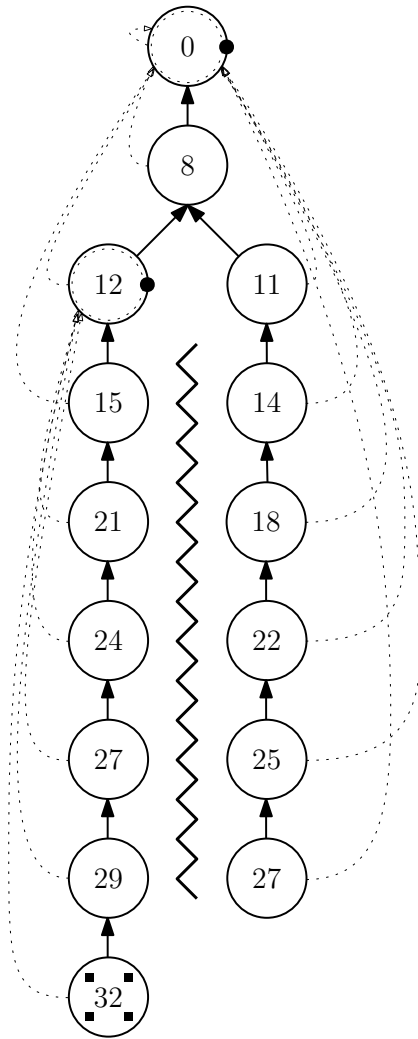
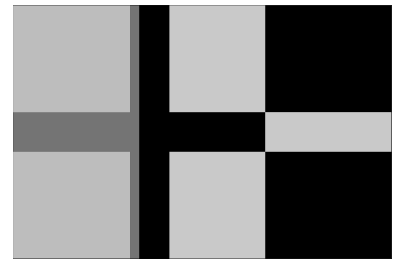


Secure Distributed Systems



An Introduction to Cryptography,
IT-Security, Distributed Systems, and
Blockchain Technology



Diego F. Aranha
Ivan Damgård
Jesper Buus Nielsen
Claudio Orlandi

Contents

1	Introduction (2025)	5
1.1	IT-Security	5
1.2	Cryptography	8
1.3	Distributed Systems	9
2	A Model of Distributed Systems (2026)[†]	29
2.1	Introduction	30
2.2	Basic System Model	30
2.3	Modelling Persistent Storage	32
2.4	Modelling Processes	32
2.5	Modelling Network Resources	37
2.6	Modelling Protocols	40
2.7	Specifying Safety Properties	40
2.8	Composition	43
2.9	Specifying Liveness Properties	43
2.10	Handling Crashes	44
2.11	Composition ^{††}	46
2.12	On Defining Eventuality ^{††}	49
3	Communication (2026)	51
3.1	Types of Communication	52
3.2	Naming	53
3.3	Let's Go!	55
3.4	The Internet	55
3.5	Programming Distributed Systems	62
3.6	Concurrency	66
3.7	Marshalling	71
3.8	Remote Procedure Calls	75
3.9	Logically Organising Connections	75
3.10	Unstructured Peer-to-Peer and Flooding	80
3.11	Structured Peer-to-Peer	85
4	Consistent Communication (2026)	87
4.1	Flooding Network	88
4.2	First In, First Out	91
4.3	Intermezzo on Modelling Distributed Systems I [†]	93
4.4	Causality	95
4.5	Intermezzo on Modelling Distributed Systems II [†]	111
4.6	Total Order	112

4.7	Chandy–Lamport Snapshot Algorithm	117
4.8	More Exercises	125
4.9	A Formal Simulation Argument†	133
5	Confidentiality (2025)	137
5.1	Confidentiality, Secret-Key Systems	138
5.2	Confidentiality, Public-Key Systems	147
5.3	Securely Implementing a Secure Channel*	155
5.4	Exercises	157
6	Authenticity (2025)	163
6.1	Cryptographic Hash Functions	164
6.2	Authenticity, Secret-Key Systems	167
6.3	Authenticity, Public-Key Systems	170
6.4	Replay attacks	174
6.5	Certificate Authorities	175
6.6	Implementing an Authenticated Channel*	177
6.7	Exercises	179
7	Synchronization (2026)	189
7.1	Physical Time	189
7.2	Clock Synchronisation	191
7.3	Round-Based Protocols	195
7.4	Exercises	199
8	Synchronous Agreement (2026)	201
8.1	Introduction	201
8.2	Defining Reliable Broadcast	205
8.3	Reliable Broadcast from a PKI	206
8.4	Lower Bound on Round-Complexity of Reliable Broadcast	214
8.5	No RB for Three Parties without Setup	219
8.6	Byzantine Agreement from Reliable Broadcast	222
8.7	Reliable Broadcast from Byzantine Agreement	222
8.8	Exercises	223
9	Key-Exchange and Key-Management (2025)	227
9.1	Authenticated Key-Exchange (AKE)	228
9.2	Diffie-Hellman Key-Exchange	231
9.3	Key-Management and User Authentication	237
9.4	Exercises	245
10	Asynchronous Agreement (2026)	251
10.1	Introduction	252
10.2	Defining Asynchronous Reliable broadcast	253
10.3	A CAP Theorem: No ARB for Three Parties	258
10.4	Intermezzo on Modelling†	261
10.5	Bracha: Asynchronous Reliable Broadcast from Authenticated Channels	263
10.6	Defining Asynchronous Byzantine Agreement	273
10.7	Implementing Asynchronous Byzantine Agreement using Coin-Flip	274
10.8	SVUP: Stabilizing Vote Updating Protocol, $n = 4$, $t = 1$	278

10.9	SVUP for General n and $t < n/3$	288
10.10	Interactive Coin-Flips*	290
10.11	Impossibility of Deterministic Asynchronous Byzantine Agreement*	290
10.12	Exercises	297
11	Privacy-Enhancing Technologies (2025)	305
11.1	Anonymous Communication	306
11.2	Deniability and Zero-Knowledge	309
11.3	Secure Multiparty Computation	312
11.4	Data Anonymization and Differential Privacy	316
11.5	Exercises	320
12	System Security Mechanisms (2025)	325
12.1	Introduction	326
12.2	The Trusted Computing Base	327
12.3	Protecting communication	329
12.4	Protecting software	332
12.5	Intrusion Detection	334
12.6	Access Control	335
12.7	Categorizing attacks	336
12.8	Attacks and Vulnerabilities	338
12.9	Exercises	346
13	Attacks and Pitfalls in Cryptography (2025)	349
13.1	Introduction	349
13.2	Randomness Generation	350
13.3	Choice of Algorithms and Parameters	354
13.4	Inadequate Trust Models	358
13.5	Certificates and Key Management	361
13.6	Covert and Side Channels	362
13.7	Exercises	364
14	State Machine Replication (2026)	367
14.1	Introduction	368
14.2	Core-Set Selection (CSS)	369
14.3	State Machines	371
14.4	State-Machine Replication (SMR)	374
14.5	SMR from CSS	376
14.6	Avoiding Total-Order	379
14.7	Reading from just one Server	386
14.8	Client-Centric Consistency	388
14.9	Exercises	390
14.10	Eventually Synchronous Implementation of Total-Order Broadcast*	397
15	Crypto Currencies* (2025)	407
15.1	Crypto Currencies*	407
15.2	Smart Contracts	408
15.3	Oracles	409
15.4	Energy	409
15.5	Building Trust from Distrust	410

16 Nakamoto-Style Consensus* (2025)	411
16.1 Blockchains: Nakamoto Style Consensus	412
16.2 Distributed Lotteries: Proof-of-Work versus Proof-of-Stake	415
16.3 A Note on Digital Identity: Parties versus Public Keys	417
16.4 Synchronous Clocks	417
16.5 Flooding System	417
16.6 Lottery System	418
16.7 Growing a Tree	420
16.8 Understanding Tree Growth	425
16.9 Making it a Total-Order Broadcast (TODO)	434
16.10 Exercises	434
17 Back Material	439
17.1 List of Exercise	443

Preface

This is the fifth edition of a book on security and distributed systems. It was first written in spring of 2018 to be used in fall 2018 in the first edition of the course Security and Distributed Systems at Aarhus University. It was heavily rewritten in spring 2019 based on the input from the first version of the course and lightly rewritten in springs 2020, 2021, and 2022 based on further teaching experience. The parts on security are to some extent reused from a previous course on security at Aarhus University, but have been rewritten and updated. The parts on distributed systems have been written from scratch, but are based on experiences from teaching distributed systems for a decade and several decades of research on provably secure multi-agent systems.

In the security part of the book, we will take a “bottom-up” approach: we first look at some technical solutions that are often part of the design of security mechanisms, including things like cryptography, network security protocols, access control, etc. Then we look at some higher level concepts, such as the security policies. Finally, we take a look at some of the reasons why systems fail in real life – in the language from this section, cases where either the threat model failed to capture the actual attacks, or where the security mechanisms failed to enforce the policy.

In the distributed systems part of the book, we have chosen to focus mainly on the so-called Byzantine agreement problem. The setting for this problem is as follow: There are some servers in a distributed system connected only by point-to-point channels. Each of them has an input which is *a priori* not known to the other servers and which might differ among the servers, for instance a vote in an election. The servers are now to agree on a valid common value, say the candidate with most votes. The problem is made hard by the assumption that we assume that some fraction, for example 33%, of the servers are evil and willing to deviate from the protocol to make the honest servers make a wrong choice or disagree on their choice.

Distributed systems is a huge area, and any attempt to cover the whole ground would lead to a superficial treatment. Thinking correctly about the security of distributed systems requires a change of mindset, which in our experience is not achieved by a superficial treatment. It is necessary to get into some of the technical details of some systems and understand why they are designed as they are. Decades of experience in teaching distributed systems has revealed that studying the Byzantine agreement problem is a good choice, as it crystallises the hard problems of secure distributed systems into a problem which is easy to study. Our hope is that the insights gained by studying the Byzantine agreement problem will give a change of mindset which will allow the reader to understand, design, and implement many other secure distributed systems.

Acknowledgements

We would like to thank the following people for helpful feedback on earlier versions of the book: Rebekah Hannah Mercer, Nikolaj Sidorenko, Mark Jensen, Kira Kutscher, Matias Frank Jensen,

Mark Simkin, Simon Holmgaard Kamp, Adam Blatchley Hansen, Peter Frands Frandsen, Kristoffer Strube, Magnus Holm Brunbjerg, Mathias Hall-Andersen, Jonathan Emil Hansen, Louise Mathiasen, Damiano Abram, Sebastian Kolby, Søren Eller Thomsen, Anders Kallesøe, Hamidreza Khoshakhlagh, Louise Mathiasen, Simon Yde Rasmussen.

Chapter 1

Introduction (2025)

Reading Guide *This chapter introduces all the concepts that will be covered in more detail later in the course. The purpose is to give you an idea what is coming up and how the concepts relate. Later chapters then zoom in on the individual concepts. The chapter is curriculum and might be queried at the oral exam.*

Contents

1.1 IT-Security	5
1.1.1 Some basic concepts	5
1.1.2 Security Policies	6
1.1.3 Threat Model	7
1.1.4 Security Mechanisms	7
1.1.5 Is it secure?	7
1.2 Cryptography	8
1.2.1 Confidentiality versus Authenticity	8
1.2.2 Unconditional versus Computational security	9
1.3 Distributed Systems	9
1.3.1 What is a Distributed System?	9
1.3.2 Middleware	10
1.3.3 Why Distributed Systems?	12
1.3.4 Replication and Consistency	14
1.3.5 Types of Failure	17
1.3.6 Security: A Rose by any Other Name Would Smell as Sweet	20
1.3.7 Specifying and Modelling Distributed Systems	21
1.3.8 Synchrony in Distributed Systems Models	24
1.3.9 Thinking like a Cryptographer in Distributed Systems	26
1.3.10 Canonical Goals	26

1.1 IT-Security

1.1.1 Some basic concepts

The subject of security is extremely difficult to define in a precise way. However, most people seem to agree that when it comes to security of IT systems, there are three fundamental aspects, sometimes referred to as *security objectives*:

Confidentiality: we want systems where information does not leak to people that should not have access. This applies at any time, i.e., whether the data is being stored, processed or transmitted.

Authenticity: we want the data in our systems to be authentic, i.e., that it has not been tampered with by people not authorized to do so. When this applies to the actual data, this property is sometimes called *data integrity*. More broadly, authenticity can also refer to beliefs about the data: common examples include the identity of the sender, the date of creation, etc. For instance, authenticity might mean that a user should not be fooled into believing that a document comes from a particular source when this is not the case.

Availability: we want systems that work when they are supposed to, i.e., legal users of the system should be able to get their data when they need them.

These objectives are just intended to be loose descriptions of basic user demands on a system. They are not linked to particular solutions, in fact there are many technically different ways to reach the goals, offering different levels of security. It should also be noted that if IT security is a fuzzy concept, the terminology of the field is even fuzzier, so you may well find other texts where the terms defined here are not used in exactly the same way. This appears to be something one simply has to live with. In this book, however, we will try to be consistent.

1.1.2 Security Policies

For any realistic situation, one needs a description of what security objectives we are after. This needs to be much more precise than just talking about confidentiality, authenticity and availability, and it is important to realize that this comes first, i.e., we have to define the objectives before we start thinking about how we are going to reach them. Such a description is called a *security policy*. In a formal, abstract model, this can take the form of specifying that certain states of the system are unsafe and should be avoided, while the others are defined to be secure. In a programming language environment, it can take the form of a set of rules for what different programs or software components are allowed to do, what data they can access, etc. When systems involve human beings as active participants, the security policy is often much less precise, and is formulated in a language that makes immediate sense to humans. While this is good from the point of view of having people understand what the goal is, the danger is that different people interpret the policy in different ways, leading to a multitude of security problems in practice.

Summarizing, a security policy, whatever form it has, is basically an attempt to characterize the events we do not want to see in a system, separating them from the allowed behavior.

Depending on whom you talk to, or which book you read, a security policy may include more than this, namely it does not only describe security objectives, it may also describe some strategies or procedures aimed at ensuring that the security objectives are reached. For instance, a company may have a security policy saying:

“This policy aims to ensure that virus attacks on our machines are prevented. All machines we use in the company must therefore use virus protection software. A single person should be made responsible for every machine, and this person must check at least every month that the virus scanner is updated”.

This somewhat blurs the distinction between security policies and other concepts in the area. But usually, high-level procedures such as the one above are thought of as part of the security policy, while lower level technical solutions are referred to as security mechanisms (see below).

Let us finally emphasize the importance of being as concrete and precise as possible, even when phrasing high-level security policies in human language. Consider the following example:

“This company takes security very seriously. We realize that virus attacks are a serious problem and we will take every possible measure to prevent them”

This is a political statement and not a security policy! Notice the differences between this and the above. Even though the texts are so short, some basic properties of a proper security policy can be identified: it defines reasonably precisely the system we are talking about and what the goal is.

1.1.3 Threat Model

Any system in real life can be attacked. An attacker can be an honest but careless user who, for instance, happens to release a piece of secret data. The attacker can be a malicious user, such as a bank employee who reprograms the system to send money to their own account. An attacker can be an intruder, such as a hacker who breaks in and takes control over a machine. And finally, the attack can come from nature itself, for instance when machines malfunction, causing loss of data.

It is almost never feasible to design a system secure against *any* attack, or if it is, the resulting system is too expensive or too slow. We therefore need a *Threat model*, that defines which attacks we are going to worry about. Whether a given system is secure or not can depend dramatically on which threat model is considered. Imagine, for instance, a multi-user computer system that stores and handles passwords internally with a very high level of security. However, the system administrator keeps a note with the administrator password on their desk. If the threat model considers only attacks that users can carry out from their own workstation, the system may be perfectly fine (and this may be reasonable if there is a high degree of physical security in the building, so that only the administrator can enter their own office). But if the threat model includes attacks where you gain undetected entrance to the administrator’s office (sometimes known as *the evil maid attack*), the system is totally insecure.

1.1.4 Security Mechanisms

Given a threat model and security policy, in order to ensure that the behavior of a system follows the security policy when subjected to the attacks identified in the threat model we need certain *security mechanisms*. These are designed to (hopefully) prevent attackers from forcing unwanted events to happen. A security mechanism can use any technique that will help us reach the goal: physical locks, ID cards, password protection, virus checkers, cryptography, etc.

For instance, if the security policy of a hospital says that only doctors should have access to the medical records of patients, encrypting the hospital’s patient database can be part of a security mechanism enforcing this policy. Another part may be a password-based scheme to ensure that only users registered as doctors can make the system decrypt data from the database.

1.1.5 Is it secure?

We have now arrived at an important question: given that we have defined a security policy, a threat model and that we have designed security mechanisms to enforce it: will the real system actually behave according to the policy? Or, put more directly, can the system be successfully attacked? This is perhaps *the* most important question in IT security, and unfortunately also the most difficult question to answer.

We first stress that security cannot be determined in an experimental fashion. Of course you could (and probably should) test whether your system is vulnerable to known attacks (this is also known as *penetration testing* and you can often hire specialised companies to try to “hack” into your system to pinpoint possible security holes before a malicious attacker does so). However even if your system is not vulnerable to any known attack, how do you know that it will not be vulnerable to some new, unknown attack?

This is why modern cryptography (and many other fields of IT security) relies on the approach known as *provable security* i.e., arguing about the security of a system by the means of formal mathematical proofs.

In provable security, we usually try to make a mathematical model of the system we want to analyze, formally define the threat model (or the class of adversaries we are trying to protect against), and finally try to prove a theorem stating that the system never shows a behavior we do not want.

However, this is often not as easy as it seems, and provable security often encounters one of the following two problems:

- The threat model is so general that we capture (almost) all possible attacks and behaviors. In this case, it is generally speaking very hard or impossible to prove anything. In several cases, it is even known that we will be up against an undecidable problem, i.e., it is known that we *cannot* prove security.
- The threat model is restricted sufficiently so that we can indeed prove that the system is secure. But then, of course, we do not know what happens if the attacker uses a strategy that is not captured in the model. Put differently: a very restricted threat model is less likely to reflect the risks that are really out there.

This is of course an unfortunate state of affairs. However, in this course, we will take a practical point of view: first, security is clearly needed in real life, and the fact that the theory is incomplete is no excuse for doing nothing about security. Second, the fact that the theory is incomplete does not mean it is useless! A proof of security in a restricted threat model can still tell us that a certain class of potential security holes have already been closed, so that we can focus our attention on other attacks.

1.2 Cryptography

This section gives a very informal introduction to some basic concepts from cryptography.

It should be apparent from the introduction that cryptography is not the only technique that is needed to obtain secure systems. In fact one can argue that cryptography can *never* be the complete answer to security problems – we return to this later. Nevertheless, cryptography is often an essential part of the security mechanisms we need.

Basically, the entire crypto-world can be divided in two orthogonal ways. The first type of division relates to the *kind of problem* we are trying to solve. The second kind relates to the type of security we obtain.

1.2.1 Confidentiality versus Authenticity

The two basic security objective we are after with a cryptographic solution may be either *confidentiality* or *authenticity*, objectives that were already described in the overview above. Cryptology is most often not useful in trying to ensure availability, since this goal is much more dependent on the way data is handled and stored physically. We will discuss availability in

more detail in the chapters on distributed systems, for instance when we cover replication and the so-called CAP theorem.

It is important to understand that the two kinds of security objectives really are different and call for different technical solutions. Simply encrypting all the data you send is *not* always the solution, and is in some cases even counter-productive!

Thus, in a given real-life scenario, you *must always* find out first whether you are dealing with an authenticity or a confidentiality problem (or both), and then choose the appropriate cryptographic technique for solving the problem.

1.2.2 Unconditional versus Computational security

The second type of division relates to the *kind of security provided*. Cryptographic constructions may provide *unconditional* or only *computational* security. For instance, a method for encrypting data may offer such strong protection against eavesdropping that even a hacker with unlimited computing power will not be able to break the encryption. Or it may be the case that the system could in principle be broken, but only by spending a completely unrealistic amount of computing power (and hence also a completely unrealistic amount of real time). In the latter case, we talk about computational security.

The unconditionally secure systems are generally speaking not very interesting from a practical point of view, at least when “practical” is to be “commercially interesting”. We know this due to some basic theory which tells us that to provide – for instance – unconditional confidentiality, we will need to either supply both sender and receiver with very long strings of shared secret bits, or rely on natural noise occurring on communication channels, or rely on non-standard ways of communication, such as quantum communication. With current state of the art, this is either expensive or hard to control in practice. So we concentrate in the following on computational security.

The world of computational security can be further subdivided, namely into *secret-key* and *public-key* systems. The meaning of this will hopefully become clear in following chapters.

1.3 Distributed Systems

1.3.1 What is a Distributed System?

A distributed system is a collection of computers which collaborate to solve a joint problem and try to appear as one consistent system towards the user or application layer. In this book we only concern ourselves with distributed systems with the following properties:

Geographic separation: The machines in the system are distributed over a large geographic area. They are typically also located in different administrative domains with different system administrators.

No common physical clock: The machines do not share a physical clock, so they cannot coordinate via some notion of global time. They might have local physical clocks, but these are near impossible to synchronise perfectly and tend to drift apart if nothing is done to synchronise them. This is an issue we discuss in more detail in Chapter 8.

Coordination is via message passing: The machines coordinate via passing messages. In this note, we will typically assume that communication is via TCP/IP. Because of the geographic separation it is hard to give an estimate on how long it takes a messages to be delivered. For that reason it can be hard to distinguish a slow message from a message that was lost or never sent.

Heterogeneous: The machines have different hardware, different operating systems and different computational power and connectivity. Some machines might be powerful servers in a server park and others might be laptops, cell phones and even cheap sensors with limited network and battery time.

An example is the Dropbox system which consists of about 10,000 servers used by Dropbox and all the millions of machines of the users.

Some computing systems falling outside the above class, but which still have flavours of distribution are multicore computers and cluster computing. Modern computers typically have many cores. They communicate via shared memory and share a physical clock. A step up from this are cluster computers which are designed for parallel execution of computationally intense tasks. In this case, a large number of homogeneous computers are connected via a high speed network and controlled by a master node. These machines share one operating system and administrator. The main problems encountered in these settings are best covered in a course on parallel computing.

1.3.2 Middleware

A distributed system is often provided via a middleware layer running on top of the different operating systems at the physically separated machines. The overall goal is to make these machines appear identical to software that interacts with them only through the middleware. In this way, the complexity of heterogeneity and distribution is confined to the middleware layer rather than being spread throughout the application logic.

More precisely, middleware aims to provide a uniform abstraction of a distributed system. The most important desired properties of such middleware are the following.

Openness and interoperability The middleware exposes well-defined interfaces (APIs) that specify the observable behavior of the system independently of its implementation. This allows independently developed components to interoperate and enables middleware implementations to be replaced or upgraded without affecting applications, as long as the interface contract is preserved. Open standards and published interfaces further promote portability and avoid vendor lock-in. A classical example is the SMTP e-mail system, which enables e-mail exchange across a wide range of operating systems and implementations.

Transparency Middleware should hide both system heterogeneity and the fact that the system is distributed. This includes hiding differences between operating systems, as well as distribution aspects such as location, replication, concurrency, and partial failures. For example, the Dropbox system allows users to access the same files across different computers and mobile devices, giving the illusion that the same files are locally present everywhere, despite very different underlying storage mechanisms.

Scalability The middleware should continue to operate efficiently as the number of users, machines, or geographic locations increases. Achieving scalability typically requires avoiding centralized bottlenecks and supporting techniques such as replication, partitioning, and load balancing.

Fault tolerance Failures are inevitable in distributed systems. Middleware should detect failures and either mask them from applications or provide structured mechanisms for recovery. This may include retries, replication, failover, or reconfiguration in response to node or network failures.

Performance efficiency While middleware introduces additional abstraction layers, it should do so with acceptable overhead. The design must balance convenience and transparency against latency, throughput, and resource consumption.

Security Middleware often provides uniform mechanisms for authentication, authorization, and secure communication. Centralizing security at the middleware layer reduces application complexity and helps enforce consistent security policies across the distributed system.

Manageability and evolvability Middleware should support monitoring, configuration, and upgrades while the system is running. This is essential for operating large-scale distributed systems and for allowing systems to evolve over time without service disruption.

Common Types of Middleware

Middleware comes in many forms, each providing different abstractions and services. Below we outline several common types, together with a representative example for each. This is purely for illustration, not for memorization. For each type we also list more examples by name only. We encourage the interested reader to explore these systems and compare their programming models.

Remote Invocation Middleware This type of middleware allows applications to invoke operations on remote services as if they were local function calls. It handles communication, marshalling, and error handling. Example: gRPC, which provides strongly typed remote procedure calls over HTTP/2. Other examples include Thrift, Java RMI, and CORBA.

Message-Oriented Middleware (MOM) Message-oriented middleware supports asynchronous communication through message queues or publish/subscribe mechanisms, decoupling senders from receivers in time and space. Example: Apache Kafka, which provides durable, high-throughput event streaming. Other examples include RabbitMQ, ActiveMQ, and MQTT-based systems.

Data Access Middleware This middleware provides uniform access to distributed or replicated data stores, hiding issues such as data placement, replication, and consistency. Example: Redis, commonly used as a distributed cache or key-value store. Other examples include Cassandra, Memcached, and distributed SQL databases.

Transaction and Coordination Middleware This class of middleware provides coordination primitives such as locks, leader election, and atomic updates across multiple machines. Example: ZooKeeper, which offers coordination services based on replicated state. Other examples include etcd and Consul.

Service Infrastructure Middleware Service infrastructure middleware supports service discovery, routing, load balancing, and policy enforcement in service-oriented systems. Example: Envoy, a programmable proxy widely used in service meshes. Other examples include HAProxy, API gateways, and service mesh frameworks.

Object and Component Middleware (Old School) Earlier distributed systems often relied on distributed objects or components with managed lifecycles and interfaces. Example: CORBA, which aimed to provide language- and platform-independent distributed objects. Other examples include DCOM and Enterprise JavaBeans. While less common today, these systems remain historically important.

Stream Processing and Workflow Middleware This type of middleware coordinates long-running computations or continuous data streams, often with built-in fault recovery. Example: Apache Flink, used for large-scale stream processing. Other examples include Spark Streaming, Airflow, and Temporal.

Security Middleware Security middleware provides shared mechanisms for identity management, authentication, authorization, and secure communication. Example: OAuth 2.0, widely used for delegated authorization. Other examples include OpenID Connect, Kerberos, and secret-management systems.

Observability Middleware Observability middleware collects metrics, logs, and traces to help operators understand and debug distributed systems. Example: OpenTelemetry, which standardizes the collection of telemetry data. Other examples include Prometheus, Jaeger, and logging frameworks.

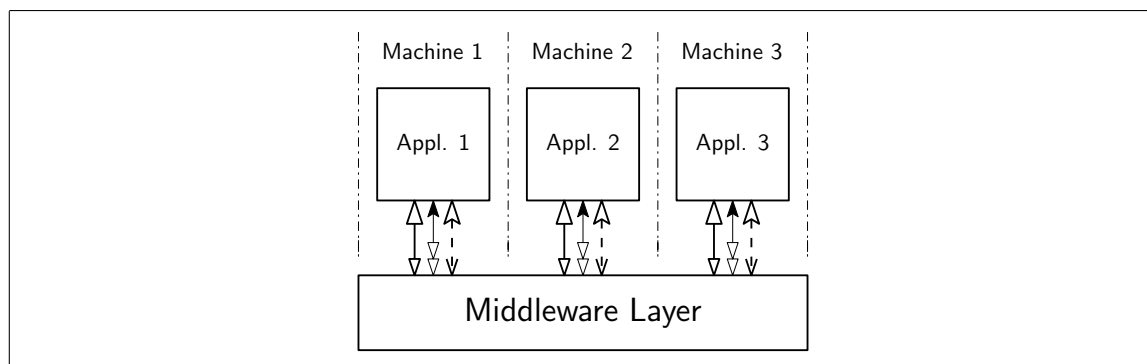


Figure 1.1: An illustration of how a good middleware layer should look like from the viewpoint of applications running on different machines. They all have some well-defined API to the system. The system looks like one coherent system to the applications. No details about the local operating system or how the middleware layer is implemented on the local machine is visible to the applications.

1.3.3 Why Distributed Systems?

Some of the main reasons for making distributed systems are:

Resource sharing: Often resources are inherently distributed and shared, like printers, large central databases or people who need to collaborate despite being geographically separated.

Fault tolerance: Having more machines gives the possibility that the system might keep operating if some of them are faulty. We might for instance replicate a resource like a file to make sure it is available even if one of the machines it was stored on has crashed. As an example Dropbox is replicating your files such that if any single server crashes no files are lost or become unavailable. This is an example of ensuring availability using replication.

Scalability / Load balancing: Having more machines that share the work can make the system faster. You can for instance store a very popular file on 100 different servers evenly distributed across the planet. If you implement a distributed system which allows

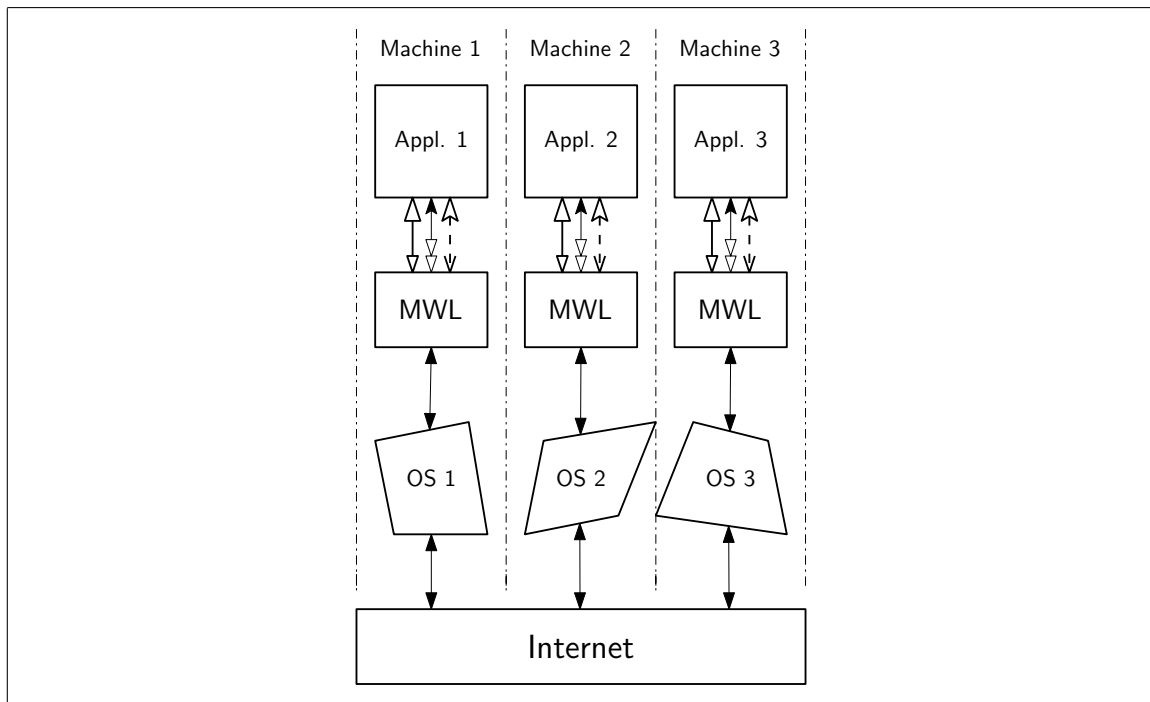


Figure 1.2: An illustration of how a middleware layer is actually implemented. On each machine there is a middleware application running, called MWL here, which via the local operating system and ultimately the Internet communicates with the MWL application on other machines.

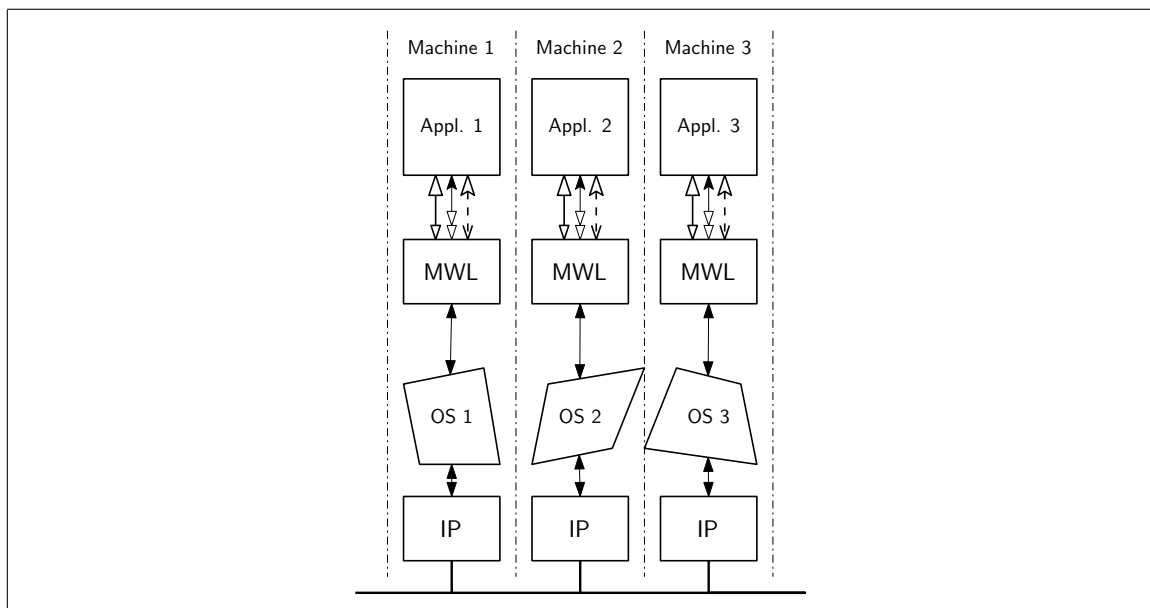


Figure 1.3: Of course the internet itself is an abstraction like a middleware layer. Via the TCP/IP layer it makes the internet look "the same" to the MWL programs on different operating systems. Each OS runs its own implementation which via a series of physical links talk to the TCP/IP implementation in other operating systems on other physical machines.

users to locate the closest server and retrieve the file from there, each server will get less workload and clients will get shorter access time.

Efficiency: It is often much more economically viable to implement a given task using a distributed system of cheap components than by constructing a single super-powerful or super-reliable machine which handles the task alone.

Autonomy: You might want your system to be autonomous in the sense that it is not controllable by a single person or organisation. This is the case for peer-to-peer file-sharing systems and the Bitcoin system which we will cover in later chapters. An autonomous system of course cannot run on a single physical machine. Otherwise, the owner of the machine could take over the system, or just cut the power cord. Neither can an autonomous system run inside a single organisation. Nor should the system run solely inside a single country, which might shut it down using legislation. To ensure that no entity could take over the system, it is necessary to set up a system where anyone can join and help run the system and where there is a large number of participants from all over the planet.

Example 1.1 (Replication for efficiency) Consider the following toy example for how to use replication for efficiency. Assume you can buy a cheap but crappy server that crashes on any given day with probability $\frac{1}{1,000}$. Assume the probability it crashes on a given day is independent of whether it crashes on any other day. Then the probability it crashes in a given year is about 31%. Assume that by buying a machine that costs 10 times as much you get a machine that crashes on a given individual day with probability only $\frac{1}{100,000}$. The probability it crashes in a given year is now about 0.4%. Here is a trick that will both save you money and give you higher reliability. Assume that we can make a distributed system consisting of three machines with the following characteristics: If at most one machine is crashed, then the system is still operational. If within a given day at most one machine crashes, then we have time to replace it with a fresh machine. Assuming machines crash independently, the only way the system can crash is that two machines crash on the same day. If you use three crappy machines, the probability this happens in a given year is about 0.1%. So we enhanced the availability of the system to be better than that of the expensive machine and only paid three times the price of a crappy machine, and much less than the super-duper machine would cost. ▼

1.3.4 Replication and Consistency

As we will see later, one of the main challenges in distributed systems is:

Consistency: When a resource is replicated in a distributed system it should be kept consistent across all replicas. For instance, if you make a change to a file on one location and later read the file on some other location, then you expect to see the change that you made. This requires that the change is propagated to other machines faster than you move and that other changes made to the same file by other clients on other machines are somehow handled in a consistent way.

Consistency is often hard even to define and even harder to implement. If a file is replicated across several servers, we cannot simply require that it is always the same on all replicas. A change will take time to propagate across the network. And even if we tried to make all replicas apply the change at the same physical time this cannot be done with perfect precision. One of the files will be written first and for a moment the system is inconsistent. Consistency is therefore more subtle to define. It is defined via a contract between the clients and the

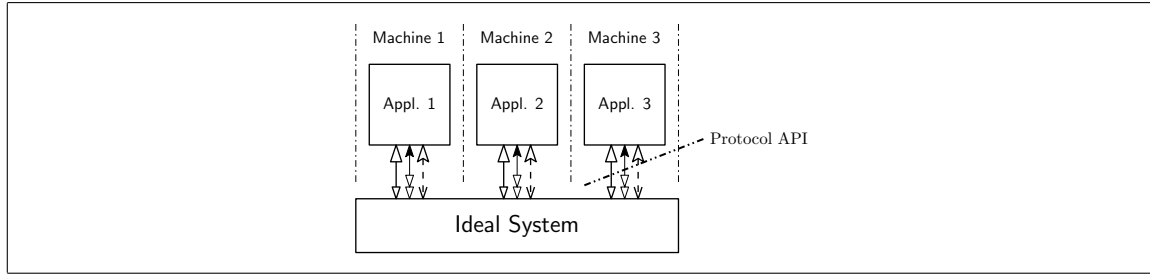


Figure 1.4: When defining what a system is supposed to do, we specify an ideal system which has the same API as the real system. It is programmed as a single object. We program it such that on any given inputs it gives the right outputs, in the right places at the right time. We think of it as a single object magically present at all sites at ones. Therefore the implementation does not have to take care of things like message passing or crashing machines. In implementing it one only focuses on making sure the input-output behaviour is as intended. It will for instance keep only one copy of a file and present that file to all users at all sites, maybe with some allowed delay. Since the ideal system is a specification of the intended behaviour, it is consistent by definition.

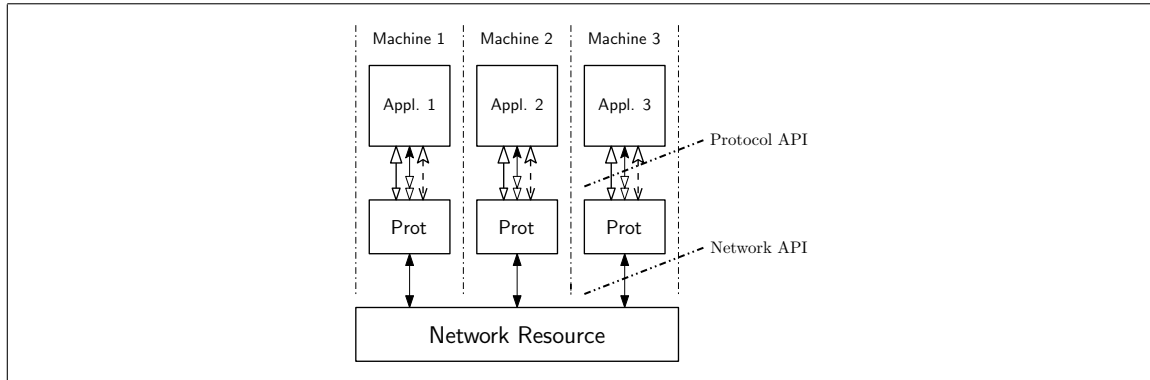


Figure 1.5: In the real system each machine has its own program and must communicate via some network resource, like the Internet. Here each machine might have its own copy of a file and try to keep them consistent via message passing over the network resource. One compares the real system to the ideal system by saying that any input-output behaviour of the real system is also possible by the ideal system. This argues that the real system does not do anything which is disallowed.

middleware. If the clients promise to behave in a particular way, then the values they receive from the middleware will be consistent according to some definition. Consistency is defined via the input-output behaviour of the distributed system and not via the particular internal representation. The specification is done in two steps. First the syntax is given, specifying which inputs and outputs the system can receive and give. Then the required relation between the inputs and outputs is specified. The last step can either be done by specifying an ideal system which has the intended input-output behaviour, or just be giving the intended relation between the inputs. We will see examples of both styles in this book.

Example 1.2 (Consistency in a replicated system) As an example of how to define and keep consistency consider the following system with three servers that try to store a file for a

single client. Initially the file is set to be the empty string ϵ . The system has a client which is the one that gets inputs and gives outputs.

First the syntax:

Read On input (`read`) the client eventually outputs (`read, f`).

Change On input (`change, C`) for some function C the client eventually outputs (`changed`).

The intended meaning of the read command is that the returned f is the current value of the file. The intended behaviour of the change command is that C is applied to the current value of the file: think of replacing the file by $C(f)$. We will now define this a bit more precisely.

First say that the input to the clients are legal if the next input is always given after the previous input is “returned”. For instance after having given input (`read`), you are not allowed to input (`read`) to the client again until after it outputs (`read, f`). We can now define consistency as follows. Consider any legal sequence of inputs to the client and let $C_1, \dots, C_n$ be all the change functions in this sequence in the order in which they appear. If the last input to the client is (`read`) and the client eventually outputs (`read, f`), then $f = C^n(\epsilon)$, where:

$$C^n = C_n \circ \dots \circ C_2 \circ C_1 .$$

Let us attempt a distributed implementation. We will use three servers. Each server stores a pair (t, f) , where t is a timestamp (which is simply an integer initially set to 0), and f is the file (initially set to the empty string). If at any point in time the values stored by the servers are $(t_1, f_1), (t_2, f_2), (t_3, f_3)$, then we define m to be the smallest $m \in \{1, 2, 3\}$ such that $t_m = \max\{t_1, t_2, t_3\}$. We call t_m the current timestamp and we call f_m the current file. Intuitively we just say that the current version of the file is the one with the highest timestamp.

If all machines work perfectly then it is easy to implement a system, so let us challenge ourself and consider a threat model where we assume that one machine might crash. The remaining two machines are expected to run honestly. To make a system that keeps working in this threat model, we make a system where the client can read or modify the file with access to only two servers. Here is how we implement the system:

Read On input (`read`) the client reads from any two servers that are available. More precisely, it could just try to read from all three and then use the first two replies to arrive. Call the received values (t_1, f_1) and (t_2, f_2) . If $t_1 \geq t_2$ it outputs (`read, f1`). Otherwise it outputs (`read, f2`).

Change On input (`change, C`) for some function C the client reads from any two servers that are available. Call the received values (t_1, f_1) and (t_2, f_2) . If $t_1 \geq t_2$ it lets $(t, f) = (t_1, f_1)$. Otherwise it lets $(t, f) = (t_2, f_2)$. Then it sets $t' = t + 1$ and $f' = C(f)$ and stores (t', f') on any two servers that are now available, not necessarily the same ones it read the file from. More precisely, it writes to all three servers, and when it hears back from the first two servers, then it outputs (`changed`).

We can now prove that the system is consistent. The trick is to prove the following invariant:

- After n change commands there exist a file f such that (n, f) is stored on at least two servers and $f = C^n(\epsilon)$;
- The third server stores a value $(n', \cdot)$ such that $n' \leq n$.

First assume the invariant is always true. Then it is easy to prove consistency. Namely, when the client reads from two servers it will get (n, f) and (n, f) or it will get (n, f) and (n', f') . In both cases it will output f since $n' \leq n$. What is left now is to prove the invariant: Clearly the invariant holds after 0 change commands as all three servers store $(0, \epsilon)$. Assume then that the invariant is true after $n - 1$ change commands. This means that at least two servers are storing $(n - 1, C^{n-1}(\epsilon))$ and the last server stores (n', f') with $n' \leq n - 1$. So the client will write back $((n - 1) + 1, C_n(C^{n-1}(\epsilon))) = (n, C^n(\epsilon))$ on two servers, maintaining the invariant. ▼

Exercise 1.1 (Generalising Replication to More Parties) We generalise the protocol in Example 1.2 to n servers. Consider the synchronous model.

1. Let r and w be positive integers. To read the file, read (t, f) from r servers and use the file with highest timestamp. To change a file, read from r servers and write back to w servers. Argue that this works if $r + w > n$. Note that this is a special case of the above example with $n = 3$ and $r = w = 2$. We call r the *read quorum* and we call w the *write quorum*.
2. We can generalize further. Assume the servers are named $0, \dots, n - 1$ and let $[n] = \{0, \dots, n - 1\}$. Let $2^{[n]}$ be the power set of $[n]$, i.e., the set of all subsets of $[n]$. Let $\mathcal{R} \subseteq 2^{[n]}$ be the so-called *read access structure*. Let $\mathcal{W} \subseteq 2^{[n]}$ be the so-called *write access structure*. To read, read (t, f) from all servers in any $R \in \mathcal{R}$. For instance, try to read (t, f) from all servers and let R be the set of servers you heard back from so far, say the fastest to respond. The set R might grow over time as slower servers respond. Wait until $R \in \mathcal{R}$. Then use the file with highest timestamp. To change, read from all servers in some $R \in \mathcal{R}$, as above, and then write to all servers in some $W \in \mathcal{W}$. What is the minimal condition on the pair $(\mathcal{R}, \mathcal{W})$ that will make the system consistent? ▼

1.3.5 Types of Failure

The main subject of a course on secure distributed systems is to construct systems which remain secure even in the presence of errors or failures. Constructing a system which works when the network is fast and delivers all messages, when all machines are present and everything runs correctly, and when no attacker is present and bandwidth is unlimited is fairly easy. But that is not how the world looks. Messages are lost, message delivery is slow, messages are tampered by adversaries; bandwidth is limited, machines are slow, physical clocks drift, machines crash, machines are taken over by hackers or subjected to denial of service attacks. Such deviations from the expected or normal behaviour are sometimes called failures. Making a distributed system which behaves correctly even in the presence of such failures is sometimes called *fault tolerance*.

There are several reasons why fault tolerance is mainly a subject of interest in distributed systems. First of all, as you add more machines, the probability that any one of them fails will go up, so you simply cannot afford to build a system which fails if any of the components fails. Also, if you are running on a single machine and the machine crashes there is little you can do. But distributed systems can have partial failures; for instance in a system with three machines when a single one crashes, it is no longer obvious that you cannot keep running. So you should try to make a system that does exactly that. Otherwise your competitors will do that and you will be out of business! Lastly, since the machines coordinate over a network, you will rely on even more machines (IP routers, DNS servers, ...) and will be subject to attacks by adversaries

on that network. You cannot ignore these possible network failures and attacks. Today these attacks are ubiquitous and persistent. If your system does anything useful, eventually someone will try to break it. Maybe someone who has an interest in the system breaking, but some people just also just like to see the world burn. And if you designed your system in an insecure way, the adversary will succeed.

Note that it is not always clear what constitutes a failure and what is considered as normal behaviour. The fact that a UDP package is dropped by the network is not really a failure, as it is within the specified behaviour of the UDP layer. You are expected to handle that yourself or use TCP/IP instead. The fact that a TCP/IP connection can be dropped is not really a failure either, maybe the machine in the other end was under such computational load that it could not handle the connection, and these things are expected to happen as load goes up and down. However, it still makes sense to divide the behaviour of the environment of the system into normal conditions and rare conditions. One then tries to design the system such that it works well and fast under fair-weather conditions, while ensuring that it handles the rare cases in a graceful manner, for instance by just getting slower or shutting down. It is essential to try to ensure that the rare cases do not result in errors which cannot be repaired later, like leaking data or handing out money from an ATM to a illegitimate recipient.

That means to specify what your system is supposed to do, you define the system's *specification*, or *input-output behaviour*, for each set of conditions that the network and machines involved may find themselves in. This defines the behaviour of the system under the given conditions. For instance: "When all machines are present and their clocks synchronised within 1 second and all messages are delivered within 1 second, then the system delivers a version of the stored file which is at most three second old. If one machine crashes and ..., then it takes 10 seconds to deliver. Under all other conditions no guarantees are provided."

We will now fix the terminology for some events that are considered rare. We will follow standard terminology and refer to them as failures.

Crash failure: A machine or process crashes, it stops running now and will never come alive again. There are three main models for this.

Fail-arbitrary: As the process crashes, the last message it sends is some arbitrary possibly faulty message.

Fail-silent: As the process crashes it sends out no message to any other process.

Fail-stop: As the process crashes it sends **Stop** to all other processes. This is an example of a theoretical failure type. It doesn't happen in practice, but as we discuss below, you can sometimes reduce real-life failure to these theoretical ones.

Crash-recovery failure: A machine or process crashes, and stops running. Later it comes alive again. Maybe someone accidentally turned it off and later turns it on again. Maybe it burned and is later replaced by a fresh machine by the local system administrator. The crash part of crash-recovery can come in the same three flavours as above defined for crash failures. When the machine comes back to life there is typically some re-entry protocol that allows it to catch up on what happened. Crash-recovery failures are harder than crash failures, as it is typically expected that when the machine comes alive again, it should start contributing to the system again, taking inputs and giving outputs. When a machine crashes and then recovers, it is assumed that all values in volatile memory (RAM) are lost. It is typically assumed that values on permanent memory (disk) are uncorrupted.

Reset failure: A machine or process is reset to some previous state. It, for instance, could forget that it already sent a given message, and so may send it again. This is another example of a theoretical failure type.

Timing failure: A machine does not run as fast as expected and so sends a message too late – or runs too fast and sends it too early.

Message delay failure: A sent message does not arrive within the expected time; the network is slow.

Byzantine process failure: A machine or process is replaced by another process running a possibly maliciously chosen algorithm. This can be due to programming error, hardware error or because the machine is taken over by a hacker. There are two main flavours:

Monolithic: If several machines are Byzantine corrupted then we assume that there is a central entity (often called the adversary) who controls and coordinates all of them. Think of this as a hacker who broke into all of them and has a network connection to all of them allowing it to orchestrate how they misbehave. We assume that the adversary's ability to coordinate is unlimited, so it is not restricted to using the available network resources. It can magically see the state of all corrupted machines and instantaneously instruct them to send nasty messages or do nasty stuff.

Local: Each Byzantine party had its code replaced by some malicious code, but they have no magical means for colluding, they have to do that via the network available to the honest correct machines.

Duplication failure: A message that should have been sent only once is sent multiple times.

Injection error: A message that should never have been sent was sent, perhaps sent over the network by an adversary trying to break your system.

Tampering failure: A message was changed while being sent, possibly by an adversary trying to break your system.

Omission failure: A message that should have been sent was never sent, or it was dropped on the network, or not received at the receiving end.

Leakage failure: A machine might leak information to the adversary that should have been kept secret, like the cryptographic keys it uses for securing its communication.

Example 1.3 (Reducing Between Error Types) As we mention above, there are tricks for reducing one type of error to some other type of error which is easier to handle. For instance, crash-recovery failure can be reduced to reset failure if you have permanent storage: now and then you write your current state to disk. If you crash and recover, you then retrieve the latest state from disk and start running from that state again. This protect against crashes as you can start running again. But it introduces reset errors as you might start running from an old state.

Reset errors can then be further reduced to duplication errors: you could make sure to write your full state to disk every time you are about to send some message m . After sending m , you write $(I \text{ sent}, m)$ to disk. If you crash and recover, read the latest state from disk. If running from this state would mean you send m , check the disk again. If there is no message $(I \text{ sent}, m)$ then send m again and write $(I \text{ sent}, m)$ to disk. Otherwise, run from the retrieved state but ignoring the instruction to send m . If you are extremely unlucky, it could happen several times

that you send m but crash before you can write $(\text{I sent}, m)$ to disk (maybe it is this attempt to write to disk which crashed the process). But this is the worst that can happen. And so crash-recovery errors have been turned into duplication errors.

As a last example, crash-silent errors can be reduced to crash-stop errors if you have precise clocks and a bound on message delivery time: each second each machine sends to each other machine a Ping message. When a machine stops sending Ping, then interpret it as the machine having sent Stop. ▼

Exercise 1.2 (Crash recovery) In this exercise you are asked to add crash-recovery resilience to the system from Example 1.2. Assume that there is a reliable way to detect when one of the three servers crashes. When this happens the local system administrator will restart it or replace it with a new server. In both cases it will be restarted with the state $(-1, \epsilon)$ so that it knows it has been crashed. Then the server will do some protocol where it talks to the other servers, called the *reentry* protocol. By definition this protocol ends when the server outputs *reentry*. During down-time the server does not respond to requests from the client. At the end of the reentry protocol the server is considered to be back in the group of servers and will again respond to the client as in Example 1.2. Describe a reentry protocol that will make the system consistent under the following conditions. Denote by the down-time interval of a server the time interval in which it is crashed plus the time interval it takes to reboot or be replaced plus the time interval it takes to run the reentry protocol. Your system should be secure as long as no two servers have overlapping down-time intervals. Notice that there is a trivial solution which makes the reentry protocol simply be an infinite loop. Try to do better. ▼

1.3.6 Security: A Rose by any Other Name Would Smell as Sweet

Different subfields of distributed systems use a number of different terms for the simple notion that a system does what it is supposed to do under some given failure model, and nothing more. Terms like Correctness, Consistency, Fault tolerance, and Security are used. And sometimes the failure model is called the Attack Model or the Adversary Model. To reduce confusion, we will fix a terminology. We will use attack model to denote the specification of the things that can happen to the system. So the attack model could for instance be “fail-stop crash failures”. To specify what a system is supposed to do, we need to specify for at least one attack model what the system is supposed to do. For instance, under the crash-recovery model where at most one out of three servers is down at a time, the system will deliver consistent files. In general we say that under some attack model $\mathcal{A}$ the system behaves according to specification $\mathcal{S}$. Then $(\mathcal{A}, \mathcal{S})$ is the full specification of the system’s guarantees. This is to some extent just a rephrasing of the notion of a threat model and security policy. But there are some qualitative differences. A security model tends to focus on the things that should be avoided, listing the bad things that are not allowed to happen: How the system is actually meant to operate in normal conditions is phrased in functional specifications. The above specification view instead specifies what the system *must* do even when under attack. Everything else is implicitly left as faulty behaviour. Security models also tend to focus on the physical system itself: we don’t want viruses and unauthorised access to the machines. The above specification view is used when specifying security not for a physical system but for a protocol like a middleware layer. In that sense they complement each other.

In the above specification model we say that a protocol π is a **secure implementation** of its specification if whenever it is run, even when under an attack allowed by $\mathcal{A}$ it behaves according to $\mathcal{S}$.

This means that security is always relative to some specification: It does not make sense to say that a system is secure without saying what it is supposed to do and under which

circumstances. Notice in particular that if a secure protocol is run in an attack model that is not allowed by $\mathcal{A}$, then all bets are off, even if π is a secure implementation of $(\mathcal{A}, \mathcal{S})$. The notions of correctness, consistency and fault tolerance are all just notions of security according to some specification. Correctness is typically just achieving security in an attack model with no or very mild failures. Consistency is simply security in the context of databases and replicated systems. Fault tolerance is security in attack models with bad failures.

To avoid confusion, we will stick with the *secure implementation of* notion of security throughout the text.

1.3.7 Specifying and Modelling Distributed Systems

We now return to exactly how we specify what a system is supposed to do and what an implementation is actually doing.

Formally, our systems will be specified via interactive agents (IAs). An interactive agent A is a computational device which receives and sends messages on named ports and which holds an internal state.

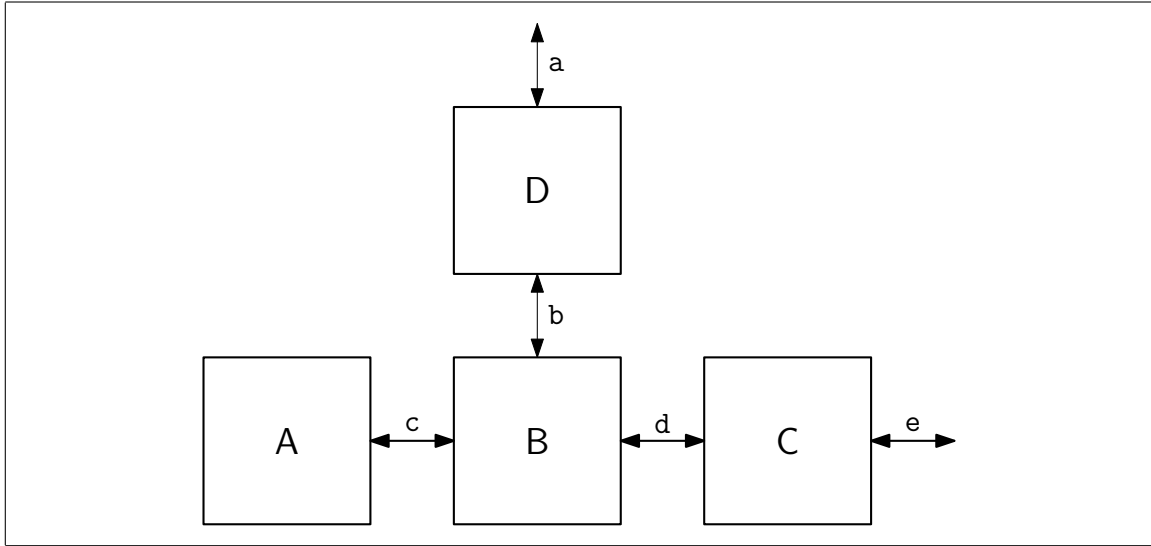


Figure 1.6: An interactive agent A with a port c ; An IA B with ports b , c and d ; An IA C with ports d and e ; And an IA D with ports b and a . Together they make up an interactive system with open ports a and e

When an IA is run it will produce a **trace**, which is just the sequence of messages exchanged between the agents in the order they were exchanged. Formally: start with the empty trace, and when m is written on port p you append (SEND, p, m) to the trace. When m is read on port p you append $(\text{RECEIVED}, p, m)$ to the trace. Most of the specifications we will consider in this book can be defined via traces. This is done via trace properties, which are just functions P which take traces as input and return true or false. If τ is a trace, then $P(\tau) = \top$ means that τ has the property P and $P(\tau) = \perp$ means τ does not have the property P . There are two important classes of trace properties:

Safety property: A safety property is something which is true until some safety condition is broken, at which point it never becomes true again. More formally, a safety property is a trace property P such that $P(\epsilon) = \top$, and if $P(\tau) = \perp$, then $P(\tau \circ \tau') = \perp$ for all traces τ' . Here ϵ is the empty list of inputs and outputs and $\circ$ denotes concatenation of lists. An

example of a safety property could be “no machines in the systems has output 42”. This is clearly true before the system has begun running, and when the property was broken it cannot be undone.

Liveness property: Liveness properties are something which can eventually become true (in some attack model). A bit more formally, a liveness property is a trace property P such that $P(\epsilon) = \perp$, and if $P(\tau) = \top$, then $P(\tau \circ \tau') = \top$ for all traces τ' . It is also usual to require that a property can always later become true by some possible execution. I.e., we require that for each τ there is τ' such that $P(\tau \circ \tau') = \top$, i.e., even if the property is false now it could always become true later. An example of a liveness property could be “some machine has output 42”. This is clearly false before the system has begun running, and when it happened once it cannot be undone. Furthermore, it is always possible in principle that it could happen that a machine outputs 42.

One has to be very careful when specifying liveness properties precisely to make sure the specification is sensical. It is notoriously hard to correctly specify liveness properties. Often for very technical reasons which does not really teach us much about real systems. As an example, say that “once a message m is sent, then m will eventually be delivered”, is not a liveness property. To begin with m is not sent, so it is true that “if m was sent then it got delivered”. But once m was sent and not yet delivered, it is no longer true that “if m was sent then it got delivered”. Once m gets delivered, it is again true that “if m was sent then it got delivered”. So we have a property which goes from true to false to true. This is not a liveness property. To phrase message delivery as a liveness property we therefore formally only consider the trace as starting after m was sent. So at the beginning of time, in the empty trace, the message m was already sent. Then we can specify the liveness property “ m gets delivered”.

Here is a related problem. We run into further complications when we want to say that “the system eventually delivers the message m ”. In any finite execution where m was not delivered it might in principle happen later that m gets delivered. So we cannot violate the property in finite runs of the systems. To speak about liveness properties the simplest way is typically to talk about infinite runs. For instance, there should be no infinite runs in which m was not delivered. However, cryptography does not like infinite runs. In infinite time, all the computational cryptography would get broken, which would probably allow to break all the nice properties it was designed to have. So coming up with a good notion of liveness which works well in combination with cryptography is rather subtle. We have chosen the formalization that at least the liveness property always in principle can be fulfilled in the future.

If you want to formally specify and prove systems secure, such technicalities are important, but not to the level of understanding we seek in this introductory book. So we will be very hand wavy about liveness properties.

On a related note. It is natural to think that the formalisation of liveness given above is wrong. Assume for instance that we want to say that some machine outputs 42. Should it not instead be something along the line of $P(\epsilon) = \perp$ and $\exists \tau < \infty$ such that $P(\tau) = \top$? Then the property would more intuitively say that some machine will eventually output 42. The reason why this does not work is that at a point where the system only ran for some finite time, it might still be that no machine output 42. So the trace does not have the property that $\exists \tau < \infty$ such that $P(\tau) = \top$. Yet, if we would let the system run a bit longer, then maybe the trace would at some point get the property. When we want to say that a system has a given liveness property, we therefore need a bit more care. We basically need to say that for all runs of the system where there is no more meaningful work to be done (like messages that need to be delivered) and where all the network resources that the protocol used also were live, the system make the liveness predicate true. We will in Section 2.9 discuss in more detail the subtleties of specifying

liveness properties. The above definition simply specifies the minimal requirements that there is always the chance that a liveness property becomes true, and once a liveness property is true it does not go away again, like the arrival of an expected message.

Mind Your Safety!

It is common in distributed systems to distinguish sharply between safety and liveness properties. You should be careful about your safety properties, if you break them it can never be undone. For liveness properties you can be more lax, because, if you do not manage to fulfil them now, there might always be a chance later.

It turns out that any trace property can be written as a conjunction of safety and liveness properties. More importantly, this can often be done in a natural way. Consider for instance this property: “Eventually exactly one machine will output 42.” It is not a liveness property, as it can become true, but then false again (when the second machine outputs 42). It is not a safety property either as it is not true from the beginning of time, where no machines have output anything yet. However, it is a conjunction of these trace properties: “Some machine has output 42” and “It did not happen that more than one machine has output 42”. The first one is a liveness property. The second one is a safety property. When they are both true, then exactly one machine has output 42.

Trying to break up the specification of your system into liveness and safety properties will help you focus on which part of the specification is the most important from a security point of view, namely the safety properties.

Exercise 1.3 (trace properties I) For each of the following trace properties, determine whether they are liveness properties, safety properties or neither.

1. If S gets input (SEND, m, R) , where R is the name of a receiver, then R will at some point output $(\text{RECEIVED}, m, S)$. Note that the trace we consider starts after (SEND, m, R) was already sent, so to begin with it holds that (SEND, m, R) was sent, but it still does not hold that a message was received.
2. If R outputs $(\text{RECEIVED}, m, S)$, then at some earlier time S got the input (SEND, m, R) .
3. If R outputs $(\text{RECEIVED}, m, S)$, then no other party will ever output $(\text{RECEIVED}, m, S)$.
4. If R outputs $(\text{RECEIVED}, m, S)$, then all other parties will eventually output $(\text{RECEIVED}, m, S)$, and no other parties will output anything else but $(\text{RECEIVED}, m, S)$. The trace to consider starts after R outputs $(\text{RECEIVED}, m, S)$.



Exercise 1.4 (trace properties II) For each of the following trace properties, write them as a conjunction of liveness properties and safety properties.

1. R outputs $(\text{RECEIVED}, m, S)$ if and only if at some earlier time S got the input (SEND, m, R) .
2. If R outputs $(\text{RECEIVED}, m, S)$, then all other parties already output $(\text{RECEIVED}, m, S)$, or will do that the next time they send something.



1.3.8 Synchrony in Distributed Systems Models

There are many ways to model the network and machines of a distributed system. A main distinction is on whether the protocols use a notion of physical time and how.

Fully synchronous: In a fully synchronous system, each process P has a clock $c(P)$ and there is a known bound Δ_{CLOCK} on how far they drift from real time. There is also a known upper bound Δ_{SEND} on how long it takes to send a message between any two parties. For sake of definition, the model often assumes some notion of global time, which is used to define what the real time is. Of course there is no such thing in real life due to the nature of time described by relativity theory, but for distributed systems located on the surface of the earth UTC is a pretty good approximation of some global time. We use N to denote nature and use $c(N)$ to denote the real time: the time of nature. So, if we define $c(X)$ to be the clock at process X , then it holds for all processes P that $|c(P) - c(N)| \leq \Delta_{\text{CLOCK}}$. Furthermore, if a message is sent at time $\text{snd}(m)$ on the clock $c(N)$ and received at $\text{rcv}(m)$ on the clock $c(N)$, then $\text{rcv}(m) - \text{snd}(m) \leq \Delta_{\text{SEND}}$. We look in much more depth at physical time and synchronous protocols in Chapter 8.

Fully asynchronous: In a fully asynchronous system, the processes P do not even have clocks, so the protocols cannot be specified by asking parties to do things at specific times. Also, there is no known upper bound on how long it takes to send a message. In fact, for each message the adversary might decide to completely drop it, or deliver it at any time of her choosing. An example of an asynchronous system is the HTTP protocol: When a client wants a web page it sends a GET request to the server and opens a port where it expects the data back. Then it just waits until the data comes. The server, when it gets the GET request retrieves the page from disk (or constructs it) and then sends it back when it is ready. We look in much more depth at asynchronous protocols in Chapter 10.

Partially Synchronous: In the partially synchronous model¹ it is assumed that if a message is sent by a correct process and that process never crashes, then the message is *eventually* delivered. There are several ways to model this, and often people do not even take the care to make precise what they mean by “eventually”. Here is one common way to do it: There is a number Δ_{SEND} , some length of time. If a message is sent by a correct process that does not crash, then it is delivered within that bound. But the value Δ_{SEND} can be arbitrarily large and it is *not known to the distributed system*. (One way of thinking of this is that you first design the system, and then some evil entity sets Δ_{SEND}). So what the synchronous model and the partially synchronous model have in common is that messages always get delivered within some time bound. The difference is that in the synchronous model the protocol designer is assumed to know the upper bound on network delivery. In the partially synchronous model the protocol designer is not allowed to use the bound, it is merely assumed to exist. It seems odd that knowing that a secret upper bound exists should be useful, but when we cover Finality Layers for Blockchains in Chapter 16, we will see an example that one can do more in the partially synchronous model than in the fully asynchronous model.

Eventually synchronous: Another popular mixed model is the eventually synchronous model, where the processors have clocks. Also two bounds Δ_{CLOCK} and Δ_{SEND} are given. It is assumed that the network in some periods is asynchronous: the clocks can be far apart and messages take arbitrarily long time to be delivered. However, it is assumed that eventually there will always come a period where the network is synchronous, i.e., during this period

¹This model was introduced in [7].

it holds for all processes P that $|c(P) - c(N)| \leq \Delta_{\text{clock}}$ and that for all messages m sent and received in this period $\text{rcv}(m) - \text{snd}(m) \leq \Delta_{\text{send}}$. The protocol does not know when the period has arrived. So what the synchronous model and the eventually synchronous model have in common is that there is a known upper bound on message delivery. The difference is that in the synchronous model the messages are always delivered within this bound. In the eventually synchronous model they are eventually delivered within this bound for a long enough period. This allows us to set the bound more liberally in the eventually synchronous model.

It can be fairly easy to design systems for the fully synchronous model. However, the systems are very hard to implement in practice as you have to constantly synchronize the clocks of the involved parties and find a way to make sure all messages that are sent are delivered within the bound. This can be hard during periods where machines crash or the Internet is slow. Moreover, all machines engage in several systems with several different and overlapping groups of servers. So if all these protocols had to be synchronous, in the end the entire Internet would have to be synchronised, which is completely infeasible as machines are constantly crashing, messages are being dropped and denial-of-service attacks are being mounted. Therefore the synchronous model is mostly of academic interest: it is the easiest test-bed for designing protocols. It can also be relevant for systems running in a very closed environment, like three database servers within the same administrative domain. However, it is one of the hard won insights in distributed systems that when you describe your system you should to the greatest extent possible not refer to any notion of real time. By experience protocols relying on real time tend to break or be very slow in practice.

In comes the asynchronous model! This model does not even give you a clock to refer to and there is no bound on message delivery time. So if you can design a system that is secure in the asynchronous model, it will also be secure when run in practice on any network. The flip side is that it is very hard to design systems for the asynchronous model. When you have no notion of time you can for instance not distinguish between a server that is crashed and a server which just has messages that take arbitrarily long time to reach you. So if your system has some special server that is important for liveness, you cannot know when it is down and so cannot know when to replace it. This sometimes makes it impossible to solve in the fully asynchronous model tasks which can be solved in the fully synchronous model.

Hence the eventually synchronous model. The intended use of this model is as follows. Assume you have a system which is specified by some safety properties $\mathcal{P} = \{S_1, \dots, S_n\}$ and some liveness properties $\mathcal{L} = \{L_1, \dots, L_m\}$. You are supposed to design your system as follows. The safety properties $\mathcal{P}$ will never be broken, neither in synchronous nor in asynchronous periods. The liveness properties need not become true in asynchronous periods, but if the system become synchronous for long enough, then your system has to be live, i.e., you for instance start to deliver messages to all receivers. For such a system, clock synchronisation and the bounds Δ_{clock} and Δ_{send} can be set to be fairly small, and clocks can be synchronized only rarely. If the clocks drift too far in some period or messages arrive very slowly, it is rarely a big issue as safety will not be broken, and we can assume that clocks will probably soon be synchronized again, and the network will be back to fair-weather conditions and so will start delivering again.

Similarly the partially synchronous model is a good balance between the too theoretical synchronous model and too demanding asynchronous model. As mentioned above, we will return to this model when we study Finality Layers.

Exercise 1.5 (detecting fail-silent crashes) Assume that process P is supposed to send a message m to Q at clock time t , i.e., when $c(P) = t$. Assume that there are no omission

errors but that P might fail-silent crash before sending the message. What is the earliest time (measured using $c(Q)$) at which Q can conclude that P must have crashed? Consider this in all of the system models we considered above, synchronous, asynchronous, . . .

1.3.9 Thinking like a Cryptographer in Distributed Systems

How should you pick your attack model? Well, the first answer is that you should take a look at your threat model. Your threat model specifies which attacks you find possible, and that should be your attack model when you analyse your system. The second answer is that you should be conservative. If you are not completely sure that your machines will never crash, you should add process failure to the attack model. If you are not completely sure that when the machine crashes it might not make the operating system send some weird message, incorporate fail-arbitrary. Or even better, maybe you are not completely sure one of your machines cannot be taken over by a crazy hacker. In that case you must let the attack model be Byzantine process failures. Sometimes you would like to be liberal when you pick the model as it will make your life easier. Do not do that. Machines will crash, messages will be seen by the adversary, and some machines will eventually be broken into. Deal with that already when you design your system.

1.3.10 Canonical Goals

So now we know how to design distributed systems and how to specify what we want them to do under which conditions. But what should we make our system do? It turns out that no matter which system you try to build, there is a small collection of problems that keep popping up. Even better, if one has a solution to these problems, then building most other systems on top is pretty easy. These are the power tools of distributed systems. We will study the most important ones. They also happen to be didactically complete. If you understand how to implement these, you have learned to think in a distributed way.

Leader Election: In many distributed systems, one can gain efficiency by having one server take on a special role, like collecting and distributing tasks. Such a server is often called the leader. The problem is that when the leader crashes, liveness typically breaks down in a very bad way. Then it is time to pick a new leader. This is called leader election. The goal is that all processes end up agreeing on some new non-crashed process that will be the new leader. We look at leader election in Chapter 16.

Broadcast: You want to send a message m to all machines in the system. This is easy when there are no failures. If there are transmission errors, various systems with acknowledgements and resends will solve the problem. In case of process failures, the problem becomes much harder. It is typically needed that all processes receive the same message, even if the sender and some of the other processes crash or are suffering Byzantine failure. The problem can be even harder when considering many messages being sent by different parties at the same time and require that they arrive in the same order at all processes, known as totally ordered broadcast. We look at Totally Ordered Broadcast in Chapter 14.

Consensus: All processes have a local input bit 0 or 1. This could for instance be a bit indicating whether they are all ready to proceed in the protocol. They should end up outputting a common decision, i.e., there is a bit b such that all correct processes eventually output b . There are several variations on how this output depends on the inputs. A popular one to study is this one: If all correct processes vote 1, then the output should be 1. If all correct processes vote 0, the output should be 0. Otherwise the output can be

arbitrary, as long as all correct processes agree on the output. We look at Consensus in Chapter 8 and Chapter 10.

MUTEX: Mutual exclusion. There are several servers in a system. The processes run the same code. There is some critical region in the code that only one server is allowed to enter at a time. This problem is already hard in multi-threaded programs. It becomes much worse in a distributed system where the processes are physically separated.

Here are some problems that can be solved with these tools.

Work distribution: You have a system where requests come in much faster than they can be solved by one machine. You make ten copies of the system and set up a special entry server, which only receives the requests and then distributes them to the workers in a balanced manner. The workload balancer crashes. The workers notice that work stop coming in. They run a leader election and the winner is promoted to the new entry server.

State machine replication: You have a critical program M that is so important that it must keep running even if the machine it runs on crashes. You then simply run it on n machines. When clients want to give an input, it is sent to all copies of m . If you use totally ordered broadcast, secure under Byzantine process errors, to broadcast the inputs to all replicas, then they will run M on the same inputs in the same order and therefore keep a consistent copy of M . Then they broadcast the outputs of M back on the totally ordered broadcast channel. The Byzantine machines might send wrong outputs on the broadcast channel, but as long as less than $n/2$ of the servers are corrupted, the correct output can be found just by taking majority of the proposed values. So this runs a consistent copy of M that can tolerate less than $n/2$ Byzantine failures.

Recovery: There has been a very bad event in your state machine replication system where a lot of machines crashed and the network completely broke down. Now all machines are back alive and the network healthy again. You wonder whether all the machines hold the same state for M . Let M_i be the state held by party i . All processes send their state M_i to all other processes. Process j lets $b_j = 1$ if and only if $M_1 = \dots = M_n$. They might get different results if some of the machines are still Byzantine corrupted and report different M_i to different parties. Therefore they run Consensus on $(b_1, \dots, b_n)$ to get a common decision b . If $b = 1$ they continue running. If $b = 0$, then they must start some clean up procedure. Note that if the output is $b = 1$, then at least one honest process had input $b_i = 1$ and therefore all correct processes hold the same state for M . So it is safe to keep running. Also, if all processes are correct and hold the same state $M_i = M$, then the output will be 1, so they keep running from M .

Sharing exclusive physical resources: A large number of machines share a common physical broadcast channel, like a cable or some frequency for Wi-fi. If two machines send at the same time, both signals are lost. Run a distributed MUTEX protocol.

Chapter 2

A Model of Distributed Systems (2026)[†]

Reading Guide *There will be a short recap of this chapter in Chapter 4, which should be enough to understand the book. If you need more details or like more formality, come back here and read this chapter. It is common in distributed systems to use a formal model for specifying what a system should do and what a protocol does. This is to allow to prove that a protocol does what it is supposed to do. Security of distributed systems is very slippery, so such proofs are needed. This chapter introduces one such formal model based on interactive agents. We use the model in some later chapters. The purpose of using a formal model in the book is to let the readers familiarise themselves with the use of such a model, i.e., to have seen the use of such a model. Studying the model in detail is not a goal in itself. This chapter is not curriculum in itself. We will not ask questions directly to this chapter at the oral exam. However, you are expected to understand the concepts and notation well enough that you can read later chapters and explain these chapter. So read the chapter to understand the notation well enough to read the coming chapters.*

Contents

2.1	Introduction	30
2.2	Basic System Model	30
2.2.1	Interactive Agents	31
2.2.2	Interactive Systems	31
2.3	Modelling Persistent Storage	32
2.4	Modelling Processes	32
2.4.1	Specifying a Process	34
2.4.2	Some Possible Special Ports	35
2.5	Modelling Network Resources	37
2.6	Modelling Protocols	40
2.6.1	Ideal Functionalities	40
2.7	Specifying Safety Properties	40
2.8	Composition	43
2.9	Specifying Liveness Properties	43
2.10	Handling Crashes	44
2.11	Composition^{††}	46
2.12	On Defining Eventuality^{††}	49

2.1 Introduction

In this book we would like to prove some results about distributed systems. To be able to prove any statement about any object, both the statement and the object need to be formalised with mathematical precision. For this a so-called system model is used, where all entities are modelled within some formal framework. In the following we will first formalise a basic framework and then show how to use it to model processes, channels, disks and other system components.

In most of the book we will only sketch proofs and will not go to the level of detail of the model given in this chapter. This chapter serves as a basis for the formally inclined reader to learn how the informal proofs in other parts of the book might be brought on more precise grounds.

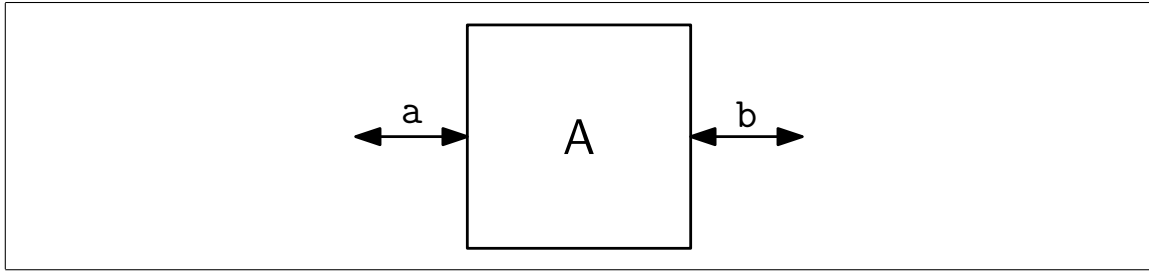


Figure 2.1: An Interactive Agent A with $\mathcal{P} = \{a, b\}$. Let $\sigma_0 := 0$. We can define $T(\sigma, P, m) = (\sigma', Q, m')$. If m is not an integer, then let $Q = P$ and let $m' = \perp$ to signal error. Otherwise, let $\sigma' = \sigma + m$ and let $Q = P$ and $m' = \sigma'$. This is then an interactive agent which keeps an accumulated sum of the integers input on its ports. When you send on some port a new number to add to the accumulator, you are returned the accumulated sum on the same port.

2.2 Basic System Model

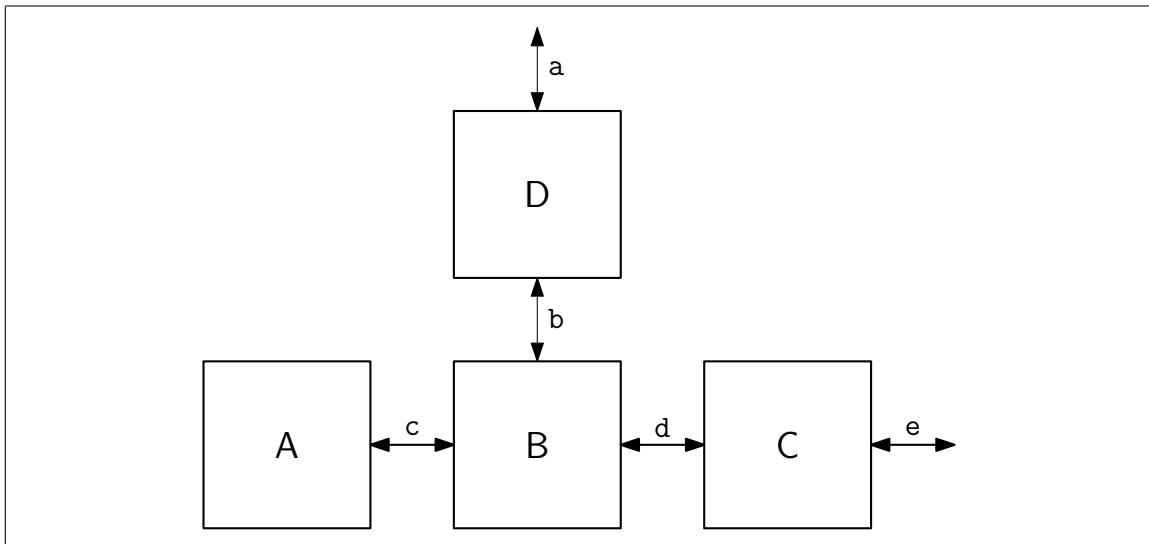


Figure 2.2: An Interactive System with open ports a and e

2.2.1 Interactive Agents

The most basic entity will be an Interactive Agent (IA) (see an example in Fig. 2.1), which is formally defined by a tuple $(\mathcal{P}, \sigma_0, T)$ where:

- $\mathcal{P}$ is a finite set containing the names of the ports on which the IA can receive inputs and outputs. Each port can be used both for sending and receiving.
- σ_0 is the initial state of the IA.
- T is the transition function. The transition function T says what the system does in response to a message being received on one of the ports. It depends on the current state σ (initially σ_0), the port name $P \in \mathcal{P}$ and the message m being received. The output of T specifies the new current state σ' , a new port $Q \in \mathcal{P}$ and some new message m' . We write $(\sigma', Q, m') \leftarrow T(\sigma, P, m)$. Notice that by this rule the transition function must always send a new message on some legal port.

We think of the IA A as a box holding its current state σ . We will therefore sometimes write $(Q, m') \leftarrow A(P, m)$ to mean: fetch the current state σ from A , compute $(\sigma', Q, m') \leftarrow T(\sigma, P, m)$ and replace the current state in A by σ' . We allow the transition functions to be randomised, i.e., they might produce one of several outputs with some probability distribution, like HEADS with probability 50% and TAILS with probability 50%. Therefore, the transition functions are not strictly “functions” in a mathematical sense. More precisely the value $T(\sigma, P, m)$ is a random variable over values of the form (σ', Q, m') in the probability theoretical sense. When we write $(\sigma', Q, m') \leftarrow T(\sigma, P, m)$ we mean that (σ', Q, m') is sampled from the random variable $T(\sigma, P, m)$. This allows to model an algorithm which for instance samples a random RSA key (n, e, d) , saves (n, d) in the state σ' and sends (n, e) on some port. An RSA key is a key used for public-key encryption and signature schemes, which we discuss in detail in Chapter 5 and Chapter 6.

Even though T is not formally a function we will still call it the transition function. We make the requirement that the transition function does not depend on the specific port names, i.e., if we renamed all port names in a unique manner the IA would still behave the same modulo the new port names. The reason for this is that if we later want to compose two systems with conflicting port names, then we can just rename the port names in one system without changing its input-output behaviour.

2.2.2 Interactive Systems

An interactive system (IS) S is just a set of IAs. If there are two IAs with the same port name, then we think of those as being connected.¹ If there are more than two IAs with the same port name, then we say that the system is malformed and we write $S = \perp$. When we have two systems S_1 and S_2 we compose them simply by taking set union $S_1 + S_2 = S_1 \cup S_2$. It is easy to see then that if S_1, S_2, S_3 are interactive systems then $S_1 + (S_2 + S_3) = (S_1 + S_2) + S_3$ and $S_1 + S_2 = S_2 + S_1$. The ports not connected to other ports are called open ports. An IS can be activated on an open port P by sending some message m . We say that we input (P, m) to S . The execution proceeds as follows. Let A_0 be the IA with port P , let $m_0 = m$

¹We could in the IA formalism have given the ports types and asked that the types of connected ports match. However, if we start putting types on IAs, we would probably have to add so-called session types to get a useful framework. A simple notion of typed ports seem to serve no real purpose, sitting between two chairs. So, we have chosen to not treat the issue with types in this book. The reader who worries about types can take the type of all ports to be the set of all finite bit-strings. All the messages being sent around are then represented as bit-strings in some canonical encoding.

and $P_0 = P$. Then we compute $(P_1, m_1) \leftarrow A_0(P_0, m_0)$. If P_1 is a closed port, then let A_1 be the other IA with port P_1 . Then compute $(P_2, m_2) \leftarrow A_1(P_1, m_1)$ and so on until we compute $(P_{t+1}, m_{t+1}) \leftarrow A_t(P_t, m_t)$ and P_{t+1} is an open port. Then let $Q = P_{t+1}$ and $m' = m_{t+1}$. Return (Q, m') . Let $\mathcal{P}$ be the open ports of S . Let the internal state of S be the internal state of all the IAs composing the S . Notice that this makes an IS a function from pairs (P, m) with $P \in \mathcal{P}$ to pairs (Q, m') with $Q \in \mathcal{P}$. In computing the output the IS also updates the internal state of the system. So an IS is in some sense just a more structured IA. There are some formal differences though. For instance, an interactive system might loop forever, whereas we require that IAs always terminate.

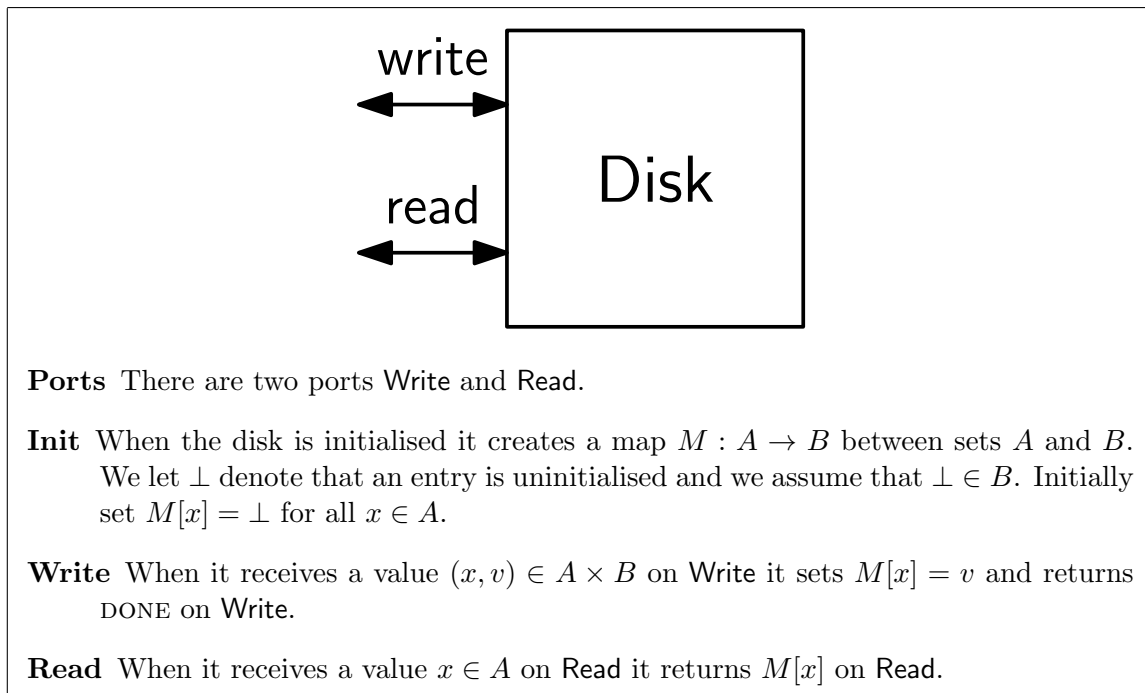


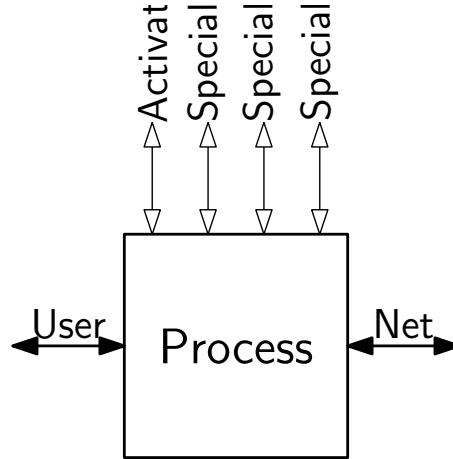
Figure 2.3: A Disk.

2.3 Modelling Persistent Storage

An important tool in fault tolerant distributed systems is some kind of permanent storage that will survive a crash. We will sometimes call it a disk and sometimes a log, or just persistent storage. We give an oversimplified model of persistent storage in Fig. 2.3. Implementing it in practice is very complicated in itself. What is hard to implement is for instance atomicity: if a process crashes while writing to the disk, either the write should succeed completely or leave no trace. In particular, the crashing process should never corrupt the disk itself.

2.4 Modelling Processes

We can also model processes. There is a wide range of choices in how to model a process in a distributed system and how they communicate. They can share memory or they can communicate solely via message passing. We will make a model based solely on message passing. We will also assume purely asynchronous programming: all inputs to a process are given as



Protocol Ports versus Special Ports The ports of the process are divided into the protocol ports and the special ports. Protocol ports correspond to the ports the process has in “real life”. The special ports are used only for modeling purpose. They are for instance used to model when a process is scheduled, when it crashes, and so on. We draw special ports with opened arrow heads. The protocol ports are in turn divided into two types of ports, the user interface ports (user ports) and the network interface ports (net ports). The user ports are those that the user of the system would call when it wants to use the system. The net ports are the ones that the process calls when it needs to talk to other processes which are part of implementing the system.

Handling Input For each protocol port P the process has a queue $P.Q$. Initially the queue is empty. When the process receives a message m on a protocol port P it adds the message to $P.Q$. Then it returns DONE on P . Note, in particular, that the process performs no actions when activated on a protocol ports. It merely stores the received message for later use.

Activate The process has a special port called **Activate**. When it receives any value on **Activate** it can do one of the following any number of times:

- Read from a message queue $P.Q$.
- Change its internal state in any other ways.
- Send messages on some ports.

The activation ends when it returns any value on **Activate**.

Figure 2.4: A Process.

messages and they are queued on the receiving process in such a way that the “calling” process can proceed its execution immediately. In particular, the caller never waits for an output. Such pure asynchronous programming in general makes it easier to develop asynchronous systems and will in most cases be needed anyway. So we make it the default case here for simplicity.

A process will receive its input asynchronously as follows: when a message m is received on some port P , the message m is added to a queue Q . Then the process returns and does

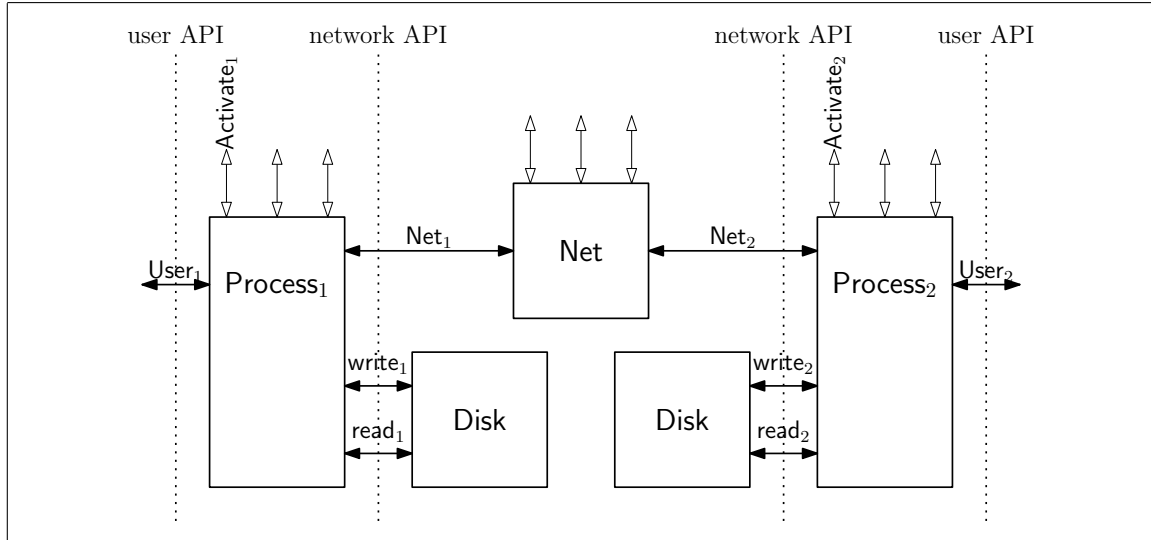


Figure 2.5: A Protocol with two processes, each with a disk. They use the network resource *Net* for communication. Note that the processes in a protocol have two types of protocol ports, the user ports where they take inputs for the protocol or middleware layer they try to implement and net ports on which they send messages to the network resources. To avoid confusing in later chapters it is important to have a clear distinction between these. We will call these ports the *user API* and the *network API* of the protocol.

nothing more. There is a separate “thread” associated to the process which is the one that does the actual work. It proceeds in steps known as activations. In each activation it is allowed to do some small amount of work and then the activation stops again. To model that processes by default might not progress at the same speed, we introduce a special port on which the process is told when to do the activations. Think of some evil daemon being connected to these ports. This means that in a larger system the daemon could try to activate the processes in a particular nasty order to break the system. This is an example of conservative modeling: we cannot by default assume that all processes proceed at the same speed, so to be on the safe side we assume that the order in which they take steps could be the worst possible one.

2.4.1 Specifying a Process

Formally a process is given by a transition function. But that is a very cumbersome formalism. Instead we will specify a process by a list of activation rules written in prose or pseudo-code. Here are some example of activation rules:

- On input (INCREMENT, a) on port I , let $x \leftarrow x + a$ and send x on port P .
- On input (PING) on port P return on P .
- On input (CONDITIONAL-INCREMENT, a, b) on any port, if $x \leq b$, let $x \leftarrow x + a$ and send x on port P .

In general a rule has a trigger part of the form

On input (NAME, $v_1, v_2, \dots$) on port P , where Cond.

In addition to the trigger part it has an **action part** A . This is the code that will be run if the rule is triggered. The action part is allowed to read the incoming queues, read local state, modify local state and send messages on outgoing ports. It is not allowed to send anything on the activation port of the IA. Putting these parts together, an **activation rule** is of the form.

On input $(NAME, v_1, v_2, \dots)$ on port P , where $Cond, A$.

The activation rule is **triggerable** if in the queue $P.Q$ there is a message m of the form $(NAME, v_1, v_2, \dots)$ and at the same time the condition $Cond$ is true. The condition might depend on the message and the internal state of the process. When a process is activated on port **Activate** it will go through all its activation rules from top to bottom. When it finds one that is triggerable, it triggers that rule.

If an activation rule is triggered, then the matching message m is removed from the queue. If the rule is triggered, then the algorithm A is run. This algorithm is allowed to run as a normal activation of a process with the restriction that it is not allowed to remove other messages from the message queues and is not allowed to send a message on the activation port. After running A , the process returns on **Activate**.

If there is no activation rule which is triggerable, then it does nothing and immediately returns on **Activate**. If there is no activation rule which is triggerable, then we say that the process is **idle**. Note that if a process is idle, then an activation cannot change that, as an activation in an idle state does not change the state of the process. Only the arrival of a new message might change the idle status of a process: the message might make some rules triggerable.

2.4.2 Some Possible Special Ports

In addition to **Activate** a process might have other special ports. These special ports are typically used to model faults. Here are some examples of such ports.

Crash On any input on the **Crash** port, delete the process' set of activation rules. This means that the machine no longer does anything when activated.

Takeover On input **AR** on **Takeover** the activation rules of the process are replaced by the activation rules in **AR**. If the process models a machine, then this corresponds to a hacker completely taking over the machine and installing its own code on the machine.

Leak On input L on **Leak**, let σ denote the internal state of the process, including all the queues of incoming messages. Compute $y = L(\sigma)$. Then return y on **Leak**. This corresponds to the process leaking some information to its environment. If the leakage function L is the identity, then it corresponds to a hacker breaking into the machine and seeing all the data on the machine. In some case more limited leakage makes sense too. We later in the book discuss security problems arising from machines leaking their power consumption or exact running time of algorithms working on secret data.

Not all processes need have all these ports. It depends on the flavour of security we want to prove. We will always be explicit about these ports if they are there. So by default, if we do not say anything, a process only has the **Activate** special port.

On Leaky Crashes*

A note about how we handle crashes. The impetus to crash is given as an input from the outside of the machine. This implies that the time of a crash cannot depend on secrets inside

the machine. We could have chosen another model, where some port is used to install a small piece of code which is watching the execution of the machine and then crashes it at a time depending on internal values on the machine. This better models crashes in real life: typically machines crash because of software bugs which occur at particular places in the code. Modeling state-dependent crashes would, however, make things very complicated when cryptography is used, since a part of the machine could be a secret key (we discuss secret keys in detail in Chapter 5 and following chapters). In this case an intruder could make the timing of the crash depend on the key, thus leaking secrets to the world via the crash. As an example, say that the intruder instructs the machine to crash after x milliseconds, where x is a 20-bit number containing the first 20 bits of the secret key. It is very hard to argue security when such leakage occurs. Such subtleties can be handled using leakage resilient cryptography, but this would go beyond the scope of this book.

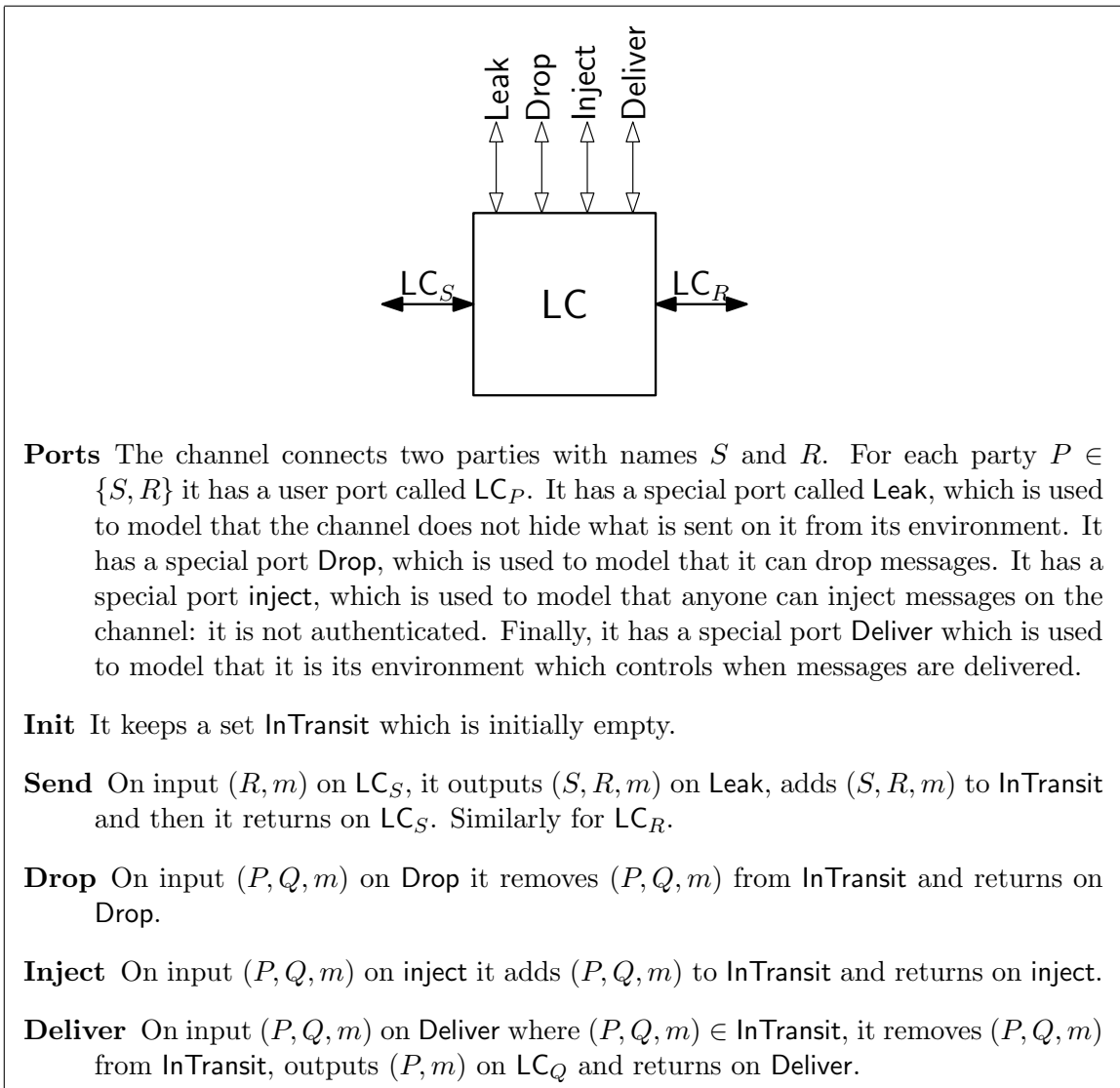


Figure 2.6: Lossy Channel

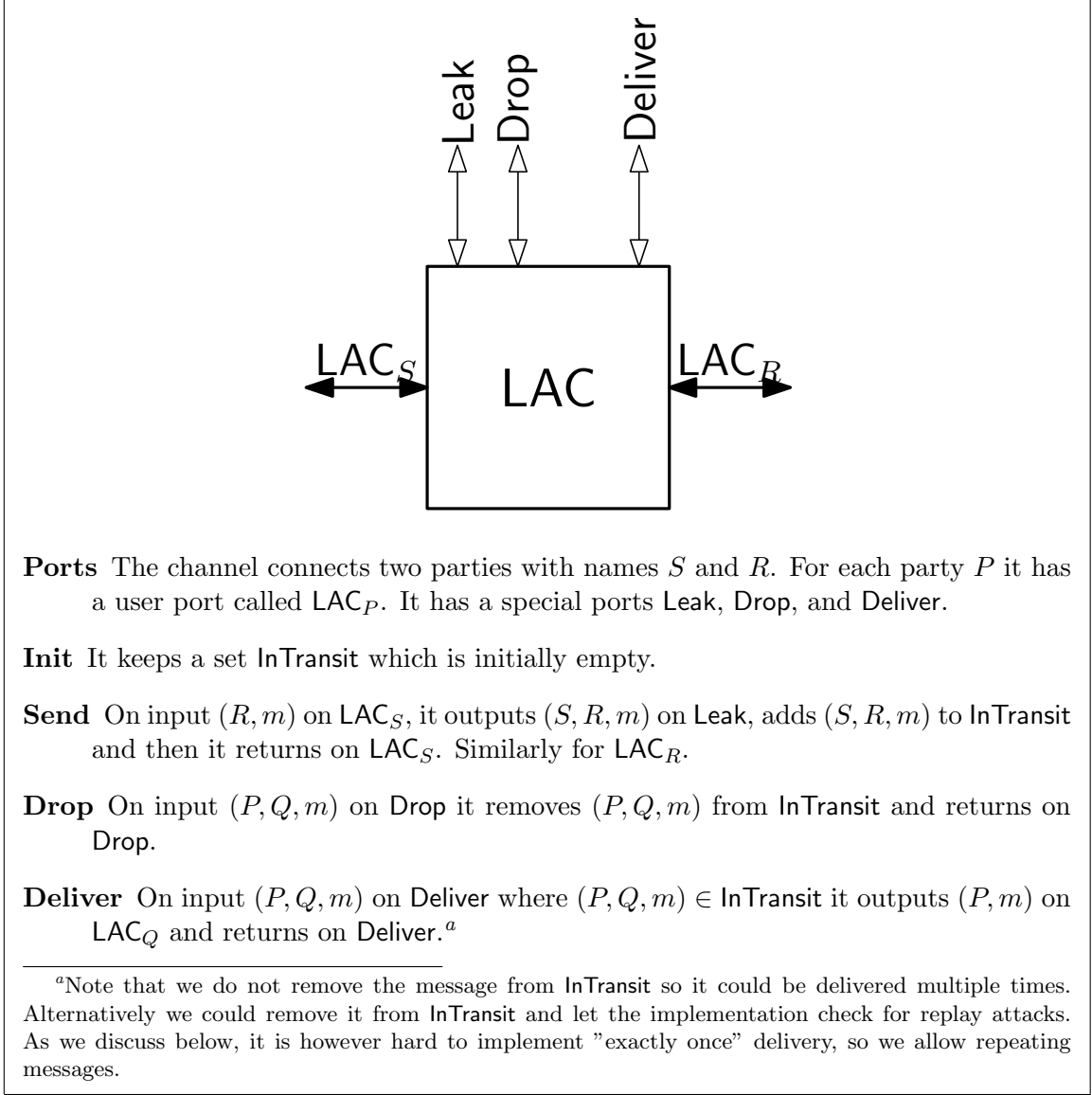


Figure 2.7: Lossy Authenticated Channel

2.5 Modelling Network Resources

When we model a protocol the parties need some means of communication. This will just be another IA modelling the means of communication we give to the protocol. We call such an IA a **network resource**. Later we will also call it an ideal functionality. The reason for this name is that IA does not go into details of how the network resource is implemented. It just specifies how the outputs should relate to inputs and magically enforces this. But sometimes we also want to assume that more complicated infrastructure is available to the parties in the protocol. This could for instance be a public-key infrastructure or a key distribution center.² In that case we use the generic names of network resource or ideal functionality.

When we model network resources we do not have additional structure, except we make the same distinction between protocol ports (the ports modelling real-life input and output) and

²We discuss these notions later in the book.

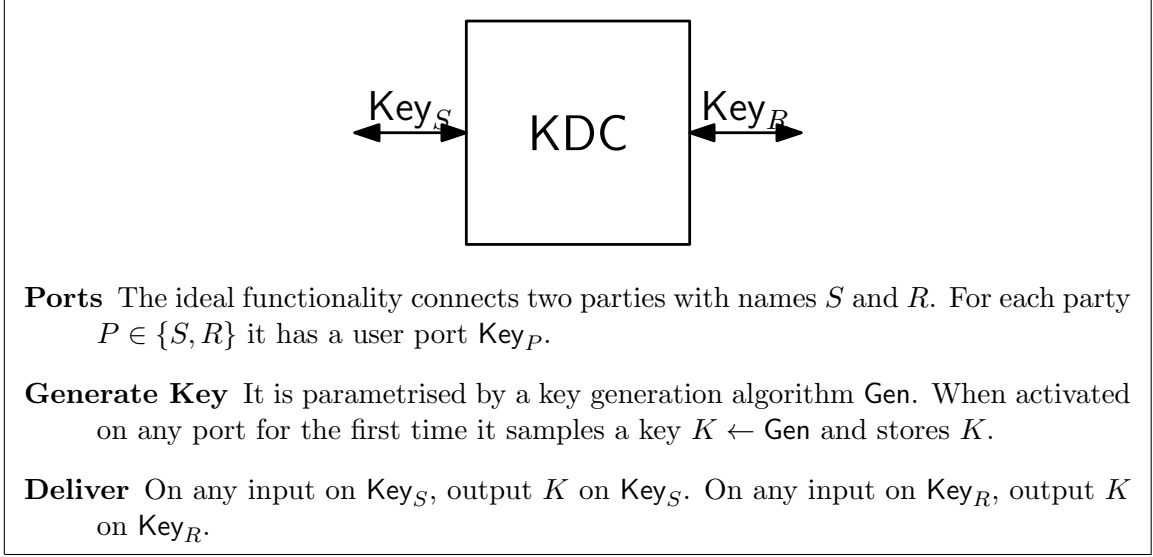


Figure 2.8: KDC

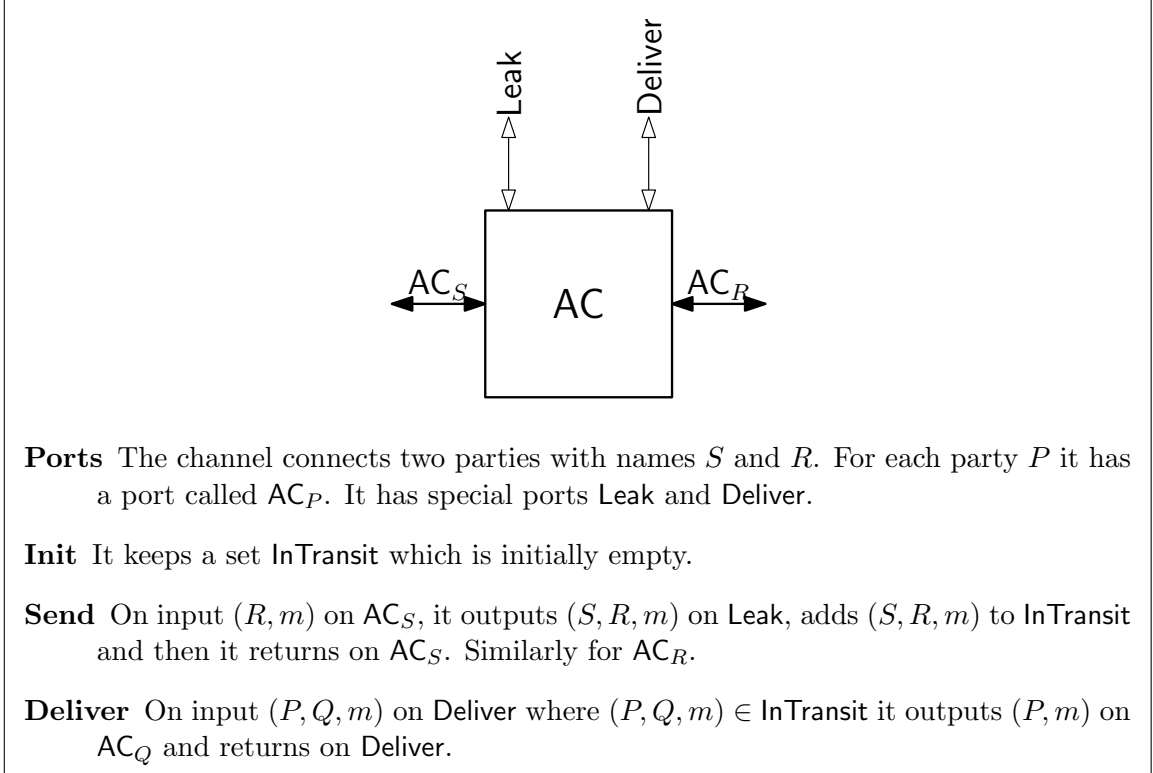


Figure 2.9: Authenticated Channel

the special ports (used to model properties of the network resource). On a network resource all the protocol ports are also called **user ports** and they are also called the **user API**. Note that this in particular means that when a process is connected to a network resource, then the network API of the process is connected to the user API of the network resource.

As an example, the IA in Fig. 2.6 models a **lossy channel (LC)**, where the environment can

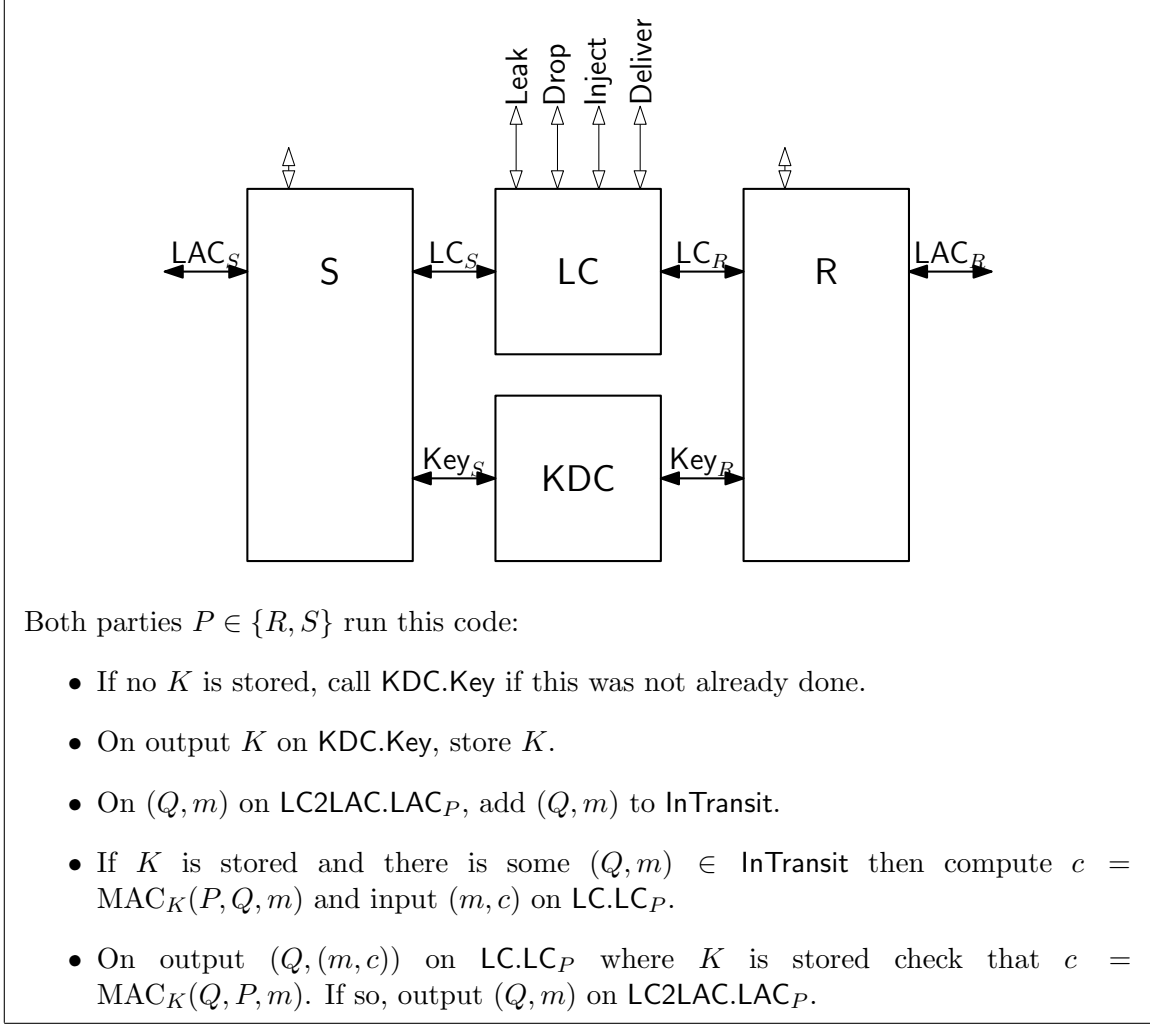


Figure 2.10: The LC2LAC protocol. It uses two network resources, LC and KDC . It has two processes, S and R . They are connected to the network resources as shown in the figure. The unnamed special port that each process has denotes the **Activate** port, which we for brevity will stop showing explicitly. Party P in addition has a port called $LC2LAC.LAC_P$ (in the figure just called LAC_P). The special ports on the parties are there simply as placeholders to remind us that the parties too could have special ports for instance to model corruption.)

see which messages are sent, it might drop a message and it might inject a message. Think of sending a message using UDP. The servers through which the message is sent can see it or drop it, and they might decide to forward a completely different message.

As another example, the IA in Fig. 2.7 models a **lossy authenticated channel (LAC)**, where the environment can see which messages are sent, and then might decide to drop a message. Think of sending a message using UDP but with some authenticity mechanism in place, for instance digital signatures (see Chapter 6). The servers through which the message is sent can see it or drop it, but they can't change it.

If we wanted to implement LAC from LC we would for instance need a symmetric key for a message authentication code (MAC). To be able to use a MAC the two parties will have to share a secret key, as we will discuss in great detail in Chapter 6. One way to put such a key in place is a so-called key distribution center which samples the key and somehow securely hands it

to both parties. In Fig. 2.8 we model a very simple key distribution center for just two parties. It will generate a random key and securely send it to each of the parties. Importantly, this is done in a way such that the environment does not see the key K . We model this by not leaking K .

As our very last example, and mostly because we need it later, consider the **authenticated channel (AC)** in Fig. 2.9. It works as a LAC, but there is no special command to drop messages. So if a message is sent, then it is eventually delivered (if the right deliver-command is eventually given).

2.6 Modelling Protocols

We are then ready to model protocols. Protocols will consist of processes, network resources and disks connected to each other in some way, see Fig. 2.5 for a generic example. Note that disks are considered special cases of network resources, which just happen to connect to only one process. As another example, consider the protocol LC2LAC in Fig. 2.10. It tries to turn a LC into a LAC. For this it uses a message authentication code (MAC). We discuss MACs later in (see Chapter 6). What is important is that for a fixed key, there is a function MAC_K which maps a string x to a code c , and you can compute $c = \text{MAC}_K(x)$ only if you know K . So when R receives c such that $c = \text{MAC}_K(S, R, m)$, then R knows that it was S who sent the message.

As another example, consider the LAC2AC protocol in Fig. 2.11. It tries to turn a LAC into an AC. We have given each party a disk. They don't use them for now, but they come in handy later.

The intuition behind the LAC2AC protocol is that we just keep sending the message until it gets through. If we try an infinitely number of times, then any reasonable channel should allow the message to get through at some point. When this happens, the message will be delivered at R 's end. At the same time an acknowledgement is sent back. If it is lost, then S keeps sending, so eventually the message gets through again and a new acknowledgement is sent. And so on. So eventually the acknowledgement must get through. At that time S stops sending. After some time all messages are then delivered or dropped and the protocol becomes silent. The only trace that will be left is that (S, m) is stored at R in **Delivered**. We will discuss later whether we can clean that up.

2.6.1 Ideal Functionalities

As should be clear from the above, network resources are used for two purposes in our model. They can be used as the network resource of a protocol, like when we use LAC in LAC2AC to implement AC. They can also be used to *specify* what a concrete protocol is supposed to do. We for instance want LAC to behave like AC. When a network resource is used to specify what a system is supposed to do we sometimes call it an **ideal functionality**. Notice that when we use AC as an ideal functionality to specify what LAC2AC is supposed to do, then there is no AC present in LAC2AC. We just compare LAC2AC to AC. On the other hand, when we use a network resource as a sub-protocol, like when we use LAC in LAC2AC, then there is in fact a copy of LAC present in LAC2AC.

2.7 Specifying Safety Properties

The above text should give a very rough idea how of we formalise what a protocol does. We now discuss how one would specify what it is *supposed* to do. Recall that we typically want to specify two different kinds of properties, **safety properties** and **liveness properties**. Safety

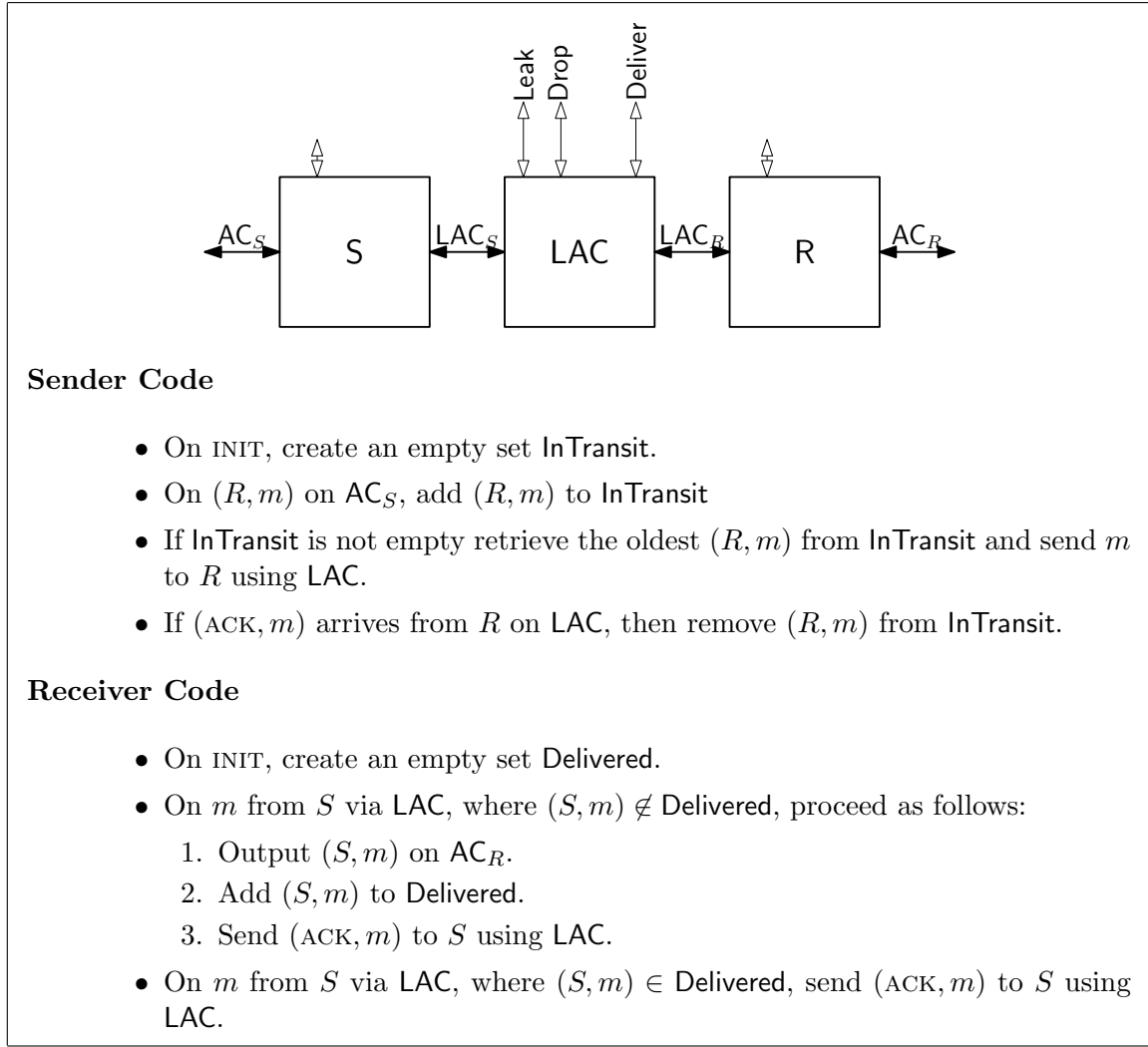


Figure 2.11: The LAC2AC protocol

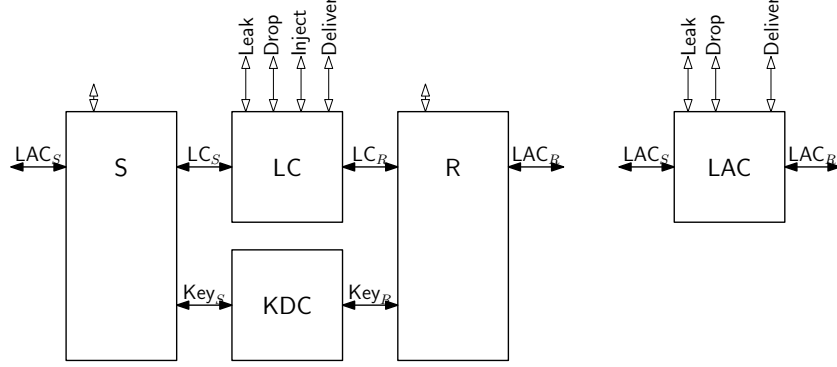
properties basically specify what is the correct relation between the inputs and outputs of the system. A safety property is only broken if the system gives a wrong output. Liveness properties specify that the system must make progress, for instance that it does not have deadlocks. A liveness property is only broken if the system ends up in a state where there is still work to do but the state of the system prevents it from ever making progress again. We start by looking at how to specify safety properties.

What is the intended behaviour of LC2LAC? Well, we stated that we wanted it to implement a LAC. And, we already specified what we think a LAC is. We did that in Fig. 2.7. So there is no reason to complicate things. We simply state that the protocol should behave as the network resource we wanted it to be. Here is the first attempt at a definition:

— *The protocol LC2LAC should behave as the network resource LAC.* —

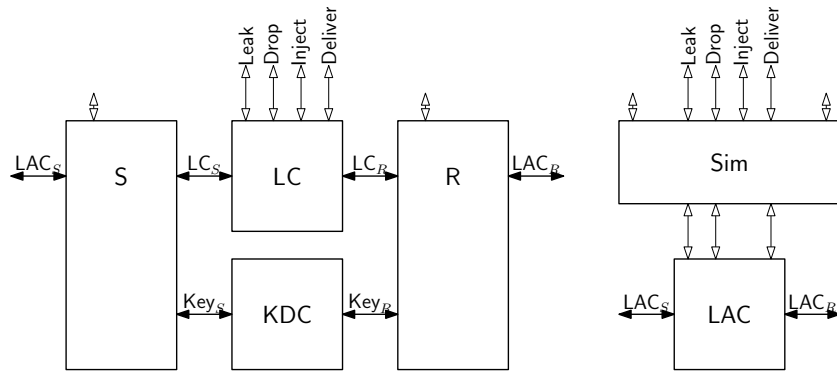
We then just have to make “*behave as*” more precise and we are done. When we address the safety of the system, what we want to say is that the protocol LC2LAC will not give any outputs that LAC could not have given. For instance, if LC2LAC would deliver to R a message m' that S did not send, then safety is broken as LAC would never deliver to R a message m' that S did

not send. We now unfold this informal definition a bit. Let us first take a look at the protocol and the network resource it is supposed to implement next to each other:



When we say that the left system behaves as the right system, we will only be concerned about the protocol ports LAC_S and LAC_R . After all, these are the ports modeling the actual IO behaviour of the protocol. The special ports just model all the open-ended parts of what the network resource might do. Yet, to know what the system actually outputs in response to inputs on the protocol ports, we also need to know what are the inputs on the special ports. We cannot require that the inputs on the special ports are the same too, as the special ports of the two systems might have different roles. After a moment of thought the right definition is to require that for each sequence of inputs to the protocol ports and special ports of $LC2LAC$ there exists a sequence of inputs to the protocol ports and special ports of LAC that makes it give the same outputs as $LC2LAC$. Furthermore, the input to the protocol ports must be the same in the two sequences, but the inputs to the special ports might differ.

There is an elegant way to turn that into a very precise definition. One requires that there exists a so-called **simulator** Sim that can be connected to the special ports of LAC such that the protocol and $LAC + Sim$ has exactly the same behaviour. Note that this means that the simulator has all the same special ports as LAC and LC . It will connect to the special port of LAC and hence the open special ports of Sim in $Sim + LAC$ will be exactly those of LC . That is why we call it a simulator: it pretends to be an LC . The job of the simulator is to make sure that $Sim + LAC$ behaves like $S + LC + KDC + R$, i.e., on any sequence of inputs they produce the same outputs.



Notice that after adding Sim the two systems have exactly the same open ports so it now makes sense to require that they have the same input-output behaviour.

Definition 2.1 (security) We say that a protocol π securely implements a network resource C if there exists an interactive agent Sim that connects only to the special ports of C and such that π and $C + Sim$ have the same input output behaviour. We write $\pi \sqsupseteq C$.

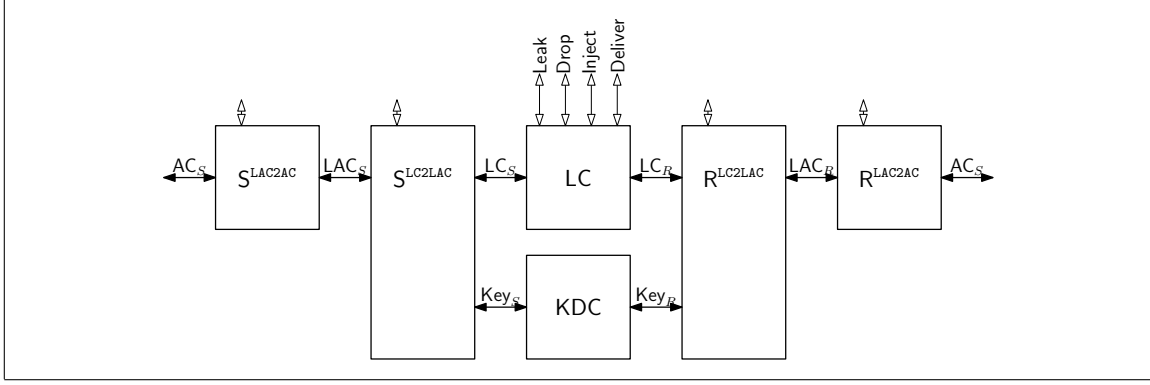


Figure 2.12: The composition $LC2AC$ of $LC2LAC$ and $LAC2AC$ running with network resource LC . In the picture we annotate S and R with the protocol from which the IA was taken to disambiguate the IA called S and R in the $LC2LAC$ and $LAC2AC$ protocols.

2.8 Composition

An important property of our model of security is so-called composition. It is easier to look at an example of what it means. Above we saw a protocol $LAC2AC$ securely implementing AC from LAC . We also saw a protocol $LC2LAC$ which implements LAC from LC . It seems reasonable that if we combine them then we can obtain a protocol $LC2AC$ which securely implements AC from LC . This is indeed the case. If we replace the LAC in $LAC2AC$ by the protocol $LC2LAC$ we get a protocol using the network resource LC and it is indeed the case that it implements AC . The composed protocol is depicted in Fig. 2.12.

It seems reasonable that $LC2AC$ securely implements AC . But can we be sure? Maybe there is some subtlety in the protocol which makes it secure when it uses LAC but insecure when it uses $LC2LAC$. Do we have to reprove security of the composed protocol to be on the safe side? It turns out that we do not have to reprove security. It can be proven once and for all that composition maintains security. If a protocol security implements a network resource then it can be plugged into any secure protocol using that resource, and security is maintained.

We will not prove here that our framework has secure composition. For the formally inclined reader, the topic is explored in more detail in Section 2.11. The topic of composition of secure protocols is explored in great detail in for example the work on the Universal Composition framework[4] introduced by Ran Canetti and the work on the Constructive Cryptography framework introduced by Ueli Maurer[11]. Our exposition here uses elements from both frameworks.

2.9 Specifying Liveness Properties

Notice that the above definition does not capture liveness properties. If $LC2LAC$ had a deadlock that at some point prevented it from ever delivering messages again, then the simulator could simply stop instructing LAC to deliver messages. The systems would have the same input-output behaviour. To have liveness we want that $LC2LAC$ is guaranteed to always make progress: if a message is sent then it is eventually delivered. It would, however, be unfair to require $LC2LAC$ to have this property unconditionally. It could be that LC drops all messages. In that case clearly $LC2LAC$ cannot deliver its own messages. So what we really want to require is that *if* the network resources used by the protocol (here KDC and LC) are live, then the protocol (here

LC2LAC) must also be live. So in some sense we want to require that the protocol is liveness preserving. For the case of LC2LAC it is easy to see that this is the case even without a formal definition.

It is perhaps a bit harder with LAC2AC: is LAC2AC liveness preserving? To answer this we first need to define what it means to be live for LAC and AC. There is no easy way to pick these definitions correctly, so the following discussion is going to be much less formal than what we did for safety properties:

- We say that **LAC** is live if it guarantees the following: any message that is sent “sufficiently many times” is eventually delivered.
- We say that **AC** is live if it guarantees the following: any message that is sent is eventually delivered.

The difference between the definitions take into account the properties of the two channels. Remember that the special ports of **LAC** allow the environment to drop messages. Therefore it does not make sense to ask that all messages are eventually delivered. On the other hand the channel **AC** does not have a special drop port, therefore messages will stay in the queue of the channel forever, waiting to be eventually delivered.

Finally note that defining what *eventually* means in a precise way is a very subtle task. However, it should be clear when looking at the LAC2AC protocol that if the underlying LAC channel guarantees to deliver any message that is sent “sufficiently many times”, then LAC2AC will also “eventually” deliver any message sent via the protocol.

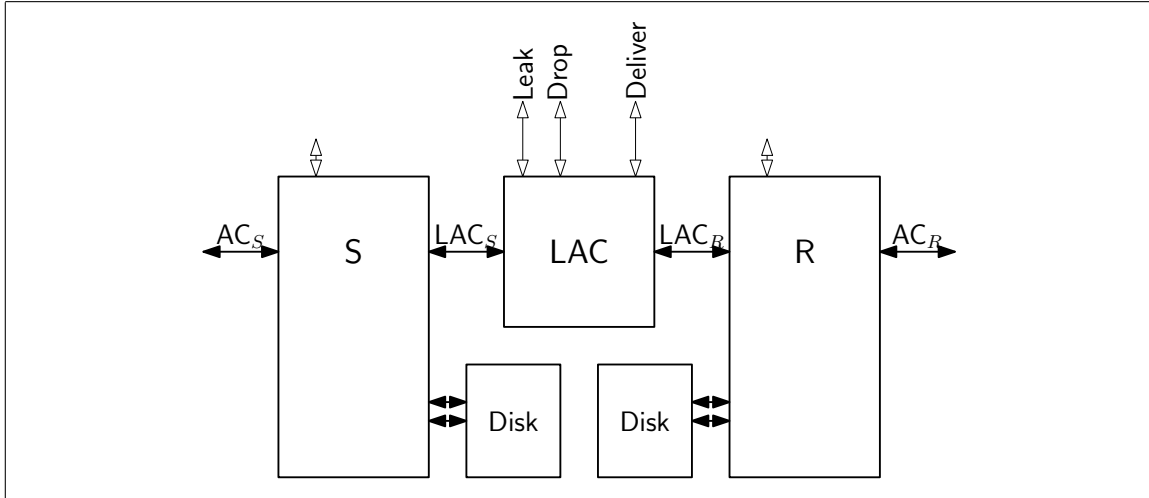


Figure 2.13: The architecture with a lossy channel and two disks

2.10 Handling Crashes

We now focus on handling crashes in the LAC2AC protocol. We will assume that both **S** and **R** can do crash-recovery in the crash-silent model (remember, this is the model in which a process crashes without sending any message to other processes). When they crash they come back to life in the initial state. Their disks survive the crash. We would like to make a system which ensures that if (R, m) is sent once at **S**, then (S, m) is delivered exactly once at **R**. A naïve solution would be the following: run as in LAC2AC, but keep the set of delivered messages

(called *Delivered* in the LAC2AC protocol) on disk. Note, however, that the following could happen: R outputs (S, m) . Then it crashes right before it gets to add (S, m) to *Delivered*. When it comes back to life, it will eventually receive (S, m) again and deliver it another time. One could then try to first add (S, m) to *Delivered* and only afterwards output it. But then clearly we get the opposite problem: R could crash right after adding (S, m) to *Delivered* and before outputting it. When it comes back to life it will find (S, m) in *Delivered* and will therefore never deliver (S, m) . The first implementation guarantees that the message is delivered at least once, but might make duplicates. The second implementation guarantees that a message is delivered at most once, but might never deliver it. What we really want is a channel which delivers the message exactly once. It turns out that this is impossible, at least with the current system architecture.

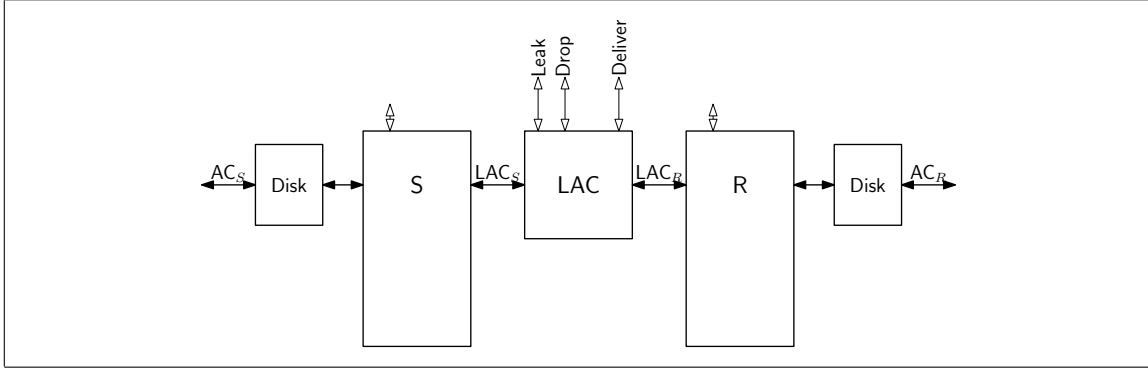


Figure 2.14: The architecture with a lossy channel and two disks used for input and output. If a party crashed the contents on the disk survives, so one cannot forget having gotten an input or having given an output.

We will use this as a chance to prove our first impossibility result. We will assume that a party cannot write to the disk and send a message in the same activation.³

Consider any protocol π for an architecture as in Fig. 2.13. We will prove that π cannot guarantee to deliver m exactly once, no matter how π works. The argument goes as follows. Send (R, m) on π and then activate all processes and channels sufficiently many times. Since π delivers m exactly once, there will be a point in time where R is about to output (S, m) . Call this point p . We can crash R just before p or just after and compare what will happen. If we crash it just before p , then it does not output (S, m) . So when it comes back to life, it will eventually deliver (S, m) at some future point q —recall that it delivers the message m once, so it must deliver it when it comes back to life, as it was not delivered yet. However, if we crash R right after delivering (S, m) and we let it recover, then it will be in exactly the same state as when we crashed it just before p . In fact, the entire system is in the same state. Hence if we run the system, from this state, then it will as before deliver (S, m) at q . But then it delivered (S, m) twice. This shows that you can never get *exactly once*.

Note that the above argument assumes that the system is crashed exactly at some bad point. Recall that our model of crash is that the environment picks the point where we crash. So maybe we could try to hide the time of the bad point from the environment. If we always place the bad point at the same point in the execution, or at some point which can be computed

³Technically, in our very simplified model a party *could* write the disk and then atomically send a message afterwards without the environment having time to crash the party in between. But this is an artefact of having used an oversimplified model of a disk. In real life a machine could of course crash between a disk access and sending a message. We model this simply by requiring the party to give the activation back between the two.

deterministically by the environment, it will not work. So we need to try to place the bad point at some random point.

Exercise 2.1 (probabilistic solution 1) Try to modify LAC2AC so that the system delivers (i.e., outputs to AC_R) a message at most once, and with probability $1/2$ will deliver it exactly once if the receiver can only be crashed once. ▼

Exercise 2.2 (probabilistic solution 2) Try to modify LAC2AC so that the system delivers (i.e., outputs to AC_R) a message at least once, and with probability $1/2$ will deliver it exactly once if the receiver can be crashed at most once. Can you get a higher probability? ▼

A popular solution to the problem that you have to pick at-least-once or at-most-one is to pick at-least-once and then make sure that the outer system using the channel can tolerate that a message is delivered more than once. One can for instance use sequence numbers.

Another popular solution is to define that a message m is delivered when R added (S, m) to its disk. Similarly it is said to be sent when (R, m) was added to the disk of S.

Exercise 2.3 (more crashes) Make a system where outputs are defined to be given via the disks and which can tolerate crash-recovery of both S and R and which implements *exactly-once*. In a bit more detail, there is a sender S and a receiver R. Each of them have a disk. The users of the channel can also write to their disk. See Fig. 2.14 for an illustration. A message m is said to have been sent when it is written to the disk of S by the user of S. It is said to have been delivered when it is written to the disk of R by R. There it will be picked up by the user of R.

- Show how to get exactly once using both disks.
- Can you get exactly-once while only using the disk of R? You can assume the message will be input at S again and again until S output DELIVERED.
- Can you get exactly-once while only using the disk of S?

▼

In the above solutions you probably left something on disk even after the protocol was done. It turns out that you have to.

Exercise 2.4 (cleaning up) Show that in a protocol for the architecture in Fig. 2.14 it is not possible to get a protocol which has exactly once delivery and where the disks are always completely empty again after the protocol terminated. ▼

2.11 Composition††

An important property of a model of security is so-called composition. It is easier to look at an example of what it means. Above we saw two protocols, LC2LAC and LAC2AC. The first one securely implements LAC from LC and is liveness preserving. The second one securely implements AC from LAC and is liveness preserving. It seems intuitive that if we combine the protocol, then we get a protocol which securely implements AC from LC. The composed protocol would look as in Fig. 2.12.

Can we be sure that the combined protocol securely implements AC? Can we be sure that it is liveness preserving?

Composability is an essential property of any definition of security. If composability holds in the model, then we can prove the properties of the combined protocol in a modular way. This

allows us to give a simpler and modular analysis where each step focuses only on one small part of the system (for instance, turning a lossy channel into a non-lossy channel). Then, composability would guarantee that security of the individual steps gives security of the composed protocol. If a model does *not* satisfy composability, then each new protocol like LC2AC would have to be analysed from scratch.

Fortunately for us the model we gave above satisfies composability. Proving this in detail is out the scope of this book, but we would like to give the interested reader an idea why this is the case.

Below we list some results about how the model composes, with some intuitive explanations. A good way to remember the composition rules is basically that behavioural equivalence, $\equiv$, is like equality on integers and secure implementation, $\sqsubseteq$, behaves as $\geq$ on integers. Another popular way to read $S_1 \sqsubseteq S_2$ is as " S_1 is at least as secure as S_2 ".

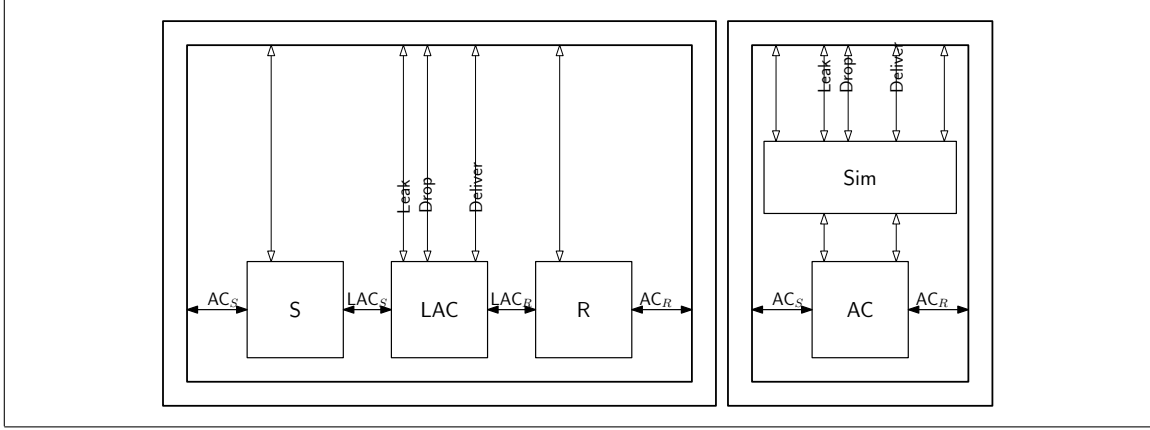


Figure 2.15: Comparing LAC2AC+LAC and AC+Sim using an environment $\mathcal{Z}$. The environment is shown as the IA surrounding the other systems.

First we need to define behavioural equivalence with a bit of care. Recall that we say that two systems S_0 and S_1 behave the same if they have the same outputs given the same inputs. It turns out this is too strict. It is OK if there is a slight difference in the probability of outputting one particular value as long as this cannot be noticed by any plausible attacker. So, if we want to compare two system like LAC2AC + LAC and AC + Sim we would like to say that it should hold for all plausible adversaries $\mathcal{Z}$ that it cannot guess whether it is in the setting $\mathcal{Z} + (\text{LAC2AC} + \text{LAC})$ or the setting $\mathcal{Z} + (\text{AC} + \text{Sim})$. See Fig. 2.15 for an illustration.

We will sketch how to define this. First let $S_0 = \text{LAC2AC} + \text{LAC}$ and $S_1 = \text{AC} + \text{Sim}$. Then sample a uniformly random bit $b \in \{0, 1\}$. Then run $S_b + \mathcal{Z}$. Assume that $\mathcal{Z}$ always outputs a guess $g \in \{0, 1\}$ at the end. This is a guess at whether it played with S_0 or S_1 . It can always make a random guess and hence win with probability $\frac{1}{2}$. So to do well it needs to be correct with probability *better* than $\frac{1}{2}$. How much it is better is called its advantage. We write it as

$$\text{Adv}_{\mathcal{Z}}(S_0, S_1) = \Pr[g = b] - 1/2 .$$

We say that the systems behave *exactly the same* if it holds for *all* $\mathcal{Z}$ that it guesses correct with probability at most $\frac{1}{2}$, i.e., $\text{Adv}_{\mathcal{Z}}(S_0, S_1) \leq 0$. We say that S_0 and S_1 are *behaviourally equivalent* if for all *plausible* $\mathcal{Z}$ it holds that $\mathcal{Z}$ guesses correctly with a probability *not too much higher* than $\frac{1}{2}$. To define "plausible" we could for instance pick some number of computational steps that we think no adversary can perform in practice, like 2^{100} , and say that a plausible adversary is one that performs less than 2^{100} computational steps. To define "not too much

higher than” we need to pick some probability ϵ that we consider negligible, for instance $\epsilon = 2^{-80}$. Then we say that the adversary should not be able to guess correctly with probability better than $1/2 + \epsilon$. When we take an IA like $\mathcal{Z}$ which connects to all open ports of S_0 (and S_1) we call it an **environment** of S_0 (and S_1). This gives us a definition as follows.

Definition 2.2 (behavioural equivalence (informal)) Let S_0 and S_1 be systems with the same open protocol ports and the same open special ports. We say that S_0 and S_1 are behaviourally equivalent if for all plausible environments $\mathcal{Z}$ there is a negligible ϵ such that $\text{ADV}_{\mathcal{Z}}(S_0, S_1) \leq \epsilon$.

The notion of secure implementation can be extended to any two systems, not just as a relation between protocols and network resources.

Definition 2.3 (general secure implementation) Let S_1 and S_2 be systems with the same protocol ports and possibly different special ports. We say that S_1 securely implements S_2 , written as $S_1 \sqsupseteq S_2$, if there exists an efficient simulator Sim such that $S_1 \equiv S_2 + \text{Sim}$.

We already argued why this definition makes sense. The reason why we require Sim to be efficient is that otherwise composition does not work. The definition of ”efficient” we leave out, but it should hold that any ”efficient” IA is also ”plausible”. Note how the definition parallels the definition of $\geq$ on natural numbers: We say that $s_1 \geq s_2$ if there exists non-negative s such that $s_1 = s_2 + s$.

Assumption 2.4 (transitivity of $\equiv$) Let S_1 , S_2 , and S_3 be systems such that $S_1 \equiv S_2$ and $S_2 \equiv S_3$. Then $S_1 \equiv S_3$.

It would be strange if a notion of ”behaving the same” was not transitive, so the above ought to be a triviality. But note that we did not define $\equiv$ very precisely yet. This can be done such that it has the above property. There are some technicalities to be handled carefully, but it is not too complicated.

Note that the parallel statement for natural number is trivial: if $s_1 = s_2$ and $s_2 = s_3$, then $s_1 = s_3$.

Assumption 2.5 (extension of $\equiv$ (informal)) Let S_1 and S_2 be systems for which $S_1 \equiv S_2$. Let S be any efficient system. Then $S_1 + S \equiv S_2 + S$.

This again ought to be the case if $\equiv$ has been defined correctly. If S_1 and S_2 give the same outputs on the same inputs, so clearly does $S_1 + S$ and $S_2 + S$. The S_i parts of the systems behave the same, and the S part is the same. So $S_1 + S$ and $S_2 + S$ will behave the same. Again, one will craft the definition of $\equiv$ and ”efficient” to have this property.

The parallel statement for natural numbers would be: if $s_1 = s_2$ then $s_1 + s = s_2 + s$.

Lemma 2.6 (extension of $\sqsupseteq$) Let S_1 and S_2 be systems for which $S_1 \sqsupseteq S_2$. Let S be an efficient system only connecting to the protocol ports of S_1 and S_2 . Then $S + S_1 \sqsupseteq S + S_2$.

PROOF The reason is that $S_1 \sqsupseteq S_2$ implies that there exists Sim such that $S_1 \equiv S_2 + \text{Sim}$. Since Sim connects only to special ports of S_2 and S connects only to protocol ports, it makes sense to consider the system $(S_2 + \text{Sim}) + S$. From $S_1 \equiv S_2 + \text{Sim}$ we get that $S_1 + S \equiv (S_2 + \text{Sim}) + S$. Clearly $(S_2 + S) + \text{Sim} = (S_2 + \text{Sim}) + S$. Putting these together we get that $S_1 + S \equiv (S_2 + S) + \text{Sim}$. This shows that $S_1 + S \sqsupseteq S_2 + S$, with Sim being the required simulator. $\square$

The parallel statement for natural numbers would be: if $s_1 \geq s_2$ then $s_1 + s \geq s_2 + s$.

Lemma 2.7 (transitivity of $\sqsubseteq$) *Let S_1, S_2, S_3 be systems for which $S_1 \sqsubseteq S_2$ and $S_2 \sqsubseteq S_3$. Then $S_1 \sqsubseteq S_3$.*

PROOF From $S_1 \sqsubseteq S_2$ we get Sim_1 such that $S_1 \equiv S_2 + \text{Sim}_1$. From $S_2 \sqsubseteq S_3$ we get Sim_2 such that $S_2 \equiv S_3 + \text{Sim}_2$. By extension of $S_2 \equiv S_3 + \text{Sim}_2$ with Sim_1 we get that $S_2 + \text{Sim}_1 \equiv S_3 + \text{Sim}_2 + \text{Sim}_1$. From transitivity of $\equiv$ and $S_1 \equiv S_2 + \text{Sim}_1$ and $S_2 + \text{Sim}_1 \equiv S_3 + \text{Sim}_2 + \text{Sim}_1$ we get that $S_1 \equiv S_3 + \text{Sim}_2 + \text{Sim}_1$. If we let $\text{Sim} = \text{Sim}_2 + \text{Sim}_1$, this gives us that $S_1 \equiv S_3 + \text{Sim}$, which shows that $S_1 \sqsubseteq S_3$. $\square$

The parallel statement for natural numbers would be: if $s_1 \geq s_2$ and $s_2 \geq s_3$, then $s_1 \geq s_3$.

Theorem 2.8 (general universal composition of $\sqsubseteq$) *Let S_1, S_2, S_3 , and S_4 be efficient systems for which $S_1 \sqsubseteq S_2$ and $S_3 + S_2 \sqsubseteq S_4$. Then $S_3 + S_1 \sqsubseteq S_4$.*

PROOF By extension of $S_1 \sqsubseteq S_2$ with S_3 we get that $S_1 + S_3 \sqsubseteq S_2 + S_3$. Then use $S_3 + S_2 = S_2 + S_3$ and $S_3 + S_2 \sqsubseteq S_4$ and transitivity of $\sqsubseteq$ and you get that $S_1 + S_3 \sqsubseteq S_4$, as desired. $\square$

From the above we have get the following result for composability of secure protocols.

Corollary 2.9 (universal composition of $\sqsubseteq$) *Let $\mathcal{N}_1, \mathcal{N}_2$, and $\mathcal{N}_3$ be network resources. Let Π_1 and Π_2 be protocols. for which $\Pi_1 + \mathcal{N}_1 \sqsubseteq \mathcal{N}_2$ and $\Pi_2 + \mathcal{N}_2 \sqsubseteq \mathcal{N}_3$. Then $\Pi_1 + \Pi_2 + \mathcal{N}_1 \sqsubseteq \mathcal{N}_3$.*

PROOF Use general universal composition with $S_1 = \Pi_1 + \mathcal{N}_1, S_2 = \mathcal{N}_2, S_3 = \Pi_2, \mathcal{N}_3 = S_4$. $\square$

The parallel statement for natural numbers would be: if $s_1 \geq s_2$ and $s_3 + s_2 \geq s_4$, then $s_3 + s_1 \geq s_4$.

2.12 On Defining Eventuality $\dagger\dagger$

For the formally inclined reader, we want to add a few details about how to define "eventual".

We take LAC as an example. Consider LAC as sitting in the LAC2AC protocol. We would like to talk about running this system. For that we need someone or something to give it inputs. For this purpose, consider an adversary $\mathcal{Z}$ connected to all the open ports of the system. This is also sometimes called an *environment*. The environment has no other ports than those connected to LAC2AC, except that it has one open port which is its activation port. This is illustrated to the left in Fig. 2.15.

We can then run $\text{LAC2AC} + \mathcal{Z}$ by activating $\mathcal{Z}$ and letting it play with LAC2AC as it wants until it terminates. So it is the environment that gives inputs on the protocol ports, receives outputs on the protocol ports and which controls all the special ports. The execution will proceed via a number of activations of LAC2AC. We want to force $\mathcal{Z}$ to eventually deliver messages sent on LAC. We can do this as follows: When the execution begins, $\mathcal{Z}$ commits to an integer $E > 0$, say by writing it down in some designated part of its state. This is now its *eventuality bound*. We can for instance use the bound E as follows: if the same message is sent on LAC at least E times, then $\mathcal{Z}$ does not drop it all E times. Furthermore, if $\mathcal{Z}$ activated the system LAC2AC at least E times while there was a message in transit in LAC2AC, it must deliver at least one of these messages. These two restrictions on $\mathcal{Z}$ forces it to sometimes deliver some message that was sent often enough. We call such environments *E-restricted*, and we write $\mathcal{Z}^E$. When we say that LAC eventually delivers a message, we then simply mean that we only consider executions that take place in *E-restricted* environment.

Note that we could make a similar definition for AC. Consider an environment $\mathcal{Z}'$ connected to all ports of AC. We say that $\mathcal{Z}'$ is *E'-restricted* if whenever it ran for E' activations while some message was in transit in AC, it will deliver at least one such message.

Recall then that we define security by requiring that there exists a simulator Sim such that $(S + \text{LAC} + R)$ and $\text{AC} + \text{Sim}$ have the same behaviour. This in particular means that $(S + \text{LAC} + R) + \mathcal{Z}$ and $(\text{AC} + \text{Sim}) + \mathcal{Z}$ have the same behaviour for all $\mathcal{Z}$. This trivially implies that $(S + \text{LAC} + R) + \mathcal{Z}^E$ and $(\text{AC} + \text{Sim}) + \mathcal{Z}^E$ have the same behaviour for all E -restricted $\mathcal{Z}^E$. Note that we can rewrite $(\text{AC} + \text{Sim}) + \mathcal{Z}^E$ as $\text{AC} + (\text{Sim} + \mathcal{Z}^E)$. In that case we can consider $\mathcal{Z}' = \text{Sim} + \mathcal{Z}^E$ as an environment for AC .

Recall that we want to define that LAC eventually delivers messages when running in $(S + \text{LAC} + R) + \mathcal{Z}^E$. But since this is a property that can be verified by only looking at the ports of the system and $(S + \text{LAC} + R) + \mathcal{Z}^E$ and $(\text{AC} + \text{Sim}) + \mathcal{Z}^E$ have the same behaviour on the open ports, this is the same as requiring that AC eventually delivers messages in $(\text{AC} + \text{Sim}) + \mathcal{Z}^E$, which is the same as saying that AC eventually delivers messages in $\text{AC} + \mathcal{Z}'$. Recall then that we say that AC eventually delivers messages if $\mathcal{Z}'$ is E' restricted for some integer E' . This leads to the following simple definition: For all E there exists E' such that $\text{Sim} + \mathcal{Z}$ is E' -restricted when $\mathcal{Z}$ is E -restricted. We call such a simulator *liveness preserving*.

To see that the definition makes sense, note that it essentially just says that if Sim is simulating an execution where LAC eventually delivers messages, then Sim must also eventually deliver messages. And since the simulation looks like the real protocol, we get that in an execution where LAC eventually delivers messages, the protocol LAC2AC also eventually delivers messages.

So, we can recap the definitions as follows: We call $S + \text{LAC} + R$ a secure implementation of AC if there exists a simulator Sim such that $S + \text{LAC} + R$ and $\text{AC} + \text{Sim}$ behave the same. We call $S + \text{LAC} + R$ a secure *and live* implementation of AC if there exists a *liveness preserving* simulator Sim such that $S + \text{LAC} + R$ and $\text{AC} + \mathcal{Z}$ behave the same.

Chapter 3

Communication (2026)

Reading Guide *This chapter serves several purposes. First of all, it recaps how the internet is built, including IP, TCP and DNS. This is not directly curriculum in this course. We will for instance not ask questions to how TCP works at the oral exam. You are assumed to know this from previous courses. Read this part of the chapter on the internet as a recap if you need a refreshment. Second, it introduces how to use the internet in Golang (the Go programming language) and gives some advise on how to program distributed systems. This is curriculum, but is covered via the mandatory exercises. You are required to use Golang in the exercises, and passing the mandatory part is the “exam” of the Golang part. We will not ask questions about the details of Golang at the oral exam. Last, the chapter introduces some concepts special to programming distributed systems, like marshalling, remote procedure calls, and how to organise machines logically. This is curriculum and might be queried at the oral exam.*

Contents

3.1	Types of Communication	52
3.2	Naming	53
3.3	Let’s Go!	55
3.4	The Internet	55
3.4.1	Physical Layer	56
3.4.2	Data Link Layer	56
3.4.3	Network Layer	57
3.4.4	Transport Layer	58
3.4.5	UDP	58
3.4.6	TCP	60
3.4.7	Naming, DNS	62
3.5	Programming Distributed Systems	62
3.5.1	Local atomicity	62
3.5.2	Roll forward, don’t rollback	65
3.5.3	Don’t rely on others	65
3.5.4	Failure is normal	66
3.5.5	Keep it simple and modular	66
3.6	Concurrency	66
3.7	Marshalling	71
3.8	Remote Procedure Calls	75
3.9	Logically Organising Connections	75

3.9.1	Fully Connected	75
3.9.2	Client-Server	77
3.9.3	Client-Replicated Server	77
3.9.4	Edge Nodes	77
3.9.5	SMTP	79
3.10	Unstructured Peer-to-Peer and Flooding	80
3.10.1	How to Build and Maintain	81
3.10.2	Gossiping	82
3.10.3	Eclipsing Attacks	82
3.11	Structured Peer-to-Peer	85

3.1 Types of Communication

The glue of a distributed system is communication, which is the subject of this chapter. We will see how the Internet allows you to communicate between two entities and how to organize communication in systems with more than two entities, including a toy example of a peer-to-peer network. We will also touch upon **naming**: how you give names to entities in a distributed system and how you locate them given that name. This is what the DNS system does on the Internet.

There are two main types of communication:

Message Passing: Processes send messages to each other and the transfer is typically between only two parties, the sender and the receiver, and the sending and the receiving are separated in time. If you communicate by playing a drum or sending smoke signals, it is message passing. E-mail and TCP/IP are also examples of message passing.

Shared Memory Space: Processes are in a setting where they can see the same variables. They communicate by writing and reading these variables. Different cores and threads on the same computer communicate via a shared memory space (the memory of the computer). Applications built on top of a database work via shared memory space, the database. In the introduction we hinted at how to implement a shared variable using message passing, by using appropriate quorums for reading and writing.

Both types of communication are worthy of being the subject of an entire book on their own. However, the course for which this book is written does not allow us the time to look into both of them in depth. Since many of the underlying principles of achieving secure distributed systems can be learned by studying either type of communication, and we want to go into some technical depth, we have chosen to focus on one of them. We have chosen message passing as we believe it will be an easier introduction to the central concepts. In addition, it allows us to set the scene for a chapter on blockchains and cryptocurrencies, which we believe any modern book on secure distributed system should cover.

Another common distinction between types of communication is as follows:

Transient: The receiver has to be alive when the message is sent to see it. If the receiver is not alive when the message is sent, the message will be lost. An example is TCP/IP.

Persistent: The message can be seen even if the receiver comes alive later. Examples are programs running on top of databases and e-mail.

Another common distinction between types of communication is as follows:

Synchronous: When sending a message, the sender waits until the receiver received the message before it continues its own execution. Similarly, if the receiver is to receive several messages, it receives them sequentially. This is a simple way to program as there is not so much concurrency, but it can be terribly slow. If it takes a second to send and process an element, and you have to send 100 elements from the same sender to the same receiver, it can be minutes instead of seconds if you do it synchronously.

Asynchronous: When sending a message, the sender continues its own execution while the message is being sent. Similarly, if the receiver is to receive several messages, it does so using separate threads. For instance, when you download a page using HTTP, then each element, for instance pictures, are sent in parallel from the server to the browser. This can speed up the application considerably. The flip side is that it is harder to program in this style.

A final common distinction between types of communication is as follows:

Push: Signals propagate by the senders actively sending them. This is a good communication pattern if messages come at random times or receivers are few and known. It is also a good pattern if you are sure the receiver needs the data. A canonical example is e-mail.

Pull: Signals propagate by the receivers asking for them. This is a good pattern if messages come at regular times or readers are many or unknown. It is also a good pattern if not all clients might need the data. An example is reading your favourite news site in the morning.

These patterns can be combined. You might for instance push a small notification to all clients that a large update is available. Then the clients will pull the update if they need it.

Exercise 3.1 (Shared Variable Space from Message Passing) Revisit Example 1.2 on replication. Recall that we considered only one client there. If there are two clients, the system does not work. It could happen that client 1 reads the current file, applies change $f_1 = C_1(f)$. Then client 2 reads the current file (before client 1 wrote back its change), applies change $f_2 = C_2(f)$. Then client 1 writes back f_1 and gets confirmations. Then client 2 writes back f_2 and gets confirmations. Now client 1 considers the change as successful, yet C_1 was never applied. The value stored in the system is $C_2(f)$. What we want if two clients each make a change is that the end state of the system stores either $C_1(C_2(f))$ or $C_2(C_1(f))$. Try to design a system which tolerates this if all servers and clients are correct. Can you make it work even if there could be one Byzantine error among the servers? Can you make it work if a client might be Byzantine corrupted? ▼

3.2 Naming

An important topic in the vicinity of communication is **naming**. This has to do with how we name things in a distributed system and how we find them. There are many naming systems, and we will look briefly at DNS below. There is some common terminology relating to naming systems which is convenient to know.

Entity Entities are the “things” being named. This could be files, computers, processes or even persons. These entities could be distributed geographically.

```

package main

import ( "fmt" )

// A function declaration: Types after variable names; Multiple return values
func multi(x int, y int) (string, int) {
    return "The answer is", x+y
}

// A class
type Named struct {
    name string // Member of a class
}

// Method on a class
func (nm *Named) PrintName(n int) {
    if n < 0 { panic(-1) } // an "exception"
    for i:=0; i<n; i++ {
        fmt.Print(nm.name + "\n")
    }
}

func main() {
    var i int // Explicit declaration of variable
    i = 21    // = is used for assignment
    j := 21   // := declares the variable based on the type of the value

    decr := func() int { // a closure
        j = j-7
        return j
    }

    str, m := multi(i, j)
    defer fmt.Println(str, m) // run after return or a panic

    fmt.Println(decr())
    fmt.Println(decr())

    nm1 := Named{ name: "Jesper" } // value
    nm2 := &Named{} // pointer

    nm1.PrintName(2) // Calling a method
    nm2.PrintName(2) // . does auto-dereference of pointer
    nm2.name = "Claudio" ; nm2.PrintName(2) // RETURN or ; as separator
    var nm3 *Named = nm2; // how to explicitly declare a pointer
    nm2.name = "Ivan"
    nm3.PrintName(2)
    nm3.PrintName(-1) // let us cause a panic
    fmt.Println("Will we make it here?")
}

```

Figure 3.1: Go by example.

Name A name is just a string of bits (or characters) used to refer to entities. An example could be `www.au.dk`. A name can refer to several entities and each entity might be referred to by many names.

Identifier An identifier is a unique name. Within a given system each entity has at most one identifier and each identifier names at most one entity. In Denmark the CPR number is an example of an identifier.

Access Point An access point is a special entity at which one can find other entities. You access the entities via the access point. For instance a web server of Aarhus university is an access point for the files on that server. You access a file like `www.au.dk/file.txt` by asking the access point for `file.txt`.

Address An address is just the name of an access point. An entity can change address, for instance if a file is moved from one web server to another.

Name Resolution Name resolution is the process of finding an entity given a name for it. For instance to find `www.au.dk/file.txt` you first resolve the address `www.au.dk` to find the IP address of the web server. Then you ask the web server for `file.txt`.

3.3 Let's Go!

We are going to present code examples in the Go programming language as it is well suited for distributed programming and allows for concise code. It is a static typed language. It is object oriented without inheritance. All types can be used as values or pointers. It allows for first order functions, a.k.a. lambda closures. If you programmed in C, C++ or Java, the syntax should not surprise you, except that it takes a while to get used to the type being mentioned *after* the name. In Fig. 3.1 there is an example of a Go program.¹ When it is run it produces the following:

```
14
7
Jesper
Jesper

Claudio
Claudio
Ivan
Ivan
The answer is 42
panic: -1
<program terminated>
```

3.4 The Internet

In this section we will briefly take a look at how we come from a situation where some machines are connected by copper cables and electromagnetic waves to a situation where any machine on the Internet in principle can send a messages to any other machine on the Internet. The explanation will be very short indeed. This is not to underestimate the importance of this

¹For a Tour of Go, see for instance <https://tour.golang.org>.

```

package main

import ( "net" ; "os" ; "fmt" ; "strconv" )

func main() {
    name, _ := os.Hostname()
    addrs, _ := net.LookupHost(name)
    fmt.Println("Name: " + name)
    for indx, addr := range addrs {
        fmt.Println("Address number " + strconv.Itoa(indx) + ": " + addr)
    }
}

```

Figure 3.2: A Go program which will display the name and IP address of your machine. Try the program to make sure you can run Go. Also try to understand the output of the program. We take the chance to introduce Go a bit more: When we use an underscore as a receiving variable it just means that we will not be using that value. The **range** keyword can be used to iterate over elements in a variety of data structures. Here **addrs** is an array of strings (containing the possible names of the host). In the loop **indx** iterates over all entries in the array and **addr = addrs[indx]**.

engineering achievement: Creating the Internet is one of the biggest engineering achievements of humanity. However we only scratch the surface of this important topic. First of all, we assume the readers of this book have already taken courses covering the basics of how to build and use TCP/IP and DNS, the backbone(s) of the Internet. Second once the Internet gives you the ability to send a message between two machines, thus establishing the smallest meaningful distributed system—two machines—and the simplest distributed task—sending a message, another equally daunting task remains ahead of us: to use this ability in a principled way to construct secure distributed systems consisting of a large collection of machines acting in concert to solve some distributed task like implementing a blockchain. We believe that studying this problem most effectively teaches the principles which will allow you to solve the problems you will meet in designing, programming, and using the distributed systems of today and tomorrow.

We expect that the reader has familiarity with the OSI (Open System Interconnectivity) model, but will recap the main layers in this section for self containment.

3.4.1 Physical Layer

There are several physical media for sending a message from two geographically separated machines, like WIFI, copper cable and pigeons. In the ISO/OSI model this is called the physical layer. It is used to move a raw unstructured package of data between two physically connected locations. How this is done depends on the underlying technology.

3.4.2 Data Link Layer

The data link layer provides a mean to send a stream of data from one node to another, including how to establish, maintain and close a connection. It detects (and possibly corrects) errors that may occur in the physical layer. But at this layer connections can only be established between two nodes that are physically connected via the physical layer. It is in turn broken into two layers:

```

package main

import ( "fmt" ; "net" )

func main() {
    ServerAddr, _ := net.ResolveUDPAddr("udp", ":10001")
    ServerConn, _ := net.ListenUDP("udp", ServerAddr)
    defer ServerConn.Close()
    buf := make([]byte, 1024)
    for {
        n, addr, err := ServerConn.ReadFromUDP(buf)
        fmt.Println("Received ", string(buf[0:n]), " from ", addr)
        if err != nil {
            fmt.Println("Error: ", err)
        }
    }
}

```

Figure 3.3: Here a UDP server receives UDP packages and prints them and a UDP client sends some packages to be printed. It listens on port 10001, which is just an arbitrary port we picked for this example. Try running them on your machine to make sure Internet works on your machine. Try running several clients at the same time. Try modifying the client and server to make them do something more interesting to get used to Go and IP. We learn a bit more Go: In Go arrays are pointer types, so when you declare an array, it is `nil` and if you try to index into it you will get a panic. Therefore, first you have to make an array of the right type and store the variables in it before you can use it. This is what happens in `buf := make([]byte, 1024)`.

Medium access control: The MAC layer is responsible for controlling how devices in a network gain access to a medium and permission to transmit data.

Logical link control: The LLC layer is responsible for identifying and encapsulating network layer protocols, and possibly has mechanisms for error detection and retransmission.

3.4.3 Network Layer

As the name indicates, the network layer provides the functional and procedural means for sending data from one node to another node to which it is connected only indirectly via a network of other nodes across “different networks”. The network layer sends so called packets i.e., variable length data sequences. To be able to send packets to a node somewhere else in the network, every node has an address. When you want to send a packet to some address, you simply give the address and packet to your OS which will then use the network layer to find a way to deliver the message to the destination node, possibly routing it through intermediate nodes. If the message is too large to be transmitted from one node to another on the data link layer between those nodes it will be split into several fragments at one node, each fragment is sent independently, and all of them are reassembled at another node.

The best known Network Layer is the Internet Protocol (IP) protocol. In IP (version 4) an address consists of 32 bits, typically shown as 4 octets of numbers from 0–255 represented in decimal form, for example 198.41.0.4. The address 127.0.0.1 has the special semantics of always pointing to the local machine.

We assume that the reader has a rudimentary understanding of IP and UDP, for example from a course on networks and operating systems.

```

package main

import ( "net" ; "time" ; "strconv" )

func main() {
    ServerAddr, _ := net.ResolveUDPAddr("udp", "127.0.0.1:10001")
    LocalAddr, _ := net.ResolveUDPAddr("udp", "127.0.0.1:0")
    conn, _ := net.DialUDP("udp", LocalAddr, ServerAddr)
    defer conn.Close()
    i := 0
    for {
        i++
        msg := strconv.Itoa(i)
        conn.Write([]byte(msg))
        time.Sleep(time.Second * 1)
    }
}

```

Figure 3.4: A UDP client sending packages to port 10001 on the local machine. If the server is running on another machine, the 127.0.0.1 would have to be replaced by the IP address of that machine.

```

package main

import ( "net" ; "fmt" )

func main() {
    ln, _ := net.Listen("tcp", ":")
    defer ln.Close()
    _, port, _ := net.SplitHostPort(ln.Addr().String())
    fmt.Println("Listening on port " + port)
}

```

Figure 3.5: A Go program which starts listening on a random local port.

3.4.4 Transport Layer

In OSI the transport layer is meant to transfer data sequences of varied lengths from a source to a destination host. In the Internet protocol suite this is maintained by the UDP and TCP protocols.

3.4.5 UDP

The User Datagram Protocol (UDP) runs on top of IP. The communication model is message oriented and stateless. Once a message is sent the sender forgets about the message. So if the message is dropped on the way, it will not be resent and the receiver will never receive it unless higher level applications ensures this themselves. Two things UDP does add compared to IP is checksums for data integrity and port numbers in order to multiplex the single channel between two machines offered by IP. A local service can register on a port with the OS to listen for packets on that port. Remote services can send packets to a specific port to have them arrive at the registered process. Multiple clients can send packets to the same port. Only one process can register locally at a port.

UDP ports are 16 bit numbers or equivalently integers between 0 and 65535. Port 0 is reserved, but is a permissible source port value if the sending process does not expect messages

```

package main

import (
    "bufio" ; "fmt" ; "net" ; "strings"
)

func handleConnection(conn net.Conn) {
    defer conn.Close()
    reader := bufio.NewReader(conn)
    for {
        msg, err := reader.ReadString('\n')
        if err != nil {
            fmt.Println("Error: " + err.Error())
            return
        } else {
            fmt.Println("From Client:", string(msg))
            titlemsg := strings.Title(msg)
            conn.Write([]byte(titlemsg))
        }
    }
}

func main() {
    fmt.Println("Listening for connection...")
    ln, _ := net.Listen("tcp", ":18081")
    defer ln.Close()
    conn, _ := ln.Accept()
    fmt.Println("Got a connection...")
    handleConnection(conn)
}

```

Figure 3.6: A TCP Server. We learn some more Go. The function `strings.Title` simply capitalises each word in the string. The `defer` keyword defers the function call until the enclosing function terminates. It is a good way to ensure that for instance connections are closed no matter what happens later: The deferred work is executed no matter how the function is exited, even if there is a panic. If you put the closing of connections at the end of the function, they would not be closed if you exit the function earlier in the code or the function terminated because of a panic.

```

package main

import (
    "bufio" ; "fmt" ; "net" ; "os" ; "strings"
)

var conn net.Conn

func main() {
    conn, _ = net.Dial("tcp", "127.0.0.1:18081")
    defer conn.Close()
    reader := bufio.NewReader(os.Stdin)
    connReader := bufio.NewReader(conn)
    for {
        fmt.Print("> ")
        text, err := reader.ReadString('\n')
        if text == "quit\n" {
            return
        }
        fmt.Fprintf(conn, text)
        msg, err := connReader.ReadString('\n')
        if err != nil {
            return
        }
        // Windows uses \r\n as a return character, TrimSpace removes the extra '\r' character
        msg = strings.TrimSpace(msg)
        fmt.Println("From server: " + msg)
    }
}

```

Figure 3.7: A TCP Client

in response. The port numbers are partitioned into three ranges. Port numbers 1 to 1023 are used for standard services, for instance SSH is assigned to port 22 and HTTP is on port 80 by default. On ports 1024 to 49151 you can register your own protocol with the IANA organisation. For instance Bitcoin is registered to port 8333. Ports 49152 to 65535 can be used for any purpose. Typically one picks a random port. Trying to pick a given hardcoded port is dangerous as it might already be taken. When writing an IP address and a port number together it is common to use the format 127.0.0.1:80.

3.4.6 TCP

The Transmission Control Protocol (TCP) is another communication protocol run on top of IP. As opposed to UDP it provides a connection or stream-oriented communication model. The sender provides a continuous stream of bytes, which TCP moves to the receiver. TCP moves data between ports as does UDP. Moreover, TCP adds some level of reliability: it detects simple corruption of data along with loss or reordering of data. This is done by adding checksums and sequence numbers to packets. However TCP does not provide any authenticity, since any node through which the data is sent could change that data, and then just change the checksums and sequence numbers appropriately.

We assume that the reader has a rudimentary understanding of TCP, for example from a course on networks and operating systems.

```

package main

import ( "net" ; "fmt" ; "strconv" )

func main() {
    addrs, _ := net.LookupHost("google.com")
    for indx, addr := range addrs {
        fmt.Println("Address number " + strconv.Itoa(indx) + ": " + addr)
    }
}

```

Figure 3.8: Let us look up the IP address of `www.google.com`.

```

package main

import ( "net" ; "fmt" ; "bufio" )

func main() {
    addrs, _ := net.LookupHost("www.google.com")
    addr := addrs[0]
    fmt.Println(addr)
    conn, err := net.Dial("tcp", addr+":80")
    if (conn!=nil) { defer conn.Close() }
    if (err!=nil) { panic(0) }
    fmt.Fprintf(conn, "GET /search?q=Secure+Distributed+Systems HTTP/1.1\n")
    fmt.Fprintf(conn, "HOST: www.google.com\n")
    fmt.Fprintf(conn, "\n")
    for {
        msg, err := bufio.NewReader(conn).ReadString('\n')
        if err != nil { panic(1) }
        fmt.Println(msg)
    }
}

```

Figure 3.9: Let us ask Google a question.

3.4.7 Naming, DNS

We assume that the reader has a rudimentary understanding of Domain Name System (DNS), for example from a course on networks and operating systems. But for completeness we will recap the DNS system here. It is a decentralized naming service which translates human readable names like `www.google.com` into IP addresses. This in turn allows one to contact for instance `www.google.com` via IP. There are 13 root servers. A root server is a server run by a designated organisation. The IP addresses of root servers are publicly known and rarely change, so that anyone can contact them. At the time of writing, these are assigned as follows.

a.root-servers.net	198.41.0.4	VeriSign, Inc.
b.root-servers.net	199.9.14.201	University of Southern California (ISI)
c.root-servers.net	192.33.4.12	Cogent Communications
d.root-servers.net	199.7.91.13	University of Maryland
e.root-servers.net	192.203.230.10	NASA (Ames Research Center)
f.root-servers.net	192.5.5.241	Internet Systems Consortium, Inc.
g.root-servers.net	192.112.36.4	US Department of Defense (NIC)
h.root-servers.net	198.97.190.53	US Army (Research Lab)
i.root-servers.net	192.36.148.17	Netnod
j.root-servers.net	192.58.128.30	VeriSign, Inc.
k.root-servers.net	193.0.14.129	RIPE NCC
l.root-servers.net	199.7.83.42	ICANN
m.root-servers.net	202.12.27.33	WIDE Project

If you own the domain `google.com` you can tell these servers what the IP address of `www.google.com` is. Then anyone else can go and ask them what the IP address of `www.google.com` is and for instance be told that it is `172.217.19.206`. Since there are few root servers and a huge amount of domains and queries, root servers do not store the IP address of all domains. Instead, a root server will return to the user the IP address of another name server which keeps track of all addresses that end in `.com`. You might in fact get a whole list of IP addresses of such machines as there might be several for fault tolerance and load balancing. Then you go ask a random of these name servers for the IP address of `www.google.com`. It might in turn give you a list of IP addresses of name servers that keeps track of all addresses that end in `.google.com`. Then you ask a random of these name servers for the IP address of `www.google.com` and might finally be told `172.217.19.206`.

3.5 Programming Distributed Systems

It is notoriously hard to program distributed systems. Here are some vague advises which will be illustrated further throughout the book and programming exercises.

3.5.1 Local atomicity

Make sure the procedures running on one machine are **atomic**, i.e., if the procedure looks at local state, then make sure that local state does not change while the procedure executes. As an example, consider the following code:

```
func getMoney(c Caller, amount int) {  
    if (amount <= account[c].amount) {  
        sendMoney(c, amount);  
        account[c].amount = account[c].amount - amount;  
    }  
}
```

```

package main

import (
    "bufio" ; "fmt" ; "net" ; "strings"
)

func handleConnection(conn net.Conn) {
    defer conn.Close()
    myEnd := conn.LocalAddr().String()
    otherEnd := conn.RemoteAddr().String()
    reader := bufio.NewReader(conn)
    for {
        msg, err := reader.ReadString('\n')
        if err != nil {
            fmt.Println("Ending session with " + otherEnd)
            return
        } else {
            fmt.Print("From " + otherEnd + " to " + myEnd + ": " + string(msg))
            titlemsg := strings.Title(msg)
            conn.Write([]byte(titlemsg))
        }
    }
}

func main() {
    ln, _ := net.Listen("tcp", ":18081")
    defer ln.Close()
    for {
        fmt.Println("Listening for connection...")
        conn, _ := ln.Accept()
        fmt.Println("Got a connection...")
        go handleConnection(conn)
    }
}

```

Figure 3.10: A multithreaded TCP Server. Try connecting multiple instances of the TCP client from Figure 3.7 to this server.

```

package main

import ( "fmt" ; "strconv" )

var sendernames = [5]string{"Alice", "Bob", "Chloe", "Dennis", "Elisa"}

var receivernames = [5]string{"Frederik", "Gary", "Hailey", "Isabel", "Jesper"}

func send(c chan string, myname string) {
    for i:=0; i<1000; i++ {
        // you send on a channel using <-
        c<- myname + "#" + strconv.Itoa(i)
    }
}

func receive(c chan string, myname string) {
    i:=0
    for {
        // you also receive from channel using <-
        msg := <-c
        fmt.Println(myname + "#" + strconv.Itoa(i) + " " + msg)
        i++
    }
}

func main() {
    c := make(chan string)
    for i:=0; i<5; i++ {
        go send(c, sendernames[i])
        go receive(c, receivernames[i])
    }
    // we do one blocking call to avoid that the program terminates
    receive(c, "Kacey")
}

```

Figure 3.11: A use of channels in Go. The purpose of the program is to illustrate the use of channels, the main tool for handling concurrency and parallelism in Go.

This code allows a party to withdraw money if it has enough on its account. The `sendMoney` procedure is distributed and synchronous, it calls the system at the bank and waits to see whether the transfer actually succeeded. The problem here is that if customer BadGuy has 100 on its account and calls `sendMoney(BadGuy, 100)` twice, then both of them will succeed. First they check that $100 \leq 100$, which is the case. Then they both send 100. Then they both subtract 100 from the account of BadBuy, bringing it to -100 . Even if BadGuy does not have full control of the scheduling of the two executions of `sendMoney` they are likely to be executed as above as `sendMoney` takes a long time to execute. The above is sometimes known as a **reentrancy problem** and sometimes as a **race condition**. They are worse in distributed systems as they could be actively exploited even if they are subtle. The way to avoid them is to keep things simple and make sure that each call to `getMoney` gets executed fully before the next one is executed. On the Ethereum blockchain a very simple reentrancy bug famously cost 55 million USD and lead to the entire blockchain breaking into two.

3.5.2 Roll forward, don't rollback

If something goes wrong in one part of a system as part of one job, it might be tempting to erase the partial effect of the job. Say you told machine C to do some part of the job and told B to do some other part. Now B crashed. There are now two options. Rollback: you ask C to drop the work it did and you retry the whole job. Roll forward: you find some other way to finish the work B should have done. It can be notoriously hard to do a rollback. Maybe machine C already did its job and then crashed. Maybe C delegated the work to D, E and F, and E finished and crashed and D is still running but unresponsive because the network is slow. Trying a rollback can be an escalating logistic nightmare. Try to avoid it.

3.5.3 Don't rely on others

Don't rely on others doing their job correctly or in time, and don't rely on acknowledgements from others. In life, and in distributed systems. A common control structure in single machine programming is the synchronous function call.

If you are used to program with synchronous function calls it is easy to drag the pattern into distributed programming and program as follows: Machine A sends some values to machine B, then B does some work and returns a value or acknowledgement to A, at which point A knows the job was done and can proceed with its code. There are many problems with this programming pattern. If the acknowledge from B to A is lost, A will wait forever. If it is very late, so will A be. This can create a rippling effect of exceptions, and coding correctly with exception can be notoriously hard. This is even more the case in distributed systems, as you do not have a global view of what went wrong.

The most robust programming style for distributed systems is asynchronous message passing. If you need machine B to know something or do something, then you send it a message M and move forward with your own stuff. Don't wait for an acknowledgement from B. Don't even have the protocol include such a thing. Instead, program your system such that if B is down, then the job gets done anyway. Maybe there is a backup server that can take B's job. Maybe you have a background thread which will monitor the system and see if it shows the progress B should make, and if not it will occasionally resend M.

The above is the ideal. Oftentimes you cannot program like that, but always try it as the first thing. In Chapter 10 we will see Bracha Broadcast which is an excellent example of this programming style.

3.5.4 Failure is normal

Program as if failure is the normal. Or better, ensure that failure is the normal by having some process in your production environment which occasionally crash some of your servers and sends crazy messages. This is an emerging trend of **chaos engineering**. It is important to stress that the mistakes here are not part of testing, it is deliberately injected into the actual production environment by some process sometimes called a **Chaos monkey**. If you have a separate evil minded department produce the Chaos monkey it will keep your programmers and designers on their toes. It is also discouraging for a potential attacker on the system. They cannot do anything to the system that it is not doing to itself on a daily bases.

3.5.5 Keep it simple and modular

This is the most important advice when programming distributed systems. If your program requires 200 lines of code, then try to break it up into two logic components of 100 lines of code, which can each easily be implemented in a simple way and which have clear and simple dependencies and interactions. In distributed systems it pays off to think a lot before you touch the keyboard and start programming. Find the right abstractions and modules which will make everything simple. Big monolithic bundles of code are ensured to fail in distributed systems. The book has several examples of such simple abstractions, like broadcast, Byzantine agreement, and replicated state machines. We will for instance see how the very complex problem of replicating state machines is a simple application of broadcast and Byzantine agreement.

3.6 Concurrency

Concurrency is already hard when programming a multi-threaded process on a single machine. The problem becomes much harder in a distributed system, where delayed messages and crashed machines introduce events you cannot observe or foresee. We assume that the reader has an understanding of basic concurrency primitives like Locks, MUTEX, and Semaphores. You will also find these in Go. However, Concurrency in Go is inspired by the theory of Communicating Sequential Processes (CSP)[10] and relies on threads synchronising via synchronising communication channels. This is a theory which handles concurrency by passing messages. Since distributed systems are all about message passing, this concurrency mechanism works well for structuring distributed systems, and we encourage the reader to try to program with it to the fullest extent possible. Synchronising channels can easily be used to emulate the well known MUTEX and Semaphores, but often a more elegant solution can be found. Try it out.

In Go calls on `Conn.Read` and `Conn.Write` are blocking. This holds for all programming languages, as these calls eventually are mapped to the equivalent calls in the OS, and these calls are blocking. For instance, this means that if you for want to receive data from two different TCP connections you cannot just make a loop where you alternate between reading from the two channels: if one of the channels run out of data, the call will block and prevent you from reading from the other channel. It *is* possible to set a timeout on a read and terminate the call after some time if no data is available. This is, however, an extremely cumbersome and error prone way to program. Instead, in programs where you have to handle several connections you want to use separate concurrent processes for doing this. In Go it is extremely easy to do concurrent programming, which is one of the reasons for choosing the language for our programming examples in this book. Consider the multithreaded TCP server in Fig. 3.10. In line 34 we added the `go` keyword in front of the call to `handleConnection`. This means that the call will be **non-blocking**. The current thread of execution will immediately terminate and therefore start the next iteration of the `for`-loop. At the same time a new thread of execution

is created which starts executing `handleConnection`. These two threads will then proceed to run concurrently.

To coordinate concurrent threads Go uses two abstractions: `MUTEX` and channels. We assume the reader is familiar with using `MUTEX` (Mutual Exclusion) to coordinate concurrent threads. In Go a `MUTEX` has the type `Mutex`. It has two operations `Mutex.Lock()` and `Mutex.Unlock()`. These are reader/writer mutual exclusion locks which can be held by a single writer but several readers. They cannot be held by a writer and a reader at the same time. Take a look at the `sync` package for more. The channel concept of Go is of more novelty. A channel has a type, which is the type of objects that can be sent over the channel. Any goroutine can send on a channel and any goroutine can receive from a channel. It is safe for many Go-routines to send and receive at the same time without using `MUTEX`s. Notice that sending is *blocking*! So when a thread sends on a channel it blocks until the message is read. Also, if a thread reads from a channel, it will block until there is a value sent on the channel. This provides a very versatile way to both communicate and coordinate between threads. See Fig. 3.11 for a toy example. Try running the code. You will see that you get a panic at the end of the program as no more Go-routines are sending on the channel but still there are channels reading. Since sending and receiving are blocking, this means that the reading threads are in a deadlock: there are no threads left in the system which can make progress. Go can detect this and then causes a panic.

Beware that the build-in `map` type of Go is not thread safe, so if you have several Go-routines use the same `map`, you must use a `Mutex`. The `map` can handle one writer or multiple readers so it is sufficient to use an `RWMutex`. See Fig. 3.12 for an example of how to secure a `map` using an `RWMutex`.

In Fig. 3.12 the `map` has been correctly secured against multiple Go-routines, but we managed to make a bug in some other position. The intent of the program was that the printed string would contain 10 occurrences of 1, 2, and 3. Yet, when one runs the program another result is printed. Even worse, the result differ from run to run. Here are some examples:

```
> go run concurrentmap.go
331111111111
> go run concurrentmap.go
33333333332222222221111111111
> go run concurrentmap.go
33333333332222211111111111
> go run concurrentmap.go
33222222222
> go run concurrentmap.go
33332222222222
> go run concurrentmap.go
331111111111
> go run concurrentmap.go
3333333333222222221111111111
```

Before reading the solution below, take a moment to see if you can realise what we did wrong? While you think, I will get coffee.—I assume you spotted it by now. The code

```
dns.Set("X", dns.Get("X") + suf)
```

can be expanded to

```
tmp := dns.Get("X")
```

```

package main

import ( "fmt" ; "sync" )

type DNS struct {
    m map[string]string
    lock sync.RWMutex
}

func (dns *DNS) Set(key string, val string) {
    dns.lock.Lock()
    defer dns.lock.Unlock()
    dns.m[key] = val
}

func (dns *DNS) Get(key string) string {
    dns.lock.RLock()
    defer dns.lock.RUnlock()
    return dns.m[key]
}

func MakeDNS() *DNS {
    dns := new(DNS)
    dns.m = make(map[string]string)
    return dns
}

func GetAndSet(suf string) {
    for i:=0; i<10; i++ { dns.Set("X", dns.Get("X") + suf) }
    c<-0
}

var c = make(chan int)
var dns = MakeDNS()

func main() {
    go GetAndSet("1") ; go GetAndSet("2") ; go GetAndSet("3")
    <-c ; <-c ; <-c // wait for the three goroutines to end
    fmt.Println(dns.Get("X"))
}

```

Figure 3.12: A Go program with a race condition. See the caption of Figure 3.6 for an explanation on the use of the `defer` keyword.

```

package main

import ( "fmt" ; "sync" )

type DNS struct {
    m map[string]string
    lock sync.Mutex
}

func MakeDNS() *DNS {
    dns := new(DNS)
    dns.m = make(map[string]string)
    return dns
}

func (dns *DNS) GetAndSetOnce(suf string) {
    dns.lock.Lock()
    defer dns.lock.Unlock()
    dns.m["X"]=dns.m["X"]+suf
}

func (dns *DNS) GetAndSet(suf string) {
    for i:=0; i<10; i++ { dns.GetAndSetOnce(suf) }
    c<-0
}

var c = make(chan int)

func main() {
    dns := MakeDNS()
    go dns.GetAndSet("1") ; go dns.GetAndSet("2") ; go dns.GetAndSet("3")
    <-c ; <-c ; <-c // wait for the three goroutines to end
    fmt.Println(dns.m["X"])
}

```

Figure 3.13: A Go program without a race condition

```
tmp = tmp + suf
dns.Set("X", tmp)
```

at which point it is easier to see that between the call to `Get` and the call to `Set` some time will elapse. During that time the lock is released, so some other goroutine might read the same current value of "X". Then the last of the these two Go-routines to call `Set` will overwrite to append done by the first one to call `Set`. What we really need to do is make the whole operation of updating atomic: no other processes should read or write while we do the update. In Fig. 3.13 is an example of how this could be done. Here some runs of the code:

```
> go run concurrentmap2.go
3333333311111111133222222222
> go run concurrentmap2.go
3111111111333333332222222222
> go run concurrentmap2.go
3111111111333333332222222222
> go run concurrentmap2.go
3333333333111111111222222222
> go run concurrentmap2.go
3111111111333333332222222222
```

Note that we do not use the synchronized calls to `Get` and `Set`. The reason is that in Go a `Mutex` does not belong to a particular goroutine, so you can deadlock yourself if you try to lock the `Mutex` while you have the `Lock`. You are deadlocking because you are waiting for yourself to release the lock! This is different from some other programming languages and therefore something to watch out for if you are already familiar with concurrent programming in other languages. One thing to take away from the example is that it was not really the access to setting and getting on the map that needed to be thread safe. Concurrent programming is much harder than that: what you need is atomicity of the critical access to your data. This is also the reason why `map` is not already synchronised in Go: it is often not needed nor is it enough, so would it be to sit between two chairs!

Exercise 3.2 (putting the pieces together)

1. Write a server and client in Go which do the following.
 - When the client is started it prompts the user for an IP address and port number. Then it makes a TCP connection to the corresponding IP address and port. Then it will prompt the user for input until `quit` is typed. The input is sent to the server. Concurrently it reads lines of text coming from the server and prints them. Notice that several Go-routines may access `Conn.Read` and `Conn.Write` concurrently.
 - When the server is started it determines its local IP address and prints it. Then it start listening for connections on a random open port and prints the port number. It should be able to handle several concurrent connections. When the server receives a string on any incoming connections, it will prepend the IP address and port number of the sender to the string and send the string back to all its current connections.

To get used to concurrency and channels in Go, use the following design for the server: each new connection is handled by a goroutine which reads the incoming lines of text. There is a single channel `outbound` of `string`, where the reading Go-routines send the strings they received after modifying them as specified. There is a single goroutine `broadcast` which is the single consumer of messages from `outbound`. It reads them iteratively and then sends them back to the clients.

```

package main

import ( "net" ; "fmt" ; "encoding/gob" ; "io" ; "log" )

type ToSend struct {
    Msg string // only exported variables are sent, so start the ...
    Number int // ... name of the fields you want send by a capital letter
}

func handleConnection(conn net.Conn) {
    defer conn.Close()
    msg := &ToSend{}
    dec := gob.NewDecoder(conn)
    for {
        err := dec.Decode(msg)
        if (err == io.EOF) {
            fmt.Println("Connection closed by " + conn.RemoteAddr().String())
            return
        }
        if (err != nil) {
            log.Println(err.Error())
            return
        }
        fmt.Println("From " + conn.RemoteAddr().String() + ":\n", msg)
    }
}

func main() {
    fmt.Println("Listening for connection...")
    ln, _ := net.Listen("tcp", ":18081")
    defer ln.Close()
    conn, _ := ln.Accept()
    fmt.Println("Got a connection from ", conn.RemoteAddr().String())
    handleConnection(conn)
}

```

Figure 3.14: A TCP server using Gob.

2. Make a version of the above where the clients send messages automatically in a way similar to Fig. 3.11. Drop the upper bound in the loop such that the clients keep sending packages forever. With one server and ten clients, what kind of throughput do you get when running on one machine. For instance, how many packages can be sent to all clients in two minutes. If you have access to several physical machines try putting the clients on different machines. What kind of throughput do you get now?



3.7 Marshalling

An important topic in communication in distributed systems is so-called **marshalling**. TCP allows to send a stream of octets a.k.a. bytes. Often we want to send more structured data, maybe even an object from an object oriented language like Go. This requires packing down the object as a stream of bytes and unpacking it at the other end. This is sometimes called **serialization**. This is extremely cumbersome and error prone to do “by hand”. Fortunately modern programming languages offer mechanisms for doing it for you. To appreciate the trouble

```

package main

import ( "bufio" ; "fmt" ; "net" ; "os" ; "encoding/gob" )

type ToSend struct {
    Msg string // only exported variables are sent, so start the ...
    Number int // ... name of the fields you want sent by a capital letter
}

func main() {
    ts := &ToSend{}
    conn, _ := net.Dial("tcp", "127.0.0.1:18081")
    defer conn.Close()
    enc := gob.NewEncoder(conn)
    for i:=0;; i++ {
        fmt.Print("> ")
        reader := bufio.NewReader(os.Stdin)
        m, err := reader.ReadString('\n')
        if err!=nil || m == "quit\n" { return }
        ts.Msg = m
        ts.Number = i
        enc.Encode(ts)
    }
}

```

Figure 3.15: A TCP client using Gob. Try removing some fields from the struct in the server, or adding some fields in the client, to appreciate the service Gob provides.

this save you, consider the so-called Big Endian-Little Endian Problem. When for instance a 32-bit integer is stored in memory, it is stored as four bytes. On some machines the most significant byte is stored first. On other machines the most significant byte is stored last. So when an integer is sent as a stream of bytes over TCP, the sender and receiver need to explicitly agree on the order. The typical way to do it is using a *network* format. It is for instance once and for all agreed that the least significant byte is sent first. One advantage of this is that one can do serialization without knowing what kind of machine the data is sent to. This is a huge advantage as this might not be known until later and maybe the data is being sent to different types of machines.

Go has a serializability system called Gob. It is very simple yet very versatile. If you used a serialization system before, Gob will be familiar to you. In Go there is a notion of a field of a struct being *exported* or not. If the name of a field starts with a capital letter, it is exported. Otherwise it is not. Whether a field is exported or not impacts how it is visible in the package system. With respect to marshalling, one thing to keep in mind with Gob is that only exported fields are sent, so be sure to start those names with a capital if you want them sent on the network. A particularity of Gob is that it is very lenient when decoding. If you send an `int8` it will allow you to read it back into a `int16`. Also, if you send a `struct` with three fields, you can read it back into a struct with just two fields: names and types will be used to determine which fields to read. The last field is just ditched. This will allow to read old serialized data later even if a protocol had its data updated to have new fields. It will also allow old and new versions of protocols to run at the same time even if packets have new optional fields added.

```

package main

import ( "fmt" ; "log" ; "net" ; "net/rpc" ; "os" ; "bufio" ; "time")

type PrintAndCount struct {
    x int
}

func (l *PrintAndCount) GetLine(line []byte, cnt *int) error {
    HardTask()
    l.x++
    fmt.Println(string(line))
    *cnt = l.x
    return nil
}

func HardTask() {
    time.Sleep(5 * time.Second)
}

func MakeTCPListener() *net.TCPListener {
    addy, err := net.ResolveTCPAddr("tcp", "0.0.0.0:42587")
    if err != nil { log.Fatal(err) }

    inbound, err := net.ListenTCP("tcp", addy)
    if err != nil { log.Fatal(err) }
    return inbound
}

func main() {
    // Register how we receive incoming connections
    go rpc.Accept(MakeTCPListener())

    // Register a PrintAndCount object
    rpc.Register(new(PrintAndCount))

    // Avoid terminating
    fmt.Println("Press any key to terminate server")
    bufio.NewReader(os.Stdin).ReadLine()
}

```

Figure 3.16: An RPC server

```

package main

import ( "log" ; "net/rpc" ; "bufio" ; "os" ; "fmt" )

func main() {
    client, err := rpc.Dial("tcp", "localhost:42587")
    if err != nil { log.Fatal(err) }

    in := bufio.NewReader(os.Stdin)
    for {
        line, _, err := in.ReadLine()
        var reply int

        // Synchronous call
        err = client.Call("PrintAndCount.GetLine", line, &reply)

        if err != nil { log.Fatal(err) }
        fmt.Println("Strings printed at server: ", reply)
    }
}

```

Figure 3.17: A synchronous RPC client

```

package main

import ( "log" ; "net/rpc" ; "bufio" ; "os" ; "fmt" )

func PrintWhenReady(call *rpc.Call) {
    <-call.Done // this channel unblocks when the call returns
    if call.Error != nil { log.Fatal(call.Error) }
    fmt.Println("Strings printed at server: ", reply)
}

var reply int

func main() {
    client, err := rpc.Dial("tcp", "localhost:42587")
    if err != nil { log.Fatal(err) }

    in := bufio.NewReader(os.Stdin)
    for {
        line, _, _ := in.ReadLine()

        // Asynchronous call
        call := client.Go("PrintAndCount.GetLine", line, &reply, nil)
        go PrintWhenReady(call) // Handles the reply when ready

        fmt.Println("See, I can still do stuff!")
        fmt.Println("See, I can still do stuff!")
        fmt.Println("OK, I'm bored...")
    }
}

```

Figure 3.18: An asynchronous RPC client

3.8 Remote Procedure Calls

With serialization in place we can conveniently move objects over the network. The next step is to add some session control. A typical control pattern in programming is the function call: some values are sent to a sub-routine and some return values are returned. This pattern also occurs naturally in distributed programming: some values are sent to a server (using serialization) and later some return values are sent back (using serialization). Remote Procedure Calls (RPC) set up standard routines and language constructions for doing this. It works as follows:

1. The parameters are sent over the network to the remote machine using serialization.
2. The function is run on the remote machine and some return value is produced.
3. The return values are sent over the network to the caller using serialization.

In object oriented programming languages, RPC is also sometimes known as Remote Method Invocation (RMI). The only difference is that when the function is run on the remote machine, the function may be a method of an object, and that method may in turn modify the state of that object.

In Fig. 3.16 and Fig. 3.17 you can find an example of how to use RPC in Go. Try to run it to make sure it works on your machine. Try to connect to the same server with several clients and see what happens. Notice that the code is not thread safe, as we have used no concurrency control whatsoever. Some things to notice:

- You make an object available on the network by registering it as shown. You can only register one object of each given type as it is accessed via the type name.
- For a method of a registered object to be visible to the clients they need to take two arguments and have return type `error`. The second argument must be a pointer. The first argument is the input. The second input is used to hold the output.
- Both arguments must be exported (or builtin) types.

The client in Fig. 3.17 makes a synchronous call: it blocks until the reply returns. It is also possible to make an asynchronous call where the RPC does not block. In that case you set up a goroutine that waits for the reply. See Fig. 3.18 for an example.

3.9 Logically Organising Connections

With the Internet in place we are now in a situation where all machines in principle can make connections to all other machines. However, maintaining a connection consumes resources in the OS so it is not viable to keep connections open to a large number of other machines. In this section we take a brief look at the ways connections and machines can be organized.

3.9.1 Fully Connected

The simplest solution in a distributed system is to let everyone in the system make a connection to everyone else in the system. This is often done in small systems where for instance a server is replicated. However, with n servers, the number of connections is in the order of n^2 . In practice this does not scale well for larger system. However in some cases it might be necessary: for instance, in systems where you want to tolerate Byzantine errors, it might not be an option to send communication via other nodes (which might be corrupted). Therefore a fully connected network is often used for small fault tolerant systems. We will in later sections study how to achieve Byzantine agreement in such systems.

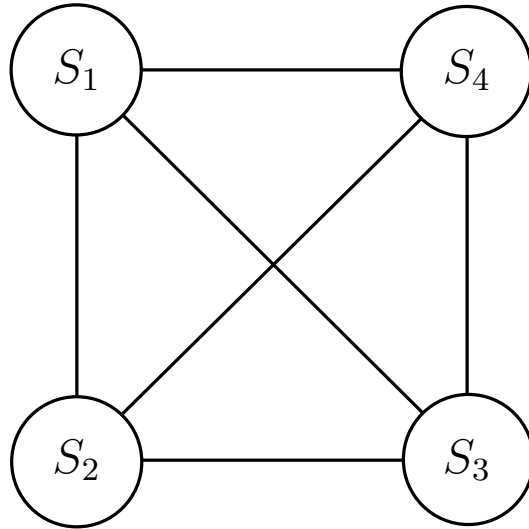


Figure 3.19: A fully connected system with four servers.

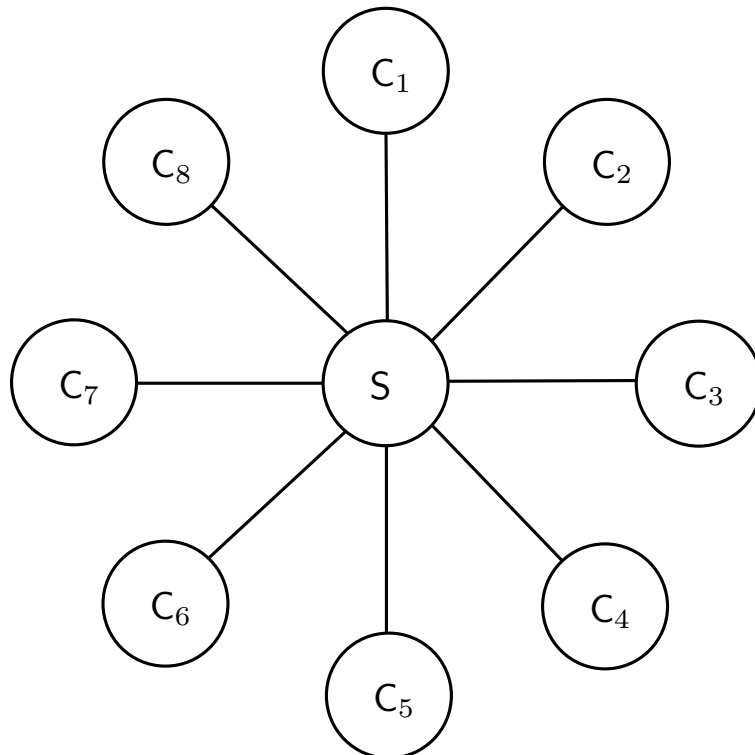


Figure 3.20: A client-server system.

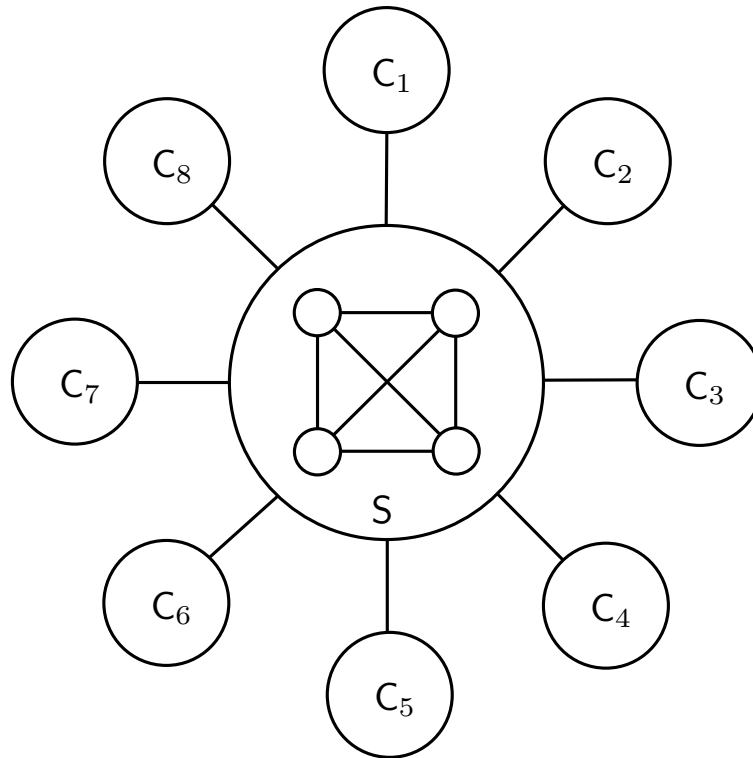


Figure 3.21: A Replicated Server

3.9.2 Client-Server

Probably the most widespread system architecture is the client-server model. There is a single server to which all clients connect. All information exchanged between clients is exchanged via the server. Platforms like Dropbox, Amazon, most gaming platforms, and whatnot are organised this way. An obvious problem with the plain client-server architecture is that it has a single point of failure/attack/trust. If the server crashes/is attacked/is corrupted the security of the system breaks down.

3.9.3 Client-Replicated Server

The obvious way to deal with the single point of attack of the client-server model is to replicate the server. This also creates the possibility to take some load off the server if it serves a huge number of clients. This is the most common architecture on the Internet. It opens up a lot of problems with keeping the copies of the replicated server consistent. We will spend a lot of time later understanding how to solve these problems.

3.9.4 Edge Nodes

When having a replicated server, it is common to put it on a protected network and let all accesses be mediated by so-called **edge nodes**. Clients contact the edge nodes, which will then distribute the workload over the replicates. When we later talk about Firewalls we will return to how this model can increase security by protecting the servers against direct contact with evildoers.

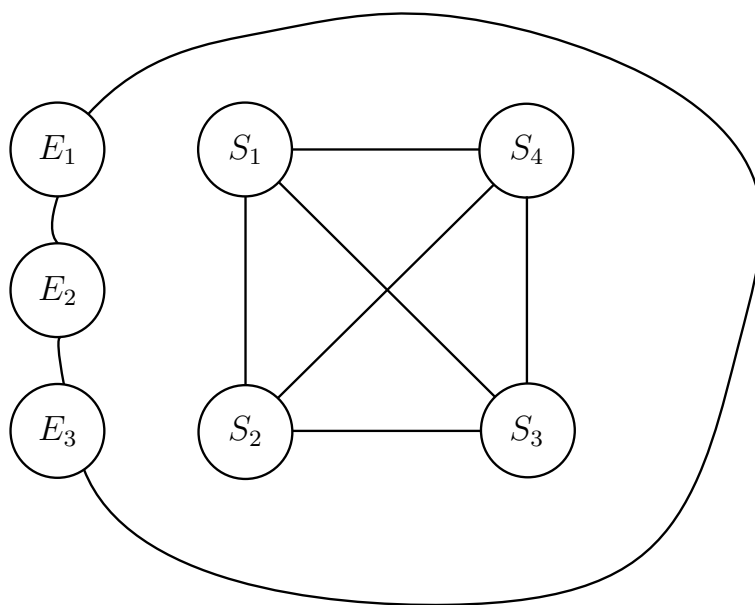


Figure 3.22: Replicated Server with Edge Nodes

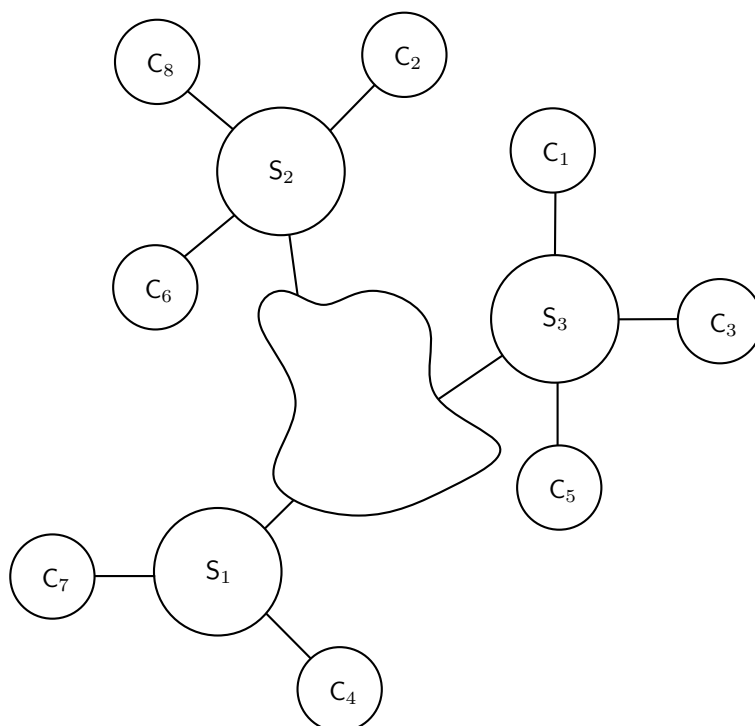


Figure 3.23: Illustration of organization of SMTP

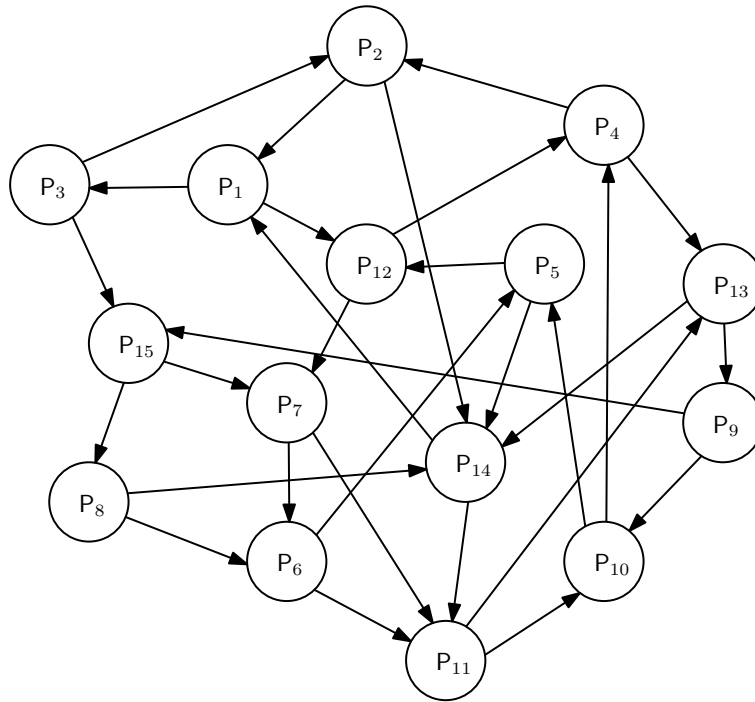


Figure 3.24: Random Peer-to-Peer

3.9.5 SMTP

As our last example of how to organize communication in a structured way, we consider the Simple Mail Transfer Protocol (SMTP). Each domain, like `au.dk` has its own mail server. The IP address of the mail server is stored in the DNS system. Here is an example of how to look up the IP address of the mail server of Aarhus University using the `dig` command on Linux. First we look up the DNS name of the server:

```
> dig au.dk MX
;; ANSWER SECTION:
au.dk. 26992 IN MX 10 au-dk.mail.protection.outlook.com.
```

The `MX` indicates that we look for the main server. We see that the name is `au-dk.mail.protection.outlook.com`. We then use an `A` to look up the IP address of that DNS name:

```
> dig au-dk.mail.protection.outlook.com A
;; ANSWER SECTION:
au-dk.mail.protection.outlook.com. 15 IN A 213.199.154.234
au-dk.mail.protection.outlook.com. 15 IN A 213.199.154.170
```

There are two addresses returned indicating that the server is indeed replicated. They might also be edge nodes with more machines behind them.

If you are at Aarhus university and send an e-mail, your mail program might send it to one of these servers that receive it for you initially. These local mail servers can then forward the e-mail to the mail server of the receiving institution. This might take several attempts if these servers are down or slow. Your mail client program is not involved in these attempts at re-sending the message, which are solely handled by the mail server. The system architecture is illustrated in Fig. 3.23.

3.10 Unstructured Peer-to-Peer and Flooding

A common theme in large distributed systems is to use **peer-to-peer networks**. All parties will not keep a connection to all other parties: it is expensive to keep a large number of open connections, and sometimes parties are not even aware of all other parties in the network. There are two main kinds of peer-to-peer networks, namely **structured peer-to-peer networks** and **unstructured peer-to-peer networks**. In structured peer-to-peer networks, the connections between nodes are established in a deterministic way. In unstructured overlay networks the connections are formed at random. In Fig. 3.24 a random peer-to-peer network is shown, where each party has chosen two other parties to connect to at random.

A peer-to-peer graph can be used to disseminate messages. If you want to send a message, simply send it to the peers you have chosen. If a node receives a message it *has not see before* then it recursively disseminate it the same way. This is called **flooding**. Consider Fig. 3.24 and consider a situation where P_3 sends a message. Assuming that each message transfer takes exactly the same time, then the message would propagate in the following way:

$$\begin{aligned}
 &\{P_3\} \\
 &\{P_3, P_2, P_{15}\} \\
 &\{P_3, P_2, P_{15}, P_1, P_{14}, P_8, P_7\} \\
 &\{P_3, P_2, P_{15}, P_1, P_{14}, P_8, P_7, P_{12}, P_{11}, P_6\} \\
 &\{P_3, P_2, P_{15}, P_1, P_{14}, P_8, P_7, P_{12}, P_{11}, P_6, P_4, P_{13}, P_{10}, P_5\} \\
 &\{P_3, P_2, P_{15}, P_1, P_{14}, P_8, P_7, P_{12}, P_{11}, P_6, P_4, P_{13}, P_{10}, P_5, P_9\}
 \end{aligned}$$

Two important measures of how good a peer-to-peer graph is are:

Diameter For each pair (P_i, P_j) of parties, define their distance to be the shortest path from P_i to P_j . If they are not connected, then $\text{Distance}(P_i, P_j) = +\infty$. Then the diameter of the graph is

$$\text{Diameter} = \max_{P_i, P_j} \text{Distance}(P_i, P_j) .$$

To compute the distance from two parties, one can flood the network starting from P_i and see how long it takes to reach P_j , as we did above for P_3 . From the above example we can see that the distance from P_3 to P_9 is 5, so the diameter of the above graph is at least 5.

Connectivity If the diameter of a graph is $+\infty$, then we say that the network is unconnected. When it is unconnected, there are two parties between which there is no path. The peer-to-peer network then does not allow these parties to communicate. The connectivity of a graph is defined to be the minimal number of edges that must be removed to make the graph unconnected.

The reason why we like to use random peer-to-peer graphs is that they are easy to form and maintain and at the same time they have low diameter and high connectivity on average.

Exercise 3.3 (diameter, connectivity) Compute the diameter and connectivity of the graph in Fig. 3.24. ▼

We now briefly investigate the connectivity of a random graph. In the graph from Fig. 3.24, each node is connected to two other nodes. We then say that the **out-degree** is 2. The following theorem shows that if you pick a random graph then the probability that it is not connected goes down exponentially with the out degree. If for instance the out degree is 20 the probability that it is not connected is less than 1 in a million.

Theorem 3.1 Assume that $n \geq 25$ and $\kappa \geq 2$. If you pick a random peer-to-peer graph on n parties with out-degree κ , then the probability that it is not connected is at most $2^{-\kappa}$.

Let P be the set of all n parties. We call a subset $S \subset P$ non-trivial if $S \neq \emptyset$ and $S \neq P$. We call a non-trivial subset $S \subset P$ a cut if there is no path from a party in S to a party in $P \setminus S$. Notice that a graph is unconnected if and only if it has a cut.

We now fix a non-trivial subset S and compute the probability that S becomes a cut if a random peer-to-peer network of out-degree κ is chosen. Let $s = |S|$. Each of these s parties will pick κ random peers to connect to. For S to be a cut, all these $s\kappa$ connections need to end up in S . If $s \leq \kappa$, then the probability of this is 0, as each of the parties in S will pick at least one peer from outside S . So assume for the rest of the analysis that $s > \kappa$. The probability that all peers are picked inside S is less than

$$\left(\frac{s}{n}\right)^{s\kappa}.$$

This number is the number we get if a peer picks the connections with possible repetition and might pick itself. It will not pick itself and will not pick the same connection twice. But this just makes the probability of getting a connection outside S higher.

There are at most 2^n different subset S to consider, as there are at most 2^n subsets of P . So by a union bound, the probability there is a cut is at most

$$2^n \max_{s > \kappa} \left(\frac{s}{n}\right)^{s\kappa}.$$

We have that

$$2^n \max_{s > \kappa} \left(\frac{s}{n}\right)^{s\kappa} = \max_{s > \kappa} \left(2^{n/\kappa} \left(\frac{s}{n}\right)^s\right)^\kappa,$$

so it is sufficient to prove that

$$\max_{s > \kappa} 2^{n/\kappa} \left(\frac{s}{n}\right)^s \leq 1/2.$$

This is the same as proving that

$$n/\kappa + \max_{s > \kappa} \log_2 \left(\left(\frac{s}{n}\right)^s\right) \leq -1,$$

or

$$n/\kappa \leq \max_{s > \kappa} s \log_2 \left(\frac{n}{s}\right) - 1.$$

The expression $s \log_2 \left(\frac{n}{s}\right)$ is maximal at $s = n/e$, where it is

$$(n/e) \log_2 e > 0.54n.$$

So, we need that

$$n/\kappa \leq 0.54n - 1.$$

If $\kappa \geq 2$, then this holds for all $n \geq 25$.

3.10.1 How to Build and Maintain

To build a random peer-to-peer network, each node could hold a list of all other nodes that are present in the network (we call this its network list), and then pick its connections at random from this list. When a new node enters the network, it will flood its presence, for instance its IP address, and all other nodes will add the new node to their network list. If a node becomes unresponsive, the other nodes will remove it from their network list. To enter the network initially, a node needs to know at least one other node of the network and ask it for its network

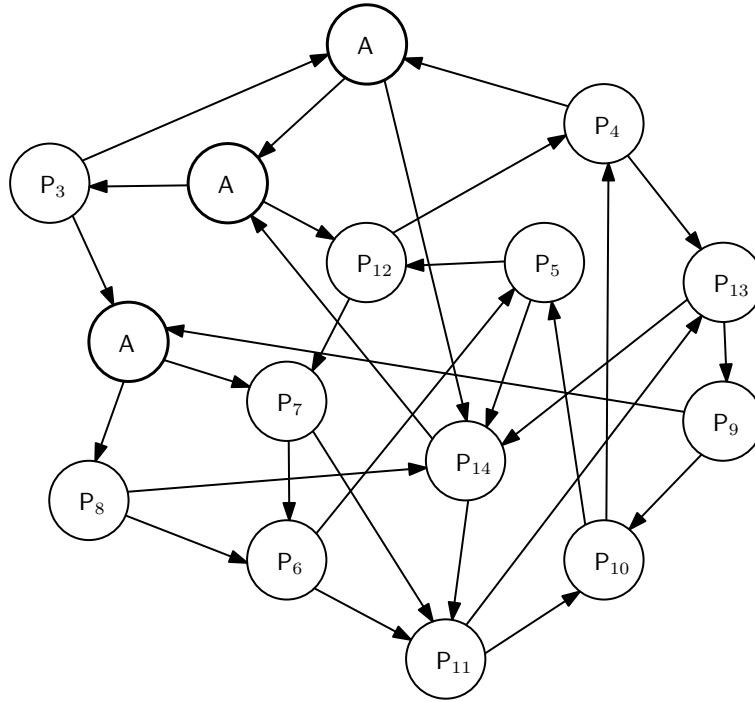


Figure 3.25: An Eclipsing Attack

list. In practice it is not possible to have a list of every node on the network, so there are other methods to build a random looking network. Each node only holds a partial view. When entering the network you get the partial view of your entry point. Then you can contact some of the known peers to get their partial view and that way learn about more peers, and yourself build a random looking partial view of the system.

3.10.2 Gossiping

Sending a large message M in a flooding network can be expensive. Each node sends and receives it many times. Here is a variant sometimes known as **gossiping**. It breaks the distribution of M up into a push and pull phase. You send M to your neighbours in sequence with a small break in between. Each time you send it the neighbour reports back if it already had M . After some number, say 10, neighbours reported that they already had the message, you take this as a signal that most nodes have the message now and stop sending it. This was the push phase. This might by chance result in some nodes not getting M . Therefore each node will occasionally pull its neighbours to check if they have new messages.

Another way to reduce the overhead of flooding is to flood only a short notification that there is some large update. Then each client will pull its neighbours for it. It will stop pulling after getting M once. Now it can reply to pull requests from its own neighbours. This can be much slower, but it ensures that each M is received only once per client.

3.10.3 Eclipsing Attacks

A dangerous type of Byzantine attack on peer-to-peer networks is called **eclipsing attacks**. Assume that the adversary controls some of the nodes $A \subset P$ of the network. If removing the connections of all these nodes creates a cut with no connections in either direction, we say that the adversary has successfully mounted an eclipsing attack, since the adversary has managed

to divide the network into two groups. All communication between the groups goes via the adversary, who can therefore control all information. This type of attack is easier to perform on a single party. For an example, consider Fig. 3.25, where P_3 has been eclipsed by an adversary A controlling only three nodes.

A particularly easy way to mount an eclipsing attack is for an adversary to create many copies of itself. It might only have a single machine, but might create millions of IP addresses and add them to the peer-to-peer system. This is called a **Sybil attack**. If the system only has 10,000 honest nodes but 1000,000 IP addresses pointing to the adversary, only one in a hundred of the IP addresses in a random selection would be to honest parties. So even a node picking a hundred random peers would with good probability be eclipsed. To avoid Sybil attacks real world peer-to-peer networks apply a number of heuristics when selecting the connections. One for instance does not pick multiple IP addresses from the same sub-domain / region.

It is particularly easy to be a victim of Eclipsing attacks when first entering a network. One needs to be sure to ask at least one honest node about its network list.

Exercise 3.4 (Warm-up: TCP Connections and Message Passing in Go) *This exercise is part of a series of practical exercises (Exercise 3.4, Exercise 4.6, Exercise 6.16, Exercise 16.2) in which you gradually build a peer-to-peer transaction ledger (sometimes referred to as a “Blockchain”). It may be useful to skim the later exercises in the series before starting, to understand which components will be extended and reused.*

Goal. Implement a Go type `Peer` that can (1) listen for TCP connections, (2) connect to another peer, and (3) send and receive typed messages. In earlier courses you have likely implemented *client-server* systems, where a server listens for connections and clients only initiate connections. In a *peer-to-peer* system there is no such fixed distinction. A `peer` is a program that is both a server and a client: it listens for incoming TCP connections *and* it actively connects to other peers. All peers run (essentially) the same code and play the same role in the system. This exercise is meant to help you shift from a client-server mindset to a peer-to-peer mindset before adding higher-level distributed-system logic.

Requirements. Your implementation should include:

1. **Listening.** A peer can start a TCP listener on a port and accept incoming connections.
2. **Connecting.** A peer can connect to another peer given an IP address (or hostname) and a port.
3. **Message format and framing.** TCP is a byte stream, so you must define how messages are encoded and how the receiver knows where one message ends and the next begins. We recommend encoding messages as JSON and prefixing each message with its length. This keeps the code simple and easy to debug. You are free to use a different encoding (for instance `encoding/gob`) if you prefer, but you must clearly document your choice. Your message type should include at least the following fields that will be serialized and sent over the network
 - a message type field, e.g., `Type string` (such as "Ping" and "Pong"),
 - a message identifier, e.g., `MsgID string` (this will be reused later for deduplication),
 - a sender identifier, e.g., `From string`,
 - a payload field, e.g., `Payload []byte` or `string`.
4. **Concurrency.** Each `Peer` must be able to receive messages while also sending messages. We suggest to use goroutines and channels, or another concurrency pattern of your choice.

Suggested API (you can change it). You may follow these function names, but you are allowed to pick your own as long as the functionality is present and clearly documented.

- `NewPeer(listenPort int) *Peer`
- `(p *Peer) Start() error` starts listening and accepting connections
- `(p *Peer) Connect(addr string, port int) error` creates an outgoing TCP connection
- `(p *Peer) Send(to string, msg *Message) error` sends a message to a connected peer
- `(p *Peer) OnMessage(from string, msg *Message)` called when a message is delivered locally

Here `to` and `from` can be any connection identifier you choose.

What your demo must do. Your hand-in must include a program `handin0.go` that:

1. Starts $n = 3$ peers on the same machine (use three different ports).
2. Connects the peers such that *each peer is connected to all others* (i.e., after setup, every peer has an open TCP connection to the two other peers).
3. Demonstrates basic message exchange using a **Ping/Pong** interaction: one peer sends a **Ping** message to another peer, and the receiving peer responds with a corresponding **Pong** message. The **Pong** should include the same `MsgID` as the **Ping**, so it is clear which request it answers.
4. Prints enough logs to show which peer sent and received each **Ping** and **Pong**, including peer identifiers and message identifiers.
5. Prints only relevant information, such that a TA can observe the required behavior in real time without excessive or missing output.

Testing. You must include at least one automatic test. You are strongly encouraged to add more tests, since they will be reused and expanded in later hand-ins. For instance:

- start two peers on free ports,
- connect one to the other,
- send k messages,
- check that all messages are received and parsed correctly.

Describe briefly in the report what you test and how to run the test.

Report. Your report should briefly describe:

1. how you implemented message framing and serialization,
2. “how your TA can run `handin0.go` (include exact commands and any required directory structure),
3. how to run your tests.

Note on reuse. In Exercise 4.6 you will extend the same **Peer** with: a peer set, flooding, and a ledger application. Design your code so that adding new message types later is straightforward. ▼

3.11 Structured Peer-to-Peer

There is also a notion of structured peer-to-peer networks. A popular structured peer-to-peer network is the Chord network. Here each node has an identity, for instance its IP address. This identity is mapped deterministically into an integer between 0 and $B - 1$ for some bound B . It is not important what B is exactly, as long as it is so large that the chance that two different nodes are mapped onto the same integer is small. We can imagine all the numbers from 0 to $B - 1$ being wrapped around a circle with 0 at the top and $B - 1$ being the last point before we hit 0 again. If we kept mapping the integers like this B would hit the same point as 0, $B + 1$ would hit the same point as 1 and so on.

Now when a number of nodes with identities $A, B, C, \dots$ enter the network, they will each map their identity into a number. We then imagine them sitting at the corresponding point on the circle. Each node will then make a connection to the node that is 1 step away on the circle, make a connection to the node that is 2 steps away, and the node 4 steps away, 8 steps away, 16, 32, and so on. If there is no node at those exact points they make a connection to the next node on the circle. So in Fig. 3.26, node A would make a connection to M, I, H , and N . And node N would have connections to F, E, K, G and M . To send a message to a recipient, a node will send it to the closest known peer before the recipient. That peer will then send it along until it reaches the recipient. If for instance A wants to send a message m to G , then it sends it to N and then N sends it to G . The particular way the connections have been chosen ensures that the diameter of the graph is only $\log_2(B)$, so it does not take too long to send a message.

Having a structured network also allows to use it as a shared variable space. Say each position i in the ring corresponds to a variable V_i . The node N_i closest to position i can be made responsible of keeping V_i . To read V_i pull it from N_i . To write it, push the update to N_i . To achieve fault tolerance, store the variable on several servers.

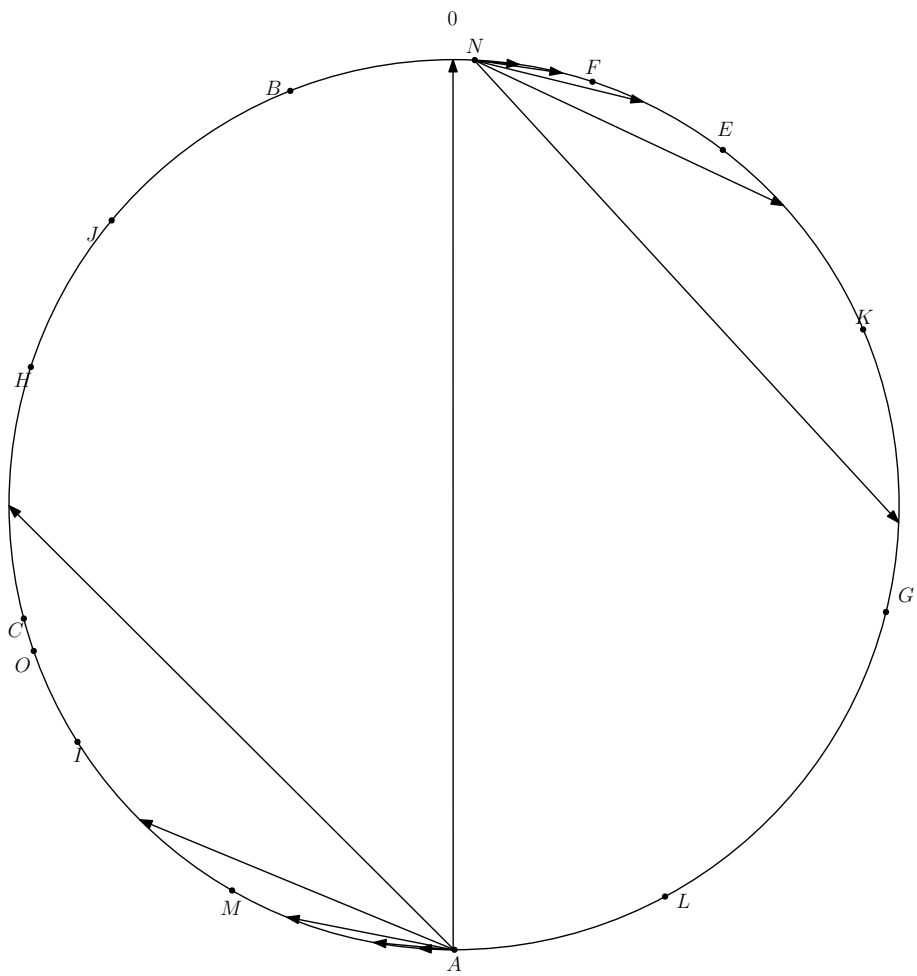


Figure 3.26: Chord Illustration

Chapter 4

Consistent Communication (2026)

Reading Guide *This section introduces the basic idea of ensuring consistency in the communication in distributed systems. This topic is about controlling in which order the same messages can arrive at different parties. We look at FIFO, causal, and total order. FIFO intuitively means that messages sent by the same party is delivered in the order they were sent. Causal order means that if two messages, possibly sent by different parties, are related in the sense that one message could have affected the other, then the affected message is delivered last at all receivers. Messages that can not have affected each other might be delivered in different order at different parties. Total order means that all messages are delivered in the same order at all parties. We will also look at how to take consistent snapshots of distributed systems to demonstrate how to work with causality. All of these consistency properties are important building blocks in many distributed systems.*

In this section we try to use the formal model for specifying how a system is supposed to behave, by giving interactive agents which guarantee exactly the desired ordering. This chapter is good for illustrating this use, as all three consistency properties, FIFO, causal and total order are easy to grasp intuitively. So the reader can compare the intuitive understanding to the formal specification and contemplate how they correspond. Read the use of the formal specifications as examples of uses of the model. There is no need to get too caught up in the formal details of the modelling if it hinders understanding for the reader. It is more important to understand intuitively what FIFO, causal and total order are. In Section 4.9 we give an example of how one formally proves that a protocol implements a specification. This is not curriculum, it is meant as an example for the formally inclined reader interested in how to use the formal model. The rest of the chapter is curriculum and might be queried at the oral exam.

Contents

4.1	Flooding Network	88
4.2	First In, First Out	91
4.3	Intermezzo on Modelling Distributed Systems I†	93
4.4	Causality	95
4.4.1	Causal Past, Present, Future	97
4.4.2	An Implementation	99
4.4.3	The Causal-Past Relation	102
4.4.4	The Causal Communication ideal functionality, Causal	106
4.4.5	Proving Security	107
4.4.6	Turning Sets into Vector Clocks	107
4.4.7	Vector Clocks as Operationalized Causality	109

4.5	Intermezzo on Modelling Distributed Systems II†	111
4.6	Total Order	112
4.6.1	An Implementation Assuming we can Detect Safe Messages	112
4.6.2	Detecting Safety	117
4.7	Chandy–Lamport Snapshot Algorithm	117
4.8	More Exercises	125
4.9	A Formal Simulation Argument†	133

Attack Model *In this chapter we assume that*

1. *All parties are correct.*
2. *The adversary cannot drop or inject messages.*
3. *The adversary is allowed to delay and reorder messages.*
4. *The parties do not have access to clocks.*

4.1 Flooding Network

In many types of networks, for instance gossip networks, messages can overtake each other. This means that you might receive a message B which depends on another message A before you receive A . It might also be you don't receive messages from P_1 in the order in which they were sent by P_1 . These behaviours might lead to inconsistencies in how the system behaves. In this section we take a look at techniques for ensuring consistent communication. Along the way we will study Lamport clocks and vector clocks, which allows us to think clearly about causality in distributed systems. This is the most important part of the chapter. It gives powerful tools for reasoning about distributed systems. These can make arguments about correctness of distributed systems clearer, where the arguments otherwise might be quite involved and complicated. We look at global snapshotting as an example of this.

We will look at how to define and implement the following orderings: Unordered, FIFO, Causal, and Total Order.

Unordered All messages might be arbitrarily delayed and be delivered in different order than sent.

FIFO Messages from a given party are at all other parties delivered in the order in which they are sent. However, messages from two different parties might be delivered in different order at different parties.

Causal Here messages from different parties which might have affected each other are delivered in the order in which the effect could have happened. For instance: Questions are delivered before their answers.

Total Order All messages are delivered in the same order at all parties. Total order is also sometimes called Atomic Broadcast.

Note that total order does not imply causal order: total order just means that all parties output in the same order. That order might not be meaningful. Answers may arrive before questions. What we typically want is therefore Causal Total Order, i.e., an order of delivery which is both Causal and totally ordered.

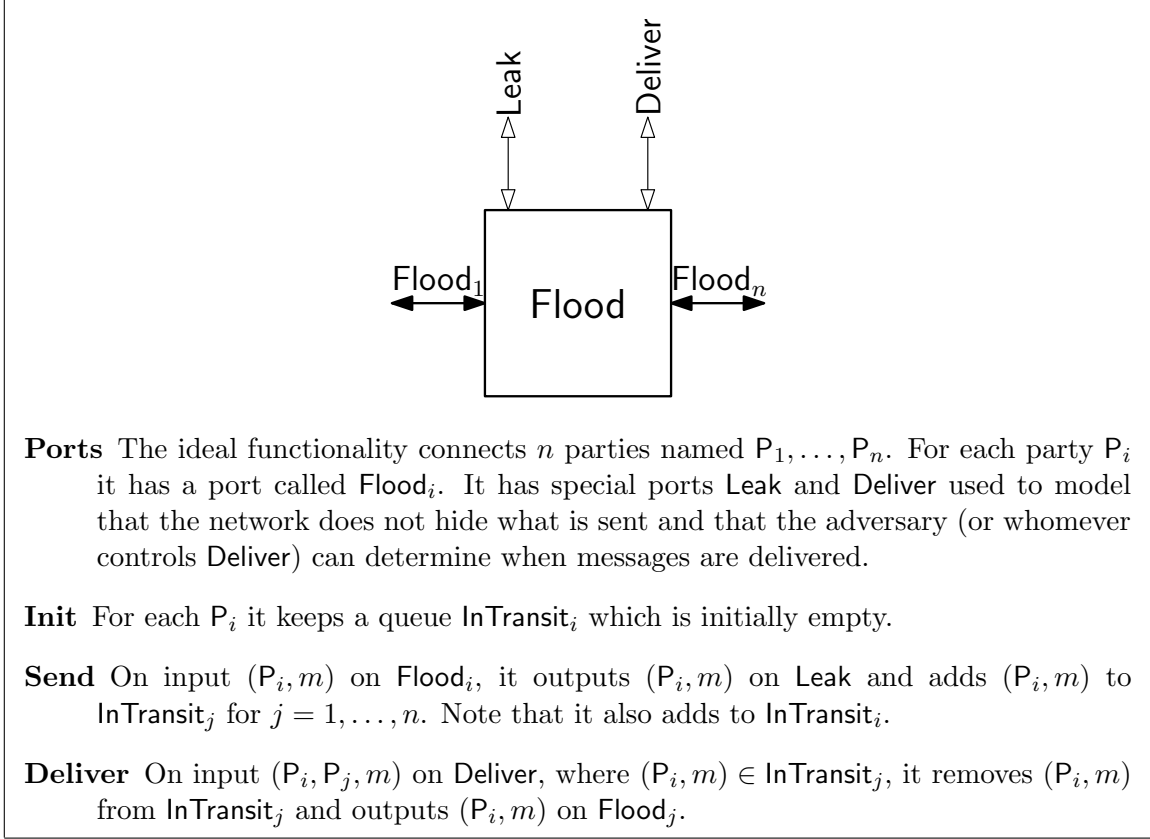


Figure 4.1: Flooding Network with Unordered messages delivery

Let us say one important thing up front: Causal Total Order implies Causal Order and therefore seems “better” than Causal Order. So why concern ourselves with Causal Order, why not just always use Causal Total Order? Well, Causal Total Order is easier to use, as it guarantees more. But, as it turns out, Causal Total Order is also much slower to implement. Causal Order can typically deliver messages much, much faster as we only need to guarantee that questions never overtake answers. Causal Total Order has to order all messages in a system, also completely unrelated ones. It is therefore a topic in implementing fast distributed systems to be able to do with “only” Causal Order on the middleware used for communication. Yet, some times Causal Total Order is what is needed. So we need to study both...

We assume that we have access to a so-called flooding system which guarantees that if a correct party sends a message then it is eventually delivered at all correct parties. As a model of this we are going to use a variant of the authenticated channel AC for n parties $P_1, \dots, P_n$. We call the network Flood , see Fig. 4.1. The reason for the name “Flooding Network” is that a natural way to ensure this property is to make sure the system is “flooded” with every new message, for instance by having each party pass on the message to anyone who has not heard it before, see also Section 3.10. We also assume that the ideal functionality is authenticated such that the receiver knows who sent a message. We model this by outputting the name of the sender along with the message and by not allowing the adversary to inject messages. We assume no other guarantees, so messages might be arbitrarily delayed and may be reordered. We look at how to design protocols on top of this with progressively stronger consistency guarantees.

We consider a system with the following syntax:

Send A party P_i can give input of the form (P_i, m) on the user port Flood_i . We say that P_i sent m (using Flood).

Deliver A party P_i can get output of the form (P_j, m) on the user port Flood_i . We say that the message m sent by P_j was delivered to P_i (by Flood).

We assume that the system has the following properties.

User Contract We require from the party P_i that it does not send the same m twice.

When the user keeps its contract, we require that the flooding system has the following safety and liveness properties.

Safety If a correct P_j outputs (P_i, m) , then earlier P_i sent (P_i, m) .

Liveness If a correct P_i sends (P_i, m) , then eventually all correct P_j deliver (P_i, m) .

As usual we are a bit lax with defining liveness. But most definitions will be sufficient for this section.

For protocols described in this chapter we will always assume that all processes are correct. Later in the book we look at solutions that work even if some processes crashes, are buggy or are malicious.

SOCRATES: Hey Plato, I have a question to the definition in Fig. 4.1. Note that when some P_i sends (P_i, m) on Flood_i , then (P_i, m) is added to InTransit_i . This means that the Deliver command can later be used to deliver (P_i, m) to P_i on Flood_i . Isn't this odd, like, a party sends a message to itself?

PLATO: Looking a bit ahead, I think it is because in a lot of the algorithms to come it says things like “P sends m to all parties” and “if a party Q received m from P then...”, and they want Q to be a generic party which could be P itself to have more concise algorithms. Without the convention that P sends to itself they would have to say things like “P sends m to all other parties” and “If a party Q received m from P or $Q = P$ and P earlier sent m , then ...”,

SOCRATES: OK, but it is easy to forget later that “P sends m to all parties” means it also sends to itself. Maybe the longer form would have been better. Anyway, there is still something odd going on I think. When a party P_i sends a message m via Flood.Flood_i it needs to wait to see m come back from Flood.Flood_i to consider it delivered locally. And the adversary—is that what they call the party controlling the special port Deliver?—controls when the message is delivered. So, there might be a delay in time between P_i sending (P_i, m_i) and the party P_i delivering (P_i, m_i) at itself. So it takes time to send to yourself?

PLATO: Yes, this is called **delayed local delivery**. It looks weird, but there is actually a good reason. Below they discuss total order broadcast, where all messages must be delivered in the same order at all parties. If two parties sending two messages at the same time also output them immediately, they cannot end up outputting them in the same order, as they each now has output a different message first.

SOCRATES: Ah, that makes sense for total order. But for a flooding network that does not need to output in the same order it seems to make less sense?

PLATO: Indeed, but I think it is because they want that these notions of consistency form a hierarchy, like causal total order implies causal order which in turn implied FIFO which in turn imply unordered delivery. If for instance FIFO was defined to have **immediate local delivery**, where a party delivers locally immediately when it sends the message, then Causal Total Order would not imply FIFO. So “all message being delivered in the same order at all parties” would not imply “messages sent by the same party are delivered in the same order at all parties”. That would also be strange.

SOCRATES: Yes, I think it is important that the formal definitions behave like our normal language, or we are apt to make mistake when thinking about them.

PLATO: There could also be real problems in practice, Like you have an application that was proven and tested secure using a FIFO middleware. Then you plug in a middleware with Causal Total Order instead and now the application breaks.

SOCRATES: Why would it break?

PLATO: Because using FIFO, if it was defined with immediate local delivery, then the application could assume immediate local delivery and somehow use that to work properly. When then you switch to Total Order, which cannot have immediate local delivery the application's assumptions might be broken and it stops working. As a stupid example consider an application which has as part of its code a switch that if it ever sees delayed local delivery, then it deliberately crashes. Using FIFO (had it been defined with immediate local delivery) it would never crash. But with Causal Total Order it would, as Total Order must sometimes do delayed local delivery to keep a global order.

SOCRATES: Oh, that is subtle.

PLATO: Yes, that is the problem with formal definitions instead of just Yada, Yada, Yada. They are hard to get right.

SOCRATES: Do you say that I just go Yada, Yada, Yada!?!

PLATO: I thought that was your thing?

Remark 4.1 (Immediate Local Delivery as Default) In Fig. 4.2 there is an example where P_1 sends m_1 via Flood and only receives m_1 after some delay. To not unnecessarily clutter future diagrams we will assume that Flood, FIFO and Causal have immediate local delivery in the diagrams.

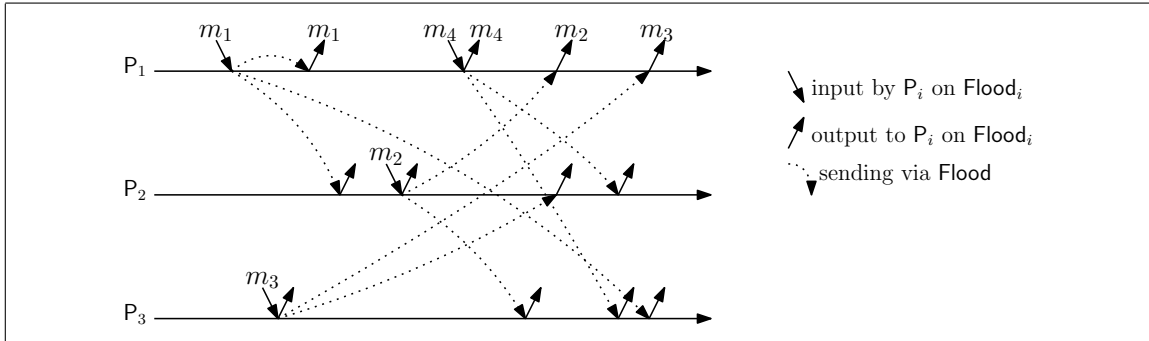


Figure 4.2: This is a timeline figure. Each line symbolizes time traveling at a local site of a party. Time travels from left to right. Events touch the timeline at the time when they happen. Party 1 is always on top in timeline figures in this chapter. In this example messages from P_1 arrive in different order at P_3 than the order in which they were sent. At P_1 the message m_1 has delayed local delivery, whereas m_4 has immediate local delivery.

4.2 First In, First Out

We first look at how to guarantee FIFO, i.e., we want to ensure that the communication preserves the order in which messages are sent by individual parties. We can formulate this as a safety property as follows:

FIFO If a correct P_i sends (P_i, m) and later sends (P_i, m') , then it holds for all correct P_j that if they deliver (P_i, m') , then they delivered (P_i, m) earlier.

The User Contract and Liveness is as for Flood.

SOCRATES: Hey, Plato, is this definition not unnecessarily convoluted? Why not say that if P_1 sends m and then m' then all other parties must deliver m and then m' ?

PLATO: I think they want to separate liveness and soundness properties. With your formulation it says that the receivers *must* deliver the messages. That is the liveness part and formulated separately. Above they only want to say that *if* the messages are delivered, then it is in the correct order.

SOCRATES: Ah, but then why not just say that if a receiver delivers both m and m' then m must be first. The above still seems to complicated...

PLATO: Well, in your formulation if a receiver delivers m' and then never delivers m , this would be OK, as you only say they must have the same order if they *both* are delivered. I'm not sure that is what they want. A receiver seeing m' might conclude that some m was never sent, rather than being overtaken and then never delivered.

SOCRATES: But if m is never delivered this would contradict liveness!

PLATO: Yes, but again, they want to separate these properties, so they do not want safety to make sense only if there is liveness, it should stand its own ground. So, they want that delivering m' without first having delivered m is illegal. I guess they could also just have said that for all receivers and all points in time it should hold that either 1) none of m and m' are delivered, or 2) only m is delivered, or 3) both are delivered, and in the order m then m' .

SOCRATES: I like that better. Sounds less highbrow. What is wrong with these people...

We model a FIFO network using an ideal functionality. See Fig. 4.3. It is simply designed to behave as Flood except that it keeps the FIFO safety property true by using a queue of messages.

Consider the following protocol.

1. Initially each party P_i will initialize a counter $c_i = 0$. This counter keeps track of how many messages the party P_i has already sent. Moreover each party P_i also initializes n counters $r_{i,j} = 0$ for $j = 1, \dots, n$. These counters keep track of how many messages party P_i has received from party P_j .
2. P_i : When sending a message m , let $c_i = c_i + 1$ and send (P_i, c_i, m) on the flooding network. This means that we tag each message with its sequence number, starting from 1.
3. P_i : When receiving (P_j, c, m) store it in a buffer until $r_{i,j} + 1 = c$. Then let $r_{i,j} = r_{i,j} + 1$ and output (P_j, m) . This makes sense as $r_{i,j}$ is the number of messages P_i received from P_j so if $r_{i,j} + 1 = c$, then (P_j, c, m) is the next message.

Clearly, if all parties are correct, this protocol ensures that all messages from each P_i are delivered by all other parties in the order in which they were sent by P_i . In particular, Flood2FIFO keeps the contract that each party sends each message at most once. This is ensured by the counter which is added to the messages sent. Therefore Flood will behave according to specification and eventually deliver the messages to all receivers exactly once. And it will not deliver any other messages. The logic of the counters then makes sure to deliver the messages in the right order. This ensure that Flood + Flood2FIFO securely implements FIFO.¹

¹One disadvantages of the protocol is that it keeps a buffer which can grow to any size. However, there is no way to solve the problem without using such a buffer either at the sender side or the receiver side. If one wants to use only fixed size buffer the sender should at some points be able to shut down for receiving new inputs from its user. This is what TCP/IP does. If the local buffer is full then on trying to send you are told EWOULDBLOCK and must try to send again later.

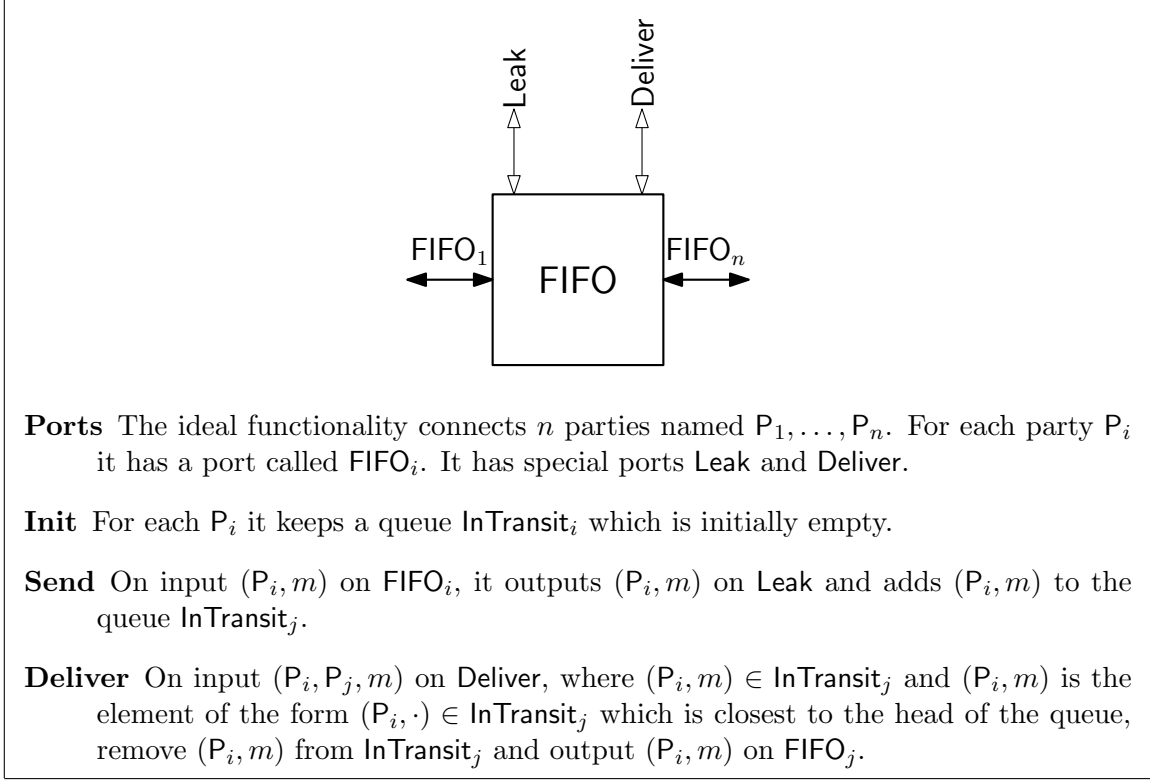


Figure 4.3: FIFO Network

4.3 Intermezzo on Modelling Distributed Systems I†

In this book we use two different styles of specifying what a protocol is supposed to do. We already discussed how to specify what a protocol should do using trace properties. We will also use the idea of ideal functionalities which originates from cryptography, specifically the sub-area of secure multiparty computation. Two examples of ideal functionalities are given in Fig. 4.1 and Fig. 4.3. When specifying what a protocol should do using an ideal functionality like **FIFO** we fully specify how it behaves by giving a step-by-step program for how to produce outputs given certain inputs. Trace properties instead only talk about the inputs and outputs and what patterns are allowed to be seen. This is very similar to the difference between operational semantics and denotational semantics in the field of semantics, and both have their advantages. We want to make the reader familiar with both styles, as they both are common in the literature.

Ideal functionalities can be used in two ways. In the top system of Fig. 4.6 **Flood** is being used as a middleware layer by the protocol **FIFO**. When an application or user at machine 1 gives an input to the **FIFO** protocol on the port $FIFO_1$ then the copy of the **FIFO** protocol on machine 1 uses the **Flood** middleware to talk to the other machines. It sends messages on $Flood_1$ and receives messages from the other machines back on $Flood_1$.

We think of the special ports of the **Flood** middleware as connected to some bad guy or hacker trying to break the protocol. So the bad guy sees all the messages sent and determines in which order they are delivered. This is a good model of TCP/IP running over the internet, where when you send a message it travels over many machines you cannot trust, and the dynamics of how TCP/IP works may give arbitrary delays. At the same time it is the bad guy which determines in which order the **FIFO**-software on individual machines are activated. This models a fully asynchronous execution. Of course, in real life no adversary has that fine control. But

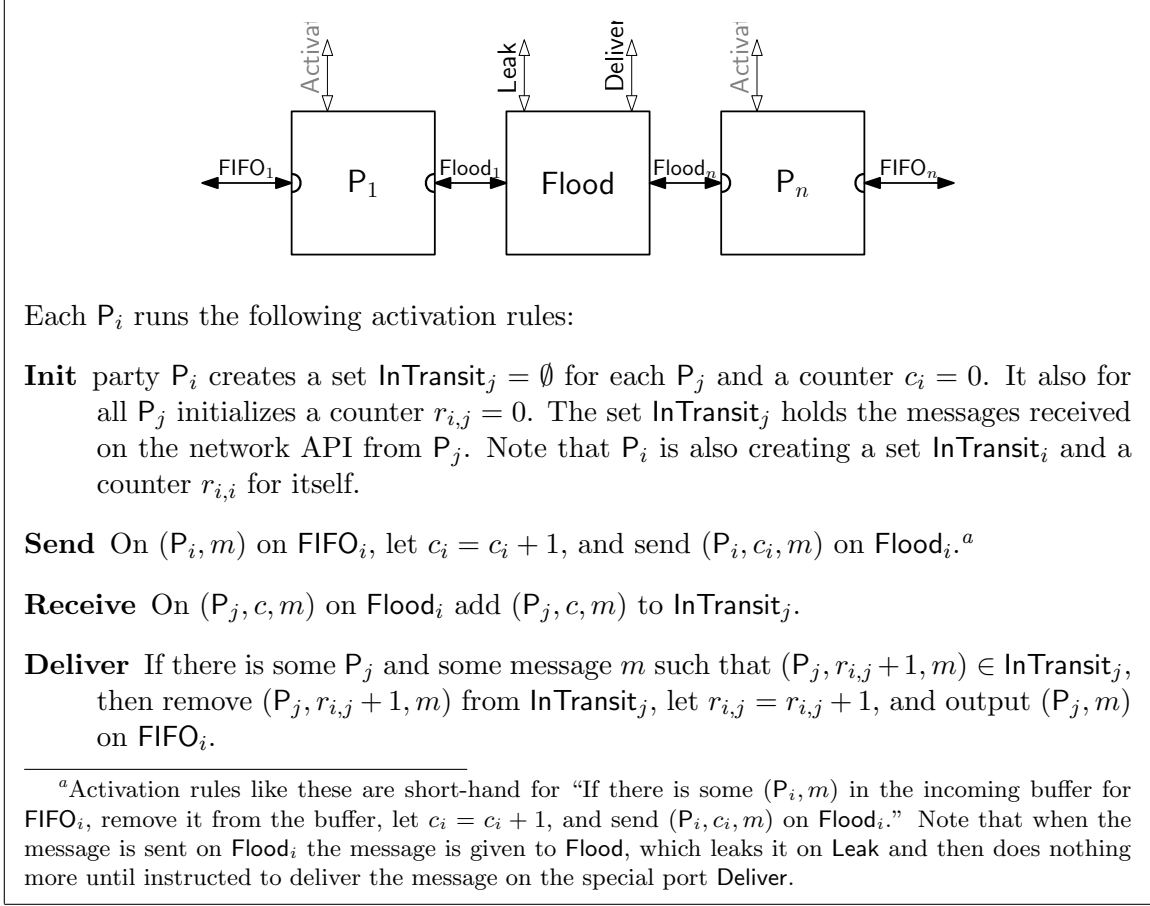


Figure 4.4: The Flood2FIFO protocol. The code P_1 is for running on machine 1, and so on for $P_2, \dots, P_n$. When a party receives an input it gets stored in an internal buffer associated to the port. This buffer is signified by the half circle. Parties only do something if activated on the **Activate**-port. Then they execute one activation run if any of them are ready to trigger. The ideal functionality **Flood** only delivers when instructed on **Deliver**. So the system does nothing except when explicitly scheduled.

the adversary might have *some* control. By modelling *full* control we are on the safe side: if we can prove security when the bad guy has full control, then we also have security when the bad guy has less control.

In the top system **Flood** is being used as a **resource** for communicating, also called a **network resource**. But one can also use ideal functionalities for *specifying* what a protocol should do. We think the protocol in Fig. 4.4 implements FIFO delivery. We define what this means by requiring that to the users it looks like they are using **FIFO** directly. Since **FIFO** also has special ports we need someone to control them when comparing the two system. Note that what we want to capture is that **Flood2FIFO** is secure if it does nothing which **FIFO** might not have done. We can show this by coming up with a good guy controlling **FIFO** such that the users cannot tell the difference between being in the top system or the bottom system. Namely, if whatever the bad guy can make **Flood2FIFO** output to a users in the top system the good guy can make **FIFO** output in the bottom system, then clearly **Flood2FIFO** does not do anything **FIFO** cannot do. That is then the definition of **Flood2FIFO** being secure: for all users of the system and all bad guys controlling **Flood** there should exists a good guy controlling **FIFO** such that the users

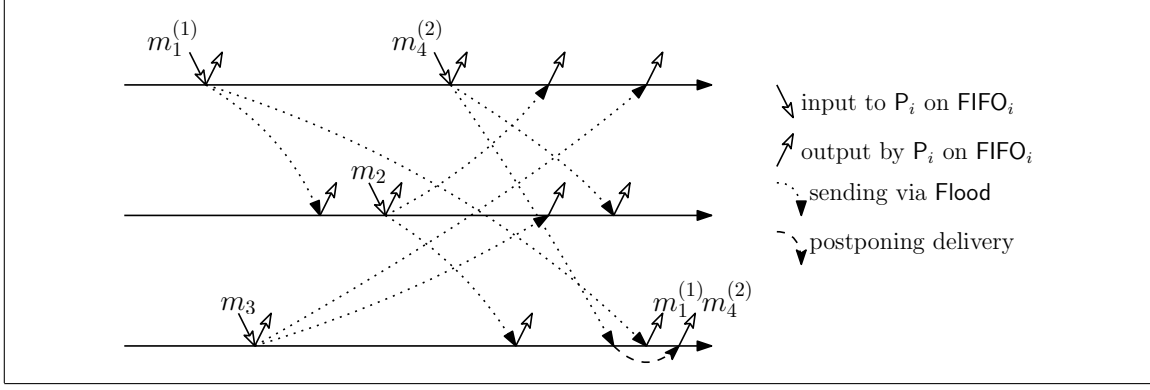


Figure 4.5: Messages from P_1 arrive in different order at P_3 than the order in which they were sent, but a counter is used to withhold m_4 until after m_1 arrives. For an example why we would like this think of a pun and its setup, like $m_1 = \text{WHY DID THE CHICKEN CROSS THE ROAD?}$ and $m_4 = \text{TO GET TO THE ORDER SIDE! [sic]}$. We want P_3 to hear the setup before the pun.

cannot tell whether they are in the top system or the bottom system. When this is the case we write $\text{Flood2FIFO} + \text{Flood} \sqsubseteq \text{FIFO}$ and we read $\sqsubseteq$ as “securely implements” and “at least as secure as”. We think of $\text{Flood2FIFO} + \text{Flood}$ as “at least as secure as FIFO” as it does not do anything FIFO would not do. For the formally inclined reader there are many more details in Chapter 2.

To drive home the connection between ideal functionalities and trace properties, recall that we earlier phrased FIFO as “If a correct P_i sends (P_i, m) and later sends (P_i, m') , then it holds for all correct P_j that if they deliver (P_i, m') , then they delivered (P_i, m) earlier.” We could have been even more precise and said that a FIFO communication protocol is a protocol with the application API $\text{FIFO}_1, \dots, \text{FIFO}_n$, where FIFO_i is available on machine i and which has the property that “If a machine i is correct and there was input (P_i, m) on FIFO_i and then later there was input (P_i, m') on FIFO_i , then it holds for all correct machines j that if there was output (P_i, m') on FIFO_j then earlier there was output (P_i, m) on FIFO_j .” It is easy to see that the ideal functionality FIFO has this property, and that it is in some sense the weakest ideal functionality having this property. It only blocks behaviours which would violate the FIFO trace property. Experience shows that phrasing a trace property correctly can be subtle, whereas formulating an ideal functionality doing (only) what one wants the protocol to do is more intuitive. Therefore ideal functionalities are gaining traction over trace properties in the literature, in particular in cryptography.

4.4 Causality

Physical time is relational, and simultaneity is not absolute, so in the extreme the only structure available to a distributed system is causality. In this section we look at how to ensure that messages are delivered in a causal order to at least have this structure to lean on in designing protocols.

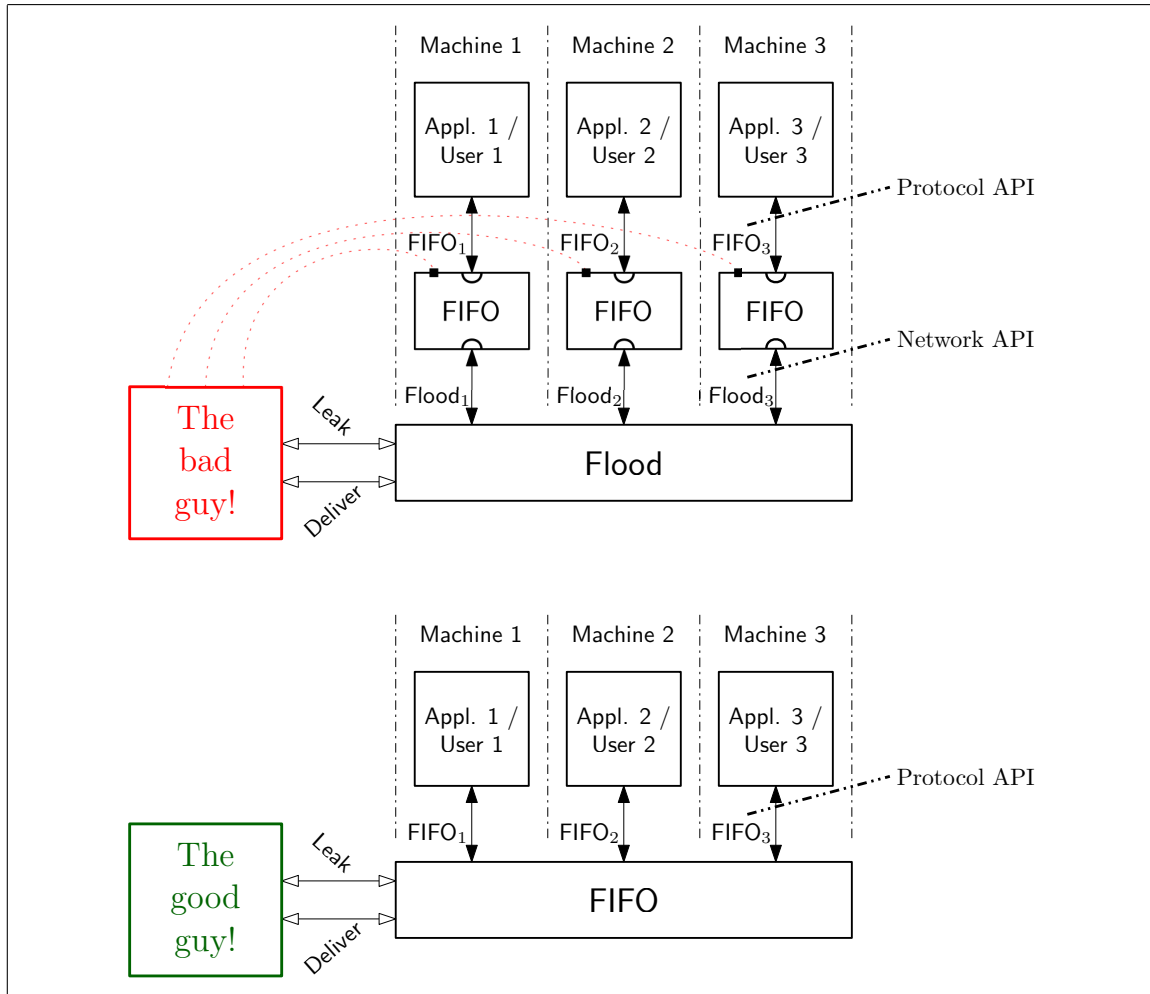


Figure 4.6: Top: the protocol Flood2FIFO + FIFO talking over the network resource Flood being controlled by a bad guy. The bad guy also controls in which order the copies of FIFO at the different parties get activated (gets run-time). Think of the software taking one step whenever the little square button is pushed by the bad guy. By running the copy at one party a lot and the copy at another party less corresponds to clocks drifting. Bottom: the ideal functionality FIFO being controlled by a good guy. Also some users, which use either the one system or the other.

“Time is nothing but the succession of our perceptions.”

—David Hume, *A Treatise of Human Nature* (1739)

“We have to take into account that all our judgments in which time plays a part are always judgments of simultaneous events. [...] simultaneity itself must be defined by an exchange of signals.”

—Albert Einstein, *On the Electrodynamics of Moving Bodies* (1905)

“There is no such thing as simultaneity of distant events.”

—Albert Einstein, *Relativity: The Special and the General Theory* (1916)

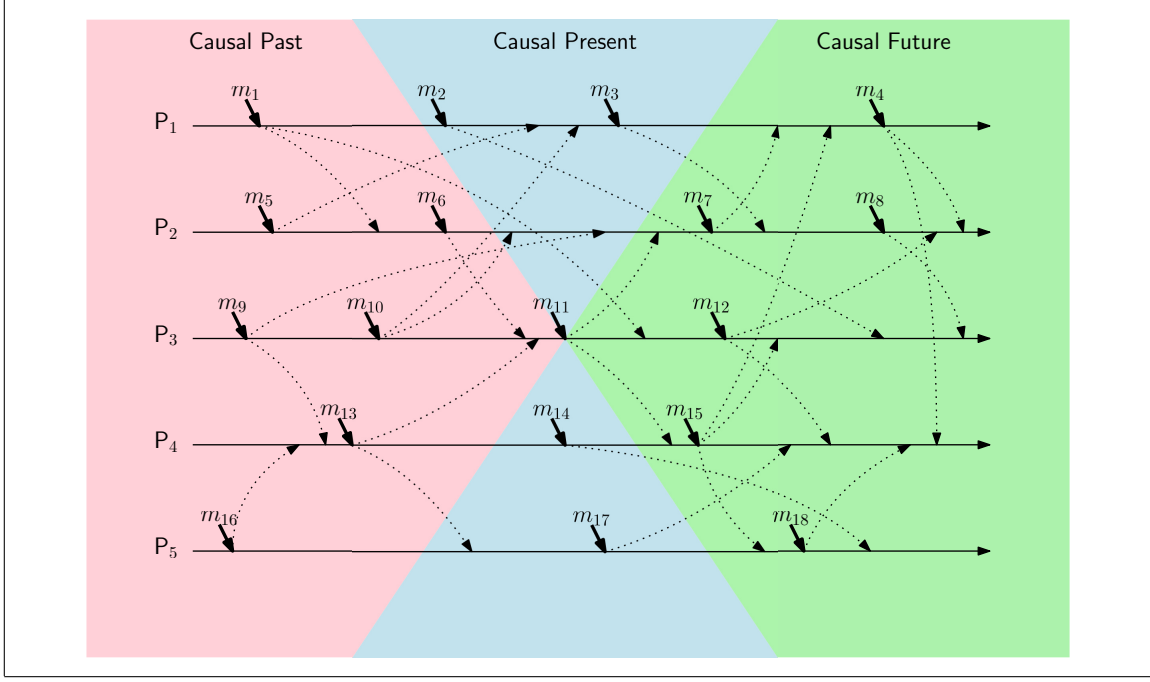


Figure 4.7: Causal past, causal present and causal future of (the event of sending) m_{11} .

When using FIFO as ideal functionality it might still happen that the messages from different parties do not arrive in a meaningful order. Consider the messages m_1 and m_2 in Fig. 4.5. When P_2 sent m_2 it already delivered m_1 . That means that the application layer using FIFO via FIFO.FIFO_2 might have chosen m_2 as a function of m_1 . We say that m_1 could have caused m_2 . For instance m_1 could be a question in some chat forum and m_2 could be the answer. But notice that at P_3 first m_2 is delivered, and then m_1 . It might confuse the application users at P_3 to read the answer before the question. In this section we will focus on defining and implementing an ideal functionality, **Causal**, which ensures that messages are delivered in a causal order: if a message m_1 could have caused m_2 , then all parties deliver m_1 before m_2 .

4.4.1 Causal Past, Present, Future

To define causality we need to be able to say when a message could have influenced another message. To do this we use the notions of causal past, present, and future. Here we do it by figure. We do the formal definitions below. The idea is illustrated in Fig. 4.7. Take a message like m_{11} . We say that the **causal past** of m_{11} is all the message which could have influence m_{11} . As an example, since m_6 arrived at P_3 before m_{11} was sent, m_6 could have influenced m_{11} . And since m_1 arrived at P_2 before m_6 was sent, m_1 could have influenced m_6 and therefore also indirectly m_{11} . Continuing this reasoning we get that the causal past of m_{11} is all messages in the shaded area labeled “Causal Past” in Fig. 4.7. It is the messages which can be reached by following arrows *backwards* from m_{11} . Technically we define the past as a *set* of messages, so

$$\text{CausalPast}(m_{11}) = \{m_1, m_5, m_6, m_9, m_{10}, m_{11}, m_{13}, m_{16}\}.$$

Notice that we include m_{11} itself in its own causal past.

Similarly, the causal future of (the event of sending) m_{11} is all (sending events of) messages which could be influenced by m_{11} . This is illustrated in Fig. 4.7. It is just the messages which

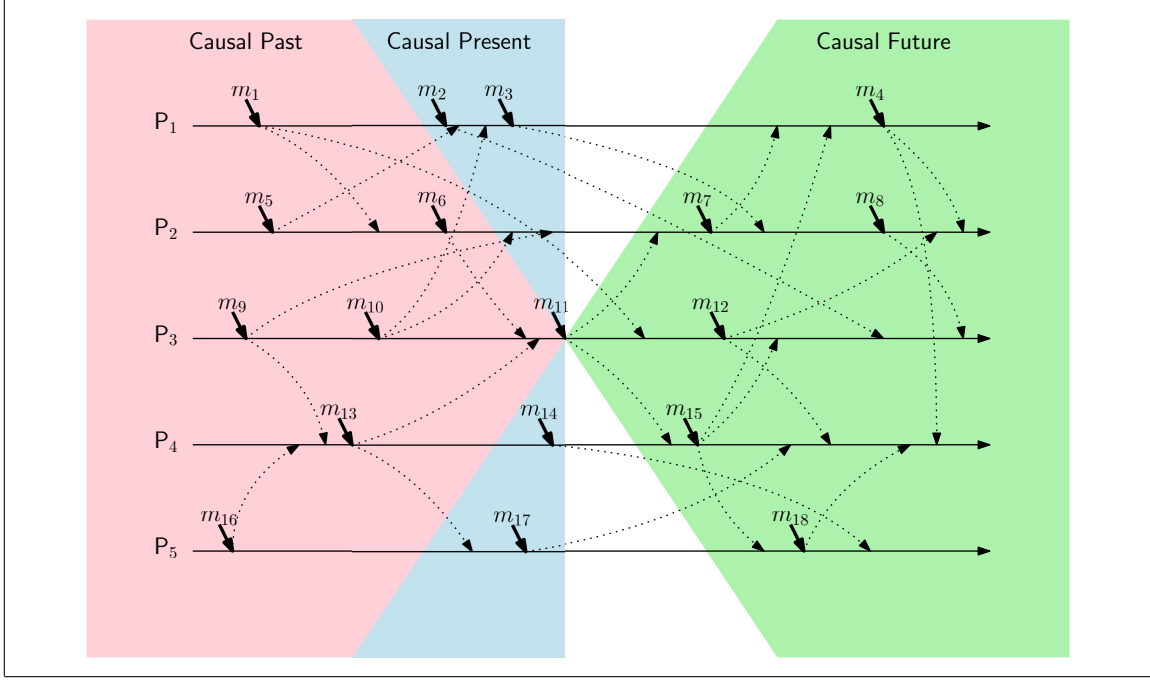


Figure 4.8: The Causal Present seen as happening in the “physical” past.

can be reached by following arrows *forward* from m_{11} . So

$$\text{CausalFuture}(m_{11}) = \{m_4, m_7, m_8, m_{11}, m_{12}, m_{15}, m_{18}\} .$$

Notice that we include m_{11} itself in its own causal future.

All the messages which are neither in the causal past nor in the causal future of m_{11} are said to be concurrent with m_{11} . They are sent in the region we call the causal present, i.e., the events that can consistently be seen to happen at the same “now” as the sending of m_{11} , from the view point of P_3 . These are the messages in the blue area in Fig. 4.7. It is just all the messages not in the causal past or future, except that we include m_{11} itself. So

$$\text{CausalPresent}(m_{11}) = \{m_2, m_3, m_{11}, m_{14}, m_{17}\} .$$

The messages sent in the causal present have the special property that they can both consistently be seen to have happened *before* the sending of m_{11} and also consistently be seen to have happened *after* the sending of m_{11} . This is illustrated in Fig. 4.8 and Fig. 4.9. In producing these alternative views we have only slid events along local timelines and kept the same order of all event. Think of events on local timelines in Fig. 4.7 as plotted using the local clock clocks of the parties, which is not synchronous with “global time” or each other. If the local clock was behind during the blue space-time area then the events get plotted too late relative to physical time, so when plotted in physical time they might have happened as in Fig. 4.8. Similarly if the local clock was ahead during the blue space-time area then the events get plotted too early relative to physical time, so when plotted in physical time they might have happened as in Fig. 4.9.

And only messages in the causal present have both these properties. Namely, events in the causal past cannot have happened after the sending of m_{11} . As an example, take m_6 . Since m_6 arrived at P_3 before m_{11} was sent, m_6 must have been sent before m_{11} in physical time. Similarly, events in the causal future cannot have happened before the sending of m_{11} . As an example, take m_7 . Since m_7 is sent after P_2 received m_{11} , m_7 must have been sent after m_{11} in physical time.

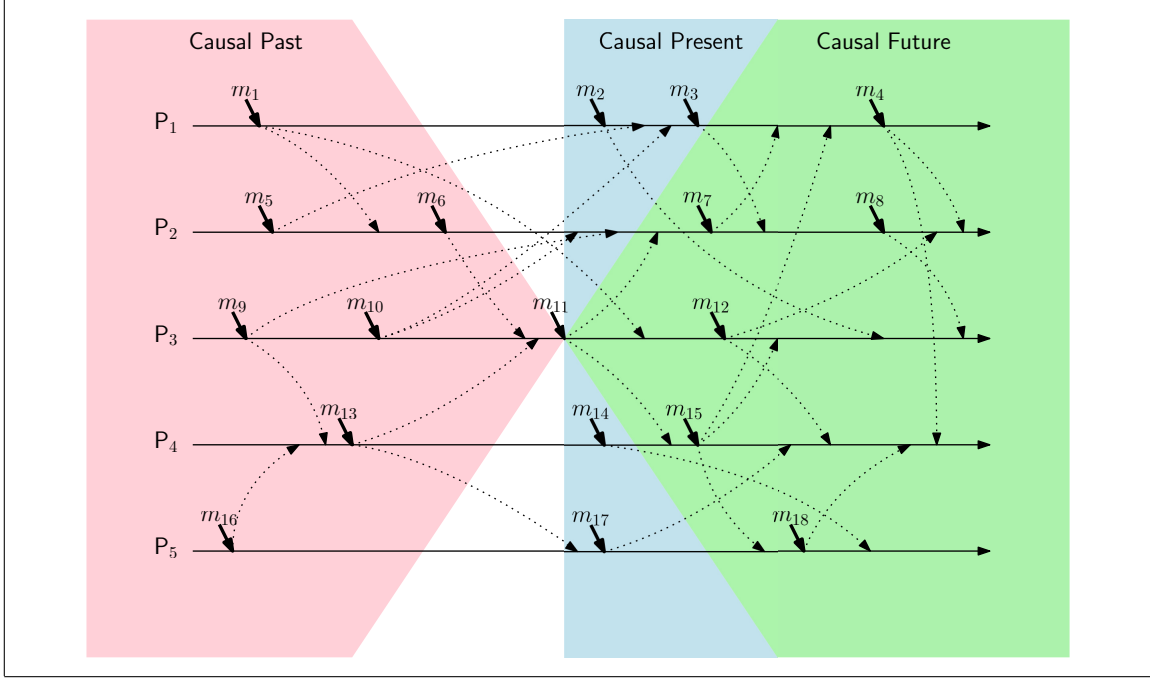


Figure 4.9: The Causal Present seen as happening in the “physical” future

4.4.2 An Implementation

When implementing causal delivery we want to ensure that when a message like m_{11} is delivered at some party, then all messages in its causal past are already delivered, i.e., all the messages in the red area. We think of a message as dragging its causal past along with it when it is sent. Informally we would also like that an implementation does not impose other restrictions on the delivery order than those needed to ensure causal delivery. For instance, in Fig. 4.7 we would like that m_3 and m_{11} can be delivered in any order at P_2 , as they are concurrent. This makes the systems as fast as possible while still ensuring causal delivery.

We will first present a protocol **FIFO2Causal** which given a **FIFO** ideal functionality implements a **Causal** ideal functionality. After that we will define what the **Causal** ideal functionality is. The reason why we do it this way around is that the protocol is actually pretty simple and almost self explanatory. The definition of causality as an ideal functionality on the other hand is a bit subtle. So let us start with the concrete and simple and then lift ourselves up to the abstract and complicated afterward.

Before describing the protocol it is convenient to have a notion of **vector clock**. For a system with n parties $P_1, \dots, P_n$ a vector clock is just a vector with n natural numbers (one for each P_i), i.e., $\text{VectorClock} \in \mathbb{N}^n$. We use $\text{VectorClock}[P_j] \in \mathbb{N}$ to denote the entry associated with P_j . Each party will have its own vector clock. We denote the vector clock of P_i by $\text{VectorClock}(P_i)$. To avoid confusion note that $\text{VectorClock}(P_i) \in \mathbb{N}^n$. So each party holds its own complete vector clock. Mixing the two notions we then have that $\text{VectorClock}(P_i)[P_j] \in \mathbb{N}$ is the entry in the vector clock of P_i associated with party P_j .

The basic idea of the vector clocks is that if $\text{VectorClock}(P_i)[P_j] = s$, then P_i knows that P_j sent at least s messages. Furthermore, there might be more messages that P_j sent via **FIFO**, but these cannot have affected P_i since P_i did not receive them, neither directly nor indirectly. When $\text{VectorClock}(P_i)[P_j] = s$, then only the s first messages sent by P_j could have affected P_i .

The protocol is described in Fig. 4.10. In Fig. 4.11 an example execution is shown. Below

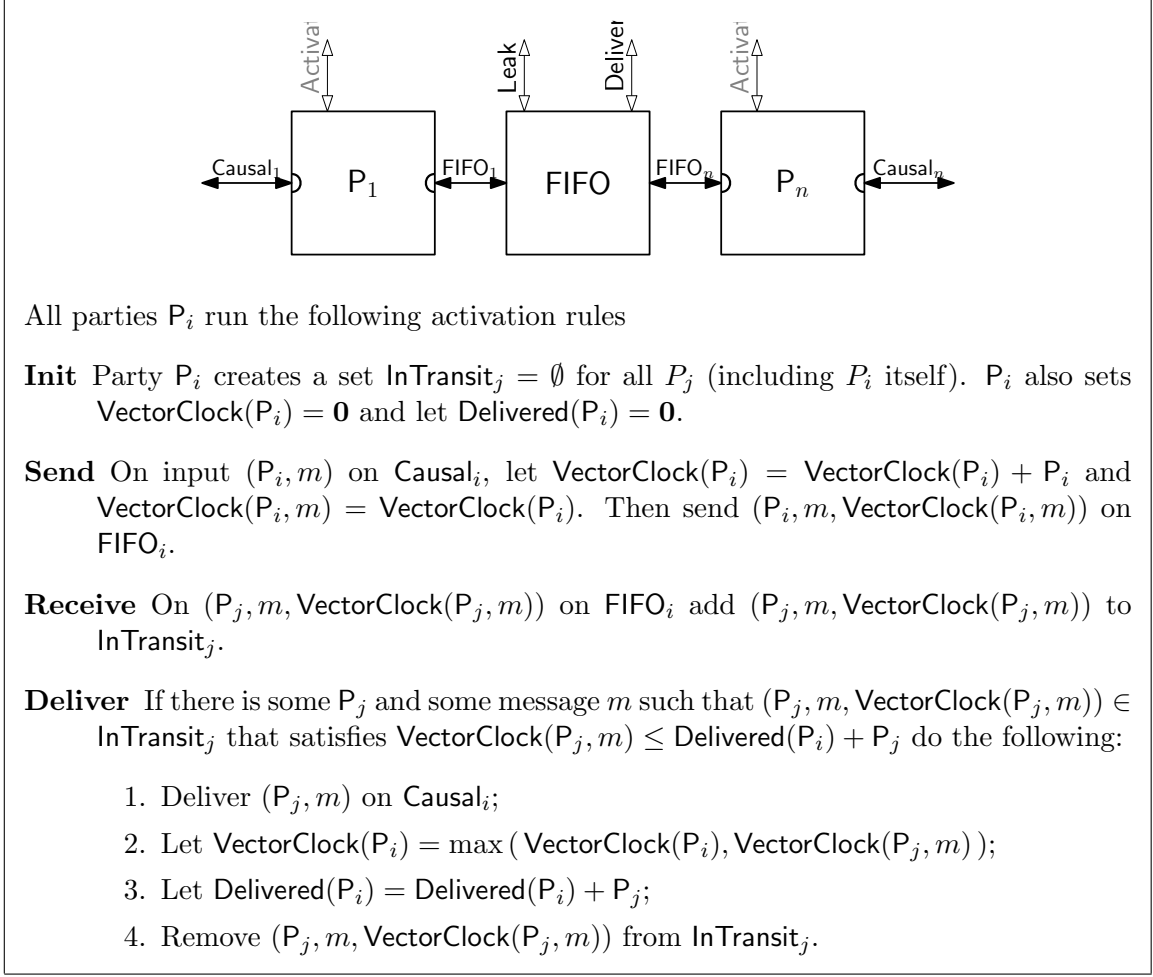


Figure 4.10: The Protocol FIFO2Causal.

follows a few simple observations that are useful in understanding the algorithm.

When P_i sends a message m it increments $\text{VectorClock}(P_i)[P_i]$ by one, as P_i just learned that P_i sent a message.

When P_i sends a message m it sends along $\text{VectorClock}(P_i)$. When P_i sends the message m that message can be influenced by exactly the messages that could have influenced P_i at the time P_i sends the message. So sending along the vector clock annotates the message m with a vector clock $\text{VectorClock}(P_i, m) = \text{VectorClock}(P_i)$ which allows to determine which messages could have influenced m . Namely, only the first $s = \text{VectorClock}(P_i, m)[P_j]$ messages from P_j could have influenced m . Therefore any receiver P_r knows that it is safe to deliver (P_i, m) once it delivered s messages from P_j : Using a FIFO ideal functionality ensures that once P_r delivered s messages from P_j they are exactly the s messages that could have influenced m .

To keep track of how many messages were delivered from each party, P_j keeps a vector clock $\text{Delivered}(P_j)$, where $\text{Delivered}(P_j)[P_i]$ is how many messages P_j delivered from P_i . This gives the rule that P_j can deliver (P_i, m) once it holds for all P_k that

$$\text{Delivered}(P_j)[P_k] \geq \text{VectorClock}(P_i, m)[P_k] .$$

There is one exception: when receiving a message m which is message number s sent by P_i , then $\text{VectorClock}(P_i, m)[P_i] = s$, as the vector clock of m counts the sending of m itself. So,

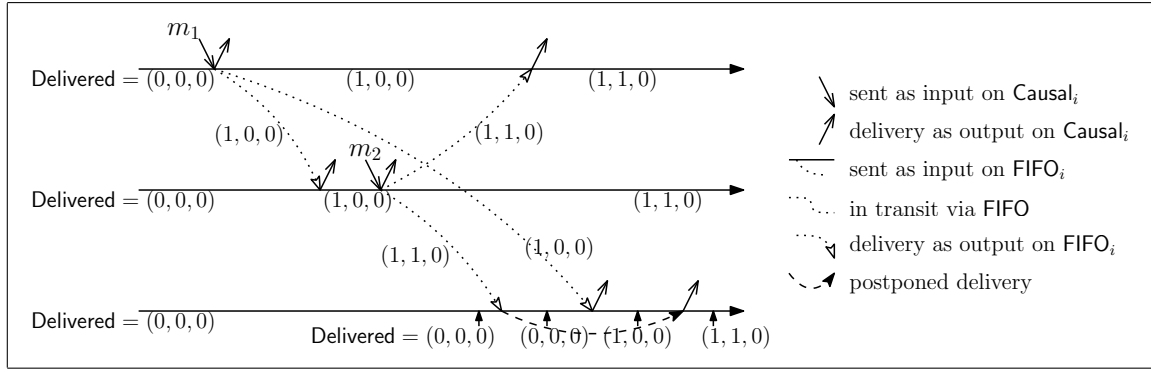


Figure 4.11: Message m_2 from P_2 , which saw one message m_1 from P_1 , overtakes m_1 . However, the counters of how many message you saw from each other party are sent along with the messages, as annotated on the dotted lines. When m_2 arrives at P_3 the message has clock $(1, 1, 0)$. The delivery clock of P_3 is $(0, 0, 0)$, so it cannot deliver a message with clock $(1, 1, 0)$ from P_2 as the first entry of $(0, 0, 0)$ shows that P_3 did not deliver any messages from P_1 and the first entry of $(1, 1, 0)$ shows that P_2 *did* deliver a messages from P_1 when it sent m_2 . Therefore P_3 will wait with delivering m_2 . When m_1 arrives at P_3 it has clock $(1, 0, 0)$ and since the delivery clock of P_3 is $(0, 0, 0)$ it can be seen that P_3 delivered all the same messages as P_1 when it sent m_1 , except m_1 itself. Therefore P_3 can output m_1 and increment its delivery clock to $(1, 0, 0)$. Now that the delivery clock is $(1, 0, 0)$ it holds for the clock $(1, 1, 0)$ of m_2 that it is the same as the delivery clock except it is one larger for the sender of m_2 . Hence m_2 can be delivered and the delivery clock upgraded to $(1, 1, 0)$. For an example why we would like causal order think of a question and its answer (from another server), like $m_1 = \text{WHY DID THE CHICKEN CROSS THE ROAD?}$ and $m_2 = \text{TO GET TO THE ORDER SIDE!}$. We want P_3 to hear the question before the answer.

for instance, if m is the first message sent by P_i , then $\text{VectorClock}(P_i, m)[P_i] = 1$. But since P_j did not receive m yet it might be the case that $\text{Delivered}(P_j)[P_i] = s - 1$. And when $\text{Delivered}(P_j)[P_i] = s - 1$ we would actually like to deliver m at P_j , as it is the next message from P_i . We therefore allow to deliver a message from P_i already when

$$\text{Delivered}(P_j)[P_i] \geq \text{VectorClock}(P_i, m)[P_i] - 1 .$$

Finally, note that if P_i receives a message (P_k, m) where for some P_j it holds that

$$\text{VectorClock}(P_k, m)[P_j] > \text{VectorClock}(P_i)[P_j] ,$$

then m could have been influenced by more messages from P_j than P_i might have been influenced by. But now these messages could have influenced P_i indirectly via m , so P_i updates $\text{VectorClock}(P_i)[P_j] = \text{VectorClock}(P_k, m)[P_j]$. Put more concisely, we could just always let $\text{VectorClock}(P_i)[P_j] = \max(\text{VectorClock}(P_i)[P_j], \text{VectorClock}(P_k, m)[P_j])$ for all P_j when receiving a new message (P_k, m) . All parties update their clocks like this in response to all incoming messages.

For more examples of how vector clocks are computed, see Fig. 4.15 appearing later in the chapter.

We now put the above observations together into an algorithm. For brevity we will use

$$\text{VectorClock}' = \text{VectorClock} + P_i$$

to denote the vector clock $\text{VectorClock}'$ for which $\text{VectorClock}'[P_j] = \text{VectorClock}[P_j]$ for all $P_j \neq P_i$ and $\text{VectorClock}'[P_i] = \text{VectorClock}[P_i] + 1$. We define

$$\text{VectorClock}_1 \leq \text{VectorClock}_2$$

to mean that

$$\forall P_k (\text{VectorClock}_1[P_k] \leq \text{VectorClock}_2[P_k]) .$$

We define

$$\text{VectorClock} = \max (\text{VectorClock}_1, \text{VectorClock}_2)$$

to mean that

$$\forall P_k (\text{VectorClock}[P_k] = \max(\text{VectorClock}_1[P_k], \text{VectorClock}_2[P_k])) .$$

We let $\mathbf{0}$ denote the all-0 vector, i.e., $\mathbf{0}[P_j] = 0$ for all P_j . The algorithm is given in Fig. 4.10.

4.4.3 The Causal-Past Relation

We would now like to prove that `FIFO2Causal` implements causal communication. For this we need to make a definition of what causal communication is. We will derive this definition via the causal-past relation, which describes when events in a distributed system might be causally related. The relation is more idiomatically called the *happens-before* relation, but we call it the *causal-past* relation here as this better captures our particular use.

The causal-past relation is a binary relation $\hookrightarrow$ on messages (P, m) . We will write it as $(P_i, m_i) \hookrightarrow (P_j, m_j)$. The intuitive meaning of this is that, in a particular run of the system, m_j may depend on m_i , for instance because P_j received m_i before it sent m_j . Note that the fact that P_j received m_i before sending m_j does not necessarily mean that m_j in fact depends on m_i , only that it *might* depend on m_i . Maybe m_j is not an answer to a question posed in m_i . Perhaps m_j is a message on a completely unrelated topic. We cannot say anything about the actual causal relationship between m_j and m_i without knowing more details about the application we are designing. Therefore, as usual, to be on the safe side, we say that if there is a chance that m_j depends on m_i , then we should treat m_j as if it in fact was causally related to m_i . Note also that not all send events are necessarily related by the causal-past relation in either direction. Two send events can easily be completely independent from each other, in which case they are not in the relation. As an example, party P_j might have sent m_j around the same time where m_i was sent by P_i but before P_j has seen m_i . Then m_i cannot depend on m_j and m_j cannot depend on m_i . We call such send events concurrent.

We use $\text{CausalPast}(P_j, m_j)$ to denote the set of (P_i, m_i) ² on which (P_j, m_j) may depend, i.e.,

$$\text{CausalPast}(P_j, m_j) = \{ (P_i, m_i) \mid (P_i, m_i) \hookrightarrow (P_j, m_j) \} .$$

Similarly we define

$$\text{CausalFuture}(P_j, m_j) = \{ (P_i, m_i) \mid (P_j, m_j) \hookrightarrow (P_i, m_i) \} .$$

²Note that as opposed to the definition by figure given earlier, here we define the causal past and future as sets of pairs (P, m) , i.e., as sets of messages together with the party that sent them. This is just a technicality as messages are just bit-strings and therefore not uniquely identifiable. Two parties might send the same message m . Therefore we need to include the party P that sent m to be able to uniquely identify the sending event of m . We tacitly assume that each party never sends the same message twice, so that the pair (P, m) uniquely identifies a sending event, say by including a counter in the message m as done in `Flood2FIFO`.

Clearly, if we know the set $\text{CausalPast}(P_j, m_j)$, then we are also able to evaluate the $\hookrightarrow$ relation for the given message, since it holds that

$$(P_i, m_i) \hookrightarrow (P_j, m_j) \iff (P_i, m_i) \in \text{CausalPast}(P_j, m_j) . \quad (4.1)$$

So, if we can define $\text{CausalPast}(P_j, m_j)$ for all (P_j, m_j) , then we also defined $(P_i, m_i) \hookrightarrow (P_j, m_j)$ for all (P_i, m_i) and (P_j, m_j) , and *vice versa*.

So, for an execution of a given protocol between a set of parties, we can determine the relation $\hookrightarrow$ for all messages by determining all the sets $\text{CausalPast}(P_j, m_j)$. We can do so operationally, going through the run of the protocol step by step, as described in the method below. The method described keeps track of auxiliary sets $\text{CausalPast}(P_i)$ for each party. These sets contain, at any given time, all messages that may have influenced P_i 's behaviour up to now. Below we use P_i to denote the process at the application layer using the causal network. It sends messages on the port Causal_i and message intended for P_i are delivered on the port Causal_i . We keep track of the message sent in the causal network and their causal pasts. It is fairly easy to see that the following is a good operational definition.

1. Initially let $\text{CausalPast}(P_i) = \emptyset$ for all P_i .

Justification: Initially no messages can have affected P_i .

2. On input (P_i, m) on Causal_i at P_i , let $\text{CausalPast}(P_i) = \text{CausalPast}(P_i) \cup \{(P_i, m)\}$ and let $\text{CausalPast}(P_i, m) = \text{CausalPast}(P_i)$.

Justification: When (P_i, m) is input on Causal_i at P_i , then clearly m now could have affected P_i , as it was P_i that input m to the system, so we let $\text{CausalPast}(P_i) = \text{CausalPast}(P_i) \cup \{(P_i, m)\}$. Furthermore, since m was input at P_i all the messages that could have affected P_i could have affected m , so we let $\text{CausalPast}(P_i, m) = \text{CausalPast}(P_i)$.

3. When (P_j, m) is delivered on Causal_i at P_i , let $\text{CausalPast}(P_i) = \text{CausalPast}(P_i) \cup \text{CausalPast}(P_j, m)$.

Justification: When m is delivered on Causal_i at P_i then the application layer connected to Causal_i might now be affected by m and therefore indirectly also the messages that might have influenced m . Therefore we set $\text{CausalPast}(P_i) = \text{CausalPast}(P_i) \cup \text{CausalPast}(P_j, m)$.

As mentioned, the causal-past relation $(P_i, m_1) \hookrightarrow (P_j, m_2)$ is supposed to capture when m_1 might have influenced the contents of m_2 . The first rule says that initially no party could have been influenced by any message. This is clear as no messages were sent so far.

The second rule says that if P_i sends a message, then the set of messages that might have influenced P_i grows by this message. This is clear as P_i has certainly seen the message. Furthermore, the message might have been influenced by all messages that may have influenced P_i —simply because the message was produced by the system running at P_i 's site.

The third rule says that if P_i has delivered a message m , then P_i may now have been influenced by all messages that may have influenced m .

The last implicit rule postulates that there is no other way m_2 could depend on m_1 . This rule is implicit in the sense that if we thought there were other ways in which messages could affect each other we would have added more explicit update rules. This fourth rule is true in a fully asynchronous system, where parties are not allowed to use a clock and where in general information only flows via message passing inside our flooding network. But the rule is not always true.³

³Assume that m_1 is a bit. Consider the case where P_i sends m_1 if $m_1 = 1$ and sends nothing when $m_1 = 0$. Assume that P_j knows when m_1 is sent and knows an upper bound on the delivery time. Then when after long

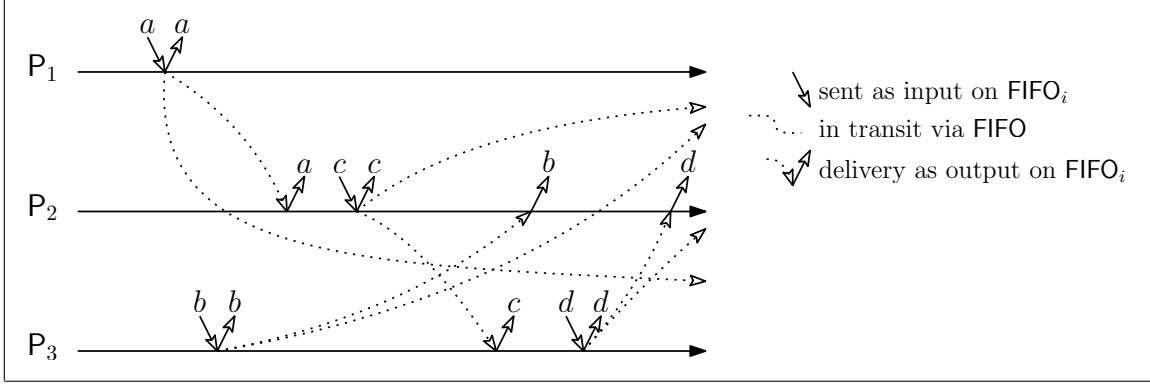


Figure 4.12: Causal Past Example. Notice that communication is via FIFO, not via Causal. Notice also that the communication is not causal, as we deliver c at P_3 before a has arrived, and $a \hookrightarrow c$ as P_2 sent c after being delivered a . We can still think of the communication in terms of causality by following arrows as in Fig. 4.7. We have that $a \hookrightarrow c$ at P_2 and $b \hookrightarrow d$ and $c \hookrightarrow d$ at P_3 . We see that a and b are concurrent as they cannot reach each other by going only forward or backwards. Similarly b and c are concurrent.

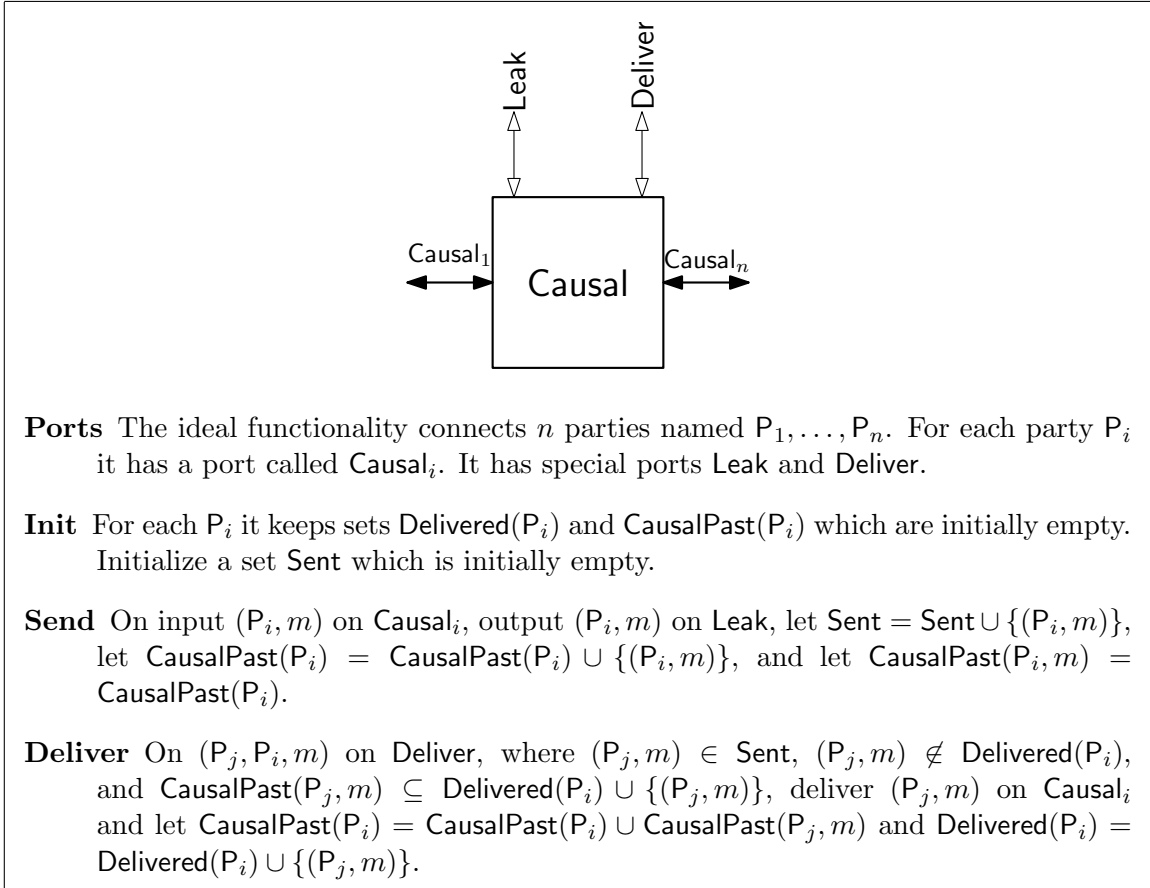


Figure 4.13: Causal Network

enough time it did not receive any message from P_i party P_j it can set $m_2 = 0$. If it does receive some m_1 it

Example 4.2 Let us look at a small concrete example where we communicate via FIFO, see Fig. 4.12.

- P_1 is told to send message a (i.e., there is input (P_1, a) on FIFO.FIFO_1). Followed by immediate local output.
- P_3 is told to send message b (i.e., there is input (P_3, b) on FIFO.FIFO_3). Followed by immediate local output.
- FIFO delivers a to P_2 (by outputting (P_1, a) on FIFO.FIFO_2 (after having being instructed to do so on FIFO.Deliver)).
- P_2 is told to send message c (i.e., there is input (P_2, c) on FIFO.FIFO_2). Followed by immediate local output.
- FIFO delivers c to P_3 (by outputting (P_2, c) on FIFO.FIFO_3).
- FIFO delivers b to P_2 (by outputting (P_3, b) on FIFO.FIFO_2).
- P_3 is told to send message d (i.e., there is input (P_3, d) on FIFO.FIFO_3). Followed by immediate local output.
- FIFO delivers d to P_2 (by outputting (P_3, d) on FIFO.FIFO_2).

In this example, we have 4 send-events $(P_1, a), (P_3, b), (P_2, c), (P_3, d)$. First, (P_1, a) and (P_3, b) are concurrent, the two involved parties have not talked to each other yet, so there is no way a could depend on b or vice versa. Second, we have $(P_1, a) \hookrightarrow (P_2, c)$ since P_2 gets a before it sends c . And finally, (P_3, b) and (P_2, c) are concurrent, because P_2 produces c before it receives b . ▼

Exercise 4.1 (properties of causality sets) Prove that the causal-past set and the causal-future set have the following properties:

Set Equivalence: It holds that

$$(P_i, m_i) \hookrightarrow (P_j, m_j)$$

if and only if

$$\text{CausalPast}(P_i, m_i) \subseteq \text{CausalPast}(P_j, m_j) .$$

Note that this does not follow by definition, as the definition says that $(P_i, m_i) \hookrightarrow (P_j, m_j)$ if and only if $(P_i, m_i) \in \text{CausalPast}(P_j, m_j)$. Note also that for the same reason, it is enough to prove that $(P_i, m_i) \in \text{CausalPast}(P_j, m_j)$ if and only if $\text{CausalPast}(P_i, m_i) \subseteq \text{CausalPast}(P_j, m_j)$.

Antisymmetric: $\text{CausalPast}(P, m) \cap \text{CausalFuture}(P, m) = \{(P, m)\}$.

sets $m_2 = 1$. So clearly m_2 always depends on m_1 , but in half of the cases no message was sent from P_i to P_j so by definition we have that $(P_i, m_1) \not\hookrightarrow (P_j, m_2)$ in those cases. The problem is that if parties are allowed to use time, then information can flow in other ways than via message passing. We will later look at other unfortunate ways information can flow if for instance parties are on the same machine. It could also be that even though no message is sent from P_i to P_j between P_i sending m_i and P_j sending m_j , they could have communicated by other means, maybe by sitting in the same room and talking. We cannot take care of that in the definition, as we only have access to the communication inside the system.

Exercise 4.2 (properties of causal-past relation) Prove that the causal-past relation $\hookrightarrow$ has the below properties. You may assume you solved the above exercise and use its result.

Transitive: If $(P_i, m_i) \hookrightarrow (P_j, m_j)$ and $(P_j, m_j) \hookrightarrow (P_k, m_k)$, then $(P_i, m_i) \hookrightarrow (P_k, m_k)$.

Reflexive: It holds for all events (P_i, m_i) that $(P_i, m_i) \hookrightarrow (P_i, m_i)$.

Antisymmetric: If $(P_i, m_i) \hookrightarrow (P_j, m_j)$ and $(P_j, m_j) \hookrightarrow (P_i, m_i)$, then $(P_i, m_i) = (P_j, m_j)$.



4.4.4 The Causal Communication ideal functionality, Causal

With the above definition of the causal-past relation it is simple to construct an ideal functionality **Causal** which only allows causal communication. It will generally work the same way as **FIFO**, but it will only allow messages to be delivered in causal order. To ensure this it keeps track of which messages were delivered at each party P_i . For this purpose it uses a set Delivered_i . Now before delivering a message m from P_j at P_i it simply checks whether $\text{CausalPast}(P_j, m) \subseteq \text{Delivered}_i$. If so, then all messages that (P_j, m) could depend on were already delivered at P_i , so it is safe to deliver (P_j, m) at P_i . The condition $\text{CausalPast}(P_j, m) \subseteq \text{Delivered}_i$ is, however, too strict. Recall that $(P_j, m) \in \text{CausalPast}(P_j, m)$. And clearly, before we deliver (P_j, m) at P_i we will have that $(P_j, m) \notin \text{Delivered}(P_i)$. So, clearly $\text{CausalPast}(P_j, m) \not\subseteq \text{Delivered}_i$. So the delivery of (P_j, m) is deadlocked, waiting for the delivery of itself. So what we want to say is that $\text{CausalPast}(P_j, m) \subseteq \text{Delivered}_i$, except that it is allowed that (P_j, m) is not in Delivered_i , which we can say by requiring that $\text{CausalPast}(P_j, m) \subseteq \text{Delivered}_i \cup \{(P_j, m)\}$. With this in mind, the ideal functionality **Causal** is defined as in Fig. 4.13.

Recall that we will in general not do full simulation proof, just argue that our protocols have the desired safety properties. Note that **Causal** behaves exactly as **FIFO** except that it delivers messages according to the following safety property.

Causal Order If $(P_i, m) \hookrightarrow (P_j, m')$ and $(P_i, m) \neq (P_j, m')$, then it holds for all correct P_k that if they deliver (P_j, m') , then they have previously delivered (P_i, m) .

The User Contract and Liveness properties are as for **Flood**.

It can be seen that **Causal** is the least restrictive ideal functionality guaranteeing this safety property. Let us argue one of the directions, namely that **Causal** maintains the above safety property. To prove this we have to assume that $(P_i, m) \hookrightarrow (P_j, m')$ and $(P_i, m) \neq (P_j, m')$ and that P_k delivered (P_j, m') . Then we have to prove that P_k previously delivered (P_i, m) .

To see this, note that Delivered_i is exactly the messages delivered at P_k . So P_k having delivered (P_j, m') is the same as $(P_j, m') \in \text{Delivered}_k$. At the point where **Causal** added (P_j, m') to Delivered_k it checked that

$$\text{CausalPast}(P_j, m') \subseteq \text{Delivered}_k \cup \{(P_j, m')\} . \quad (4.2)$$

Since $(P_i, m) \hookrightarrow (P_j, m')$ we have by definition that $(P_i, m) \in \text{CausalPast}(P_j, m')$. So from Eq. 4.2 we get that

$$(P_i, m) \in \text{Delivered}_k \cup \{(P_j, m')\} . \quad (4.3)$$

Since $(P_i, m) \neq (P_j, m')$ it follows from Eq. 4.3 that

$$(P_i, m) \in \text{Delivered}_k .$$

Therefore (P_i, m) was already delivered before the point where (P_j, m') was delivered. The other direction is argued similarly.

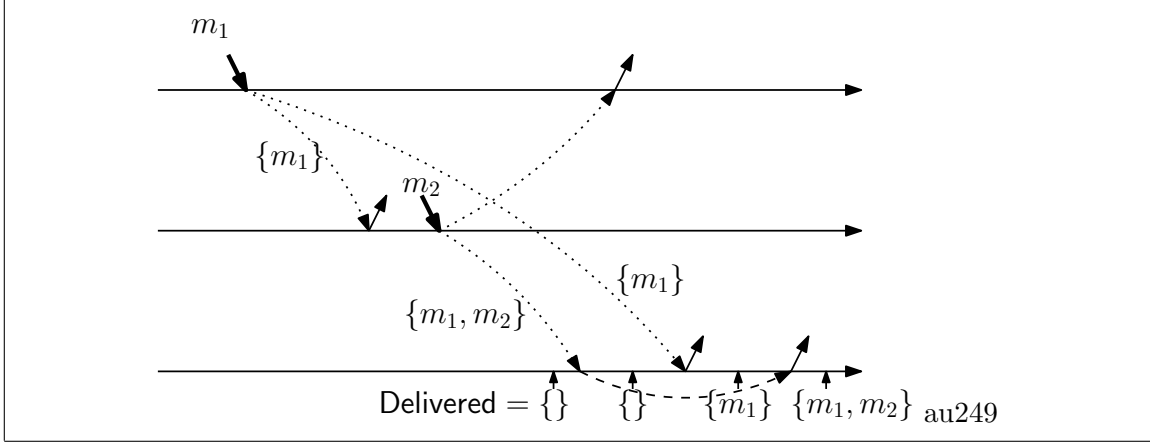


Figure 4.14: Message m_2 from P_2 , which has m_1 in its causal past, overtakes m_1 . However, the causal past of messages are sent along with them, as annotated on the dotted lines. When m_2 arrives at P_3 it has causal past $\{m_1, m_2\}$ and P_3 has $\text{Delivered} = \emptyset$, so m_2 cannot be delivered. Later when m_1 arrives at P_3 it has causal past $\{m_1\}$ and P_3 has $\text{Delivered} = \emptyset$, so m_1 can be delivered. After this m_2 still has causal past $\{m_1, m_2\}$ but P_3 now has $\text{Delivered} = \{m_1\}$, so now m_2 can be delivered. After this P_3 has $\text{Delivered} = \{m_1, m_2\}$.

4.4.5 Proving Security

We now prove the following theorem.

Theorem 4.3 $\text{FIFO2Causal} + \text{FIFO} \sqsubseteq \text{Causal}$.

Since we are not doing full simulation proofs, we just have to argue that $\text{FIFO2Causal} + \text{FIFO}$ produces outputs according to the safety property Causal Consistency defined above. As a step towards this, we first give an implementation for which this is trivial. The protocol we give is very inefficient but trivially a safe protocol. It will simply keep track of which messages were delivered and then only deliver a message m if all the messages that may have influenced m have already been delivered. All parties P_i run the following code:

Init Initially let $\text{CausalPast}(P_i) = \emptyset$ for all P_i and let $\text{Delivered}(P_i) = \emptyset$

Send On input (P_i, m) on Causal_i , let $\text{CausalPast}(P_i) = \text{CausalPast}(P_i) \cup \{(P_i, m)\}$ and let $\text{CausalPast}(P_i, m) = \text{CausalPast}(P_i)$. Then send $(P_i, m, \text{CausalPast}(P_i, m))$ on Flood_i .

Deliver On receiving $(P_j, m, \text{CausalPast}(P_j, m))$ on Causal_i wait until $\text{CausalPast}(P_j, m) \subseteq \text{Delivered}(P_i) \cup \{(P_j, m)\}$. Then deliver (P_j, m) on Causal_i , and let $\text{CausalPast}(P_i) = \text{CausalPast}(P_i) \cup \text{CausalPast}(P_j, m)$. Also add (P_j, m) to $\text{Delivered}(P_i)$.

Let us call this protocol Flood2Causal . There is an example of an execution in Fig. 4.14. Note that the sets are being defined exactly as in Causal . So it is completely trivial that

Lemma 4.4 $\text{Flood2Causal} + \text{Flood} \sqsubseteq \text{Causal}$.

4.4.6 Turning Sets into Vector Clocks

As we have just argued, the above protocol Flood2Causal satisfies the desired properties of safety and liveness (in a pretty much straightforward way). It is, however, horribly inefficient, since

every time a message m is sent, the entire set of messages that might have influenced m is sent along. We now give a much more efficient protocol. Instead of all parties keeping track of all messages received from P_j they could just use the FIFO network instead of the Flood network, and then only keep track of how *many* messages they received. The crucial observation is that if two parties P_i and P_j receive the same messages in the same order from P_k and they received the same *number* of messages, then they also received the same *set* of messages. So the overall idea is to replace the sets of messages from the inefficient protocol by counters.

More precisely this means the following: instead of maintaining its causal past, each P_i maintains an array $\text{VectorClock}(P_i)$ of integers, where $\text{VectorClock}(P_i)[P_k]$ is the number of messages that have currently been received from P_k . When a message m_i is sent, P_i will append the current state of his vector clock to the message, this is denoted $\text{VectorClock}(P_i, m_i)$, and thus $\text{VectorClock}(P_i, m_i)[P_k]$ is the number of messages P_i has received from P_k at the time it sent m_i . Moreover, instead of remembering all messages that were delivered, each P_i will maintain an array $\text{Delivered}(P_i)$, of the same type as vector clocks, but where $\text{Delivered}(P_i)[P_k]$ contains the number of messages from P_k that were delivered. With this in mind, we can look at Flood2Causal and FIFO2Causal next to each other:

Init

Flood2Causal: Initially let $\text{CausalPast}(P_i) = \emptyset$ for all P_i and let $\text{Delivered}(P_i) = \emptyset$

FIFO2Causal: P_i : Initially let $\text{VectorClock}(P_i) = \mathbf{0}$ and let $\text{Delivered}(P_i) = \mathbf{0}$.

Send

Flood2Causal: On input (P_i, m) on Causal_i , let $\text{CausalPast}(P_i) = \text{CausalPast}(P_i) \cup \{(P_i, m)\}$ and let $\text{CausalPast}(P_i, m) = \text{CausalPast}(P_i)$. Then send $(P_i, m, \text{CausalPast}(P_i, m))$ on Flood_i .

FIFO2Causal: On input (P_i, m) on Causal_i , let $\text{VectorClock}(P_i) = \text{VectorClock}(P_i) + P_i$ and $\text{VectorClock}(P_i, m) = \text{VectorClock}(P_i)$. Then send $(P_i, m, \text{VectorClock}(P_i, m))$ on FIFO_i .

Deliver

Flood2Causal: On receiving $(P_j, m, \text{CausalPast}(P_j, m))$ on Causal_i wait until $\text{CausalPast}(P_j, m) \subseteq \text{Delivered}(P_i) \cup \{(P_j, m)\}$. Then deliver (P_j, m) on Causal_i , and let

$$\text{CausalPast}(P_i) = \text{CausalPast}(P_i) \cup \text{CausalPast}(P_j, m) .$$

Also add (P_j, m) to $\text{Delivered}(P_i)$.

FIFO2Causal: On $(P_j, m, \text{VectorClock}(P_j, m))$ on FIFO_i at P_i , where $\text{VectorClock}(P_j, m) \leq \text{Delivered}(P_i) + P_j$, deliver (P_j, m) on Causal_i and let

$$\text{VectorClock}(P_i) = \max (\text{VectorClock}(P_i), \text{VectorClock}(P_j, m))$$

and $\text{Delivered}(P_i) = \text{Delivered}(P_i) + P_j$.

It should be clear that when FIFO2Causal is run on top of a FIFO protocol, then it does exactly the same things as the inefficient protocol Flood2Causal and therefore is safe and live for exactly the same reasons. For instance, when using FIFO flooding, then the condition $\text{CausalPast}(P_j, m) \subseteq \text{Delivered}(P_i) \cup \{(P_j, m)\}$ is true if and only if $\text{VectorClock}(P_j, m) \leq \text{Delivered}(P_i) + P_j$. In this sense, vector clocks is just an efficient compressing of causal pasts on a FIFO network.

This argues the following result

Lemma 4.5 $\text{FIFO2Causal} + \text{FIFO} = \text{Flood2Causal} + \text{FIFO}$.

It is clear that

$$\text{FIFO} \sqsubseteq \text{Flood} .$$

The reason is that the behaviour of **FIFO** is a subset of the allowed behaviour of **Flood**. So it is safe to use **FIFO** whenever it is safe to use **Flood**. From $\text{FIFO} \sqsubseteq \text{Flood}$ we get that

$$\text{Flood2Causal} + \text{FIFO} \sqsubseteq \text{Flood2Causal} + \text{Flood} .$$

We have already proven that

$$\text{Flood2Causal} + \text{Flood} \sqsubseteq \text{Causal} .$$

Putting these together we have that

$$\text{FIFO2Causal} + \text{FIFO} = \text{Flood2Causal} + \text{FIFO} \sqsubseteq \text{Flood2Causal} + \text{Flood} \sqsubseteq \text{Causal} ,$$

which implies that

$$\text{FIFO2Causal} + \text{FIFO} \sqsubseteq \text{Causal} .$$

We have argued the following result:

Theorem 4.6 $\text{FIFO2Causal} + \text{FIFO} \sqsubseteq \text{Causal}$.

4.4.7 Vector Clocks as Operationalized Causality

Notice that once all messages have been tagged with vector clocks, we can efficiently check if they are in each others causal pasts or whether they are concurrent. It is easy to see that

$$(P_i, m_i) \hookrightarrow (P_j, m_j) \iff \text{VectorClock}(P_i, m_i) \leq \text{VectorClock}(P_j, m_j) .$$

This gives an efficient implementation of causality testing. We simply keep track of the vector clocks, and when we want to know whether $(P_i, m_i) \hookrightarrow (P_j, m_j)$ we simply check whether $\text{VectorClock}(P_i, m_i) \leq \text{VectorClock}(P_j, m_j)$.

When we have two messages (P_i, m_i) and (P_j, m_j) for which

$$\text{VectorClock}(P_i, m_i) \not\leq \text{VectorClock}(P_j, m_j) \text{ and } \text{VectorClock}(P_j, m_j) \not\leq \text{VectorClock}(P_i, m_i) ,$$

we say that the messages are **concurrent**. Neither of these messages have affected each other. When two messages are concurrent we write

$$(P_j, m_j) \parallel (P_i, m_i) .$$

Exercise 4.3 (Causal Past) Recall the series of events we considered in Example 4.2. Recall that the communication in Example 4.2 is not causal. We can, however, still compute the vector clocks of messages. Assume the parties computed vector clocks for all messages using the following algorithm:

Init Party P_i sets $\text{VectorClock}(P_i) = \mathbf{0}$.

Send Whenever inputting some (P_i, m) on FIFO_i , let $\text{VectorClock}(P_i) = \text{VectorClock}(P_i) + P_i$ and $\text{VectorClock}(P_i, m) = \text{VectorClock}(P_i)$.

Receive Whenever receiving output $(P_j, m, \text{VectorClock}(P_j, m))$ on FIFO_i , let $\text{VectorClock}(P_i) = \max(\text{VectorClock}(P_i), \text{VectorClock}(P_j, m))$.

What would the vector clocks of the four messages be? Recall that in Example 4.2 we concluded which messages are causally related and which are concurrent by “following arrows” as in Fig. 4.7. There are 4 messages sent, and hence 6 pairs of messages. For each such pair, use the vector clocks associated to the messages to determine if the messages in the pair are concurrent, or the pair is in the causal past relation. Does it give the same conclusion as when “following arrows”.

Assume parties use the above protocol based on vector clocks to communicate. For each send-event, specify the vector clock that accompanies the message that is sent. There are 4 messages sent, and hence 6 pairs of messages. For each such pair, use the vector clocks associated to the messages to determine if the messages in the pair are concurrent, or the pair is in the causal past relation.

Notice that one can nonetheless compute the vector clocks of the messages. Imagine using the algorithm in Fig. 4.10 but without

Exercise 4.4 (Safety and liveness of Vector Clocks) Above we said the vector-clock protocol needs to run on top of a FIFO flooding network. It turns out, however, that the protocol also works if run on a flooding network without FIFO. Prove that the protocol achieves causal delivery also when run on a network without FIFO. I.e., you are to prove that $\text{FIFO2Causal} + \text{Flood} \sqsubseteq \text{Causal}$. The trick is to prove that FIFO2Causal is itself ensuring FIFO delivery.

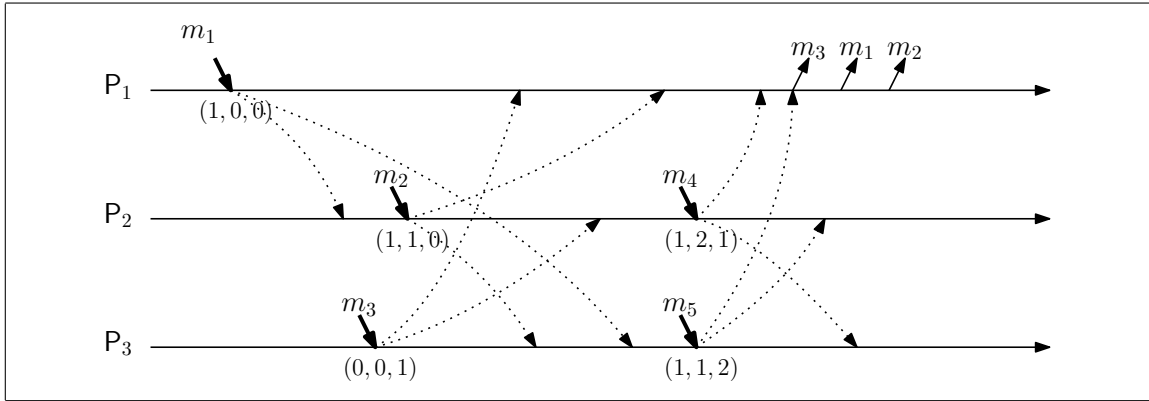


Figure 4.15: An illustration for total order. If we insist on causal order, then $\{m_1, m_2, m_5\}$ should be delivered in the order m_1, m_2, m_5 as $m_1 \hookrightarrow m_2 \hookrightarrow m_5$. And $\{m_2, m_4\}$ should be delivered in the order (m_2, m_4) as $m_2 \hookrightarrow m_4$. Similarly m_3 must be delivered before m_4 and m_5 . Messages m_4 and m_5 are concurrent, and m_3 is concurrent with both m_1 and m_2 . This gives a number of possible causal orders, for instance $(m_1, m_2, m_3, m_4, m_5)$ and $(m_2, m_1, m_3, m_5, m_4)$. But if we insist on total order, all parties have to deliver the messages in the same order. If we use the rule that we sort concurrent message on their vector clocks lexicographically, then m_3 should come first as $(0, 0, 1)$ is lexicographically less than $(1, 0, 0)$. And m_5 should come before m_4 . So the total order should be $(m_3, m_1, m_2, m_5, m_4)$. Recall that a message m is deliverable when you received from all other parties a message with a vector clock larger than that of m . Therefore, when P_1 received m_4 and m_5 the messages m_1, m_2, m_3 become deliverable. They are delivered in the specified order. No other messages are deliverable at the current point in time. Do you see why?

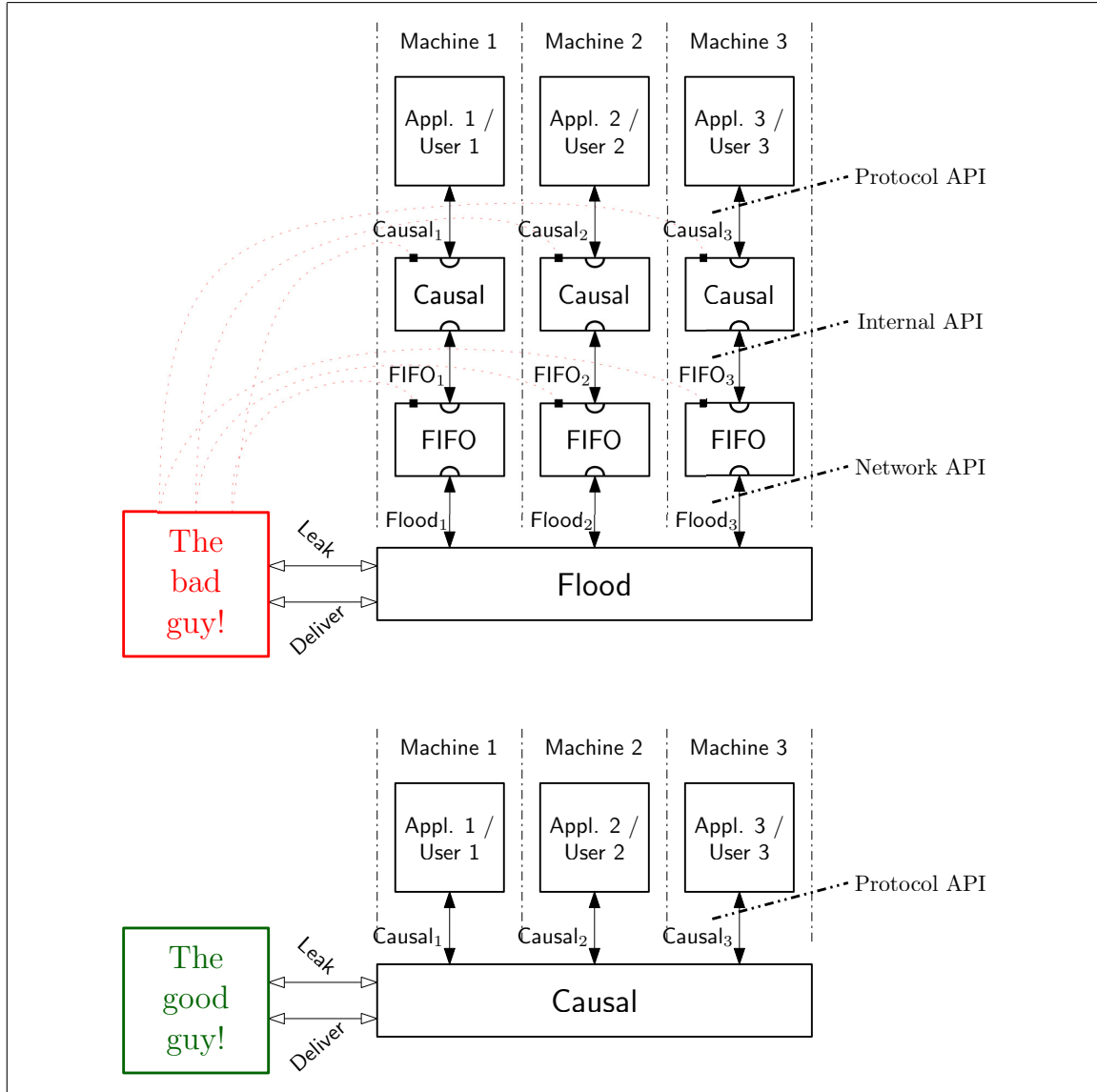


Figure 4.16: Top: the protocol $\text{Flood2FIFO} + \text{FIFO2Causal}$ talking over the network resource Flood being controlled by a bad guy. Bottom: the ideal functionality Causal being controlled by a good guy. Also some users, which use either the one system or the other.

4.5 Intermezzo on Modelling Distributed Systems II†

One reason why using ideal functionalities is gaining more traction over trace properties is that one gets composition for free. We have now proven that $\text{Flood2FIFO} + \text{Flood}$ securely implements FIFO. And we have proven that $\text{FIFO2Causal} + \text{FIFO}$ securely implements Causal. Note that $\text{FIFO2Causal} + \text{FIFO}$ needs FIFO for communication. And we have that $\text{Flood2FIFO} + \text{Flood}$ implements FIFO. It would be natural to expect we can just plug in $\text{Flood2FIFO} + \text{Flood}$ for FIFO in $\text{FIFO2Causal} + \text{FIFO}$ to get a new protocol $\text{FIFO2Causal} + \text{Flood2FIFO} + \text{Flood}$ which still

securely implements **Causal**. We can write this symbolically as

$$\begin{aligned} \text{Flood2FIFO} + \text{Flood} \sqsubseteq \text{FIFO} \quad \wedge \quad \text{FIFO2Causal} + \text{FIFO} \sqsubseteq \text{Causal} \\ \Downarrow \\ \text{FIFO2Causal} + \text{Flood2FIFO} + \text{Flood} \sqsubseteq \text{Causal} . \end{aligned}$$

Note that if we treated the protocols and ideal functionalities as integers and replaced $\sqsubseteq$ by $\geq$, then the above would be true too. Having a notion of security with composition therefore gives a convenient “arithmetic” for security. For the formally inclined reader there are many more details in Chapter 2.

4.6 Total Order

We proceed to study total order.

Total Order If a correct P_k delivered (P_i, m) and then later delivered (P_j, m') , then it holds for all correct P_m that if they deliver (P_j, m') , then they earlier delivered (P_i, m) .

For total order we can think of a question and answers to the question given by two different servers concurrently. Maybe we still need to agree who answered “first”, see Fig. 4.18.

Note that for this setting we need both Causal Order and Total Order. Total order in itself does not imply FIFO or Causal Consistency, but can be combined with them. The reason is that, even if all players deliver received messages in the same order, it could still be that they are delivered in the wrong causal order, say, an answer is delivered before the question. Our solution below will give both.

In Fig. 4.17 we give an ideal functionality TOB which has totally ordered delivery. We will now discuss how to implement TOB given Causal.

When we require total order, we need to allow delayed local delivery, as two messages sent at the same time at different parties need to be delivered in the same order at all parties. So at least one of the parties have to wait with delivering its own message. As an illustration, consider Fig. 4.19.

Below we give an implementation which achieves both Causal Order and Total Order (we called this Causal Total Order above). It does so by first achieving Causal Order and then sorting the concurrent messages.

4.6.1 An Implementation Assuming we can Detect Safe Messages

We already know how to implement Causal Order, so let us start with assuming we have a flooding network Causal with Causal Order. Note then that only concurrent messages can get delivered in different order at different parties, as $(P_i, m_i) \hookrightarrow (P_j, m_j)$ implies that (P_i, m_i) always gets delivered before (P_j, m_j) . Also, we can output the concurrent messages in any order we want without risking to break Causal Order. For the concurrent messages we can then just let all parties sort their concurrent messages using any deterministic total ordering.

As an example, they could sort all messages by lexicographic order of the vector clock, so a vector clock $(0, 0, 1)$ would be smaller than $(1, 0, 0)$. We write this as $\text{VectorClock}(P_i, m_i) < \text{VectorClock}(P_j, m_j)$. For brevity we just write $(P_i, m_i) < (P_j, m_j)$ instead of $\text{VectorClock}(P_i, m_i) < \text{VectorClock}(P_j, m_j)$. Given any causal order $\hookrightarrow$ on messages we can then extend it into a total+causal order $\rightarrow$ as follows. We say that

$$(P_i, m_i) \rightarrow (P_j, m_j)$$

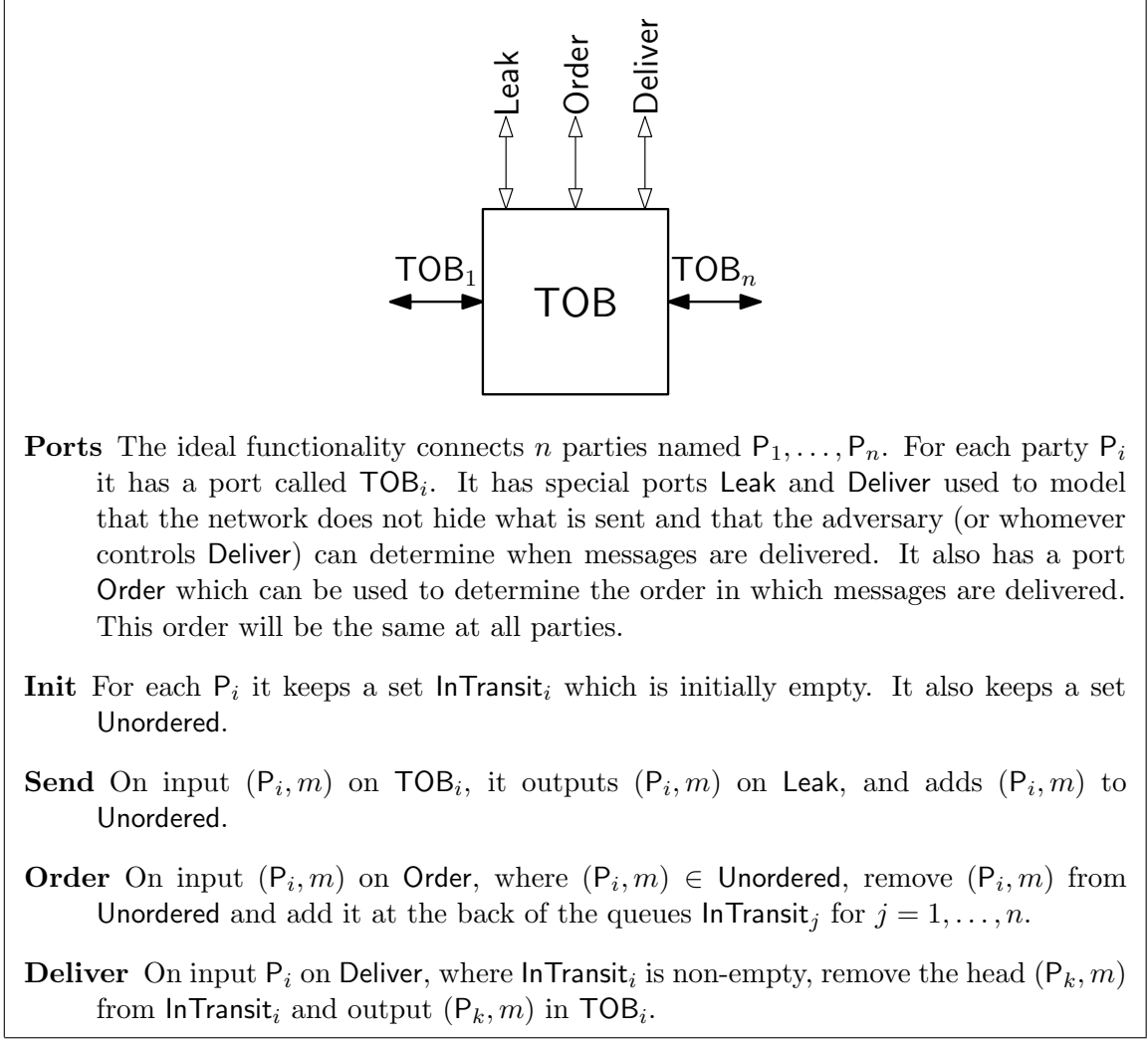


Figure 4.17: TOB Network

if and only if

$$((P_i, m_i) \hookrightarrow (P_j, m_j)) \vee ((P_i, m_i) \parallel (P_j, m_j) \wedge (P_i, m_i) < (P_j, m_j)) .$$

Recall that $(P_i, m_i) \parallel (P_j, m_j)$ means that the messages are concurrent, i.e., $(P_i, m_i) \not\rightarrow (P_j, m_j)$ and $(P_j, m_j) \not\rightarrow (P_i, m_i)$. So, in words we say that $(P_i, m_i) \rightarrow (P_j, m_j)$ if either the messages are causally related or they are concurrent and (P_i, m_i) is lexicographically smaller than (P_j, m_j) .

Our goal is now to construct a protocol which delivers according to $\rightarrow$. There is one big problem to be solved to achieve this. It is hard to know in a system communicating via **Causal** whether more concurrent messages are on their way. For an illustration of this, consider Fig. 4.20. We need to come up with a way such that we for all messages (P_i, m_i) eventually can determine that no message which is concurrent with (P_i, m_i) is still on its way.

We first assume that we can solve this problem and show that then we are done. We are going to assume that we have a procedure **IsSafe** that the parties can run locally. It takes as input a message (P_j, m_j) that was sent via **Causal** and it outputs $\top$ or $\perp$. When it holds that $\text{IsSafe}(P_j, m_j) = \top$ (at P_i), then we call (P_j, m_j) *safe* (at P_i). The procedure **IsSafe** should have a safety and a liveness property:

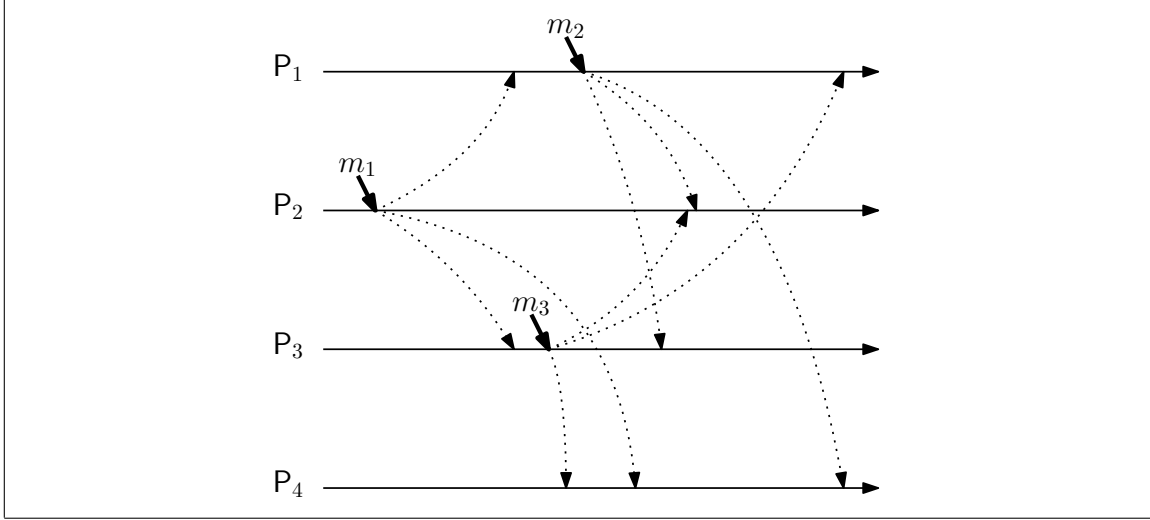


Figure 4.18: For an example why we would like total order think of a question and its answer from two different servers, like $m_1 = \text{WHY DID THE CHICKEN CROSS THE ROAD?}$ and $m_2 = m_3 = \text{TO GET TO THE ORDER SIDE!}$. Maybe there is great prestige in who answered first, so a dispute might lead P_1 and P_3 to resolve the disagreement through kinetic negotiation. We therefore need that P_1, P_2, P_3, P_4 agree on who answered first. The answers being concurrent they could have happened in any order in “physical” time, so we need another convention.

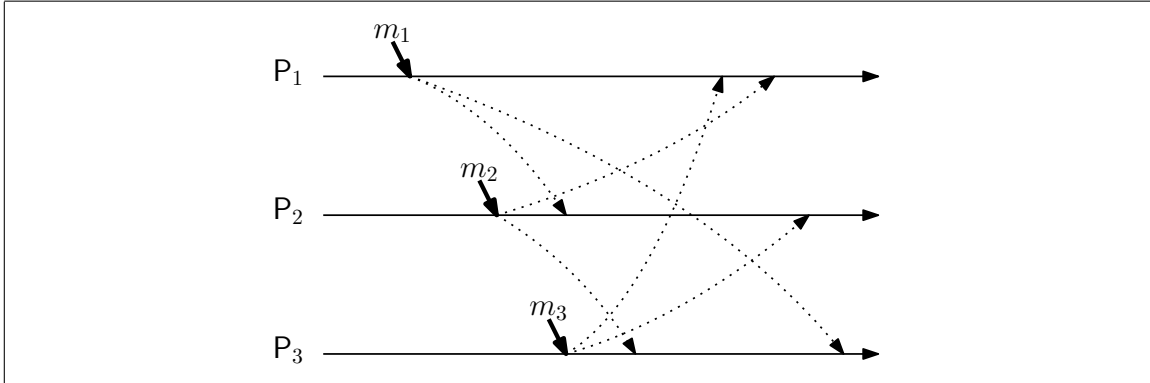


Figure 4.19: Three parties are sending three concurrent messages using **Causal**. They might arrive in different orders at different parties as they are concurrent. Assuming that **Causal** has immediate local delivery, they are delivered by **Causal** in the following ordering: $P_1 : (m_1, m_3, m_2)$; $P_2 : (m_2, m_1, m_3)$; $P_3 : (m_3, m_2, m_1)$. But a protocol implementing **TOB** needs to deliver them in the same order. So two of the parties must wait with delivering their own message on TOB_i until receiving the message that must be output first. If we order concurrent messages lexicographically on their vector clocks, then the correct total order would be (m_3, m_2, m_1) . So P_1 can for instance not deliver m_1 locally before having delivered m_3 .

Safety If at some point in time t_1 it holds that $\text{IsSafe}(P_j, m_j) = \top$ at party P_i and it holds at some later point in time t_2 that P_i receives (P_k, m_k) on Causal_i , then $(P_j, m_j) \hookrightarrow (P_k, m_k)$.

Liveness It holds for all P_i and all messages (P_j, m_j) received at P_i from **Causal** that eventually $\text{IsSafe}(P_j, m_j) = \top$ at party P_i . And when it happens at some point in time that

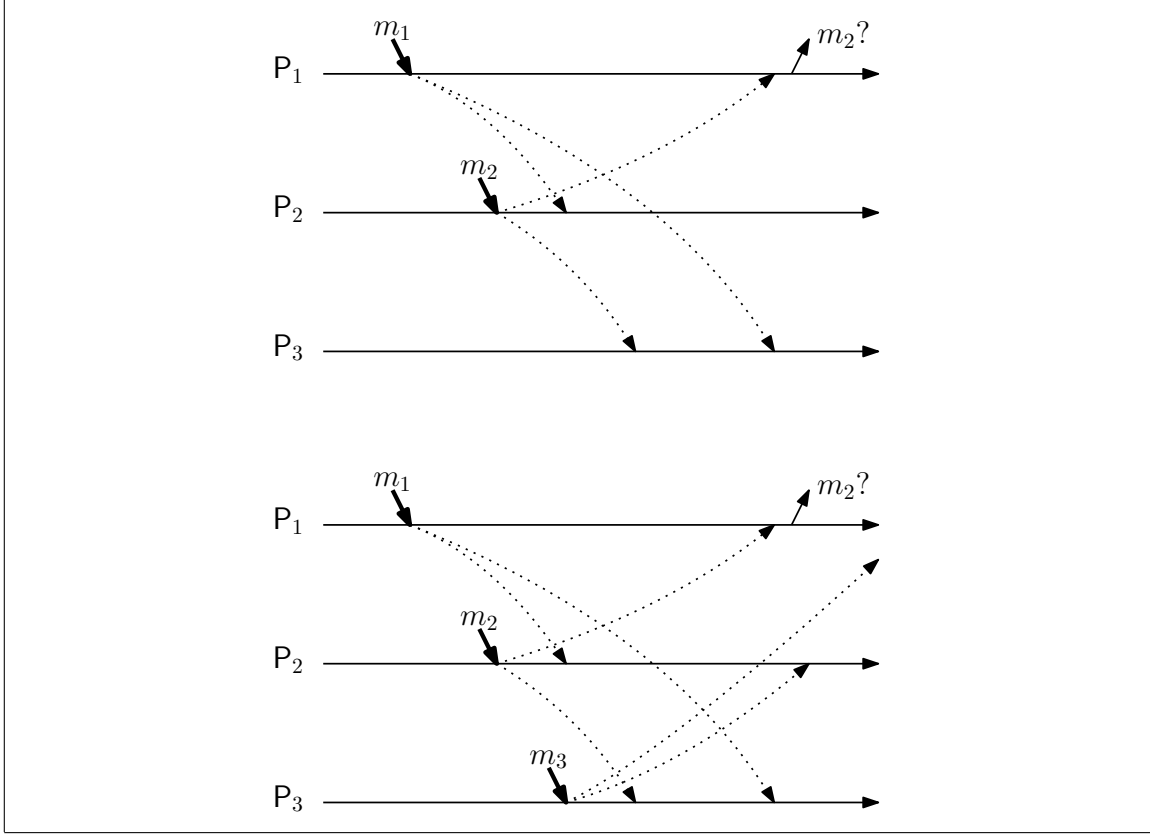


Figure 4.20: Two executions similar to Fig. 4.19, but with P_3 not sending a message in the top execution, and with P_3 's message to P_1 still being in transit in the bottom execution. In the top execution the correct total order is (m_2, m_1) as we sort lexicographically on vector clocks for concurrent messages, and the vector clock of m_1 is $(1, 0, 0)$ and the vector clock of m_2 is $(0, 1, 0)$ and $(0, 1, 0) < (1, 0, 0)$. In the bottom configuration the correct total order is (m_3, m_2, m_1) . Now consider the two executions from the point of view of P_1 . It receives the same messages at the same times. So it cannot know whether it is in the top execution or the bottom execution. Therefore it cannot deliver any message in either setting. In the bottom setting it did not get m_3 yet. And in the top setting it does not know whether there might be an m_3 on its way from P_3 . If no more messages are ever sent in the system, this situation would go on forever. At any point in time it could be that P_3 just sent the message m_3 and that P_1 therefore should wait a bit longer. So the system is in an apparent deadlock. Any causal total order broadcast running on top of a Causal ideal functionality needs to find a way to rule out whether there are still concurrent messages on their way. When we can be sure for a message m that no message concurrent with m can ever arrive, we call m safe. In the above executions there are no safe messages.

$\text{IsSafe}(P_j, m_j) = \top$, then it holds for all future times too.

The liveness property just says that eventually we will consider all messages safe. The safety property says that once (P_j, m_j) is considered safe, then all later messages to arrive will be in the causal future of (P_j, m_j) . This is also the minimal property making IsSafe useful. What we want is that there is no message concurrent to (P_j, m_j) on its way. This is the same as saying that for all later messages it should hold that $(P_j, m_j) \not\parallel (P_k, m_k)$. This is the same as requiring that $(P_j, m_j) \hookrightarrow (P_k, m_k)$ or $(P_k, m_k) \hookrightarrow (P_j, m_j)$. But since (P_k, m_k) arrives later

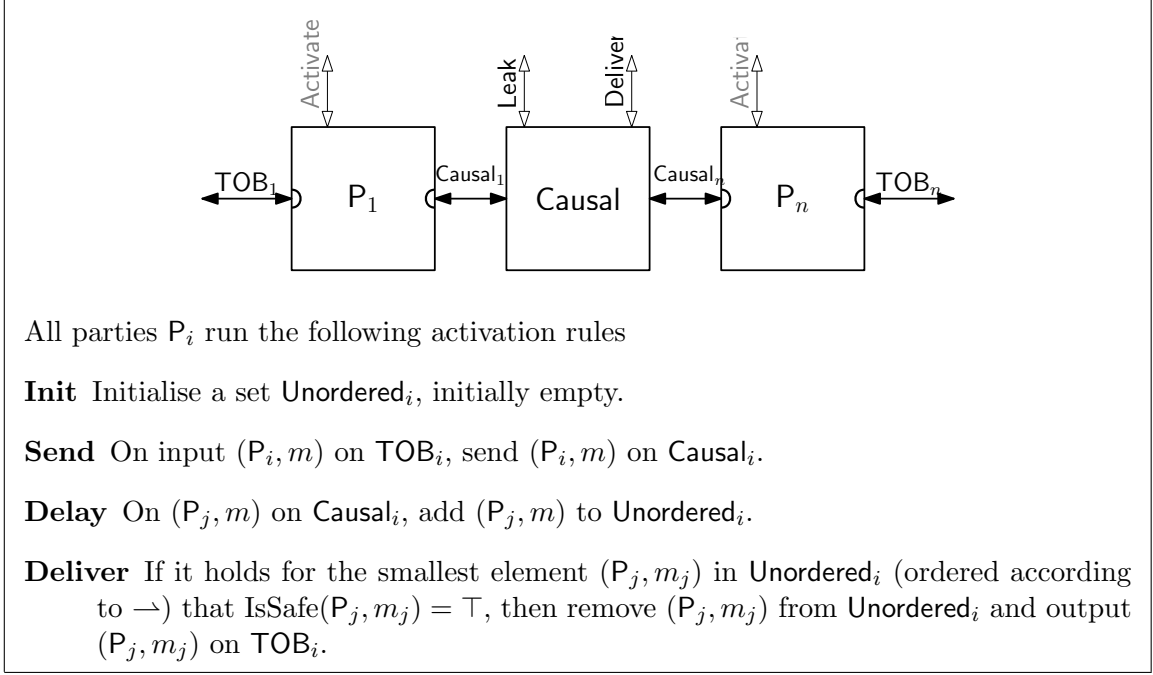


Figure 4.21: The Protocol Causal2TOB.

than (P_j, m_j) it cannot be the case that $(P_k, m_k) \hookrightarrow (P_j, m_j)$, as **Causal** promises to deliver in the order $\hookrightarrow$. So it must be the case that $(P_j, m_j) \hookrightarrow (P_k, m_k)$.

If we have a safe and live procedure IsSafe , then we can implement **TOB** using the protocol **Causal2TOB** in Fig. 4.21.

Theorem 4.7 *Assuming that IsSafe is live and safe we have that $\text{Causal2TOB} + \text{Causal} \sqsubseteq \text{TOB}$.*

PROOF We first argue safety. Assume that some (P_j, m_j) is output on TOB_i by P_i . We want to argue that if $(P_k, m_k) \rightarrow (P_j, m_j)$, then (P_k, m_k) was already output. When (P_j, m_j) was output it was the case that $\text{IsSafe}(P_j, m_j) = \top$ by construction of the protocol. It follows from safety of IsSafe and $(P_k, m_k) \rightarrow (P_j, m_j)$ that when $\text{IsSafe}(P_j, m_j) = \top$ then (P_k, m_k) already arrived at P_i . To see this note that when $(P_k, m_k) \rightarrow (P_j, m_j)$ then $(P_k, m_k) \hookrightarrow (P_j, m_j)$ or $(P_k, m_k) \parallel (P_j, m_j)$. If $(P_k, m_k) \hookrightarrow (P_j, m_j)$, then (P_k, m_k) arrived because we communicate via **Causal**. If $(P_k, m_k) \parallel (P_j, m_j)$, then (P_k, m_k) arrived because $\text{IsSafe}(P_j, m_j) = \top$ and IsSafe is safe. So, we can conclude that (P_k, m_k) arrived. This ensures that (P_k, m_k) is either in Unordered_i or already output. If it was already output we are done, as this is what we should ensure. And it cannot be the case that it is in Unordered_i and at the same time that $(P_k, m_k) \rightarrow (P_j, m_j)$, as we pick (P_j, m_j) to be the smallest element in Unordered_i according to $\rightarrow$. This concludes the proof of safety.

We then argue liveness. We would like to argue that eventually all messages get delivered on TOB_i . At any point in time in an execution consider a message (P_j, m_j) which has been sent. We argue that it eventually gets delivered on TOB_i . At the point in time where it is delivered at all parties on Causal_i at most a finite number of elements were sent. Call the set of these messages X . Note, this is the set of all messages ever sent up to the point where (P_j, m_j) arrived at all parties. Let Y be the set of messages sent on **Causal** which are not in X . Note that all message in Y are in the causal future of (P_j, m_j) . Therefore, if $(P_k, m_k) \rightarrow (P_j, m_j)$, then $(P_k, m_k) \in X$.

For all elements in X it holds that they eventually arrive. Since there are finitely many of them, they will all eventually arrive. And then it will also eventually happen that $\text{IsSafe}(\mathbf{P}_k, m_k) = \top$ for all of them. At this point one of them, call it $(\mathbf{P}_k, m_k)$, will be the smallest message in Unordered_i . Since $\text{IsSafe}(\mathbf{P}_k, m_k) = \top$ it will get delivered. Then the second smallest gets delivered and so on. Since there are finitely many messages smaller than $(\mathbf{P}_j, m_j)$ it will eventually be the turn of $(\mathbf{P}_j, m_j)$ to be delivered. $\square$

4.6.2 Detecting Safety

We now discuss how to implement IsSafe . Recall that we for a given message $(\mathbf{P}_j, m_j)$ need to detect when all messages $(\mathbf{P}_k, m_k)$ which could be concurrent with $(\mathbf{P}_j, m_j)$ have arrived at $\mathbf{P}_i$.

Let $(\mathbf{P}, m)$ be a message received by $\mathbf{P}_i$. We call $(\mathbf{P}, m)$ *safe* (i.e., we let $\text{IsSafe}(\mathbf{P}, m) = \top$) if $\mathbf{P}_i$ received from all $\mathbf{P}_j$ a message $(\mathbf{P}_j, m_j)$ such that $(\mathbf{P}, m) \hookrightarrow (\mathbf{P}_j, m_j)$, i.e., $\mathbf{P}_j$ sent $\mathbf{P}_i$ something in the causal future of $(\mathbf{P}, m)$. In fact, the condition only needs to be true for all $\mathbf{P}_j$ which are not the sender $\mathbf{P}$ or $\mathbf{P}_i$ itself. The reason why we only consider $\mathbf{P}_j \neq \mathbf{P}_i$ is that nothing concurrent to $(\mathbf{P}, m)$ can be on its way to $\mathbf{P}_i$ from $\mathbf{P}_i$ as we assume $\mathbf{P}_i$ immediately delivers to itself. Furthermore, since $(\mathbf{P}, m)$ is in the causal future of itself, $\mathbf{P}$ already sent something in the causal future of $(\mathbf{P}, m)$ to $\mathbf{P}_i$, as $\mathbf{P}_i$ received $(\mathbf{P}, m)$. We therefore just need to receive from all other parties which are not $\mathbf{P}$ or $\mathbf{P}_i$ something in the causal future of $(\mathbf{P}, m)$ to consider it safe. This is illustrated in Fig. 4.22.

We now argue that the above IsSafe is safe. We have to argue that when $\text{IsSafe}(\mathbf{P}, m) = \top$ then no concurrent messages are on their way. It is enough to argue that it holds for all $\mathbf{P}_j$ that there are no concurrent messages on their way from $\mathbf{P}_j$. So assume $\mathbf{P}_i$ received a message $(\mathbf{P}_j, m_j)$ such that $(\mathbf{P}, m) \hookrightarrow (\mathbf{P}_j, m_j)$. We break the set of messages sent by $\mathbf{P}_j$ into two. The set B of messages sent before sending $(\mathbf{P}_j, m_j)$ and the set A of messages sent after sending $(\mathbf{P}_j, m_j)$. For a message $(\mathbf{P}_j, m') \in A$ we know that $(\mathbf{P}_j, m_j) \hookrightarrow (\mathbf{P}_j, m')$. We get from transitivity and $(\mathbf{P}, m) \hookrightarrow (\mathbf{P}_j, m_j)$ that $(\mathbf{P}, m) \hookrightarrow (\mathbf{P}_j, m')$. Therefore $(\mathbf{P}, m) \not\parallel (\mathbf{P}_j, m')$.

For a message $(\mathbf{P}_j, m') \in B$ we know that $(\mathbf{P}_j, m') \hookrightarrow (\mathbf{P}_j, m_j)$. Therefore $\mathbf{P}_i$ received $(\mathbf{P}_j, m')$ before receiving $(\mathbf{P}_j, m_j)$ (we use *Causal* as ideal functionality). Therefore $(\mathbf{P}_j, m')$ is no longer on its way to $\mathbf{P}_i$. Fig. 4.23 illustrates the argument.

So it holds for all messages sent from $\mathbf{P}_j$ that either they are not concurrent to $(\mathbf{P}_j, m)$, or they are already delivered to $\mathbf{P}_i$. In other words, all concurrent messages were already delivered. This is what we had to show.

This method also has liveness if it is guaranteed that each party always sends a message again at some point in time. To ensure thus we can cheat a bit and assume that each party has a clock which is an integer and which is always eventually progressing, i.e., it never gets stuck forever. This will allow the parties to do something “now and then”. By “now and then” we just mean that a party always will do it again, for instance each time the clock progressed one click. Now and then each party $\mathbf{P}_i$ will then send out a message $(\text{FLUSH}, \mathbf{P}_i, f_i)$, where f_i is the clock. This establishes the above situation, where all parties are always sending messages. Hence IsSafe is live and safe.

4.7 Chandy–Lamport Snapshot Algorithm

We present the Chandy–Lamport snapshot algorithm. It exemplifies how causality makes it easier to design and analyze distributed algorithms. It is also an important building block in many distributed algorithms. It allows servers to create a global backup of their state (the snapshot) from which they can restart in case they crash. The setting is a distributed

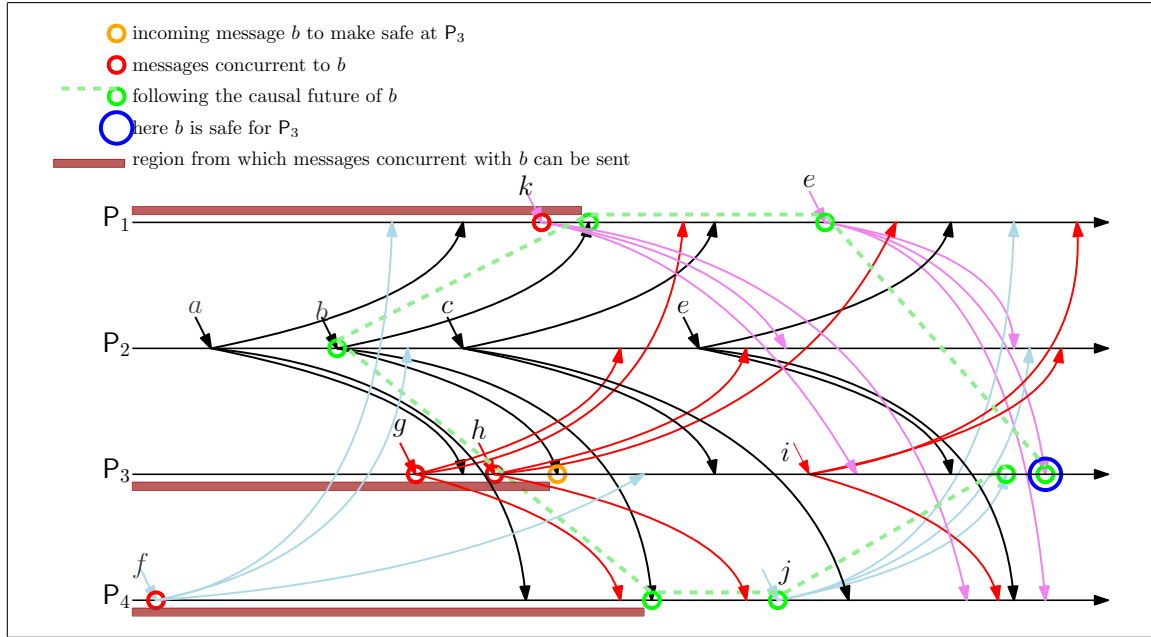


Figure 4.22: P_3 received b from P_2 and wants to be sure that it is safe, i.e., no message concurrent with b is on its way to P_3 . It waits until it has heard from P_1 and P_4 something in the causal future of b . Via P_4 first b arrives at P_4 and after that j is sent and arrives as P_3 . Via P_1 first b arrives at P_1 and after that e is sent and arrives as P_3 . At this point b is safe at P_3 . Note that there are 4 messages in the system concurrent with b , namely k , g , h , and f . Messages g and h were sent by P_3 itself and therefore also delivered at P_3 (as we assume immediate local delivery). Message k must have been delivered at P_3 as it was sent before e at P_1 and e was already delivered at P_3 . And note that any message from P_1 which is concurrent with b must be sent in the brown region before b arrived (or it would be in the causal future of b and hence not concurrent with b), so the argument which shows that k must have arrived at P_3 shows that *any* concurrent message from P_1 arrived too. Similarly, message f must have been delivered at P_3 as it was sent before j at P_1 and e was already delivered at P_3 .

system consisting of some servers which communicate via causal message passing.⁴ Each server takes inputs from a local user, it keeps a local state, and the communication channels can have messages in transit. When a server receives an input or a message from another server it updates its local state based on the message and its current state. Think of dropbox servers getting updates from users and communicating updates to each other. The updates are used to update files and then forgotten. Besides the local files, we also need to record the updates in transit. This is called a **global state** or a **snapshot**. One use of this is to be able to restart the system from a known good state in case of failures. Other uses include distributed garbage collection, distributed deadlock detection, and distributed debugging. We will not discuss these other applications of snapshots. We want the following properties to hold for a snapshot protocol.

Definition 4.8 (Snapshotting (informal))

Local State Recording Each server records a local state.

⁴The algorithm can also be run atop a FIFO order, but we here assume Causal Order to make the analysis easier and to be able to work with the notions of causality.

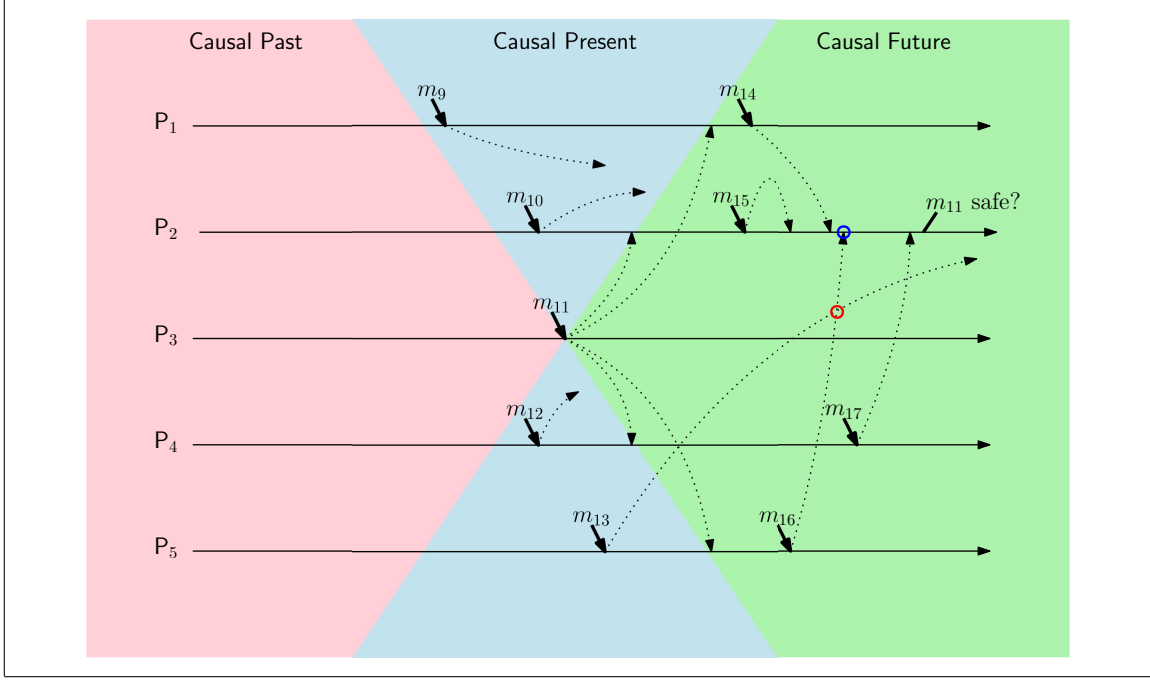


Figure 4.23: We illustrate safety detection via common causal pasts. Consider a message m_{11} sent by P_3 and received by P_2 . All messages are sent via a protocol which already has causal order. The green area is the causal future of m_{11} . We have ignored all messages in the causal past of m_{11} for clarity. Since messages are sent with causal order the causal future of m_{11} starts at the other parties when it arrives, as no message in the causal future of m_{11} can arrive before m_{11} . Consider P_2 which have received from all parties a message sent from the causal future of m_{11} , namely m_{14} from P_1 , m_{15} from P_2 , m_{11} from P_3 , m_{17} from P_4 , and m_{16} from P_5 . The claim is that no message concurrent with m_{11} can be on its way. Concurrent messages are those sent from the causal present of m_{11} , like m_9, m_{10}, m_{12} , and m_{13} . Could one of these still not be received at P_2 ? Let us consider m_{13} for concreteness? By assumption m_{13} is sent from the blue area (the causal present of m_{11}) and, by assumption, m_{16} is sent from the green area (the causal future of m_{11}). And the causal future is after the causal present at any party, so m_{13} is sent before m_{16} at P_5 . Since messages are sent with causal order, m_{13} must therefore be received *before* m_{16} at P_2 . But, by assumption, m_{16} was already received by P_2 , so m_{13} must also have been received. Ergo m_{13} cannot cross the path of m_{16} as on the picture. The same argument holds for all other concurrent messages. Therefore no concurrent message can still be on its way at the mark where we ask if m_{11} is safe.

Channel State Recording Each server records the set of messages in transit on each of its incoming⁵ channels, i.e., from each of the other parties.

Consistency The recorded global states are consistent in the following sense.

Consist States The recorded states could have occurred simultaneously in a real execution of the system. In particular, they are all in the causal present of each other. In other words, they are concurrent. Or in yet other words, no recorded state is in the causal future of another recorded state. To understand the importance of local snapshots being taken at concurrent times, see Fig. 4.24 and the explanation in its

⁵We have chosen the convention that messages in transit are recorded at the receiver side. An alternative convention is to record them at the sender side. Both conventions are used in the literature.

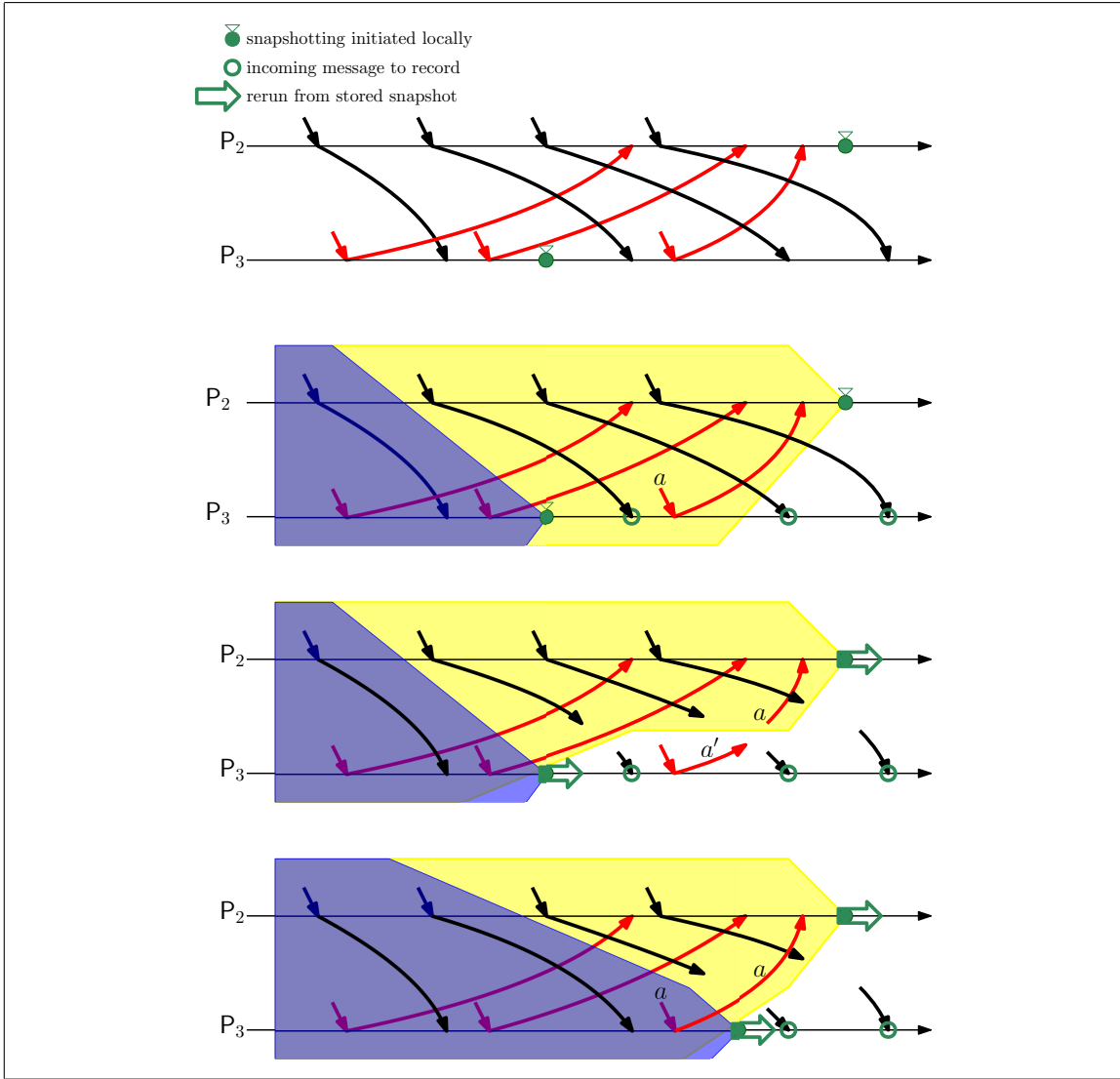


Figure 4.24: Consider two servers having run a protocol. Assume they each record their local states at the non-concurrent green dots in the top timeline diagram, and let us see what goes wrong. The local states of the parties are based on the messages in the causal pasts of the snapshots, illustrated in the second diagram. Party P_3 will also have to record the three messages in transit, as P_2 will not send these again if it resumes from its snapshot. So it keeps them and adds them to its incoming queue when restarting, as illustrated in the third diagram. Notice, however, the message a sent in the causal future of the snapshot of P_3 but in the causal past of the snapshot of P_2 . Party P_2 resumes from a state updated with receiving a . But when P_3 resumes execution, it might receive another input a' . It might well be that a was an input from the local user at P_3 which might now give another input. This leads to an inconsistent execution. The protocol will most likely fail spectacularly. Party P_3 must therefore also remember that a was received as local input, but this is the same as snapshotting later as we just update the state of P_3 to be consistent with having received a , illustrated in the last diagram. Note that now the snapshots are concurrent. Still P_3 has to record the two messages in transit and add them to its incoming queue when restarting.

caption.

Consist Messages All messages recorded as sent by some recorded state are either recorded as received by another recorded state, or are recorded as in transit. No message is recorded as received if it was not recorded as sent. See the third diagram in Fig. 4.24 to see what goes wrong if a messages is marked as received but not as sent.

No Halting The snapshot protocol does not interfere with the normal operation of the system and generates minimal extra communication. In particular, the system continues to operate while the snapshot is being recorded.

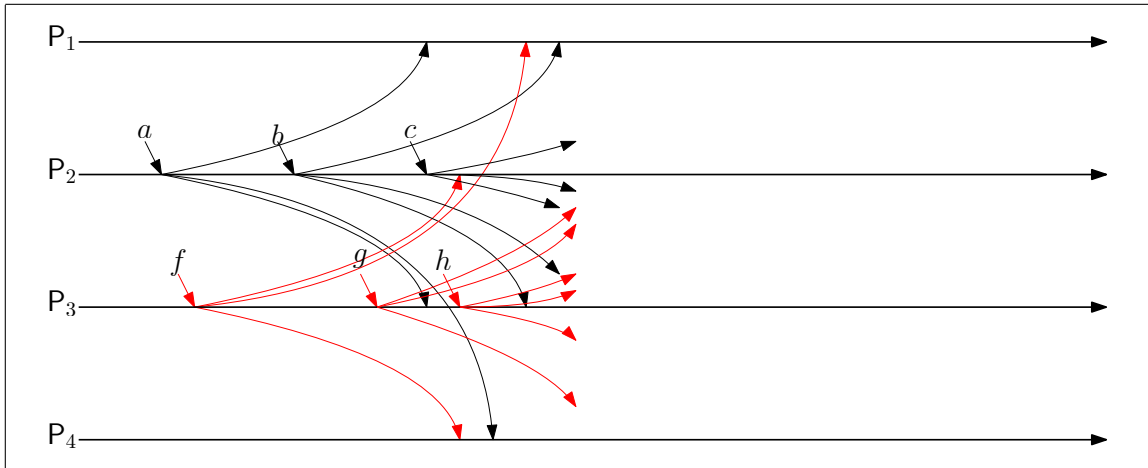


Figure 4.25: Four servers have sent six messages of which some have been received and some are in transit.

We now sketch the algorithm by Chandy and Lamport which achieves the above properties. The algorithm is illustrated in Fig. 4.25–Fig. 4.28.

Local Initiation of Snapshotting Any server can locally initiate snapshotting by sending out a special SNAPSHOT message to all other servers. We call the point in (causal time) where it does this the **snapshot point** of that server. The snapshot point marks the point in time where the local state is recorded. Local initiation can only be done if the server did not see another server initiate snapshotting earlier. This ensures that all locally initiated snapshot points are concurrent. See Fig. 4.26.

Reactive Initiation of Snapshotting When a server receives a SNAPSHOT message for the first time it immediately records its local state and sends out SNAPSHOT messages to all other servers, if it did not already do a local initiation of snapshotting. It employs a small hack to ensure that the snapshot point is before receiving the SNAPSHOT message. It temporarily buffers the SNAPSHOT message until after it has recorded its local state and sent out its own SNAPSHOT messages. This ensures that also all reactive snapshot points are concurrent with all other snapshot points. See Fig. 4.27.

Wait for Common Causal Future Each servers then waits to see a SNAPSHOT message from all other servers. See Fig. 4.28.

Compute Messages in Transit When a server has seen a SNAPSHOT message from all other servers, it knows that all messages which were sent before the snapshot point of another server have been received. This is just because it already saw those snapshot messages

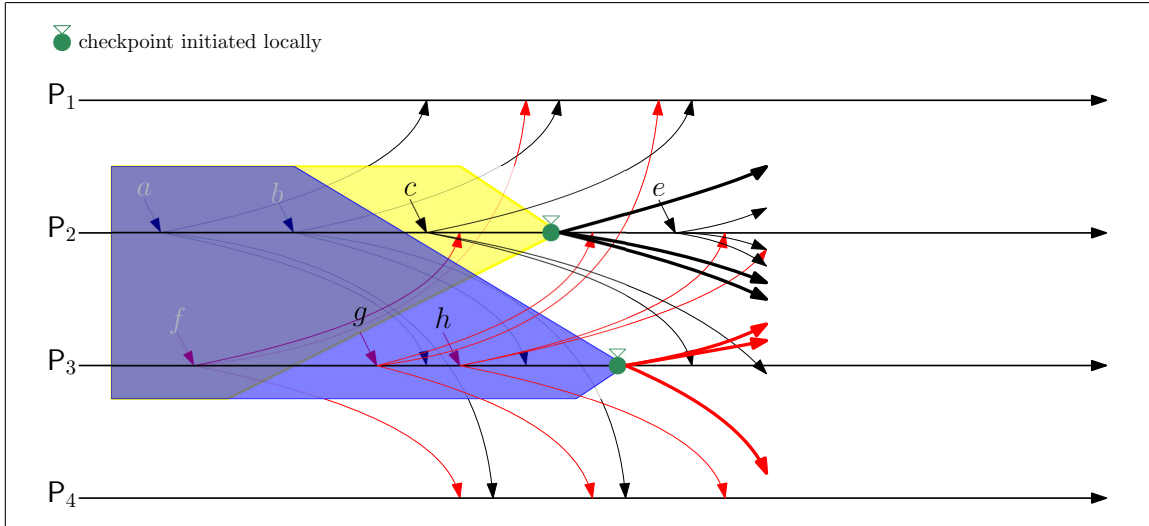


Figure 4.26: Servers P_2 and P_3 concurrently initiate snapshotting. They send out messages informing the other servers. The causal past regions show that the snapshotting points are concurrent: neither is included in the other snapshot point's causal past. The protocol does not stop during snapshotting: server P_2 sends a new message e .

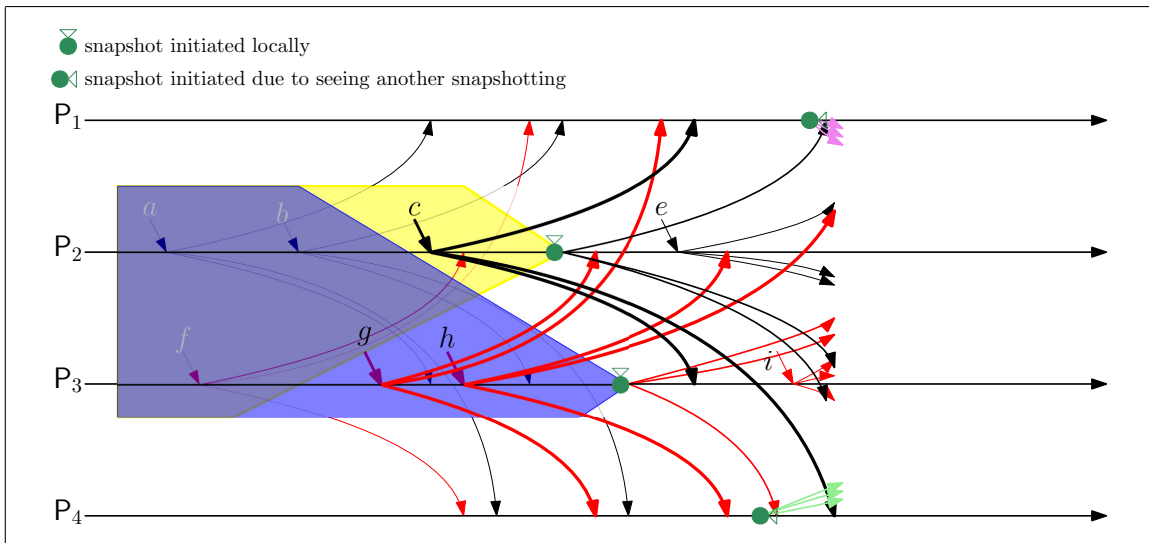


Figure 4.27: When server P_4 learns that P_3 started a snapshot, it quickly initiates its own snapshot just before *receiving* the message that P_3 is snapshotting. It just holds this message from P_3 in a temporary buffer, starts its own snapshot, and then receives the message from P_3 as usual. This ensures that the snapshot points are concurrent. It sends out messages telling the other servers about its own snapshot. Similarly when P_1 learns that P_2 started a snapshot, it quickly initiates its own snapshot just before *receiving* the message and tells the other servers about its own snapshot.

and communication is causal, so any message sent before the snapshot point of another server must have been received too. It can therefore now record the set of messages in transit on each of its incoming channels as those messages received after its own snapshot

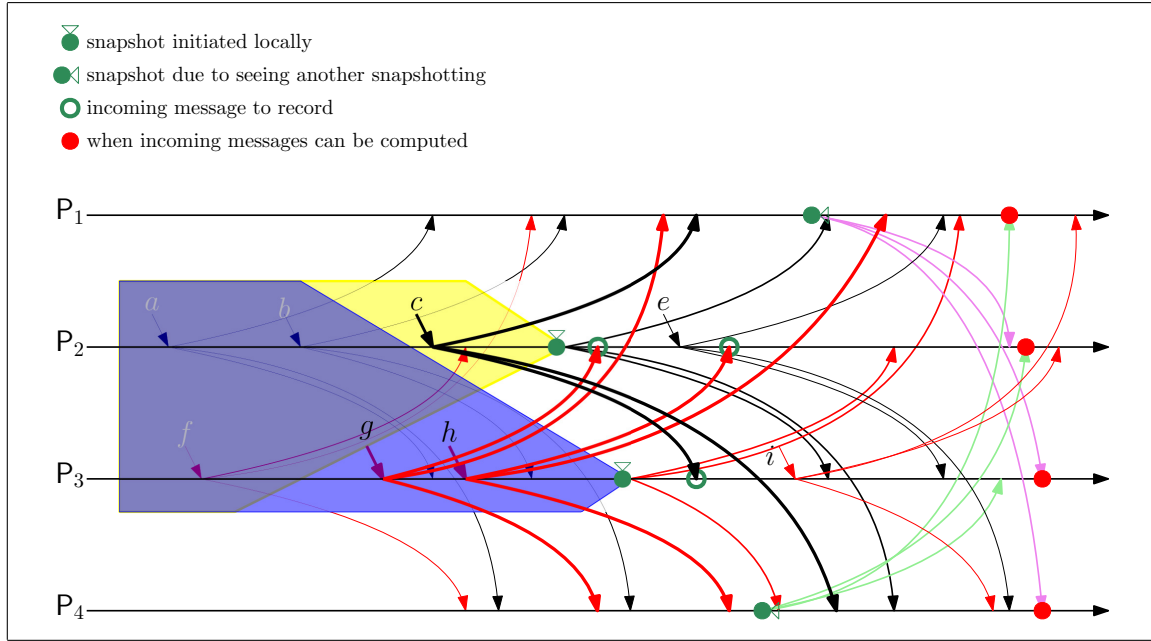


Figure 4.28: After P_2 started a snapshot and learned that P_3 started a snapshot, learned that P_4 started a snapshot, and that P_1 started a snapshot (in that order), it had enough information to record the incoming messages in transit. This point is marked by a red dot. It now knows that no messages sent in the causal past of another server's snapshot point and concurrent with its own snapshot point are in transit. It therefore knows that g and h were the only messages in transit from the causal past of a snapshot point. It records these along with its local state at its snapshot point. Similarly, after P_3 learned that P_3, P_2, P_4, P_1 started a snapshot (in that order), it knows that c was the only message in transit from the causal past of a snapshot point. The messages to be stored by P_1 and P_4 are not marked, as computing them has been left as an exercise.

point but before receiving the SNAPSHOT message from the corresponding sender. A bit more formally, for each server P_j , let snap_j be the SNAPSHOT message sent by P_j . Each server P_i will record the set of messages in transit to P_i as those messages in the set

$$\text{Incoming}_i := \left(\bigcup_{j=1}^n \text{CausalPast}(\text{snap}_j) \right) \setminus \text{CausalPast}(\text{snap}_i) .$$

Namely, $\bigcup_{j=1}^n \text{CausalPast}(\text{snap}_j)$ is the set of all messages ever sent before the snapshot points of all servers. All of these could be in transit to P_i . However, $\text{CausalPast}(\text{snap}_i)$ is the set of messages already included in the local state of P_i at its snapshot point. Therefore, only the messages in the difference need to be recorded as in transit. See Fig. 4.28.

The above protocol was phrased abstractly using Causal Past. In an implementation we would use vector clocks. See Exercise 4.12 for the details.

EINSTEIN: Hey, Hume, wake up! Fascinating how this idea of vector clocks captures how “time” is not a thing in itself, but rather a relation between events. Isn't that your idea?

HUME: Well, kind of, but I never ventured into this business that things cannot happen at the same “time”. I think I implicitly assumed there were a global sequence of phenomena, of which each of us observes only a subsequence.

EINSTEIN: Like Total Order?

HUME: Yes, except parties only see a sub-sequence of the global total order. But you did pick up on my idea, didn’t you, and argued that time at different locations cannot be related except via light signals. And then you observed that for moving observers there is in fact no global simultaneity. If one event could not have been reached by a light signal from another, then the two may be seen in different orders by different observers. Nice catch!

EINSTEIN: Well, kind of. But I am not sure I picked that up from you. I read Kant, and he seemed to think that the time we experience behaves much as you describe—mere succession of events, since that is the only way we can think about the world. Yet, he adds explicitly that two events either are simultaneous or are not, as a matter of objective experience, and that this does not depend on any observer. That just doesn’t survive once you take finite signal speed seriously!

HUME: Indeed, Kant took my observations seriously, though he felt compelled to secure them with principles going somewhat beyond experience. I confined myself to what succession alone presents to us, and left questions of simultaneity there.

EINSTEIN: Anyway, that is not why I woke you up. I have questions about this Chandy-Lamport Snapshot Algorithm. There is a great deal of trouble taken to ensure that the right messages in transit are captured. Why is that? Why not simply wait until there are no messages in transit?

HUME: Because in an asynchronous system one can never be sure that there are no messages in transit. There may always be a message on its way. In practice, systems communicate almost continuously: user inputs, periodic heartbeats, reconfiguration of which files reside on which servers, and so on. Waiting for the system to become quiet is therefore not a viable option.

EINSTEIN: All right, but why not just stop the system for a while in order to take the snapshot?

HUME: Like having Dropbox suddenly pause for a while? That would hardly recommend itself to the users. The market, I fear, shows little patience for hanging system. Moreover, stopping a distributed system is no simple matter. One would have to coordinate all parties to stop at the same time, and then ensure that they all restart correctly. That is complicated and error-prone.

EINSTEIN: Fair enough. But then why record the incoming messages as a separate thing? Could one not simply run the recorded state of the server on those messages, producing a new state, and then store only that state?

HUME: As you run the server on the incoming messages, the server would most likely generate new *outgoing* messages as well. You would then be storing a state together with a set of outgoing messages. The difficulty appears much the same, at least in matters of implementation.

EINSTEIN: Ha! That just gave me a good idea! Why not record the state at the snapshot point together with the set of outgoing messages that have not yet been received by the other servers at their snapshot points? Like in Fig. 4.27, have P_3 store g and h instead of having P_2 do it? The sender already *has* these messages, so it is preferable to have the sender record these rather than have the receiver wait for incoming messages still in transit, no?

HUME: That could work as well, I suppose. I am not confident that it gains us much. One would then have to resend those messages upon restart. And how would one know which messages had not yet been received by the other servers at their snapshot points? That too would require careful bookkeeping. Whether this is simpler than tracking incoming messages

is not immediately clear to me. I'll go back to being a dead philosopher now. Good night, Einstein.

If you want to help Einstein with his idea then solve Exercise 4.13.

4.8 More Exercises

Exercise 4.5 (Lamport Clocks) This exercise explores Lamport clocks as a way to achieve total order different from the method described earlier. We assume a reliable FIFO network: messages can be arbitrarily delayed, but messages from the same party are received in send order. We also assume we do not worry about liveness here, and we assume that there is always a next message which will eventually arrive and flush the queue.

Lamport clocks (LC) are more compact than vector clocks: each consists of a single integer that is sent along with every message. The algorithm for maintaining and using Lamport clocks is as follows:

1. Each party P_i holds a counter c_i , initially set to 0.
2. When P_i sends a message m , it increments $c_i = c_i + 1$, sets $LC(P_i, m) = c_i$, and inputs the tuple $(P_i, m, LC(P_i, m))$ to the FIFO network.
3. When a message $(P_j, m, LC(P_j, m))$ arrives from the FIFO network at P_i , it is inserted into InTransit_i . (Note: A party also receives its own messages via the FIFO network, and must treat them like any other message.)
4. A message $(P_j, m, LC(P_j, m))$ in InTransit_i is deliverable if:
 - (a) it has the smallest Lamport clock among all messages in InTransit_i , breaking ties by party identifier, and
 - (b) for every party $P_k \notin \{P_i, P_j\}$, there is some message from P_k in InTransit_i with Lamport clock strictly larger than $LC(P_j, m)$.
5. When P_i delivers $(P_j, m, LC(P_j, m))$, it updates its local counter as

$$c_i = \max(c_i, LC(P_j, m)) + 1.$$

Consider the following scenarios; determine the delivery order at all parties:

- Two parties send messages periodically without network delays: P_1 sends a message at times $1 + 2t$, and P_2 at times $2 + 2t$, for $t = 0, 1, 2, \dots$. Messages are received in less than one unit of time (so each party receives a message from the other before sending their own).
- Like above but with three parties: P_1 sends at times $1 + 3t$, P_2 at $2 + 3t$, and P_3 at $3 + 3t$, for $t = 0, 1, 2, \dots$. Messages from the other parties are still received quickly, like in the example above, but P_3 always receives messages from P_2 before those from P_1 , even if they were sent earlier.
- Two parties send messages concurrently at each time step: P_1 and P_2 both send a message at times $t = 1, 2, 3, \dots$. Messages are received less than one second after being sent.
- Like above but with three parties: P_1 , P_2 , and P_3 each send a message at every time $t = 1, 2, 3, \dots$. Messages are received less than one second after being sent. Additionally, P_3 always receives messages from P_2 before those from P_1 , even if sent earlier.

- Can you generalize from the examples above whether Lamport clocks always yield a total order of delivered messages?



SOCRATES: Psst, Plato, attention. Now I'm confused. The above idea with Lamport clocks seem much simpler than going via Causal Order. So why not just do it as above?

PLATO: Maybe it does not give Causal Order?

SOCRATES: I'm sure it does. A receiver updates its local clock to be larger than that of the message m it received, so the local time becomes larger than any clock of a message which could have influenced m .

PLATO: That true. Hmm, maybe it is because Lamport clocks does not give precise information about concurrency. All messages have a clock which is just an integer, so it can easily happen that two messages m and m' , which are concurrent has clocks like 1 and 2 such that it looks like m' is after m .

SOCRATES: How?

PLATO: Like party 1 sends m , giving it clock 1. Concurrently party 2 sends a and then m' giving them clocks 1 and then 2. Since party 2 did not see m its message m' is concurrent with m . But this cannot be seen from the clocks.

SOCRATES: How would that matter?

PLATO: Maybe some parts of the application are fine with processing messages in causal order, while others need total order. By going via vector clocks you can get both. The more relaxed part of the application can process messages as they come in. The more strict part can then wait until it knows all concurrent messages arrived and process them in some global order.

SOCRATES: Again, why would that matter?

PLATO: Well looking at Fig. 4.22 the relaxed part of an application could process the message b at P_3 when it arrives in the orange circle. The strict part would have to wait until the blue circle. So you do not have to unnecessarily slow down the parts of your application which does not rely on total order.

SOCRATES: Does that ever matter in practice?

PLATO: Well, consider a system with files where parties can acquire a lock on a file and then afterwards can edit it exclusively. Now edits on different files can safely be applied in causal order, as only one party is editing a given file and hence there cannot be concurrent edits on a single file. But to acquire the locks it is important who acquired the lock first, so total order is required.

SOCRATES: So it would be slow to get a lock, but once you have it, editing the file is fast?

PLATO: Yup!

SOCRATES: Makes sense.

Exercise 4.6 (Peer-to-Peer Flooding and a Simple Ledger) *This exercise is part of a series of practical exercises (Exercise 3.4, Exercise 4.6, Exercise 6.16, Exercise 16.2) in which you gradually build a peer-to-peer transaction ledger (sometimes referred to as a “Blockchain”). It may be useful to skim the later exercises in the series before starting, to understand which components will be extended and reused.*

Goal. Implement a simple distributed ledger on top of a peer-to-peer flooding network. Each peer maintains a local copy of the ledger. Transactions are flooded through the network and applied locally. The system should achieve **eventual consistency**: if no new transactions are issued, then eventually all peers hold the same ledger.

```
package account

import ( "sync" )

type Ledger struct {
    Accounts map[string]int
    lock sync.Mutex
}

func MakeLedger() *Ledger {
    ledger := new(Ledger)
    ledger.Accounts = make(map[string]int)
    return ledger
}
```

Figure 4.29: The Ledger type.

```
package account

type Transaction struct {
    ID string
    From string
    To string
    Amount int
}

func (l *Ledger) Transaction(t *Transaction) {
    l.lock.Lock() ; defer l.lock.Unlock()

    l.Accounts[t.From] -= t.Amount
    l.Accounts[t.To] += t.Amount
}
```

Figure 4.30: The Transaction type.

Peers vs. users. It is important to distinguish between:

- *Peers*: the programs you run (processes) that form the peer-to-peer network and execute the protocol. Each peer runs the same code and maintains a local copy of the ledger.
- *Users*: logical entities that own accounts in the ledger. Users are identified only by account names (strings) and are not network participants.

To reiterate, you can think of peers as banks and users as bank customers. The banks need to agree on the balance of all its customers, and for a customer to transfer money, they go to their local bank, which handles the transfer

A peer may create transactions on behalf of users, but users themselves do not send network messages. Network communication happens *only* between peers.

Background. You already implemented a **Peer** that can:

- listen for TCP connections,
- connect to other peers,
- send and receive typed messages.

In this exercise, you extend this peer with:

- a dynamically growing set of peers,
- flooding of messages,
- a simple application-layer protocol (a ledger).

Ledger and transactions. The ledger is a map from account names to balances.

A transaction consists of:

- **TxID** (string): a unique transaction identifier,
- **FromAccount** (string): the account to debit,
- **ToAccount** (string): the account to credit,
- **Amount** (integer).

Executing a transaction subtracts **Amount** from **FromAccount** and adds it to **ToAccount**. (The fields **FromAccount** and **ToAccount** refer to ledger accounts and are unrelated to which peer sent or received the network message carrying the transaction.)

Part 1: Forming a Peer-to-Peer Network. Extend your **Peer** with support for joining a network.

1. When a peer starts, it is given an IP address and port of an existing peer. If the connection fails, it starts a new network consisting only of itself.
2. Each peer maintains a local set of known peers (addresses).
3. When a peer connects to an existing peer, it requests the existing peer's set of known peers.

4. The joining peer adds itself to its own peer set and floods a *Join Message* announcing its presence.
5. When a peer receives a Join Message for a previously unknown peer, it adds that peer to its peer set.

We recommend that you maintain a fully connected network, meaning that each peer keeps an open TCP connection to every peer in its peer set. This keeps the flooding logic simple and is sufficient for the scale of this exercise.

If you choose a different network topology (for instance, keeping connections only to a subset of peers), you must ensure that your flooding mechanism still works correctly and argue this in your report.

Part 2: Flooding Messages. Add a method

```
Peer.FloodMessage(msg *Message)
```

that disseminates messages to all peers. You may choose between two designs:

Simple flooding. When flooding a message, send it to all peers in your peer set.

Deduplicating flooding (optional, more advanced). Implement flooding with duplicate suppression as follows:

1. Each message has a unique identifier `MsgID`. Each peer keeps a set of message identifiers it has already *flooded*.
2. To flood a message m , a peer sends m to all peers in its peer set *except* those peers from which it already received m .
3. When a peer receives a message m from another peer, it floods m unless it already flooded m .
4. When a peer R joins by connecting to an existing peer S , then S also sends to R all *previous* messages that S has seen. This way R should end up having seen the same messages as the other peers even if joining after the first messages were sent.

Clearly state in your report which approach you implemented.

Part 3: Flooding Transactions and Maintaining the Ledger. Define a message type for transactions. Add a method

```
Peer.FloodTransaction(tx *Transaction)
```

that floods a transaction using `FloodMessage`.

When a peer receives a transaction:

- it delivers the transaction locally,
- it updates its local ledger by executing the transaction.

In this exercise, transactions are always executed when delivered, even if this causes an account balance to become negative. Handling or rejecting overdrafts is intentionally deferred to later exercises.

Hand-in and Demo. Your hand-in must include:

1. The source code of your system.
2. A short PDF report describing: the design of your system; how your TA can run the system, including exact commands; how you tested the system; your answers to the discussion questions below.
3. A program `handin.go` that:
 - (a) starts $n = 10$ peers on the same machine (use fewer if needed),
 - (b) connects them into a single network,
 - (c) lets each peer send $\tau = 10$ transactions involving the same 5 accounts,
 - (d) sends transactions concurrently from all peers,
 - (e) waits until all transactions are delivered,
 - (f) checks that all peers hold identical ledgers.
 - (g) Prints only relevant information, such that a TA can observe the required behavior in real time without excessive or missing output.

Testing. You must test your system and describe how you tested it. You are strongly encouraged to use automatic tests. For instance, tests may check that:

- all peers eventually learn the same peer set,
- all flooded transactions are delivered exactly once (or at least once, depending on your design),
- all ledgers converge to the same state.

Discussion questions (report). Answer the following in your report:

1. If you implemented simple flooding: describe one execution (peer join order and timing of transactions) where eventual consistency is achieved, and one where it is not.
2. If you implemented deduplicating flooding: argue why your system achieves eventual consistency, or describe an execution (peer join order and timing of transactions) where it fails.
3. Assume transactions that would make an account negative are rejected upon delivery. Does your system still achieve eventual consistency? Explain why or why not.



Exercise 4.7 (better total-order) Look at Fig. 4.15 again. Neither P_2 nor P_3 can deliver m_3 . This sort of makes sense. If P_1 had sent a new message right after sending m_1 it would have vector clock $(2, 0, 0)$ and be concurrent with m_3 . So depending on how we sort concurrent messages this message might have to be delivered before m_3 . However, we have decided to use a specific way to order concurrent messages, namely the lexicographic order with the sender identity being the most significant “digit” and P_1 being the first to be considered. For this exercise we change the convention and say that higher numbered parties are considered first. So if P_1 and P_3 sends messages concurrently with vector clocks $(1, 0, 0)$ and $(0, 0, 1)$, then we deliver $(0, 0, 1)$. With this knowledge we can do better. So, a message with vector clock $(2, 0, 0)$

would come after m_3 . In fact, it is easy to see that with the way we sort, the vector clock $(0, 0, 1)$ of m_3 is the smallest possible vector clock of any message according to $\rightarrow$, so all parties could deliver m_3 immediately when receiving it! This proposes the following more efficiently rule for delivering a message: A message m becomes **deliverable** when it follows from the vector clocks of the messages seen that no message with a smaller vector clock according to $\rightarrow$ could be on its way to you. Deliver all deliverable messages in the order $\rightarrow$. With this new and better rule, when would the various messages get delivered in Fig. 4.15, and which messages still cannot be delivered. ▼

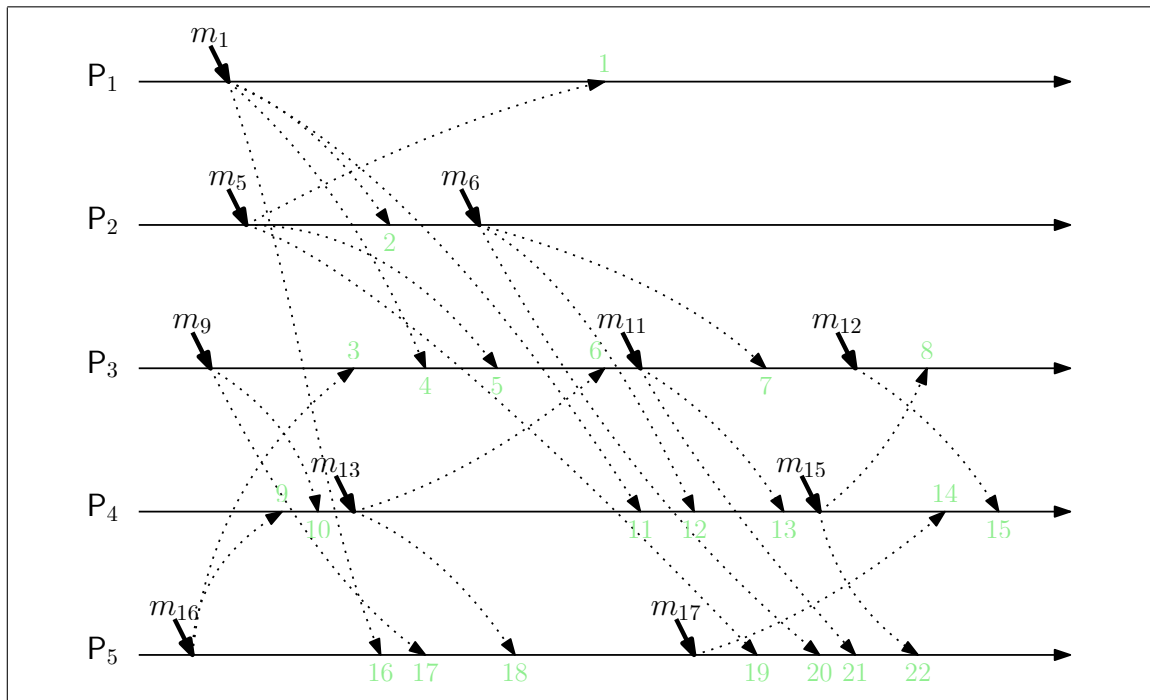


Figure 4.31: An example of causal communication (with one mistake). When a dotted lines arrives at a timeline consider it a receive event in causal communication. Remember that when a message is received then all messages in its causal past must already have been received. Can you find one missing delivery to add to make the whole pattern safe according to Causal Order? Once fixed, for the 22 other receive events, which one can individually be removed without violating Causal Order?

Exercise 4.8 (Causal Regions) In Fig. 4.7 compute the causal past, causal present and causal future of m_{10} . Do the same for m_2 . ▼

Exercise 4.9 (Causal Present I) Is the following statement true: If m is in the causal present of m' , then m' is in the causal present of m . If you claim it is true, prove it. If you claim it is false, give a counter example. ▼

Exercise 4.10 (Causal Present II) Is the following statement true: If m is in the causal present of m' and m' is in the causal present of m'' , then m is in the causal present of m'' . If you claim it is true, prove it. If you claim it is false, give a counter example. ▼

Exercise 4.11 (Causal Order) See Fig. 4.31. The communication is not safe according to Causal Order. However, by adding one more arrow to the communication pattern it can be made safe. Where should it be? Draw an arrow from the message to be delivered to where and when it arrives. Deliver it at the receiver as early as possible to make the answer unique. After thus making the communication safe, for each of the 22 other receive events, determine which ones can individually be deleted without violating Causal Order? If you claim a delivery of m cannot be removed, give an example why by specifying a delivery m' which is no longer safe according to Causal Order after the delivery of m was deleted. Note that m' must be received at the party P where the delivery of m was deleted. And it must be received after the point at which m was delivered before deleted. Pick the first m' at P to violate Causal Order to make the answer unique. ▼

Exercise 4.12 (Implementing Chandy-Lamport Snapshotting) In describing the Chandy-Lamport snapshotting algorithm we said that the messages in transit to P_i are the messages in the set

$$\left(\bigcup_{j=1}^n \text{CausalPast}(\text{snap}_j) \right) \setminus \text{CausalPast}(\text{snap}_i) . \quad (4.4)$$

In an implementation, one would use vector clocks to determine which messages are in the causal past of the various snapshot messages. Let us get closer to an implementation by rephrasing the condition using vector clocks. Let

$$\text{vc}_i := \text{VectorClock}(\text{snap}_i)$$

and let

$$\text{vc}_{\text{ALLSENT}} := \max_{j=1}^n \text{vc}_j .$$

For each message m received at P_i , let $\text{vc}_m := \text{VectorClock}(m)$. Show that m is to be stored according to Eq. 4.4 if and only if

$$\text{vc}_m \leq \text{vc}_{\text{ALLSENT}} \wedge \text{vc}_m \not\leq \text{vc}_i .$$

▼

We first write

$$m \in \bigcup_{j=1}^n \text{CausalPast}(\text{snap}_j)$$

via vector clocks. This is the same as

$$\bigvee_{j=1}^n m \in \text{CausalPast}(\text{snap}_j) .$$

This is the same as

$$\bigvee_{j=1}^n \text{vc}_m \leq \text{vc}_j .$$

We already argued that this is the same as

$$\text{vc}_m \leq \max_{j=1}^n \text{vc}_j .$$

Exercise 4.13 (Helping Einstein) Help Einstein analyze his idea of snapshotting by recording the local state together with the set of *outgoing* messages that have not yet been received by the other servers at their snapshot points. Thus, in Fig. 4.27, messages g and h would be recorded by P_3 as outgoing messages not yet received by P_2 , and so on.

Fill in the details of how this would work. When can the servers record the snapshot? How do they know which messages were not yet received by the other servers at their snapshot points? Argue that the recorded global states of your solution are consistent in the sense of *Consist States* and *Consist Messages* above (except that we now record outgoing messages).

Compare the new protocol to the original one recording incoming messages. Discuss their respective pros and cons, considering implementation complexity, the amount of data to store, the amount of data to send on restart, how early the snapshot can be taken, and any other parameters you find relevant. Make a principled choice of which protocol you prefer and argue for your decision. ▼

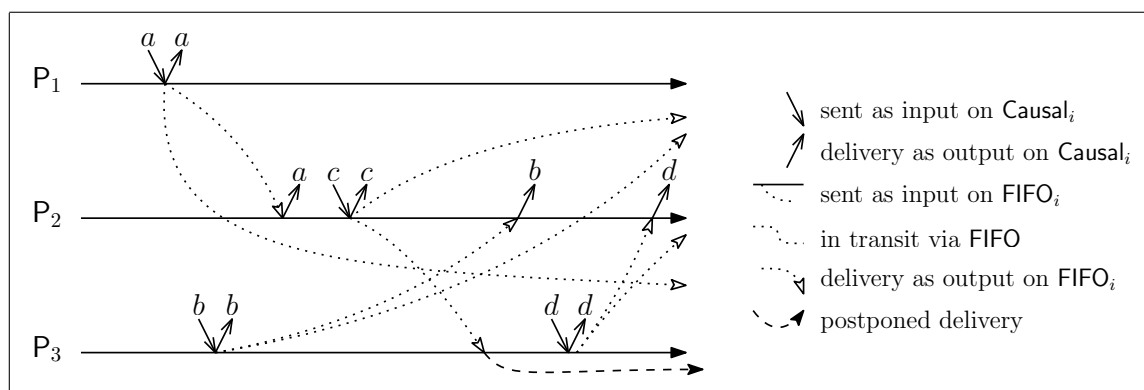


Figure 4.32: Causal Communication Example. The communication via FIFO is the same communication as in Fig. 4.12. Notice, in particular, that the communication via FIFO is not causal. However, now the inputs and outputs on the API ports are for Causal, and this communication *is* casual as P_3 delays the delivery of c .

Exercise 4.14 (Causal Communication) Consider the implementation of Causal using FIFO in Fig. 4.32. It is a run of Fig. 4.10. Update the vector clocks as in Fig. 4.10. What would the vector clocks of the four messages be? There are 4 messages sent, and hence 6 pairs of messages. For each such pair, use the vector clocks associated to the messages to determine if the messages in the pair are concurrent, or the pair is in the causal past relation. ▼

4.9 A Formal Simulation Argument†

This section is not curriculum, it is for the formally inclined reader who cannot get enough formalism. Earlier in the chapter we gave the intuition why **Flood** + **Flood2FIFO** securely implements FIFO. Namely, whatever behaviour **Flood** + **Flood2FIFO** is capable of is included in the possible behaviour of FIFO. To argue this formally one would have to argue that for any possible behaviour of **Flood** + **Flood2FIFO** there is a way to use the special ports of FIFO to make it have the same behaviour. We show the proof below with all details explained, as an example of what such a proof looks like. In the rest of the book, we will not be this formal, but the interested reader can construct such proofs along the lines of the example.

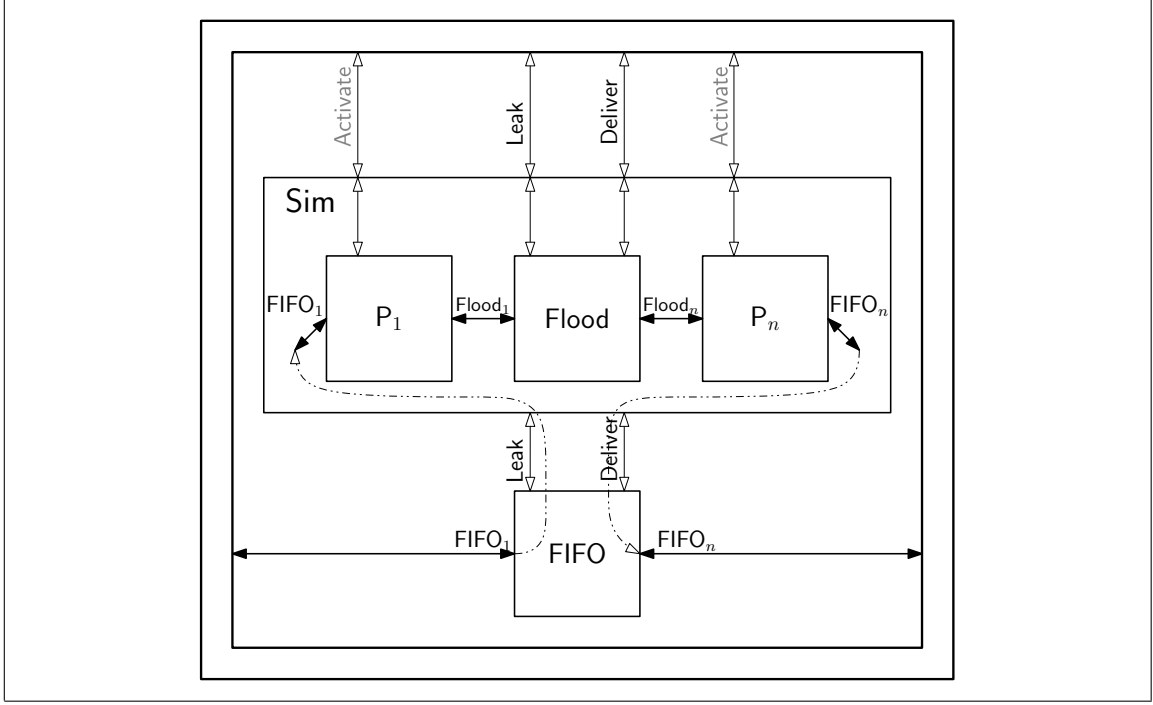
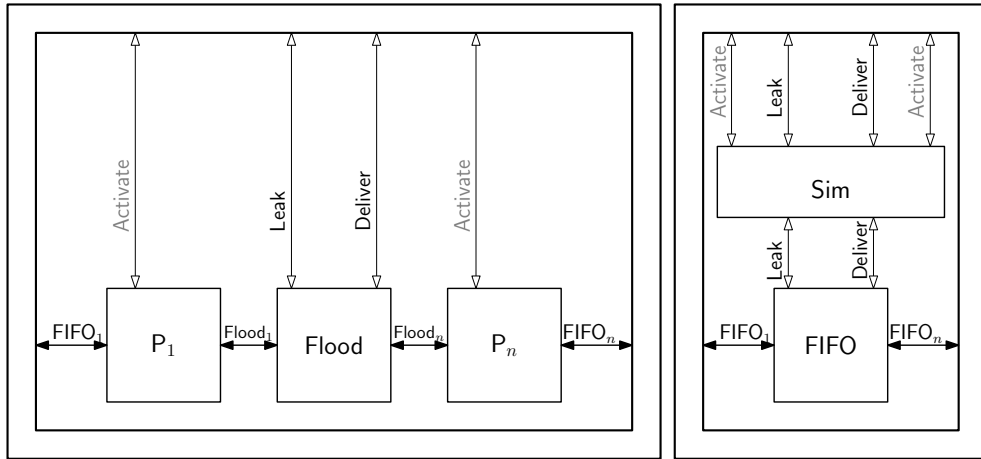


Figure 4.33: Illustration of simulating just by running the protocol. This is possible whenever the ideal functionality being implemented leaks all inputs.

Theorem 4.9 Let $\text{Flood2FIFO} = \{P_1, \dots, P_n\}$ where P_i is defined in Fig. 4.4. Then $\text{Flood} + \text{Flood2FIFO} \sqsubseteq \text{FIFO}$, i.e., the protocol Flood2FIFO when run with the ideal functionality Flood behaves as the ideal functionality FIFO.

PROOF Recall that $\text{Flood} + \text{Flood2FIFO} \sqsubseteq \text{FIFO}$ is defined to mean that there exists a simulator Sim such that $\text{Flood} + \text{Flood2FIFO}$ and $\text{Sim} + \text{FIFO}$ behave the same. This in turn means that for any IA, called the environment $\mathcal{Z}$, that could connect to the systems, they will give the same outputs on the same input. The setup is illustrated here:



We call the configuration $\mathcal{Z} + \text{Flood2FIFO} + \text{Flood}$ on the left the **real world** and we call the configuration $\mathcal{Z} + \text{Sim} + \text{FIFO}$ on the right the **simulation**.

We can make a very simple simulator Sim which basically just internally runs a copy of $\text{Flood} + \text{Flood2FIFO}$, called the **simulated protocol**. This setup is illustrated in Fig. 4.33.

When in $\mathcal{Z} + \text{Sim} + \text{FIFO}$ the environment inputs a message on FIFO_i , it is by design given to the simulator Sim on Leak , so the simulator can just input it on $(\text{Flood} + \text{Flood2FIFO}).\text{FIFO}_i$ in the simulated protocol. In addition the simulator will route messages to and from $\text{Flood}.\text{Leak}$ and $\mathcal{Z}.\text{Leak}$. Similarly for Deliver and the activation ports of the parties in the simulated protocol. Therefore the simulated protocol $\text{Flood} + \text{Flood2FIFO}$ is run on exactly the same sequence of inputs as the real world protocol would have been when interacting with $\mathcal{Z}$. Finally, when the simulated protocol outputs (P_i, m) on $(\text{Flood} + \text{Flood2FIFO}).\text{FIFO}_j$ the simulator will input (P_i, P_j, m) on $\text{FIFO}.\text{Deliver}$ to try to make FIFO also output (P_i, m) on FIFO_j . Notice that if this always makes FIFO output (P_i, m) on FIFO_j , then the simulation would be perfect: $\mathcal{Z}$ would always see the same input-output behaviour in the real world and the simulation.

So we only need to argue that when the simulator Sim inputs (P_i, P_j, m) on $\text{FIFO}.\text{Deliver}$ then FIFO will output (P_i, m) on FIFO_j . For this to be the case, we need that 1) $(P_i, m) \in \text{FIFO}.\text{InTransit}_j$ and 2) that (P_i, m) is the message of the form $(P_i, \cdot) \in \text{FIFO}.\text{InTransit}_j$ closest to the head of $\text{FIFO}.\text{InTransit}_j$. When this is that case the ideal functionality FIFO will indeed deliver (P_i, m) on FIFO_j , as desired. To see that the two conditions hold, note that when Sim inputs (P_i, P_j, m) on $\text{FIFO}.\text{Deliver}$ it is because it just saw P_j output (P_i, m) in the simulated protocol. This means that just before P_j did so, it was the case that $(P_i, \cdot, m) \in P_j.\text{InTransit}_j$. It is easy to see that this implies that $(P_i, m) \in \text{FIFO}.\text{InTransit}_j$, as we make the simulated protocol and FIFO send and deliver the same messages at the same times. It is in addition easy to see that $(P_i, m) \in \text{FIFO}.\text{InTransit}_j$ is the message of the form $(P_i, \cdot) \in \text{FIFO}.\text{InTransit}_j$ closest to the head of $\text{FIFO}.\text{InTransit}_j$. This is because the messages in FIFO are added to InTransit_j in the order they were sent by P_i and because Flood2FIFO delivers them on FIFO_j in the order they were sent by P_i . $\square$

Clearly the above proof strategy is very generic. If we implement an ideal functionality which leaks all inputs on Leak , then we can simulate simply by running the real world protocol. All that is needed is then that the protocol implements the safety properties of the ideal functionality that we implement. When this is the case, the simulator can simply instruct the ideal functionality to give the same outputs as the simulated protocol. In the following we will therefore only argue that our protocols implement the desired safety properties. We will not in each proof go through the above tedious simulation argument. We just wanted to be sure the curious and theoretically inclined reader saw it at least once.

Chapter 5

Confidentiality (2025)

Reading Guide *This section introduces some concepts and techniques of confidentiality, i.e., how information is hidden. All of the chapter is curriculum and can be queried at the oral exam, except Section 5.3.*

Contents

5.1	Confidentiality, Secret-Key Systems	138
5.1.1	The One-Time Pad and Perfect Secrecy	138
5.1.2	Practical systems, Definition of Security	142
5.1.3	Exhaustive Search	143
5.1.4	Price of an Attack	144
5.1.5	Stream Ciphers	144
5.1.6	Block Ciphers	145
5.2	Confidentiality, Public-Key Systems	147
5.2.1	RSA	149
5.2.2	Basic Facts In Modular Arithmetic	150
5.2.3	Chosen Plaintext Security of RSA	151
5.2.4	Chosen-Ciphertext Security	152
5.2.5	Optimal Asymmetric Encryption Padding	152
5.2.6	Hybrid Encryption	153
5.2.7	Elliptic Curve Cryptography	154
5.2.8	Quantum Computing and Encryption Security	154
5.3	Securely Implementing a Secure Channel*	155
5.4	Exercises	157

In this chapter we will introduce several kind of cryptosystems that allow to turn plaintext data into encrypted data or ciphertext, thus preserving the confidentiality of the underlying data. The first ingredient in a cryptosystem is an **encryption algorithm**. This is a function, usually denoted by E , which takes as input some plaintext data m (and usually other values as described below), and produces some ciphertext c . (Usually, an encryption algorithm comes with a corresponding **decryption algorithm** D that can be used by the rightful receiver of the data to invert the process). Intuitively it is “safe” to transmit or store c on a resource that can be observed by the adversary, since the encryption algorithm – in ways that we will see later – is able to turn the plaintext into some “gibberish” value from which it is very hard to recover any information about the original plaintext. In modern cryptography we always

assume that the encryption algorithm is known to everyone, even to the adversary. This is known as the *Kerckhoffs' principle*. The opposite of this is *security by obscurity*, which is instead the process of trying to achieve security by hiding the algorithms we are using from the adversary. Intuitively, it might seem a good idea to hide as much as possible from our potential adversaries. However experience shows that this is a very near sighted approach. It is very hard to hide the functioning of a computer system – after all the adversary might be able to obtain a copy of our software and reverse-engineer it. Moreover, making our encryption algorithms public helps us gain confidence in their security, since it allows researchers to study, analyse, and try to break them. Thus, if you want to encrypt some data, you should not try to invent your own encryption algorithm – this is very likely to end up in a security disaster. Instead, you should look at some of the encryption algorithms that the cryptographic community has properly studied and analysed. In fact, there is only a handful of encryption algorithms that are used to encrypt the vast majority of all internet traffic, and those algorithms are chosen by means of public international standardisation process. Now, since everyone uses the same algorithms to encrypt their data, we need something to make sure that the way we encrypt our data is unique and cannot be understood by others. This is achieved by using a unique **cryptographic key**. While the encryption algorithms are public, it is important to keep your keys secure. In this chapter we will see two kind of cryptosystems: **symmetric** or **secret-key** cryptosystems vs. **asymmetric** or **public-key** cryptosystems. In the former, the same key is used to encrypt and to decrypt, and this key should of course be kept secret. In the latter instead two different keys are used to encrypt and decrypt respectively. In this case, the encryption key is usually known as the **public-key** that, as the name suggests, can be made public while the decryption key, usually known as the **secret-key** or **private-key**, should be kept private.

Finally, a crucial ingredient in modern cryptosystems is some source of **randomness** or otherwise non-repeating value. This will play a crucial role to achieve meaningful notions of confidentiality, as we shall see.

5.1 Confidentiality, Secret-Key Systems

We start with secret-key cryptosystems, which are also historically the first kind of cryptosystems to have been discovered and used. A secret-key cryptosystem consists of three algorithms (G, E, D) , one for generating a key, one for encryption and one for decryption. The algorithm G takes as input a key length and usually produces a key by just outputting a randomly chosen bit string of that length. The algorithm E takes as input a key k and a plaintext m and outputs a ciphertext c . The algorithm D takes a ciphertext c and a key k and produces a plaintext $D_k(c)$. The basic requirement for a secret-key cryptosystem is *correctness* which means that if the *same* key is used for encryption and decryption, the original plaintext is recovered. In other words, it must hold that:

$$m = D_k(E_k(m)).$$

Intuitively, we would like the system to be *secure*, in the sense that an adversary who sees c but does not know k should have no idea whatsoever about which plaintext c represents. We will attempt to define more formally what this means later on.

5.1.1 The One-Time Pad and Perfect Secrecy

The first example of a secret-key cryptosystem is the *one-time pad*, an unbreakable encryption algorithm that, as the name suggests, works under the assumption that each encryption key is only used once. The one-time pad works as follows: Assuming the message is a string of bits labeled $m_1, \dots, m_t$, the key will be a random bit-string of the same length labeled $k_1, \dots, k_t$. We

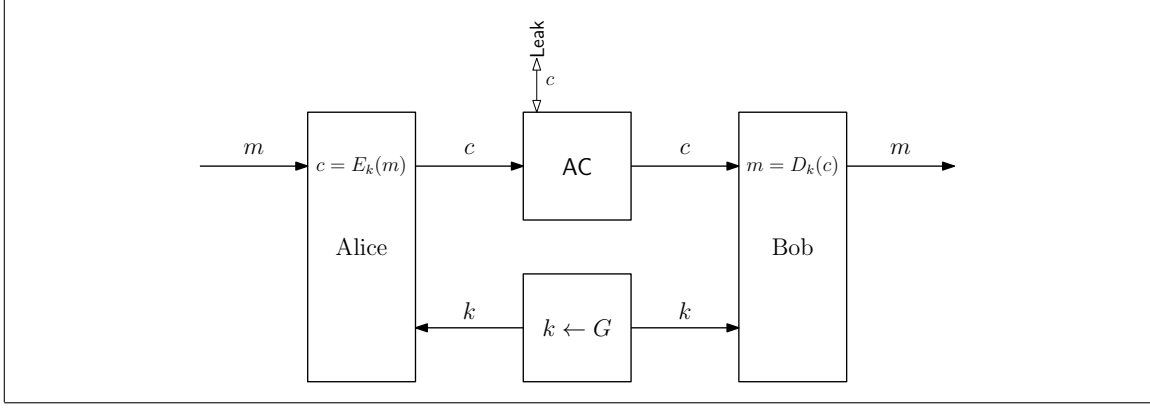


Figure 5.1: An illustration of how a symmetric encryption scheme (G, E, D) is used to send a single message. Alice has to send a message m to Bob over an authenticated channel which might leak the transmitted data. Somehow they managed to run the key generator G in the past and learn a common key k . When m arrives, Alice can compute and send $c = E_k(m)$. This ciphertext is allowed to leak. When receiving c , Bob can compute $m = D_k(c)$ and output m . It is essential that k is known only by Alice and Bob. We call the protocol $\Pi_{G,E,D}$.

encrypt by taking the bit-wise XOR of the strings, i.e., the ciphertext $c_1, \dots, c_t$ is defined by $c_i = m_i \oplus k_i$.¹

The receiver, who also knows the same key, can recover the plaintext by decrypting each i 'th bit as:

$$c_i \oplus k_i = (m_i \oplus k_i) \oplus k_i = m_i \oplus (k_i \oplus k_i) = m_i.$$

The intuition for security is as follows: given only a ciphertext $c_1, \dots, c_t$, any key string $k_1, \dots, k_t$ could have been used during encryption, and for each such key we would get a different plaintext. Thus any ciphertext could decrypt to any plaintext, so the eavesdropper learns nothing about the message from seeing the ciphertext. A more precise result is easy to show:

Theorem 5.1 *When one-time pad is used for encryption, the ciphertext is always a uniformly distributed bit string, in particular, it is independent of the plaintext.*

PROOF For illustration, we first show the simple case where we encrypt just one bit. Since the key bit is chosen independently of the message, we calculate as follows:

$$\Pr[c = 0] = \Pr[m = 0, k = 0] + \Pr[m = 1, k = 1] \quad (5.1)$$

$$= \Pr[m = 0] \Pr[k = 0] + \Pr[m = 1] \Pr[k = 1] \quad (5.2)$$

$$= \Pr[m = 0] \cdot \frac{1}{2} + \Pr[m = 1] \cdot \frac{1}{2} \quad (5.3)$$

$$= \frac{1}{2} (\Pr[m = 0] + \Pr[m = 1]) = \frac{1}{2} \quad (5.4)$$

In case you are not familiar with notation and rules in probability theory, let's see what is going on in the calculation above: we are computing the probability that the ciphertext ends up being 0. In 5.1 we observe that due to the nature of the XOR function, this only happens when the message and the key happen to be equal, so the probability of c being 0 is the sum of

¹XOR or *exclusive OR* is the Boolean function that takes as input two bits and outputs 0 if the bits are equal and 1 if the bits are different.

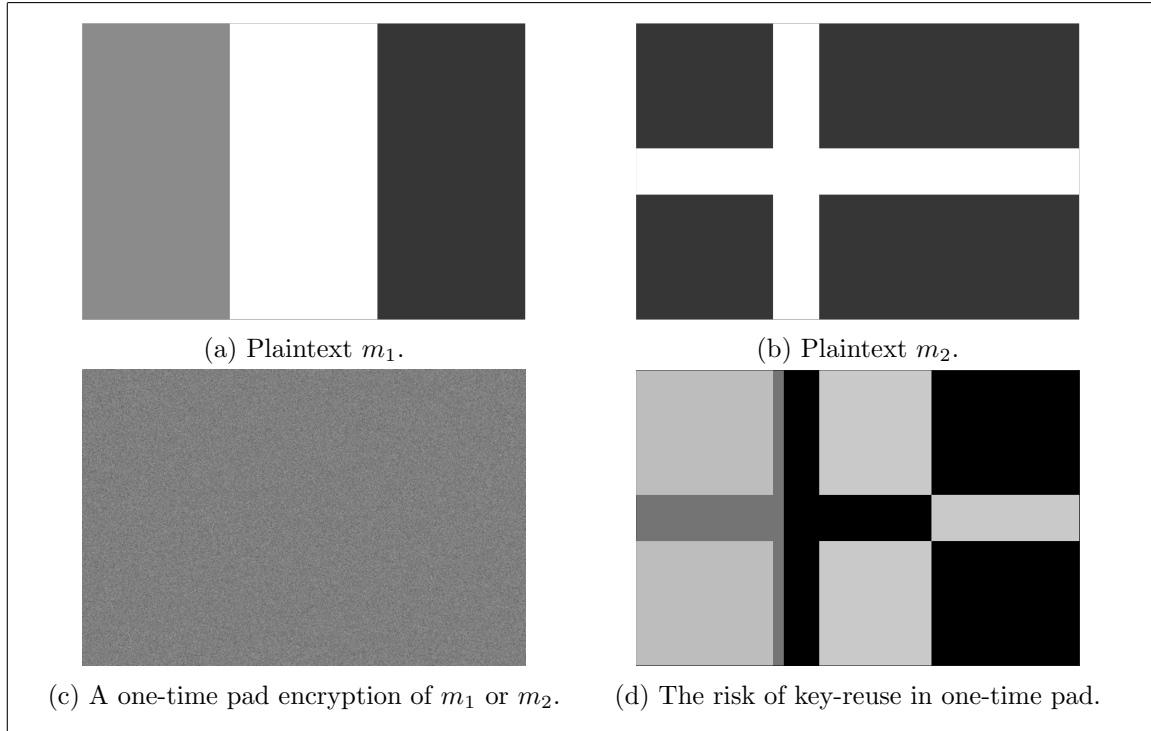


Figure 5.2: A visual representation of one-time pad encryption and the danger of key-reuse. Fig. 5.2a and Fig. 5.2b show two plaintexts, the grayscale representations of a tripartite flag with vertical lines and a flag with a nordic cross. Fig. 5.2c shows a one-time pad encryption of one of the two messages. The picture only shows noise, and it is impossible to tell which picture we started from without the key. Fig. 5.2d shows the risk of key-reuse in one-time pad. The figure shows $m_1 \oplus m_2$, namely the information that is leaked encrypting both m_1 and m_2 using the same key. The picture shows distinct features from the original plaintexts.

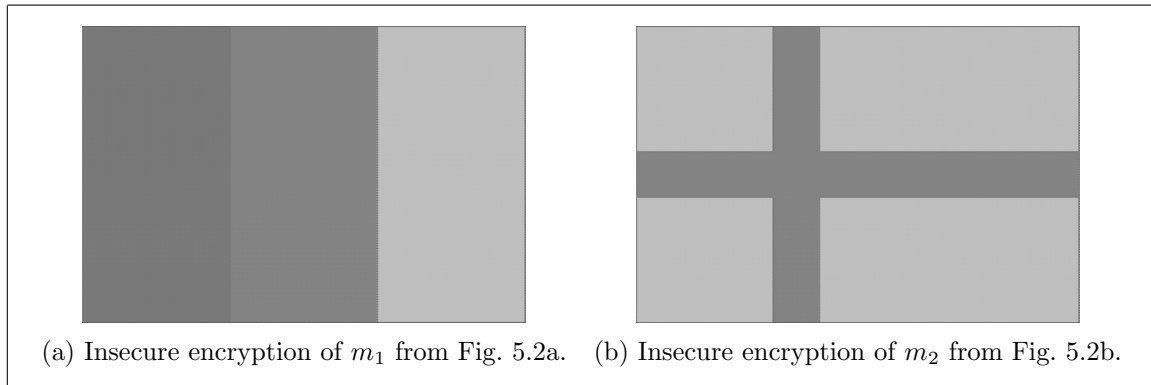


Figure 5.3: A visual representation of the risks of encrypting a long message using a block-cipher in a block-by-block fashion without a mode of operation. Equal plaintext blocks result in equal ciphertext blocks, revealing significant information about the underlying plaintext. Fig. 5.3a and Fig. 5.3b show the encryptions of Fig. 5.2a and Fig. 5.2b respectively. While each original color has been replaced by a new color by the block-cipher, the structure of the original flags is completely visible.

the probabilities of the case in which both m, k are 0, plus the probability they are both 1. In 5.2 we note that since the key is chosen independently from the message, the joint probability of m and k being equal to some values is equal to the products of the individual probabilities of m, k being equal to those values. In 5.3 we remember that the key is chosen uniformly at random, so the probability that the bit is 0 or 1 are both equal to $1/2$. Finally, while we don't want to make any assumption on how the message is chosen (after all, the cryptosystem should be secure regardless of which message is being encrypted), in 5.4 we remember that the message can only be either 0 or 1 so the sum of the two probabilities must be 1, thus reaching the desired result.

In the general case, we also have to use that the key bits are chosen independently of each other, but otherwise the basic idea is the same: we can show that for any message length t , each ciphertext occurs with probability 2^{-t} . For simplicity, we do this only for the all-0 ciphertext of length t : $00 \dots 0 = 0^t$.² Note that for the ciphertext to be 0, the plaintext and key must take the same value, but this can be any value x .

$$\begin{aligned}
\Pr [c = 0^t] &= \sum_{x \in \{0,1\}^t} \Pr [m = x, k = x] \\
&= \sum_{x \in \{0,1\}^t} \Pr [m = x] \Pr [k = x] \\
&= \sum_{x \in \{0,1\}^t} \Pr [m = x] 2^{-t} \\
&= 2^{-t} \sum_{x \in \{0,1\}^t} \Pr [m = x] \\
&= 2^{-t}
\end{aligned}$$

□

What this shows is that the ciphertext has the same distribution *no matter what the plaintext was*, which of course means that there is no way to learn anything about the plaintext from seeing the ciphertext. We say that a cryptosystem has *perfect secrecy* when the ciphertext is independent of the plaintext. Note that perfect secrecy holds no matter how much computing power the eavesdropper has – in other words, the one-time pad is unconditionally secure.

However, the argument above works only if each key string is used *once*, otherwise the key bits would not be independent: some of the key bits used at later positions would be copies of bits used before. So this means that we need as many random key bits as we have message bits, and so the one-time pad is quite useless in practice. It is not hard to see that this is no coincidence: no matter how smart we try to be, there is no way around this problem:

Theorem 5.2 *Suppose a cryptosystem can handle $\mathcal{M}$ possible plaintexts, and uses $\mathcal{C}$ ciphertexts and $\mathcal{K}$ keys. If the system has perfect secrecy, then it must be the case that $\mathcal{K} \geq \mathcal{C} \geq \mathcal{M}$.*

PROOF For any cryptosystem it must hold that $\mathcal{C} \geq \mathcal{M}$, otherwise encryption would have to send different plaintexts to the same ciphertext, and we could not decrypt correctly. Now, consider some plaintext m . Note that for every ciphertext c , there must exist some key K_c for which $E_{K_c}(m) = c$. If this was not the case, there would be some ciphertext c_0 that could not

²In theoretical computer science the notation a^t indicates the string obtained concatenating t times the symbol a . When the symbol a is a number this leads to ambiguity between the two meanings of e.g., 0^t , which could either be 0 raised to the t (equaling 0), or the string containing t zeros. Usually it is clear from the context which of the two meaning is to be assumed.

occur when the plaintext is m . Then we would not have perfect secrecy: if the adversary sees that ciphertext c_0 was sent, they could conclude that the plaintext was not m . So we conclude that there is at least one key for every ciphertext, i.e., $\mathcal{K} \geq \mathcal{C}$. $\square$

To see what this result means, note that if we did try to use the same “one-time” pad key twice, then the number of possible keys would be *smaller* than the number of possible messages, and the above result says that perfect secrecy is not possible. More concretely, re-using the key in one-time pad leads to leaking information about the underlying plaintexts, as visualized in Figure Fig. 5.2.

5.1.2 Practical systems, Definition of Security

In real life, it is clear that we will have to use the same key for many different messages, or at least for a message much longer than the key. This means in practical systems we typically have to settle for computational security, i.e., we build systems that we hope the adversary cannot break because they would have to spend an unrealistically large amount of time to do so.

This has several consequences for how we should design cryptosystems and use them: First, the adversary should have no idea what we are sending even if we use the system several times, so if for instance we send the same message twice, an eavesdropper should not be able to tell that this was what happened. If the encryption algorithm really only takes key and message as inputs, then if we send the same message twice, two identical ciphertexts will be produced, and this will be visible to an eavesdropper.

Therefore, good encryption algorithms work not only with k and m as input, but they also need some additional variable whose value changes from one encryption operation to the next. Such a variable is often called a *nonce* (“number used once”). It must be chosen in some way that guarantees that we will not use the same value twice. A nonce can take the form of a counter or it may consist of random bits that are freshly chosen every time we encrypt something (different encryption schemes might require different kind of nonces).

As a result, if we encrypt the same message twice, the value of the nonce n will be different in the two cases, and so (if we design the encryption algorithm properly) this gives us a chance to make sure that even two encryptions of the same message look like unrelated, random strings of bits. When we want to emphasize that a nonce n was used in the encryption process we write

$$c = E_k(m, n),$$

where it must of course still be the case that $m = D_k(E_k(m, n))$. If the particular value of n is not important, we will sometimes just write $E_k(m)$ even if the encryption algorithm is defined to choose a nonce internally.³

Here is an informal version of the definition of security one usually wants from cryptosystems with computational security: Consider an adversary who plays the following game: they send a message m to an oracle O who has a secret key k inside. (The oracle is just a “black-box” that the adversary can play with, and it is a convenient abstract way to model what kind of access the adversary has to our system). The oracle sends back a ciphertext c , which is computed in one of two ways: In case 1) $c = E_k(m, n)$, in case 2) $c = E_k(r, n)$ where r is random message

³Note that if you use an encryption library it is crucial to understand which of the two conventions is being used e.g., whether the programmer should take care of sampling a good nonce *outside* the encryption function and provide it as a parameter, or whether the nonce is chosen *internally* by the encryption function. In the old days the previous convention was more common, as it gives more flexibility to the user of the encryption library. However, after many years of noticing how users were not using encryption libraries properly, the more modern approach tends to hide as much as possible about the details of the encryption from the programmer. This can also create problems, as the user might be in doubt whether the encryption algorithm is using a nonce at all!

of the same length as m . In both cases a different nonce n is used for each encryption. The adversary may send as many messages as they like but the oracle stays in case 1) or 2) all the time. The security definition can then be stated as follows.

Definition 5.3 Consider any adversary who plays the above game and whose computing power is limited in the sense that whatever algorithm they run terminates in time much less than the time it takes to try all possible keys in the cryptosystem (see below for a detailed discussion on exhaustive key search). No such adversary can guess whether they are in case 1) or 2) (with probability better than essentially a random guess).

What this definition is saying is that in real life when an adversary looks at encrypted data, even if they know (or can even to some extent control) the data that is encrypted, for all they know, the encrypted data may as well contain unrelated random garbage. So in a very strong sense, the adversary has no idea about what is encrypted.

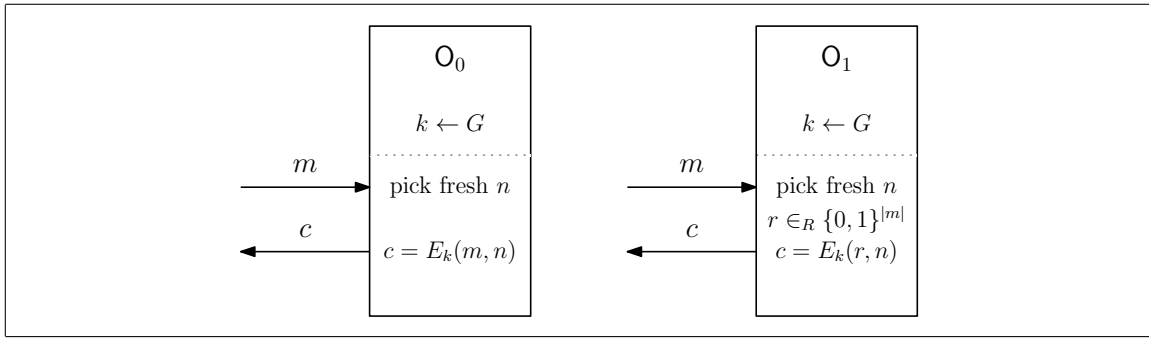


Figure 5.4: An illustration of the security notion *indistinguishability under chosen-plaintext attack* or IND-CPA for short. On the left there is an IA O_0 which when it is given m returns an encryption of m (using a fresh nonce). On the right there is an IA O_1 which when it is given m returns an encryption of a random message of the same length of m . The security notion asks that these two IAs look the same to any plausible adversary, i.e., $O_0 \equiv O_1$.

This definition is clearly not mathematically precise. It can be completely formalised, and if we do so, we get the notion of *indistinguishability under chosen-plaintext attack (IND-CPA)* (also known as *semantic security*), but the details of this are outside the scope of this course. The more formal definition is illustrated in Fig. 5.4 without going into details.

5.1.3 Exhaustive Search

Another consequence of reusing the same key many times is that the system can only be secure if the adversary's resources are limited.

More specifically, we have to assume that the adversary does not have enough computing power to run through all possibilities for the key k . This is because we have to take into account the possibility that the attacker finds out (by guessing or spying) one or more plaintext(s) m corresponding to ciphertext(s) c . Now they can execute the following simple algorithm known as *exhaustive key search*:

1. Initialize an empty list L .
2. For every possible key k' : compute $D_{k'}(c)$ and check if $D_{k'}(c) = m$ for all the plaintext/-ciphertext pairs (m, c) that are known to the adversary. If this is the case, add k' to the list L .

Of course, the correct key k will always be on the list L once the algorithm terminates. But if the list is very long, running the algorithm does not help the adversary much. However, a long list is unlikely if the adversary knows enough plaintext: say the key has t bits and the adversary knows u bits of plaintext in total. If we make the reasonable assumption that decrypting with an incorrect key gives you something random, then the probability that an incorrect key happens to produce a result that matches all u bits of plaintext is 2^{-u} . There are $2^t - 1$ incorrect keys, so we expect that $2^{-u}(2^t - 1)$ incorrect keys will survive the test. If $u > t$, then this is less than 1, so the adversary can expect that only the right key will survive the test. So we conclude:

FACT: If the adversary knows u bits of plaintext (and matching ciphertext), and u is larger than the length of the key, then we can expect that exhaustive search will identify the correct key.

It is important to understand that exhaustive search can be used against *any* system where the same key is used to send several messages, that is, any practical system. To make sure that exhaustive search is infeasible, a secure system these days must use keys of length about 128 bits or more – since doing 2^{128} repetitions of some non-trivial computation is currently considered completely infeasible⁴. Note that the key length by itself is no guarantee for security, it is only a necessary condition: given m, c , there might be a much faster way to find the key than simply trying all possibilities.

5.1.4 Price of an Attack

Thinking in terms of exhaustive search is part of estimating the **cost of an attack** and balancing that with the **probability of an attack**. Say that if someone breaks your system you will lose C dollars, and there is a probability p that an attacker can break into the system. Then the expected cost of attacks on the system is pC dollars. You want the amount pC to be negligible, i.e., something you don't mind paying. Let us for illustration say we find one dollar to be negligible. In that case we would call a probability $p = 1/C$ that the system is broken into a **negligible probability**. In general, to know what probability you can consider negligible, you need to know the cost of a break each time the system is run and how many times it is run per day and for how long.

If we think like that, how long encryption keys should we use? Let us try to do a rather conservative estimate. The global computing power in the foreseeable future is about 2^{80} instructions per second. Imagine each instruction has some small probability p of causing a bad event that will cost you a million dollars, which is about 2^{20} dollars. Imagine this is repeated for 10 years, which is about 2^{28} seconds. Then the expected economic cost of the bad event over ten years would be $p2^{20+80+28} = p2^{128}$ USD. So if we say that an expected cost of one USD from a possible unlucky event over the next ten years is a negligible cost, then we could adopt $p = 2^{-128}$ as a negligible probability.

5.1.5 Stream Ciphers

Practical secret-key algorithms come in two flavors: *stream ciphers* and *block ciphers*.

A stream cipher is basically an algorithm X that expands a short key k and a nonce n to a much longer random *looking* string $X(k, n)$, which is then used to encrypt the message *as if* it was a one-time pad, that is, we take the output from the stream cipher and XOR it bitwise with the message m to get the ciphertext $c = m \oplus X(k, n)$. To decrypt, we compute

⁴At the time of writing in 2024, the entire Bitcoin network was performing around 2^{69} “cryptographic operations” per second, meaning that it would take around 2^{59} seconds for the entire Bitcoin network to try all possibilities. The age of the universe is currently estimated to be $\approx 2^{58}$ seconds.

$c \oplus X(k, n) = m \oplus X(k, n) \oplus X(k, n) = m$. Note that the receiver must know not only k but also n to do this. However, n is not secret so it can just be sent along with the ciphertext.

It is easy to see that the output string cannot really be random: say we expand a 128 bit key to a 1000 bit long output. Each possible key can only result in a possible output. So the stream cipher cannot possibly output more than 2^{128} different strings, whereas a really random 1000 bit string might take on any of the 2^{1000} possibilities. Even though the output string is not really random, the hope is that an adversary with limited computing power will not be able to tell the difference and hence cannot break the resulting encryption.

As the name suggests, stream ciphers do not generate the entire output at once, but one bit (or chunk of bits) at the time in a streaming fashion. In fact one does not even need to know the required length in advance. Instead, the algorithm is initialised with inputs k, n and can then be queried for the next bit or byte of the output string. This can in principle go on for as long as you wish.

This means that a stream cipher fits nicely with applications where the input to be encrypted arrives as a stream, say byte by byte and we do not know when it ends. An example is a user that types away at a keyboard. We can encrypt every byte immediately as it arrives, and send off the encrypted data.

A well known example of a stream cipher is RC4 which used to be popular, but after some known weaknesses were found has decreased in popularity. In recent years other, very fast and secure stream ciphers have been proposed, some examples that can be found in the literature are SALSA20 and SNOW.

One should note that any good block cipher (which we look at in the following section) can also be used to build a secure stream cipher.

Finally, we note that there is a related concept to stream ciphers, that we will return to later in the course, namely the concept of **pseudorandom generator (PRG)** or **deterministic random bit generators (DRBG)**. This is a crucial component of any modern operating system or programming language, and allows a computer (which is a deterministic machine) to simulate random strings of bits using algorithms similar to stream ciphers. PRGs are crucially used in security applications to generate, among other things, cryptographic keys to be used in encryption schemes. While the underlying principles behind stream ciphers and PRGs are similar, the details differ due to their different use cases.

5.1.6 Block Ciphers

Block ciphers encrypt in their basic form a fixed size block of data, and output a block of the same size as the input. Historical examples are the former US standards DES (56 bit keys, 64 bit blocks) and triple-DES (112 or 168 bit keys, 64 bit blocks). DES and triple-DES are now considered insecure and should not be used, but we will discuss them in the book since understanding *why* they are insecure helps us understanding important principles in cryptography. In particular DES has essentially always been considered insecure due to the very short key.

The current standard block cipher is AES (128, 192 or 256 bit keys and 128 bit blocks). It is incredibly popular and most modern CPUs implement AES directly in hardware, giving extremely fast encryption times.

Block ciphers alone do not yield secure encryption schemes, and a block cipher should always be used together with a suitable **mode of operation**, which are general methods that allow using a block cipher to encrypt a string of data of any length, and also to achieve security as required in Definition 5.3.

Without a mode of operation, encryption cannot be secure: assume you want to encrypt a long message m , that can be decomposed in blocks $m_1, \dots, m_t$ that fit the input size of the block

cipher (we assume here for simplicity that the length of the original message is a perfect multiple of the block size – if this isn't the case the message can be suitably padded). What happens if you simply encrypt each block using the block cipher e.g., if you compute each ciphertext block c_i as $c_i = E_K(m_i)$? As the block cipher is a deterministic function once the key is fixed, if two blocks of the message happen to be the same, their corresponding ciphertexts blocks will also be equal i.e., if $m_i = m_j$ then $c_i = c_j$. This leaks information about the structure of the plaintext, as shown in Figure Fig. 5.3, and therefore simply encrypting block by block without a mode of operation does not yield a secure encryption scheme.

Examples of mode of operations that do not suffer from this problem include Cipher Block Chaining (CBC) mode, Counter (CTR) Mode, and others that we do not cover in this book. These modes need as input not only the key and the message, but also a nonce, as described above: at least one block of data that can change from one encryption operation to the next. In modes-of-operation lingo, the nonce is called an *initialization vector*, IV .

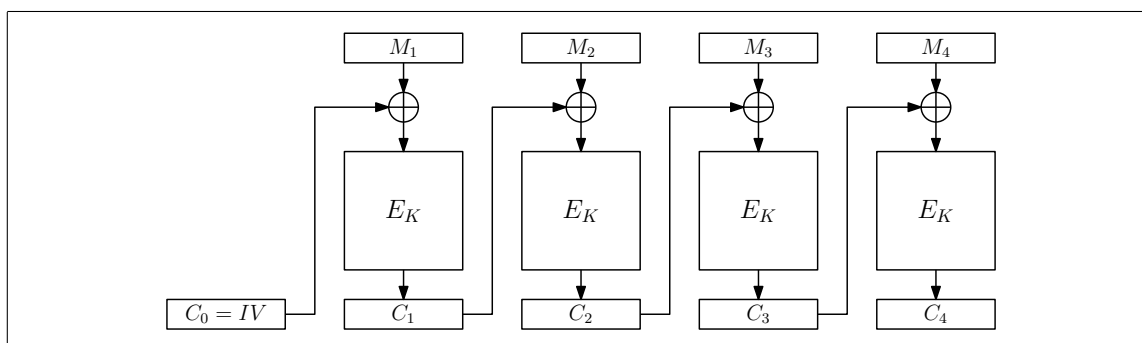


Figure 5.5: An illustration of CBC mode. In this example, four blocks are encrypted, producing a ciphertext with five blocks.

An historically commonly used mode is CBC mode. Assume the message consists of 128 bit blocks $M_1, \dots, M_t$, where we pad the last block t in some way, if it does not fill the required block length. Then the ciphertext will be $t + 1$ blocks $C_0, \dots, C_t$, where $C_0 = IV$ and for $i = 1, \dots, t$,

$$C_i = \text{AES}_K(M_i \oplus C_{i-1}) .$$

Here, $M_i \oplus C_{i-1}$ means bit-wise XOR of the two blocks. The idea here is to have the encryption of block i depend in a complicated way on the encryptions of all previous blocks, including of course the first block C_0 aka the IV . Thus, even if you encrypt the same message twice, the change in the IV will propagate to all subsequent blocks. Note that in CBC-mode the IV must be a random string to satisfy *indistinguishability under chosen-plaintext attack*. That is, the IV cannot be implemented as a counter in CBC mode (See Exercise 5.5 to see why this is the case).

The second mode we describe is also the most popular one at the time of writing, namely **counter mode (CTR)**, where again the message is $M_1, \dots, M_t$, and the ciphertext depends on an IV . Again the ciphertext will be $t + 1$ blocks $C_0, \dots, C_t$, where $C_0 = IV$ and for $i = 1, \dots, t$,

$$C_i = \text{AES}_K(IV + i) \oplus M_i .$$

Here $IV + i$ means: think of IV as a 128 bit number and add i to this number modulo 2^{128} (or in other words, ignore any carry beyond bit position 128). CTR mode has the advantage over CBC that it is easy to encrypt several blocks in parallel, and that we only need the encryption function for the block cipher, even to decrypt. As it should be apparent from its description,

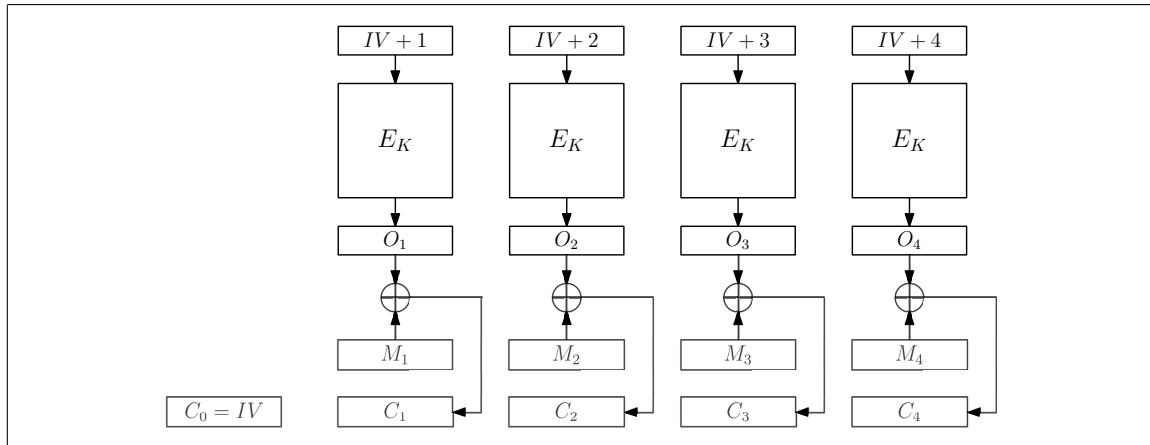


Figure 5.6: An illustration of CTR mode. In this example, four output blocks are produced. After this they are used for OTP encryption of the message. When used for encryption the IV is sent along to allow the receiver to regenerate the output sequence.

CTR mode essentially turns the AES block cipher into a stream cipher, which produces a long string of random looking bits that can be then used to mask the message in a “one-time pad” fashion.

As is apparent from the above modes, the *IV* is *not* secret, nevertheless it is important to get high quality encryption that the *IV* is chosen in the right way. The point is that the *IV* plays the role of the nonce, the extra input to the encryption we mentioned above, that is needed to ensure that the ciphertext will change, even if key and message are the same. Therefore the *IV* should be random, or chosen in some other way ensuring that values are reused with only very small probability. Unfortunately, several software implementations either ignore the problem of choosing *IV*, or circumvent it, for instance by fixing it to be the all-zero string. While this idea may make the implementation simpler, it degrades security and should be avoided.

In conclusion, as a rule of thumb, you will need a good block cipher, an appropriate mode of operation and a reasonable way to choose initialization vectors to get a useful encryption scheme.

Secret-key systems are generally quite fast, typically, one can get throughputs of hundreds of Mbytes/sec in software on a decent PC. And of course much more in a dedicated hardware implementation.

The basic problem with secret-key systems is that one must have the key in place at both the receiver and sender before sending data. This is not always an easy problem in practice (and is also the basic reason why no one wants to use the one-time pad in practice). The problem is particularly difficult if we have a large number of users where each user potentially may want to communicate privately with any other user. Then we need a separate key for each pair of users, and hence the number of keys increases quadratically with the number of users.

5.2 Confidentiality, Public-Key Systems

A public-key cryptosystem also has three algorithms (G, E, D), for key generation, encryption and decryption. The algorithm G takes as input a key length and outputs a pair of keys of the right length.

In contrast to secret-key systems, where the same key was used for both encryption and decryption, a public-key system makes use of a pair of matching keys, a **public-key** pk and a

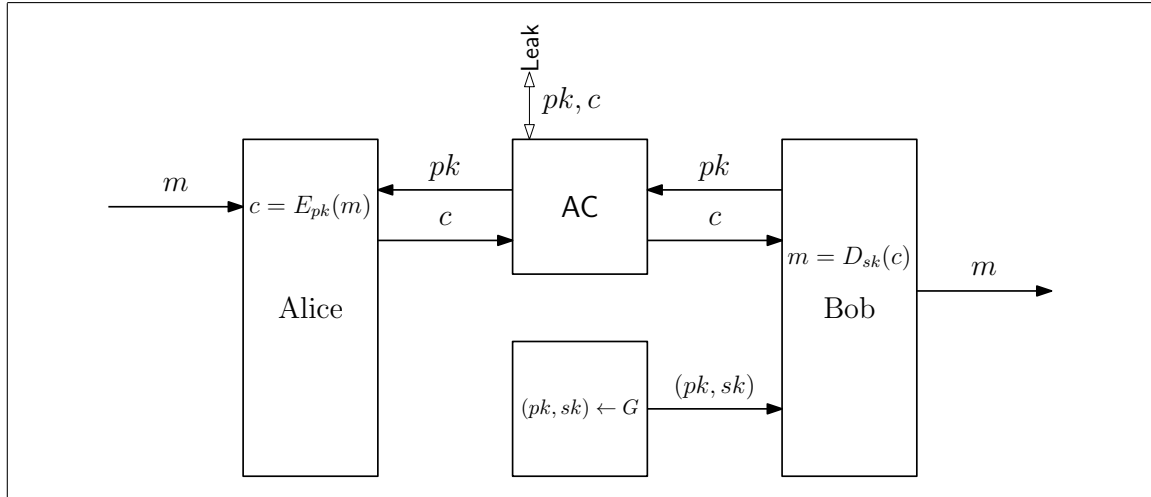


Figure 5.7: An illustration of how an asymmetric encryption scheme (G, E, D) is used. Alice has to send a message m to Bob over an authenticated channel which might leak the transmitted data. First Bob creates a key pair using G and sends pk to Alice. This public key is allowed to leak. When m arrives, Alice can compute and send $c = E_{pk}(m)$. This ciphertext is allowed to leak. When receiving c , Bob can compute $m = D_{sk}(c)$ (using the secret key) and output m . It is essential that sk is known only by Bob.

secret-key sk . Sometimes these keys can also be called **encryption-key** and **decryption-key**, if one wants to make clear that the keys are to be used for confidentiality (as opposed to public-key systems for authentication as we will see in the next chapter).

The idea is that a user A of such a system will generate such a pair of keys in private on their own machine. They will then publish pk , but keep sk private. For this to make sense, it clearly must be the case that even though there is a connection between pk and sk , it must be a very difficult computational problem to compute sk from pk .

The key generation algorithm is therefore usually more complex than for secret-key systems, because it has to produce pairs of keys with a carefully constructed relation between them. The goal is to ensure that anyone can encrypt a message m intended for A by running E on input m, pk to get $c = E_{pk}(m)$. Then A can decrypt this by running D on input sk, c because the system is constructed in such a way that for any message m we have:

$$m = D_{sk}(E_{pk}(m)).$$

We emphasize that, as for secret-key systems, the encryption algorithm may make random choices during the process, so that in fact the concrete value of $E_{pk}(m)$ may depend on those random choices, not just on pk and m . Indeed this is necessary for security as we will see in a moment.

Security here means that an adversary must have no idea about the plaintext corresponding to c , *even when given the public key pk* . The reason for this is that when using public-key cryptography everyone is allowed to know pk , and this of course includes the adversary.

We can formulate a notion of security for public key cryptosystems similar to what we did for secret key systems in Definition 5.3. The definition is exactly the same, with the only exception that the adversary is given the public key when the game starts.

Note that this immediately implies that the encryption function in a public-key cryptosystem must be *randomized* i.e., if one encrypts the same message twice then two different ciphertexts

will come out. If this was not the case, it would be very easy for the adversary to win the game in Definition 5.3 in the following way:

1. The adversary submits a message m to the oracle;
2. The adversary receives a ciphertext c from the oracle;
3. The adversary encrypts $c' = E_{pk}(m)$ and concludes that c is an encryption of m if $c' = c$ and an encryption of a random r otherwise.

The definition is sketched in Fig. 5.8.

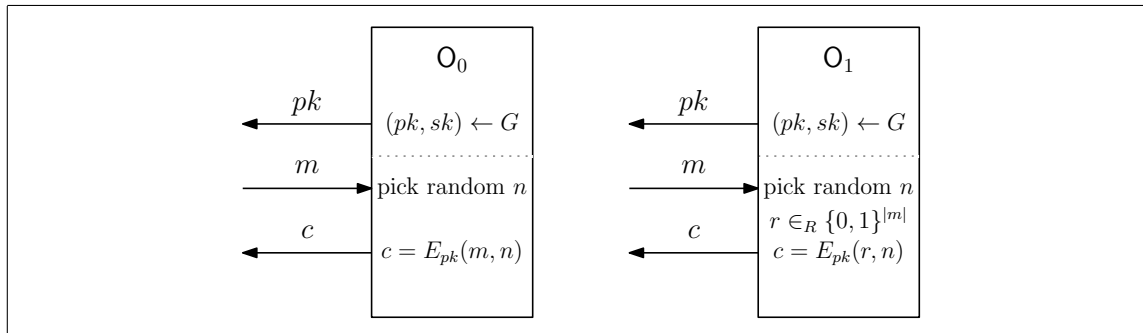


Figure 5.8: An illustration of the security notion indistinguishability under chosen-plaintext attack for public-key encryption. On the left there is an IA O_0 which hands out the public key and which when it is given m returns a random encryption of m . On the right there is a similar IA O_1 which when it is given m returns an encryption of a random message of the same length of m . The security notion asks that these two IAs look the same to any plausible adversary, i.e., $O_0 \equiv O_1$.

Public-key cryptosystems can also be broken using exhaustive search: Usually, there is only one private key corresponding to a given public key, so the adversary can just try all possible secret keys until they find the one that matches pk . However, for all known public key systems, there are algorithms known for computing sk from pk that are *much faster* than just trying all possibilities. Therefore the size of keys for public-key systems are usually much bigger than they are for secret-key systems.

Public-key systems solve the problem of distributing secret keys that we had before, but one still has to be careful: the sender of an encrypted message must make sure that the public key they are using actually belongs to the intended receiver. Otherwise someone else will be able to decrypt the message. Coming up with good solutions to this is less trivial in practice than one might expect – we return to this problem later, in the chapter on key management.

5.2.1 RSA

An important historical example (which is however becoming less popular in practice) is the RSA public-key system. RSA was the first publicly discovered public-key encryption scheme and has been the main public-key encryption scheme from the 80s until the mid 2010s. In RSA the public key consists of two numbers n and e , whereas the private key consists of numbers n and d . The number n is called the *modulus* and is the product of two prime numbers p, q . The numbers e and d are chosen to satisfy a particular relation that involves in an essential way the prime factors p, q . More precisely, one computes $d = f(e, p, q)$ for some particular easy to compute function f , the details of which we return to below.

It can be shown that computing the private key from the public one is as hard as solving the *factoring problem*, i.e., the problem of finding p, q from n . Conversely, it is clear that if one could factor n and find p, q , one could also compute d in the same way as when keys are generated. So breaking RSA is no harder than factoring.

We describe here the basic (or *vanilla*) version of RSA, which is deterministic and therefore insecure as described above. We will later discuss how to transform vanilla RSA into a *secure* public-key cryptosystem: in the basic version of RSA, messages are numbers in the interval $[0..n-1]$, and to encrypt a number m , one computes $c = m^e \bmod n$, i.e., we raise m to exponent e , divide by n and take the remainder to be our result. In the same way, one decrypts by computing $c^d \bmod n$, and the special choice of e, d ensures that we always have

$$c^d \bmod n = (m^e \bmod n)^d \bmod n = m$$

As mentioned, the only known threat to this system is the possibility of factoring n . This is of course harder, the larger n is. The best known algorithms for factoring are capable of factoring a k -bit RSA modulus in time $2^{O(\sqrt[3]{k})}$ (very roughly and ignoring many details). The performance of these algorithms in practice is such that the recommended RSA keys size today is in the range 2048-4096 bits. But this size is not fixed for all times, it must increase with improvements in hardware and the increasing efficiency of the known factoring methods, and take into account how long into the future the encrypted data should stay private. For instance, when RSA was invented back in 1978, 512 bits long moduli were thought to be sufficient. In 2015, such a modulus could be factored in under 4 hours for 75\$ using standard cloud computing platforms. Several national and international agencies periodically release guidelines about recommended key sizes, including the European ENISA, the American NIST, the German BSI, etc.

For completeness we now explain how to compute d from e, p and q , i.e., how to compute the function f from above: first, e must be chosen such that the greatest common divisor of e and $(p-1)(q-1)$ is 1. Then one computes d such that $ed \bmod (p-1)(q-1) = 1$. The condition on e ensures that a suitable d always exists. This is usually written as follows:

$$d = e^{-1} \bmod (p-1)(q-1).$$

RSA used to be one of the most commonly used public-key systems in practice, but it is declining in popularity. One important reason for why it was so popular is that when generating keys, it is possible to choose the public key (n, e) such that e is a small number, such as 3 (2 cannot be used since $(p-1)(q-1)$ is always even). Since the time needed to do the encryption or decryption is directly proportional to the number of bits in the involved exponent, this means that RSA encryption can be made several orders of magnitude faster. RSA is the only public-key system for which such an optimisation is available. Note that a similar optimisation cannot be used for the secret key (n, d) , since d will be too easy to guess if it becomes too small.

5.2.2 Basic Facts In Modular Arithmetic

In case this is the first time you see modular arithmetic, this subsection will explain how the calculation in RSA described above are performed. If you are familiar with modular arithmetic you can safely skip to the next subsection.

As the name suggest modular arithmetic is the arithmetic performed modulo some number, the modulus n . This simply means that when n is reached we start back from 0. Everyone performs modular arithmetic daily when thinking about the time, even without realising it. For instance, if it's 11am and I tell you to meet me in 3 hours you know we will meet at 2pm because once you get to 12 you reset the counter to 0. More formally, when writing $a \bmod n$ we mean the remainder after you have divided a by n . For instance $17 \bmod 5 = 2$ since $17 = 3 \cdot 5 + 2$.

If you have that $a \bmod n = b \bmod n$ we say that a and b are congruent modulo n . This just means that through the lens of modular arithmetic a and b are equivalent, they are the same. For instance, 17 and 12 are congruent modulo 5 since the remainder is in both cases 2.

Modular additions, subtractions, multiplications and exponentiations work in the same way as they do in “regular” arithmetic. So for instance if you have that a, b are congruent modulo n then it will also hold that

$$x \cdot a^y + z \bmod n = x \cdot b^y + z \bmod n$$

for some integers x, y, z . It is also useful to note that when you do multiple calculations, you get the same result whether you perform the modular reductions “on the way” or you wait until the end i.e.,

$$ab \bmod n = (a \bmod n) \cdot (b \bmod n) \bmod n$$

and in fact this can be used to our advantage when computing with very large numbers (like when we perform RSA decryption) to ensure that the intermediate results do not “explode” in size.

Most cryptographic algorithms in this book crucially use exponentiations, and therefore it is useful to remember that

$$(a^x \bmod n)^y \bmod n = a^{(xy)} \bmod n$$

Another important concept in modular arithmetic is the concept of a **modular inverse**. We say that “ b is the modular inverse of a modulo n ”, and we write $b = a^{-1} \bmod n$, if it holds that

$$ab \bmod n = 1$$

For instance, the modular inverse of 2 modulo 5 is 3 and we can write $3 = 2^{-1} \bmod 5$ since $2 \cdot 3 \bmod 5 = 6 \bmod 5 = 1$. Note that the modular inverse is not always well defined. For instance, you cannot find the modular inverse of 2 modulo 6. That is, there is no solution to the equation $2x \bmod 6 = 1$. This is because a value a only has a modular inverse modulo n if a and n do not share any divisor larger than 1, or as we wrote above, if the greatest common divisor (gcd) of a and n is 1. Since 2 and 6 are both divisible by 2, $\gcd(2, 6) = 2$ and there is no inverse.

If you want to learn more about cryptography, it is a good idea to get some basic background in algebra. Algebraic abstractions such as groups, rings and fields are often needed to reason about modern cryptosystems.

5.2.3 Chosen Plaintext Security of RSA

The “vanilla” version of RSA described above clearly does not satisfy the notion of indistinguishability under chosen plaintext security. The easiest way to see this is that RSA as described above is deterministic and therefore if you encrypt the same message twice you will get the same result twice. To achieve meaningful security we need the encryption algorithm to include some additional value or nonce. Remember that in secret-key cryptosystems (such as AES in CBC mode), the nonce was called initialisation vector and was made public as part of the ciphertext. Note that in the case of public-key cryptography we cannot make the nonce public. The difference is of course that in public-key cryptography the adversary knows the public-key and is therefore able to try and guess the plaintext and encrypt it using the public-key and the nonce to see if it matches the ciphertext. Intuitively this is the reason why in public-key cryptography we pick the nonce to be a uniform random value large enough that it would be infeasible for the adversary to guess it correctly.

Now, we need a way to “mix” the message and the random nonce together to be suitable for RSA encryption. Here is a simple, but inefficient way, to get a secure version of RSA which uses a nonce:

1. Assume that the plaintext is a single bit $b \in \{0, 1\}$.
2. Pick a uniformly random value r between 0 and B such that $2B + 1 < n$.
3. Encrypt $c = (2r + b)^e \bmod n$.
4. When decrypting, output $b = (c^d \bmod n) \bmod 2$.

The solution above “wastes” a lot of the plaintext space, as it uses a whole RSA ciphertext (which is in the order of thousands of bits) to encrypt a single bit. While some ciphertext expansion is somehow necessary (to make room for the random nonce) we can certainly hope to do better than this.

5.2.4 Chosen-Ciphertext Security

Further complications arise if the adversary can manipulate the input to the decryption algorithm and see how it reacts. Contrary to what you might think at first, this is a very realistic possibility – for instance, if the decryption algorithm is being used inside a web server. When setting up a secure connection with such a web server, it usually expects to receive a ciphertext, which it will decrypt and then communicate to the client whether the decryption was successful, and if not, what error occurred. This opens the possibility for an adversary to choose any string they like and send this to the server, just to see whether the server can decrypt successfully, and if not, which error message it will send. This may reveal information on the secret key, and there are examples known where such an attack can actually break an encryption scheme. Such an attack is known as a *chosen ciphertext attack* (CCA).

The main way of ensuring CCA security is to create a cryptosystem in which the decryption fails unless the ciphertext was generated “honestly” using the encryption algorithm. The intuition for this is as follows: if the only way in which the adversary can produce a ciphertext that will be decrypted is by encrypting some message of their own choice, they will learn no new information from receiving the decryption oracle – the decryption oracle just tells back to the adversary something they already knew! This simple yet powerful idea can be used to build a CCA version of RSA as we will see below.

5.2.5 Optimal Asymmetric Encryption Padding

The Optimal Asymmetric Encryption Padding (OAEP) is a default industry standard to encrypt messages in a secure way using RSA, and is described in the PKCS series of standards from RSA Laboratories.

We will describe here some of the main ideas without delving into too many details: OAEP is an *invertible* function that takes as input a message m and some randomness r and produces an output $\text{OAEP}(m, r)$ which is suitable for encryption using RSA. This includes producing an output which is approximately the same size of the modulus n (for technical reasons, it is not secure to encrypt small messages using RSA). Since OAEP is invertible, the receiver can decrypt the “outer layer” of RSA and then invert the OAEP process to retrieve the original message and randomness. In other words, then using OAEP the flow is as follows:

1. The sender computes $x = \text{OAEP}(m, r)$, then generates $c = x^e \bmod n$.
2. The receiver retrieves $x = c^d \bmod n$, and then inverts $(m, r) = \text{OAEP}^{-1}(x)$.

Under the hood, OAEP allows to mix a medium size random string r with a longer message m into a “random looking” string of the right length that can be suitably encrypted with RSA. For concreteness, if the modulo n is 4096 bits long, the nonce r could be 256 bits and the message m could be 3600 bits (the numbers are only indicative). We will just say “short” for the length of r and “long” for the length of m . In a bit more detail, OAEP uses a function G to expand the short string r into a long random looking string, which is then XORed with m . Think of this G as a stream cipher from the previous sections.

Then, OAEP also mixes the message into the whole output, using a different function H which shrinks (a function of) the message into a short value of the same length of r . This function H can be seen as a cryptographic hash function, which will be defined in the next chapter. In addition, we append some 0 bits after the message before starting the process. This ensures that there is some “redundancy” that can be checked during decryption to ensure that no one has tampered with the ciphertext. At a high-level and ignoring many details, on input a message m and a random value r , OAEP does the following:

1. Let $u = G(r) \oplus (m, 0^\ell)$ where 0^ℓ is a string of ℓ zeros, think of $\ell = 128$ in practice.
2. Compute $v = H(u) \oplus r$.
3. Output (u, v) (to be then RSA encrypted).

One can see that OAEP can be inverted verifying that the receiver on input (u, v) can recompute:

1. $r = H(u) \oplus v$.
2. $(m, \text{check}) = G(r) \oplus u$.
3. Output m if $\text{check} = 0^\ell$ and abort otherwise.

The main idea is that if any bit of (u, v) is changed during transport by the adversary, the value check will not be equal to 0^ℓ except with probability $2^{-\ell}$.

While the details above might be more than what you can digest at this point in your study, the important fact for us is to know that usage of OAEP with RSA has been proven to be secure even in a threat model where the adversary can do a CCA attack, i.e., play with the decryption equipment as sketched in the previous paragraph. The intuition here is that if the adversary tries to construct some ciphertext in some “illegal” way, decryption will always fail because the string coming out of the RSA decryption will not be in the correct format.

In summary: public-key schemes, including RSA, should never be used in practice without use of a secure padding method such as OAEP! Well designed software packages ensure this by default, but you cannot be completely sure unless you check.

5.2.6 Hybrid Encryption

In practice no one uses public-key encryption schemes to encrypt large messages i.e., messages that do not fit within the OAEP padding as described above. The reason for this is that public-key systems are typically slower than secret key systems by several orders of magnitude, and hence cannot be used directly for bulk encryption of data. What one usually does is therefore to use the public-key system just once to communicate a key for a secret-key system, which is then used to encrypt the actual data. This is sometimes known as **key enveloping**, **key encapsulation** or **hybrid encryption**. In practice, it might look something like this: To encrypt a large plaintext M :

1. Generate a random AES key k .

2. Encrypt M using k and AES (using a suitable mode of operation e.g., AES in counter mode).
3. Encrypt k using RSA (using a suitable padding scheme e.g., OAEP)

On the receiving side, the receiver will first retrieve k from the RSA ciphertext, and then M from the AES ciphertext.

5.2.7 Elliptic Curve Cryptography

Other examples of public-key systems are the ElGamal crypto system based on the discrete logarithm problem. (We will introduce the discrete logarithm problem and its applications to cryptography in more detail later in the book). One way of implementing this system is based on a notion from algebraic geometry known as Elliptic Curves. ElGamal based on elliptic curves is known as *Elliptic Curve Cryptography* (ECC).

It is important to note that because of the different nature of the computational problems involved in different public-key systems, one cannot compare their security based on key sizes alone. In particular, systems based on ECC have the interesting property that no known attacks on them perform significantly better than brute force attacks. That is, a k -bit key can be broken in time $2^{O(k)}$, which is much slower than what can be done for RSA.

Therefore ECC can be used with much smaller keys than RSA. In practice today an ECC public key can be around 256 bits offering a security similar to RSA with key-size of 2048 – 4096 bits. For this reason, in the last few years, ECC is quickly replacing RSA as the de-facto standard and is used in many places where small keys and ciphertexts are important.

5.2.8 Quantum Computing and Encryption Security

Public-key cryptography based on both RSA and ECC is not secure if the adversary has access to a suitable *quantum computer*. Many experts believe that the question is not *if* but *when* such quantum computers will become a reality, and therefore cryptographers are currently working on designing *post-quantum* public-key cryptosystems, which are cryptosystems that are secure even against quantum computers. Note that while such computers do not exist yet an adversary might decide to record large amount of encrypted communication today to be decrypted later, once quantum computing becomes available – this is known as the *store now, decrypt later* threat model. Therefore it is important that the transition to post-quantum cryptography takes place much earlier than the expected advent of quantum computers.

In particular, in 2017 NIST started an open competition to identify alternatives to RSA and ECC which are secure even if the adversary has access to a quantum computers. As of 2024, the winning encryption schemes based on *lattice-based cryptography* are starting to be deployed over the internet by some of the major browsers and encrypted communication apps.

Note however that on the other hand the impact of quantum computers on the security of secret-key cryptosystems is very limited, and adjusting the key-length of current systems (e.g., switching from 128 to 256 bit keys in AES) is enough to ensure security even in a post-quantum world.

Finally, we point the difference between post-quantum cryptography (PQC) and quantum-key distribution (QKD). PQC is a set of cryptographic techniques that can be used on today's computers and using today's communication technology (the internet), but is resistant to quantum attacks. Therefore PQC can be implemented today to protect against future threats. On the other hand QKD is a technology that exploits the law of physics to construct encryption schemes with unconditional security that do not rely on mathematical assumptions. Unfortunately, QKD requires the sender and the receiver to be directly connected by quantum channels,

and at the time of writing QKD has managed to send secure messages within a radius of at most $100km$. While this can have applications in closed networks where it is possible to establish such connections (e.g., a military base, government buildings, etc.) it does not seem to be a suitable replacement to for mass encryption over the internet or mobile networks.

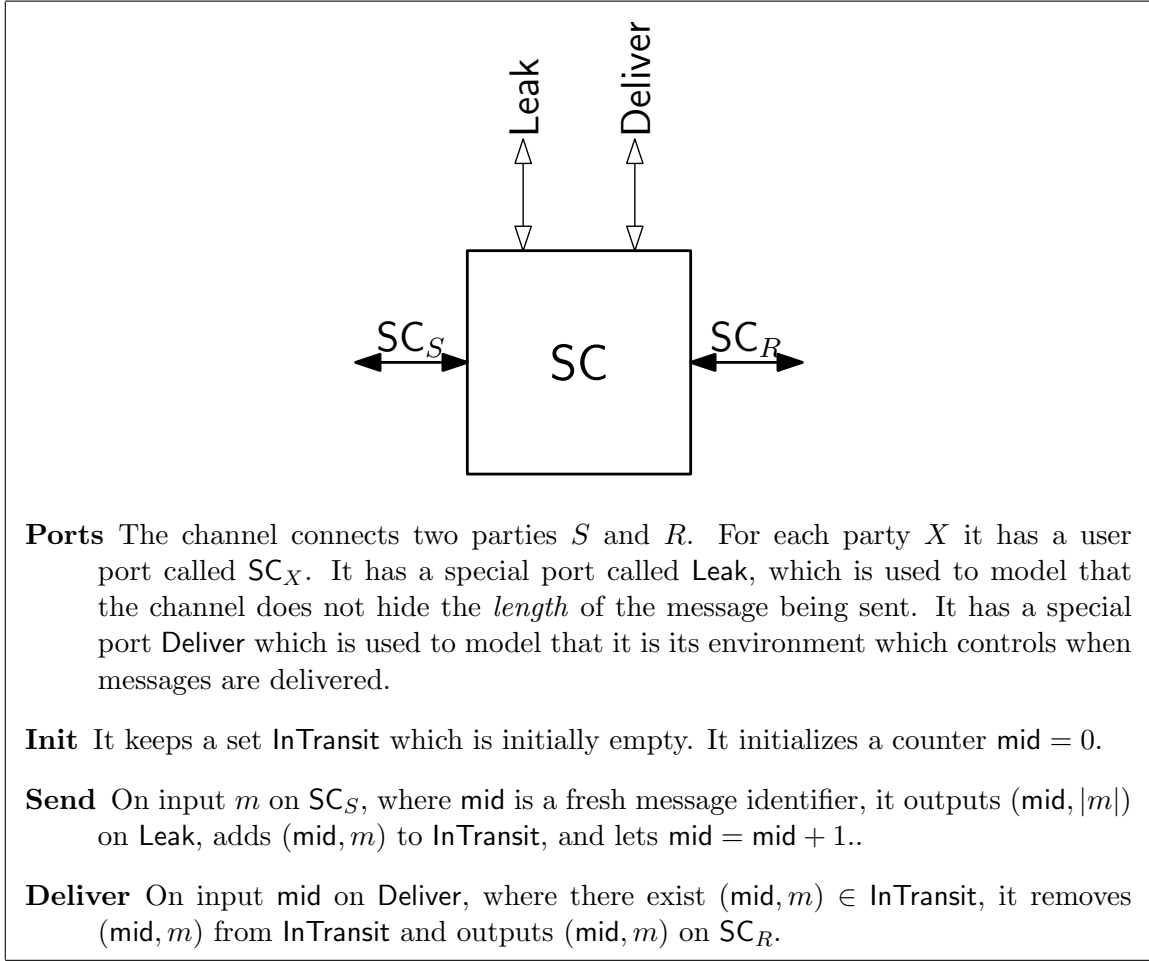


Figure 5.9: The SC network resource modeling a secure channel.

5.3 Securely Implementing a Secure Channel*

We now relate the notion of a secure symmetric encryption scheme with the notion of securely implementing a secure channel. This section can safely be skipped. It is included for the reader interested in how the formal model based on specification via interactive agents relate to the more informal presentation in this chapter.

We use the network resource SC in Fig. 5.9 to model a secure channel. Notice that as opposed to AC the network resource SC does *not* leak the message m . That is what makes it a model of a secure channel. It does, however, leak the length of m . This is because we cannot expect an implementation to completely hide the length of the message being sent. An encryption of a single bit is hopefully shorter than an encryption of a two hour movie.

Recall now that the canonical way we envision to use a symmetric encryption scheme is as in Fig. 5.1. We could ask the question: when it holds for the protocol $\Pi_{G,E,D}$ in Fig. 5.1 that

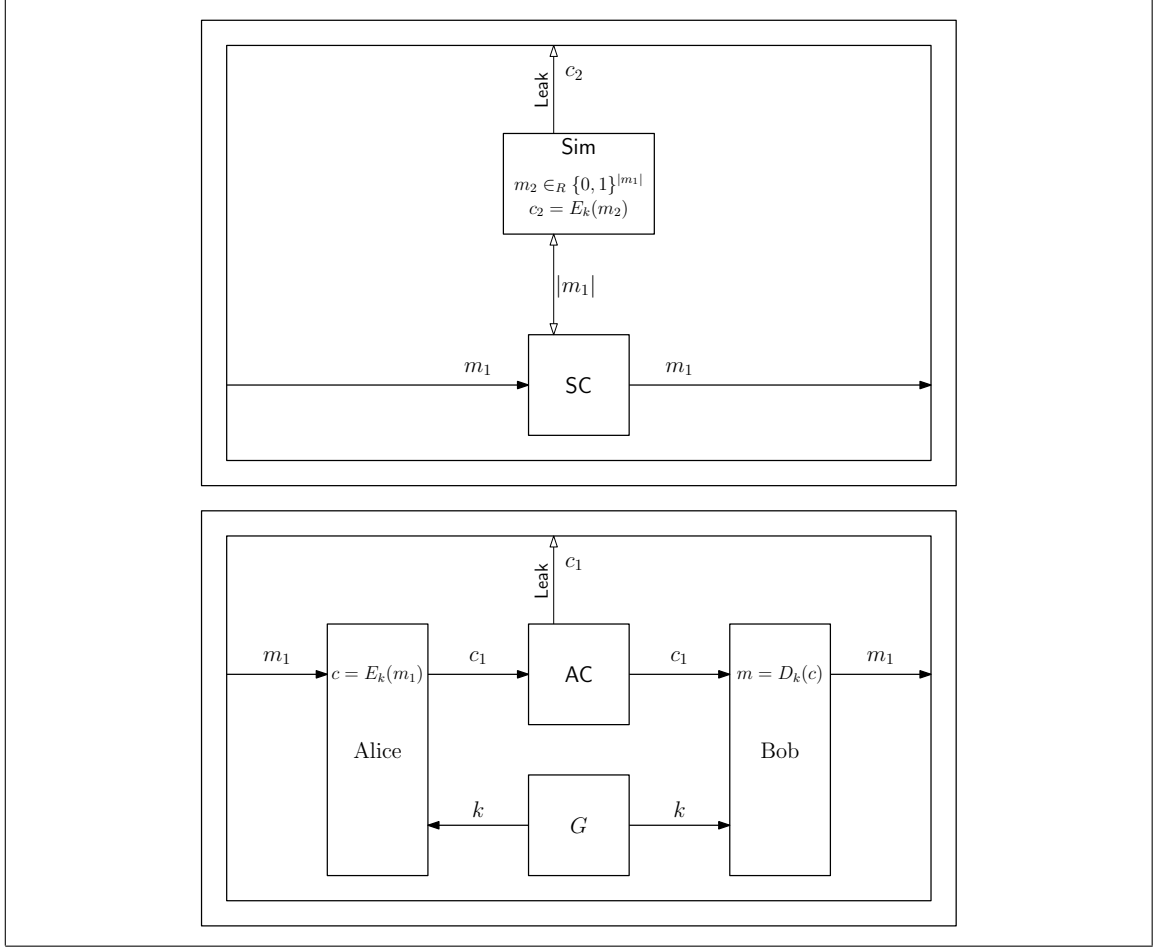


Figure 5.10: $\Pi_{G,E,D}$ and $SC + Sim$ in juxtaposition. The simulator outputs a uniformly random message m_2 of the same length as m_1 . This ensures that $\Pi_{G,E,D} \equiv SC + Sim$ when (G, E, D) is the one-time pad and only one message m_1 is sent and $|k| = |m_1|$.

it securely implements SC , i.e., when is it the case that $\Pi_{G,E,D} \sqsupseteq SC$? Recall that to prove $\Pi_{G,E,D} \sqsupseteq SC$ we need to come up with a simulator Sim such that $\Pi_{G,E,D} \equiv SC + Sim$. Recall that when a message m_1 is sent in $\Pi_{G,E,D}$ the channel AC leaks $E_K(m_1)$. When a message m_1 is sent in $SC + Sim$, then SC leaks only $|m_1|$. Fig. 5.10 shows $\Pi_{G,E,D}$ and $SC + Sim$ in juxtaposition. So, the job of Sim is to get as input $|m_1|$ and then output $E_K(m_1)$. This might seem impossible at first: the simulator is given only the length of m_1 , so it cannot compute m_1 itself, as there are many messages with the same length. But recall that when k is unknown and (G, E, D) is the one-time pad, then $E_K(m_1)$ and $E_K(m_2)$ have the exact same distributions for all m_1 and m_2 of the same length. Therefore Sim can just output $E_K(m_2)$ for some m_2 with $|m_2| = |m_1|$. The simulator in Fig. 5.10 picks a uniformly random such message m_2 , but any message of the right length will do.

It should be clear that if (G, E, D) satisfies indistinguishability under chosen-plaintext attack, then the protocol in Fig. 5.11 securely implements SC . The simulator will simply encrypt random messages as in Fig. 5.10.

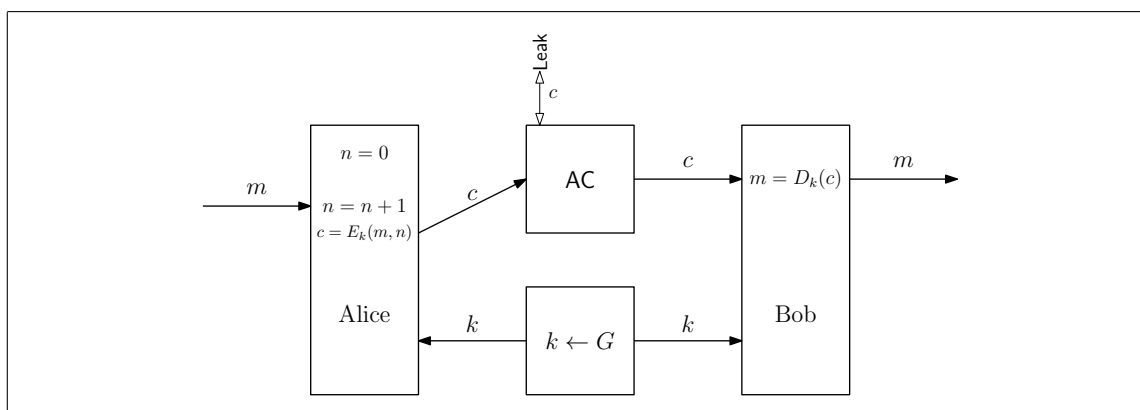


Figure 5.11: An illustration of how a symmetric encryption scheme (G, E, D) is used for sending multiple messages. It works as in Fig. 5.1, but each encryption also uses a nonce, in this particular example implemented as a counter. Initially the counter is set to $n = 0$. Each time a new message is sent, it is incremented and given to the encryption algorithm E as input.

5.4 Exercises

Exercise 5.1 (One-Time Pad Theory) A security officer in a bank has read a book on cryptology and learned that the one-time pad is completely unbreakable. So they insist that the payment orders sent from bank costumers over the net to the bank should be encrypted using the one-time pad (hardly a practical idea, but in this exercise we ignore the difficulties with distributing enough bits for the key). Assume that the format of such payment orders is as follows: the amount in kroner a to transfer from one account to another is in the range $[0, 2^{21})$ and is therefore stored in bits 0, 20 of the string sent, with the least significant bit in position 0.

Shortly after, an employee of the well known company Hackers Unlimited intercepts on the net an encrypted payment order c . Based on the time at which it was sent, they guess it contains a request to transfer their salary for next month to their account.

1. Assume the original salary was less than a million kr. per month. What is the value of bit 20 in the payment order? The attacker is able to stop the transfer of c and send instead a different ciphertext c' . Show how the attacker can construct c' given their knowledge of c and the original message in such a way that they will receive more than a million kr. extra next month.
2. Is the security problem you have seen here a confidentiality problem or an authenticity problem?
3. This book claims that the one-time pad cannot be broken, and yet we have identified a security problem here. Why is this not a contradiction?
4. Does your attack work on any of the other secret-key encryption schemes from the chapter? Stream ciphers? AES in CBC mode? AES in counter mode? Why or why not?
5. (Optional) Suppose the amount was encrypted using vanilla RSA instead. Is there any strategy that the employee can use to increase their salary in this case?



Exercise 5.2 (When to use Encryption?) A manufacturer of windows (the kind you have in buildings, that is!) is about to launch a new product line: remote controlled windows that can be opened by just pushing a button on a little handheld remote control unit. The controller communicates with a window unit that can then open the window. Very useful for windows that are hard to reach physically. However, insurance companies have said that to insure objects in buildings with these windows, they will demand that it is ensured that a window can only be opened by the designated controller – and not by a controller that has been stolen elsewhere, for instance. Someone suggests that all communication between controller and window unit should be encrypted. Is this a good idea? Why or why not? ▼

Exercise 5.3 (Insecurity of Deterministic Encryption) This exercise is about why deterministic public-key encryption cannot be secure. Consider the following way to use RSA: the input is a sequence of letters in the English alphabet. Each letter is converted to a number such that a becomes 0, b becomes 1, etc. So we represent the plaintext by a sequence of numbers, each in the interval $[0..25]$. Finally, each number is encrypted individually using the RSA public key (n, e) . Describe how this encryption scheme can be easily broken without having to factor n . Test your method by finding the cleartext for the ciphertext

365, 0, 4845, 14930, 2608, 2608, 0

and public key $(18721, 25)$, without factoring 18721. ▼

Exercise 5.4 (CBC Decryption) Consider the CBC mode of use for block ciphers. Describe how to decrypt a CBC ciphertext. Suppose a ciphertext consisting of many blocks is sent, but during transmission, 1 bit in a single block is changed. How large a part of the resulting decryption result might be incorrect? What if CTR mode is used instead? ▼

Exercise 5.5 (CBC-mode with predictable IV) In this exercise you are asked to figure out why CBC-mode is not secure according to the definition of security presented in Fig. 5.4 if the IVs are not chosen in a random way. Assume e.g., that the IVs are implemented using counters. Can you find a sequence of messages $m_1, \dots, m_n$ that the adversary can submit to the encryption oracle to distinguish between O_0 and O_1 ? (Hint: can you choose m_1, m_2 such that the left oracle O_0 returns $c_1 = c_2$? What does the right oracle O_1 return on the same inputs?). ▼

Exercise 5.6 (On The Need For Padding in RSA) This exercise is about the problem that occurs when one needs to encrypt a secret 128-bit AES key under RSA. As discussed in the chapter RSA moduli today are at least 2048 bits long. Since AES keys are random anyway, one might be tempted to assume that it might be safe to directly encrypt them using RSA (after all, the “guessing” attack is not relevant in this case). You will see here why this is not true, and why it is so important to use OAEP when encrypting AES keys using RSA.

Assume that we use the variant of RSA with $e = 3$, and therefore we encrypt the key $k \in [0, 2^{128})$ (interpreted as an integer with 128 bits) simply as

$$c = k^3 \bmod n$$

Can you find a way of easily recovering k from c ? Hint: consider the difference between integer values k^3 and $k^3 \bmod n$.

Now suppose the public exponent is instead 17, and the modulus n has length 2150 bits. Show that the adversary can compute k from $k^{17} \bmod n$ while spending a factor roughly 2^{26} more time than in the previous question – which is certainly still feasible. ▼

Exercise 5.7 (DES vs. AES Security) Assume you have obtained some ciphertext that has been encrypted with the DES algorithm, and that you have also somehow obtained the corresponding plaintext. This means you can test whether a given DES key is the unknown key you are looking for. Assume you can test one key in time 10^{-10} seconds. How many days would it take to test all DES keys? What would the answer be if we had used AES instead? ▼

Exercise 5.8 (Meet-in-The Middle Attack) In view of the fact that the 56 bits of key in the DES algorithm is not quite enough it was suggested that one could just encrypt each plaintext block twice under two independently chosen keys K_1, K_2 , that is, we would encrypt 64 bit block M to ciphertext C as $C = E_{K_2}(E_{K_1}(M))$, where E here stands for DES encryption. It seems that this is just a new block cipher, but now with $2 \cdot 56 = 112$ bits of key.

It turns out that this is not really true. Given enough memory space and a pair M, C of corresponding plaintext and ciphertext, one can break this cipher much faster than by trying all 2^{112} possibilities for the key. Here is the start of the algorithm:

1. Let variable K run through all possible 56-bit DES keys, for each possibility, compute $D_K(C)$ (the decryption of C under key K). Store all these results in a sorted table.
2. ??

Fill in step 2, to obtain an algorithm that finds the correct key K_1, K_2 using a total of $2 \cdot 2^{56}$ DES encryptions and decryptions.

This so called *meet-in-the-middle attack* is the reason why DES these days is recommended for use only in the *two-key triple-DES* construction, where we set $C = E_{K_1}(D_{K_2}(E_{K_1}(M)))$. Would the attack above work against this cipher? (Note that the design of Triple-DES makes it backward-compatible with regular DES by setting $K_1 = K_2$, due to the alternation of E , D and E).

Exercise 5.9 (Permutation Ciphers) Consider a cryptosystem where plaintext messages are sequences of letters from some alphabet – for concreteness let us assume the English one, which has 26 letters and that messages are written in English. A key is a permutation of the alphabet, i.e., a 1-to-1 mapping ϕ of the alphabet to itself. We encrypt by replacing the i 'th letter m_i in the plaintext by $\phi(m_i)$. How many possible keys does this cryptosystem have? Is it possible to perform exhaustive key search against this cryptosystem? Argue that it is not necessary to do exhaustive key search to find information on the plaintext from the ciphertext, by proposing one or more methods that might help to do it faster than by exhaustive search. ▼

Exercise 5.10 (Understanding Modular Arithmetic) The public key for the RSA cryptosystem is a pair of numbers (n, e) where $n = pq$, the product of two primes p, q . The secret key is (n, d) . Suppose we have $p = 3, q = 11$ so that $n = 33$ and messages are numbers in the interval $[0..32]$. It turns out that $e = 7, d = 3$ can be used. Encrypt the messages 2 and 5 and check that decryption works as expected. ▼

Exercise 5.11 (RSA and AES Encryption)

Goal. The goal of this exercise is twofold. First, you will implement RSA key generation, encryption, and decryption yourself. Second, you will use Go's standard cryptographic library to encrypt and decrypt data using AES.

Part 1: RSA encryption. Create a Go package with methods `KeyGen`, `Encrypt`, and `Decrypt` that implement RSA key generation, encryption, and decryption. Your solution should use integers from the `math/big` package.

The `KeyGen` method should take as input an integer k , such that the bit length of the generated modulus $n = pq$ is k . The primes p, q do not need to be primes with certainty; they only need to be probable primes.

The public exponent must be $e = 3$. This means that the primes p, q must satisfy

$$\gcd(3, p-1) = \gcd(3, q-1) = 1.$$

Recall that d must satisfy

$$3d \bmod (p-1)(q-1) = 1,$$

i.e., $d = 3^{-1} \bmod (p-1)(q-1)$.

Suggested API (you can change it).

- `KeyGen(k int) (n, e, d *big.Int, err error)`
- `Encrypt(m, e, n *big.Int) *big.Int`
- `Decrypt(c, d, n *big.Int) *big.Int`

Implementation hints. You may find the following useful:

- The `Mod` and `ModInverse` methods in `math/big`.
- Cryptographically secure randomness from `crypto/rand`, in particular the function `rand.Prime`.

Testing (RSA). Test your solution by verifying at least:

- that the modulus has the required bit length,
- that encryption followed by decryption of random plaintexts returns the original plaintext.

Plaintexts and ciphertexts can be treated as numbers; no string conversion is required.

Part 2: AES encryption. Implement methods `EncryptToFile` and `DecryptFromFile` that encrypt and decrypt data using AES in counter mode with a supplied key.

Suggested API (you can change it).

- `EncryptToFile(key []byte, plaintext []byte, filename string) error`
- `DecryptFromFile(key []byte, filename string) ([]byte, error)`

The method `EncryptToFile` should write the ciphertext to a file. The method `DecryptFromFile` should read the ciphertext from a file, decrypt it, and output the plaintext.

Test your solution by encrypting an RSA secret key to a file, decrypting it, and checking that the recovered key can still be used for RSA decryption.

Go's official `crypto` package provides AES support via `crypto/aes`. The key passed to `aes.NewCipher` must have length 16, 24, or 32 bytes.



Exercise 5.12 (Implementing a Secure Channel I*) Consider a network resource SC' defined as SC except that it does not leak $|m|$ on $Leak$. It only leak a fixed message $A_MESSAGE_WAS_SENT$. Prove that it is not the case that $\Pi_{G,E,D} \sqsupseteq SC'$ when G, E, D is the one-time pad encryption scheme. Hint: consider the environment which flips a bit b and sends 1 if $b = 0$ and 11 if $b = 1$. ▼

Exercise 5.13 (Implementing a Secure Channel II*) Consider the network resource SC in Fig. 5.9. Let $\Pi_{G,E,D}$ be the protocol from Fig. 5.7. Argue that when (G, E, D) is indistinguishable under chosen-plaintext attack then $\Pi_{G,E,D} \sqsupseteq SC$. ▼

Chapter 6

Authenticity (2025)

Reading Guide *This section introduces some concepts and techniques of authenticity. All of the chapter is curriculum and can be queried at the oral exam, except Section 6.6.*

Contents

6.1	Cryptographic Hash Functions	164
6.1.1	The Birthday Bound	165
6.1.2	Second Preimage Resistance	165
6.1.3	Hash-Functions in Practice	166
6.2	Authenticity, Secret-Key Systems	167
6.2.1	Unconditional Authentication	167
6.2.2	Practical systems and exhaustive search	168
6.2.3	Example MAC algorithms	169
6.2.4	Authenticated Encryption	170
6.3	Authenticity, Public-Key Systems	170
6.3.1	Security of public-key systems	171
6.3.2	Difference to secret-key: Transferability, Non-repudiation	171
6.3.3	RSA Digital Signatures – Vanilla Version	172
6.3.4	Attack on Vanilla RSA Signatures	172
6.3.5	Hash-and-Sign Signatures	173
6.3.6	Using Digital Signatures in Practice	173
6.4	Replay attacks	174
6.5	Certificate Authorities	175
6.6	Implementing an Authenticated Channel*	177
6.7	Exercises	179

In this chapter we will introduce several techniques that allow to produce authentication tags to be used in our communication. Such a tag can be appended to the messages that are exchanged by the parties, and used to verify that the communication has not been tampered along the way. As in the previous chapters, we will start with secret-key systems for authentication, to then move to public-key systems. When considering authentication, the difference between secret-key systems and public-key systems is deeper than what was the case for encryption. This is because public-key authentication systems, also known as **digital signature schemes**, can be used in applications in which one wants to prove to a third party that someone has sent some message. As we shall see, this is not the case for symmetric-key authentication schemes,

also known as **message authentication codes (MAC)**. This wider difference between the two kind of authentication systems is reflected in the different terminology (MACs vs. signatures) and notation used in the two cases.

From a technical point of view, this chapter will re-use most of the cryptographic technologies seen in the previous chapter (block-ciphers, RSA, etc.). The similarity in the underlying tools should not fool the reader into thinking that authenticity and confidentiality mechanisms are one and the same. They are deeply different tools which solves different problems, and in practice it is crucial to understand which tools should be used in which case.

Before talking about authentication mechanisms, we introduce a new tool called **cryptographic hash functions**. Hash functions are a fundamental building block in authentication schemes and many other cryptographic primitives.

6.1 Cryptographic Hash Functions

We give a brief introduction to **cryptographic hash functions**. We stress the term *cryptographic* here because you might have encountered different kind of hash functions, for instance in an algorithm course. Even if later in the chapter (or the book) we only write “hash function” for brevity, we still mean a cryptographic hash function as described in this section.

What’s common for all hash functions (cryptographic or not) is that they can “shrink” inputs of long size into short “digests”. Intuitively these short digests should be a short representation of the entire original input in such a way that the hash of different inputs results in different digests. When using cryptographic hash functions, we require that this property holds in a very strong sense, even against a motivated adversary that is actively trying to make the system fail (whereas in non-security related applications it might be fine to use hash functions that only satisfy the properties when inputs come from a “natural” source).

In general, a cryptographic hash function h should have the following properties:

Shrinking It should be able to take as input a message x of any length and it should produce an output of fixed length k .

Efficient It should be fast to compute (e.g., same speed as symmetric-key encryption schemes).

One-Way Function Given an output $y = h(x)$ computed on a long-enough random input x , it should be computationally unfeasible for an adversary to recover x .

Collision Resistant It should be computationally unfeasible for an adversary to find a collision i.e., two inputs x_1, x_2 such that $x_1 \neq x_2$ but $h(x_1) = h(x_2)$.

Some remarks: When we write “computationally unfeasible” above we mean that it should take some unrealistically large amount of cryptographic operations for an adversary to break the property (for instance time comparable to what it takes to brute force an AES key – 2^{128}). You have in fact already encountered the notion of “one-way function” in the confidentiality chapter (even if we did not use the term), when working with public-key systems (remember that the public key pk is typically computed from the secret key sk , and that it should be hard to recover sk from pk). Note finally that the fact that the function is shrinking clearly implies that there are plenty of collisions e.g., pairs (x_1, x_2) with $x_1 \neq x_2$ s.t., $h(x_1) = h(x_2)$. This simply follows from the fact that there are so many more possible inputs than outputs. However, we demand that such collisions are hard to find.

Cryptographic hash functions are commonly used for integrity checks: if you download a big file from the internet, sometimes the website will list a cryptographic digest next to the file. Once you have downloaded the file, you can recompute the hash on your machine and check if

the value you computed matches with the one on the webpage. It is important to understand that this is only a countermeasure against accidental communication faults: if the adversary controls the website that publishes the file you download, they can also clearly control the digest published next to the file.

However, if you know you are receiving the hash from a “trusted source” (maybe the sender reads it to you over the phone) and you verify that the file you received has the same hash, then collision resistance of hash functions implies that you received the right file. In some sense, hash functions can be seen as a way of turning the problem of authenticating large files into the problem of authenticating short digests.

6.1.1 The Birthday Bound

There is an important relationship between the output size k of a hash function and (an upper bound on) the time it takes to compute a collision, namely it is always possible to find collisions for a hash function with output size k in time approximately $2^{k/2}$. The reason for this is the so-called “birthday attack” that we describe here.

The attack is really simple, and just involves computing the hash function $t = 2^{k/2}$ times on random inputs $x_1, \dots, x_t$ and hope that the list of outputs $y_1, \dots, y_t$ contains a collision.

We now estimate the probability that a collision will be found. We first make the simplifying assumption that the outputs $y_1, \dots, y_t$ are uniformly random k -bit strings, which is a good enough approximation to simplify our analysis (cryptographic hash functions are designed to provide “random looking” outputs, and we fed them with long-enough uniformly random input $x_1, \dots, x_t$).

Then, the probability that any pair of outputs i, j collide is equal to $\Pr[y_i = y_j] = 2^{-k}$. But since we are just interested in finding *any* collision in the list, we count the number of possible pairs of outputs, which is approximately $t^2/2$ (more precisely, $t(t-1)/2$). We make a new simplifying assumption and assume that the probability of a collision in each pair is independent from the other pairs (this is not technically true, but again, it is good enough for our argument) and therefore we get that there is a very high probability of finding a collision in the list:

$$\Pr[\text{There exist } i \neq j \text{ such that } h(x_i) = h(x_j)] \approx t^2/2 \cdot \Pr[h(x_i) = h(x_j)] \approx t^2/2 \cdot 2^{-k} \approx 1/2$$

The curious name of this attack stems from the so called “birthday paradox” (which is not actually a paradox, but just a somewhat surprising fact from probability theory), which talks about the number of people you need to have in a room before you have significant probability of two of them sharing a birthday. While the initial intuition might say that you need almost as many people as there are days in a year, you can calculate (using more precise calculations than we did above) that it is enough to have 23 people in a room to have more than 50% probability of a collision.

6.1.2 Second Preimage Resistance

It can be shown (as you will do in the exercise) that the list of properties above is redundant, and any collision resistant function must also be one-way. This means, it is potentially *easier* to find collisions in a hash function than to be able to invert it. (As we will see below, there are practical hash functions whose collision resistance has been broken but are still believed to be one-way).

One-way hash functions are also sometimes known as **preimage resistant** functions, since it should be hard to find a preimage for a given output. There is a third security requirement for

hash functions that lies in between one-wayness/preimage resistance and collision resistance, namely the notion of **second-preimage resistance**:

Second-Preimage Resistance A hash function is second pre-image resistant if it is computationally unfeasible for an adversary, on input some value x_1 , to find a different x_2 such that $h(x_1) = h(x_2)$.

Clearly, breaking second-preimage resistance is at least as hard as breaking collision resistance: if given x_1 you can find a second-preimage x_2 then you also found a collision, namely (x_1, x_2) . Intuitively, finding second-preimages is usually harder than finding collisions since when searching for collisions the adversary can freely choose both of the values, while when searching for second-preimages the adversary has to find a match for x_1 .

It is also interesting to understand the relationship between second-preimage resistance and one-wayness/preimage resistance, and you will be asked to do so in the exercises.

6.1.3 Hash-Functions in Practice

Modern hash functions must have output length of at least 160 or, better, 256 bits. This is a direct consequence of the birthday attack described above. We provide a brief history of hash functions and their security below:

MD5 (for message-digest algorithm) was an historically widely used hash-function, with output size 128 bits. It should already be clear from its output size that MD5 is not suitable for modern applications (due to the birthday bound). More importantly, the function has been completely broken, and collisions can be found in matters of seconds on regular computers. Finding MD5 in any security-related application should immediately raise a red-flag.

SHA-1 is part of the SHA family (for secure-hashing algorithm), a family of hash functions standardized by the US National Institute for Standard and Technologies (NIST). SHA-1 was designed by the National Security Agency (NSA) to replace MD5, and has output length of 160 bits. Since 2005, the cryptographic community has been pointing out weaknesses in SHA-1. Unfortunately, this has not been taken seriously by the wider community until 2017 when a team of researchers from Google and CWI Amsterdam found the first known collision in SHA-1. This has required a significant computational effort, but their results can be re-used to produce further collisions thanks (among other things) to redundancy in file-encodings. Even after 2017, some prominent voices have argued that the attack was not practical enough since it only found collisions and not second-preimages. The crucial lesson to learn here is that, as the saying goes, *attacks only ever get better*, and replacing cryptographic algorithms in deployed software can take a long time. Therefore, one should start reacting when the first crack is visible in the wall, and not when the whole house comes down.

SHA-256 is the natural successor of SHA-1. It has output length of 256 bits and it is still considered secure. However, its internal mechanics resemble those of SHA-1, which prompted NIST to look for suitable alternatives.

SHA-3 is the first entry in the SHA family that was chosen by means of a public international competition. Cryptographers from all over the world, including Denmark, submitted their proposals and spent years trying to evaluate the security of each others' proposals. Finally the Keccak submission from a team of Belgian researchers was chosen as the basis for the

current standard. The main reason behind the choice of Keccak is that its internal design is very different from SHA-256, so we do no longer “put all our eggs in one basket”. The SHA-3 design, known as the “sponge construction”, allows to have variants of SHA-3 with different output size (including 256, or 512 bits providing security against up to 2^{256} running time). It can also be naturally used to provide outputs of arbitrary length, which is very useful in certain applications.

6.2 Authenticity, Secret-Key Systems

A secret-key system for authenticity consists of three algorithms (G, MAC, V) , where G generates a key, MAC is used to authenticate a message (MAC stands for Message Authentication Code), and V is used to verify a received message. G usually produces a key by just outputting a random bit string of fixed length. MAC takes as input a message m and a key k and outputs a message authentication code (MAC), $c = \text{MAC}_k(m)$. Then the idea is to send (m, c) to the receiver who will run V on input (k, m, c) . The result $V_k(m, c)$ is either $\top$ (accept) or $\perp$ (reject).

An authentication scheme must have the property that if no one tried to modify the message, the receiver will accept, i.e., we demand that

$$V_k(m, \text{MAC}_k(m)) = \top$$

always. Of course, this condition says nothing about security. For this, we demand the following: the adversary is allowed to specify any number of messages he wants, say $m_1, \dots, m_t$ and he is given valid MACs $c_1, \dots, c_t$ for these messages. He is also allowed to specify pairs of form m, c and will be told if c is a valid MAC on m (he cannot decide himself as he does not know the secret key). We now have the following (informal) definition:

Definition 6.1 Consider any adversary who runs in time much less than what exhaustive key search would take. The authentication scheme is secure if no such adversary can play the game specified above and in the end produce a message m_0 and a MAC c_0 such that $V_k(m_0, c_0) = \top$ and $m_0 \notin \{m_1, \dots, m_t\}$.

In more human language: even if the adversary has seen some number of messages with valid MACs, he still cannot come up with a message that was not sent and a valid MAC for that message. This is the case even if the adversary can choose the valid messages to be sent.

Note that there is no requirement that the MAC algorithm should be randomized i.e., it is perfectly fine that, fixed a key k and a message m , a MAC scheme always outputs the same MAC c . This was very different for encryption: given two ciphertexts it should be difficult to tell if they are an encryption of the same message or not.

Note also that a MAC typically has a fixed size independent of the size of the message m . Again this was very different for encryption: since it should be possible to recover the plaintext from the ciphertext, the ciphertext must be at least as long as the plaintext. But in authentication there is no requirement of extracting the message from the MAC. Instead, one sends the message and the code, and the receiver has to check that they are valid together.

6.2.1 Unconditional Authentication

Just as for encryption, there are unconditionally secure ways to do message authentication, that are, however, quite useless in practice. For instance, if there is a finite number of different messages that could be sent, the sender and receiver could agree in advance on a table containing for each message a random independently chosen t -bit MAC. The table then functions as the

key. Of course, an adversary cannot find a correct MAC for a message that was not sent, except with probability 2^{-t} , regardless of how much computing power they have. Note that seeing other messages with legal MAC's does not help the adversary: Since the MAC's were independently chosen, faking the MAC of a new message requires them to guess a random t -bit string.

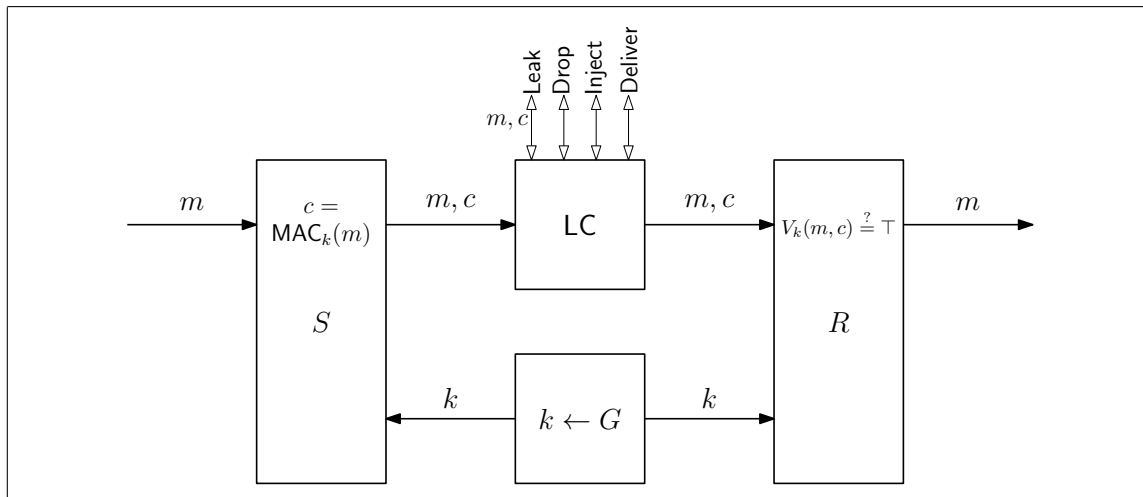


Figure 6.1: An illustration of how a symmetric authentication scheme (G, MAC, V) is used to send messages in an authenticated way. The protocol tries to implement a lossy authenticated replay channel LARC. Alice has to send a message m to Bob over an unauthenticated channel which leaks the transmitted data and which allows the adversary to inject messages. Somehow they managed to run the key generator G in the past and learn a common key k . When m arrives, Alice can compute and send a MAC $c = \text{MAC}_k(m)$. This MAC is allowed to leak. When receiving m, c , Bob can check that $V_k(m, c) = \top$ and output m if this is the case. It is essential that k is known only by Alice and Bob. We call the protocol $\text{MAC2LARC}_{G, \text{MAC}, V}$. To make Bob deliver a message m_0 never sent by Alice the adversary would have to inject a message (m_0, c_0) such that $\text{MAC}_k(m_0, c_0) = \top$ without Alice having MACed m_0 and without knowing k . Security of a MAC scheme exactly states that this is hard.

This system has the property that the key gets larger the more possible messages we have, and if we wanted to send, say n messages, we would need n tables, since just like for the one-time pad, we can not reuse the key. There are systems for unconditional authentication that are not quite as inefficient as this simple example, but there is no way around the fact that you need more key material, the more messages you want to be able to send.

6.2.2 Practical systems and exhaustive search

For real-life applications, it is clear that a table as above will be much too large to handle. We need systems where we can use a small, fixed key, so that there will be much fewer possibilities for the key than there are possible messages. This means that after the adversary has seen a few valid messages and MAC's, the key is most likely uniquely determined: only a single key-value would lead to MAC's consistent with those the adversary has seen. The key can then be found by exhaustive search: the adversary simply runs through all possibilities for the key. For each candidate, the adversary generates MAC's for all messages that were actually sent. If all these MAC's are identical to those that were sent by the legitimate sender, the adversary assumes they found the correct key. To ensure that such an exhaustive search is infeasible, the same

constraints on key lengths (≥ 128) holds for this type of secret-key system as for the case of confidentiality. In addition, the MAC itself cannot be too short, otherwise an adversary might be able to simply guess a correct MAC for a falsified message. Usually, MAC's of 64 bits (and preferably more) are used.

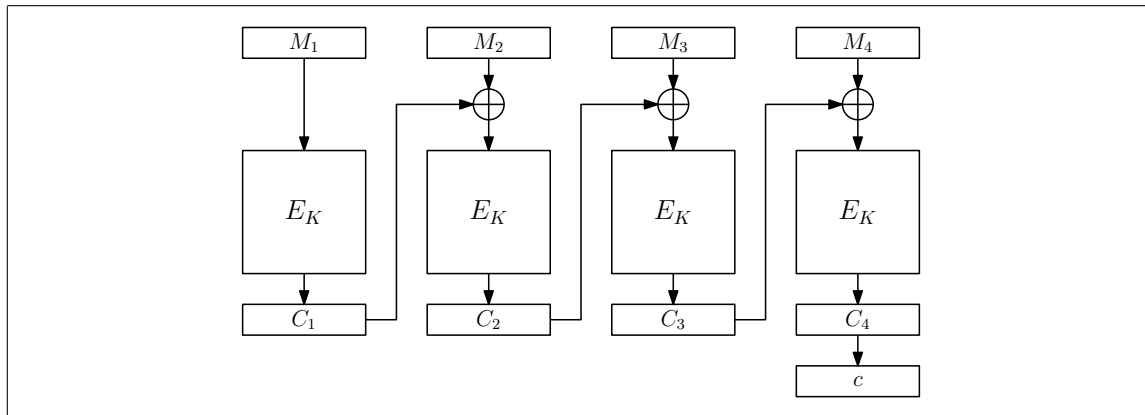


Figure 6.2: An illustration of CBC-MAC. In this example, four blocks are MACed. Only the last block is the MAC.

6.2.3 Example MAC algorithms

We describe two examples of MAC algorithms that are widely used in practice, but neglect many details to simplify the presentation:

HMAC: The conceptually simplest construction for a MAC is probably HMAC, which is also a quite popular choice in Internet standards. HMAC builds a secure MAC from any secure cryptographic hash function H .

In its simplified form, HMAC on message m and key K is just $H(K, m)$, where K, m means K concatenated by m . The actual construction contains more (complicated) steps which will be discussed in the exercises. In any case, the intuition is that the hash function produces a fixed-length, random looking value that depends on every bit of both the key and the message and, since the adversary does not know the key, they are unable to forge the MAC for other messages. Moreover, since the hash function is a hard to invert one-way function, the adversary cannot recover the key from a MAC.

CBC-MAC: It is also possible to construct a MAC scheme starting from a secure block-cipher (e.g., AES). The construction, known as the CBC-MAC, follows the same process as in CBC encryption (as described in the previous chapter) but using the all-zero IV . Then the MAC is just *the last block of the ciphertext*. Such a MAC can be verified simply by recomputing the MAC from the received message and comparing to the received MAC. The intuition is that the last block of the CBC ciphertext depends in a complicated way on both the key and the entire message. Therefore any change to the message would result in a completely different last block that is hard for an adversary to predict. It can therefore be used as a MAC. Note that, as it is, this MAC scheme cannot be used for messages of variable length, as it will become clear if you solve the exercises.

MAC algorithms are generally as fast as secret-key encryption, but suffer of course from

the same key distribution problem that exist for any secret-key construction: we must have the secret key in place at sender and receiver before we can send anything.

6.2.4 Authenticated Encryption

Most applications require both confidentiality and authenticity at the same time. One may of course do this combining any secret-key encryption from the previous chapter with a MAC scheme from above. However, combining these tools in a secure way is far from trivial.

First, if one wants to generically combine encryption and MAC one must use different and independent keys for the two purposes: cryptography is hard and security might be lost if the same key is used in two different contexts.

Moreover, it is not trivial to decide in which order the two operations should be performed. What should you encrypt? What should you authenticate? We will return to this in the exercises.

Luckily, in practice, you do not need to worry about this. Cryptographers have developed special modes of operation that provide both confidentiality and authenticity at the same time, using only one key. Such encryption schemes are known as **authenticated encryption**, and are becoming increasingly popular in practice, since they are simpler, more secure, and more efficient than an ad-hoc combination of encryption and MAC schemes.

Examples of authenticated encryption schemes are OCB, GCM and CCM.

6.3 Authenticity, Public-Key Systems

A public-key system for authenticity consists of three algorithms (G, S, V) , where the algorithm used to generate the authentication tag is now called S , known as a **digital signature algorithm**.

In contrast to secret-key systems for authentication, where the same key was used for both authentication and verification, a public-key system for authenticity makes use of a pair of matching keys, a **public-key** pk and a **secret-key** sk . Sometimes these keys can also be called **verification-key** vk and **signing key** sk , if one wants to make clear that the keys are to be used for authenticity (as opposed to public-key systems for confidentiality that we have seen in the previous chapter).

Just like for public-key encryption, G is more complicated than for MAC schemes because it must output a pair of keys with the right relation between them. The idea is that a user A of such a system will generate such a pair of keys in private on their own machine, will then publish pk , but keep sk private. Again, it must be a very difficult computational problem to compute sk from pk .

This setup can be used by A to send an authenticated message m by computing the signature $s = S_{sk}(m)$ and send m, s . The receiver who (like anyone) knows A 's public key pk can run V to get a result $V_{pk}(m, s)$ which is $\top$ (accept) or $\perp$ (reject) as before. Also as before, we must require that the receiver accepts only if nothing happened to the message, i.e., for any message m and matching keys pk, sk we have

$$V_{pk}(m, S_{sk}(m)) = \top$$

On the other hand, it should be the case that an adversary who knows (of course) the public key of A and has seen a few messages authenticated by A , cannot find a valid signature on any message m' for which A did not provide a signature. The definition is similar to what we saw for the secret-key schemes:

Definition 6.2 The adversary is given the public key pk . They are allowed to specify any number of messages, say $m_1, \dots, m_t$ and they are given valid signatures $s_1 = S_{sk}(m_1), \dots, s_t =$

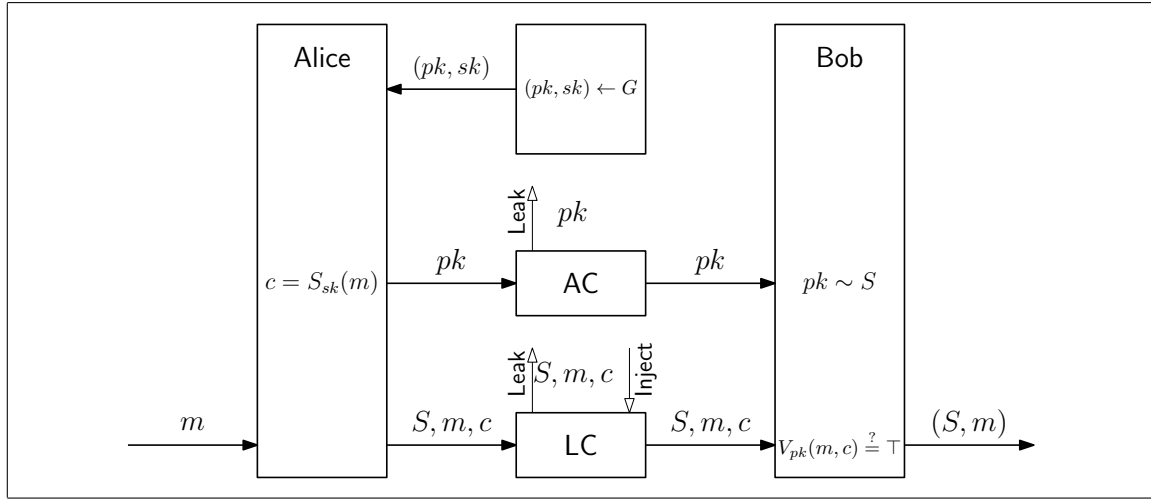


Figure 6.3: An illustration of how a digital signature scheme (G, S, D) is used to send messages in an authenticated way. The protocol tries to implement a lossy authenticated replay channel LARC. The sender, Alice, has to send a message m to a receiver, Bob, over an unauthenticated channel which leaks the transmitted data and which allows the adversary to inject messages. Somehow they managed to briefly have access to an authenticated channel at some point in time. Alice generates a key pair (pk, sk) and sends pk to Bob over the authenticated channel. Bob stores that pk is the public key of Alice. Later when a message arrives while Alice and Bob only has access to an unauthenticated channel LC, Alice can compute and send along a digital signature $s = S_{sk}(m)$. This digital signature is allowed to leak. Alice also sends along her identifier S . When receiving S, m, s , Bob can use S to fetch pk and check that $V_{pk}(m, s) = \top$ and output S, m if this is the case. It is essential that sk is known only by Alice. We call the protocol $\text{Sig2LARC}_{G,S,V}$. To make Bob deliver a message m_0 never sent by Alice the adversary would have to inject a message (S, m_0, s_0) such that $V_{pk}(m_0, s_0) = \top$ without Alice having signed m_0 and without knowing sk . Security of a digital signature scheme exactly states that this is hard.

$S_{sk}(m_t)$ for these messages. The digital signature scheme is secure if the adversary cannot efficiently produce a message m_0 and a signature s_0 such that $V_{pk}(m_0, s_0) = \top$, and $m_0 \notin \{m_1, \dots, m_t\}$.

A system satisfying the above definition is said to be **unforgeable under chosen message attack**.

6.3.1 Security of public-key systems

For similar reasons as with public-key crypto-systems, exhaustive key search is possible, but the adversary can usually find the secret key from the public one much faster than by exhaustive search. So keys must in general be much larger for public-key authentication than for the secret-key case.

6.3.2 Difference to secret-key: Transferability, Non-repudiation

Note here a very important difference between public and secret-key authentication: in the secret-key case, if B receives a message from A and accepts it, B knows that the message came from A , but will not be able to convince anyone else: both A and B have the secret key, and so

anything B claims to have received from A could as well have been generated by B . This does not happen in the public-key case: the value $S_{sk}(m)$ can only be computed by someone who knows sk , and so it could only have been produced by A . This means that digital signatures are **transferable**, and digital signature schemes satisfy the property of **non-repudiation**, meaning that the signer A cannot first sign a message m and then later claim this they never did that.

Note that, as for public-key encryption, it is important that the receiver uses the *right* public key when checking a signature, otherwise one can be made to believe that a message comes from A when this is not the case. We will discuss **certificates** later in this chapter to address this problem.

6.3.3 RSA Digital Signatures – Vanilla Version

Most of the techniques that can be used to construct public-key encryption schemes (RSA, discrete logarithm, elliptic curves, ...) can also be used to construct digital signature algorithms.

In the case of RSA signature that we will see in a moment, there will be a strict resemblance between the operations used to RSA encrypt/decrypt and to RSA verify/sign respectively. This sometimes leads to confusion, such as believing that encryptions and signatures are the dual of each other. We stress that the strict resemblance between the operations performed in RSA encryption and signatures respectively is somehow a coincidence and, for instance, if you use methods based on elliptic curves then encryption and signatures use very different algorithms.

We start with a simple but insecure version of the RSA signature scheme, with public-key $pk = (n, e)$ and secret-key $sk = (n, d)$. To generate a signature of a message $m \in [0, n)$ the signer, who is the only one who knows the secret-key, will do something to the message that no one else could have done, namely apply the secret-key operation to the message. That is, the signature on m could be

$$s = m^d \bmod n$$

It turns out that the special way e and d are chosen will ensure that raising s to power e modulo n will produce the original message, i.e., we have

$$s^e \bmod n = (m^d \bmod n)^e \bmod n = m$$

So when you receive a pair (m, s) you can check the signature by verifying whether $s^e \bmod n = m$ and reject otherwise.

Remember that in RSA encryption we had a possible optimization where e was chosen as a small number such as 3. This can also be done in the context of signatures, thus allowing for fast verification. This is important in applications where a signature is generated once, but verified many times.

6.3.4 Attack on Vanilla RSA Signatures

The first attempt we just made to use RSA for signatures is actually insecure and does not satisfy the definition of security we gave earlier: an adversary could choose some s , compute $m = s^e \bmod n$ and claim that m is a signed message. Indeed, this will pass the verification, since m and s precisely relate as they should in order for a signature to be valid. This may not seem like a complete disaster: as the adversary cannot control the value of m – most likely m is not meaningful at all. Nevertheless this is represent a security problem and therefore vanilla RSA should not be used in practice.

6.3.5 Hash-and-Sign Signatures

We are going to fix the above problem combining the RSA algorithm with cryptographic hash functions as described at the beginning of the chapter.

The basic idea for using a hash function h in connection with signatures is that computing $h(m)$ produces a digest of m , and that no one can find any other message with the same digest (this would lead to a collision). Therefore, intuitively, signing $h(m)$ is just as convincing as signing m itself. In practice what we do is to fix a hash function h to be used by all users, and then this can be combined with a basic signature scheme as the RSA one described above.

So, an RSA signature on a message m is computed as

$$S_{sk}(m) = h(m)^d \bmod n$$

In other words, we first hash the message and then raise it to the secret-key¹. Accordingly, we also define a verification algorithm $V_{pk}(m, s)$, that on input message m and signature s accepts if and only if

$$h(m) = s^e \bmod n$$

Intuitively, hash-and-sign prevents the attack described above: of course, the adversary can still choose some s and compute $s^e \bmod n$. With basic RSA, this could be claimed to be a message. But with the hash-and-sign RSA scheme, the adversary instead has to find m such that $h(m) = s^e \bmod n$. This is the problem of inverting h , i.e., given some large random looking value (such as $s^e \bmod n$) in the image of the function, finding a preimage of this value.

As we discussed above, cryptographic hash functions are one-way functions, which are hard to invert. Of course now we have just argued that a specific simple attack does not work anymore, but it turns out that one can actually prove that hash-and-sign RSA signatures satisfy the unforgeability property.

In the rest of the book, when we talk about *RSA signatures*, we will always mean a secure variant, where hashing is used, even if we do not refer explicitly to the hash function used.

Finally, using hash-and-sign RSA also has the advantage that one can sign messages of arbitrary length by doing a single RSA exponentiation which, as discussed earlier, takes much more time than e.g., hashing.

6.3.6 Using Digital Signatures in Practice

We end this section with some thoughts on usage of digital signatures in real life. In particular, can digital signatures have legal value, just like hand written signatures? The answer is yes, in fact the legislation in many countries, including Denmark, allows for digital signatures for all sort of important documents. For this to be secure in practice some conditions need to be met. Some are obvious, such as using large enough key sizes, and storing the private key securely. Other things are more subtle: clearly, it should be possible for an honest user to report it if their private key has been stolen or compromised, and we then have to assert that all signatures that match the old public key should now be considered invalid. Note here that even documents with an older date cannot be trusted anymore since using the private key, one can forge documents with any desired date.

However, since digital signatures can be used to bind the signer to some obligation, it may be in a cheating signer's interest to report their private key stolen even if it is not, just to repudiate a signed document. There are several solutions to this. One solution is to have an independent time stamping service, who is asked to countersign important documents. Since

¹Note that the hash value is much shorter than the size of the RSA modulo, and some appropriate padding is needed to make the sizes fit.

the signature of such a service remains valid, even if users have their keys compromised, this would solve the above problem that even old documents cannot be trusted when the private key is stolen.

A supplementary solution which is virtually always necessary, is to have legal documents where users agree to certain regulations for the use of digital signatures. For instance, a user typically has to agree they assume responsibility for their own private key, including reporting it stolen as soon as they suspect its security is harmed. This means one can have rules stating that if a user does not protest against the consequences of having digitally signed something before some deadline, they cannot get out the obligation by claiming that the private key was compromised. Such agreements resemble a lot the rules that apply to credit cards, or other digital tools.

A final thing to note about practical applications of signatures is that

**A digital signature commits the signer to
exactly what was signed and nothing else.**

While this seems blatantly obvious when stated explicitly as here, it is surprisingly often overlooked in applications: suppose, for instance, we have a system for signing e-mails. Is there a difference between the case where only the message content is signed and the case where all fields of the mail, including the intended recipient's address, are signed? Despite what you may think, there is in fact a very big difference. The latter method ensures that the sender always commits to the identity of the recipient, the former only does this if the recipient is explicitly mentioned in the text of the mail. As a silly –but illustrative– example: suppose Bob is in love with Alice and sends her a signed mail saying “I love you, will you marry me?”. Then with the former method, anyone can claim to have a signed offer of marriage from Bob!

A more subtle problem of this type is the case when a sender encrypts a message and then signs the ciphertext. This signature does not commit the sender to the plaintext message! The simple reason is that anyone can eavesdrop a ciphertext from the net and sign it, without knowing anything about the plaintext.

6.4 Replay attacks

The secret-key and public-key approach to authenticating messages are actually not satisfactory by themselves in all cases. The reason for this is that if I receive for instance a message m with a digital signature from A , this only proves that *at some point*, A produced this message. It leaves open the possibility for an adversary to take a copy of the signed message and send it to me as many times as they want. This is known as a **replay attack**. If A is a bank customer, the receiver is a bank, and the message is a request to transfer money from A 's account to someone else, the security problems with replay should be evident.

So what we really want in practice is often not just to protect the integrity of messages, but to have a real authenticated channel, that is, we want B to receive the exact same sequence of messages that A sends, and if this not possible, we want to come as close to this as we can. If we do not have physical control over the communication line, we cannot prevent an adversary from physically blocking some messages or from reordering them before they are received, but we should at least be able to make sure that replayed messages are filtered out. We describe three approaches to this, all of them with their pro and cons. In the description below we assume MACs are being used to authenticate, but the same would work for signatures.

Counters One trivial way to prevent replay attacks is have the sender make sure that they never send exactly the same message twice, for instance by appending a sequence number,

and also add a MAC (computed over both message and sequence number). Then we can have the receiver store every message ever received (or at least the sequence numbers). This will allow the receiver to filter out every replayed message, and also to correctly place all received messages in the order they were sent.

Of course, this is hardly a practical solution. Even if we do use sequence numbers, we cannot expect the receiver to store more than the sequence number of a small number of recently accepted message. Suppose, for instance, that they stores only the last accepted message. Then if the receiver has stored number n and the next message does not have number $n + 1$, we need a rule for the receiver to decide what to do. The problem is that in some cases, messages may get lost or be reordered even if no attacker is present. One possibility is to require that the next message must have number $\geq n$ to be accepted. This will prevent replay, but may also mean that good messages get rejected if they arrive too late.

Timestamps A different approach is to append a timestamp to messages. To be of any use, this of course requires that the receiver checks the time stamp. Again, one may in principle store all received time stamps. This leads to security against replay, but is usually not practical. Another method is to check time stamps against your own system time, to ensure that it is not too old. Some compromise has to be made here, so that on one hand good messages are accepted, but on the other hand replayed messages are rejected. This requires that there is some synchronization between sender and receiver, and also that messages are delivered quickly enough. On the other hand, there is no need here to remember any information about earlier messages.

Challenge-Response Finally, one may use interaction: we can have the receiver first send a challenge i.e., a number R to the sender. This number can be chosen at random, or be a sequence number, the only real requirement is that it has never been used before (i.e., it is what we called a nonce earlier). Then the sender sends the message plus a MAC computed over the message and R . This will prevent replay, and there is no need for synchronization, but one does need to at least remember some state in order to ensure that R -values are not reused. In addition this is not trivial to implement securely if the receiver has to handle several connections in parallel – for instance, he has to keep track of which R -values belong to which connections.

You might have encountered such measures against replay attacks when using authentication methods for logging into sensitive services or using a credit card online, often when using two-factor authentication mechanisms, which will also be discussed in a later chapter.

6.5 Certificate Authorities

The main issue that is left to address, when using public-key systems (encryption or authentication), is how to securely retrieve the public-key of the users or the website we want to communicate with. Note that the fact that those keys can be made *public* does not imply that there necessarily is a consensus on which key belongs to whom, especially as the adversary might want lie about their identity.

Essentially, the solution to this problem is to have some form of authentication chain. Say you know, for some reason, that user A really owns public key pk_A (maybe they have given you their public-key on a piece of paper) and that you trust them. Then, A can send you a digitally signed message informing you that user B is the owner of public-key pk_B and you can add (B, pk_B) to your “address book”. There are many variants of this simple idea in practice:

privacy activists have for years held *key signing parties* where they would verify each others' identity and then sign each other's public key, to generate a decentralised web of trust.

A more common way of achieving the same goal is to use central **certificate authorities (CA)**. A certificate authority is an entity (usually a commercial one) whose job is to provide users (typically owners of websites) with **certificates**, that establish a link between the identity of the user and their public key.

A certificate authority owns its own key pair (sk_{CA}, pk_{CA}) , and we must assume that users know the identities of the trusted certificate authorities and their public keys (these typically come pre-installed with your operating system or with your browser, see the exercises).

Once a user A wants to obtain a certificate, they contact the CA, send their public-key pk_A to the CA, and finally identify themselves in some way. It is important to understand that the way the user identifies themselves cannot be via cryptography only: before A has registered, A and the CA do not share any secret-keys, and the CA does not know any public-key it can safely assume belongs to A . Therefore, the CA must verify A 's identity is some non-cryptographic "physical" way, and it must be able to verify that the public key it gets, actually came from the person it identified. In practice, this takes many forms: in some cases users can receive a certificate by meeting in person at some citizen-service desk, or showing some IDs at their bank. When certificates are used for web-applications, the CA might ask users to publish some file containing a unique ID as a proof of ownership of the website. If the CA sees that you published the requested content on your website, they will assume you own that URL.

When the CA accepts the identity of the user, it will issue a so called **certificate**, which in its simplest form contains the identity of the user, their public key, and a signature on those values under the secret-key of the CA

$$Cert_{CA}(A) = (ID_A, pk_A, \dots, S_{sk_{CA}}(ID_A, pk_A, \dots))$$

This can be thought of as a kind of receipt for having registered – the CA certifies that it identified the user as being A , and that pk_A was the key that was presented to it. Because all users were assumed to have pk_{CA} , if A now wants to communicate with B , they can simply send $Cert_{CA}(A)$ to B . Now B can check that the certificate is valid (by checking the signature using pk_{CA}). If the certificate is valid, B will retrieve A 's public-key which they can then use to encrypt data or to verify signatures.

In real life, things are not as simple, however: if a user suspects that their private key has been compromised, it should be possible for them to report this, and have their certificate revoked, much like credit cards can be reported stolen. This means that there must also be an option to check whether a given certificate is still valid. Typically CAs publish black-lists of revoked certificates. This is one of the reasons why certificates should also contain a validity period as part of what is signed by the CA, since if a certificate never expires, black lists would grow indefinitely. In general, certificates may contain all sorts of other control information, so a certificate is typically a rather long data record with for instance the following fields:

- Name of the certificate owner
- Name of the CA who signed the certificate
- Method used by CA to check identity of certificate owner
- Date issued
- Validity period
- Rights and privileges of certificate owner.

- Crypto algorithm to be used for checking this certificate
- Crypto algorithm used by certificate owner
- Public key of certificate owner

All of these data should of course be signed by the certificate authority.

Another complication in real life is that there is of course more than one CA in the world. In the US, companies like Verisign and AT&T issue certificates, in Denmark for instance DanID offer such services. So it will often be necessary to have methods for handling the situation where two users do not have certificates from the same CA. One way to do this is if the two CA's certify each other's public keys. Some notation to describe this: a certificate issued by certification authority CA to user A with public key pk is denoted $Cert_{CA}(A, pk)$.

Now, say user A has a certificate from CA_1 and user B has one from CA_2 . Now A receives $Cert_{CA_2}(B, pk_B)$, which they cannot directly check as A does not have CA_2 's public key. However, A does have CA_1 's public key, so if A also gets a certificate $Cert_{CA_1}(CA_2, pk_{CA_2})$, then A can first verify this certificate, which provides evidence that the public key of CA_2 really is pk_{CA_2} . Thus A can now verify $Cert_{CA_2}(B, pk_B)$ to get convinced about pk_B .

A more general way is to use so called *certificate chains*: such a chain is an ordered list of certificates, of the following form

$$Cert_{CA_1}(B, pk_B), Cert_{CA_2}(CA_1, pk_{CA_1}), \dots, Cert_{CA_n}(CA_{n-1}, pk_{CA_{n-1}})$$

where the first entry is a certificate where CA_1 certifies the public key of the user (B) in question. In the second entry, CA_2 certifies the public key of CA_1 , etc. until the last entry where CA_n certifies the public key of CA_{n-1} . If another user A is able to collect enough certificates to form a chain as above, where A already knows the public key of CA_n , then A can verify the entire chain and hence also the public key of B .

It is important to understand, however, two limitations of certificate chains: A should only trust the end point (B 's public key) to the extent that they trust that *every* CA involved in the chain has not issued fake certificates (and this has happened in practice – CAs can be hacked, their keys can be stolen, etc.). Second, certificate chains do not remove the need for at least one public key that must be known to users initially, in particular, verifying a certificate chain cannot be done unless one knows beforehand an authentic copy of pk_{CA_n} .

If you look at the certificates installed on your machine, you will notice that certificate chains are also used in a hierarchical-manner. One starts from few, highly trusted **root certificates** and then go down the tree to certify other certificate authorities, that finally certify users.

One curious thing about root certificates is that they are self-signed i.e., the public-key is signed under its own secret-key. Therefore, that signature does not prove anything in itself – the *only* trust one can place in a public-key obtained this way is the trust that follows from having received it from a trusted source.

Finally, while up to few years ago it was relatively expensive and complicated for small organizations to get a certificate from one of the existing commercial certificate authorities, the situation changed when a number of profit and non-profit organizations started *Let's Encrypt*, a free certificate authority with the goal to push encrypted communication as the default setting for the entire world wide web.

6.6 Implementing an Authenticated Channel*

We show how the tools introduced in this chapter can be used to formally implement the functionality of an authenticated channel.

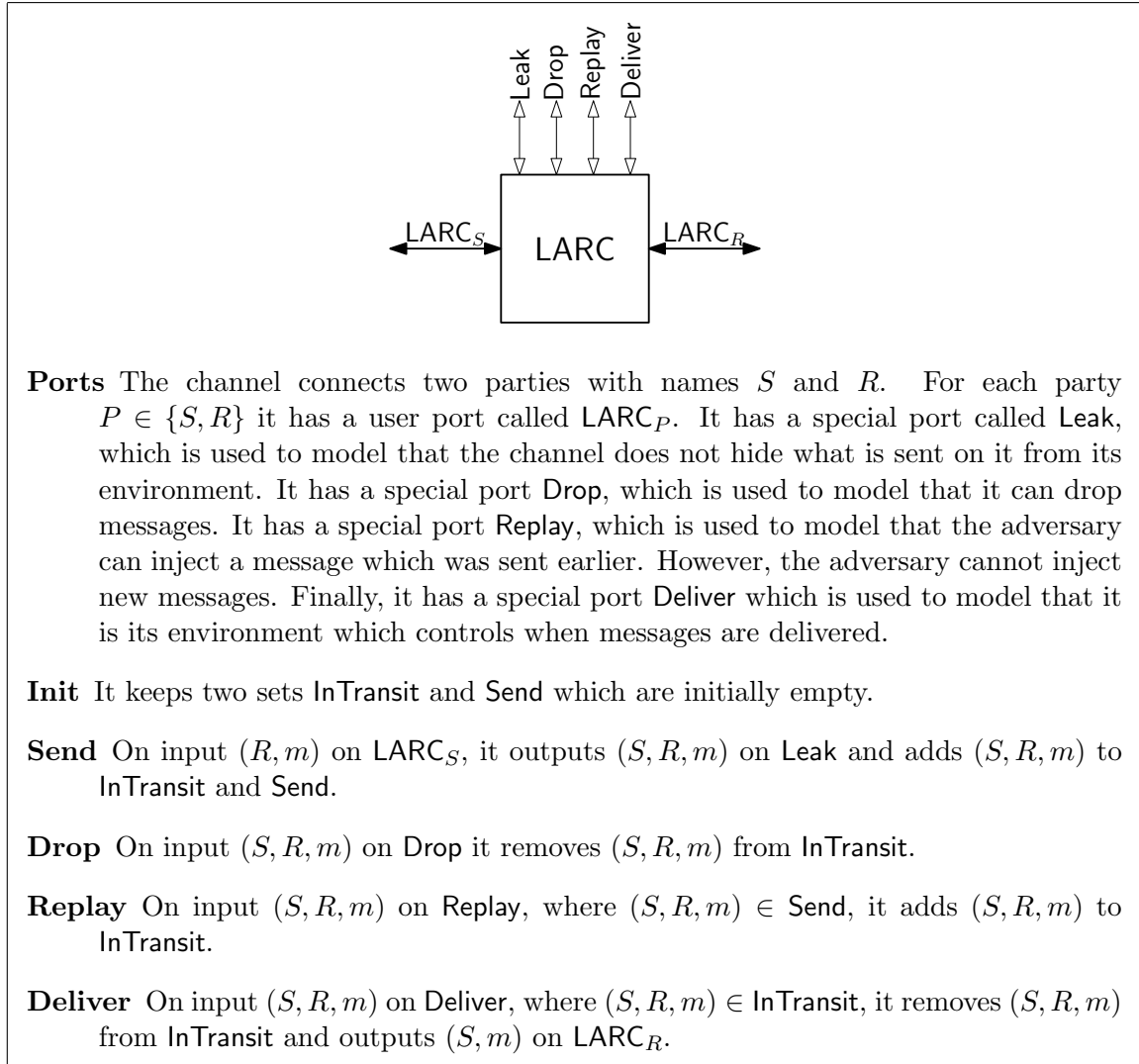


Figure 6.4: Lossy Authenticated Replay Channel

We can use a secure MAC to implement an authenticated channel. We can even prove security of this in the sense of a protocol implementing an ideal functionality in Chapter 2. For this purpose we need to formally specify the intended behaviour of the protocol in terms of an ideal functionality. We do this in Fig. 6.4. Then we need to formally specify how the protocol works. We do this in Fig. 6.1. Then we need to argue that the possible input-output behaviour of the protocol is included in the possible input-output behaviour of the ideal functionality.

In Fig. 6.4 we model an authenticated channel as an ideal functionality, with the slight problem that old messages can be replayed. This means that the channel does not allow to inject new messages, but if a message m was sent once, the adversary can inject it again later. We introduce this slight weakness to make it easier to implement LARC. If one later wants to implement LAC from LARC one could use some of the techniques to prevent replay attacks described earlier. But for now we focus on implementing an ideal functionality allowing replay but not allowing injection of new messages.

Intuitively the protocol's behaviour is included in that of the LARC as the protocol does not deliver message that were not sent by the sender because of the use of the MACs. Of course

there is a tiny chance that the adversary could, e.g., guess the key of the MAC and use this to inject new messages. But this happens with a probability small enough to be negligible. And, we allowed protocols to behave different from the ideal functionalities with a tiny negligible probability in our notion of securely implementing an ideal functionality.

For the formally inclined reader the below theorem adds a bit more detail to how the proof is done, namely by giving a simulator which constructively shows that the behaviour of the protocol is included in that of LARC.

Theorem 6.3 *Let $\text{MAC2LARC}_{G,\text{MAC},V}$ be the protocol in Fig. 6.1. If (G, MAC, V) is a secure MAC then $\text{MAC2LARC}_{G,\text{MAC},V} \sqsupseteq \text{LARC}$.*

PROOF (INFORMAL)* To prove $\text{MAC2LARC}_{G,\text{MAC},V} \sqsupseteq \text{LARC}$ we have to come up with a simulator Sim such that $\text{MAC2LARC}_{G,\text{MAC},V} \equiv \text{LARC} + \text{Sim}$. A simulator Sim can simulate $\text{MAC2LARC}_{G,\text{MAC},V}$ given access to LARC simply by running the protocol "in the head" as in Fig. 4.33. When a message is dropped or delivered in $\text{MAC2LARC}_{G,\text{MAC},V}$ the simulator does the same on LARC. When a message is injected in $\text{MAC2LARC}_{G,\text{MAC},V}$ and Bob outputs m , then simulator will replay m on LARC. This will be allowed except when the adversary succeeds in making Bob deliver an m that was never sent by Alice. When the adversary is sufficiently computationally bounded and the MAC scheme is secure this happens with negligible probability. Therefore $\text{MAC2LARC}_{G,\text{MAC},V} \equiv \text{LARC} + \text{Sim}$. $\square$

It can also be shown that, for public-key systems, the notion of unforgeability under chosen message attack is exactly what is needed for the protocol in Fig. 6.3 to securely implement LARC. We state this result without a proof.

Theorem 6.4 *Let $\text{Sig2LARC}_{G,S,V}$ be the protocol in Fig. 6.3. If (G, S, V) is unforgeable under chosen message attack, then $\text{Sig2LARC}_{G,S,V} \sqsupseteq \text{LARC}$.*

6.7 Exercises

Exercise 6.1 (Enc-then-sign or Sign-then-encrypt) You are consultant to a company that offers access to a database D . Different users have access to different parts of the database and moreover, the database contains sensitive information. Therefore the security policy says:

- When a request for information arrives, D should be able to determine which user sent the request.
- A user should not be able to get information about which data other users asked for.

We can assume that every user A has a private RSA key sk_A of their own, and the database stores a list of public keys of all users. Also all users know the public key pk_D of D .

The threat model assumes that the database is sufficiently protected that it will not be hacked, but network traffic can be attacked and manipulated.

One suggestion to ensure that the policy is followed is: when user A wants to send a request R to D , they will send

$$E_{pk_D}(R), S_{sk_A}(E_{pk_D}(R)), A$$

to D , that is, encrypt the request under D 's public key, append A 's signature on the encrypted request, and then append A 's username. D will look up pk_A , verify the signature, and if it is correct, will decrypt the request, and return an answer of form $E_{pk_A}(\text{data})$, that is, the answer to the request, encrypted under A 's public key.

Another suggestion goes as follows: when user A wants to send a request R to D , they will send

$$E_{pk_D}(R, S_{sk_A}(R)), A$$

to D , that is, append A 's signature on the request, encrypt this under D 's public key, and then append A 's username. D will decrypt the request and signature, look up pk_A , verify the signature, and if it is correct, will return an answer of the form $E_{pk_A}(data)$, that is, the answer to the request, encrypted under A 's public key.

One of these solutions fails to satisfy the security policy. Which one, and why? Are there any general conclusions one could draw from this example?

▼

Exercise 6.2 (CBC-MAC Padding) For MAC constructions based on block ciphers such as CBC-MAC, one needs to split the input message into blocks of fixed size such as 128 if we use AES. A padding method is then needed to append some bits to the message in order make its length after padding divisible by 128. Consider the following method: “If the message length is divisible by 128, do nothing, otherwise append a minimal number of zero bits so that the resulting length is divisible by 128”.

Explain why this method would be insecure, no matter which MAC algorithm we use. I.e., show how a MAC on one message can be turned into a MAC on a different message without using the secret key.

Consider instead the following: “If the message length is divisible by 128, do nothing, otherwise append a minimal number of zero bits so that the resulting length is divisible by 128. Finally, append a block containing in binary the bit-length of the message”.

Explain why this method does not suffer from the problem(s) you identified with the first method. Note: you are not expected to prove that, for instance, CBC-MAC is secure with this second padding scheme, this is not even known to be true in all attack models. You are only expected to argue why the generic problem with the first scheme is solved in the second.

Clearly, the second method makes the message one block longer than we would need if we just wanted to store the message in an integral number of blocks. Is it possible to have a padding method that never makes the message longer in this way, and yet does not have the problem of the “only-zero” padding?

▼

Exercise 6.3 (Merkle-Damgård Construction) Both the SHA-1 and the SHA-256 hash functions are constructed using the same basic design idea: first, a so called compression function f is constructed, that takes as input two strings m and s , of lengths 512 bits and 160 bits, respectively. Next, a 160 bit block s_0 is fixed as part of the description of the hash function. Now, given a message M to hash, we split it in 512 bit blocks $m_1, \dots, m_t$, padding the last block with 0's if it is incomplete. Finally a block m_{t+1} is appended that contains the original length of the message, written in binary. We now compute, for $i = 1, \dots, t+1$, $s_i = f(m_i, s_{i-1})$. The final hash value is defined to be s_{t+1} .

Show that if one can find a collision, i.e., different messages M, M' producing the same hash value, then one can easily find a collision for the function f .

Thus if we believe it is hard to find collisions for f , we have to believe it is hard as well for the entire hash function.

Hint: When we hash M , we process blocks $m_1, \dots, m_{t+1}$ to get values $s_1, \dots, s_{t+1}$. When we hash M' , we process blocks $m'_1, \dots, m'_{t'+1}$ to get values $s'_1, \dots, s'_{t'+1}$. Note that if the hash of M is the same as the hash of M' , this means that we have

$$f(m_{t+1}, s_t) = s_{t+1} = s'_{t'+1} = f(m'_{t'+1}, s'_{t'})$$

This may immediately give a collision for f , and we are done. If not, then $(m_{t+1}, s_t) = (m'_{t'+1}, s'_{t'})$. What can you conclude from this? ▼

Exercise 6.4 (HMAC and Length-Extension Attack) In the chapter we described HMAC for a message M and key K as $H(K, M)$ i.e., the hash applied to the concatenation of K and M , where K is of fixed size and M is a message of variable length. We will see here why the actual construction needs to be more complicated (the actual HMAC construction has a structure of the form $H_1(K, H_2(K, M))$).

As you have seen in the previous exercise, computing $H(K, M)$ involves in fact repeatedly invoking some “inner-function” f . For concreteness, say the message fits exactly in two blocks i.e., $M = (m_1, m_2)$ then

$$c = H(K, M) = f(m_2, f(m_1, f(K, s_0)))$$

Is this a secure MAC? If you are given the message $M = (m_1, m_2)$ and the MAC c (but not the key K). Can you forge a MAC on a different message?

To avoid the problem you have identified above, some have suggested instead to compute the MAC as

$$c = H(M, K) = f(K, f(m_2, f(m_1, s_0)))$$

Does this solve the problem you have found above? Why or why not? ▼

Exercise 6.5 (Hash For Authentication) Alice has a PC and is worried that someone might get access to her machine and modify some important files she has while she is away on holiday. In order to at least be able to detect any such changes, she uses a hash function h , for which it is infeasible to find collisions. For each relevant file, she applies h to the contents of the file and stores all output values in a separate directory on her hard disk. When she comes back, she will verify that the hash values on the files still match the values she stored. Explain why this solution is unlikely to work.

Alice thinks again and comes up with two solutions that seem better:

1. She stores the hash values on a USB stick that she takes with her, when she comes back she verifies the hash values on the stick as above.
2. She chooses a secret key K , and for every file f , she computes $\text{MAC}_K(f)$ using some secure authentication scheme. She stores all the MAC values in a special directory and brings K with her on a USB stick. When she comes back she verifies all the MACs.

Will both solutions work if Alice goes on holiday once? Is the answer the same if she does it twice or more? ▼

Exercise 6.6 (Replay Attacks) Many car companies offer remote controls for their cars, so that the car can be remotely unlocked. One potential security problem is that since the communication between car and remote control is wireless, it is quite easy using standard equipment to eavesdrop the communication relating to a target car. An adversary could try to use the information obtained this way to later trick the car into unlocking. Design one or more methods for handling the communication between car and remote control that would be secure against this threat model. ▼

Exercise 6.7 (Not a Signature) Here is a suggestion for doing message authentication based on public key technology, but without hash functions: suppose the receiver B knows the sender A 's public key pk_A . B chooses an AES key K , and sends to A the encryption $E_{pk_A}(K)$ (concretely, this could be RSA plus OAEP). A will decrypt to get K , and then sends to B the message M plus the message authentication code $CBC - MAC_K(M)$.

Explain why this cannot be used as a digital signature scheme, even if we assume that both the public-key encryption and the CBC-MAC construction are secure. ▼

Exercise 6.8 (RSA Malleability) Another reason why hash functions are necessary ingredients in signature schemes: Suppose we use RSA for digital signatures, but without using a hash function. Concretely, this means that every message m is a number in the interval $[0..n-1]$ where (n, e) is the public key and (n, d) is the private one. The signature on m is $m^d \bmod n$. Suppose an adversary is given valid signatures s_1, s_2 on messages m_1, m_2 . Show that the adversary can now easily generate on his own a valid signature on $m_1 \cdot m_2 \bmod n$, namely $s_1 \cdot s_2 \bmod n$ is a valid signature on $m_1 \cdot m_2 \bmod n$. Hint: for any numbers a, b it holds that

$$((a \bmod n) \cdot (b \bmod n)) \bmod n = ab \bmod n.$$

▼

Exercise 6.9 (Hash-and-Sign) In hash-and-sign RSA a signature on a message m is computed as

$$s = H(m)^d \bmod n$$

and verified by checking that $s^e \bmod n = H(m)$. Suppose that you discover that the hash-function is not collision resistant (but still second-preimage resistant). How does this affect the security of the signature scheme? You should think both about the unforgeability security game described in the chapter, as well as concrete attack scenarios.

Now, suppose that you discover that the function is not even second-preimage resistant (but still one-way). How does this affect the security of the signature scheme? ▼

Exercise 6.10 (Combining Encryption and MACs) As explained in the chapter, if you need to achieve both confidentiality and authenticity in the symmetric-key case, you should simply use an authenticated encryption scheme instead of attempting to do an ad-hoc combination of an encryption scheme and a MAC. This exercises exposes some of the pitfalls that might arise if you try to do things yourself. Given an encryption key ek , an authentication key ak , and a message M , discuss the pros and cons of the following options:

Encrypt-and-MAC As the name suggest we encrypt and MAC the message independently i.e., output $(c, t) = (E_{ek}(M), MAC_{ak}(M))$;

MAC-then-Encrypt Compute $t = MAC_{ak}(M)$, output $c = E_{ek}(M, t)$;

Encrypt-then-MAC Compute $c = E_{ek}(M)$, compute $t = MAC_{ak}(c)$, output (c, t) ;

Hints: Since you are evaluating the security of the combination of the two tools *in general*, you are welcome to replace the MAC or the encryption with concrete ones if it helps you come up with an attack. For confidentiality: what happens if you encrypt the same message twice? Which of the above proposals leak this information, and which do not?

For authenticity, given the knowledge you have now it should be harder to come up with conclusive attacks. However, you should still be able to argue that the different modes behave

differently when the adversary tries to tamper with the ciphertext. To do so, you must consider how the three different methods will be decrypted. You can also assume that the adversary can modify ciphertext and submit them to a decryption oracle (thus learning the result) in the style of *chosen-ciphertext attacks* described in the previous chapter. ▼

Exercise 6.11 (CRC in WEP) This exercise is about a problem in the first version of the WEP standard, i.e., the protocol that is used to establish secure channels on wireless networks. WEP assumes that sender and receiver have a preshared key K , so no public-key technology is used. Instead, the protocol uses the stream cipher $RC4$, which is basically a function that takes K as input and outputs a random looking string that can be as long as one wishes. This output is used as a one-time pad on the message, so that encryption of message M will be $M \oplus RC4(K)$. The protocol has some built-in procedures for changing the key so that the same output string will not be used on different messages, but this is not relevant here.

The designers of WEP also wanted to ensure authenticity. For this, they used a so-called cyclic redundancy check (CRC). This is a function that takes any string as input and outputs a fixed length string $CRC(M)$. It is normally used to check for transmission errors coming from naturally occurring noise. One sends the concatenation $M||CRC(M)$, and the idea is that if M or $CRC(M)$ is modified, there is a good chance that the last part will no longer be CRC applied to the first part, so that errors can be detected.

Question 1: Explain why this way to use CRC (without encryption) will not help to obtain authenticity.

The idea of WEP was to send the encryption $RC4(K) \oplus (M||CRC(M))$, and this was then supposed to ensure both confidentiality and authenticity. Probably, the designers of WEP believed that the problem with authenticity addressed in the previous question would not occur here because of the encryption. They were wrong because they did not take into account the way the CRC function was defined: each bit in $CRC(M)$ is in fact the x-or of a fixed set of bit positions in M , for instance, the first bit in $CRC(M)$ might be the xor of bits number 1,3,5,7, etc of M .

Question 2: Explain how an adversary can modify an encrypted message so that the underlying message is changed, but the CRC-check after decryption will still be OK. ▼

Exercise 6.12 (Certificates Authorities) Find out which certificate authorities are trusted by your operating system/browser. The exact procedure to do so might change between different OS and browsers. Try to investigate the details of the certificates you have. Also try to find the root certificates you have installed. Your browser will trust any site with a certificate that can be verified using any of the root certificates on the list. What did you see? Are you surprised? Worried? ▼

Exercise 6.13 (Hash Trees) Suppose you are given a hash function h that accepts input strings of length 256 bits and produces 128 bit outputs. We assume h is a good cryptographic hash function: h can be computed efficiently, but it has the basic property required for hash functions, namely it is not in practice possible to find inputs $x \neq y$ such that $h(x) = h(y)$, a so-called collision for h . Consider the hash function H defined as follows: it takes as input a string X of length 512 bits. We split X in two strings X_0, X_1 each of length 256 bits, such that X is the concatenation of X_0 and X_1 , written $X = X_0||X_1$. We define

$$H(X) = h(h(X_0)||h(X_1))$$

Show that if one could find a collision for H , i.e., $X \neq Y$ such that $H(X) = H(Y)$, one could easily find a collision for h as well. Therefore, finding a collision for H is as hard as finding one for h . Generalize this idea to construct from h a secure hash function that takes inputs of length $2^j \cdot 256$ bits, for any j , and argue that it is hard to find collisions for your function, if it is hard to find them for h .



Exercise 6.14 (CR $\Rightarrow$ OWF) This exercise is concerned with proving the fact that if a hash function h is collision-resistant then it is also one-way. In this exercise we will assume for simplicity that $h : \{0, 1\}^{k+1} \mapsto \{0, 1\}^k$, and that $Im(h) = \{0, 1\}^k$. In other words, h maps $k + 1$ -bit strings to k bit strings and h is surjective. Both restrictions can be removed quite easily.

To show that inverting h is at least as hard as finding collisions, we assume that we are given an algorithm Alg that inverts h , i.e., on input $a \in Im(h)$ it returns x such that $h(x) = a$. We then construct an algorithm that will use Alg to find collisions with good probability, as follows:

1. Choose $y \in \{0, 1\}^{k+1}$ uniformly at random and compute $a = h(y)$
2. Execute Alg on input a and let x be the output
3. Output x, y .

Does this work? Note first that it is always the case that $h(x) = a = h(y)$, but for (x, y) to be a collision, it must also be the case that $x \neq y$. To estimate the chance of this, do the following:

1. Let $h^{-1}(a)$ be the preimage of a , i.e., the set of inputs that are mapped to a by h , and let $|h^{-1}(a)|$ be the size of this set. Show that when we choose y uniformly as above and set $a = h(y)$, then $Pr(|h^{-1}(a)| \geq 2) \geq 1/2$.
2. Let E be the event that $|h^{-1}(a)| \geq 2$. Show that $Pr(x \neq y | E) \geq 1/2$.
3. Show that $Pr(x \neq y) \geq 1/4$.

Hence the probability that our algorithm successfully produces a collision is at least $1/4$. Hint for item 1): construct a set $A \subset \{0, 1\}^{k+1}$ as follows. For every element a in the output set $\{0, 1\}^k$, choose an input u such that $h(u) = a$ and add u to A . Note that now A has exactly 2^k elements. Therefore, when you choose y uniformly it will be the case that $Pr(y \notin A) = 1/2$.

Can you now prove that if h is second-preimage resistant, then it is also one-way? You need to show that, given Alg that finds pre-images, you can find second-preimages as well. (Hint: try to use the algorithm and proof above as a starting point, and see if and where that proof fails).



Exercise 6.15 (RSA Signatures)

Goal. Extend your Go package from Exercise 5.11 to support RSA signatures. Messages are first hashed using SHA-256, and the hash value is then signed using RSA.

```

package account

type SignedTransaction struct {
    ID          string // Any string
    From        string // A verification key coded as a string
    To          string // A verification key coded as a string
    Amount      int    // Amount to transfer
    Signature   string // Potential signature coded as string
}

func (l *Ledger) SignedTransaction(t *SignedTransaction) {
    l.lock.Lock() ; defer l.lock.Unlock()

    /* We verify that the t.Signature is a valid RSA
     * signature on the rest of the fields in t under
     * the public key t.From.
     */
    validSignature := true

    if validSignature {
        l.Accounts[t.From] -= t.Amount
        l.Accounts[t.To] += t.Amount
    }
}

```

Figure 6.5: The SignedTransaction type.

Task. Implement methods to generate and verify RSA signatures. Hashing should be done using the `crypto/sha256` package. The hash value may be converted directly to an integer before signing.

Suggested API (you can change it).

- `RSASign(message []byte, d, n *big.Int) *big.Int`
- `RSASign(message []byte, signature, e, n *big.Int) bool`

Note. In real-world standards, the hash value is padded before applying RSA. You may ignore padding in this exercise.

Validation and measurements. In addition to the code, your solution must include the following:

1. Verify that you can generate and verify a signature on at least one message. Modify the message and check that verification fails.
2. Measure the hashing speed in bits per second. Hash messages significantly longer than a hash value (e.g., 10KB).
3. Measure the time required to generate an RSA signature on a hash value using a 2000-bit RSA key.
4. Assume RSA were used directly on the entire message. Use the result from the previous item to estimate the achievable throughput in bits per second. Compare this to the hashing speed and discuss whether hashing improves efficiency.



Exercise 6.16 (Authenticated Transactions in a Peer-to-Peer Ledger) *This exercise builds directly on Exercise 4.6. You will extend the ledger with authenticated transactions using RSA signatures.*

Goal. Extend your peer-to-peer ledger so that transactions are authenticated. Only the owner of an account should be able to spend funds from that account. This is achieved by signing transactions with RSA secret keys and verifying signatures upon delivery.

Background. From Exercise 4.6, you already have:

- a peer-to-peer network with flooding,
- a replicated ledger,
- transactions that are delivered and executed at all peers.

From Exercise 6.15, you have implemented RSA signatures. In this exercise, you add cryptographic authentication to transactions. You do *not* add total order or overdraft protection yet.

Accounts and keys. Each account is associated with an RSA public key. Account names are string encodings of RSA public keys. Let $\text{encode}(\text{pk})$ denote a deterministic encoding of an RSA public key pk as a string (for instance you can use base64). A user creates an account by generating an RSA key pair (pk, sk) ; the account name is $\text{encode}(\text{pk})$. Only the holder of the corresponding secret key sk can authorize transactions from this account.

Signed transactions. Replace plain transactions with `SignedTransactions` consisting of:

- `TxID` (string): a unique transaction identifier,
- `FromAccount` (string): $\text{encode}(\text{pk})$ of the sender,
- `ToAccount` (string): $\text{encode}(\text{pk})$ of the receiver,
- `Amount` (integer),
- `Signature`: an RSA signature.

All fields except `Signature` are signed using the sender's secret key. To do this, you must define an unambiguous serialization of the transaction fields (excluding the signature) before signing and verification, for instance by marshaling the fields in a fixed order using JSON. A signed transaction is *valid* if the signature verifies under the public key encoded in `FromAccount`.

Ledger update rule. When a peer receives a signed transaction:

- it verifies the signature,
- if the signature is valid and the `TxID` has not been seen before, it executes the transaction on the local ledger,
- if the signature is invalid, the transaction is ignored.
- if the `TxID` has already been executed, the transaction is ignored (replay protection),

As in Exercise 4.6, transactions are executed even if this causes an account balance to become negative.

Flooding. Signed transactions are flooded using the same flooding mechanism implemented in Exercise 4.6. No changes to the flooding logic are required.

Hand-in and Demo. Your hand-in must include:

1. The source code of your system.
2. A short PDF report (see below).
3. A single demo program that:
 - (a) starts a peer-to-peer network as in exercise Exercise 4.6,
 - (b) creates several accounts (RSA key pairs),
 - (c) sends both valid and invalid signed transactions,
 - (d) shows that only valid transactions affect the ledger,
 - (e) verifies that all peers end with identical ledgers.
 - (f) Prints only relevant information, such that a TA can observe the required behavior in real time without excessive or missing output.

Describe clearly in the report where this program is located and how to run it.

Testing. You must test your system at least as thoroughly as in Exercise 4.6. In addition, include tests that check:

- valid signed transactions are accepted,
- transactions with invalid signatures are rejected,
- transactions signed under the wrong key are rejected,
- ledgers still converge across peers.

Describe what you test and how you run the tests.

Report. Your report should include:

1. A brief description of how signed transactions are implemented.
2. A description of how RSA public keys are encoded as account identifiers.
3. A discussion of why it is hard for an adversary to take control of another user's account.
4. A description of how you tested the system, including tests that check that invalid transactions are rejected.
5. Instructions for your TA explaining how to run your code and reproduce your demo.



Chapter 7

Synchronization (2026)

Reading Guide *This chapter introduces some basic concepts and techniques relating to time and synchronisation in distributed systems. All of the sections are curriculum. The material is fairly basic and sets the stage for the Dolev-Strong protocol in Chapter 8.*

Contents

7.1	Physical Time	189
7.2	Clock Synchronisation	191
7.2.1	GPS Clock Synchronization	191
7.2.2	NTP Clock Synchronisation	191
7.3	Round-Based Protocols	195
7.3.1	When a Sent Message Arrives	196
7.3.2	Accounting for Computation Time	196
7.4	Exercises	199

7.1 Physical Time

When building distributed systems it is sometimes useful, or even necessary, to have access to the physical time. If for nothing else, it is nice if your calendar system can remind you when it is time for lunch. For the sake of measuring time, most consumer grade computers are equipped with a quartz clock which measures time using an electronic oscillator regulated by a quartz crystal. Typical quartz clocks drift away from the real physical time by about 1 second in about 10 days. This is certainly enough to get you to lunch at a reasonable time, since it would take 50 years to drift half an hour. But note that by the above number a quartz clock drifts about 2^{-20} seconds per second, and a modern CPU can execute about 1000 instructions in 2^{-20} seconds. For a modern computer, 2^{-20} seconds is a long time, so when judged by modern computers, quartz clocks are horrible at keeping time.

Another alternative is atomic clocks which use the electron transition frequency of atoms for measuring time. These clocks lose about 1 second in 1 million years and are used for defining international standards of time: since we do not have a more precise way of measuring time, these devices become our *de facto* standard for what time is. However, atomic clocks are too bulky and expensive to be put in consumer grade computers.

Since computers have fairly poor clocks they will tend to drift from the perfect clock. The time on the clock minus the time on the perfect clock is called the **clock drift**, see Fig. 7.2 for an illustration.

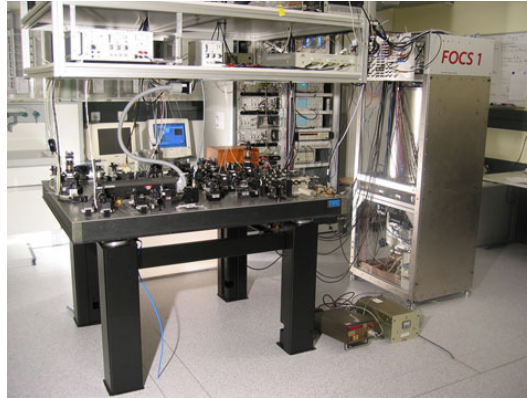


Figure 7.1: FOCS 1, an atomic clock in Switzerland which drifts at most one second in 30 million years

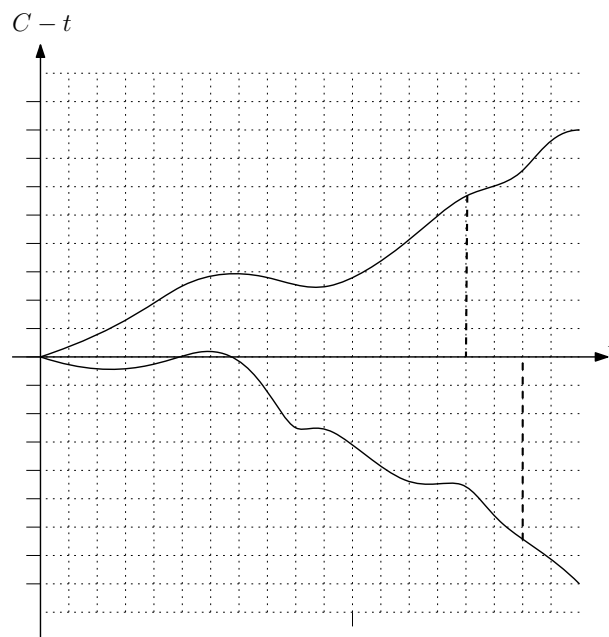


Figure 7.2: Plotting the drift of two clocks. The horizontal axis is time t , or equivalently, the perfect clock. The vertical axis is $C - t$, where C is the clock. So we plot the clock drift. A perfect clock would run along the horizontal axis. The drift of the two clocks are shown as dashed lines at two different points in time. The top clock has positive drift: it is ahead of time. The bottom one has negative drift: it is behind time. Note that the drift between the two clocks can be twice that of the drift from the perfect clock.

7.2 Clock Synchronisation

When you buy a new watch, you would typically set the time to be the same as some other clock that you believe is correctly showing the current time. Similarly, computing devices can synchronise their clock by interacting with more accurate clocks to which they are connected via a computing network.

In practice, this means there are a few organizations owning very expensive and very precise atomic clocks (which provide essentially error-free measures of time), who can help the billions of computing devices equipped with low-accuracy quartz clocks keep their measure of time from drifting too much.

It turns out that clock synchronisation in distributed systems is harder than setting the time on your own watch. In particular, here is a solution that *does not* work: the device with the lousy clock asks the device with the precise clock for its current time, and then sets it as its own. The problem with this solution is clearly that transmitting messages takes time: by the time you receive the time measured by the atomic clock, some more time has passed, so the time you received is not the current time anymore! It is like setting the date on you clock using some old news paper you find laying around. Therefore, protocols for clock synchronisation are a bit more complicated than the one described above.

Two of the main systems for clock synchronisation are the GPS system and the NTP protocol.

7.2.1 GPS Clock Synchronization

The global positioning system consists of a large number of satellites in low orbit. Each of them is equipped with an atomic clock and they constantly transmit their position and their time. Their positions consists of three spatial coordinates (x, y, z) and the time consist of one time coordinate t . Together this gives a 4-dimensional coordinate (x, y, z, t) which the satellites broadcast publicly and can be received by anyone with a GPS receiver which has the satellite in line of sight. Notice that if you receive signals (x_1, y_1, z_1, t_1) and (x_2, y_2, z_2, t_2) from two different satellites, the time coordinates t_1 and t_2 might not be the same, since the time it takes for the signal to reach you is a function of your distance to the satellites, and your distance to the two satellites might not be the same. However, it turns out that if you receive four signals $(x_1, y_1, z_1, t_1), \dots, (x_4, y_4, z_4, t_4)$, then you can compute your own position and time (x_0, y_0, z_0, t_0) . Oversimplifying a bit, this corresponds to having a system with four equations and four unknowns, which can then be solved.¹ We will not go into details of the GPS equations here (See Figure 7.3 for a simplified example with a single spatial dimension).

7.2.2 NTP Clock Synchronisation

We will instead have a look at the details of the Network Time Protocol. In this protocol we imagine a server S which has an atomic clock, or some other very precise clock. There is a client C with a drifting clock which occasionally needs to be synchronised.

The algorithm works under two assumptions:

- (A1) During the time it takes to run the protocol, the clocks of the server and the client only drift negligibly apart.
- (A2) The time it takes to send a package from the client to the server is the same as it takes to send from the server to the client.

¹It is not that simple, as the equations are not linear, but four coordinates are indeed enough.

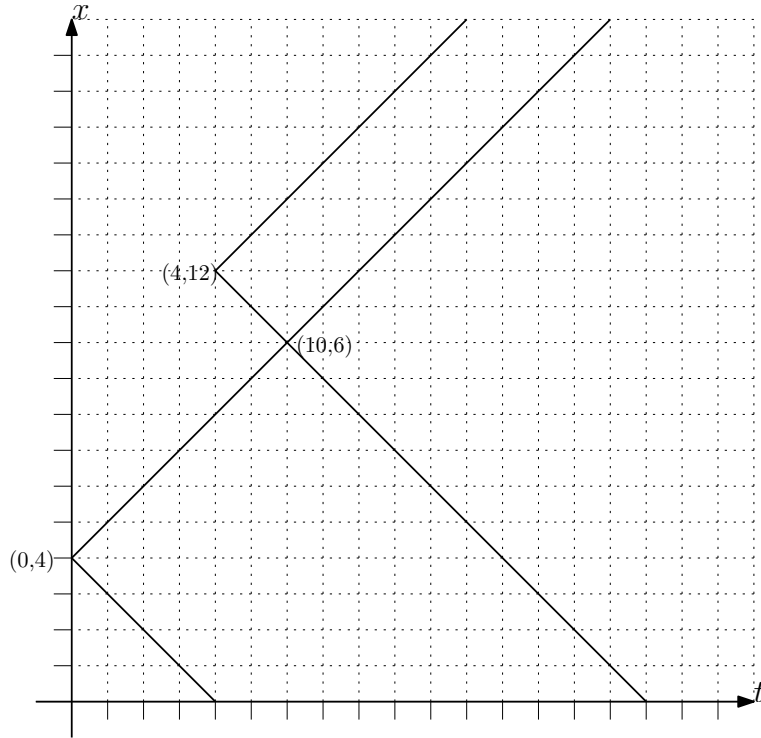


Figure 7.3: GPS in two dimensions. Time runs rightward (unit: 1 second) and space runs upward (unit: c meters (where c is the speed of light in meters per second)). A satellite at position $x = 4$ and at time $t = 0$ sends the signal $(0,4)$. When you receive it you can be anywhere on the bottom triangle. Another satellite at position $x = 12$ and at time $t = 4$ sends the signal $(4,12)$. When you receive the signal you can be anywhere on the top triangle. But if you receive both signals at the same time, you know that you are at position 10 and that the time is 6, as this is the only position which is on both triangles. Similarly, for four dimensions you need readings from four satellites.

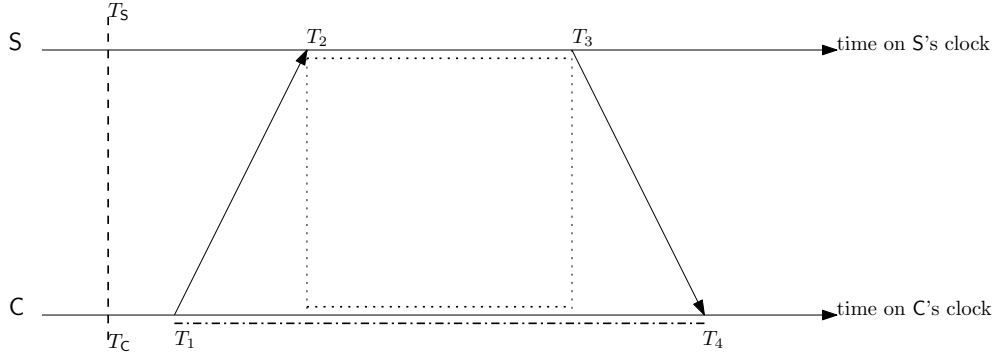


Figure 7.4: The Core of the NTP Protocol illustrated.

It is natural to ask whether these assumptions are satisfied in practice. Assume that the server has an atomic clock, that the client has a quartz clock, and the entire protocol takes about a second to terminate. Then the drift between the clocks will be about 2^{-20} seconds, which is small enough to be ignored and therefore assumption A1 holds. Assumption A2 is less straightforward: while it might be true that the time it takes to send packets between two devices is approximately the same on average, network errors, packet retransmission and unstable connections can easily negate assumption A2 in practice, as they might only affect the communication on one direction. We will return to this later on.

Given the two assumptions, we are now ready to describe the main idea behind the NTP protocol (with many details omitted). When a client wants to synchronise its clock, the protocol in Fig. 7.5 is executed.

1. C sends a “time request” message to S and stores its current system time T_1 .
2. Upon receiving the “time request” message, the server S stores its current system time T_2 .
3. The server S prepares a “time response” message, which includes the time T_2 . Right before sending the response to the client C, the server S measure its current system time again, call it T_3 , and includes it in the message.
4. Upon receiving the message (“time response”, T_2 , T_3), the client C measures its current system time T_4 . It can now compute two numbers:

$$\text{TransEst} = \frac{(T_4 - T_1) - (T_3 - T_2)}{2}$$

$$\text{OffsetEst} = (T_1 + \text{TransEst}) - T_2 .$$

The number **OffsetEst** is an estimate of how much the client’s clock is ahead of server’s clock. The client adjusts its own clock using this estimate.

Figure 7.5: The core of the NTP protocol. See also Fig. 7.4

To understand why the protocol works the way it does, consider Fig. 7.4. The top vertical line is time measured on the clock of the server. The bottom vertical line is time measured on the clock of the client. Times as measured on the clocks are plotted at the same physical time. Now take the clocks at the physical times indicated by the dashed vertical line to the

left. The clock at the server is T_S . The clock at the client is T_C . So, the clock of the client is $\text{Offset} = T_C - T_S$ ahead. We call this the **offset** of the client's clock (relative to the clock of the server). Notice that with this definition of offset, it holds that $T_S = T_C - \text{Offset}$. In the rest of the text, an offset is always how much the clock is *ahead*. One of the goals of the NTP protocol is to estimate this offset.

Assumption A1 can now be rephrased as saying that the offset is more or less constant during the execution of the protocol. In other words, the offset could have been measured by subtracting the clock of the server from the clock of the client at any fixed physical time.

Now, let **Trans** be the time it takes for a message to travel from the client to the server or the other way around (remember that by assumption A2 the two times are equal). Then, at the physical time when the server measures the time T_2 , the clock of the client would show the time $T_1 + \text{Trans}$ (this is where the first vertical dotted line touches the clock line of the client in the picture).

Then the offset could be measured as $\text{Offset} = (T_1 + \text{Trans}) - T_2$. Therefore, to compute the offset, it is enough to be able to estimate the transmission time.

Unfortunately, since the clock of the client and server are not synchronised initially, there is no way to compute the transmission time precisely. For instance, the time it takes to send a message from the client to the server is *not* $T_2 - T_1$ as T_1 is measured on the client's clock and T_2 is measured on the server's clock, and these two clocks are not in sync. The NTP algorithm therefore uses the following trick instead: Notice that $T_4 - T_1$ is the total time it took to run the protocol. This is a meaningful measure of time since T_1 and T_4 are measured on the same clock. Similarly, the time spent on processing at the server side is $T_3 - T_2$, which is also a meaningful measure of time since it is performed on the same clock. Therefore, the total time it took to send both messages was $(T_4 - T_1) - (T_3 - T_2)$, since clocks do not drift noticeably during the protocol (assumption A1). Under the assumption A2 (that the message transfer time is the same in both directions) we can divide this by 2 to get an estimate of **Trans**, and hence an estimate of **Offset**, so this explains the computation done by the client in the protocol.

Adjusting the Clock

Now, if the estimated offset is positive, then the client's clock is ahead and it will turn down the speed of its clock a bit to fall back. If the offset is negative, then the client's clock is behind and it will turn up the speed of its clock a bit to catch up. Notice that the client does *not* just set its clock to the new estimate of what the time is at the server. When it is ahead, this would force time to travel backwards on the clock of the client which will be bad for many processes on the client. But sudden jumps forward in time can be disturbing too. Imagine that you used your computer to time the boiling of an egg for breakfast, and then the clock of the computer suddenly jumps forward by five minutes. When the timer goes off, the egg might not be to your taste. It is therefore better to adjust the speed of the clock a bit. In many cases this is also better at preventing future drifts.

Revisiting the Assumptions

As already mentioned, the assumption that the transfer time is the same in both directions is a brave one. Even if this is true when the network is in good condition, a burst error affecting only one direction could falsify the assumption. To avoid such a situation NTP runs the core protocol several times and then adopts the execution where **TransEst** was lowest. The rationale is that when errors occur, then transmission times tend to get higher, so throwing away the executions with the highest execution times will in general also throw away the executions where there were network errors. Even with these added bells and whistles the assumption is somewhat brave.

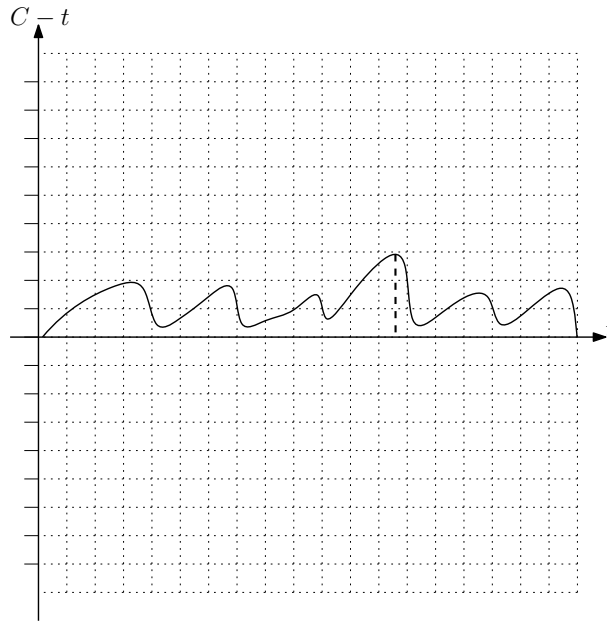


Figure 7.6: Plotting the drift of a clock that is occasionally synchronized. See also Fig. 7.2. The maximal drift MaxDrift is plotted as a dash lines. This is the maximal drift the clock managed to have before being synchronized.

Sometimes there is just a difference in the up link and down link of a computer. Exercise 7.2 lets you think a bit about this situation.

Max Clock Drift

If one has a bound on network delay and how fast a clock drifts one can occasionally run a clock synchronization protocol like NTP to bring the clock close to the perfect clock again. From bounds on the network delay, how fast the clocks drift and how often one runs synchronization one can calculate a so-called max clock drift MaxDrift which is the maximal drift a clock might ever experience.

7.3 Round-Based Protocols

To study the general power of using time in distributed systems it is usual to first consider the fully synchronous round-based model. In the simplest instantiation there are n parties or processes called $P_1, \dots, P_n$. The protocol, denoted by π , proceeds in rounds. The idea is that in each round each process is allowed to send a message to each other process. Sending nothing is an option, which is denoted NoMsg in the following. In this model, we assume that all processes have perfectly synchronised clocks and that transmission time is fixed such that a message sent by a correct process in a certain round will always be available to the receiver when the next round starts.²

²Since we cannot guarantee that something will *always* happen—the computers of a party could blow up and all network cables melt—what we are actually doing when setting the timeout is accept that any party not sending within this timeline will be counted as corrupted. And then we hope that so few will miss the timeline that counting them as corrupted we are still within the number of corruptions the system is designed to tolerate.

7.3.1 When a Sent Message Arrives

While this model is simple and easy to work with, it is clear from our previous discussion that the assumptions made are hardly realistic. There is some hope, however. Even if clocks are not perfectly synchronised, we can still hope to set bounds on clock drift and transmission time, and predict when a message ought to arrive. More precisely, suppose a client expects that a server will send a message at time t . Assume that the client knows that the absolute offsets of the server's clock and its own clock relative to real time are at most **Offset**. Assume also that the client knows that it takes at most **Trans** seconds to send a message. Then, if nothing was received at time $t + 2\text{Offset} + \text{Trans}$ on the clock of the client it can conclude that the message was not sent. Namely, the server will send the message at time t on its own clock, which is no later than at real time $t + \text{Offset}$. Therefore the message arrives no later than at real time $t + \text{Offset} + \text{Trans}$. At real time $t + \text{Offset} + \text{Trans}$ the clock of the client shows no more than

$$(t + \text{Offset} + \text{Trans}) + \text{Offset} = t + 2\text{Offset} + \text{Trans} .$$

7.3.2 Accounting for Computation Time

We now generalise the above idea to a protocol π consisting of many rounds. We will argue that if we know bounds on clock drift, transmission time and computation time of the computations done in each round, we can set timeouts in such a way that we avoid dropping messages which were actually sent. This will imply that we can run a protocol that was designed for the fully synchronised round-based model in the more realistic setting of imperfect clocks and varying transmission times.

For simplicity we will assume that the computation that needs to be done in each round takes about the same time in each round. We use **MaxComp** to denote a time bound on how long it takes any party to complete the computation of any round once all the inputs are ready. We also assume a time bound **MaxTrans** on how long it maximally takes to send a message between two correct processes and a time bound **MaxDrift** on how long any correct process drifts from the real time. Note that we can use clock synchronisation to keep **MaxDrift** down.

From these numbers one can compute the slot length:

$$\text{SlotLength} = 2\text{MaxDrift} + \text{MaxTrans} + \text{MaxComp} .$$

Note that this is just the time on the clock of the receiver that it would take for a sent message to arrive, but including now also the time it takes to compute the message. By making the rounds this long, in each round each honest party has time to compute the messages to send, send them and have them arrive at the other honest parties in time (on the receiver's clocks).

We assume that there is a time t_0 at which the parties agree that they start running the protocol. At this time we also assume that they received their inputs for the protocol. We can assume that the input to each party P_i is (t_0, x_i) and that all honest parties have the same t_0 as input. We assume that (t_0, x_i) arrives before the clock of P_i reach t_0 such that P_i can start running the protocol at (local) time t_0 .

Each round runs within a time slot. Rounds are indexed by natural numbers r . Round r begins at (local) time

$$\text{SlotBegin}^r = t_0 + r \cdot \text{SlotLength} .$$

We assume that the computation performed by P_i is described by an algorithm **Compute**. It takes as input the round number r and the messages received in the previous round. It outputs the messages to be sent in the next round. Note that P_i also sends a message to itself. It is just stored locally. This message can be used to store the local state of the parties, i.e., the values it needs to remember for future rounds. In round $r = 0$ it only takes the input of P_i as input.

A round-based protocol now proceeds as in Fig. 7.8.

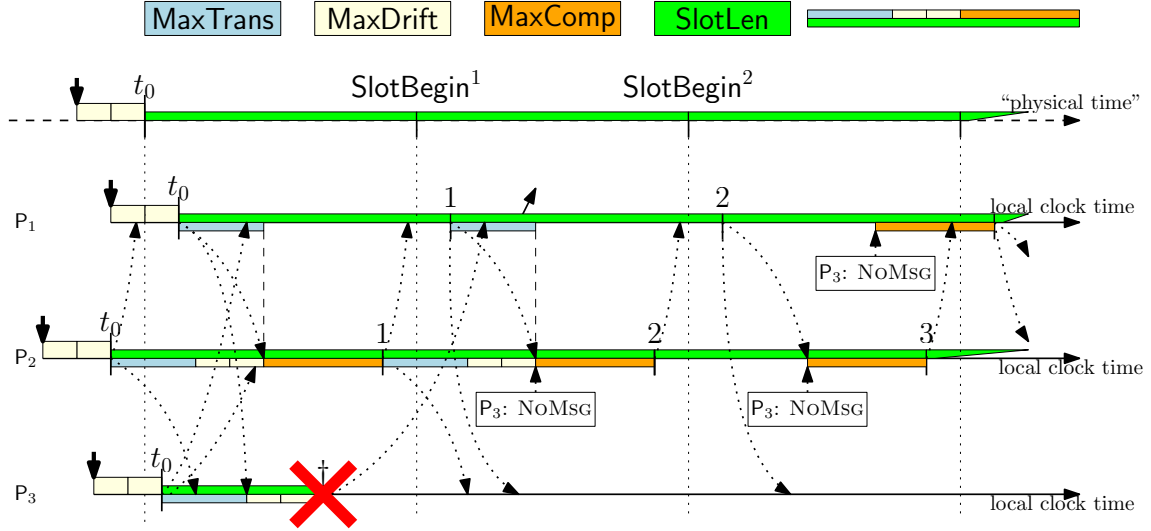


Figure 7.7: Illustration of round-based protocols. We know an upper bound on the maximal transmission time of a message, MaxTrans , a maximal amount of time local clocks can drift from “physical time”, MaxDrift , and a maximal time it takes to compute the messages to send in the next round once you have the messages from the previous round, MaxComp . We let the length of a round or slot be $\text{SlotLength} = \text{MaxTrans} + 2\text{MaxDrift} + \text{MaxComp}$. The parties agree on a time t_0 to begin the protocol, the beginning of slot 0. Here they send out the first messages. Since local clocks can drift, some parties like P_2 may think slots begin MaxDrift too early. Other parties, like P_1 may think slots begin MaxDrift too late. Still, an early party like P_2 will have received all messages by time $t_0 + \text{MaxTrans} + 2\text{MaxDrift}$, which allows it to compute for MaxComp time and be ready to send out messages at the beginning of (its local) slot 1. Since all parties are thus ready to send out messages when their local slots begin it is maintained that all messages also arrive on time for the computation phase. Therefore, if a party like P_3 crashes and does not send a message for P_2 for round 1, then P_3 can know that this happened when it has to begin computing (at the orange interval). It can then use some dummy message, NoMSG from P_3 instead. Note that a crashing or malicious party may send a message to some parties and not others. Here P_3 sends to P_1 for round 1, so not until round 2 does P_1 detect that P_3 is crashed or malicious. Parties wake up 2MaxDrift before the first round of sending, to be sure to be awake to receive from early parties. See, for instance, the first message from P_2 to P_1 .

1. Each P_i gets input (t_0, x_i) before local time $t_0 - 2\text{MaxDrift}$ and computes SlotBegin^r for $r = 0, 1, \dots$
2. At time SlotBegin^0 party P_i computes $(m_{i,1}^0, \dots, m_{i,n}^0, \text{State}_{i,0}) = \text{Compute}(0, x_i)$. The local time at P_i is now at most $\text{SlotBegin}^0 + \text{MaxComp}$.
3. Now send $(\text{MSG}, 0, m_{i,j}^0)$ to each P_j . This message arrives at local time at most $\text{SlotBegin}^0 + \text{MaxComp} + 2\text{MaxDrift} + \text{MaxTrans}$ at P_j . And $\text{SlotBegin}^0 + \text{MaxComp} + 2\text{MaxDrift} + \text{MaxTrans} \leq \text{SlotBegin}^1$. The message arrives at time at time at least $\text{SlotBegin}^0 - 2\text{MaxDrift}$ at P_j (if computation and transmission is instantaneous and the clocks maximally apart), at which part P_j will be awake to receive.
4. In rounds $r = 1, 2, \dots$, party P_i runs as follows:
 - (a) Until local time SlotBegin^r , receive and store messages of the form $(\text{MSG}, r - 1, m_{i,j}^{r-1})$ from P_j .
 - (b) At time SlotBegin^r , for each P_j where no $m_{j,i}^{r-1}$ is stored, define $m_{j,i}^{r-1} = \text{NoMSG}$. Then compute $(m_{i,1}^r, \dots, m_{i,n}^r) = \text{Compute}(m_{1,i}^{r-1}, \dots, m_{n,i}^{r-1})$. Now the local time is at most $\text{SlotBegin}^r + \text{MaxComp}$.
 - (c) Now send $(\text{MSG}, r, m_{i,j}^r)$ to each P_j . This message arrives at local time at most $\text{SlotBegin}^{r-1} + \text{MaxComp} + 2\text{MaxDrift} + \text{MaxTrans}$ at P_j . And $\text{SlotBegin}^{r-1} + \text{MaxComp} + 2\text{MaxDrift} + \text{MaxTrans} \leq \text{SlotBegin}^r$.

Figure 7.8: A generic round-based protocol.

Using Clocks Makes You Slow

If an honest party sends a message which does not arrive before the timeout (and hence is replaced by **NOMSG**), then we say that a correct message was **dropped by the timeout**. Note that if all our assumptions on **MaxTrans**, **MaxDrift**, and **MaxComp** are correct, then it will never happen that a correct message is dropped by the timeout. This follows from the way we compute the slot length.

A warning about the round-based model is in order, however. We argued above that if a process is correct, then its messages will never be dropped by a timeout. Conversely, this means that if a process sends **NOMSG**, in a round where the protocol says something should be sent, then that process is not correct. In the Byzantine faults model this allows P_i , when it receives **NOMSG** from P_j , to assume that P_j is Byzantine corrupted. Since our models typically do not guarantee anything about the safety of corrupted parties, it follows that the protocol after this point can ignore the safety of party P_j . It can therefore have devastating consequences to miss a timeout. Therefore all the time bounds have to be put such that even in unfortunate conditions they can be met by all honest processes except with negligible probability. Most computers should be able to synchronise their clocks within a second of real time. The time to deliver a message, however, under worst case conditions over the internet is problematic to set conservatively. Even a bound as high as five seconds may be too daring: recall, it must never happen that it takes this long to deliver a message. Even with these bounds, each round will suffer a penalty of seven seconds. This means that a protocol with a 1000 rounds would suffer a delay of about two hours. Since the typical delivery time of a network is in the millisecond range, a protocol that does not care about timeouts could in principle execute 1000 rounds of

communication in about a minute. So it costs us the difference between a minute and hours to work with round-based synchronous protocols as opposed to the asynchronous protocols. These are only rough estimates, but their purpose is to illustrate the potential penalty of working with explicit time and timeouts in distributed systems.

The takeaway message is that in fully synchronous systems, each round takes the worst-case time to send a message, which might be much, much longer than the typical network delivery time. Therefore synchronous systems tend to be much slower than asynchronous systems.

7.4 Exercises

Exercise 7.1 (precise guarantee) Assume that the clocks of the server and client drift at most ϵ during the execution, i.e., the change in their offsets does not change by more than ϵ . Assume that the time it takes to send the message in one direction is no more than δ plus the time it takes to send the message in the other direction. Assume for simplicity that the client right after measuring T_4 subtracts Offset from its clock. Assume that the client updates its clock by subtracting OffsetEst at the end of the protocol. Give an upper bound on the absolute offset of the client's clock at the end of the protocol. ▼

Exercise 7.2 (different up and down latencies) Consider a system where the transmission times in the two directions are not the same. For simplicity assume that they are always t_1 respectively t_2 . But you do not know these values, you are just guaranteed that whenever you send a message from the client to the server it takes exactly t_1 seconds, and whenever you send a message from the server to the client it takes exactly t_2 seconds. For simplicity assume also that the clocks of the server and client do not drift during the execution of the protocol, i.e., assumption A1 holds perfectly. However, you start from a situation where the clocks of the server and the client can be arbitrarily far apart. Can you design a protocol to synchronise the clock of the client with the one of the server? How precise can you make your protocol? ▼

Exercise 7.3 (quorum clock synchronisation I) One big problem with NTP is that the client puts all its trust in the server. If the server is corrupted it can set the clock of the client to something wrong, which might have disastrous consequences for the safety of the client. Assume that you are a client and that you have access to the three clock servers of which you are willing to trust that at least two are honest and meet assumptions A1 and A2 perfectly. Assume that the clocks of the honest servers are perfectly synchronised. Try to design a protocol which mitigates the influence of a Byzantine server. ▼

Exercise 7.4 (quorum clock synchronisation II) This is the same exercise as above, but we consider a more realistic setting. In practice A1 and A2 do not hold perfectly and the clocks of honest servers will not be perfectly the same. We have to account for that in our algorithm. As a first step, assume that A1 and A2 still holds perfectly for the two honest servers. But assume that the clocks of the two honest servers can be a bit away from the real time and that they might be different. Specifically, assume there is some δ such that both clocks of the honest servers are within δ from the real time. Try to design a protocol which mitigates the influence of a Byzantine server. How close to the real time can you guarantee to be? Try to make an algorithm that is allowed to use δ and one which does not use δ . Can you do better when you can assume that you know δ ? ▼

Chapter 8

Synchronous Agreement (2026)

Reading Guide *The crucial material in this chapter is the Dolev-Strong protocol and the definition of what it achieves, namely reliable broadcast. It is also good to come away with an understanding of the two lower bounds on the number of corruptions tolerable and the number of communication rounds.*

Contents

8.1	Introduction	201
8.2	Defining Reliable Broadcast	205
8.3	Reliable Broadcast from a PKI	206
8.3.1	Dolev-Strong for Less Corruptions	213
8.4	Lower Bound on Round-Complexity of Reliable Broadcast	214
8.4.1	Generalizing the Result	219
8.5	No RB for Three Parties without Setup	219
8.6	Byzantine Agreement from Reliable Broadcast	222
8.7	Reliable Broadcast from Byzantine Agreement	222
8.8	Exercises	223

Attack Model *In this chapter we assume that*

1. *There are n parties, also sometimes called servers.*
2. *Up to t parties might be Byzantine corrupted, i.e., they may behave in any way and desire to break the protocol. We will consider two limits on t , namely $t \leq n - 2$ and $t \leq n/3$.*
3. *The adversary cannot drop messages sent by honest parties.*
4. *The adversary cannot inject messages and make them look like coming from other parties, which we ensure by using signatures or authenticated channels.*
5. *Protocols are in the synchronous round-based model. This implies that:*
 - (a) *The adversary is allowed to delay messages at most some known amount of time.*
 - (b) *The parties have access to synchronised clocks.*

8.1 Introduction

In this chapter we study protocols which allow several parties in a distributed system to agree on a decision even in the presence of a Byzantine adversary which tries to make them disagree. In general we are interested in three types of properties when considering a protocol trying to solve an agreement problem.

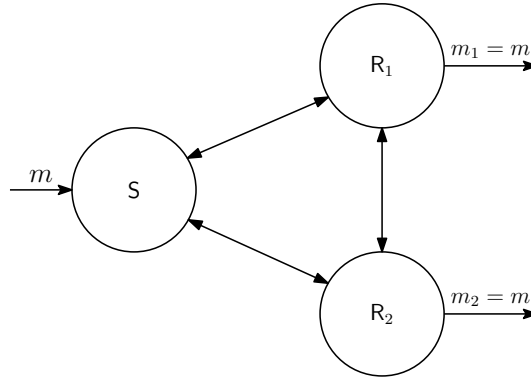


Figure 8.1: A sender sending the same message to two receivers.

Agreement All honest parties make the same decision.

Validity The decision that is made must be sensible in some sense.

Termination If all parties start running the protocol then all honest parties end up making a decision.

As an example consider the setting in Fig. 8.1. There are three parties connected by authenticated channels. There is a sender S which must send the same message m to two receivers R_1 and R_2 . We want to tolerate that one of the three parties might be Byzantine corrupted. When we design the protocol we do not know which party will be corrupted, it may be any of them, the sender or one of the receivers. Think of it this way. First we design the protocol, and then some evil adversary decides which party to corrupt and now controls this party completely. We want that a corrupted receiver cannot manipulate the message sent by S to the other party. We also want that a corrupted sender cannot make R_1 and R_2 output different messages. We could formulate it as follows:

Agreement If both R_1 and R_2 are honest and R_1 outputs m_1 and R_2 outputs m_2 , then it always holds that $m_1 = m_2$.¹

Validity If both S and R_1 are honest, then $m_1 = m$. Similarly, if both S and R_2 are honest, then $m_2 = m$.

Termination If all honest parties start running the protocol, then eventually every honest R_i outputs some m_i .

The reader which likes puzzles can at this point pause the reading and see if they can come up with a protocol with all the above three properties.

When solving distributed problems, there is almost always an agreement problem hiding somewhere. The system should do something valid and we want different parts of the system to agree on what has happened in the past and act in a coordinated manner in the future. Since the agreement problem is inherent to distributed systems, no book can list all possible agreement problems along with their solutions. The purpose of this chapter is to instead open up the mind of the reader towards the aspect of agreement in synchronous systems. Learning to see the problems and to solve them.

In this chapter we will look at two agreement problems which capture the core of the problem.

¹Here it is implicit that it holds even if S is Byzantine corrupted, as the only premise we give is that R_1 and R_2 are honest.

Reliable Broadcast The sender S is to send a single message at some agreed time. All receivers eventually output a message or NoMSG and agree on what they output. If the sender S is honest, then only the intended message can be output as coming from S . In particular, if S is honest, then no honest receiver outputs NoMSG . This is the generalisation of the above example to the case with more receivers.

Byzantine Agreement There are n parties $P_1, \dots, P_n$. Each has a single bit v_i as input. We call these the votes. They output a common decision d , also a bit. All parties should agree on d . Furthermore, if all honest parties have the same vote v as input, then $d = v$.

In this chapter we will see some examples of solving these problems. We will see that the two problems are tightly connected. We will also see some lower bounds which explore how well we can expect to solve these problems.

Here are some naïve protocols for reliable broadcast with three parties and one Byzantine corruption that do *not* work.

Example 8.1 S sends m to R_1 and R_2 and R_i outputs $m_i = m$. The problem is that if S is Byzantine corrupted it could send m_1 to R_1 and $m_2 \neq m_1$ to R_2 . It is not supposed to do this, but since S is Byzantine corrupted it might choose to deviate from the protocol. So, the protocol does not have Agreement. Note that it *does* have Validity and Termination. ▼

Example 8.2 S sends m to R_1 and R_2 . Receiver R_i receives m_i and relays it to the other receiver. Now both receivers have m_1 and m_2 . If $m_1 \neq m_2$ then R_i outputs NoMSG . Otherwise R_i outputs m_i . The problem is that if S and R_1 are honest and R_2 is corrupted, then R_2 might choose to send some $m_2 \neq m$ to R_1 . Then $m_1 \neq m_2$ at R_1 and hence R_1 outputs NoMSG . But R_1 is supposed to output $m_1 = m$ when S and R_2 are honest. The protocol does not have Validity. Note that it *does* have Agreement and Termination. ▼

Example 8.3 S sends m to R_1 and R_2 . Receiver R_i receives m_i and relays it to the other receiver. Now both receivers have m_1 and m_2 . If $m_1 \neq m_2$ then R_i outputs nothing. Otherwise R_i outputs m_i . The problem is that if for instance a corrupt S sends m_1 to R_1 and $m_2 \neq m_1$ to R_2 , then the receivers never output anything. The protocol does not have Termination. Note that it *does* have Agreement and Validity. ▼

As shown above, getting two out of three of Agreement, Validity and Termination is relatively simple. It is getting all three at the same time that is hard. Sometimes impossible. The first student to present a protocol which always has Agreement, Validity and Termination for the above setting—with authenticated channels and a single Byzantine corruption—will instantaneously pass the course with an A. Email the solution to your instructor.²

Before we progress, we give an example that works for three parties S , R_1 and R_2 . It uses signatures and a PKI.

Example 8.4 We assume that R_1 and R_2 know and agree on the verification key of S , and similar for the other parties. First S computes a signature σ on m . Then S sends m, σ to R_1 and R_2 . Receiver R_i forwards what it gets to the other receiver (along with its own signature to prove that it was the one forwarding). Based on what they received R_i makes a decision as follows.

²The essential part about only using authenticated channels is that the parties are not allowed to use a PKI. So they do not know each other verification keys before the protocol is run. Not having a PKI makes the problem impossible. One might think of just sending the public keys around. However, agreeing on the verification key vk_1 of P_1 is an agreement problem in itself. Actually the same agreement problem!

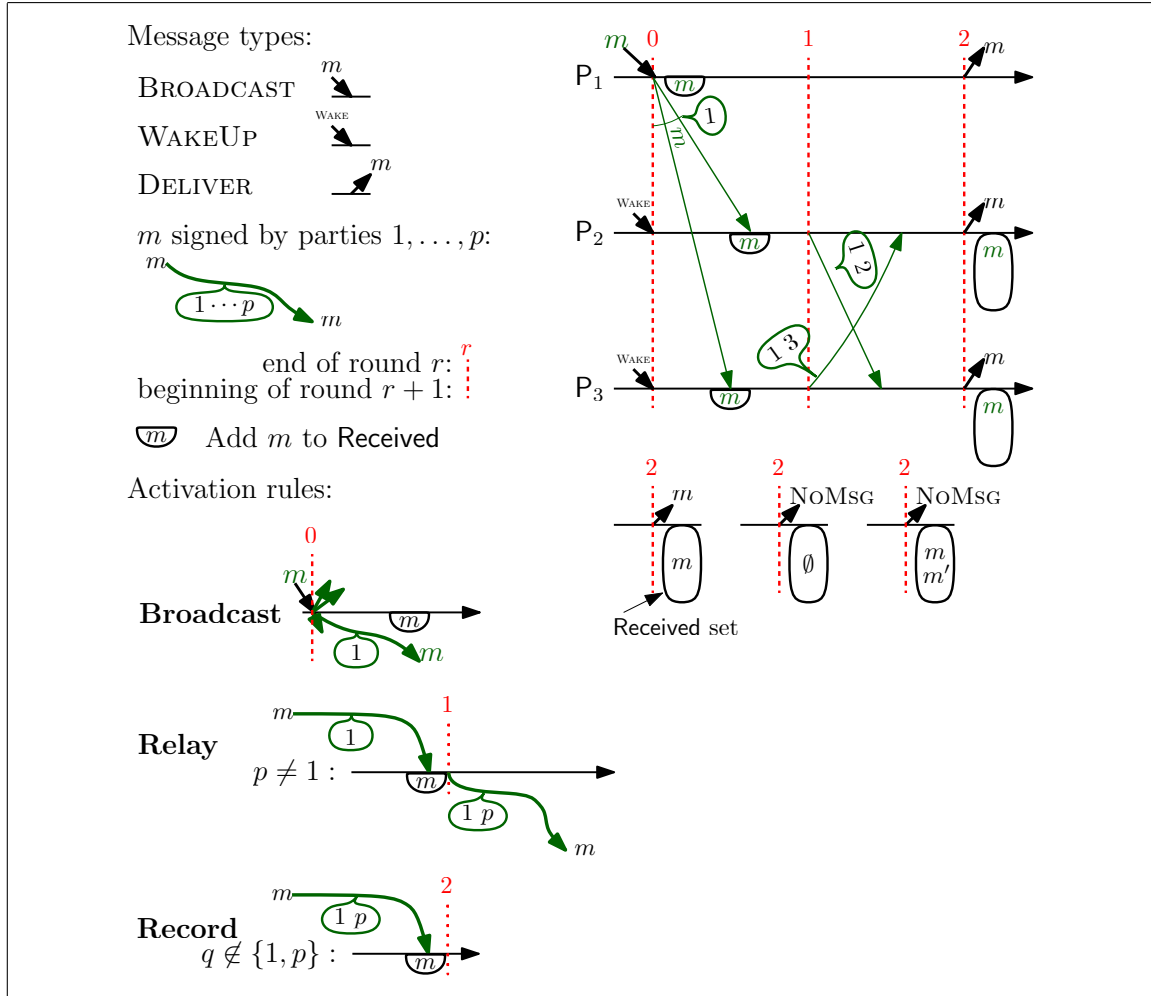


Figure 8.2: An illustration of Dolev-Strong for $n = 3$ and $t = 1$.

- If it received no message signed by S , neither from S directly, or indirectly via the other receiver, then it outputs $m_i = \text{NoMSG}$.
- If it received some m signed by S directly from S , but from the other receiver received another $m' \neq m$ also signed by S , then it outputs $m_i = \text{NoMSG}$.
- If it received some m signed by S directly from S and did not via the other receiver receive another $m' \neq m$ also signed by S , then it outputs $m_i = m$.
- If it received no m signed by S directly from S but indirectly via the other receiver received some m signed by S , then it outputs $m_i = m$.

This can be phrased concisely as follows:

1. Let Received be the set of messages m for which an m signed by S was received via S or via the other receiver.
2. If $|\text{Received}| \neq 1$ then output NoMSG. Otherwise, let m be the single element in Received and output $m_i = m$.

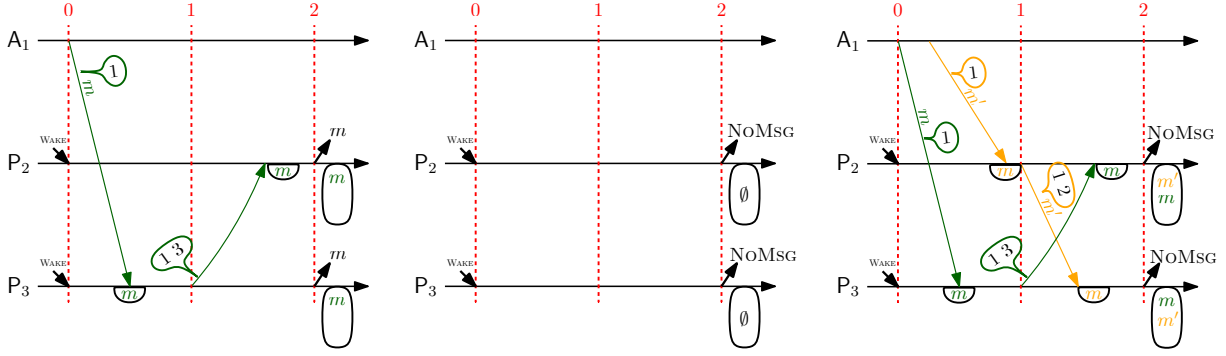


Figure 8.3: An illustration of Dolev-Strong for $n = 3$ and $t = 1$. Three types of corrupted sender: Sending only to one party, sending nothing, and sending different messages to different parties. In the last example, notice how it is important that we only use the *set* of messages received not their order, which differs between the parties.

The protocol is illustrated in Fig. 8.2 and Fig. 8.3.

We now argue that the protocol has all three properties required to be a Reliable Broadcast.

Agreement: R_1 and R_2 forward what they receive to the other party, so they end up with the same set *Received*. Each compute its output from *Received* using the same function. The same function applied to the same input gives the same output. This is illustrated in Fig. 8.3.

Validity: If S is honest, then it only signs one m . So, since we assume the signature scheme will not be broken, the receiver R_i will receive a unique signed m —it might receive m both from S and via the other receiver, but it will be the same value. So, $\text{Received} = \{m\}$ and therefore $|\text{Received}| = 1$. Hence the receivers output m .

Termination: no matter what they receive the receivers outputs something in round 2. ▼

Dolev-Strong is a generalisation of the above example, where all parties P forward a message m when they see it the first time. I.e., if they get it from S directly, they forward. But, also if they get a forwarded message in round 2 for the first time, then they re-forward it, and so on. This is done to make sure that all parties saw it: maybe it was a corrupted Q who sent m to P and Q did not send m to the other parties. To prevent that corrupted Q can forward some m very late, one must forward the set of signatures from earlier forwarders. So a message comes with a list a parties which forwarded it. Since there are only n parties, by round n there can be no more forwards and the protocol stops. We will put more detail on this below, but let us first define the problem properly.

8.2 Defining Reliable Broadcast

The first problem we will consider is the **reliable broadcast** problem. There are n parties $P_1, \dots, P_n$. One of them sends a message m to each of the other parties, which are supposed to receive the same message. For some bound t we want to tolerate that up to t parties are Byzantine corrupted. We discussed Agreement, Validity and Termination informally already. We now add a bit of detail.

First of all, to distinguish different runs of the protocol we introduce broadcast identifiers *bid*. When we receive a message m we need to know where it “goes”. It is just a way to name a particular broadcast, like *bid* = “the second broadcast of the protocol named Foo”. A value like (bid, P_i, m) then means the message m that P_i sent under the identifier *bid*.

Ports: For each party $P_i \in \{P_1, \dots, P_n\}$ there is a protocol port RB_i .

Syntax: The inputs on RB_i from parties P_i can be of the form (bid, P_i, m) , where bid is a broadcast identifier and m is a message. An honest party never sends $m = \text{NoMsg}$.

We require **unique identifiers**: each honest P_i only gives one input of the form $(\text{bid}, P_i, \cdot)$ per bid , i.e., honest parties never try to send different m under the same bid .

Furthermore, parties $P_j \neq P_i$ can give inputs of the form $(\text{bid}, P_i, \text{WAKEUP})$ indicating that they know that P_i is about to broadcast and that the protocol starts in the round where they themselves got this input.

Parties P_j can give outputs of the form (bid, P_i, m) indicating that they think P_i has broadcast m under bid .

When running synchronous protocols we use timeouts to determine when a sent message should have arrived, but if the sender did not know it should send the message, this is of little help. We therefore require from the users using the protocol that:

Synchronous Inputs: If in some round any honest party gives an input of the form $(\text{bid}, P_i, \cdot)$, then in that round all honest parties must give an input of the form $(\text{bid}, P_i, \cdot)$.

Note that if the sender P_i is corrupted, then the above means that all the honest parties input $(\text{bid}, P_i, \text{WAKEUP})$ in the same round. So, they all know that they are waiting for a message from P_i even if the corrupted P_i does not send the message. This allows to use a timeout and agree that the message was never sent. In our case this will lead to a special output NoMsg which is never sent by an honest party.

If the users fail to give synchronous inputs like this, all bets are off. We only require agreement and validity when the inputs were given like this. Conditioned on unique identifiers and synchronous inputs, the protocol should then have these properties:

Agreement If honest P_j outputs (bid, P_i, m_j) and honest P_k outputs (bid, P_i, m_k) , then $m_j = m_k$.

Validity If honest P_j outputs (bid, P_i, m_j) and the sender P_i is honest, then earlier P_i had input (bid, P_i, m_i) with $m_i = m_j$.

Strong Termination If an honest party gets an input of the form $(\text{bid}, P_i, \cdot)$ then eventually it also gives an output of the form $(\text{bid}, P_i, \cdot)$.

Strong termination is called such to distinguish it from asynchronous termination which does not guarantee termination when the sender is corrupted. There is a long discussion of that in Section 10.2.2.

8.3 Reliable Broadcast from a PKI

We now look at a solution to the reliable broadcast problem which works for any number of parties n and any number of Byzantine corruptions $t \leq n - 2$. In fact, the protocol also work for $t = n$ and $t = n - 1$, but these cases are trivial. The case of $t = n$ is trivial as there are no honest parties, and the protocol is only required to guarantee something about the inputs and outputs of the honest parties. When there are none, any protocol is secure. The case of $t = n - 1$ is also trivial as there is only one honest parties. It means that this party has to agree with itself, which it is trivial. Validity is slightly non-trivial: the protocol has to have the property that when, e.g., P_1 broadcasts m then P_1 will always output m , no matter what happens, but this is easy to ensure: let an honest sender of m always output m no matter what happens in the protocol. So, the first interesting case is $t = n - 2$. So here we, e.g., look at

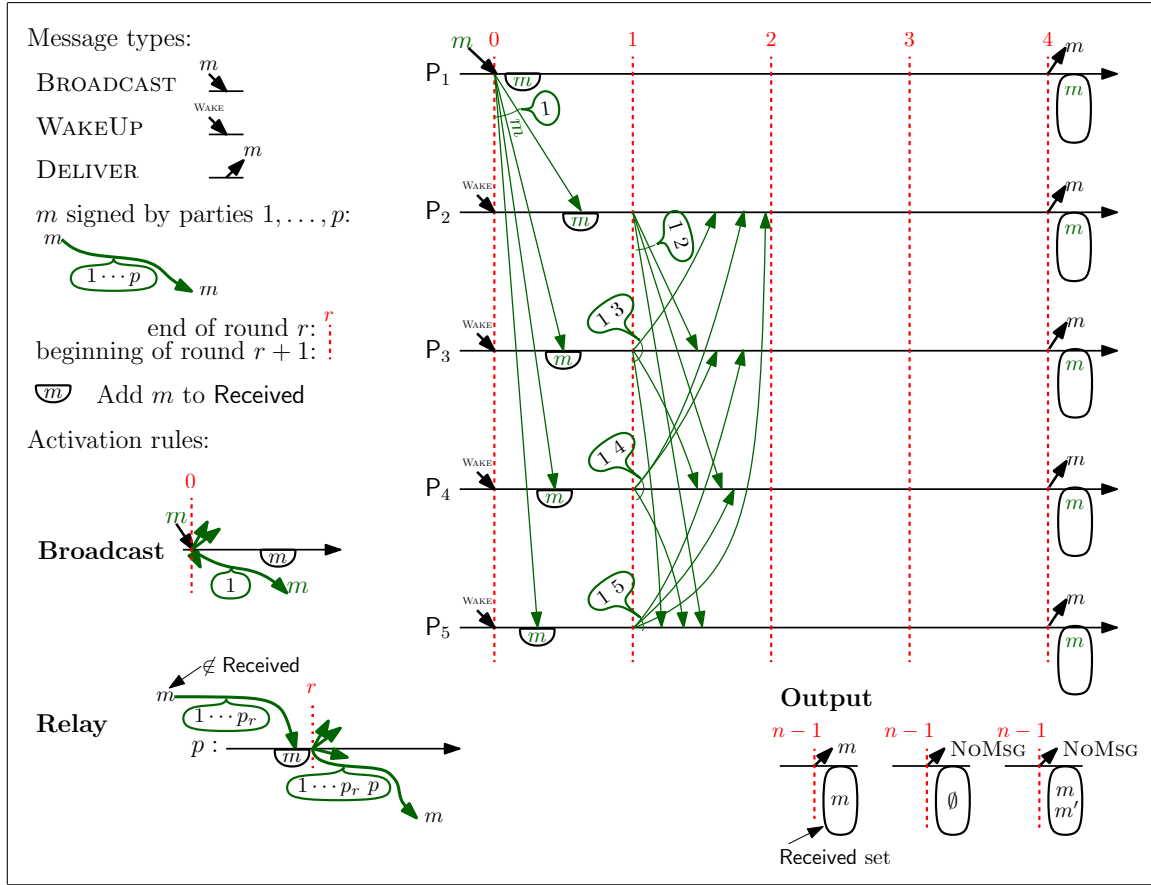


Figure 8.4: A run of DolevStrong with $n = 5$ and all parties honest. We explain the graphical notation. A BROADCAST message is used to tell a broadcaster, here P_1 , to broadcast a message. Other parties receive a WAKEUP message to indicate the beginning of round 1. A DELIVER message gives output from the protocol. We annotate a line with the message sent and who signed the message. If the message is not explicitly given, but the color of the line is preserved from a previous line, it means the same message was sent. A red, dashed line marked r indicates the end of round r , which is at the same time the beginning of round $r + 1$. A message m in a half ellipses means the party adds m to the set Received. The **Broadcast** activation rule say that on input m at the beginning of round 1 sign it and send it to all parties along with the signature. Then add m to Received. The **Relay** activation rule say that on receiving m signed by r parties during round r , and signed by the broadcaster P_1 , party p adds its own signature and sends m and all signatures to all parties. It is tacit in the rule that the message is only sent to parties which did not already sign the message. Then p adds m to Received. All this is not done if m was already in Received. The output is given in round $n - 1$, here 4. The output rule has three cases. If only one m was added to Received output m . If no message was added to Received, output NoMSG. If several messages were added to Received, output NoMSG. In the run of the protocol P_1 is honest and sends a signed m to all parties. They all add their signature and send m to all other parties. Then the protocol goes silent as all parties already have m in Received. At the end Received only contains m , so this becomes the output. It is tempting to ask whether the parties could not terminate earlier. We will later discuss why they cannot do this safely

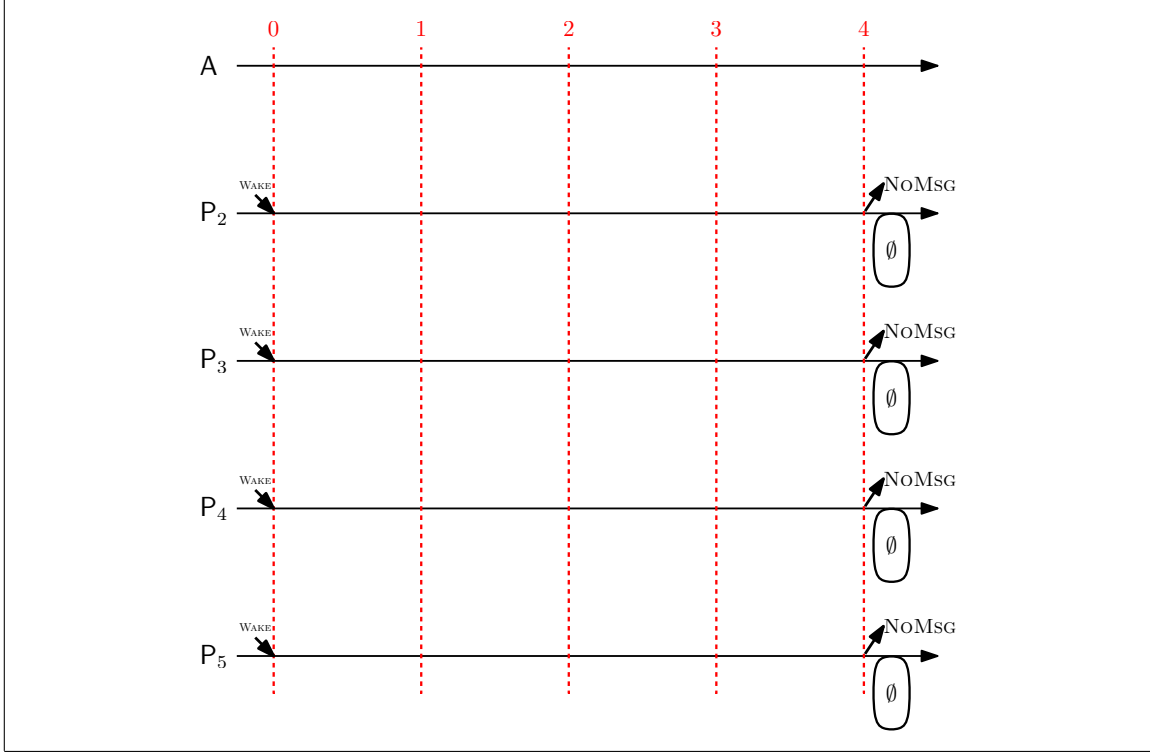


Figure 8.5: A run of **DolevStrong** with $n = 5$ and a corrupted sender, which sends no message. The parties wait until round 4, they see that $\text{Received} = \emptyset$ and output **NOMSG**. It is tempting to ask whether the parties could not terminate earlier. We will later discuss why they cannot do this safely.

1000 parties of which 998 are corrupted. Still the remaining two honest parties must end up agreeing. And they do not know who each other are!

Yet, there is a protocol solving this problem, called the **Dolev-Strong** protocol after the inventors. It is a protocol for the synchronous round-based model, where we assume that in each round, each honest party P has time to send a message to each other honest party. So if a message does not arrive from some party P , then we know that P deliberately did not send it. We assume there is some public-key infrastructure ensuring that each party P_i has a signing key sk_i for a signature scheme and that all other parties know the verification key vk_i , and importantly, that they *agree* on the verification key. That way, if a party accepts a signature from P_i it knows that it can relay it and have it accepted by the other parties. The protocol proceeds as follows:

Broadcast: When the sender gets some m to broadcast it signs m and sends m and the signature to all parties. For our example, let us assume P_1 is the sender. When P_1 is told to broadcast a message m it sets $r_1 = 0$ to indicate it is the beginning of round 1. It increments r_1 whenever one round elapses. Therefore, m is by definition always sent out at the beginning of round 1. Party P_1 also adds m to a set **Received** of messages it received so far.

Wake up: When a receiver $P_2, \dots, P_n$ gets input **WAKEUP** it indicates the beginning of round 1. Party P_i sets $r_i = 0$ and increments r_i whenever a round elapses. Recall that we assume all parties get their first input in the same round, so that all parties agree on what round

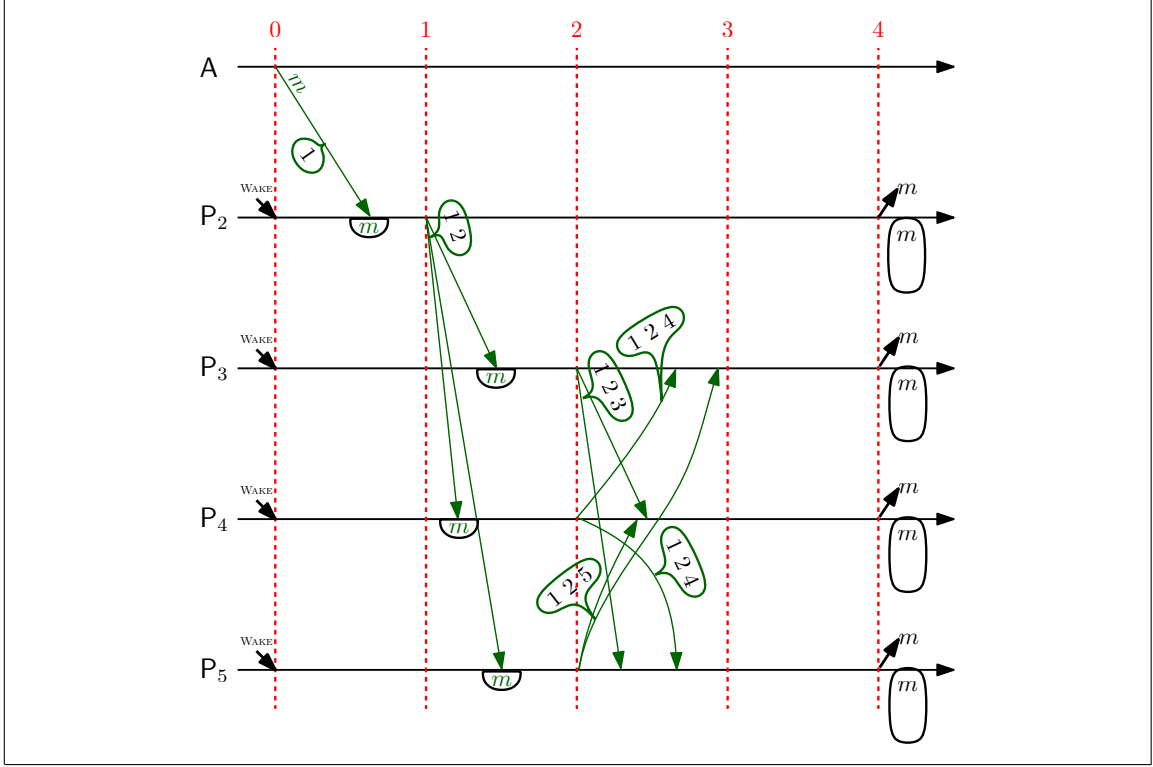


Figure 8.6: A run of DolevStrong with $n = 5$ and P_1 corrupted. The sender P_1 only sends the signed m to P_2 . Party P_2 adds m to **Received** and then forwards it to all parties, adding its own signature. It has to forward as it knows P_1 might be corrupted and not have sent m to all other parties. On receiving m signed by P_1 and P_2 in round 2 the other parties add m to their sets **Received**, add their own signature and forward to all other parties, who did not already sign the received message. On receiving these messages the parties do nothing as they already have m in **Received**. At the end **Received** = $\{m\}$ at all parties, so $|\text{Received}| = 1$ and they all output m .

they are in.

Relay 1: If during round 1 a party P_j received some m signed by P_1 then P_j also signs m and then sends m and the two signatures, the original from P_1 and the new one from P_j , to all other P_k , i.e., to all P_k which are not P_1 or P_j . This is always done at the beginning of round 2. **Record:** It also adds m to a set **Received** of messages it received so far.

Relay 2: If during round 2 a party P_k received some m signed by P_1 and one other party P_j , then P_k also signs m and then sends m and the three signatures, the two from P_1 and P_j and the new one from P_k , to all other P_ℓ , i.e., to all parties which did not already sign m . This is always done at the beginning of round 3. **Record:** It also adds m to a set **Received** of messages it received so far.

Relay r : In general, if during round r a party P_i received some m signed by P_1 and $r - 1$ other parties, then P_i also signs m and then sends m and the $r + 1$ signatures, the r it received and the new one from P_i , to all other P_k , i.e., to all parties which did not already sign m . This is always done at the beginning of round $r + 1$. **Record:** It also adds m to a set **Received** of messages it received so far.

Output At the end of round $n - 1$ all parties P_i give output m_i as follows.

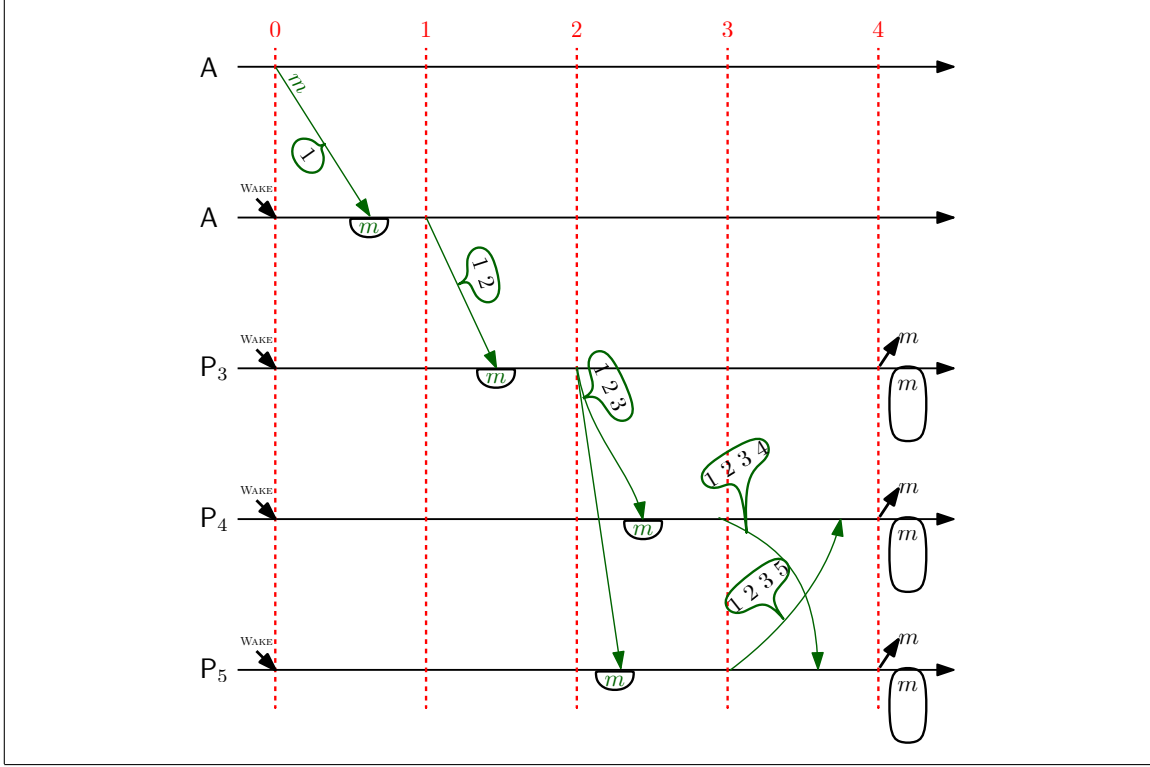


Figure 8.7: A run of DolevStrong with $n = 5$ and P_1 and P_2 corrupted. Party P_1 only sends m to P_2 , which in turn only sends it to P_3 , which then forwards it to P_4 and P_5 , which are the only parties not having signed m yet. These parties add their own signature and forward to the other. All honest only see m and therefore in the end output m .

- If $|\text{Received}| \neq 1$ then output NoMsg.
- Otherwise, let m be the single element in Received and output $m_i = m$.

Pruned Relay: Parties only forward a given message m once. The purpose of forwarding is to make sure all honest parties saw the same *set* of messages, so there is no need to forward the same m several times. Parties also do not forward m to a party P' from which they saw a signature on m . Such a party clearly already knows m .

In Fig. 8.4 there is an honest run. All parties end up sending one message to each other party and the protocol goes silent by the end of round 2, as parties only forward m the first time they see it.

In Fig. 8.5 there is an illustration of what happens when the corrupted broadcaster sends nothing. We later discuss why the parties have to run to the end in these examples before giving output. But, basically, at any point in time a new messages signed by S could arrive at just one honest party, which changes the output of that party. The party needs time to report this new message to the other parties to change their outputs too.

In Fig. 8.6 there is a run with a corrupted sender. It tries to create disagreement by sending m only to P_2 . Forwarding ensures agreement. Had the parties terminated in round 1 they would have had different outputs.

In Fig. 8.7 there is a run showing why the parties need to do the forward in round 2 in Fig. 8.6. Party P_3 , for instance, cannot rule out that both P_1 and P_2 are corrupted and only

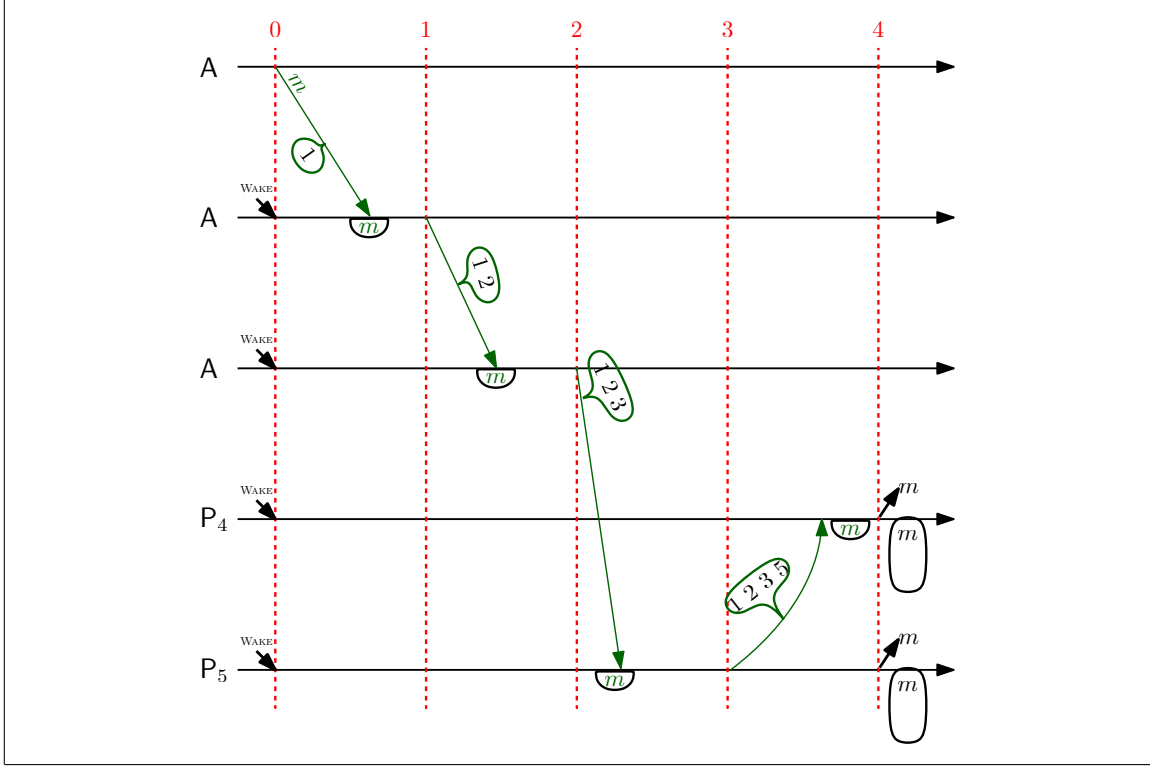


Figure 8.8: A run of DolevStrong with $n = 5$ and P_1 , P_2 and P_3 corrupted. Party P_1 only sends m to P_2 , which in turn only sends it to P_3 , which in turn only sends it to P_5 , which then forwards it to P_4 . Party P_4 only see m and therefore in the end outputs m . Note that P_5 hears nothing until the last round, so if it had terminated earlier with output NoMSG it would have made a mistake.

sent m to P_3 , so it forwards to ensure P_4 and P_5 saw m too. Had the parties terminated in round 2 they would have had different outputs.

In the same way Fig. 8.8 shows why the parties need to do the forward in round 3 in Fig. 8.7. Party P_5 , for instance, cannot rule out that all of P_1 , P_2 , and P_3 are corrupted and only sent m to P_5 , so it forwards to ensure P_4 saw m too. Had the parties terminated in round 3 they would have had different outputs.

However, by round 4 party P_4 does not have to forward m anymore. It is already signed by P_1, P_2, P_3, P_5 , and only these parties can sign on behalf of themselves, so the signatures show that they already saw m . So all other parties already saw m , ergo, no reason to forward it.

To further illustrate the protocol, see Fig. 8.9 where a corrupted sender and its accomplices P_2 and P_4 try to create disagreement by sending several messages. But since the honest parties see both, the honest parties output NoMSG and thus will agree. The crucial idea is that for all message m and m' that a corrupted party sends, they are all forwarded. So all honest parties will see the same *set* of messages. Here $\text{Received}_i = \text{Received}_j = \{m, m'\}$. And we take the decision based on the set only, not the order in which messages were seen.

Finally, Fig. 8.10 illustrates why parties cannot terminate early either after seeing an otherwise honest looking run. Another message might arrive later and at different times at different parties. It might be tempting to ignore this “second” message, as it clearly shows that P_1 is corrupted, and in this example it even arrives from a corrupted party. But trying to ignore it opens up all the same problems as before. For instance, in Fig. 8.11 the parties do not agree

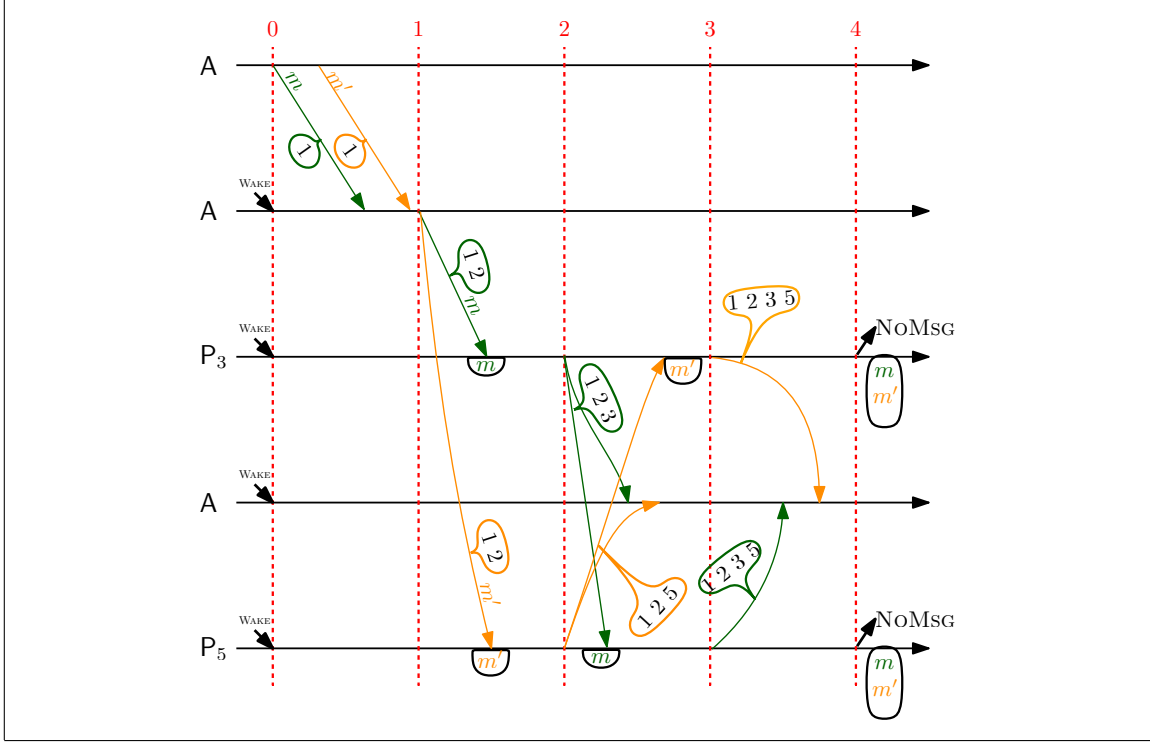


Figure 8.9: DolevStrong with $n = 5$ and P_1, P_2, P_4 corrupt. It illustrates why we should not use the order in which messages were seen in the decision rule. Party P_3 sees m first (in round 2) and then m' (in round 3). Party P_5 sees m' first and then m . However, the Received-sets will be the same as $\{m, m'\} = \{m', m\}$. We must therefore base the decision only on the sets of received messages if we want agreement.

on which message is second. In the next chapter we will prove that there is no protocol which allows to terminate early. This lower bound is under the restriction that all parties must give outputs in the same round. If we allow that they sometimes give outputs in different rounds there are situation where one could give outputs earlier. See for instance Fig. 8.12, where the parties give outputs after having added the second message to Received, at which point they know that the output will be NoMsg.

With these examples under the skin, it should be clear that Dolev-Strong implements RB. We here only draw out the main points. Strong Termination is trivial as all parties stop after having run for $n - 1$ rounds.

The crucial idea behind Agreement is:

Because a party always relays a received message m to the parties which might not have seen it yet, at the end all parties will always have seen the same set of message. And the output only depends on the set of messages seen.

I.e., if Received_i is the set of honest P_i and Received_j is the set of honest P_j then $\text{Received}_i = \text{Received}_j$, so the parties will output the same message. This gives Agreement.

The crucial idea behind Validity is:

Because a message is only accepted if signed by the original sender, only the message intended to be sent by an honest sender will be accepted.

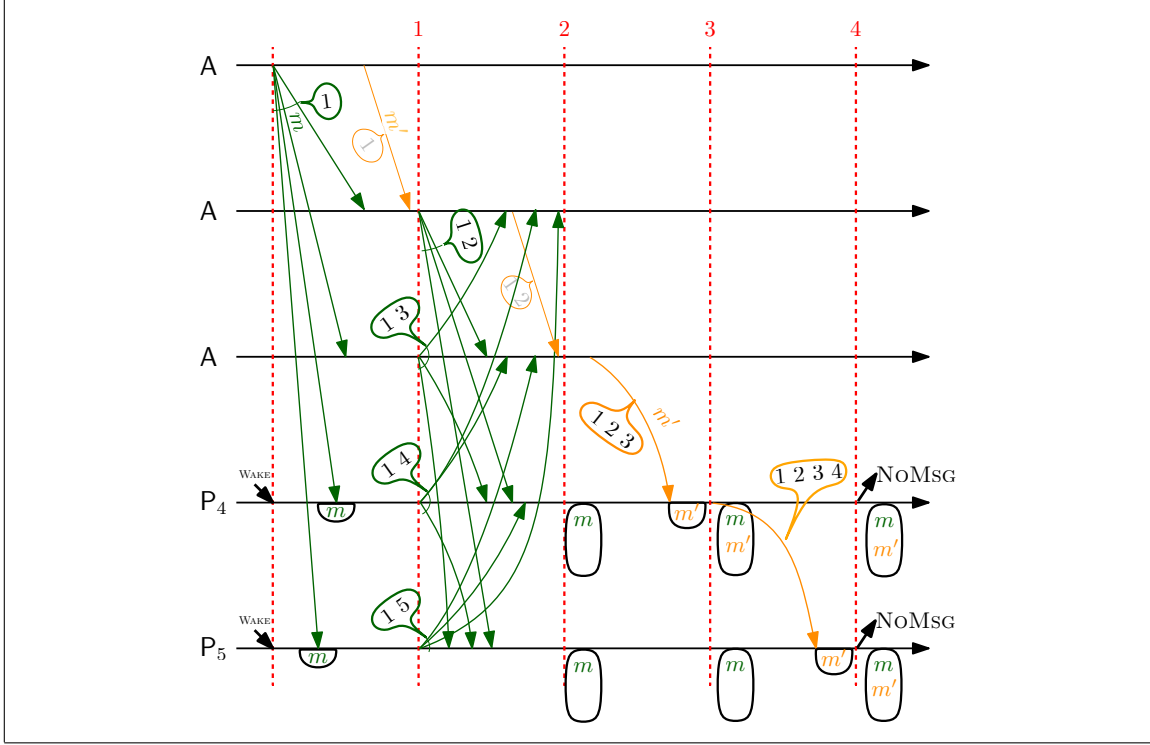


Figure 8.10: A run of DolevStrong with $n = 5$ and P_1 , P_2 and P_3 corrupted. It illustrates why an honest run like Fig. 8.4 cannot terminate early. All can look good until round 3, where the corrupted P_3 throws in a new message m' cooked up with its colluders P_1 and P_2 . Now P_4 needs time to relay to P_5 , so one cannot terminate at the end of round 3 either if all looks good, as it does for P_5 . The parties could not have terminated at the end of round 2 either. Imagine that P_2 was honest and the first to see m' . Now P_2 outputs NoMsg and needs time to relay m' to make P_5 do the same.

Since P_1 sends the signed m to all parties in the first round, it ends up in Received in **Relay 1**. And by the above observation, no honest party will add another m' to Received. So at the end Received = $\{m\}$. This immediately gives Validity.

8.3.1 Dolev-Strong for Less Corruptions

If one does not want to tolerate the maximal number of corruptions $t = n - 2$, then one can make the protocol use less rounds. We sketch how. Take any number of parties and any $t < n - 2$. This could for instance be $n = 1000$ servers and we want to tolerate $t = 10$ corruptions. Then one can run Dolev-Strong as above but compute the outputs already in round $t + 1$. The same rule is used to compute the outputs from the value of Received, but the parties just use the value of Received at the end of round $t + 1$, so in our example that would be round 11 instead of running for 999 rounds.

The crucial observation is the following.

If a party P received a message m in round $t + 1$, then it was signed by $t + 1$ parties. Since there are at most t corrupted parties, we know that at least one honest party signed it already.

This honest party already sent m to all other honest parties as it should.

So, if a party P received a message m in round $t + 1$, all honest parties received m already. Then

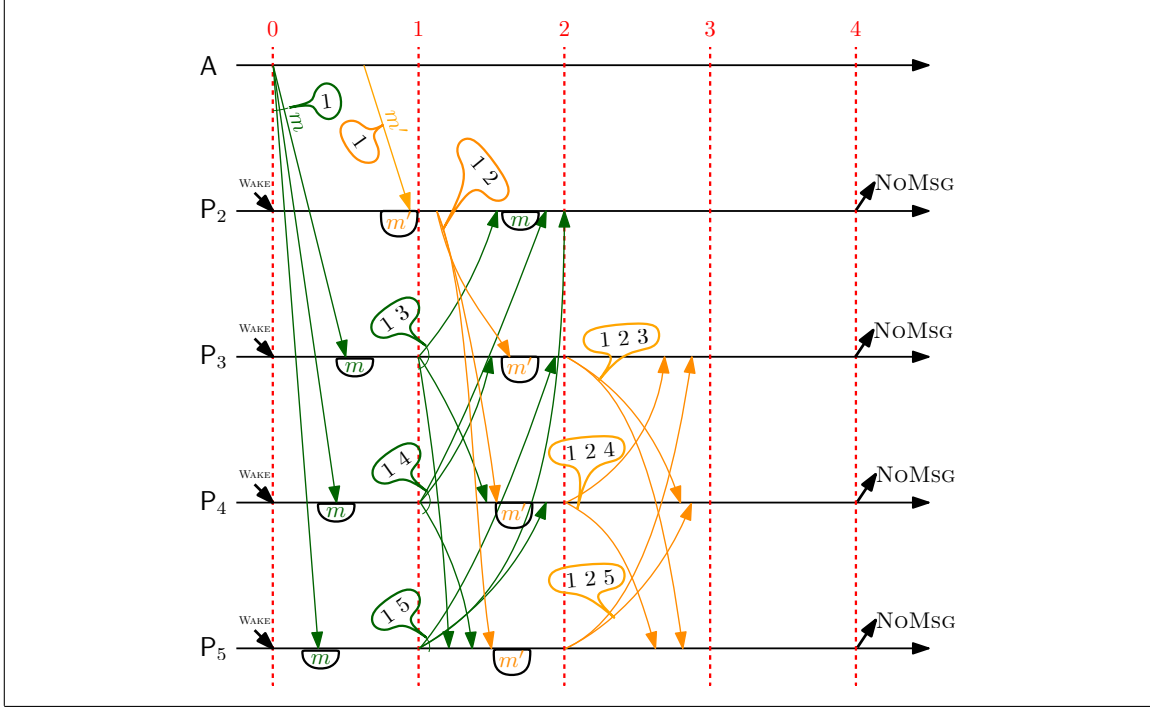


Figure 8.11: An illustration why parties cannot just ignore the “second” message, assuming it comes from corrupted parties. Party P_2 might be honest and not have seen m when it sent m' . So, if P_3 ignored m' it would ignore a message from an honest party and end up with another Received set. Notice that here P_2 does not send out m . It already saw m signed by P_1 , P_3 , P_4 and P_5 , so all parties saw it, so no need to forward. Note how in this example, all honest parties know that the result will be NoMsg by round 2. The reason why parties cannot give an output of NoMsg when learning that this will be the result is that we require that they must terminate in the same round. Here they would, but see Fig. 8.12.

there is no reason for P to relay it. If on the other hand P received a message m in round t or earlier, it still has time to relay it. As an example consider Fig. 8.7. If P_4 knew that there could be at most two corrupted parties, then when receiving m signed by P_1 , P_2 and P_3 in round 3 it would not have to relay it to P_5 , as it would know that some honest party already did so—in Fig. 8.7 it is P_3 which was honest and already relayed the message. And a party like P_3 still have time to relay before round $t + 1 = 3$.

8.4 Lower Bound on Round-Complexity of Reliable Broadcast

Recall that the DolevStrong protocol runs in $n - 1$ rounds, i.e., the outputs are given at the end of round $n - 1$ when tolerating $t = n - 2$ corruptions. One should wonder if that many rounds are really needed? Or did we overlook some neat protocol using only three rounds no matter the number of corruptions?

No, if an RB protocol tolerates $t = n - 2$ Byzantine corruptions, then it *must* use $n - 1$ rounds. In this section we prove that fact. We will do a proof by contradiction. We will assume that for some n there exists a protocol which does Reliable Broadcast, which tolerates $n - 2$ corruptions and never uses more than $n - 2$ rounds of communication. From this we reach an absurdity. This shows that the assumption is false, i.e., for all n there does not exist a protocol

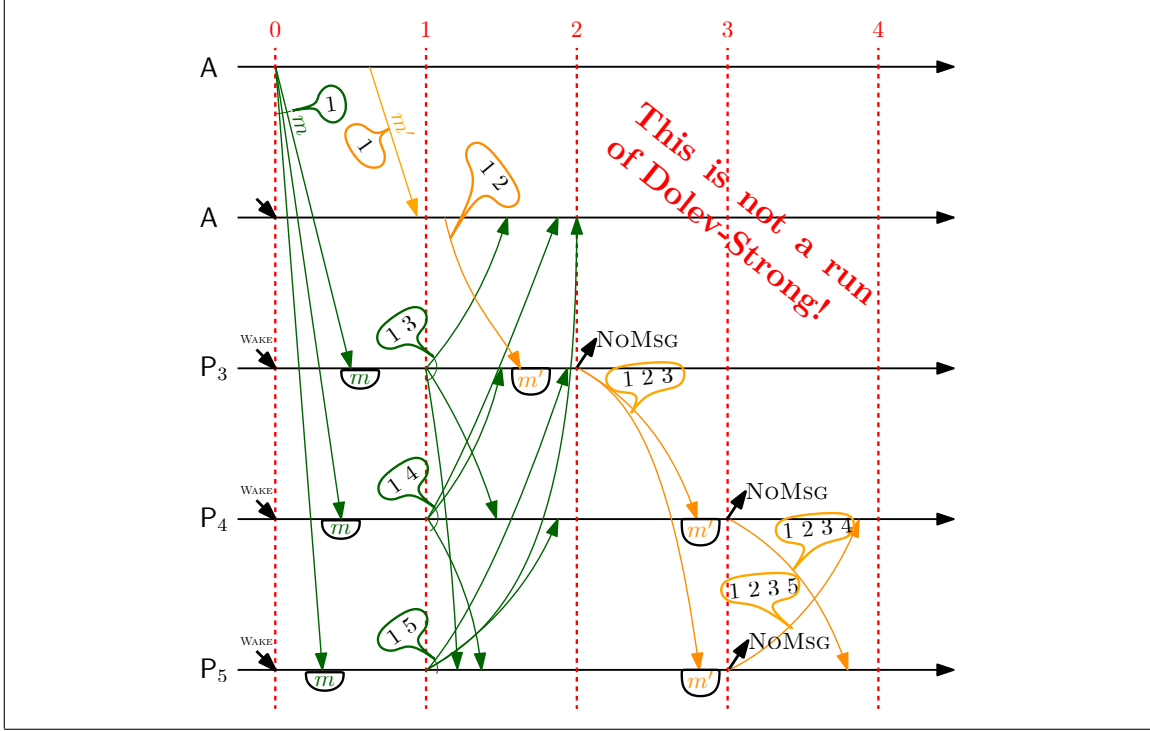


Figure 8.12: An illustration why parties cannot just output when they learn the result is NoMsg. It could have them output in different rounds.

which does Reliable Broadcast, tolerates $n - 2$ corruptions and never uses more than $n - 2$ rounds of communication. Ergo, for all n it holds that if a protocol does Reliable Broadcast and tolerates $n - 2$ corruptions, then it sometimes uses more than $n - 2$ rounds of communication, i.e., $n - 1$ rounds, as we desired to prove.

So, assume that for some n there exists a protocol which does Reliable Broadcast, i.e., it has Validity and Agreement, which tolerates $n - 2$ corruptions and never uses more than $n - 2$ rounds of communication.

We consider protocols which use setup, i.e., we imagine some trusted party generated possible correlated values and gave them to the parties. It could for instance be that each P_i gets a random secret key sk_i for a signature scheme and the other parties are given the verification key vk_i . But we allow the setup to be arbitrary, not just a PKI. Having arbitrary setup just makes the result stronger. We show that $n - 1$ rounds are needed no matter what kind of help the parties get as long as this help is independent of m .³ A protocol for $n = 5$ parties with setup and $n - 2 = 3$ rounds is illustrated in the top of Fig. 8.13.

To work our way towards the absurdity, consider the top diagram in Fig. 8.14. Here P_3, P_4, P_5 are corrupted. They run honestly, except that they ignore the incoming messages which crossed the orange line. They just replace them with NoMsg, and then otherwise run honestly. This is an allowed execution in our model, as we only use $t = 3$ corruptions. Since we assume the protocol has Validity against three corruptions we can conclude that P_2 outputs $m_2 = m$. Namely, the sender is honest, so all honest receivers output the input of the sender.

Then consider the mid diagram in the same figure. Here P_1, P_4, P_5 are corrupted. Party P_1 will drop the outgoing messages which would cross the orange line and otherwise run honestly.

³We could, in fact, allow the parties to get a fresh round of help in each round, as long as it is independent of m , but it would clutter the figure.

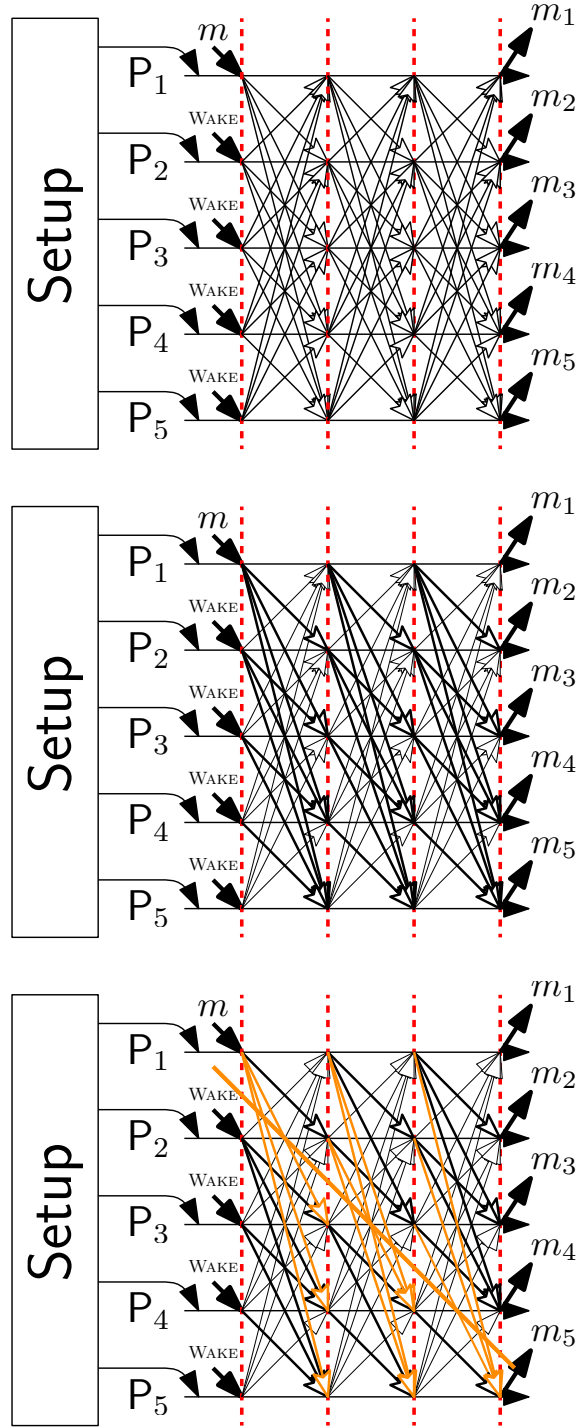


Figure 8.13: Illustration of the impossibility of RB in $n - 2$ rounds with $t = n - 2$ corruptions. Top: a protocol with $n = 5$ parties and $n - 2 = 3$ rounds. There is some trusted party handing out a secret signing key for each party and the corresponding verification key to all other parties. This allows to use signatures, as in Dolev-Strong. There are three rounds of communication. There is an arrow for each possible message in the protocol. Middle: Our proof will try to prevent the input m of P_1 to reach P_5 , so we are focused on “downwards” communication and ignore the “upwards” ones. Bottom: we can draw a diagonal line separating P_1 from P_5 . If we can stop the 10 messages crossing this line in the downwards direction, then information about m cannot reach P_5 .

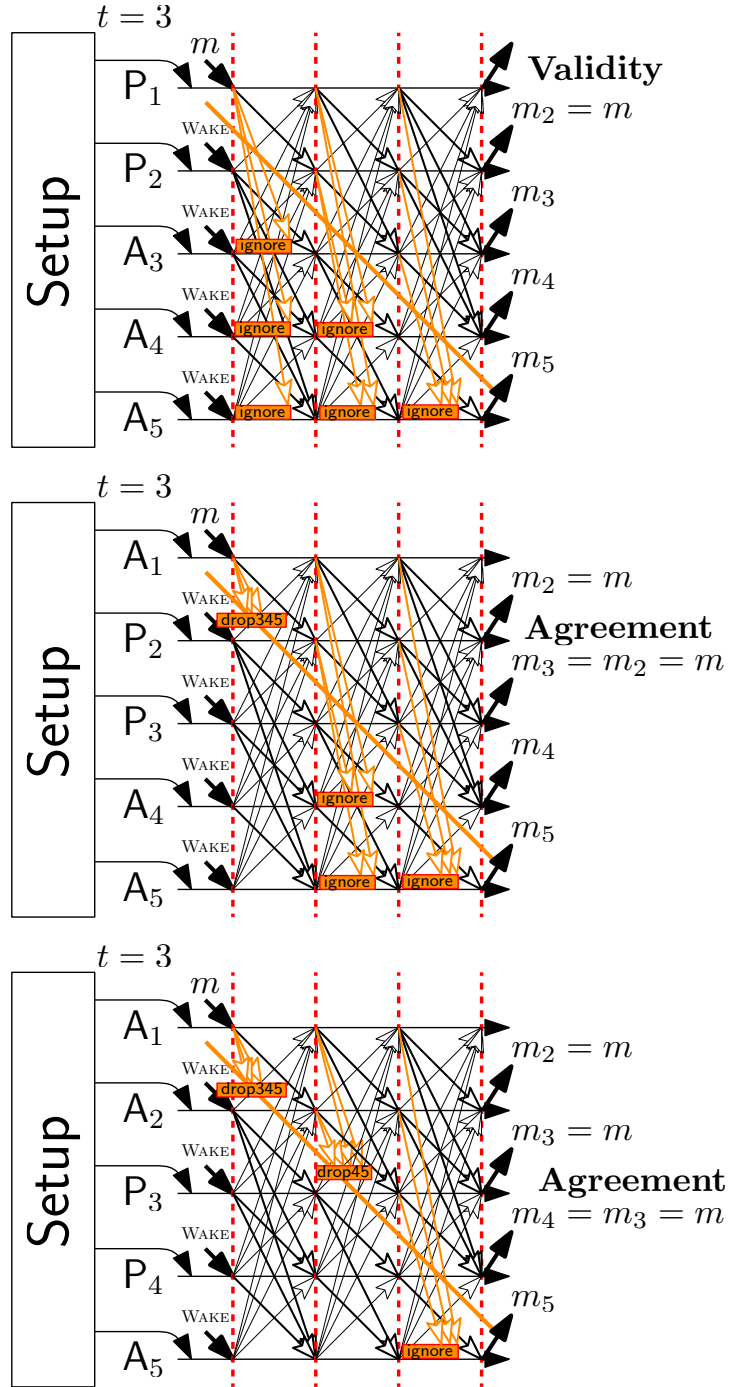


Figure 8.14: Illustration of the impossibility of RB in $n - 2$ rounds with $t = n - 2$ corruptions. Top: We corrupt P₃, P₄, and P₅. They run almost correctly, except they ignore messages which crossed the orange line. For instance A₃ runs a copy of the honest P₃, except that in the first round it replaces the message from P₁ by NoMSG, so that it looks to P₃ as if P₁ was corrupted and did not send anything in the first round. Middle: now it is actually P₁ who is corrupted. Party A₁ runs the honest P₁ except that it drops the messages to parties P₃, P₄, P₅ in the first round. Parties P₄ and P₅ still ignore messages which crossed the orange line. The same 10 messages are being dropped, now just at the sender for three of them. So the execution is the same and ergo it still holds that $m_2 = m$. Now, by agreement the honest P₂ and P₃ output the same, so additionally $m_3 = m_2$. Bottom: P₁, P₂ are corrupted and drop messages crossing the orange line. Party P₅ ignores the same messages as before. The same execution is produced, so still $m_3 = m$. By agreement the honest P₃ and P₄ output the same, so $m_4 = m_3$.

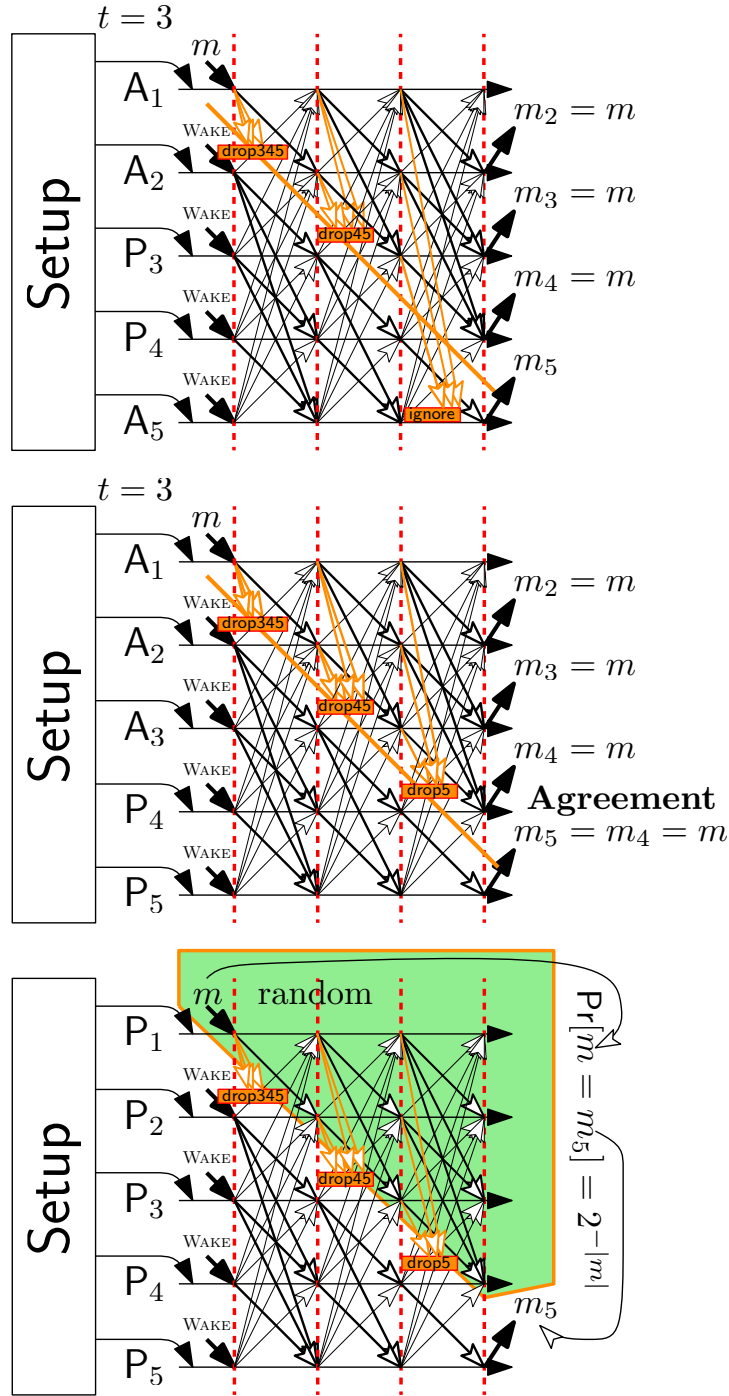


Figure 8.15: Illustration of the impossibility of RB in $n - 2$ rounds with $t = n - 2$ corruptions. Top: continued from Fig. 8.14. Mid: We corrupt P_1, P_2, P_3 and let them drop messages about to cross the orange line. The same execution results, so still $m_4 = m$. By agreement the two honest parties output the same, so $m_5 = m_4$. All in all, this gives us that $m_5 = m$. Bottom: information does not leave the green region and m is inside and m_5 outside. So if m is a uniformly random bit-string, the chance that $m_5 = m$ cannot be better than guessing m . But then we both conclude that $m_5 = m$ and that $m_5 \neq m$.

Parties P_4 and P_5 also run honestly, except that they drop the incoming messages which were dropped in the top diagram. This is again an allowed execution in our model, as we only use $t = 3$ corruptions. Since we assume the protocol has Agreement against three corruptions we can conclude that the two honest parties P_2 and P_3 output the same, i.e., $m_3 = m_2$. Since we generate the same execution as in the top diagram we still have that $m_2 = m$. Therefore $m_3 = m$.

Consider the bottom diagram then. Here P_1, P_2, P_5 are corrupted. Parties P_1 and P_2 will drop the outgoing messages which would cross the orange line and otherwise run honestly. Party P_5 also runs honestly, except that it ignores the incoming messages which crossed the orange line. We only use $t = 3$ corruptions, so by Agreement we have that $m_4 = m_3$. Since the execution is the same as in the middle diagram we still have $m_3 = m$, ergo $m_4 = m$.

In Fig. 8.15 we start with the bottom diagram from Fig. 8.14. In the middle diagram P_1, P_2, P_3 are corrupted. They run honestly except that they all drop the outgoing messages which would cross the orange line. We only use $t = 3$ corruptions, so by Agreement we have that $m_5 = m_4$. We generate the same execution as in the top diagram so $m_4 = m$, and thus $m_5 = m$.

But, now consider what would happen if we input a uniformly random 100-bit message to P_1 . All arrows going out of the green region are dropped. Therefore, no information about m can travel outside the green region.⁴ Therefore m_5 is independent of m . Since m is a 100-bit uniformly random message it follows that $\Pr[m = m_5] = 2^{-100}$. In general $\Pr[m = m_5] = 2^{-|m|}$. But this clearly contradicts that $m_5 = m$.

8.4.1 Generalizing the Result

One can generalize the above result in several ways. One can prove that for any t , not just $t = n - 2$, one needs $t + 1$ rounds for RB. So for $n = 1000$ servers and a protocol which should tolerate just $t = 10$ corruptions, one needs 11 rounds. This result is surprisingly much more complicated to prove and we will not discuss it further. The interested reader will find a proof in [6].

The result for $t = n - 2$ holds even if Validity and Agreement are allowed to fail with probability $\leq 1/n$. In this case we can use the same arguments as above to conclude that $\Pr[m_5 \neq m_4] \leq 1/n$, $\Pr[m_4 \neq m_3] \leq 1/n$, $\Pr[m_3 \neq m_2] \leq 1/n$, and $\Pr[m_2 \neq m] \leq 1/n$. From these we can conclude that $\Pr[m_5 \neq m] \leq 4/n$, or equivalently $\Pr[m_5 = m] \geq 1/n$, as $n = 5$. In general one gets that $\Pr[m_n = m] \geq 1/n$. But we can still prove that $\Pr[m_n = m] \leq 2^{-|m|}$. Setting $|m| = \log_2(n) + 1$ this gives us that $\Pr[m_n = m] \leq \frac{1}{2n}$, which contradicts $\Pr[m_n = m] \geq 1/n$.

8.5 No RB for Three Parties without Setup

Above we constructed a reliable broadcast protocol for any $t \leq n - 2$ Byzantine corruptions using signatures. We could ask ourselves if we could do the same using only authenticated channels. It turns out that $t < n/3$ is the best we can do. In this section we present a proof

⁴This is not completely trivial: if for instance P_3 dropped a message to P_5 when the first bit of m was 0 and sent a message when the first bit was 1, then when the first bit is 0, there would be no message leaving the green section, yet P_5 would learn the first bit of m : when no message arrives then the first bit is 0. But here we fixed that no messages would be sent *before* m was sampled and then clearly information cannot cross out of the green region. The green region is the causal future of m even in the synchronous model. An easy way to see this is that since no messages are sent out of the green region, we could compute m_5 before we input m to P_1 . So, clearly m_5 cannot depend on m .

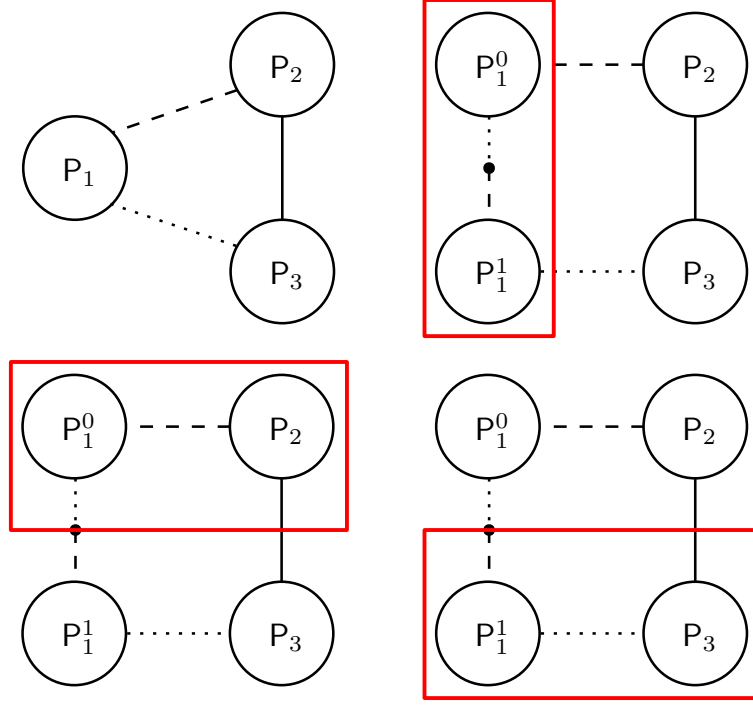


Figure 8.16: Illustration of the impossibility of RB for $n = 3$ and $t = 1$ without using setup.

of this. What we prove is that if $n = 3t$ then no protocol for Reliable Broadcast can tolerate t Byzantine corruptions if it is allowed to only use authenticated channels.⁵

We first prove the result for $n = 3$ and $t = 1$. Assume for the sake of contradiction that we have a protocol π with three parties P_1, P_2, P_3 which solves the reliable broadcast problem with tolerance against $t = 1$ Byzantine corruptions. Assume the protocol is for the model with pairwise authenticated channels *and no setup*. The protocol is illustrated as the upper left system in Fig. 8.16.

We now describe a (rather strange) way in which a corrupted broadcaster might behave, let us call it $\widetilde{P}_1$. What it will do is to run two copies of the *honest* algorithm for P_1 : one with input 0 and one with input 1. Call them P_1^0 and P_1^1 . When executing the protocol, they will be interconnected as follows:

- Whenever P_1^0 tries to send a message to P_3 , instead send it to P_1^1 . Whenever P_1^0 receives a message from P_1^1 treat it as if it comes from P_3 .
- Whenever P_1^1 tries to send a message to P_2 , instead send it to P_1^0 . Whenever P_1^1 receives a message from P_1^0 treat it as if coming from P_2 .

Finally, connect $\widetilde{P}_1$ to P_2 and P_3 as follows:

- Connect P_1^0 to P_2 via the channel they normally would use.
- Connect P_1^1 to P_3 via the channel they normally would use.

⁵In Section 10.3 we prove a similar result. We show that in the asynchronous model there is no reliable broadcast for three parties, even when allowing the use of signatures. This result is incomparable to the one we prove here, i.e., neither of them imply the other. The impossibility result in Section 10.3 is stronger in the sense that we rule out RB even when using signatures. On the other hand the impossibility result in Section 10.3 is weaker in the sense that it is for the asynchronous model, where one is not allowed to use timeouts, so it is easier for tasks to become impossible.

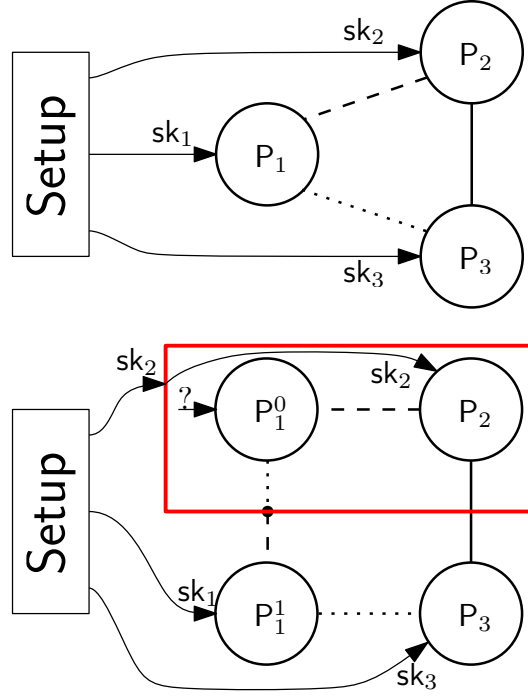


Figure 8.17: Why the proof in Fig. 8.16 does not apply when there is setup. A corrupted P_2 cannot run a copy of P_1 as it does not know the secret values sk_1 of P_1 given out by the trusted party.

The result is illustrated in the upper, right corner of Fig. 8.16. Recall that even if there are actually four parties in the protocol, we may think of $\widetilde{P}_1 = \{P_1^0, P_1^1\}$ as one corrupted party. Since π can tolerate one Byzantine corruption it follows from the agreement property that whatever $\widetilde{P}_1$ is doing, party P_2 and P_3 will get the same output. So when this strange four-party system is run we can conclude that $m_2 = m_3$.

Consider now the system in the lower, left corner. It is the *same* system as before, but now we consider $\{P_1^0, P_2\}$ as a single corrupted version of P_2 . Note, in particular, that it is connected to P_1^1 and P_3 exactly as P_2 would be. Since P_1^1 is running honestly as broadcaster with input $m = 1$ and since P_3 is honest and π tolerates one corrupted party (here P_2) it follows from the validity property that P_3 will output 1. Furthermore, since P_3 's view of the protocol is completely the same in this case as in the former case where P_1 was corrupt, its output is the same in both cases. So we conclude that $m_3 = 1$.

Consider now the system in the lower, right corner. It is the same system as before, but now we consider $\{P_1^1, P_3\}$ as a single corrupted version of P_3 . Since P_1^0 is running honestly as broadcaster with input $m = 0$ and since P_2 is honest and π tolerates one corrupted party (here P_3) it follows from the validity property that P_2 will output $m_2 = 0$.

From the last two ways of looking at the system, we can conclude that $0 = m_2 \neq m_3 = 1$, but we also concluded before that $m_2 = m_3$ which is of course a contradiction. So, it must be that no 3-party protocol exists that tolerates 1 Byzantine corruption.

To see why this proof would not work for the Dolev-Strong protocol, note that when we use a PKI then P_1 receives a secret key sk_1 that only P_1 knows. Then the corrupted $\{P_1^0, P_2\}$ in Fig. 8.16 cannot run as in the protocol. In the protocol P_1^0 should have sk_1 as input. But this is only given as input to the honest P_1^1 . This is illustrated in Fig. 8.17.

This, in particular, means that we were well justified in using a PKI for the case $t = n - 2$.

8.6 Byzantine Agreement from Reliable Broadcast

In our next study of the round-based synchronous model we will look at Byzantine Agreement (BA). In an agreement protocol each party has a vote and the goal is to end up with a common, meaningful decision.

In more detail, a Byzantine Agreement protocol is a protocol between n parties, which as before are denoted as $P_1, \dots, P_n$. The input of P_i is a vote $v_i \in \{0, 1\}$. The output of P_i is a decision $d_i \in \{0, 1\}$. We require the following properties.

Validity: If honest P_i outputs d_i then there exists an honest party P_j such that $d_i = v_j$.

Agreement: If honest P_i outputs d_i and honest P_j output d_j , then $d_i = d_j$.

Strong Termination: If P_j is honest and is given input, then eventually it gives output.

Strong Termination says that all honest parties who start running the protocol eventually compute a result. Agreement just says that all honest parties will agree on the result they compute. Validity says that the result must be a value that was voted for by at least one honest party (this is the part making the result meaningful). In particular, if all honest parties vote for the same value, then all honest parties must eventually agree on this value.

Notice that if there are n parties and up to t Byzantine parties, then Byzantine agreement is impossible if $t \geq n/2$. This is for purely definitional reasons and does not involve an actual attack. To see this, assume that $n = 2t$. Consider an execution where $P_1, \dots, P_t$ have input 0 and $P_{t+1}, \dots, P_n$ have input 1. Assume that half the parties are Byzantine corrupted but follow the protocol honestly. By termination and agreement, the parties will eventually agree on some output b . Assume now that the honest parties were $P_1, \dots, P_t$. Then validity requires that $b = 0$. But if instead the honest parties were $P_{t+1}, \dots, P_n$, then validity requires that $b = 1$. Since all parties follow the protocol as prescribed, there is no algorithmic way to determine who is corrupted, so the algorithm cannot determine which case it is in. So whatever it outputs, it will always be wrong in one of the cases. When we work with Byzantine agreement, we therefore always assume that $t < n/2$ of the parties are Byzantine corrupted.

So BA is impossible when $t \geq n/2$. On the other hand, it is possible whenever $t < n/2$ if we have access to an ideal functionality RB for reliable broadcast, which is secure for the same t . We simply run n instances of the protocol, where each P_i broadcasts their vote v_i . This ensures that all parties learn a vector $(v_1, \dots, v_n)$ of votes that they all agree on. If some $v_i = \text{NoMsg}$, then simply let $v_i = 0$. Then, take the value v which occurs most often in this vector and let the result be $d_i = v$.

Termination follows from termination of the underlying reliable broadcast protocol. Agreement follows from agreement of the reliable broadcast protocol: the parties will agree on $(v_1, \dots, v_n)$ and will therefore compute the same output. Let us then argue validity. Assume that there are two honest parties, one with $v_i = 0$ and one with input $v_j = 1$. Then validity is trivially fulfilled as all parties output 0 or 1. Assume then that there is a value v such that $v_i = v$ for all honest P_i . Then by validity of the reliable broadcast protocol, the value v occurs $n - t$ times in $(v_1, \dots, v_n)$ as there are at least $n - t$ honest parties and they all have input v . Since $n - t > t$, it follows that all honest parties take their output to be $d_i = v$.

8.7 Reliable Broadcast from Byzantine Agreement

In this section we argue that one can implement Reliable Broadcast given access to an ideal functionality BA for Byzantine agreement. The protocol is very simple, so we will skip most of

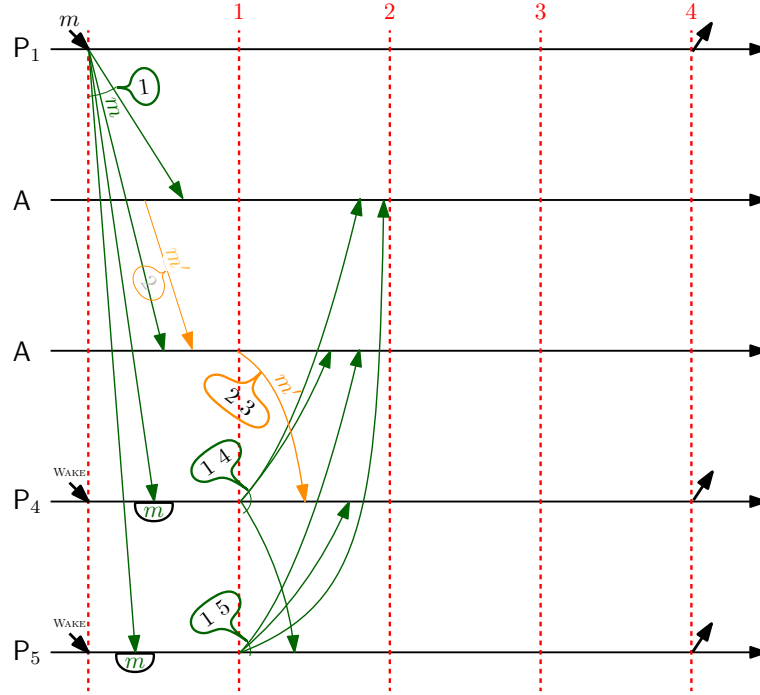


Figure 8.18: Parties P_2 and P_3 were corrupted. Start the execution at the beginning of round 3. What messages will the honest parties send from now on: what messages in what rounds? And what will their outputs be?

the details. Assume that a sender wants to reliable broadcast a bit b to n parties $P_1, \dots, P_n$. It sends b to all parties. Let b_i be the bit received by P_i . Then $P_1, \dots, P_n$ run BA on $b_1, \dots, b_n$. Let the result be r . All parties output r . Agreement of Reliable Broadcast trivially follows from Agreement on BA. Termination of Reliable Broadcast trivially follows from Termination of BA. Validity of Reliable Broadcast follows from Validity of BA. Namely, if the sender is honest it sends b to all honest parties, so all honest P_i have input $b_i = b$. By Validity of BA the result r must be the input for an honest party, so $r = b_i = b$. This allows to broadcast a single bit, but a sender wants to send a longer message it can just send it bit by bit.

A disadvantage of the above protocol is that if a sender wants to send a 1000-bit message it would involve running 1000 instances of a BA, which would be very expensive. Exercise 8.8 asks you to do better.

8.8 Exercises

Exercise 8.1 (Dolev-Strong I) Consider Fig. 8.18. Parties P_2 and P_3 were corrupted. Start the execution at the beginning of round 3. What messages will the honest parties send from now on: what messages in what rounds? In what global round do they give output, and what will their outputs be? Argue for your answers. ▼

Exercise 8.2 (Dolev-Strong II) Consider the run of Dolev-Strong in Fig. 8.19. All parties are honest, but P_3 was woken up one round too early. Note that the red round numbers are now the global round numbers. The system has been running for a while and is in round 42 as P_3 is told that the protocol begins. It sets $r_3 = 0$ and thinks it is at the beginning of round 1 of the Dolev-Strong instance. Only a round later are the other parties woken up. In global round 43 they set $r_i = 0$ and think they are at the beginning of round 1 of the Dolev-Strong instance.

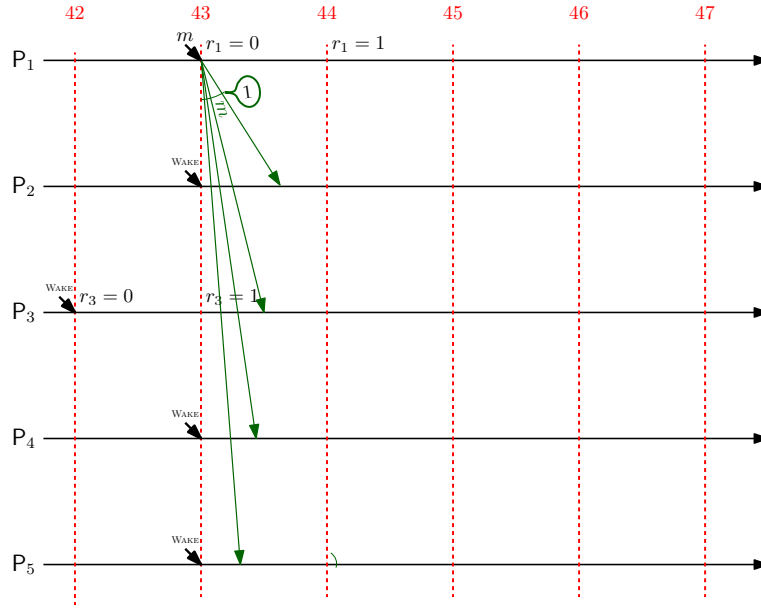


Figure 8.19: A run of Dolev-Strong. All parties are honest, but P_3 was woken up one round too early. What will happen now?

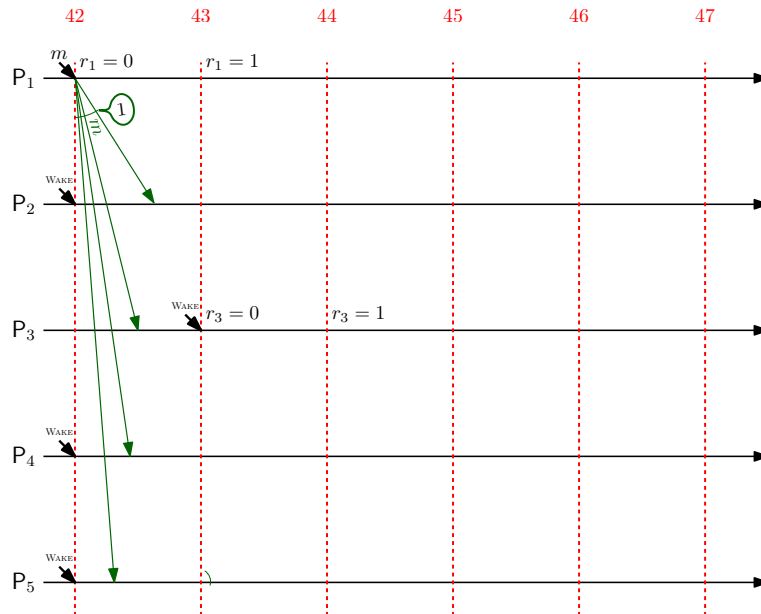


Figure 8.20: A run of Dolev-Strong. All parties are honest, but P_3 was woken up one round too late. What will happen now?

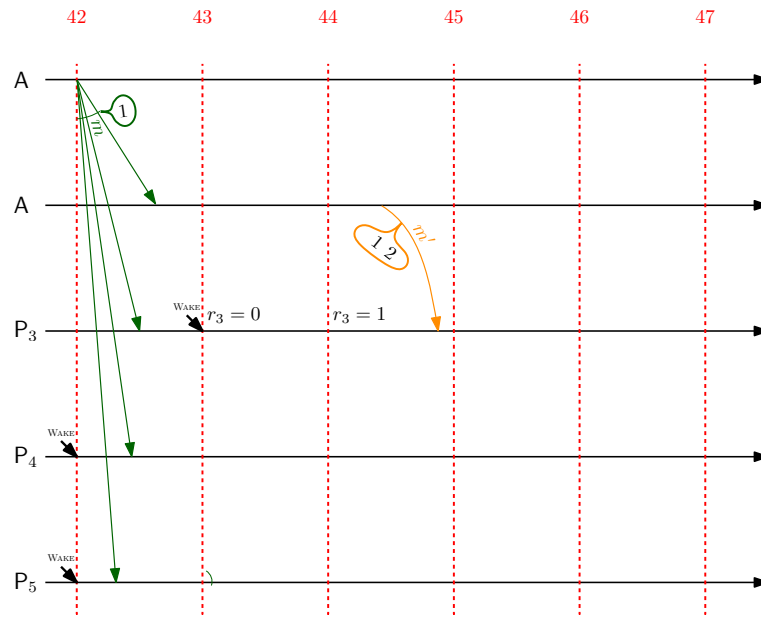


Figure 8.21: Two Byzantine corrupted parties and a late riser. What will happen?

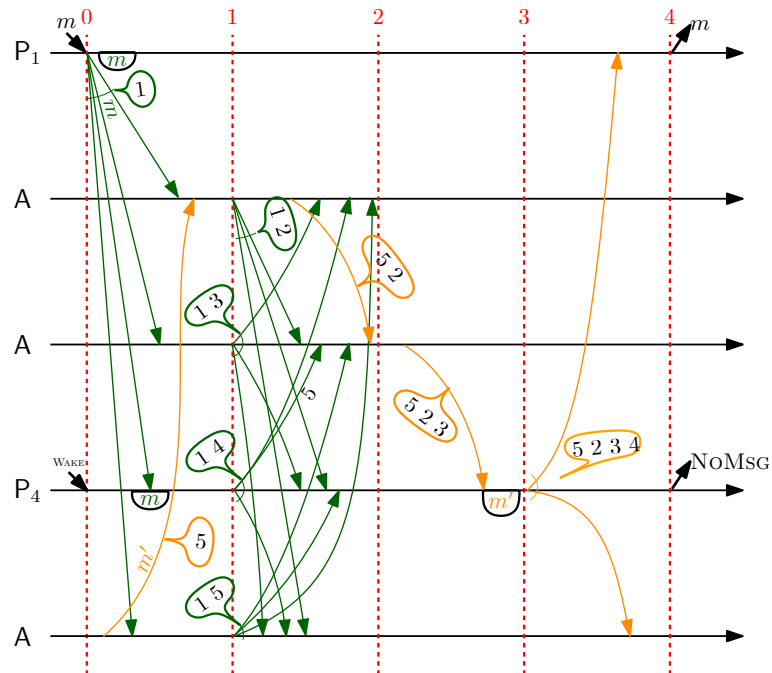


Figure 8.22: Parties P₁ and P₄ are honest and ran the software they were given. The other parties were corrupted and exploited a bug they found in the software to break both Validity and Agreement. How do you simplest explain the behavior? If you worked as a debugger for the company producing the software, what bug would you start by looking for and in the code for which activation rule?

Start the execution at the beginning of global round 44. What messages will the honest parties send from now on: what messages in what rounds? In what global round do they give output, and what will their outputs be? ▼

Exercise 8.3 (Dolev-Strong III) Consider Fig. 8.20. All parties are honest, but P_3 was woken up one round too late. Note that the red round numbers are now the global round numbers. The system has been running for a while and is in round 43 as P_3 is told that the protocol begins. It set $r_3 = 0$ and thinks it is at the beginning of round 1 of the Dolev-Strong. But one round earlier the other parties were woken up. In global round 42 they set $r_i = 0$ and think they are at the beginning of round 1 of the Dolev-Strong instance. Start the execution at the beginning of global round 43. What messages will the honest parties send from now on: what messages in what rounds? In what global round do they give output, and what will their outputs be? ▼

Exercise 8.4 (Dolev-Strong IV) Consider Fig. 8.21. The corrupted parties will not send other messages than those indicated. Start the execution at the beginning of global round 43. What messages will the honest parties send from now on: what messages in what rounds? In what global round do they give output, and what will their outputs be? ▼

Exercise 8.5 (Dolev-Strong V) Consider Fig. 8.22. The corrupted parties exploited a bug to make the two honest parties violate Validity and Agreement. How do you simplest explain the behavior? If you worked as a debugger for the company producing the software, what bug would you start by looking for and in the code for which activation rule? ▼

Exercise 8.6 (Impossibility for more parties) Show that it holds for all $t > 0$ and $n = 3t$ that it is impossible to get reliable broadcast with n parties which tolerates t Byzantine corruptions in the round-based synchronous model with authenticated channels. [Hint: Assume for the sake of contradiction that such an n -party protocol exists. Group the n parties into three groups of t parties and use them to construct a 3-party protocol for reliable broadcast tolerating 1 Byzantine corruption.] ▼

Exercise 8.7 (Message complexity of Dolev-Strong) In DolevStrong, if the sender is Byzantine corrupted it can force the honest parties to send a large number of messages by sending signed (bid, m) for a large number of different m to different parties. Consider a version of Dolev-Strong where parties only forward the first two messages which they mark as received. In a bit more detail, a message m is only relayed if $m \notin \text{Received}$ and $|\text{Received}| \leq 1$. Argue that the protocol is still secure. How many messages are sent in the worst case? ▼

Exercise 8.8 (Better Reliable Broadcast from BA) Show how to broadcast a 1000-bit message using only a single BA. Assume that you have access to a PKI and make the protocol secure against $t < n/2$ Byzantine corruptions. The protocol should be of the following general form: In round 1 the sender signs m and sends it to all parties. In round 2 all parties forward what they received so far to all other parties. In round 3 all parties forward what they received so far to all other parties. In round 4 the parties run one BA. If the result is 0 they output NOMSG. From among all signed messages they saw above they pick one, let us call it m , as their output. This protocol clearly has Termination. Your job is to decide how to compute the votes for the BA and the message m when the result is 1. Argue Validity and Agreement. [Hint: try to vote on whether contradicting messages were signed and pick m to be the unique message when there were no contradictions. Beware that the adversary might report contradictions to some parties and not others. Also beware that all party need to know this m .] ▼

Chapter 9

Key-Exchange and Key-Management (2025)

Reading Guide *This chapter introduces techniques for key management and key exchange. All of the chapter is curriculum and can be queried at the oral exam.*

Contents

9.1	Authenticated Key-Exchange (AKE)	228
9.1.1	Key Distribution Centers (KDC)	229
9.1.2	AKE from Public-Key Cryptography	230
9.1.3	A Strawman Protocol	230
9.1.4	How to not to do it – The Needham-Schroeder Protocol	230
9.2	Diffie-Hellman Key-Exchange	231
9.2.1	Cryptography Based on Discrete Logarithm	232
9.2.2	Diffie-Hellman Key Exchange	232
9.2.3	Authenticated Diffie-Hellman Key-Exchange	233
9.2.4	AKE in Practice	234
9.3	Key-Management and User Authentication	237
9.3.1	Password Security	237
9.3.2	Choosing and Guessing Passwords	237
9.3.3	Using and eavesdropping passwords	239
9.3.4	Storing and stealing passwords, the user side	239
9.3.5	Storing and stealing passwords, the verifier side	240
9.3.6	A note on password managers	241
9.3.7	Password-Based Encryption	242
9.3.8	Password-Based Key-Exchange	242
9.3.9	Hardware Security	243
9.3.10	Biometrics	245
9.4	Exercises	245

Previous chapters have discussed cryptographic mechanisms and discussed how they can solve crucial confidentiality and authentication problems. However, any cryptographic solution is only as secure as the keys that are used within the cryptographic algorithm. For this reason, the saying goes that *cryptography is a tool that turns all sorts of problems into key-management problems*.

This chapter will focus on how keys can be stored, transmitted, and exchanged between parties that want to securely communicate over the Internet. Before delving into technical solutions we state some general principles here:

Any secret runs a greater risk of being revealed, the longer time you keep it constant, and the more you use it.

This principle tells us that even strong cryptographic keys do not have eternal life. The more you use a key to e.g., encrypt data, the more material you provide an adversary to mount possible cryptanalytic attacks. One has to understand that this principle must be applied to the specific context, and in particular it does not mean that cryptographic keys cannot be re-used at all¹. To give a concrete example, there are known attacks on the CBC mode of operation that kick-in after $2^{n/2}$ blocks have been encrypted, where n is the size of the block cipher. Depending on the block size this might be a completely unrealistic attack, but it can be exploited in practice e.g., against older block ciphers with block size 64 bits.

One common way of dealing with the problem listed above is to use so called **ephemeral-keys** or **session-keys** (depending on the context). Those are keys that are used for a limited period of time, or on a limited amount of data. We will see different techniques to perform **authenticated key-exchange**, which allows two parties to establish a session key. The term “authenticated” here refers to the fact that the session-key must be linked to some long-term, well established key. This leads to the problem of managing those long-lived keys, leading to another important principle:

The security of any cryptographic system will eventually depend on some physical or otherwise non-cryptographic means, used to protect and/or validate some cryptographic key.

You have already encountered this principle in the authentication chapter, where we discussed certificate authorities. Certificate authorities serve the purpose of establishing an authentic link between the identity of a user and their public key. Once this link has been established, it is possible to perform chains of authentication using only cryptographic methods. However, as we have discussed, establishing this first link requires the CA to verify the identity of users in a non-cryptographic way (such as checking their passport). Similarly, the user also needs to keep their secret-key secret. While they could encrypt this key under some other key, eventually some non-cryptographic tool will have to be used. We will see different tools that are commonly used to protect long-term keys, such as **passwords**, **secure hardware**, **biometrics**, etc.. You are probably already familiar with these and have used them many times (at least) for the purpose of user identification, which will also be briefly discussed in the chapter.

9.1 Authenticated Key-Exchange (AKE)

We have seen how symmetric-key cryptography is an efficient and secure way to establish a confidential and authenticated channel between two parties (remember that this is best done using an **authenticated encryption** scheme that provides confidentiality and authenticity at once). Of course, to use symmetric-key cryptography, one needs a way to let the communicating parties, let's call them Alice and Bob or A, B , to agree on some random key K . At the same time we need that no-one else (in particular the evil eavesdropper E), is able to gain any information about K .

¹With the exception of unconditionally secure solutions such as one-time pad.

As discussed above, it is best to have a new key K every time A, B establish a new connection. That is, A, B want to establish a so-called **session key** K . The process of establishing a session key between two parties is usually known as **authenticated key-exchange (AKE)**.

9.1.1 Key Distribution Centers (KDC)

There is a very simple way of achieving authenticated key-exchange if there is some trusted third-party that both A, B trust. Such third-parties are called **key distribution centers (KDC)** and are used in real-world systems like Kerberos. In a nutshell, and avoiding many details a KDC works as follows:

1. We assume that each user shares a long-term key with the KDC. For instance, A and the KDC share K_A , while B and the KDC share K_B .
2. If A wants to establish a session key with B , A will send a request to the KDC (using K_A to protect and authenticate the message).
3. The KDC will generate a session key K_{AB} and send to both A, B the tuple (A, B, K_{AB}) (using K_A and K_B respectively to protect and authenticate the message).
4. B sends an **ack** message to A , encrypted and authenticated using K_{AB} .

Note that using a KDC each user only needs to store a long-term key for the KDC, independently on the number of users of the system (if every user had to store a long-term key for every other user the total number of keys in the system would grow quadratically). Also, the protocol above only relies on secret-key cryptography, which is generally faster than its public-key counterpart.

However, it should be clear that the KDC represents a single point of failure in the system. In particular:

- All users must trust the KDC completely, since the KDC can decrypt or forge all traffic if it decides to misuse its knowledge.
- If the KDC is down, the entire secure communication system becomes unavailable.

While the KDC represents a single point of failure, the solution described above actually satisfies many useful properties *under the assumption that the KDC is honest*. We can therefore use the KDC solution as a benchmark to evaluate the security of the other AKE protocols will present later.

In particular, the KDC-based AKE protocol above satisfies (*when the KDC is honest*):

Agreement Both A, B agree on the same session-key K .

Secrecy No one other than A, B learns any information about the session-key K .

Freshness Each session-key K is chosen uniformly at random and independently of any other key in the system.

Authentication If A receives an **ack** from B , then A knows that B is online and wants to talk to A . Similarly, when B receives a message from the KDC, B knows that A is online and wants to talk to B .

Note that we do not state that both parties agree on whether the protocol was successful or not. This is in general an impossible problem if we assume that the adversary is controlling the communication channel, as the last message of the protocol could be dropped by the adversary (or maybe even just get lost due to a communication error). Therefore, B might accept without A learning about it. Adding an extra *ack* from A to B would not help, as now this message might get dropped. And so on.

9.1.2 AKE from Public-Key Cryptography

We now study how public-key cryptography can enable us to achieve the same properties as the protocol described above, but *without* using a KDC.

For this to be possible, we will assume that A, B somehow know each other's public-keys pk_A, pk_B . This can be done e.g., using certificates as described in the authentication chapter. Thus, it would be incorrect to state that the solution we are about to describe does not rely on any trusted third-party. However, it should be easy to see that there is a significant difference in the trust we need to put in the KDC or the CAs.

9.1.3 A Strawman Protocol

Before moving on, consider the following, simplistic protocol for key-exchange.

1. A picks a key K , and sends to B the ciphertext $c = E_{pk_B}(K)$.
2. B decrypts c and retrieves K .

Before reading on, take a moment to think about which of the desirable properties of the KDC protocol are (not) satisfied by this protocol and why.

As it should be evident, neither A nor B have any idea whether the other party was there at all. Moreover, the key is only chosen by A , thus could be biased if A is controlled by an adversary.

9.1.4 How to not to do it – The Needham-Schroeder Protocol

While the strawman protocol above was clearly unsuitable, we will see that designing good authenticated key-exchange protocols is harder than it may seem. A good example is the following protocol, proposed in 1978 by Needham and Schroeder. We assume in the description that $E_{pk}()$ is an secure encryption scheme as discussed in Chapter 5, and that A and B already know each other's public keys:

1. A chooses nonce n_A and sends $E_{pk_B}(ID_A, n_A)$ to B .
2. B decrypts, checks ID_A , chooses nonce n_B and sends $E_{pk_A}(n_A, n_B)$ to A .
3. A decrypts, checks that the correct value of n_A appears in the result, and sends $E_{pk_B}(n_B)$ to B .
4. B decrypts and checks that the correct value of n_B appears in the result.
5. If the checks executed are ok, each party computes the session key as some fixed function of n_A, n_B .

Recall that *nonces* are values chosen randomly from a large domain, so that we may assume they will repeat with negligible probability.

It may seem very plausible that if both A and B end up accepting the values they see, we have essentially what we wanted above, namely A, B agree on the values of n_A, n_B , and no one else knows these values. After all, only B can decrypt values encrypted with pk_B , and similarly for A . So it should be secure to generate the session key from the nonces.

Unfortunately, this turns out to be false! It is possible for a third user E with a certified public key to mix two instances of the protocol, and fool B . This goes as follows:

1. Suppose A starts a session with E , thus A will send $E_{pk_E}(ID_A, n_A)$ to E .
2. E decrypts this and starts up a session with B , pretending to be A . So E will send $E_{pk_B}(ID_A, n_A)$ to B .
3. B decrypts this, finds the right ID in the result, and sends $E_{pk_A}(n_A, n_B)$ to E .
4. E is of course not able to decrypt this, but can instead simply forward the message $E_{pk_A}(n_A, n_B)$ to A .
5. A will decrypt this, find a result that has exactly the form he expected, and will therefore return $E_{pk_E}(n_B)$ to E .
6. E can decrypt, find n_B , and is now able to send $E_{pk_B}(n_B)$ to B .
7. When B decrypts, he will accept since he finds the right value of n_B in the result.

As a result, if we went on to use this as a basis for creating a session key, B would believe he was communicating securely with A , but is in fact talking to E , and this of course completely defeats the purpose of the protocol.

However, the intuition that suggests that the protocol should be secure is not completely unreasonable: If only one instance of the protocol is running at any given time, no attacks are known. Indeed, the attack we saw is crucially based on the idea that several instances of the protocol are running at the same time, and the adversary is able to “mix” these instances together in an unfortunate way.

Finally note that in the attack above the user A actually initiates a protocol with an evil party E . One might ask, why would A do that? However, think of how many servers you connect to, daily and willingly (e.g., a website you visit) or unwillingly (e.g., a website serving ads as part of a website you actually wanted to visit). It is unrealistic to assume that everyone you connect to is honest and incorruptible, and therefore any realistic threat model for authenticated key exchange must include the scenario in which a user connects to a maliciously controlled, or Byzantine, server.

9.2 Diffie-Hellman Key-Exchange

The Diffie-Hellman key-exchange protocol represents the birth of modern cryptography. While RSA was the first publicly discovered public-key encryption scheme, the *idea* of public-key encryption was presented in a paper by Diffie and Hellman a few years earlier than the discovery of RSA. The Diffie-Hellman paper also introduced the key-exchange protocol that we will present below. (Interestingly, this key-exchange protocol can very easily be turned into a public-key encryption scheme, but it took a few years for this to be noticed, and the resulting public-key encryption scheme is known as the ElGamal encryption scheme).

9.2.1 Cryptography Based on Discrete Logarithm

The discrete-logarithm problem (DL) is a hard mathematical problem that can be used to construct public-key cryptosystems (encryption and signatures) and can be used as an alternative to RSA. As mentioned in previous chapters, the main advantage of DL-based cryptography is that, when DL is implemented over elliptic curves, it is possible to use very short keys compared to RSA (in practice, 256 bits instead of 2048 – 4096 bits). As you probably do not have the mathematical background necessary to work with elliptic curves, we will here present the DL problem when instantiated with modular arithmetic, which uses key-size equivalent to RSA. This is almost never used in practice due to its inefficiency, but it is much easier to present.

As the name suggests, the DL problem concerns computing logarithms i.e., solving equations of the form $g^x = h$ for known g, h . Computing (regular) logarithms is an easy computational problem. Instead, the DL problem asks to solve equations of the form:

$$g^x \bmod p = h$$

With p being a large prime (e.g., 2048 – 4096 bits). Note that there are significant differences between the DL and RSA: while they both work with modular exponentiations, the modulo is a large prime number in DL (while it was a composite, the product of two large prime numbers in RSA). Also, in DL the base g is public and x is the “secret” (while in RSA the encryption exponent was public and the base contained the secret). Finally, the RSA function could be inverted having knowledge of d , while there is no secret value that allows to easily “decrypt” the DL one-way function.

The reason why it is possible to use the DL one-way function to build key-exchange protocols (as well as public-key encryption, digital signatures, etc.), is the algebraic nature of the one-way function. In particular, thanks to the properties of exponentiation we have that:

$$(g^x \bmod p)^y \bmod p = g^{xy} \bmod p$$

9.2.2 Diffie-Hellman Key Exchange

When using the Diffie-Hellman key-exchange, A, B need to use the same parameters g (which is usually called the **generator**) and the prime modulus p . Then, the DH-KE protocol between two parties A, B works as follows:

1. A chooses a random large enough number² x , computes $X = g^x \bmod p$ and sends X to B .
2. Likewise, B chooses a random large enough number y , computes $Y = g^y \bmod p$ and sends Y to A .
3. Now, both A, B can compute the same key $K = g^{xy} \bmod p$ respectively as:

$$K = X^y \bmod p \text{ and } K = Y^x \bmod p$$

We proceed in evaluating the properties of this protocol, by benchmarking it against the KDC-based solution:

Agreement It is easy to verify that both parties end up with the same key since:

$$X^y \bmod p = (g^x \bmod p)^y \bmod p = g^{xy} \bmod p = (g^y \bmod p)^x \bmod p = Y^x \bmod p$$

²For concreteness, you can think about the range $0, \dots, p-1$, but the exact details are a bit more complicated.

Secrecy Consider the setting where A, B are both honest and an adversary E eavesdrop the values exchanged by A, B , namely X, Y . Is it now possible for E to compute K ? It should be clear that if the adversary can break the DL problem, then the adversary can break the DH-KE protocol by e.g., recovering x from X and then computing $Y^x \bmod p$. However, the converse is not known to hold in general. That is, the security of the DH-KE protocol relies on the so-called **Diffie-Hellman problem**, which states exactly that no adversary with access to (g, p, X, Y) but not x, y can efficiently compute $g^{xy} \bmod p$. (There is also a different and stronger variant, the **decisional Diffie-Hellman problem**, which states that given (g, p, X, Y) the adversary has no information at all about $g^{xy} \bmod p$.)

As discussed above the hardness of the DL and DH problems is comparable to the hardness of RSA. One peculiar property of DL is that both A, B need to use the same system parameters g, p for the protocol to succeed (as opposed to RSA where each user has their own modulus n). How do they agree on these values, and what impact does this have for security? There are at least two school of thoughts on this. The first says that g, p should be the same for every user in the system, hardwired in the software. This has the advantage that agreeing on the values becomes trivial, and that one can optimise the implementation for the hardwired parameters. In this case, the parameters could have been chosen as part of a standardisation process. This has the further advantage that the parameters are (hopefully) chosen in a way to withstand common attacks or pitfalls against DL-based system. On the other hand, one could worry that those behind the standardisation process “planted” a backdoor in the system. Or more in general, that the fact that everyone uses the same parameters would make the job of an adversary easier. This is actually the case for the **index-calculus algorithm**, the best known attack against the DL problem (with modular arithmetic), where it is easier to attack many instances of the DL problem for the same g, p rather than to break many independent instances of DL. On the other hand, if everyone chooses their parameters then we must have that either A or B chooses the parameters, and that the other party must run some sort of sanity-check to ensure the parameters to do lead to a weak instance of the DL problem (you will be asked to think about weak DL parameters in the exercises). This is necessary as one of the users might be malicious, or simply just incompetent in their choice of the parameters.

Freshness Assuming the protocol is used with “safe” (g, p) , then the protocol ensures freshness since both parties contribute with their own random values x, y . In fact, it is enough for one of the two parties to contribute proper randomness for the final value to be random-looking.

Authentication As it should be clear from the description of the protocol, the parties have no idea who they are talking to. This will be addressed next.

9.2.3 Authenticated Diffie-Hellman Key-Exchange

To ensure that the DH-KE protocol satisfies authentication, we will need to use both signatures and MACs as described in the authentication chapter. Intuitively, signatures will be used to link the execution of the protocol with some known public-key for the user we want to communicate with, while MACs will be used to ensure that the other party indeed has access to the ephemeral Diffie-Hellman secret and it is not just a dishonest user replying some values they collected somewhere else.

Once again, we will assume that A, B know each other’s public-key pk_A, pk_B (for instance, they have already exchanged certificates, or they can do so during the AKE protocol). Then,

after performing the steps of the DH-KE protocol described above they will perform a round of authentication where each party signs and MACs the transcripts of the protocol $\text{tx} = (pk_A, pk_B, X, Y)$ using their long-term secret-key sk_A, sk_B , and the derived key K respectively. To be more precise, we write tx_A to represent the transcripts observed by A , and K_A to be the key derived by A , and tx_B, K_B for the respective values on B 's side. (In an honest execution of the protocol $(\text{tx}_A, K_A) = (\text{tx}_B, K_B)$ but if an adversary is tampering with the communication this might not be the case.)

1. A sends (t_A, s_A) to B where $s_A = S_{sk_A}(\text{tx}_A)$ and $t_A = \text{MAC}_{K_A}(\text{"fromAtoB"}, \text{tx}_A)$.
2. B sends (t_B, s_B) to A where $s_B = S_{sk_B}(\text{tx}_B)$ and $t_B = \text{MAC}_{K_B}(\text{"fromBtoA"}, \text{tx}_B)$.
3. Upon receiving (t_A, s_A) , B checks that $\text{Ver}_{pk_A}(tx_B, s_A) = 1$ and $\text{Ver}_{K_B}(\text{"fromBtoA"}, tx_B, t_A) = 1$ and aborts otherwise.
4. Upon receiving (t_B, s_B) , A checks that $\text{Ver}_{pk_B}(tx_A, s_B) = 1$ and $\text{Ver}_{K_A}(\text{"fromAtoB"}, tx_A, t_B) = 1$ and aborts otherwise.

The protocol now enjoys authenticity in a strong sense. From A 's point of view (since the protocol is symmetric the same would hold for B), receiving a signature on the transcript of the messages under B 's secret key is a proof that B is online and actively participating in the protocol, since the value X computed by A is a large enough *challenge* (reminiscent of the countermeasures against replay attacks we saw in the authenticity chapter). Thus s_B was computed in the context of this protocol execution, and is not being replayed from an older transcript. Moreover, since the transcripts contain A 's public key we know that B indeed believes he is talking to A , and not someone else (remember that in the Needham-Schroeder protocol the adversary was exploiting the lack of "recipient authentication" to mix-and-match messages from different executions). Finally, when A receives a valid MAC t_B under the key K_A , then she has a proof that B also knows the same key K_B . Due to the secrecy property of the DH-KE protocol, the only other party in the world that can compute K_A is the one that knows the secret exponent y . This means that B indeed knows y and is not just using an old Y from some other protocol instance. Finally, note that when computing the two MACs t_A, t_B , Alice and Bob prepend the strings *"fromAtoB"* and *"fromBtoA"* respectively. This is to make sure that the MACs encode information on what direction they should be used in. Without this extra information we would have that $t_A = t_B$ (since $k_A = k_B$ and $tx_A = tx_B$), and therefore there would be a trivial attack in which a corrupt B can simply send $t_B := t_A$ back to Alice.

The technique where A, B finish the protocol by confirming they have observed exactly the same protocol transcript is sometimes known as *final authentication of views* and is used in many AKE protocols. This essentially forces the attacker to only look at messages sent by the peers and forward these truthfully and prevents man-in-the-middle attacks of the type that broke the Needham-Schroeder protocol

You are welcome to think whether a simpler protocol would also satisfy the same properties, or whether removing any of the steps would decrease security. In fact, you will be asked to do in the exercises.

9.2.4 AKE in Practice

We briefly review some of the many uses of the Diffie-Hellman AKE in practice.

TLS The TLS protocol (Transport Layer Security) is arguably the most widely deployed and used cryptographic protocol in existence. It has a long history dating back to the mid-90s where it started under the name SSL (Secure Socket Layer). The protocol was designed at Netscape, a major browser at the time. The protocol had a rough start, with the first version of SSL being broken even before being published. To make a long story short, SSL evolved by being repeatedly broken and fixed. Its name eventually evolved into TLS, as a wider community beyond the original designers took ownership of the protocol. Despite this, the protocol kept being broken and fixed, until it was decided that TLS should be completely redesigned under the *provable security* approach. This major update was released under the anti-climatic name of TLS 1.3, which is the current version of the protocol. TLS is a suite of protocols including the *handshake* protocol that implements a version of the Diffie-Hellman Authenticated Key-Exchange described above. The TLS suite however also contains a *record* protocol (specifying how the data should be encrypted after the key is established) as well as many other “housekeeping” protocols (such as *heart-beat*, *alert*, etc.).

The main differences between the DH-AKE described in this book and the current version of TLS can be summarized as follows: in addition to the ephemeral DH keys X, Y , in TLS each party also picks a random nonce that is included in the key-derivation process (this helps to ensure enough randomness in cases where the parties – perhaps due to limited computing power – decide to re-use their values of X, Y across different handshakes). Also, the output of TLS is not a single key, rather a set of keys to be used for different purposes: This includes different keys to be used for encryption and authentication *during* the hand-shake and *after* the handshake (this e.g., helps proving security of the protocol in a modular way) and also different keys to be used in the different directions of communication i.e., different keys are used for sending data from A to B and from B to A (this helps e.g., for preventing replay attacks). Another difference is that the mutual authentication steps of the protocol are encrypted under the generated DH key (this helps hiding the identity of the participating parties which might otherwise be revealed by the signatures). Finally, while it is possible to use TLS when both parties have a public-key, the most commonly deployed version of TLS allows users without a public-key to establish an authenticated connection to a server owning a certificate. This is the version you commonly use when connecting to any website, and is known as TLS with *one-sided authentication*. (As the signature from the client to the server is removed, the server needs other ways to authenticate the users. This is typically done at the application level using a username and password, as you most likely have experienced many times in your life).

HTTPS The HTTPS protocol (Hypertext Transfer Protocol Secure) or *secure HTTP* is simply the extension of HTTP that runs on top of TLS and is mainly used to browse the internet securely. You have most likely noticed that your browser lists `https://www...` in front of the URL you are visiting (and perhaps display a lock) when an encrypted connection is established.

IPSec The TLS protocol lives between the transport layer (TCP) and the application layer. This means that TLS enables to establish secure connections *between applications*. The advantage of this is that the data is encrypted by the *application* running on your machine, and is decrypted by the *application* on the machine you are communicating to. This implies that applications running on the same machines (or other users sharing the same machines) do not ever see the plaintext. However, this also means that only applications which were developed to use TLS can use it, and also that communication at lower layers of the communication stack is not encrypted. Making a long story as short as possible, IPSec is a set of protocols that do a job very similar to what TLS can do, but on a different layer in the standard protocol stack, namely between the IP layer and the transport layer. IPSec can create a secure connection

(also known as a *security association*) between two IP addresses. The fact that the protocol is on a lower layer means, for instance, that the sequence numbers and other control information that is used to handle a TCP connection are encrypted while in transit (this would not be the case for TLS). On the other hand, data from different applications will go through the same “encrypted tunnel”, so IPSec gives in itself no separation between different applications that run on the same machine. This means that the “secure tunnel” goes between such IP nodes and as soon as the network adapter delivers data to the machine it sits in, the data is no longer protected. On the other hand, all applications on the machine can use the protected connection without being aware of the encryption. Most instances of IPSec use the Diffie-Hellman protocol for key-exchange.

VPN Virtual Private networks (VPN) solutions are designed to set up secure connections from a local area network to a remote PC. During the set-up, which happens between the PC and a VPN gateway, the remote PC is assigned an IP-address as if it was a part of the local network. After setting up the connection all traffic between the PC and the VPN gateway is encrypted and checked for authenticity. This requires, of course, that an authenticated key-exchange is done during the set-up, using e.g., the Diffie-Hellman protocol. Most VPN products are built on top of the IPSec protocol.

Your PC can now work as if it was physically a part of the local network. The idea is that a company can for instance allow employees to work from home, while still having access to confidential company data as if they were at work. VPNs can be also used to prevent location-based censorship.

Forward Secrecy Forward secrecy is a desirable property that an AKE should satisfy, and has gotten into the spotlight especially after the Snowden revelations. In a nutshell, forward secrecy states that an adversary learning a long-term key should not be immediately able to decrypt all previous conversations. If you go back to the Needham-Schroeder protocol (or even the simple strawman protocol) you will notice that an adversary learning a long-term secret key, for instance sk_B , can immediately decrypt the protocol transcript and learn all the material that is used to generate the session key. Thus, such an adversary can immediately learn the entire content of all previous sessions.

This is a direct consequence of long-term secrets being used for *confidentiality* purposes, and can be exploited e.g., by a powerful adversary (such as a government agency). If you go back to the description of the Diffie-Hellman AKE, you will notice that the long-term secret keys are only used for authenticity, and the confidentiality of each session is protected by the ephemeral Diffie-Hellman secrets x, y . This means that if an adversary manages to learn your secret-key by stealing it or breaking it in some other way, they would still have a hard time decrypting older conversations. (But they can of course impersonate you from now on).

Older version of TLS used RSA to transmit a session-key from one party to the other and were therefore not forward secure. The RSA-based handshake protocol has therefore been deprecated since TLS 1.3.

Signal Protocol The Signal Protocol is the most commonly used protocol in end-to-end encrypted chat apps such as Signal, Whatsapp, Facebook Messenger, Skype, etc. The protocol is open-source and is generally considered to have a very solid design. It uses a variant of Diffie-Hellman known as *double ratchet*, which in a nutshell allows to evolve the keys to ensure that old conversations will be kept private even if a (derived) session key is leaked in the future. It also has other interesting features, like the possibility of performing a key-exchange with a user which is not currently online, as this often the expected use case for messaging apps.

9.3 Key-Management and User Authentication

As we discussed in the introduction of the chapter, systems for key management often tend to be hierarchic: one key is protected from modification or leakage by another key, which is again protected by a third key, etc. Sometimes the structure is not a simple tree structure, and one may have keys that are protected by different keys at different locations. But any such structure must of course have an end: one or more keys that are not protected by other keys, and so can only be protected by non-cryptographic means.

Therefore, a real-life analysis of any such system cannot be limited to cryptography alone. This is the main motivation for this section that will cover (mostly) passwords, and (to a limited extent) secure hardware and biometrics. More broadly these three categories represent something users *know* (a password), something they *have* (some hardware) or something they *are* (biometrics).

Those are tools that you are most likely already very familiar with, since they are commonly used for user authentication for accessing all sorts of resources (your laptop, your phone, etc.) and not only cryptographic keys.

9.3.1 Password Security

Passwords or PIN codes are an essential part of security in many systems. Unfortunately, they are often the weakest link in the chain because they have to be remembered by humans. This often has the effect that people either choose passwords that are easy to guess, or store them in ways that degrades security.

In the following we will look at the general case where a user (knowing a password) needs to convince a verifier (knowing the same password or more commonly a function of the password) of their identity by proving knowledge of the password. Since knowledge of the password can let anyone claim the user's identity, we need to make sure that the way the password is managed does not let anyone else learn the secret. There are (at least) 4 important security aspects of password security, and corresponding to these, there are 4 types of attacks we must protect against:

- How is the password chosen – Can the adversary guess the password and verify their guess?
- How is the password transmitted between the password user and verifier – Can the adversary get hold of the password while it is in transit?
- How is the password stored by the password user – Can the adversary steal the password from the user?
- How is the password stored by the password verifier – Can the adversary steal the password from the verifier?

9.3.2 Choosing and Guessing Passwords

Choosing a Password. It is clear that from a security point of view we want passwords that are hard to guess for an adversary. We will return later on how often and how many times an adversary might be able to try to guess your password, but regardless we can already state that we need the password to be chosen *as random as possible from a large enough set*. The set from which a password can be chosen is a simple function of its length and the number of possible choices for each character. For instance, credit cards in Denmark generally use 4-digit numerical PINs leading to $10000 \approx 2^{13}$ different PIN codes. In comparison, there are $62^8 \approx 2^{47}$ possible

passwords that include 8 characters including uppercase, lowercase, and numerical digits. In general, if the password is chosen from a character set of size C and has length ℓ , then there are C^ℓ possible passwords. This number grows much faster as a function of ℓ than as a function of C , so if one wants a large number of possibilities, then increasing ℓ is typically better.

However, we must also use passwords that can be remembered and entered correctly. Experimental studies show that 12 digits seem to be the maximum one can expect anyone to enter correctly under stressful circumstances.

Passphrases. At the same time, the number of possible passwords is not a measure of quality by itself. We also have to worry about how they are chosen. Passwords that common users can remember are usually not uniformly distributed and are therefore often much easier to guess than what the password length might lead you to believe.

One idea that many have advocated for is to have people choose, not a password, but a *passphrase* which is generally easier to remember for a human, and can be harder to crack. For instance, the Electronic Frontier Foundation (EFF) suggests a method to generate a passphrase rolling dice as follows: you roll 5 dice and find a corresponding word in their list. For instance, if your dice roll is 54321 the corresponding word is “shuffle”. If you repeat the process a few times you might end up with a passphrase such as

shuffle moustache sequel coming consonant overbook

Such a passphrase is chosen uniformly at random from a set of size $(6^5)^6 \approx 2^{77}$, which is likely to be enough for most applications, but it is arguably easier to remember (and type!) than an equivalently strong password made of random characters and symbols such as Q[3!AaB.t6-X.

Password Policies. Most systems nowadays are designed knowing that users are bad at choosing passwords and therefore implement countermeasures, and try to “nudge” users into choosing good passwords, by enforcing certain password policies at the time of registration. Unfortunately, sometimes even well intended policies might lead to unexpected consequences. Consider for instance a website that forces users to choose a password containing both upper- and lowercase characters, numbers and special symbols. Perhaps the user ends up picking **Password1!** as their password. Is this a better or worse choice than the (fully lowercase) passphrase from above? Another point of contention is how often users should be forced to change their password. Clearly changing password fits the same idea from the beginning of the chapter, where ephemeral session keys were used instead of long-term ones. But if you force users to change passwords too often, they might end up choosing passwords of the form **password2023**, **password2024**, etc. making it very easy for an adversary that steals one of the password to predict future or past ones. In general, it still is a good idea to help the users choose good passwords (for instance, to avoid recently used ones or passwords that have been leaked online), but one should also remember to implement reasonable policies that do not push users into bad security practices.

Guessing a Password. A separate question in this context is: how easy is it for the adversary to verify their guess? There are two main quantity to consider, one is *how many* guesses can the adversary make, and the other is *how fast* can the adversary verify their guesses.

Thus, there is a huge difference between **offline attacks** and **online attacks**. Here by offline we mean that the adversary can steal some information about the password that allows them to verify their guesses in an environment they control. Typically this lets the adversary try as many guesses as they like, and the only thing we can do is to limit their speed (we return to this later). Instead, when talking about online attacks, we mean the setting where the adversary has

to interact with a system in our control to verify their guesses, for instance by trying to log-in. Clearly, it is much easier to detect and prevent an adversary guessing passwords in an online fashion, since we can implement countermeasures in the system under attack. For instance, one could decide that an account should be frozen after 3 failed login attempts, as limiting the number of login attempts may help security significantly. But one has to take care: if the attacker just wants to break into *some* account, they can just try once for every account, so in this case security depends on the total number of users, as well as the number of passwords and the max number of failed logins allowed. Another issue is that the attacker's goal might not be to get hold of a password, but to make the availability break down, which they can certainly succeeding in doing if they make a large number of accounts freeze by just trying incorrect passwords. So whether limiting the number of failed login attempts is a good or a bad security mechanism depends on the assumed threat model. A better idea can be to have the system wait for some time after a failed login attempt before it allows the user to try again. And to make this time longer, the more failed attempts are made. For honest users, this will be a minor problem, but an attacker will be slowed down significantly.

9.3.3 Using and eavesdropping passwords

Stealing the password when it is used can take many forms. Watching over someone's shoulder while the PIN code is entered is a trivial but often very effective attack. A fancier variant is the scam where an ATM (Automatic Teller Machine) is equipped with a false front and a video camera, that records both the magnetic stripe of your credit card and your PIN code, thus enabling the crook to copy your card and start emptying your account minutes later. Finally, passwords sent in clear over a local area network are very easy indeed to detect and grab, and this has been done in practice on many occasions (for instance, attackers sitting at the local cafe intercepting other customers using the free wifi to login onto web services which do not make use of encrypted communication). Note also that looking over someone's shoulder can also be done electronically using so called spyware, where you get code to run on the victim's machine in a unnoticed way. The program will record passwords and send them to the attacker. Another variant of the same attack (which lead to many cases of identity theft in Denmark a few years back) uses a keylogger, a small hardware device that stores all keystrokes plugged between the keyboard and the computer. A criminal can install such device at some public PC (e.g., at the library) and then collect it every few weeks to learn the passwords of those who used that PC in that period of time.

General precautions are hard to define because the threat models are so diverse, but in general passwords should be always sent through encrypted channels. Moreover, passwords can be made stronger when used in connection to other methods of authentication from the upcoming sections. This is known as two- or multi-factor authentication (2FA, MFA).

9.3.4 Storing and stealing passwords, the user side

Stealing the password from the user's permanent storage can be easy if it is written down (either on a post-it on the computer monitor, or stored in plaintext on a hard-disk so that the user does not have to type it every time) but other methods can also be effective, such as trying to fool users into revealing their passwords under false pretenses. This is known as *social engineering*. In a study at University of Sydney, 336 students were sent an email asking them to send their password to a certain mail address. The mail claimed that this was necessary to validate the password database after a suspected attack. 138 of them returned a valid password. Many were suspicious and changed their passwords, but essentially no one reported the incident to the system administrator. A variant of this idea is the *phishing attack*, where you send a mail

to the victim, claiming you are their bank, for instance. The mails asks the user to follow a link, leading to a fake web page that claims to be their bank. Here the user is asked to type their user name and password, to “validate the account”, for instance.

According to many people, including several hackers, social engineering is one of the most effective attacks on real systems, and systematically preventing it is very difficult. The problem seems to be that we are up against some basic human psychologic factors: it seems to be hard for us to assign value to passwords, perhaps because they are not a tangible objects we can hold in our hands. And therefore fooling people into giving them away seems much easier than, for instance, getting someone to give you the key to their house.

It seems the only mechanism that has some effect is to inform and train users, or to use multi-factor authentication.

9.3.5 Storing and stealing passwords, the verifier side

Stealing passwords from the verifying party may also be possible. In some really bad cases systems store passwords in cleartext. UNIX (and several other systems) is somewhat better, in that the password file does not contain the passwords themselves, but instead a record (u, h) containing username u and $h = H(pw)$ where pw is the password of user u and H is a hash-function (as described in Chapter 6). Then, when the user tries to login they present their username and password (u, pw') , the verifier retrieves the h corresponding to u from their storage, and verifies whether $h = H(pw')$.

Remember that cryptographic hash-functions are also one-way functions, meaning there is no easy way to invert the hash and recover the password. Thus the idea is that even if the adversary learns the hash value h they will still have a hard time finding the password pw . Therefore, sometimes the process of hashing passwords like this is referred to as **one-way encryption**.

This is somehow confusing, since encryption (as described in the Confidentiality chapter) typically uses (secret) keys and is randomised, whereas hashing is keyless and deterministic. This means that if an adversary sees a hash $h = H(pw)$ and wants to check if pw' is the right password, they can just compute $H(pw')$ and see if they get the same result h . They can then try the next password, and so on. This process is known as **password cracking**. This is a very relevant attack practice since in many systems, like UNIX, the password file is not too difficult to steal. (Over the last decades billions of password hashes have been leaked this way as a consequences of several breaches of security at large organisations).

Password crackers are programs that use dictionaries with probable passwords and try to match them with entries in a given password file. Modern password crackers make educated guesses at what a password might be using common words, known password databases, as well as implementing common substitutions that users are likely to employ when generating passwords (for instance using `pa55w0rd` instead of `password`). Practical studies indicate that up to 25% of the passwords used in real life can be cracked this way, when people are not guided in any way or warned against bad choices of passwords.

There are three main countermeasures against these kind of attacks:

- *Choosing better passwords:* as discussed this might include encouraging (or enforcing) that users do not choose weak passwords as well as using password managers (see below).
- *Slow the attacker down:* similarly to what was suggested for online login attempts, it is possible to make the life of the adversary harder by making sure that the function H is “slow enough” to slow down the number of attempts a password cracker can run per second, while at the same time “fast enough” for the system to be efficient enough. Some

hash functions (such as e.g., bcrypt or PBKDF2), have been designed specifically for this purpose, and can get as input a parameter specifying how “slow” the function should be. Another very effective countermeasure is salting, which involves using some random number (called salt) when computing the hash of the function. (See more about salting in Exercise ?? at the end of this chapter).

- *Removing the single point of failure:* an increasingly common architecture for storing passwords used e.g., by Facebook, is the following: passwords are hashed together with a secret key K which is stored on a different machine than the one storing the password hash. Therefore, the adversary must now get hold of both the hashed password and the key K to be able to verify their guesses. This of course also require the second server to be involved in every password verification attempt, but this seems to be a reasonable price to pay for the added security. More in detail, when the user u register their password pw , this is sent to the “hashing server” that computes $h = H(K, pw)$ and returns this value to the “authentication server” that stores the pair (u, h) . When the user later contacts the authentication server, the authentication server sends the received password pw' and the hashed password h to the “hashing server” who checks whether $H(K, pw') = h$ or not.

It is noteworthy that in some contexts passwords are being dropped entirely. The latest iteration of the Danish ID system MitID employees a passwordless approach. However, after researchers have pointed out that an attacker could easily lock users out of the system (the standardly assigned usernames were very easy to guess), the MitID designers encouraged users to choose hard to guess usernames, so that current usernames lie somewhere between standard usernames and passwords.

Another modern passwordless approach are so-called **passkeys**. A passkey is nothing else but a public/secret-key pair that can be used for authentication: at registration time the user generates a key pair, sends the public-key to the server and stores the secret-key securely (e.g., in a secure enclave or in the operating system key-chain). At login time, the server sends a random challenge and waits for a signature on the challenge. This is very similar to one of the countermeasures against replay attack we saw in the authentication chapter. Note that since passkeys require a way of securely storing the secret-key, they do not really fall under the umbrella of *something the user knows* and cannot be used as drop-in replacements for all applications of passwords.

9.3.6 A note on password managers

An increasingly popular way of dealing with creating and managing passwords are so called password managers. In a nutshell, a password manager is a piece of software (or in rare cases hardware) that helps users generate “truly random” passwords and helps users by storing their passwords for them. Password managers solve the problem of users choosing easy to guess passwords, and users re-using the same passwords over multiple websites. Thus, they can greatly help against attackers who can get access to the hashes of the passwords stored at the verifier side. However, using password managers introduces new security issues: where and how are the password stored? Early *in browser* password managers would store the password in plaintext at the client side, making it very easy for an attacker that can hack into the user’s device to recover the user passwords. More modern solutions store the passwords encrypted under a master password (using a method very similar to what is described in Section 9.3.7). Often, password managers allow users to access their password on multiple devices (computers, phones, etc.), and therefore store the encrypted passwords (sometimes referred to as one’s *vault*) on some remote server or *in the cloud*. But, as someone famously said “there is no cloud, it’s just someone else’s computer”. Therefore, if an attacker can now hack into the password storage

server, the attacker can try to brute force the master password. As very often in security, it is not possible to make absolute statements such as “A is more secure than B”. Instead, one has to look at a specific threat model, and then say “In thread model T_1 , A is more secure than B” (which means that in thread model T_2 , B might be more secure than A).

9.3.7 Password-Based Encryption

After having discussed passwords in general and as a mean of user authentication, we return to the original motivation to introduce passwords in this chapter, namely as a mean to protect cryptographic keys.

Suppose you need to protect an RSA secret key sk using a password pw . Very roughly, the solution is to first hash the password $h = H(pw)$ and then use h as a cryptographic key for a symmetric block cipher (e.g., AES in a suitable mode of operation) and finally encrypt as $c = E_h(sk)$. Remember that the output length of hash functions is typically larger than the length of keys for symmetric crypto, so in practice one can just truncate h to the appropriate length.

Note that this works also for all sort of other password-based encryption: for instance, a password manager can encrypt site passwords under the master password, or an encrypted file storage system can encrypt arbitrary files using a password.

Now, it should be clear that the security of password-based encryption will depend on the security of the password, and it will be much lower than the security of standard symmetric encryption. However, one way to make password-based encryption more secure is to use (again!) the approach of purposefully slowing down the time it takes to verify a guess. That is, instead of a standard hash function (such as SHA) to hash the password, we use some special purpose password-hashing algorithm such as bcrypt or PBKDF2. Assume for concreteness that it takes one millisecond to evaluate SHA and one second to evaluate bcrypt. Then, this will lead to a small slow-down for the user (it takes one second, instead of one millisecond, to decrypt the data), while it makes brute force attack going from somehow feasible to unfeasible (say it takes 80 years, instead of a month, to brute force the password).

How do such slow functions work? In a nutshell, and omitting many details, you can imagine slow hashing to just iteratively perform some faster hashing i.e., we build the slow hash function H' as

$$H'(pw) = H(H(\dots H(pw)))$$

Such functions, designed to be slow but not too slow, are sometimes called *moderately slow functions*.

9.3.8 Password-Based Key-Exchange

We conclude the discussion on passwords with a connection with the key-exchange section from the beginning of the chapter. As we have seen, authenticated key-exchange is often used in contexts where users do not have a public-key and must therefore anyway authenticate themselves using a password afterwards. Therefore researchers have suggested to instead integrate the password check within the AKE protocol. The original Diffie-Hellman protocol is changed as follows:

1. A chooses a random large enough number x , computes $X = g^x \bmod p$ and sends $X' = E_{H(pw)}(X)$ to B .
2. Likewise, B chooses a random large enough number y , computes $Y = g^y \bmod p$ and sends $Y' = E_{H(pw)}(Y)$ to A .

3. Now, both A, B can compute the same key $K = g^{xy} \bmod p$ respectively as:

$$K = D_{H(pw)}(X')^y \bmod p \text{ and } K = D_{H(pw)}(Y')^x \bmod p$$

Essentially, the only difference is that the parties exchange g^x and g^y encrypted under the password, instead that doing so in plaintext. The protocol clearly still works if both parties know the same password. To argue for security we assume that decrypting X' or Y' under the wrong password will simply lead to a uniformly random value. This is a reasonable assumption for most cryptosystems. Note that now the adversary has no way of brute-forcing the password from X' and Y' , since the adversary has no way to tell whether they are using the wrong password (in which case they get a random value) or the right password (in which case they get g^x , respectively g^y , which also looks like a random value!). Thus, password-authenticated key-exchange is free from *offline attacks* where the adversary steals the transcripts of the protocol and tries to crack the password offline. The adversary can of course still perform *online attacks* against the system, but as we discussed this is much easier to protect against.

9.3.9 Hardware Security

Why use secure hardware? Hardware units with some degree of physical security are very useful in many secure systems. The more obvious purpose is of course to prevent the adversary from getting hold of our secret keys. But it is very important to understand that this is not the only issue – it is also important to make sure that a certain secret parameter only exists in “one copy”.

For instance, when you encrypt your key under a password the adversary might be able to copy the ciphertext and then attempt to brute-force it offline.

Another example are credit cards with magnetic stripes, that are very easy to copy without the user noticing it. Modern credit cards instead keep the key inside a chip, a hardware unit that the attacker cannot easily break into – and definitely without spending a significant amount of resources and time. Now even if the adversary manages to get hold of your chip for a few seconds (say, you give the card to a store clerk), they will not be able to break into it and steal the key.

Tamper-evident hardware. Consequently, it is interesting to have hardware units that are *difficult* to break into, even if it is not *impossible* to do so. The unit may still be useful in improving security, as long as there is a significant cost, and if the attack will take some time and/or may leave some trace on the device showing it was attacked. Such hardware is often called *tamper-evident* hardware.

Chip-cards are examples of such units. They are basically cards with their own computer on board, complete with CPU, storage, operating system and various software put in by the manufacturer or distributor. They often come with built-in co-processors for encryption, in some cases even with a dedicated hardware implementation of RSA. The storage is physically protected such that access to using the keys on board should be only through the card’s own CPU. This means you must know the PIN code that opens the card and follow the communication protocol. The best cards on the market have various protective layers put on top of the CPU and storage devices so that tampering with the card is hard without destroying the data on the card. Current state of the art is that breaking into a smart card is not impossible, but it requires a skilled and determined attacker, it takes non-negligible time, and requires specialized equipment. Thus chip-cards may be a very useful part of many systems.

Side-Channel Attacks. One has to be careful, however, with the way one uses the processor(s) on the card. An example of this: chipcards have to rely on external power-sources to operate, so an attacker may be able to continuously monitor how much power the card is consuming. The problem with this is that sometimes the card's power consumption depends on the instructions the CPU is currently handling, simply because some CPU commands require more power than others to execute. If the critical parts of code are not carefully done, this may reveal information on the keys used by the card. An example: the standard algorithm for doing an RSA decryption scans the secret exponent d bit by bit; it executes one set of operations if the bit is 0, another if the bit is 1. In a straightforward implementation, the operations needed for a 1-bit may consume a significantly different amount of power than those for a 0-bit, and so by plotting power consumption against time, the adversary will see a distinct pattern for every 1-bit, and so can read off the secret key directly!

In general, this shows that information can escape from hardware devices in unexpected ways, and one should not just consider the possibilities for output that the device was designed for. Attacks that break cryptography by capturing physical information about the computation process (such as time, power consumption, electromagnetic radiation, sound, or more) are usually known as **side-channel attacks**.

Two- and Multi-factor Authentication. A primary application for secure hardware is for so-called **two-factor authentication (2FA)** or **multifactor authentication (MFA)**.

The idea is to authenticate a user in two ways: first by a password and second by checking that they have a certain hardware unit in their possession.

The basic idea is (again!) just the same as the protection from replay-attacks in the authenticity chapter. That is, one can put a secret key K inside the unit. This key is also held by the verifying party. The verifier issues a challenge c (a nonce) which the user forwards to the hardware unit, which then returns a response $t = \text{MAC}_K(c)$ that can be verified by the other party. The main difference from previous uses of the same idea is that this time it is the job of the (human) user to read the MAC from the device and type it somewhere else. Thus, the code t is much shorter than in standard cryptographic authentication. In modern applications such codes are called **one-time passwords** and are typically composed of six numerical digits. Often one-time passwords have very short expiration times, and sometimes the challenge c is replaced by the current time, this frees the user from manually forwarding c to the device.

You might have encountered one-time passwords in small hardware units that can be used e.g., in connection with the Danish MitID system. You might also have encountered one-time password as part of some authenticator app running on your smartphone. Finally, you might have received one-time passwords via SMS or your email as a way to prove possession of that particular phone number or email address.

The big security advantage of two-factor authentication is of course that it is no longer possible to steal a password from the user and then freely impersonate them. However, it is important to realize that two-factor schemes can still be attacked, because the one-time password only depends on the user key and the nonce, and not on what the user actually wants to do once they are logged in. As a result one can do so called *real-time phishing*: say the user wants to log into their bank. The adversary fools the user to go to their web page rather than that of the bank and log in there instead. At exactly the same time the adversary logs into the bank in the name of the user, forwards the nonce to the user and relays their response to the bank. They can now spend money in the user's name.

This is why modern usage of one-time passwords e.g., when used to authorize credit card transactions, usually contains the context of the transaction to be approved e.g., *here is code 123456 to approve a payment of 100kr on webshop.com*.

In other cases, secure-hardware units are used to approve transactions by pressing some button on the hardware unit (which might be connected e.g., via USB to your laptop). Also in this case it is a good idea for the device to show (to the possible extent since such devices typically have very small screens) some context about the transaction to be approved.

Tamper-Resistant hardware. Other types hardware units are much more difficult to break into. These are called *tamper resistant* or *tamper-proof*. There are devices from IBM, for instance the IBM 4758, that have been certified by the US administration to the highest possible level of tamper resistance. This means that it is estimated that not even a well funded organisation with expert knowledge can break into the device. The 4758 is a plug-in card for PC's. Its protection mechanisms are alive at all times and will erase all internal keys if an attack is detected. It has its own CPU, storage and battery back-up and can do all the standard cryptographic algorithms internally. Banks use these units to protect particularly sensitive and long-lived data, such as PIN codes for credit cards. Also CA's use this kind of technology to store their private keys used for issuing certificates. Currently no attacks are known to break into a 4758.

9.3.10 Biometrics

An increasingly popular way to replace passwords as tools to protect access to hardware units are biometric. Most laptop today come with fingerprint readers, and most smartphone can let a user access the device via face-recognition. Other signals can be used as well, such as voice or eyes (although they are not as popular). There are a few important things to understand about biometrics.

Fuzzy Verification. When verifying biometric signals one cannot use an “exact match” approach like it is typically done with PINs or passwords, since biometric signals will naturally change due to time and other external conditions: Did the user shave today? Are they wearing make-up? Are we in the middle of a pandemic and everyone is using face-masks? Thus, verification has to be flexible enough not to lock out honest users while at the same time strict enough not to give access to an attacker. For some technologies, such as face-recognition, there is also an effort to ensure that the technique cannot be easily bypassed using e.g., a picture of the user, by attempting to verify that the user is “alive”.

Permanency and Link to Identity. Another tradeoff to be considered when using biometrics is the following: of course biometrics are convenient since you typically carry your body with you. However this also means that they cannot be easily revoked or changed if you are afraid someone has stolen them. Similarly, a user can choose a different password and username that are unrelated to their physical identity for different services, while biometrics will stay the same and might help an attacker linking users between different platforms, with negative consequences for users' privacy and anonymity.

Finally, when biometrics are used we still have to address the same challenges as in password security, namely ensuring that an attacker cannot steal the biometric signal in transit or at the verifier side.

9.4 Exercises

Exercise 9.1 (Yet Another AKE Protocol) Consider the following proposal for set-up of a secure channel: Assume A has already the public encryption key pk_B of B . Assume also that

ciphertexts made with pk_B are from the same set as the plaintexts – as is the case for RSA, for instance.

1. A chooses a random message R and sends to B $E_{pk_B}(E_{pk_B}(R))$.
2. B decrypts twice to get R and sends $E_{pk_B}(R)$ back to A
3. A checks that the value he received is indeed $E_{pk_B}(R)$, and if so he sends OK to B .
4. Both parties now use R as a secret key for the following communication. Say, we derive from R both a key for encryption and for message authentication codes, and every message sent is encrypted and tagged with an authentication code.

Suppose first that B is honest, and receives OK in step 3. When B receives a message after step 3 with a correct MAC, can he trust that this message came from A ? When B sends an encrypted message, can he trust that only A will be able to decrypt? Why or why not?

Suppose now that A is honest and the check in step 3 is OK. When A receives a message after step 3 with a correct MAC, can he trust that this message came from B ? When A sends an encrypted message, can he trust that only B will be able to decrypt? Why or why not? Hint: the answer to this one is more subtle than you may think. An adversary could start a new session with B and use data he knows from the past. ▼

Exercise 9.2 (Needham-Schroeder) Consider the Needham-Schroeder protocol. Suppose we remove the last message from the protocol, i.e., A simply doesn't send $E_{pk_B}(n_B)$. What happens now to the attack described in the notes – does it still work? Does the protocol still satisfy the properties required from an authenticated key exchange protocol? (Hint: look at the situation from B 's point of view after the protocol is finished.)

Can you find other ways of making the Needham-Schroeder protocol more secure? ▼

Exercise 9.3 (Liveness Check) Consider the following authentication protocol for two parties S, R who are assumed to share a secret key K_{SR} for symmetric encryption. The protocol is not designed to do exchange of a secret session key, but only tries to ensure for S (R) that R (S) is currently active at the other end of the communication channel.

1. S sends a nonce n_S to R .
2. R chooses a nonce n_R and sends $E_{K_{SR}}(n_R), E_{K_{SR}}(n_S)$ to S .
3. S decrypts under K_{SR} what he receives and checks that the last encryption of the two contains the n_S he chose in Step 1. He returns the result of the first decryption to R .
4. R checks that what S returned equals the n_R he chose in Step 2.

The intuition is that R would not be able to encrypt n_S correctly, nor would S be able to decrypt n_R correctly, without knowing K_{SR} . And so (hopefully) the protocol would not be successful unless both parties were active at the same time. However, consider the following application of the protocol:

A is a machine which communicates with several parties, so it stores for each other party B a secret key K_{AB} , and we assume that B is the only one, other than A , who knows K_{AB} . When another party B wants to start a session with A , the following happens:

- B sends his username to A .

- This causes A to start a new process which will handle the new session. The process looks up K_{AB} in A 's database, and conducts the above protocol with B , where A sends the first message, i.e., A plays the role of S and B plays the role of R .

When A wants to start a session with another party B , the following happens:

- A starts a new process which will handle the new session. The process looks up K_{AB} in A 's database, and sends the user name of A to B .
- The process then conducts the above protocol with B , where B sends the first message, i.e., A plays the role of R and B plays the role of S .

Question 1: Show that if the processes run by machine A are independent, i.e., do not communicate with each other, then when A makes a call to B , an adversary can intercept the call, do further interaction with A , and complete the protocol successfully, i.e., A will think he is talking to B , even though B is not present at all.

Question 2: How can the way A handles the protocol be changed such that the security problem you identified above is avoided? if you see several ways to solve the problem, feel free to mention them all.



Exercise 9.4 (Weak Parameters for Diffie-Hellman) Assume that you are an attacker who got access to the code repository of some Diffie-Hellman repository with hardwired (g, p) parameters, and want to weaken the security of the system by replacing the original (g, p) with some values that will allow you to more easily break the security of the Diffie-Hellman protocol. In particular, focus on secrecy i.e., you want to make it easier for yourself to find the key $K = g^{xy} \bmod p$ having only access to $X = g^x \bmod p$ and $Y = g^y \bmod p$. Can you find any choice for g , that ensures that $K = g^{xy} \bmod p$ can ever only be a single value regardless of the choices of x and y ? Can you find some value for g for which K can only assume a value from a very small subset?

Can you imagine any attack in which you replace the value of p instead?



Exercise 9.5 (Variants of DH-AKE) Consider different variants of the authentication steps in the Diffie-Hellman authenticated key-exchange and see if you can find any attack. Consider for instance:

1. A variant in which the MAC and the signature are computed only on (X, Y) (and not pk_A, pk_B).
2. A variant in which the MAC and the signature are computed only on (pk_A, pk_B) (and not X, Y).



Exercise 9.6 (Password vs. Passkeys) Consider the difference between using password and passkeys when logging in into a website. For concreteness:

1. When using password: the user sends (u, pw) to the server that fetches (u, h) and checks whether $h = H(pw)$.

2. When using passkeys: the server sends u to the server. The server fetches (u, pk) , picks a random challenge c and sends it to the user. The user replies with $s = S_{sk}(c)$. The server checks the signature as $Ver_{pk}(c)$.

Question 1: Consider a passive attacker who observes the communication of a user and then later tries to impersonate them. How does the security of the two systems compare if host and user communicate in the clear? What if an encrypted connection is used? For the passkey solution, you are welcome to think of different variants, for instance should the user only sign the challenge or would it help to sign something else as well?

Question 2: Consider an attacker who breaks into the host and reads the file with information on users (password file or file with public keys). How does security of the systems compare if the attacker can only read (and not modify) this file? What happens if they can modify it?

Question 3: From a general point of view (including for instance also attacks that try to break into a user's PC), which system would you prefer (motivate your answer)?



Exercise 9.7 (Salt) Salt is a random value which is added when hashing passwords at the server side, which is stored at the server side together with the username and password. For concreteness, when the user registers the server picks a random salt s and stores $(u, s, h = H(pw, s))$.

It is recommended to always use salt when hashing passwords. Explain why salt makes the system more secure when the password file is known to the attacker.



Exercise 9.8 (Using Secure Hardware) Consider the following system for providing digital signatures, for instance in a home banking system.

The private key of each user is stored in a chipcard held by the user. The card must be inserted in a card reader attached to the user's PC and activated by typing a PIN code. When a digital signature is to be generated, software on the PC computes a hash value on the message to be signed. This hash value is sent to the card and the card returns the signature to the PC.

Question 1: Some card readers come with their own keyboard and the PIN code must be typed on this keyboard. Other readers are without keyboard and the PIN is typed on the user's PC and transmitted to the reader. Compare the security of the two types. What must an attacker do to break the security in the two cases?

Question 2: Describe possible security problems that may exist if the software computing the hash value of the message is not reliable, for instance it is infected by a virus.

Question 3: Some systems require that the PIN code is typed for every message signed, i.e., the user must explicitly confirm that they actually want to sign each message. Would such a requirement solve the security problems you identified in Question 2? Why or why not?

Question 4: Suppose private keys are instead stored in a hardware unit attached to a central server, and a user must authenticate themselves to the hardware unit in order to have the unit use their private key for signing messages. Compare the security of such a system to the one above. Note this solution is essentially the one that DanID used for the Danish digital signature system from the fall of 2009. Here the authentication uses one-time passwords in the first version.



Exercise 9.9 (Multiuser Security) Consider a password system controlling access to a multiuser system. Let L be the lifetime of a password, S the number of possible different passwords, N the number of users, and R the maximal login rate, i.e., the maximal number of login attempts for a single user that the system will accept in one time unit before an alarm is sounded. Concretely, this means for instance that if time is measured in minutes and $R = 10$ then if the system detects more than 10 failed login attempts from the same user within one minute, then this is assumed to be an attempted attack, and the system administrator is alerted.

Now, assuming passwords are chosen at random, and that we only consider attacks where the adversary tries to log in as a particular user using some guess at the password, derive an expression in terms of L, S and R for the probability that the adversary finds the password of one particular user during one life cycle of a password, without the attack being detected.

Assume for simplicity that all users change passwords simultaneously. Find an expression in terms of L, S, N and R for the probability that the adversary could find the password of SOME user, during one life cycle, without the attack being detected. ▼

Exercise 9.10 (Rainbow Tables) In this exercise, we consider the case where an attacker learns $H(pw)$ for a password pw and a hash function H , and wants to find pw . If there are N possible passwords, one can of course find pw by spending $O(N)$ time trying all possibilities.

But now suppose the attacker knows the function H ahead of time and is able to invest large resources in doing precomputation, so that once they get $H(pw)$, they can find pw much faster than if they had to start from scratch.

One trivial way to do this is to make a sorted list with entries of the form

$$(H(pw_1), pw_1), (H(pw_2), pw_2), \dots$$

where pw_i runs through all possible passwords. With such a table one can just look up the target value $H(pw)$ and read out the desired value of pw directly. This uses almost no on-line computation, but one needs $O(N)$ memory which is usually not practical.

Here is an idea for a different method that uses less memory at the expense of more on-line time and can therefore lead to a more practical attack:

We let g be a function that can take a hash value as input and output a string that might be a password. For instance, if H is SHA-256 and passwords are 7 characters, then g could simply output the first 56 bits (7 bytes) of the input. We define a function F by setting $F(pw) = g(H(pw))$. Note g is not introduced because we hope it will return the target password, there is no real chance that this will happen. We just want a way to construct from H a new function that maps into the password domain, since this turns out to be useful later.

Now, the pre-computation works as follows, using parameters t and n :

- For $i = 1$ to t : choose random password pw_i , and then compute $y_i = F^n(pw_i)$.
- Store a table T with entries (pw_i, y_i) , sorted by the y_i -values.

Here, F^n means F iterated n times, $F^n(pw) = F(F(\dots F(pw) \dots))$.

We will now consider a matrix M with t rows and $n + 1$ columns, where $M[i, j] = F^j(pw_i)$. We let j run from 0 to n and define $F^0(pw_i) = pw_i$. Note that this means that T consists of the first and last column of M . Note also that we are not going to store M , this would be much too much data, we just need to define M , in order to explain what computation we need to do when we search for a password. The i 'th row of M looks as follows, where the first and last element is what we store in T :

$$pw_i, \quad F(pw_i), \quad F(F(pw_i)), \quad \dots \quad F^{n-1}(pw_i), \quad F^n(pw_i) = y_i$$

1. Describe an algorithm that uses T to find pw from $H(pw)$, under the assumption that all the y_i 's are distinct, and that the pw we are looking for is found somewhere in the first n columns of M . So we assume that for some i, j where $j \leq n$, we have $pw = M[i, j]$.

Remember that we do not store M , only T .

Hint: Note that, because you know $H(pw)$, you can compute $g(H(pw)) = F(pw)$. Now, begin by finding out which row of M contains pw . Next, use that information to find pw . For this, it is useful to note that our assumption $pw = M[i, j] = F^j(pw_i)$ implies that $F(pw) = F^{j+1}(pw_i)$.

2. How many times does your algorithm need to compute F ? Argue that your answer is correct.
3. What might go wrong if not all y_i 's are distinct and (optionally) what could you do in the preprocessing to avoid this problem?

In general there is no guarantee that the right pw is found by the algorithm, since the target value may not be in the matrix M at all. But Martin Hellman, who had the basic idea, developed it further using several different tables and a smart choice of t and n to get an algorithm that uses memory and on-line time $O(N^{2/3})$ where N is the number of possible passwords. ▼

Exercise 9.11 (Software Wallet) Use your solution from Exercise 6.15 to create a software wallet for an RSA secret key. It should have these functions.

1. `Generate(filename string, password string) string` which generates a public key and secret key and places the secret key on disk in the file with filename in `filename` encrypted under the password in `password`. The function returns the public key.
2. `Sign(filename string, password string, msg []byte) Signature` which if the filename and password match that of `Generate` will sign `msg` and return the signature. You pick what the type `Signature` is.

Your solution should:

1. Make measures that make it costly for an adversary which gets its hands on the keyfile to bruteforce the password.
2. Describe clearly what measure have been taken.
3. Explain why the system was designed the way it was and, in particular, argue why the system achieves the desired security properties.
4. Test the system and describe how it was tested.
5. Describe how your TA can run the system and how to run the test.

▼

Chapter 10

Asynchronous Agreement (2026)

Contents

10.1 Introduction	252
10.2 Defining Asynchronous Reliable broadcast	253
10.2.1 Why is this not just Flooding?	254
10.2.2 The Godot Problem	254
10.3 A CAP Theorem: No ARB for Three Parties	258
10.4 Intermezzo on Modelling†	261
10.5 Bracha: Asynchronous Reliable Broadcast from Authenticated Channels	263
10.5.1 Analysing Bracha Broadcast	269
10.6 Defining Asynchronous Byzantine Agreement	273
10.7 Implementing Asynchronous Byzantine Agreement using Coin-Flip	274
10.7.1 The Reader is Visited by the Goddess of Chance	274
10.8 SVUP: Stabilizing Vote Updating Protocol, $n = 4$, $t = 1$	278
10.9 SVUP for General n and $t < n/3$	288
10.10 Interactive Coin-Flips*	290
10.11 Impossibility of Deterministic Asynchronous Byzantine Agreement*	290
10.11.1 Dealing with Random Coins	295
10.11.2 Stable/Unstable States/Messages	295
10.11.3 There are Unstable Initial States	296
10.11.4 The Forbidden States	296
10.11.5 Unstable States have Unstabilizing messages	297
10.12 Exercises	297

Attack Model *In this chapter we assume that:*

1. *There are n parties.*
2. *We assume that there are at most t Byzantine corruption and $n = 3t + 1$ such that $t < n/3$.*
3. *The adversary cannot drop messages.*
4. *The adversary can delay messages arbitrarily, but must eventually deliver all messages.*
5. *The adversary cannot inject messages: we assume authenticated channels.*

6. *The parties do not have a clock.*

7. *We do not assume flooding, only point-to-point channels.*¹

10.1 Introduction

In this chapter we study protocols which allow several parties in a distributed system to agree on a decision even in the presence of a Byzantine adversary, which tries to make them disagree by deviating from the protocol. This adversary gets to control a number t of the participants in the protocol. It is allowed to make them behave arbitrarily badly. We will study reliable broadcast and Byzantine Agreement.

Reliable broadcast means that a sender should send a message m to a set of receivers. The receivers must agree on the message even if the sender is Byzantine and tries to make them output different messages. Even worse, they must agree even if some of the receivers are corrupted and collude with the sender.

In Byzantine Agreement instead each party has as input a single bit, 0 or 1. We can think of the input as a vote *No!* or *Yes!*. The goal is for the honest parties to agree on an output. The only requirement is that if all honest vote *No!* then the common output of all honest must be *No!*, and if all vote *Yes!*, then the common output must be *Yes!*. If the inputs of the honest parties are mixed, the output may be either *No!* or *Yes!*, but all honest parties must *agree* on the output. A concise way to say this is: the honest parties must agree on a value sent by an honest party.

We will study the problems in the *asynchronous* model. The main characteristic of this model is that messages can be delayed for an arbitrarily long time. This means that if a party is supposed to send you a message and it does not arrive, you cannot conclude that the party is corrupted (Byzantine) and did not send the message. Namely, it might just be that the message is very, very slow. In such a model, there is no use for a clock. So we simply do not give parties access to clocks. They are state machines which only send messages in response to receiving messages.

We will solve the problem assuming we can already do authenticated point-to-point communication. I.e., parties can send messages directly to each other, and they can trust who the messages come from. We can model it as follows.

Syntax A party S can get an input of the form (R, m) . It means that S should send m to R .

A party R can get an output of the form (S, m) . It means that S sent m to R .

Authenticity If an honest R gets an output (S, m) and S is honest, then at some point S gave the input (R, m) .

Liveness If an honest S gets input (R, m) and R is honest, then eventually R gets the output (S, m) .

When we solve agreement problems, we are interested in three properties.

Agreement All honest parties make the same decision.

Validity The decision that is made must be sensible in some sense.

¹I.e., a corrupted party may send a message to some parties and not others. The reason for not assuming flooding is that implementing reliable flooding (flooding against Byzantine corruptions) is the very topic of the chapter.

Liveness/Termination If all parties start running the protocol, then all honest parties eventually end up making a decision.

Below we will define these properties more carefully for Asynchronous Reliable Broadcast (ARB) and Asynchronous Byzantine Agreement (ABA). We will give one example of a secure protocol for each task. The protocol we give for asynchronous broadcast can only tolerate that strictly less than one third of the parties are corrupted. We will also argue that this is the best we can do if we only use authenticated channels. The protocol we give for asynchronous Byzantine agreement will use randomness. At the end of the chapter we will argue that it is impossible to solve Asynchronous Byzantine Agreement without using random choices.

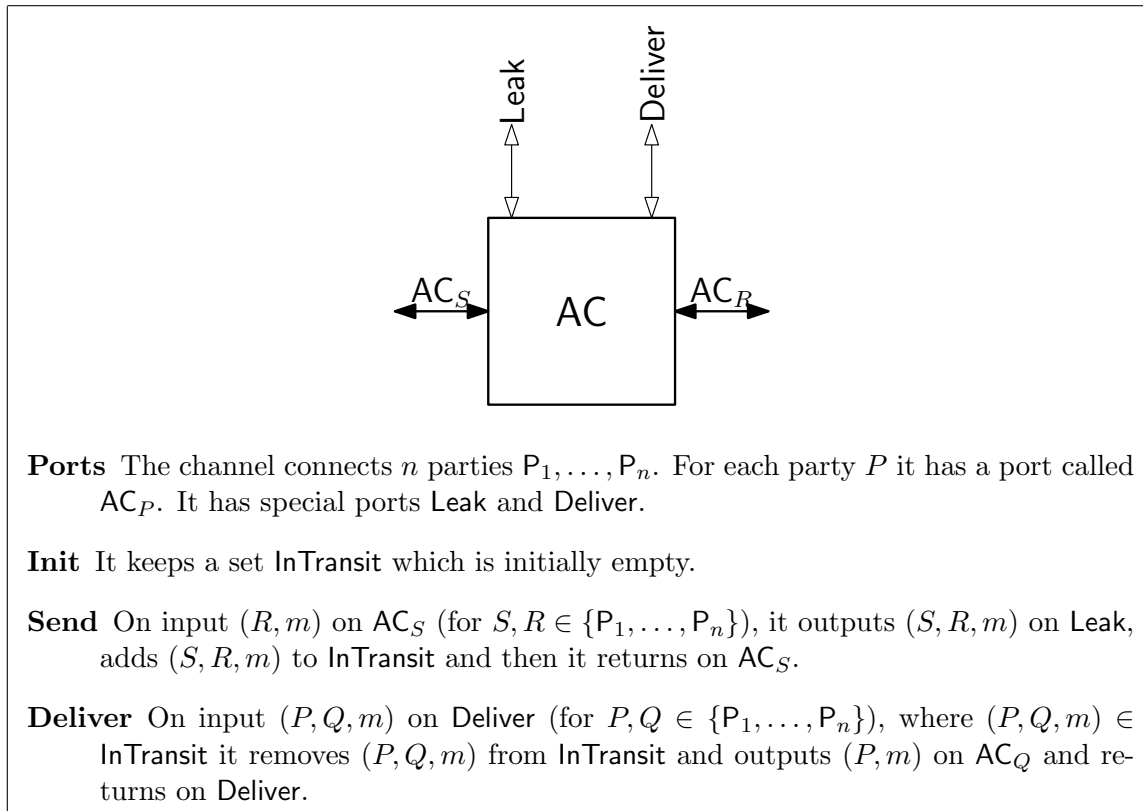


Figure 10.1: Authenticated Channel

10.2 Defining Asynchronous Reliable broadcast

In this chapter we assume we start from point-to-point communication. A particularity of point-to-point channels is that you cannot see what other parties are sending to each other. This is opposed to flooding, where all parties eventually see the same messages. However, sometimes we would like to know what others sent to others. Say we have a point in a protocol where a sender S is to send a message m to all other parties. Maybe it is important that the same message is sent to all receivers. We sometimes call this a broadcast and we call S the broadcaster. But with point-to-point channels a Byzantine corrupted broadcaster could send different messages to different parties. The parties would not know. Our goal is to solve this problem. We want to find a way for the receivers to check if the broadcaster was cheating, and if so, agree on what the output should be. If the broadcaster is honest of course they should agree on the message

which the broadcaster intended to send. If the broadcaster is corrupted, they are allowed to agree on anything, but it is important that they agree. A protocol guaranteeing this is called **asynchronous reliable broadcast** or **ARB**. There is a designed party P_i who is the broadcaster. It broadcasts a message m and each party P_j will then output some m_j and the goal is that $m_j = m$.

In a bit more detail. Party P can get input of the form (P, bid, m) meaning that it broadcasts m under the broadcast identifier bid . Since many broadcasts can happen at the same time, it is hard to talk about what it means that the parties agree on *the* message being sent without being able to name the sessions. They should agree on the message sent under each pair (P_i, bid) , i.e., what P_i broadcast for bid . We tacitly assume that honest parties only use each bid once and only once outputs something of the form $(P_j, \text{bid}, \cdot)$ for each (P_j, bid) . An ARB should have these properties:

Agreement If two honest parties P_j and P_k output (P_i, bid, m_j) respectively (P_i, bid, m_k) then it must always be the case that $m_j = m_k$.

Validity If some honest P_i gets input (P_i, bid, m) and some honest P_j outputs (P_i, bid, m_j) , then it is always the case that $m_j = m$.

Termination 1 If P_i is honest and gets input (P_i, bid, m) , then eventually all honest parties P_j output some (P_i, bid, m_j) .

Termination 2 If P_i is corrupted and some honest P_k gives an output (P_i, bid, m_k) , then eventually all honest parties P_j output some (P_i, bid, m_j) .

We also formalize asynchronous reliable broadcast a bit more precisely as an ideal functionality—see Fig. 10.4 on page 261—but the above trace-based definition will be sufficient for presenting and analyzing the upcoming protocols.

10.2.1 Why is this not just Flooding?

The reader may be forgiven for asking themselves, *why is this not just flooding?* We discussed in Section 3.10 how we can flood messages. Can't we just do that here too? Something like this: If you—directly or indirectly—see a message (P, bid, m) from P for the first time, then send it to all parties. Surely this gives Termination 1 and Termination 2. The real problem is Agreement. What if P is corrupted and sends (P, bid, m) to some parties and $(P, \text{bid}, m' \neq m)$ to other parties? Now we do not have a unique message sent under bid . We cannot make the rule: *use the m received first*, as parties might receive the messages in different orders.

Add counters as in FIFO? Well, the corrupted sender can just lie again: it sends $(P, \text{bid}, 1, m)$ to some parties and $(P, \text{bid}, 1, m' \neq m)$ to other parties.

OK, maybe say that if two different (P, bid, m) and $(P, \text{bid}, m' \neq m)$ were sent, then deliver nothing. We know the sender is corrupted, so not delivering anything is allowed according to Termination 1. Well, then what if the sender is honest and only sends (P, bid, m) ? When do we output? What if we output m after 3 minutes and then some other $(P, \text{bid}, m' \neq m)$ arrives and we realize we made a mistake outputting m . When are we done waiting?

10.2.2 The Godot Problem

Before we solve the ARB problem, let us dive a bit more into the definition and the problem with waiting in the asynchronous model. Consider a broadcast, say with name bid , where all parties know that some P_i ought to do a broadcast with name bid . Note that if the broadcaster is corrupt, then the protocol might not produce any output even if all honest parties start

running the protocol. Termination 1 only guarantees that there will be output for bid if *the broadcaster is honest*. So if the broadcaster is corrupt, we get no guarantee from Termination 1. And Termination 2 only guarantees that there will be output at all honest parties if at least one honest party gave output. The reader might wonder why we did not ask for something simpler and stronger, like:

Strong Termination If all honest parties expect a broadcast for bid then eventually all honest parties P_j output some m_j .

It turns out it is impossible to implement because we cannot distinguish the situation where the corrupted sender never sends anything and the case where an honest sender sends its intended messages, but the messages get delayed for a very long time. The receivers cannot decide on something in the first case without risking deciding on something wrong in the latter case. We will call this the **Godot Problem**, as it is reminiscent of the play *Waiting for Godot* by Samuel Beckett. In the play, two characters wait for a third character, Godot, who never arrives. They cannot decide to leave because they might miss Godot if he arrives just as they are leaving. So they wait forever.

(A road. A tree. A network. No messages arrive.)

ESTRAGON: Let's leave!

VLADIMIR: But Godot was supposed to ARB us a bit, Estragon!

ESTRAGON: We've been running the ARB protocol for three days—

VLADIMIR: —Time is not well-defined here, we are in the asynchronous model

ESTRAGON: Don't quibble. We've waited long enough. If nothing arrives, we should just decide.

VLADIMIR: Decide what?

ESTRAGON: Something reasonable. Zero. Everyone likes zero.

VLADIMIR: That is not a protocol argument.

ESTRAGON: It is a human argument. Surely the protocol shouldn't just... stall.

VLADIMIR: You are asking for Strong Termination again.

ESTRAGON: Of course I am! If all honest parties run, then eventually everyone outputs. End of story.

VLADIMIR: End of Validity, actually.

ESTRAGON: Look, suppose Godot is corrupt.

VLADIMIR: And sent nothing.

ESTRAGON: Right. Total silence. So we wait.

VLADIMIR: Or, under Strong Termination, we eventually decide anyway.

ESTRAGON: Good! Progress!

VLADIMIR: But when do we decide?

ESTRAGON: It has been three days! We decide now. 0!

VLADIMIR: Now suppose Godot is honest and has broadcast 1.

ESTRAGON: Ah! Then we should output 1.

VLADIMIR: But maybe all his messages were delayed for four days.

ESTRAGON: That's unlucky.

VLADIMIR: And indistinguishable.

ESTRAGON: From what?

VLADIMIR: From the corrupt Godot who sent nothing at all.

ESTRAGON: Oh.

VLADIMIR: And you wanted to make a decision after three days?

ESTRAGON: Yes... and output 0.

(Pause.)

ESTRAGON: Oh. So if we hear nothing for three days, then we output 1!

VLADIMIR: But what if the honest Godot has sent 0?—

VLADIMIR: —Same delays. Same silence. Same behavior.

ESTRAGON: So we must also output 0.

VLADIMIR: After three days.

ESTRAGON: Without having heard anything.

VLADIMIR: And now you see the problem.

ESTRAGON: Zero equals one.

VLADIMIR: Again.

ESTRAGON: I hate when that happens.

(Pause.)

ESTRAGON: Fine. Then let's change Validity.

VLADIMIR: No, if Godot is honest we must wait for his message.

ESTRAGON: Then... maybe we guess?

VLADIMIR: Now you are just being stupid!

ESTRAGON: So we just... wait forever?

VLADIMIR: If Godot is corrupt, yes.

ESTRAGON: That seems terrible.

VLADIMIR: It is the best we can do without breaking Safety.

(Pause.)

ESTRAGON: Then why not give up entirely? Why have any termination conditions at all?

VLADIMIR: Because if the sender is honest, we can indeed guarantee to hear something.

ESTRAGON: When?

VLADIMIR: Eventually...

ESTRAGON: But what is Termination 2 about then?

VLADIMIR: Because something even worse can happen.

ESTRAGON: Worse than waiting forever?

VLADIMIR: Yes. Waiting alone.

ESTRAGON: I don't follow.

VLADIMIR: Suppose the broadcaster is corrupt. He sends just enough garbage to convince one honest party to output a value.

ESTRAGON: Lucky fellow.

VLADIMIR: The others see nothing convincing.

ESTRAGON: Unlucky fellows.

VLADIMIR: Now one honest party has decided and moved on. The others wait forever.

ESTRAGON: So?

VLADIMIR: So the system has forked in time. One part of the system believes the protocol finished. The rest believes it never did.

ESTRAGON: That sounds... awkward.

VLADIMIR: It is disastrous for composition.

ESTRAGON: Composition? I don't follow...

VLADIMIR: If you later want to build something stronger—say, total-order broadcast—

VLADIMIR: —you will rely on the fact that once any honest party delivers a message, all honest parties will eventually deliver something.

ESTRAGON: Preferably the same thing.

VLADIMIR: Otherwise one replica applies an update...

ESTRAGON: ...and the others wait forever, stuck at the old state.

ESTRAGON: So if someone hears, everyone must eventually hear.

VLADIMIR: Exactly.

ESTRAGON: Even if the broadcaster is corrupt?

VLADIMIR: Especially then.

ESTRAGON: So we accept this:

If the broadcaster is honest, everyone eventually outputs.

If the broadcaster is corrupt, we might wait forever.

But if anyone outputs, no one is left behind.

VLADIMIR: That is the price of asynchrony.

ESTRAGON: We sacrifice liveness to preserve safety.

VLADIMIR: And we add just enough liveness back to make composition possible.

ESTRAGON: We wait.

VLADIMIR: But we wait together.

(*Pause.*)

ESTRAGON: Still no messages, huh?

VLADIMIR: Maybe the Internet died?

ESTRAGON: Or no messages were sent?

(*Long pause.*)

ESTRAGON: Wait, I don't get it...

VLADIMIR: What?

ESTRAGON: If the broadcaster is corrupted we wait *forever*?

VLADIMIR: Yes.

ESTRAGON: If the broadcaster is honest then we get the message *eventually*?

VLADIMIR: Yes?

ESTRAGON: What is the difference between waiting forever and arbitrarily long? It feels the same to me.

VLADIMIR: It's about what *eventual* termination really means.

ESTRAGON: Not that it happens after three days apparently.

VLADIMIR: No, we say that a broadcast protocol *eventually* gives output if it is guaranteed to give output once all messages sent in the protocol are delivered.

ESTRAGON: So the outputs *are* guaranteed to be given!

VLADIMIR: Yes, when all the messages which the broadcast protocol sent have been delivered!

ESTRAGON: And when does that happen?

VLADIMIR: It is not guaranteed to happen; we only know that *if* all messages are delivered, *then* the good thing happens.

ESTRAGON: What good thing?

VLADIMIR: Like, if some honest party had output, then all other honest parties will also have output.

ESTRAGON: And without Termination 2?

VLADIMIR: Well, it could happen that some honest party had an output and that all messages of the protocol were delivered—

ESTRAGON: —and still some other honest party did not have output—

VLADIMIR: Right.

ESTRAGON: Maybe new messages will be *sent*?

VLADIMIR: It is the asynchronous model. Parties only send messages in response to receiving messages—

ESTRAGON: —or getting inputs?—

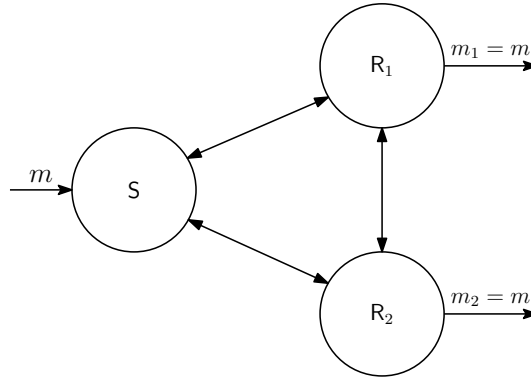


Figure 10.2: A broadcaster sending the same message to two receivers.

VLADIMIR: —Yes, they also send messages when they receive inputs. But inputs were given, all messages delivered. Nothing new will ever happen.

ESTRAGON: Yet, one honest party had output and another did not.

VLADIMIR: Yes!

ESTRAGON: That's bad!

VLADIMIR: Yes.

ESTRAGON: So, we should just require that if some honest party gave output and all honest parties had input and all messages were delivered in the protocol, then all other honest parties also gave an output!

VLADIMIR: We did. That is Termination 2!

ESTRAGON: Right.

(Pause.)

ESTRAGON: So. We've waited long enough. If nothing arrives, we should just decide.

10.3 A CAP Theorem: No ARB for Three Parties

Before diving into solving ARB, we first want to build some more intuition for why the problem is hard, and thus, why the solution looks so complicated.

As an example of the ARB problem consider the setting in Fig. 10.2. There are three parties connected by authenticated channels. The parties are also allowed to use signature if they want, as their transferability is stronger than what authenticated channels provide. In fact, they are allowed to use any kind of cryptography, as long as all communication is via the pairwise channels.

There is a sender S which must send the same message m to two receivers R_1 and R_2 . We want to tolerate that one of the three parties might be Byzantine corrupted. When we design the protocol we do not know which party will be corrupted, it may be any of them, the sender or one of the receivers. Think of it this way. First we design the protocol, and then some evil adversary decides which party to corrupt and now controls this party completely. We want that a corrupted receiver cannot manipulate the message sent by S to the other party. We also want that a corrupted sender cannot make R_1 and R_2 output different messages. We could formulate it as follows:

Agreement If both R_1 and R_2 are honest and R_1 outputs m_1 and R_2 outputs m_2 , then it always holds that $m_1 = m_2$.²

²Here it is implicit that it holds even if S is Byzantine corrupted, as the only premise we give is that R_1 and

Validity If both S and R_1 are honest, then $m_1 = m$. Similarly, if both S and R_2 are honest, then $m_2 = m$.

Termination If the sender is honest, then R_1 and R_2 both end up outputting something.

A protocol with these properties is called a **reliable broadcast**. When it works in the asynchronous model it is called an **asynchronous reliable broadcast** or ARB.

ARB sounds like a simple problem, but it is actually very hard. Sometimes so hard that it is impossible. In fact, it is impossible to solve for three parties as sketched above, at least if we are to tolerate that one of the parties may be Byzantine corrupted. It is even impossible if we are promised that the receivers are not Byzantine corrupted, the only way in which they may fail is that they may crash. So, we look at a model where either S is Byzantine corrupted or R_1 may crash or R_2 may crash, but at most one of these faults at a time.

We can use Fig. 10.3 to argue the claim.

If S sends FOO and R_2 is crashed we are guaranteed by Validity and Termination 1 that R_1 still outputs FOO. This is the left figure. Notice that when R_2 is crashed party R_1 will not know. At any point in time it just knows that it did not receive any messages from R_2 yet. Similar for S : it only sees that no messages have arrived—yet. It cannot know if R_2 crashed or the messages are on their way but very, very slow. However, the assumption is that somehow S and R_1 make it work out, and R_1 ends up outputting the intended message FOO; This is what Termination 1 plus Validity means: at some point R_2 gives an output and it is the right thing.

Similarly, if S sends BAR and R_1 is crashed we are guaranteed by Validity and Termination 1 that R_2 still outputs BAR. This is the right figure. When R_1 is crashed party R_2 will not know. It just sees that it received no messages from R_1 yet. Similarly for S . Nonetheless, at some point R_2 decides to output the intended message BAR (by Termination 1 and Validity).

Consider then the nasty Byzantine corrupted S illustrated in the bottom figure. It runs a copy of the honest S with input FOO and lets it talk to R_1 . It gives it no inputs from R_2 , as if R_2 was crashed. It also runs a correct instance of the honest S with input BAR and lets it talk to R_2 . It gives it no inputs from R_1 , as if R_1 was crashed. Furthermore, it delays the messages between R_1 and R_2 for a very, very long time. Recall that it is the adversary who schedules messages. It will deliver the messages later, but first it needs something bad to happen.

From the left figure we know that eventually R_1 will output FOO. The adversary will deliver messages between $S(\text{FOO})$ and R_1 until this output is given. During this it delivers no messages from R_2 to R_1 . The view of R_1 of what is going on is therefore the same as when R_2 is crashed (no messages arriving), so it will do the same as when it is running in the left figure. From the right figure we know that eventually R_2 will output BAR. Its view of what is going on is the same, so it must do the same.

After the two outputs were produced the adversary delivers all messages sent between R_1 and R_2 —the rules of the game are that all messages must be delivered eventually. Now they were delivered. The receivers might now realize that they made a mistake, but it is too late. They already gave outputs. And the outputs are different. They violated Agreement.

Proofs like this are often called CAP theorems in the literature, the original being proven in [9]. These theorems cap what is possible, like you shall not have Termination, Validity *and* Agreement. They are also called CAP as an acronym for Consistency-Availability-Partitioning. Here Consistency is just another word for what we called Agreement. Availability is another word for Termination: the system is available in that it is doing something: producing outputs. And Partitioning is another word for two regions of a system not being able to talk because messages get delayed for a very, very long time.

R_2 are honest.

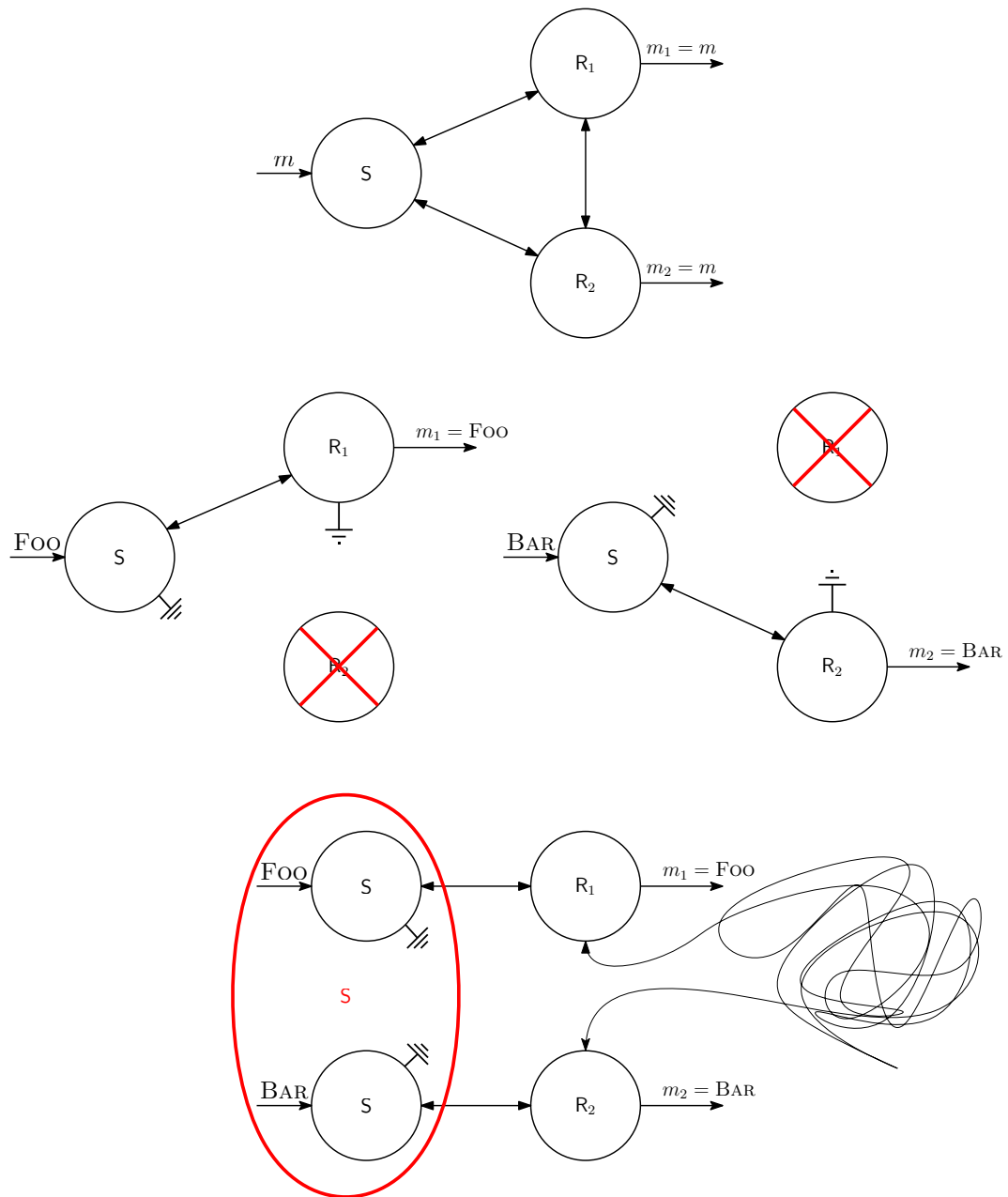


Figure 10.3: There is no reliable broadcast in the asynchronous model for three parties which tolerates that the sender may be Byzantine corrupted or that a single receiver may be crashed. If the protocol has termination and validity when R₁ is crashed and when R₂ is crashed, then it can be made to violate agreement when S is Byzantine and messages between R₁ and R₂ are delayed for a very long time—the receivers will both think the other receiver is crashed and will then trust S.

So, is the chapter done? Is the problem we try to solve impossible? No, it turns out that if we have $n = 4$ parties, one broadcaster and 3 receivers R_1, R_2, R_3 and if we trust that at most one party is Byzantine corrupted, then we *can* solve the problem. In general, if we want to tolerate t corrupted parties, then we need to begin with $n = 3t + 1$ serves and we can argue that $3t$ is not enough. So to tolerate 2 Byzantine corruptions, we can do it with 7 parties and we can argue that 6 are not enough. So let us get to it.

10.4 Intermezzo on Modelling†

To be sure we agree exactly what we are talking about, we can model both authenticated channels and ARB as ideal functionalities. This section is for the formally inclined reader. If you just want to see the protocols, you can skip. If you ever get a doubt that you understood the setting or the problem, you can come back here.

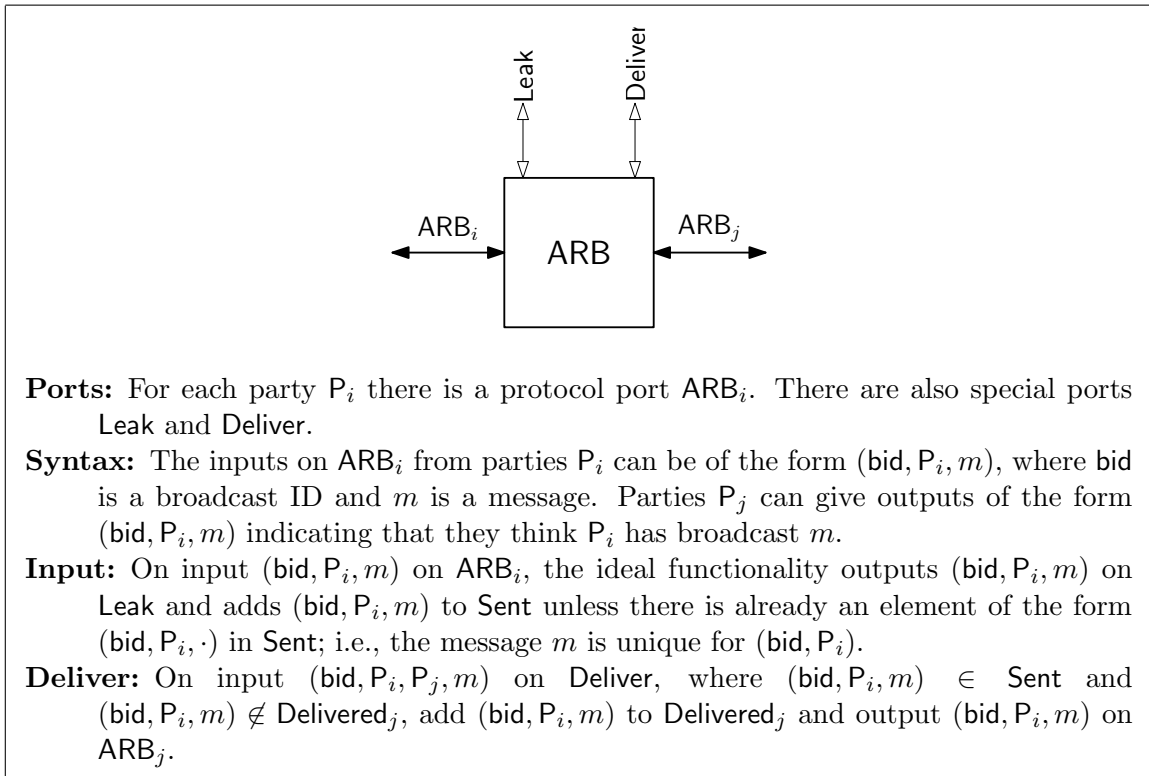


Figure 10.4: An ideal functionality ARB for Asynchronous reliable broadcast.

We start with a network of authenticated channels with eventual delivery. They can be modelled using Fig. 10.1. Note that when a message (P, m) is delivered it is always a message that was sent by P at some earlier point; this is what makes the communication authenticated. We discuss in Chapter 6 and Chapter 9 how such channels can be implemented. Note also that there is no guarantee about when a message will be delivered: delivery is controlled by the adversary connected to $Deliver$. We do not explicitly model “eventual delivery”. We only require that the ARB delivers messages when AC has delivered all messages.

We require from the implementation of the parties P_i that if they were activated and did not send a message, then if we activate them again—while no new message arrived—then they again send no message. This prevents that they count the number of times they were activated

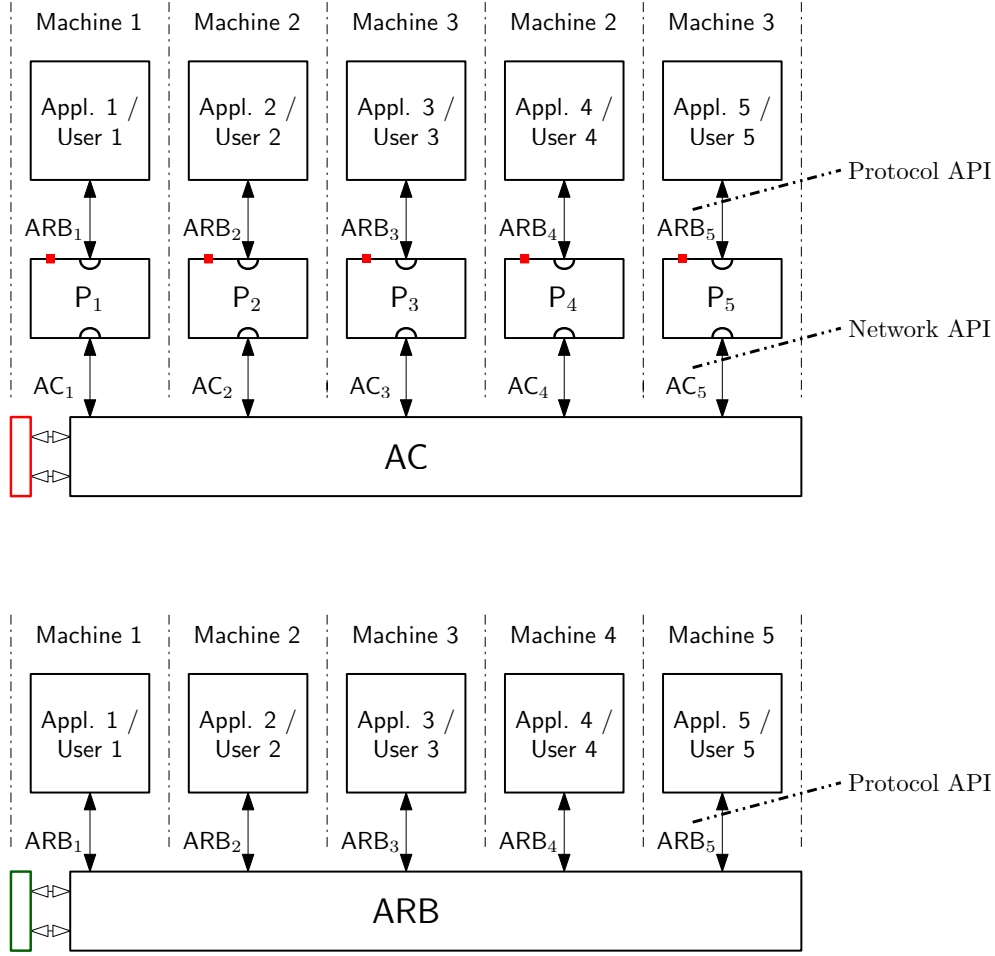


Figure 10.5: Building Asynchronous Reliable Broadcast from Authenticated Communication. The top system should look like the system below to the users. In the top system we have indicated that there is an adversary scheduling the delivery pattern of AC. Similarly, it schedules the activation pattern of the parties P_i . We say that the adversary **schedules the execution**. Looking like the system below just means that there is some way to schedule the lower system to make it look like the top one.

and use this as some rudimentary form of clock.

We can then define asynchronous reliable broadcast as an ideal functionality; see Fig. 10.4. The main difference is that we introduce some session identifiers **bid**, called broadcast identifiers. These allow several broadcasts to take place at the same time. If, for instance, P_i wants to send the same message HELLO twice, we need a way to distinguish the two executions. We then add the requirement that each **bid** is used at most once.

Notice that the ideal functionality ARB does not include anything about termination. Before reading on, think for a minute about why we did not add the following to the ideal functionality:

Termination 1: If P_i and P_j are honest and an input of the form $(\text{bid}, P_i, \cdot)$ is input on ARB_i , then eventually an output of the form $(\text{bid}, P_i, \cdot)$ must be given on ARB_j .

Termination 2: If P_j and P_k are honest and an output of the form $(\text{bid}, P_i, \cdot)$ is given on ARB_j , then eventually an output of the form $(\text{bid}, P_i, \cdot)$ must be given on ARB_k .

As the reader probably realized, the term “eventually” is not well-defined, and neither of course is “must”. We have delegated scheduling to the adversary exactly because we do not want

ARB itself to decide when to deliver. If anything, it is the adversary which must “eventually” deliver all messages. But even that is hard to define. What we will do instead is to say that when we implement ARB based on AC, then ARB must have the above liveness properties when AC delivered all messages sent on AC. Asking a protocol implementing ARB using AC to deliver messages when AC does not would be unrealistic anyway.

When we implement ARB from AC, what we mean is then that we cook up a protocol consisting of n parties, $P_1, \dots, P_n$ such that when we plug them into the protocol at the top of Fig. 10.5, then the system looks like ARB towards the users. The way we model liveness is then just by saying that when we compare the top system to the below system, then the only excuse for the protocol $\{P_1, \dots, P_n\} + AC$ to not have delivered a message sent by an honest party is that in AC there is a message sent by an honest party which was not delivered yet or some honest party received a new message but was not activated yet. This way we sneak out of defining “eventually”. We just let it mean that whenever AC gets around to do what the honest P_i asked it to do, then protocol $\{P_1, \dots, P_n\} + AC$ will also do what it was asked to do by the users of the system. We can also call this “liveness preservation”: when AC is live, then $\{P_1, \dots, P_n\} + AC$ is live.

Relating to the above we have the notion of an **exhaustive scheduling**, where no message is in transit and where no honest party if activated would send a new message.

Finally, we can address how we model Byzantine corruptions. It is illustrated in Fig. 10.6. We simply let the bad guy—a.k.a. the adversary—take over the Byzantine parties. Since now it is the bad guy which decides the inputs of the corrupted parties, we also let the good guy control these parties in the comparison. If the adversary makes P_1 send FOO on the protocol, maybe just by honestly following the protocol, then we need that the good guy can tell ARB that party 1 should send FOO. But note that by the way we programmed ARB, the good guy cannot make P_1 send different messages to different parties. So if the $\{P_3, P_4, P_5\} + AC$ would allow P_1 to send different messages, then no good guy can make the system look the same.

10.5 Bracha: Asynchronous Reliable Broadcast from Authenticated Channels

In this section we look at a Byzantine reliable broadcast protocol, called Bracha Broadcast, for the fully connected model with authenticated channels and asynchronous communication with eventual delivery. Recall that in the asynchronous model we do not use time and there is no known upper bound on the delivery time of messages. It tolerates $t < n/3$ corruptions when there are n parties.

The protocol was first discovered by Gabriel Bracha in 1984[3] and is arguably one of the most elegant asynchronous protocols. It concisely illustrates some concepts and techniques crucial for programming the asynchronous model, so we present it here and elaborate on some of these points.

Below we will simplify notation a bit. We will assume that it is always P_1 who is the broadcaster. When P_1 gets input (P_1, m) on ARB_1 to start a broadcast, we say that it got input $(BROADCAST, P_1, m)$. When a party outputs (P_1, bid, m) on ARB_j we say that it has output $(DELIVER, P_1, bid, m)$. This way we do not have to keep naming the ports.

Let n be the number of parties and let $t < n/3$ be the number of Byzantine corruptions to tolerate. The protocol is presented in Fig. 10.7. Note that if you receive $(SEND, P_1, bid, m)$ and $(SEND, P_1, bid, m')$ for $m' \neq m$, then you only echo the first one. This ensures that the broadcaster only is able to send one m per bid.

In Fig. 10.8 there is an example run with $n = 4$ and $t = 1$. It illustrates how the protocol runs and how parties stop triggering activation rules after they terminated. In Fig. 10.9 there

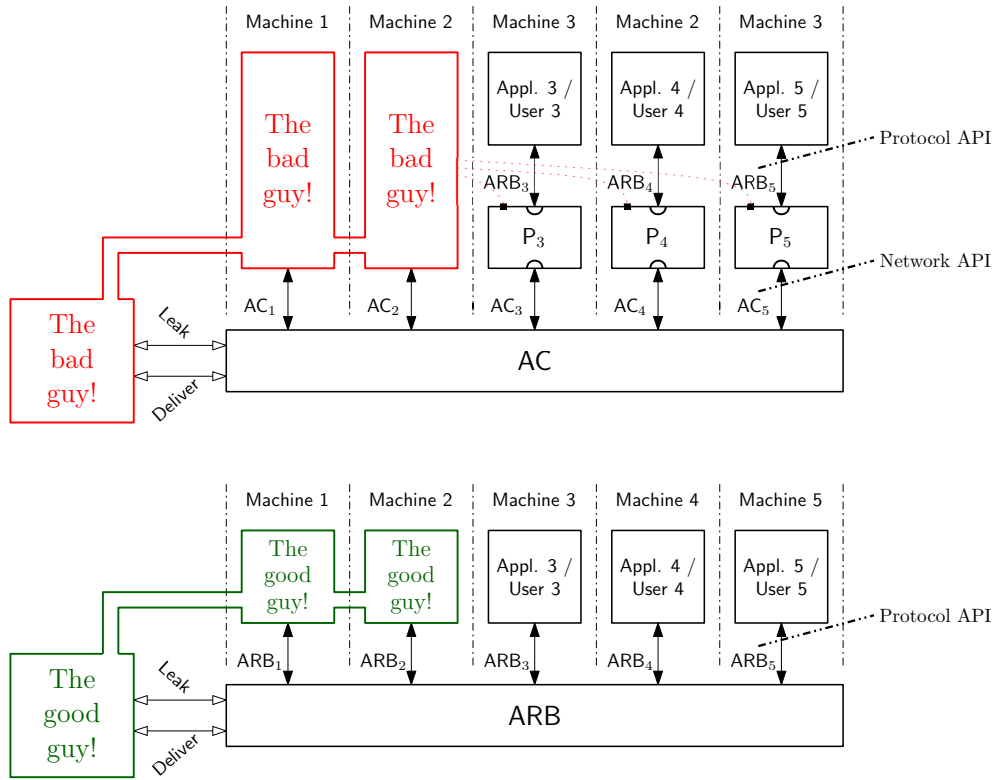


Figure 10.6: An adversary controlling two parties out of five, i.e., $n = 5$ and $t = 2$. The most important thing to take away from this is that the adversary is what we call **monolithic**, i.e., it is present everywhere. So we should expect any kind of arbitrarily crazy attack, not just attacks which make sense for the individual parties or which is “practical”. Think of an adversary which has broken into P_1 and P_2 and every other machine on the internet, except P_3, P_4, P_5 . If we can design a protocol working in that setting, it should also work in practice. On the other hand, trying to limit a formal model to consider only “practical” attacks has turned out to be hard.

Send P_1 :	On input $(\text{BROADCAST}, P_1, \text{bid}, m)$, send $(\text{SEND}, P_1, \text{bid}, m)$ to all parties. And honest party will do this at most once per bid.
Echo P_i :	On message $(\text{SEND}, P_1, \text{bid}, m)$ from P_1 , send $(\text{ECHO}, P_1, \text{bid}, m)$ to all parties. This is not done if you earlier received a message of the form $(\text{SEND}, P_1, \text{bid}, \cdot)$, i.e., with the same bid.
Ready 1 P_i :	If the message $(\text{ECHO}, P_1, \text{bid}, m)$ has been received from $n - t$ distinct parties, send $(\text{READY}, P_1, \text{bid}, m)$ to all parties, if this message was not already sent.
Ready 2 P_i :	If the message $(\text{READY}, P_1, \text{bid}, m)$ has been received from $t + 1$ distinct parties, send $(\text{READY}, P_1, \text{bid}, m)$ to all parties, if this message was not already sent.
Deliver P_i :	If the message $(\text{READY}, P_1, \text{bid}, m)$ has been received from $n - t$ distinct parties, output $(\text{DELIVER}, P_1, \text{bid}, m)$ and terminate the protocol.

Figure 10.7: Bracha Broadcast

is another example with corrupted sender. It illustrates why the **Ready 2** rule is needed for Termination 2.

Before we analyze the protocol, we want to highlight some general principles that are important for any fully asynchronous protocol: it is non-sequential, there are only activation rules, there is a point where parties wait for message from $n - t$ parties, there is a point where they wait for messages from $t + 1$ parties, and we have that $n > 3t$ to get a big enough overlap between the information of honest parties. Let us take them one by one.

Non-Sequentiality Each of the rules *Send*, *Echo*, *Ready 1*, *Ready 2*, and *Deliver* are independent activation rules. They trigger whenever their precondition is fulfilled. Recall furthermore the convention that if several activation rules have their precondition fulfilled, then the topmost one will be activated.

Note in particular that it might easily happen that a party triggers *Ready 2* before it triggers *Ready 1*. It might even happen that a party triggers *Deliver* and terminates before triggering *Ready 1*; in that case it will never trigger *Ready 1*. It might seem dangerous that some party terminates before running *Ready 1*, but in fact the protocol works exactly because we allow this. Consider a run with four parties. Assume that P_1 broadcasts and is corrupted. Note that $n = 4$ and $t = 1$, so $n - t = 3$. Consider the execution in Fig. 10.9. P_3 receives three READY messages and triggers *Deliver* to output and terminate. One of these READY messages is sent by the corrupted sender, which did not send it to the other parties. So the other parties only receive two READY messages, which is not enough to trigger *Deliver*. However, since P_3 triggered *Ready 2* it sends out a third READY message, which makes the other parties also get three READY messages and trigger *Deliver* to terminate. If we would have required that parties first trigger *Ready 1* before they can trigger *Ready 2*, then P_3 would never have triggered *Ready 2* and the other parties would never have terminated.

Gathering Maximal Information If you wait for messages from $n - t$ processes you are trying to collect as much information as you can *without deadlocking*. There are n parties in total and t of them might be corrupt and never send their messages. So if you wait for $n - t + 1$

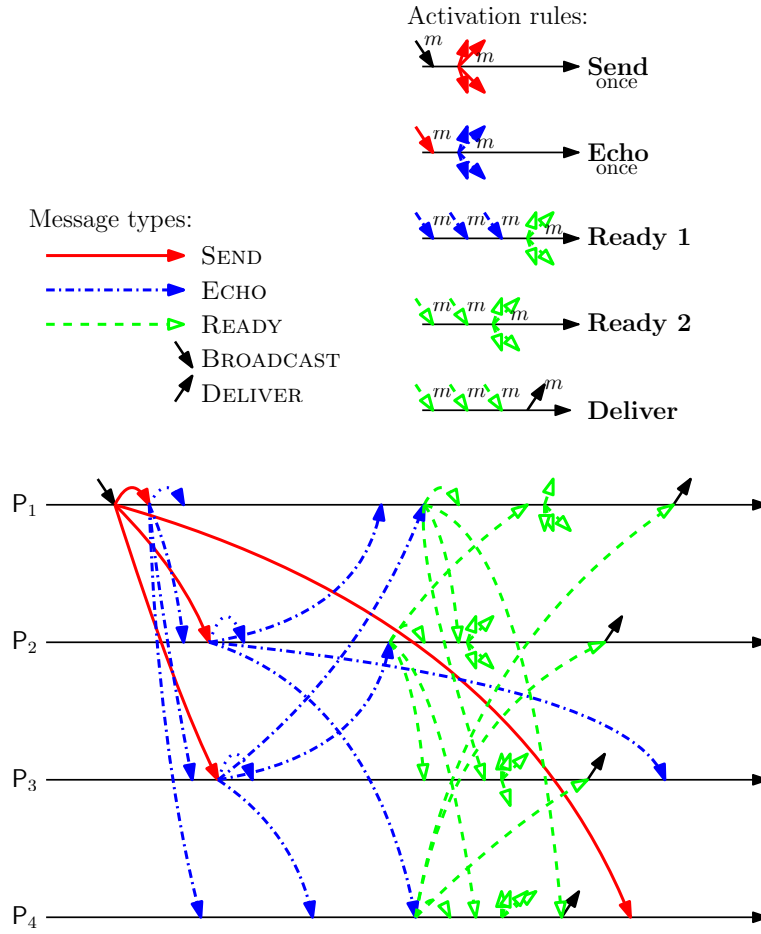


Figure 10.8: Example execution of Bracha Broadcast with all parties honest. The broadcaster sends out the (SEND, m) message—we do not show the message m in the timeline figure, only the type of message. It is tacitly understood that all types of messages are for the same m . When a party gets the SEND-message it sends an ECHO message to all parties. When the parties received three ECHO messages they send READY to all. When they received three READY message they deliver an output and terminates. Notice that after seeing two READY messages the parties trigger the **Ready 2** rule and send READY to all parties. We have only indicated those send events, not delivered them. Think of them as being delayed forever. As we see, this is not a problem when the broadcaster is honest, the parties will terminate anyway. The **Ready 2** rule is only needed when the broadcaster is corrupted: it is used to guarantee Termination **2**—see Fig. 10.9. Notice that parties do not process messages received after termination. For instance, P_4 will not send out ECHO messages as it receives the SEND message after terminating. Similarly, P_3 will never trigger the **Ready 1** rule as it receives the third ECHO message after it terminated.

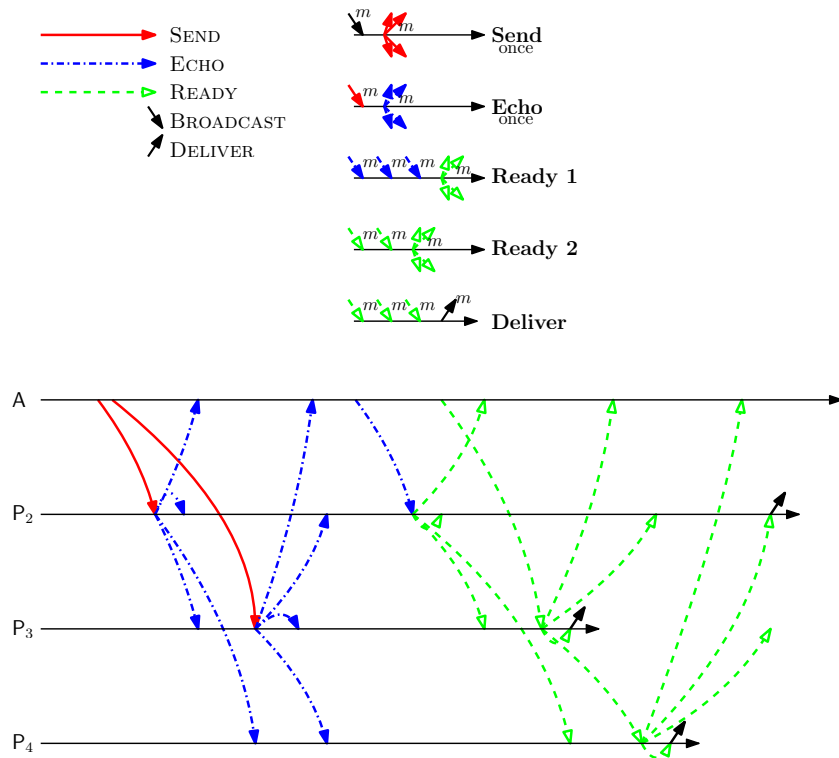


Figure 10.9: Example execution of Bracha Broadcast with a corrupted P_1 . It illustrates why the **Ready 2** rule is needed for Termination 2. We see that P_3 terminates after getting three READY messages, the last of these messages being sent by itself after executing the **Ready 2 rule**. However, one of the three messages was sent by the corrupted sender, which did not send it to the other parties. But since P_3 triggers **Ready 2** the other parties will also eventually get two READY messages and then executed **Ready 2** to get a third READY message and terminate.

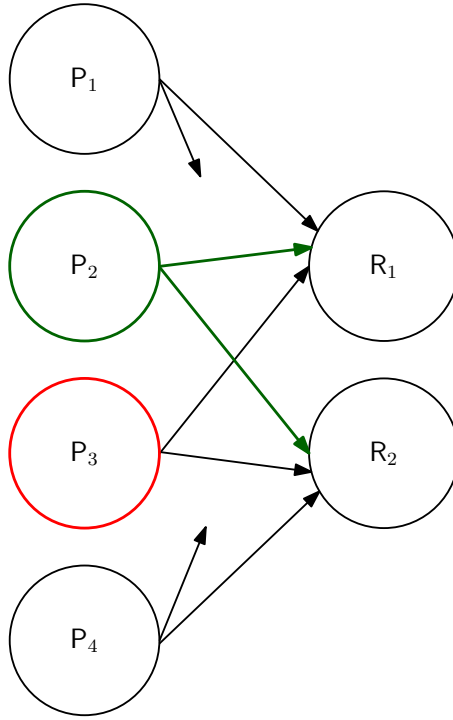


Figure 10.10: With four parties and at most one corrupted party, receivers can wait for three messages and be guaranteed to have heard from at least one common honest party, here P_2 . One cannot do with less than four, as different messages can be slow towards different receivers, so they cannot be guaranteed to hear from the same three parties. And they might be unlucky that the corrupted party is among the two common parties they hear from.

messages, then the last one might never arrive. And remember that in the asynchronous model, there is no way to distinguish a message that was not sent from a message that was sent by an honest process but is just very slow. So you can never wait for more than $n - t$ messages. In the above protocol the parties wait for the same message being echoed from $n - t$ parties to ensure that they all received the same m , as they should. We would of course have liked to check that all n did so, but we cannot wait for n messages.

Hearing from an Honest Party If you wait for messages from $t + 1$ parties, then you are essentially only ensuring that you hear from at least one honest party. There are at most t corrupt parties, so if you get $t + 1$ messages from distinct parties, at least one of them was sent by a correct one. In the above protocol this is used to learn that some honest party sent $(\text{READY}, P_1, \text{bid}, m)$, from which we conclude that some correct process saw the same m from $n - t$ other parties. We will see below how that is used.

Hearing from a Common Honest Party Why the $n > 3t$ condition? Well, if two different parties P_1 and P_2 each hear from $n - t$ parties, then the number of parties they both heard from might be as low as $n - 2t$: it could for instance be that P_1 heard from $P_1, \dots, P_{n-t}$ and that P_2 heard from $P_{t+1}, \dots, P_n$, giving the shared “informants” $P_{t+1}, \dots, P_{n-t}$, and if $n = 3t$, then $n - t - (t + 1) + 1 = n - 2t$. Out of these $n - 2t$ there might be t parties that are corrupted. So, the number of honest parties they both heard from might be as low as $n - 3t$. So, we need $n > 3t$ to guarantee that there is at least one shared honest party they both heard from. This is illustrated in Fig. 10.10.

It turns out that if P_1 and P_2 never hear from a common honest party, then it might be that information never was able to pass between P_1 and P_2 , and then they cannot accomplish agreement. They are basically Partitioned, so they cannot be both Available and Consistency, in the words of CAP Theorems. We argued this for $n = 3$ and $t = 1$ above. It will be an exercise to generalize to $n = 3t$.

Running Forever? A particular thing to notice about Bracha Broadcast is that if, for instance, P_1 is corrupt and does not send its message, then the correct parties run forever. The reason is that they will forever be waiting in *Echo*. And there is no way they can avoid this. They cannot distinguish a message not sent by a corrupt party from a message sent by an honest party that was delayed indefinitely. So if they terminate before getting something from P_1 , they will jeopardise the safety of the protocol when P_1 is honest but “infinitely slow”. There was a more elaborate discussion of this Waiting for Godot problem above.

10.5.1 Analysing Bracha Broadcast

We now proceed to analyse the protocol.

Theorem 10.1 *Bracha Broadcast securely implements ARB given AC when there are $t < n/3$ Byzantine corruptions.*

PROOF For the sake of the proof we assume exactly t parties are corrupted and $n = 3t + 1$. So there are $2t + 1$ honest parties. For less corruptions the proof follows similarly.

We first argue Termination 1. We have to argue that if P_1 is honest and broadcast, then all parties will eventually make an output. If P_1 gets input $(\text{BROADCAST}, P_1, \text{bid}, m)$ it sends $(\text{SEND}, P_1, \text{bid}, m)$ to all parties. The $n - t$ honest parties will eventually get the message and send $(\text{ECHO}, P_1, \text{bid}, m)$ to all parties. Therefore all $n - t$ honest parties will get $n - t$ such messages and send $(\text{READY}, P_1, \text{bid}, m)$ to all parties if they did not do so already. Therefore all honest parties eventually get $n - t$ such messages and output $(\text{DELIVER}, P_1, \text{bid}, m)$. This is illustrated in Fig. 10.11.

For arguing Agreement and Validity we use the following crucial observation: if an honest P_i outputs $(\text{DELIVER}, P_1, \text{bid}, m_i)$, then $t + 1$ honest parties sent $(\text{ECHO}, P_1, \text{bid}, m_i)$.

The argument goes as follows. We assumed that P_i outputs $(\text{DELIVER}, P_1, \text{bid}, m_i)$. Inspecting the protocol we see that then P_i must have received $(\text{READY}, P_1, \text{bid}, m_i)$ from $n - t$ parties. We have that $n > 3t$, so $n - t > 2t$. So, P_i must have received $(\text{READY}, P_1, \text{bid}, m_i)$ from at least one honest party. So, some honest party sent $(\text{READY}, P_1, \text{bid}, m_i)$. But then there is also a *first* honest party to send $(\text{READY}, P_1, \text{bid}, m_i)$. This was done in the rule **Ready 1**. Namely, if an honest party P_c sends $(\text{READY}, P_1, \text{bid}, m_i)$ in **Ready 2**, then it saw $(\text{READY}, P_1, \text{bid}, m_i)$ sent by $t + 1$ parties. There are only t corrupted parties. Ergo, at least one honest party sent $(\text{READY}, P_1, \text{bid}, m_i)$ before P_c . So, P_c was not first. So, we know some honest party sent $(\text{READY}, P_1, \text{bid}, m_i)$ in **Ready 1**. But then it saw $n - t$ parties send $(\text{ECHO}, P_1, \text{bid}, m_i)$. So, at least $(n - t) - t \geq t + 1$ honest parties sent $(\text{ECHO}, P_1, \text{bid}, m_i)$ as we claimed. This is illustrated in Fig. 10.12.

We then argue Validity. Assume that P_1 is honest and some honest P_i outputs $(\text{DELIVER}, P_1, \text{bid}, m_i)$. By the crucial observation $t + 1$ honest parties sent $(\text{ECHO}, P_1, \text{bid}, m_i)$. They only do that if P_1 sent $(\text{SEND}, P_1, \text{bid}, m_i)$, which means P_1 had input $(\text{BROADCAST}, P_1, \text{bid}, m_i)$. This is illustrated in Fig. 10.12.

We then argue Agreement. Assume that P_i outputs $(\text{DELIVER}, P_1, \text{bid}, m_i)$ and P_j outputs $(\text{DELIVER}, P_1, \text{bid}, m_j)$. Using the crucial observation twice we get that $t + 1$ honest parties

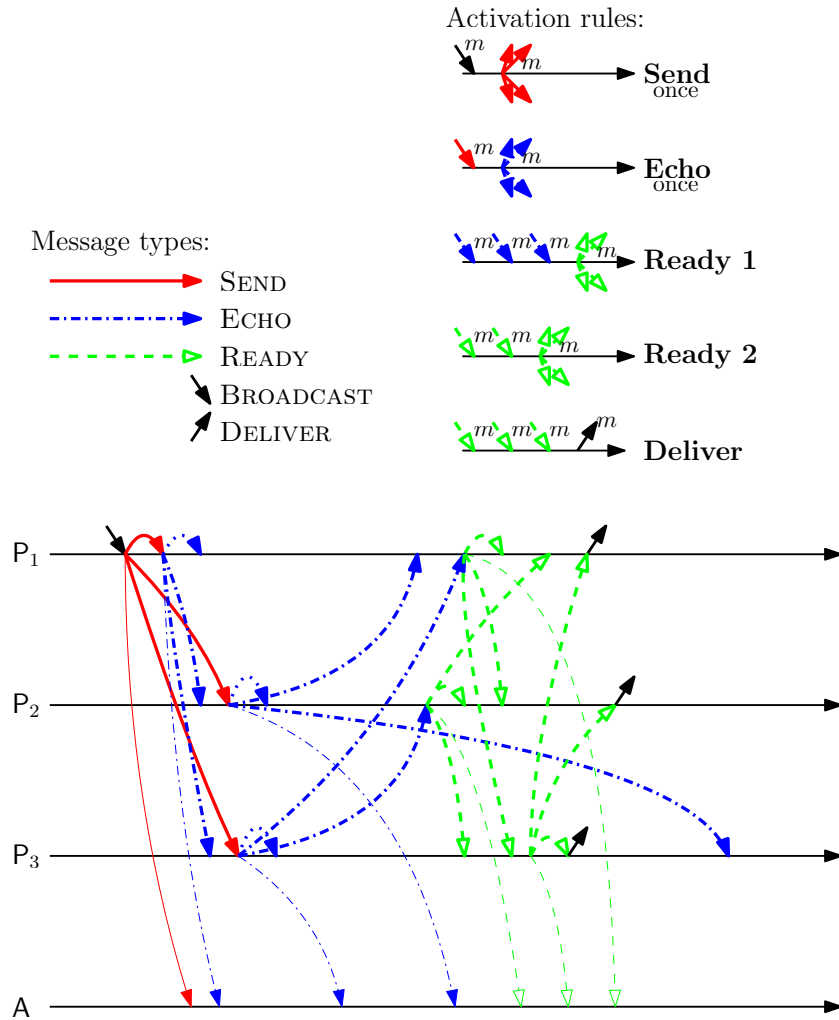


Figure 10.11: Example execution of Bracha Broadcast with a corrupted P_4 . It illustrates how the protocol always terminated even if t parties are corrupted. All honest parties get the SEND message. They all send ECHO. So all parties get three of these and send READY in **Ready 1** unless they already did this in **Ready 2** like P_3 . So all three honest parties send READY. So they all get three READY and terminate.

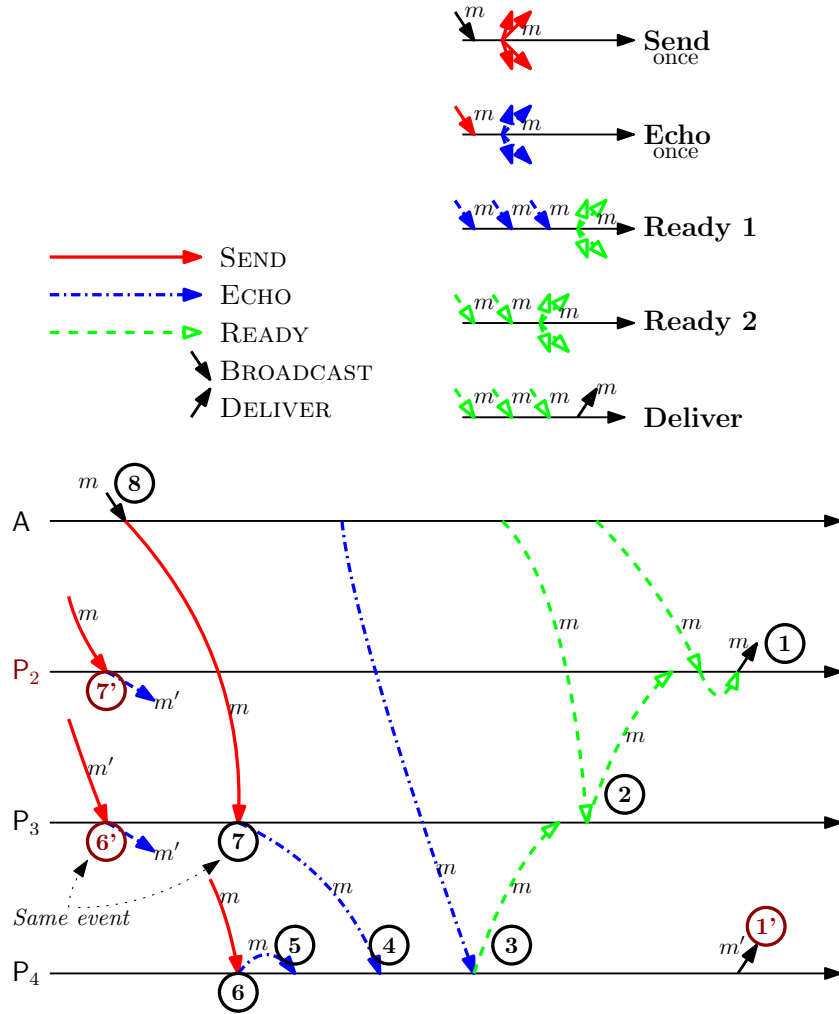


Figure 10.12: Illustration of the crucial observation for Agreement and Validity in Bracha Broadcast. The argument works backwards from event (1): if an honest party outputs a value m , it must have received 3 READY messages for m , so one must be from an honest party. This honest party may have executed **Ready 2**, like P_3 at (2), but there must be a first honest party to send READY for m , here P_4 at (3). This party must have sent it in **Ready 1**, so it received 3 ECHO messages for m , here at (3), (4) and (5). Therefore, at least 2 honest parties sent ECHO for m , here at (6) and (7). For Validity, since P_3 sent ECHO for m , it must have received SEND from P_1 . But then P_1 sent m , giving Validity. For Agreement, assume two honest parties output m and m' , here at (1) and (1'). Then the crucial argument gives us that two honest parties sent ECHO for m (here at (6) and (7)) and that two parties sent ECHO for m' (here at (6') and (7')). Since there are only three honest parties, one of them sent both m and m' , here P_3 at (6') and (7). But honest parties only trigger **Echo** once in the same broadcast, so (6') and (7) is the same event, so m and m' was sent in the same **Echo** event. This implies that $m' = m$ as P_3 is honest and **Echo** instructs to send identical messages to all parties.

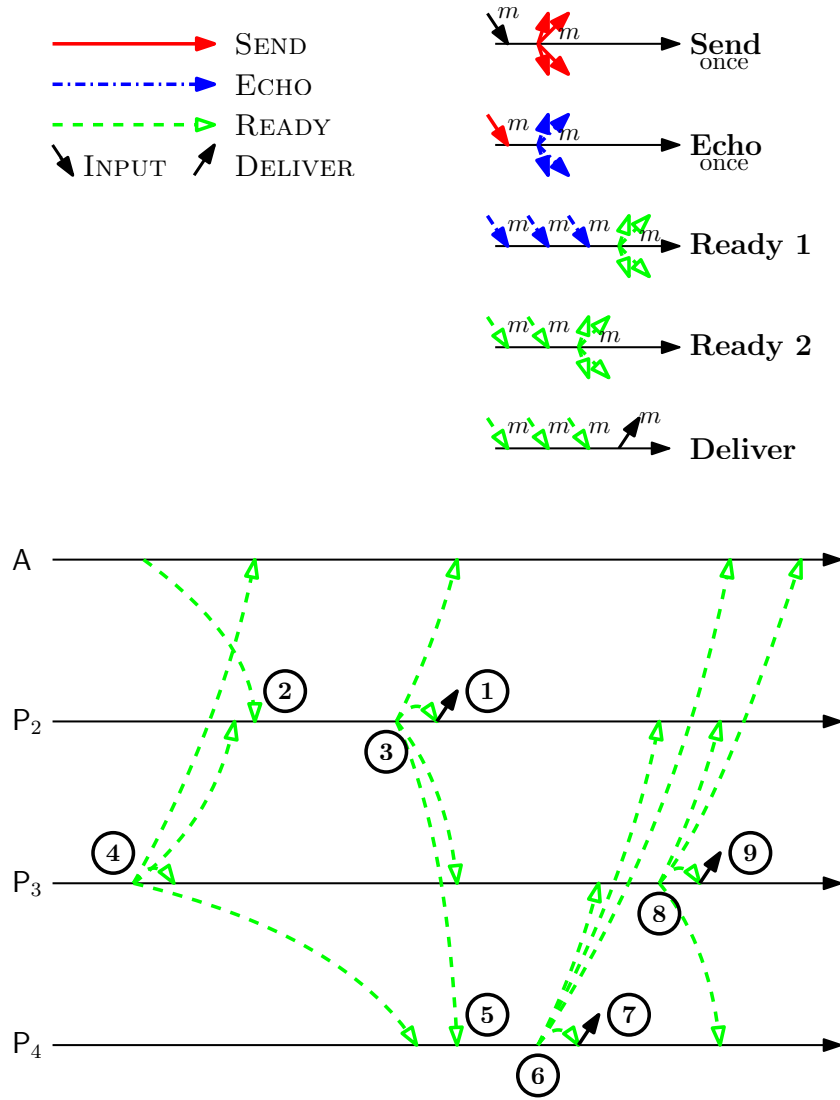


Figure 10.13: An illustration of the argument for Bracha Broadcast having Termination 2. Here the sender is corrupted and P_2, P_3, P_4 are honest. Assume that some honest party had output, say P_2 . This happens as (1). We need to argue that then the two other honest parties had output too. We reason backwards. From P_2 having output we know that it applied **Deliver**. So it received 3 READY. From that we can conclude a place where it received 2 READY, see (2). When having received 2 READY the party can apply **Ready 2**. So we conclude that P_2 did so. This is (3). Furthermore, since P_2 received 2 READY messages at (2) and there is at most one corrupted party, we know that P_2 received one of the messages from an honest party. In this case it is P_3 . Since P_3 is honest we know that when it sent the message to P_2 it sent it to all parties. This is (4). So now we have argued that there are two spots in which an honest parties sends READY to all honest parties, namely (3) and (4). These message will eventually arrive at P_4 , here at spot (5), which then runs **Ready 2**, here at spot (6). When it receives its own READY messages it has seen 3 and runs **Deliver**, spot (7). Similarly, the two READY messages sent at (3) and (4) will eventually arrive at P_3 , which then runs **Ready 2**, here at spot (8). When it receives its own READY messages it has seen 3 and runs **Deliver**, here at spot (9). Now all parties terminated.

sent $(\text{ECHO}, P_1, \text{bid}, m_i)$ and that that $t + 1$ honest parties sent $(\text{ECHO}, P_1, \text{bid}, m_j)$. Since $(t+1)+(t+1) = 2t+2$ and there are only $2t+1$ honest parties we conclude that at least one honest party both sent $(\text{ECHO}, P_1, \text{bid}, m_i)$ and $(\text{ECHO}, P_1, \text{bid}, m_j)$. Since honest parties only send something of the form $(\text{ECHO}, P_1, \text{bid}, \cdot)$ we get that $(\text{ECHO}, P_1, \text{bid}, m_i) = (\text{ECHO}, P_1, \text{bid}, m_j)$. Ergo, $m_i = m_j$. This is illustrated in Fig. 10.12.

We finally argue Termination 2. We have to argue that if some honest party terminates then all honest parties eventually terminate, even if P_1 is corrupt. The above cases were fairly simple. They did not use all the details of the protocol. This will happen now. The reason this is harder than it looks is that P_1 might be corrupted. So we start from the assumption that a correct P_i outputs $(\text{DELIVER}, P_1, \text{bid}, m')$ (we now write m' since, as P_1 is corrupted, we do not know what the “real” input of P_1 was). Then we have to analyse why this could happen. Then we need to argue that in those cases it would force any other correct P_j to also output $(\text{DELIVER}, P_1, \text{bid}, m')$. So, assume that some P_i outputs $(\text{DELIVER}, P_1, \text{bid}, m')$. Inspecting the protocol we see that then P_i must have received $(\text{READY}, P_1, \text{bid}, m')$ from $n - t$ parties. We have that $n > 3t$, so $n - t > 2t$. So P_i must have received $(\text{READY}, P_1, \text{bid}, m')$ from at least $2t + 1$ parties. There are at most t corrupted parties, so we can conclude that P_i must have received $(\text{READY}, P_1, \text{bid}, m')$ from at least $t + 1$ correct parties. By inspection of the code we see that when these $t + 1$ parties sent $(\text{READY}, P_1, \text{bid}, m')$ to P_i , they also sent $(\text{READY}, P_1, \text{bid}, m')$ to all other parties. Therefore all correct parties will eventually receive $t + 1$ messages of the form $(\text{READY}, P_1, \text{bid}, m')$. Hence they will all trigger Ready 2 and send $(\text{READY}, P_1, \text{bid}, m')$ to all other parties. Now *all* of the $n - t$ honest parties sent $(\text{READY}, P_1, \text{bid}, m')$ to all honest parties. Therefore all honest parties will eventually receive $n - t$ messages of the form $(\text{READY}, P_1, \text{bid}, m')$ and output $(\text{DELIVER}, P_1, \text{bid}, m')$. What a beautiful protocol!

□

The above protocol tolerates $t < n/3$ Byzantine corruptions. This can be seen to be optimal as it is optimal already in synchronous networks. However, it turns out that the bound holds in a much stronger sense in asynchronous networks. The following exercise asks you to explore this.

10.6 Defining Asynchronous Byzantine Agreement

We now move on to study asynchronous Byzantine agreement. In Byzantine agreement each party P_i gets an input (VOTE, v_i) , where $v_i \in \{0, 1\}$ is called the **vote** of party P_i . At some point each P_i will have to output $(\text{DECISION}, d_i)$ where $d_i \in \{0, 1\}$ is called the **decision** of party P_i . We want that the parties make the same decision. And we want that if all honest parties vote for the same bit, then that must be the decision that is made. Another way to phrase that is to say that the decision must be on a bit that some honest party voted for. A bit more formally:

Validity: If some honest P_i gives an output $(\text{DECISION}, d)$, then at some earlier point some honest P_j had the input (VOTE, d) .

Agreement: If some honest P_i and P_j output $(\text{DECISION}, d_i)$ and $(\text{DECISION}, d_j)$, then $d_i = d_j$.

Termination: If all honest parties give an input of the form $(\text{VOTE}, \cdot)$, then eventually all honest parties give an output of the form $(\text{DECISION}, \cdot)$.

As for ARB we also use identifiers, they are now called **baid** for BA identifier. It is for identifying different runs of the protocol. They were suppressed in the above definition, as we trust the readers can tacitly add them in their heads by now. If we want to be more formal we can also define asynchronous Byzantine agreement via the ideal functionality **ABA** in Fig. 10.14.

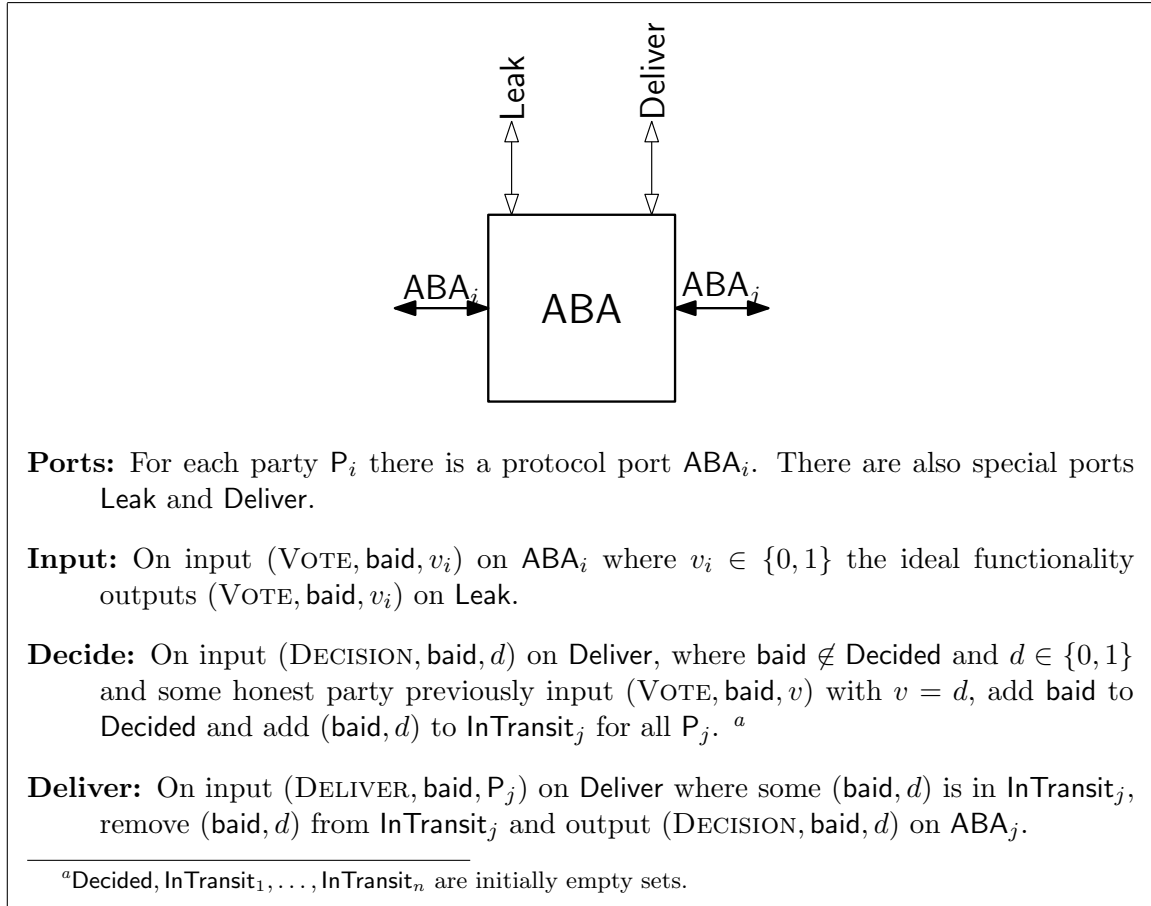


Figure 10.14: An ideal functionality ABA for Asynchronous Byzantine Agreement.

10.7 Implementing Asynchronous Byzantine Agreement using Coin-Flip

We now present a protocol for Asynchronous Byzantine Agreement (ABA). We call it SVUP, for stabilizing, vote-updating protocol. It might look a bit odd at first sight, so to demystify it we will present the protocol in the style of letting the reader “rediscovering” it for themselves for the case with $n = 4$ parties and $t = 1$ corruption.

10.7.1 The Reader is Visited by the Goddess of Chance

The reader will get a bit of help from the Greek goddess of chance, and why exactly this goddess took the job shall become clear along the way.

TYCHE: What is the problem?

THE READER: Each party has a vote, either 0 or 1. We want them to decide on the same bit and the decision must be on a bit that some honest party voted for.

TYCHE: Can they decide without seeing each other’s votes?

THE READER: I guess not!

TYCHE: What to do first then?

THE READER: Well, let us start by having everyone send their votes to everyone else.

TYCHE: How to send the votes?

THE READER: We could use ARB such that the corrupted party cannot send different votes to different parties?

TYCHE: Good idea, and then what? Do we ask the parties to wait for all four votes?

THE READER: No, there is the Godot problem.

TYCHE: The what?

THE READER: If one of the parties is corrupted, then it might never send its vote. So if we wait for all four votes, then we might wait forever and violate Termination.

TYCHE: So what to do?

THE READER: All parties wait for three votes. This is the best we can do.

TYCHE: And then how do you compute the decision based on the three votes?

THE READER: Well if all three votes are the same, then we can just decide on that—

TYCHE: Why?

THE READER: We have to make a decision now, might never hear from more parties. If vote 0, then we can hardly go for 1 as decision!?!

TYCHE: Sounds right. And if some honest parties hears 0, 0, 0 is it guaranteed that they all do so?

THE READER: —No?—

THE READER: —No, it could be that the parties P_1, P_2, P_3, P_4 had inputs 0, 0, 0, 1 respectively, and the adversary could schedule such that P_1 hears from P_1, P_2, P_3 first and P_4 hears from P_2, P_3, P_4 first. Then P_1 hears 0, 0, 0 and decides 0, but P_4 hears 0, 0, 1.

TYCHE: Is that a problem?

THE READER: Well, the parties have to decide on the same bit. So if P_1 decides 0, then P_4 must also decide 0. So I guess that if parties receive two votes for 0 and one vote for 1, then they must also decide 0.

TYCHE: And if parties receive two votes for 1 and one vote for 0?

THE READER: Well, then they decide 1. It could be that the parties P_1, P_2, P_3, P_4 had inputs 0, 1, 1, 1 respectively, and the adversary scheduled such that P_4 hears from P_2, P_3, P_4 first and decide 1, and P_1 hears from P_1, P_2, P_3 first, and hears 0, 1, 1. So, P_1 needs to go with 1 as well.

TYCHE: So, what is the rule?

THE READER: Decide on the bit getting at least two out of three votes.

TYCHE: And that always gives us agreement?

THE READER: I think so!

TYCHE: What happens if the inputs are 1, 1, 1, 0.

THE READER: They all decide 1! No party can see two votes for 0, and they will all see two votes for 1.

TYCHE: And inputs 1, 1, 1, 1.

THE READER: Even better, they all decide 1.

TYCHE: And 0, 0, 0, 0 or 0, 0, 0, 1?

THE READER: Well, no party can see two votes for 1, and will eventually get two for 0, so they all decide 0.

TYCHE: Is there any case left?

THE READER: Well, I got the feeling you were waiting with the case 0, 0, 1, 1.

TYCHE: Does your rule work here?—

THE READER: —no, I guess some parties might see two votes for 0 and some might see two votes for 1.

TYCHE: You guess?

THE READER: I know! Parties P_1, P_2 could hear from P_1, P_2, P_3 first and P_3, P_4 could hear from P_2, P_3, P_4 first. Then P_1, P_2 will decide 0 as they see two votes for 0 and one vote for 1. P_3, P_4 will decide 1 as they see two votes for 1 and one vote for 0.

(pause)

THE READER: This unbiased case is annoying!

TYCHE: What about trying again from the updated votes?

THE READER: Like, run the whole thing again, but now with the votes being the decisions from the previous round?

TYCHE: Yes!

THE READER: Well, the decisions are 0, 0, 1, 1. We are back where we started!

TYCHE: Would another rule work here? The problem is that with biased inputs like 0, 0, 1, 1 some could hear 0, 0, 1 and some could hear 0, 1, 1. What about deciding 0 in both cases?

THE READER: So 0, 0, 0 goes to 0 and 1, 1, 1 goes to 1. And in both mixed cases we decide 0?

TYCHE: Yes!

THE READER: ... Well, then it works for 0, 0, 1, 1, but now input 0, 1, 1, 1 is the troublesome case. Party P_1 could hear 0, 1, 1 and P_4 could hear 1, 1, 1. Then P_1 would decide 0 and P_4 would decide 1.

TYCHE: So no rule works?

THE READER: No, it seems that for each rule there is a troublesome input and a nasty way to schedule such that agreement is violated.

TYCHE: This is so. At least for all fixed rules. But chance is your friend!

THE READER: Eh?

TYCHE: Chance is your friend!—

TYCHE: —Try this rule: if you see three votes for the same bit, then decide on that bit. Otherwise you pick your updated vote at random.—

THE READER: At random?

TYCHE: —fifty-fifty between 0 and 1.

THE READER: Now you are just being stupid!

TYCHE: Careful mortal!!

THE READER: Sorry...

THE READER: But how will picking at random help?

TYCHE: For all fixed rules the adversary were making you run in an infinite loop by scheduling. Try the random rule with inputs 0, 0, 1, 1 and try to schedule in a bad way. I will flip the coins.

THE READER: Well, I let P_1 and P_2 hear from P_1, P_2, P_3 first and P_3 and P_4 hear from P_2, P_3, P_4 first.

TYCHE: Good, so P_1 and P_2 see 0, 0, 1 and P_3 and P_4 see 0, 1, 1. They all flip a coin—wait, let me get my coins out—

TYCHE: So, P_1 flipped 0, P_2 flipped 1, P_3 flipped 0 and P_4 flipped 0.

THE READER: So, they decide 0, 1, 0, 0?

TYCHE: Are you happy?

THE READER: No, I am not happy, they still do not agree!

TYCHE: Above, when we talked about just trying again, why did that not work?

THE READER: Well, we started from the inputs 0, 0, 1, 1 and we ended up with the same votes 0, 0, 1, 1. So we were back where we started.

(pause)

THE READER: Oh, we are no longer back where we started, we are at 0, 1, 0, 0!

TYCHE: What happens if we try again from 0, 1, 0, 0?

THE READER: Well, they all ARB their new votes. All parties will collect at least two votes on 0 and update their vote to 0. So, they end up with updated votes 0, 0, 0, 0.

TYCHE: Tada!

THE READER: But what if they had flipped 0, 0, 1, 1 instead of 0, 1, 0, 0?

TYCHE: They try again.

THE READER: And if they flip 0, 0, 1, 1 again forever?

TYCHE: That will not happen, they will eventually flip biased votes.

THE READER: Says who?

TYCHE: Says I!

THE READER: Okaay...

TYCHE: Mortal, I am the Goddess of Chance, I am the one who decides when you get biased coins. I might have my fun with you and give you unbiased votes for a while, but eventually I will give you biased votes, I promise you that.

THE READER: So, are we done?

TYCHE: When do you give outputs?

THE READER: Oh, I guess we can give outputs when we see three votes for the same bit.

TYCHE: Well, now let me schedule. Inputs are 1, 1, 1, 0. I let P_1 hear from P_1, P_2, P_3 first and P_2, P_3, P_4 hear from P_2, P_3, P_4 first. They all flip coins. They all flip 0. What happens?

THE READER: Well P_1 sees 1, 1, 1 and decides 1. The others see 1, 1, 0, flip 0, and decide 0.

TYCHE: So, they do not agree?

THE READER: No, they do not agree. But in the next round they all see two votes for 0 all update their votes 0. Then they all see 0, 0, 0 and output 0.

TYCHE: But you said that P_1 when it sees 1, 1, 1 then it outputs 1. So P_1 already outputted 1 and terminated.

THE READER: Oh crap!

TYCHE: What went wrong?

THE READER: Well, we had the rule that 0, 1, 1 goes to 1 to avoid this. Because then a party with 1, 1, 1 would know that all other parties update to vote 1. But now a party with 0, 1, 1 might decide 0 using a coin-flip.

TYCHE: What to do?

THE READER: Can't you just tell me by now, I'm getting tired of this!

TYCHE: Offerings are a nice way to say please!

(The reader pours honey and wine on the ground and offers a few coins.)

TYCHE: Nice, I like that. Ok, here is the deal. You do alternating rounds of fixed rules and random rules. In the fixed rounds you use the rule that 0, 1, 1 goes to 1 and 0, 0, 1 goes to 0. In these rounds, if you see 1, 1, 1 or 0, 0, 0 you decide on that and give output. This works as every one follows your decision using the fixed rule.—

THE READER: —ah, and when I use the random rule, if I see 1, 1, 1 or 0, 0, 0 I stay with that vote but do *not* give output!

TYCHE: Yes, exactly. In the random rounds you do not give output, you just update your vote.

THE READER: —and if you see 0, 1, 1 or 0, 0, 1 you flip a coin and decide on the outcome of the coin-flip—

TYCHE: —yes—

THE READER: —and run another round to see if we got lucky!

TYCHE: Yes, exactly.

THE READER: This will sometimes bias the votes towards 0 or 1 and then the next fixed round will detect this.

TYCHE: Exactly, chance is your friend!

THE READER: Thanks! Very nice ideas!

TYCHE: Well, they are not mine.

THE READER: No?

TYCHE: No, the SVUP protocol is strongly inspired by the protocol in the paper [8] by Feldmann and Micali from 1988. Using randomness for SVUPs was first proposed by Michael Ben-Or[1] in 1983. I whisper by his ear though...

THE READER: OK.

TYCHE: And you are not done...

THE READER: No?!?

TYCHE: You assumed above that corrupted parties played nice when updating their votes. But they might not do that. What if a corrupted party seeing 1, 1, 0 in a fixed round lies and votes 0 instead of 1? And then schedules its own vote to arrive first. It would ensure that no party ever sees three votes for the same bit and thus never gives output.

THE READER: Oh, crap!

(*pause*)

THE READER: You want more honey and wine, right?

TYCHE: No, I will let you solve this one. Think back to the chapter on causal communication. The two votes on 1 that a party must see in the fixed round to be justified to vote 1, they are in the *causal past* of the updated vote for 1, right?

THE READER: Yes?!

TYCHE: If the communication was causal, how would you catch the cheating adversary above who saw 1, 1, 0 and then voted 0?

THE READER: Hmm, give me a moment to think about this...

In Fig. 10.15 there is an illustration of why just taking majority of the three first votes received will violate agreement. In Fig. 10.16 there is an illustration of why picking random votes is needed. In Fig. 10.17 there is an illustration of why picking random votes will not violate validity.

10.8 SVUP: Stabilizing Vote Updating Protocol, $n = 4$, $t = 1$

We now sketch the protocol for $n = 4$ parties and $t = 1$ corruption. We will then generalize it to arbitrary n and t . The protocol is given in Fig. 10.22. We first explain the notation of the figure and then explain the protocol. We go through the graphical notation from top to bottom, left to right.

There are two types of rounds, majority rounds and random rounds. There are correspondingly two types of votes, MVOTES, which are the results of the majority voting rule, where for instance 0, 0, 1 goes to 0, and RVOTES, which are the results of the random voting rule, where 0, 0, 1 goes to a coin-flip. RVOTES will have a single arrow head and MVOTES will have a double arrow head in the figure (symbolizing the two RVOTES “sent along” to justify them). Blue will indicate 0 and orange will indicate 1. Coins have a dollar sign on them. A coin-flip where the value does not matter is indicated by a gray coin. A coin not flipped yet is indicated by half blue and half orange. One majority round of voting followed by one random round is called an **attempt** (at stabilizing the votes) and is indexed by a . An attempt includes the initial MVOTES and the following RVOTES. If we talk about the updated votes of attempt a we mean the RVOTES of attempt $a + 1$.

An attempt where the RVOTES start out unbiased is called unstable, as the next round of RVOTES might end up being anything depending on scheduling. This is illustrated in Fig. 10.20. The inputs start out as 0, 1, 1, 0. Then at (1) to (4) the RVOTES are made 0, 0, 1, 1 by the

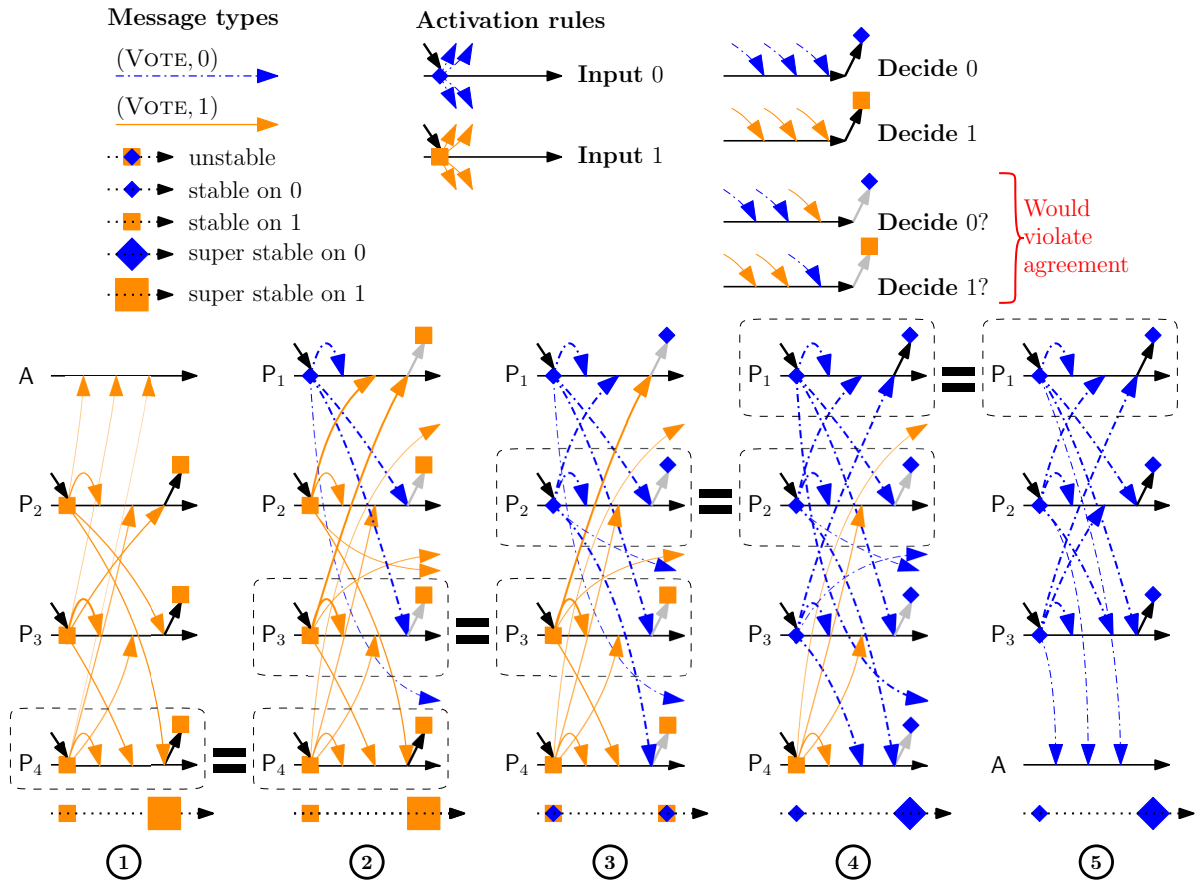


Figure 10.15: Illustration of why a simple majority voting will not do the job. To the left: all honest parties have input 1 and the adversarial P_1 is crashed. This is optimal, as all honest parties wait to hear from all honest parties. They only hear 1-votes. In this scenario they must vote 1, when else? Second timeline from the left: from the view point of P_4 it could also have happened that P_1 was honest and had input 0, but P_4 just happened to hear from the other parties first. It has the same view, so there is no way it can do anything different. Therefore it outputs 1. The other parties hear from P_1 first and therefore have two votes for 1 and one for 0. So, to maintain agreement they must votes 1 in that case too, as P_4 did. Similarly, starting from the right, if all honest parties have input 0 and P_4 is crashed all honest parties must output 0. Therefore P_1 will also do this if P_4 is honest with input 1, but P_1 does not hear from P_4 before it must vote. To keep agreement the three parties hearing from P_4 and hearing two times 0 and one times 1 must vote 0 too. But we have the problem that P_2 does not hear from P_3 in timeline (4). So, if we change the input of P_3 to 1 as in timeline (3), then P_2 has the same view as in timeline (4). Ergo it produces the same output: 0. Similarly, P_3 does not hear from P_2 in timeline (2), so we can change the input of P_2 to 0 to get timeline (3). Having the same view P_3 still outputs 1. But now we see that majority voting is problematic. It works fine in timelines like (1), (2), (4) and (5), where the inputs are stable (three parties have the same vote). There it ensures that *all* parties have the same updated vote, as no party can get two votes for the other bit. We call this super-stable. But for unstable inputs like (3), where no bit has three votes, the majority rule can produce any possible mix of outputs depending on scheduling, hence the name unstable.

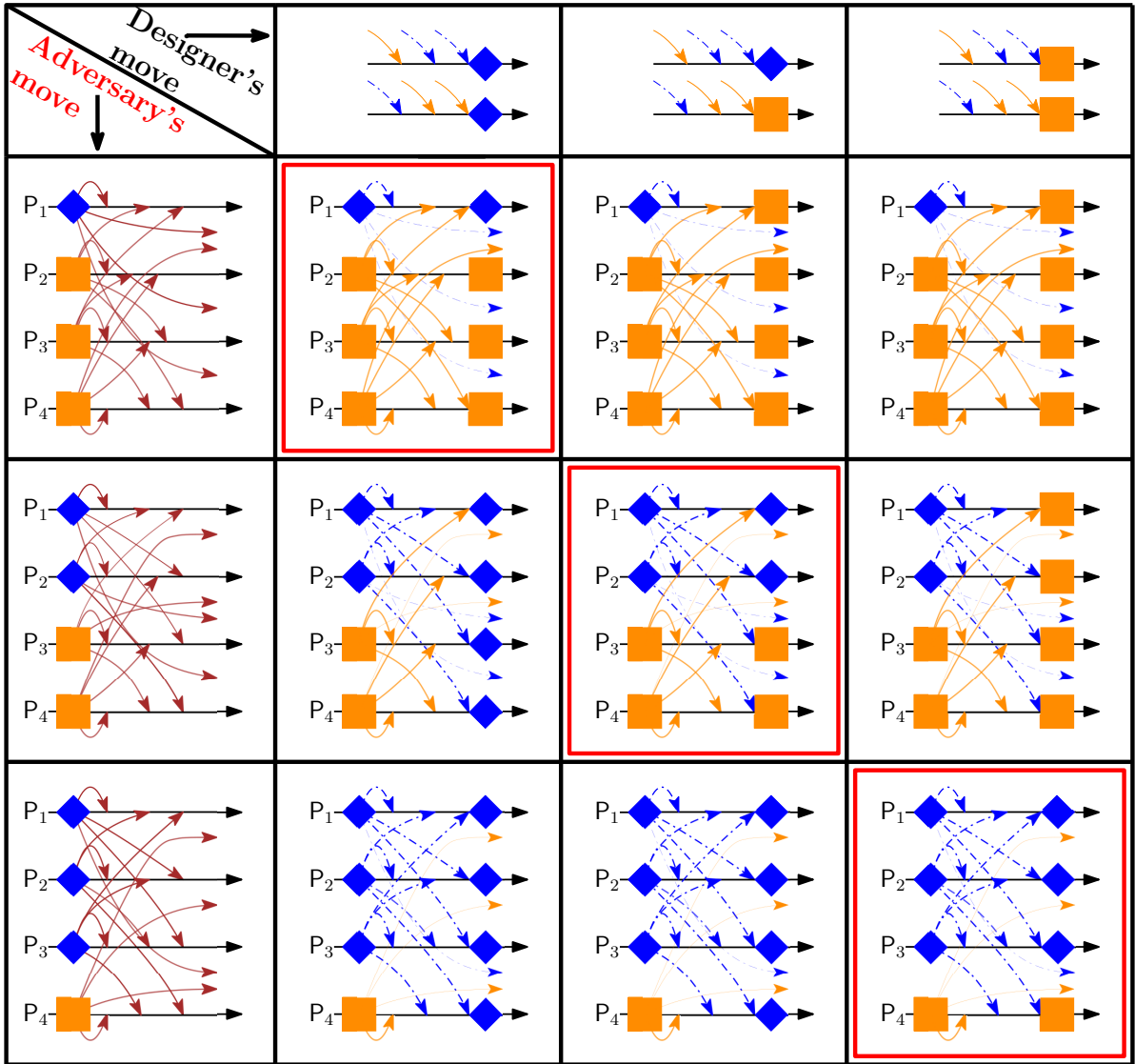


Figure 10.16: Designing an ABA protocol can be seen as a game between the protocol designer and the adversary. The protocol designer picks a rule for updating mixed votes. Here either “always 0”, “majority”, or “always 1”. This is the top row. The adversary picks the initial votes and the schedule. This is the left-most column. For every rule there is a setting of inputs and schedule such that the updated votes do not become super-stable. This is the diagonal. For all other combinations the protocol designer “wins”: the updated votes are super-stable. If the protocol designer picks a fixed rule the adversary can always win. This is like always playing Rock in Rock-Scissors-Paper. When we play Rock-Scissors-Paper we play a random move to not be exploited like this. Similarly, the protocol designer needs to use a random rule picked *after* the adversary scheduled. Then they will sometimes win.

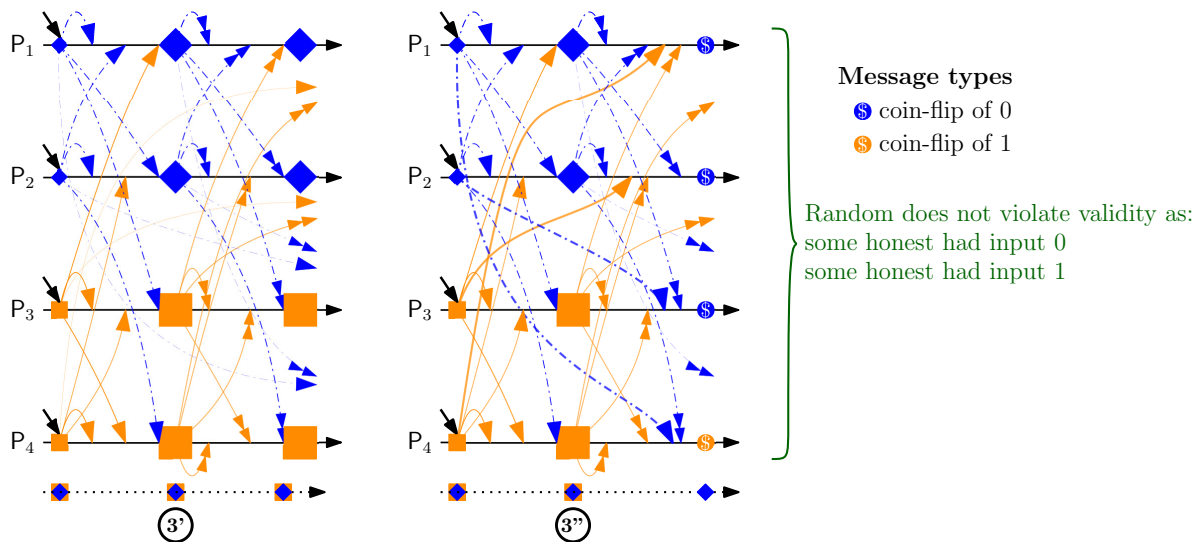


Figure 10.17: To the left, an illustration why we cannot deterministically get out of an unstable state by majority voting, even when all parties are honest as assumed here. The adversary can keep using the same scheduling to keep us unstable. To the right, an illustration how randomness can help. Instead of using the majority voting rule in the new round, the parties just pick a uniformly random vote, fifty-fifty 0 or 1. With a good probability this will give a stable setting of the votes: it does not always happen that exactly two parties vote 0 and two vote 1. Voting at random might seem crazy: what about Validity? No worries, a party like P₁ who sees a second-round vote for 0 knows that two parties had input 0, or P₁ would not have voted 0 in the second round. So, since two parties voted 0 and there is at most one corrupted party, at least one honest party must have voted 0. A party like P₁ who *also* sees a second-round vote for 1 also knows that two parties had original input 1, or P₃ would not have voted 1 in the second round. So, at least one honest party voted 1. Validity requires that the final decision will be some bit an honest party had as input. But both 0 and 1 were inputs of some honest parties, so the decision can be whatever. What about Agreement then? Here P₄ had a minority vote of 1 after the coin-flipping. The trick is to run the whole thing again. After the random voting, use the votes as input to a new round of majority voting as in (4) in Fig. 10.15. After this the votes becomes super-stable. See Fig. 10.18 of how the above behaves if a party is corrupted.

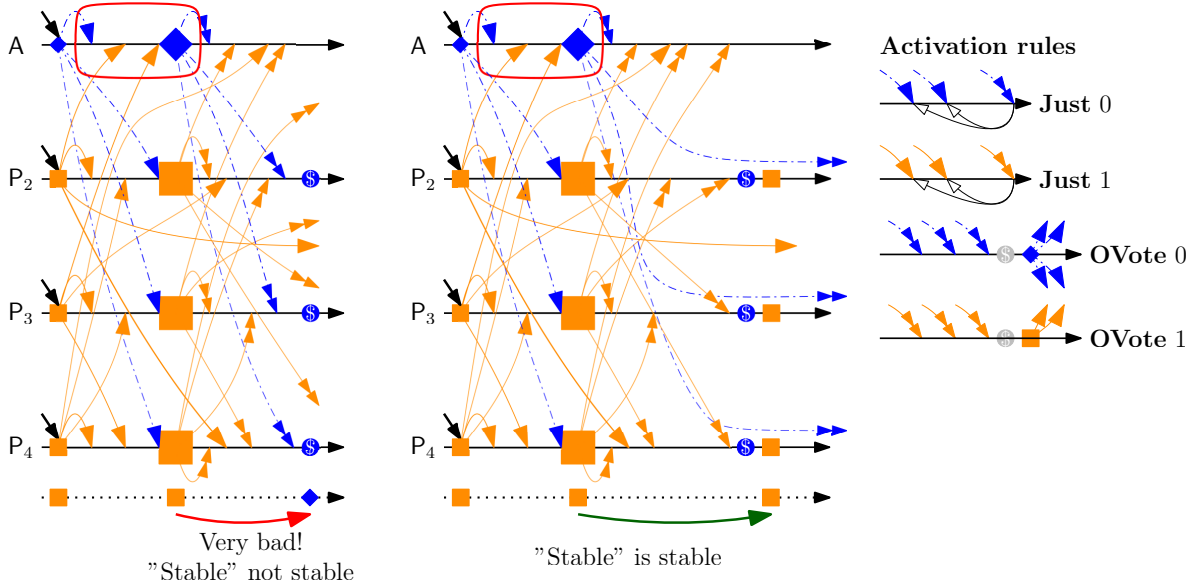


Figure 10.18: In Fig. 10.17 it was assumed all parties were honest, so no party would lie in their second-round votes. This allowed us to conclude from seeing a mixed second-round vote that both 0 and 1 were inputs of honest parties. Here we illustrate how to handle a corrupted party. To the left: a corrupted party votes 0 after having seen two votes for 1 in a stable network—like (2) in Fig. 10.15. This makes the honest parties see a mixed second-round voting and tricks them into voting at random, when they should actually stick to their 1-vote. To avoid this we make the Justification rule, **Just b** , which says that to accept a second-round vote for b you must first have seen two first-round votes for b . This prevents corrupted parties from fake votes accepted as genuine. Notice how the second-round vote of 0 from P_1 is never received as the condition for receiving it is not fulfilled. This is an instance of causal communication, as discussed in Section 4.4, where we insist on seeing the messages having caused the 0-vote before receiving it. We also make the overwrite voting rule, **OVote b** , saying that on seeing three identical second-round votes, go for that vote no matter the coin-flip. Notice how all parties update their votes to 1 despite their coins landing on 0. Together these activation rules ensure that if the original inputs are stable, then the updated votes two rounds later are super-stable.

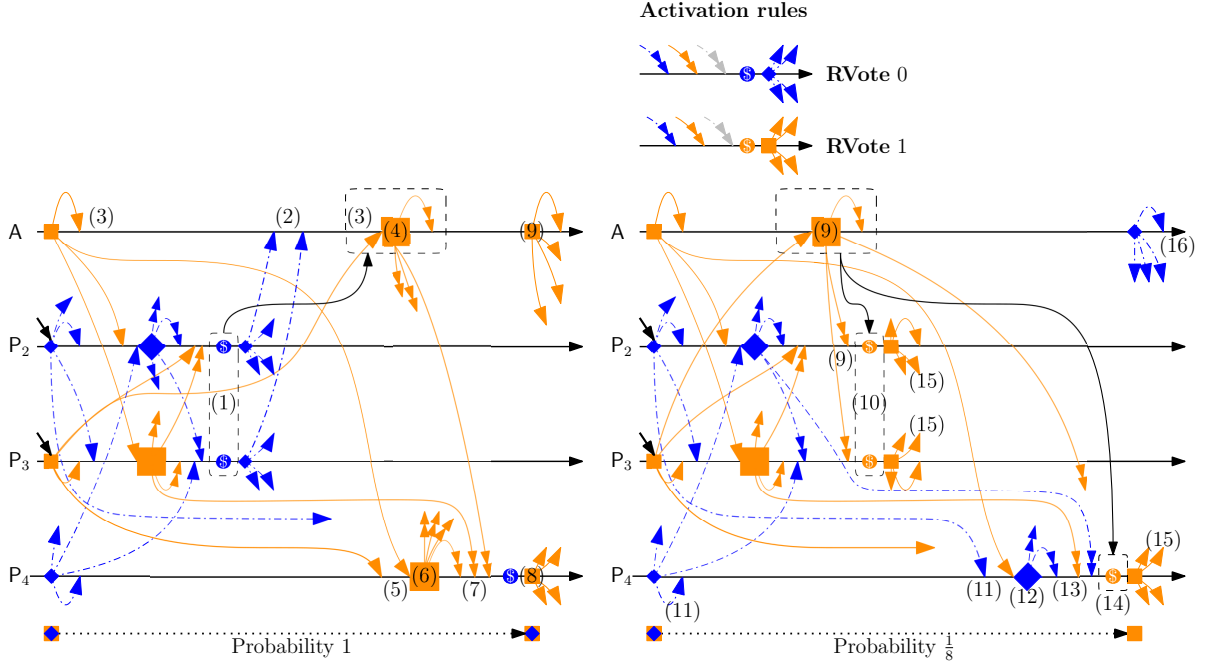


Figure 10.19: Left: the problem with coin-flipping after seeing only two votes (1). The adversary, here P_1 , stalls and learns the coin-flips of the two first honest parties (2). Even if they are lucky and vote the same as the input of the third honest party, the adversary picks the opposite second-round vote (4) using the votes of P_1 and P_3 as justification (3). It also delivers these votes to P_4 (5), which also picks 1 as second-round vote (6). It then delivers the three second-round votes for 1 to P_4 (7), which therefore does an overwrite vote for 1 (8), ignoring the coin-flip which would have made the votes stable. Right: we introduce the **RVote** b activation rule which say that on a mixed vote, before flipping the coin, wait for a third vote to arrive (the value does not matter). Now the adversary needs to first pick its second-round vote and deliver it to P_2 and P_3 before they do the coin-flip (9). Here the adversary picked 1. No matter what it picks there is now a chance that the parties flip the same, here they get lucky and both flip 1 (10). This is the lucky value as the adversary cannot make P_4 do an overwrite vote on 0. The best it can do is to deliver to P_4 the two votes of 0 from P_2 and P_4 (11) to make P_4 do the second-round vote of 0 (12). But since P_3 had a second-round vote of 1 and P_1 had to commit to its vote of 1 already in (9), party P_4 can never see three second-round votes for 0 at (13), so it uses the coin. If the honest parties get lucky again and P_4 also flips 1 at (14), then all honest parties have vote 1 (15). Therefore the vote will be stable on 1 even if the adversary cheats and picks it new vote to be 0 (16). So, if three fifty-fifty coin-flips go the right way, then the votes will stabilize. This means that no matter what the adversary does, the honest parties have a probability of at least $1/8$ of their votes stabilizing. The big difference between the left scenario and the right scenario, is that at the left the adversary can pick its second-round vote *after* seeing the coin-flips of the honest parties. In the right scenario it has to pick its second-round vote *before* seeing the coin-flips of the honest parties, which makes the coin-flips independent of the second-round vote of the adversary.

scheduling. These can then be used to give the updated votes at (5) to (8) of 0, 1, 1, 0. The reader will see that all other combinations of updated votes could be produced different scheduling.

An attempt where the initial RVOTES are biased is called *stable*, as the updated RVOTE must end up biased towards the same bit. This is illustrated in Fig. 10.21. The RVOTES at (13) to (16) are stable on 0—there are three votes on 0. Therefore, at the majority voting at (17) to (20) all parties will receive at most one RVOTE for 1 and at least 2 RVOTES for 0. Thus the majority votes will all be on 0 and the next attempt is stable on 0 too.

We indicate on the fifth, dotted line of the timeline whether the attempt is stable or unstable. If for some attempt not enough RVOTES have been sent to know if it is stable or unstable, it is indicated with a grey square.

On input b the parties will ARB an RVOTE for b for attempt 1. This activation rule is called **Input b** .

Then follows a round with voting using the majority rule, i.e., the parties wait for two RVOTES on the same bit b and then ARB an MVOTE for b . This activation rule is called **MVote b** . Since there are three honest parties, there will always arrive three RVOTES, so there will always be one bit getting at least two RVOTES, unless some honest party terminated already.

To *receive* an MVOTE for b in attempt a , a party needs to first have received two RVOTES for b for the same attempt a . This activation rule is called **Just b** (for *Justification* for voting b). Think of this as a pruned application of how we implemented causal communication in Section 4.4. If a party sends an MVOTE for b for attempt a , then it implicitly claims to have seen two RVOTES for b for attempt a . So, to prevent cheating we just wait to see these two RVOTES on b too before accepting the MVOTE on b . Note that we might not see the *same* two votes, but seeing that there are two votes for b is enough. If a corrupted party cheats and sends an MVOTE for 0 without there being two RVOTES for 0 around, then no honest party will ever accept this MVOTE. It will forever sit in the incoming queue of the receiver and will never be considered.³

The random voting rule has two cases. If a party see two MVOTES for different bits, then it flips a coin and uses the outcome as the initial RVOTE for the next attempt. Even if the first two votes collected are different the rule may not be executed until three votes were collected. This activation rule is called **RVote b** , for Random Voting b .

If a party sees *three* MVOTES for the same bit b , then it ignores the coin and uses b as the RVOTE for the next attempt. This activation rule is called **OVoting b** , for Overrule Voting b : the three votes for b makes us overrule the coin.⁴

If a party sees three RVOTES for the same bit b from the same attempt a , then it decides on b and terminates. This activation rule is called **Decide b** . Note that the three votes need not be from the “current attempt”, there is no such notion. It can be from any attempt, as long as all three votes stem from the same attempt.

Let us sketch why the protocol securely implements ABA.

We start by the crucial part and show that if an attempt a is unstable, there is a chance of at least $1/8$ that it ends up stable, i.e., the RVOTES for attempt $a + 1$ are biased to some bit. This is illustrated in Fig. 10.20. Consider the RVOTES at (5) to (8). They are unbiased, 0, 1, 1, 0, so attempt 2 is unstable. At (9) to (12) we see that the adversary chooses to schedule

³A natural worry here is whether the fact that such an unjustified vote never arrives could be a problem for liveness? No, the corrupted party could also just have *not* sent the MVOTE at all. And the protocol is designed to work even if some honest parties never see messages from corrupted parties. No honest party ever waits for more than three messages. So this activation rule is also live in the sense that it will eventually be triggered unless some honest party terminated already.

⁴If four votes were collected, then it might be the case that both random voting and overrule voting is possible. The party may do any of them.

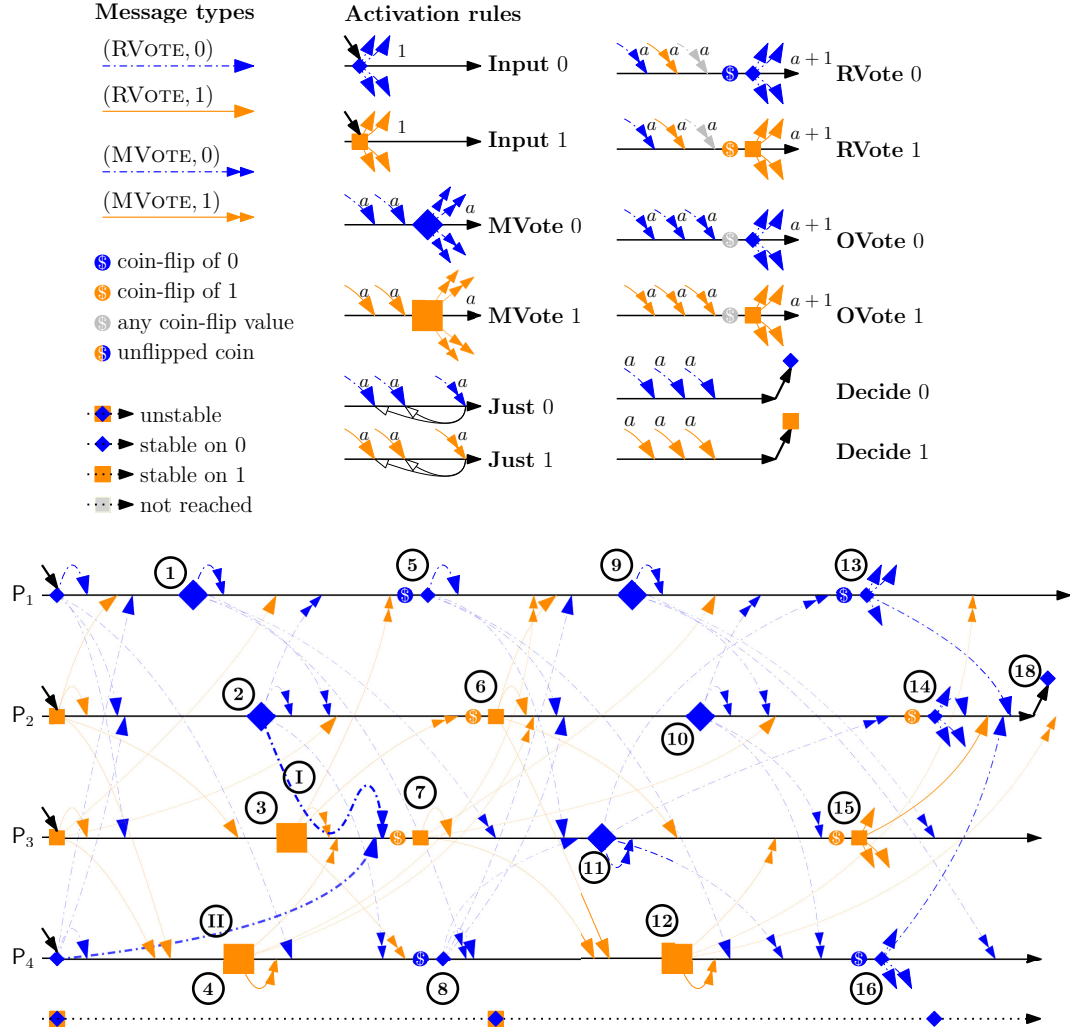


Figure 10.20: Illustration of a partial run of the SVUP protocol for $n = 4$ parties and $t = 1$ corruption. In the run parties have inputs 0, 1, 1, 0 and ARB these as RVOTES (**Input** at the beginning of the timelines). P_1 receives 0, 1, 0, in that order, and sends an MVOTE for 0 (**MVote** at (1)). P_2 receives 1, 0, 0 and MVOTES 0 at (2). P_3 receives 1, 0, 1 and MVOTES 1 at (3). P_4 receives 0, 1, 1 and MVOTES 1 at (4). Then at (5) P_1 receives MVOTES 0, 0, 1 and uses its coin, which is 0, to RVOTE 0 for the next attempt (**RVote** at (5)). At (6) P_2 receives MVOTES 0, 0, 1 and uses its 1-coin to RVOTE 1 (**RVote** at (6)). At (7) P_3 receives MVOTES 1, 1, 0 and uses its 1-coin to RVOTE 1. Note that the MVOTE on 0 received by P_3 from P_2 arrives early at (I), but has its delivery postponed until two RVOTES for 0 were received by P_3 from the same attempt. The second message is (II)—the first message arrived from P_1 at the beginning. At (8) P_4 receives MVOTES 1, 0, 1 and uses its 0-coin to RVOTE 0. The votes are back to 0, 1, 1, 0, so they are still unstable. In the next attempt P_1, P_2, P_3 receive two 0-votes and get MVOTE 0 by taking majority (**MVote**). Party P_4 gets 0, 1, 1 and thus MVOTE 1. Parties P_1 and P_2 at (13) and (14) get three MVOTES for 0 and use **OVote** to vote 0 for the next attempt. Parties P_3 and P_4 at (15) and (16) get three mixed MVOTES and use **RVote** to vote. Tyche is kind and they get coins 1 respectively 0, and thus get RVOTES 1 and 0. Now there are three RVOTES for 0 and attempt 3 is stable on 0. At (18) P_2 received all three RVOTES on 0 from attempt 3 and uses **Decide** to output 0 and terminate. The execution continues in Fig. 10.21.

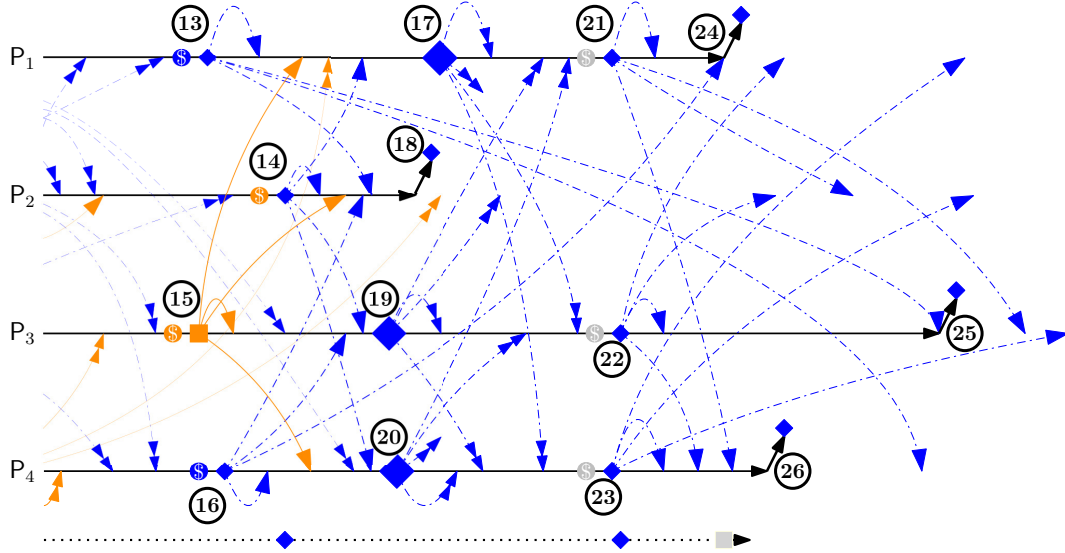


Figure 10.21: Illustration of the termination of a run of the SVUP protocol for $n = 4$ parties and $t = 1$ corruption, continuing Fig. 10.20. Party P_2 terminated at (18), receiving three RVOTES for 0 from attempt 3. The same three votes arrive at P_3 which also decide 0 at (25). Before that P_1 , P_3 and P_4 sent RVOTES for attempt 4, which is also stable on 0. Namely, since attempt 3 was stable on 0 it was a given that the MVOTES at (17), (19) and (20) would be on 0, as the parties can receive at most one RVOTE on 1, making the majority always 0. At (21), (22) and (23) these three MVOTES on 0 from attempt 4 arrive at P_1 , P_3 and P_4 which **OVotes** 0. These three RVOTES on 0 from attempt 4 all reach P_4 at (26) which decide 0. So, P_4 decides 0 from attempt 4 at (26) before P_3 decides 0 from attempt 3 at (25). This demonstrates the non-sequentiality of asynchronous protocols. However, since **Decide** based on votes from attempt a implies that a is stable, and because stability is preserved from attempt to attempt, it does not matter if parties terminate by using **Decide** on RVOTES from different attempts—the first stable attempt defines what the outputs can be.

as to make the MVOTES be 0,0,0,1. It could have chosen any other combination. Now watch the RVOTES at (13) to (16). In general there might be some parties doing overrule voting (here P_1, P_2) and some parties doing random voting (here P_3, P_4). Parties doing overrule voting will always vote the same, as there must be some parties they both heard from. Here they both heard from P_1, P_2, P_3 , but in general there will always be at least two common parties they heard from, as they both heard from three out of four. Then what about the random voters? Well, consider the *first* honest party to random vote. Here it is P_3 at (15). When P_3 is about to flip its coin at (15) we know two crucial observations.

1. Two MVOTES of the attempt have already been fixed to the same bit p . Here we see that at (15) P_3 received three MVOTES which were 0,1,0, so $p = 0$ got two votes. In general a party always collects three MVOTES before doing random voting, so some bit must occur twice.
2. All the coins used by honest random voters of the attempts are still unflipped. At (15) neither P_3 nor P_4 flipped their coins yet. This is a simple consequence of the fact that we are looking at the *first* honest party to do random voting.

The significance of the first observation is that when some bit p has two votes, then the other bit, let us call it the excluded bit x , can never get three votes. Ergo, no party will ever do overrule voting on x . In other words, *if* some honest parties did or will ever do overrule voting in the same attempt, then it will be on p (hence called the possible bit). Here we saw that P_1 and P_2 did overrule voting at (13) and (14), and it was indeed on $p = 0$.

The significance of the second observation is that the honest parties doing random voting will flip their coins *after* the possible bit p has been fixed. So, for each of the coins there is a fifty-fifty chance that they hit the possible bit p . So the chance that the coins of all three parties hit p is $\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{1}{2} = 1/8$.⁵

Let us then argue Validity, Agreement and Termination.

We first argue Termination. We first argue that eventually some attempt becomes stable. Namely, for each unstable attempt, the probability that it does not become stable is at most $7/8$. So, the probability that we run a events without the attempts becoming stable is at most

$$\left(\frac{7}{8}\right)^a,$$

and

$$\lim_{a \rightarrow \infty} \left(\frac{7}{8}\right)^a = 0,$$

so with probability 1 some attempt will eventually become stable.

And once an attempt is stable we eventually get termination. To be concrete say attempt 3 is stable on 0 as in Fig. 10.20 (see (13) to (16)). So, there are three RVOTES on 0. These three votes will eventually reach all honest parties. So, all honest parties will eventually see three RVOTES for 0 and terminate. The argument is only slightly complicated by the fact that it might happen that some honest parties terminated *before* they saw these three votes, like P_4 in Fig. 10.21, which at (26) hears three votes on 0 from attempt 4 and terminate. But all parties which did not terminate already will eventually hear the votes from attempt 3, as P_3 at (25) in Fig. 10.21. All in all, all parties eventually terminate.

Notice that there are in fact infinitely long runs of the protocol. However, they have probability 0. We need to keep getting unlucky infinitely many times for them to happen. We will

⁵In our example only two parties do random voting, but in general all three honest parties might do random voting.

leave the reader to contemplate the difference between *always* and *with probability 1*, but for the record let us state that we have argued something slightly weaker:

Probabilistic Termination: If all honest parties give an input of the form $(\text{VOTE}, \cdot)$, then with probability 1 all honest parties give an output of the form $(\text{DECISION}, \cdot)$.

That was Termination, let us then argue Validity. We have to argue that if all honest parties have the same input b , then all honest parties who output something will output b . To be concrete say P_3 is corrupted and the honest parties have input 0, as in (13) to (16) in Fig. 10.21. We already argued above how that leads to the next attempt being biased on 0 too. This will continue like this forever, all attempts being stable on 0. Therefore no party can use the **Decide** rule on 1. To output 1 in Fig. 10.21 a party would need to see three RVOTES on 1 from some attempt, but all attempts are biased to 0, so this is impossible, you will never see more than one vote for 1.

We then argue Agreement. Assume that some honest party P_i outputs 0. We then have to argue that no honest party outputs 1. If P_i outputs 0, then P_i saw three RVOTES for 0 from the same attempt, say attempt 11 to be concrete. If P_j outputs 1, then P_j saw three RVOTES for 1 from some attempt (same or other), say attempt 42 to be concrete. But if attempt 11 is stable on 0, then using the same argument as for validity, all subsequent attempts 12, 13, ... are stable on 0 too. So attempt 42 is stable on 0. So, P_j cannot see three RVOTES for 1 from attempt 42. So, P_j must have output 0. The case where P_i outputs last or they output based on votes from the same attempt is argued in the same way. To be more concrete, consider P_2 outputting at (18) in Fig. 10.21 and P_4 outputting at (26) in Fig. 10.21. P_2 saw three 0-votes from attempt 3. Therefore attempt 3 was stable on 0. Therefore attempt 4 is also stable on 0. Since P_4 output at (26) after hearing three votes from attempt 4 it must also therefore always output 0—it must have heard some of the votes that the attempt is stable on.

10.9 SVUP for General n and $t < n/3$

We now analyze SVUP. Assume for simplicity that there are exactly t Byzantine corruptions and $n = 3t + 1$ parties.⁶ So, there are exactly $n - t = 2t + 1$ honest parties.

We only give the reader the hints needed to modify the arguments for $n = 4$ and $t = 1$, as the arguments are very similar. We already saw the main tricks to argue the security.

We now call an attempt **stable on b** if $n - t$ of the RVOTES are b . Note that again, clearly an attempt cannot be stable on both 0 and 1: to be stable on 0 we have $n - t$ votes on 0. Since the votes are sent by ARB, also the votes of corrupted parties are well-defined—either 0 or 1. Since $n - t$ votes are 0 and there are at most n votes, it follows that at most t votes are 1. Since $t < 2t + 1 = n - t$ there cannot be $n - t$ votes on 1. So, stable on 0 excludes stable on 1.

Note that we have something even stronger. Namely, if attempt a is stable on 0 it excludes seeing $t + 1$ RVOTES on 1 from attempt a . So, if attempt a is stable on 0, then there cannot be a justified MVOTE on 1 in attempt a . Similarly, if attempt a is stable on 1 it excludes seeing $t + 1$ RVOTES on 0 from attempt a , so if attempt a is stable on 1, then there cannot be a justified MVOTE on 0 in attempt a .

The crucial point is again to argue that each unstable attempt has a non-zero probability of becoming stable. We again look at the point of time in an execution where the first honest party is about to do random voting in an attempt—just before it flips its coin. We observe that:

⁶It is easier to argue for the case where $n \geq 3t + 1$, as there are more honest parties compared to corrupted parties. It is also easier to argue for the case where less parties are corrupted than the maximum of t corruptions that the protocol tolerates. So, our argument applies to $t < n/3$ in general.

Each P_i runs all of the following activation rules. If more than one can be triggered during an activation, any of them may be triggered.

Input On input $(\text{VOTE}, v_i \in \{0, 1\})$ on ABA_i , let $R_i^{(1)} = v_i$ and $\text{ARB}(\text{RVOTE}, 1, R_i^{(1)})$.

MVote^(a) If for some attempt $a \in \mathbb{N}_+$ it happens that $(\text{RVOTE}, a, R_j^{(a)})$ arrived via ARB from $t + 1$ distinct parties, for which $R_j^{(a)} = v$ for a common bit $v \in \{0, 1\}$, let $M_i^{(a)} = v$ and $\text{ARB}(\text{MVOTE}, a, M_i^{(a)})$. We call such a bit $M_i^{(a)}$ a **majority vote**.

Just^(a) To receive a vote $(\text{MVOTE}, a, M_j^{(a)})$ from P_j for attempt $a \in \mathbb{N}_+$ party P_i needs to first see the majority vote $(\text{MVOTE}, a, M_j^{(a)})$ come via ARB from P_j . But it also needs to see $(\text{RVOTE}, a, R_k^{(a)})$ arrive via ARB from $t + 1$ distinct parties P_k with $R_k^{(a)} = M_j^{(a)}$. If this does not happen it simply behaves as if $(\text{MVOTE}, a, M_j^{(a)})$ never arrived. We call such a MVOTE accompanied by $t + 1$ matching RVOTES a **justified vote**.

OVote^(a) If for some attempt $a \in \mathbb{N}_+$ it happens that justified votes $(\text{MVOTE}, a, M_j^{(a)})$ were received from $n - t$ distinct parties for which $M_j^{(a)} = J$ for a common bit $J \in \{0, 1\}$, then let $R_i^{(a+1)} = J$ and $\text{ARB}(\text{RVOTE}, a + 1, R_i^{(a+1)})$. We call this an **overwrite vote**.

RVote^(a) If for some attempt $a \in \mathbb{N}_+$ it happens that justified votes $(\text{MVOTE}, a, M_j^{(a)})$ were received from $n - t$ distinct parties and two of them were different, i.e., $M_j^{(a)} \neq M_k^{(a)}$, then pick a uniformly random bit $R_i^{(a+1)} \in_R \{0, 1\}$ and $\text{ARB}(\text{RVOTE}, a + 1, R_i^{(a+1)})$. We call this a **random vote**.

Decide^(a) If for some attempt $a \in \mathbb{N}_+$ it happens that votes $(\text{RVOTE}, a, R_j^{(a)})$ arrived via ARB from $n - t$ distinct parties for which $R_j^{(a)} = d$ for a common bit $d \in \{0, 1\}$, then output $(\text{DECISION}, d)$ on ABA_i and terminate.

Figure 10.22: The Vote Updating Protocol SVUP for $n = 3t + 1$ parties of which at most t are Byzantine corrupted.

1. At least $t + 1$ of the MVOTES of the attempt have already been fixed to the same bit p . Namely, a party always collects $n - t$ MVOTES before doing random voting, and $n - t = 2t + 1$. Therefore it cannot be that at most t are on 0 and at most t are on 1, because then there would be only $2t$ votes. The missing vote must be 0 or 1, bring some tally to $t + 1$.
2. All the coins used by honest random voters of the attempts are still unflipped. This is a simple consequence of the fact that we are looking at the *first* honest party to do random voting.

The significance of the first observation is that when some bit p has $t + 1$ votes, then the other bit, let us call it the **excluded bit** x , can never get $n - t$ votes. Ergo, no party will ever do overrule voting on x . In other words, *if* some honest parties did or will ever do overrule voting in the same attempt, then it will be on p (hence called the **possible bit**).

The significance of the second observation is that the honest parties doing random voting will flip their coins *after* the possible bit p has been fixed. So, for each of the coins there is

a fifty-fifty chance that they hit the possible bit p . So the chance that the coins of all honest parties hit p is

$$\left(\frac{1}{2}\right)^{n-t} \neq 0.$$

Now solve exercises Exercise 10.8 and Exercise 10.9.

10.10 Interactive Coin-Flips*

The above SVUP uses a local coin-flip. Notice that what is needed to get lucky is that the coin is random and unpredictable and that all honest parties uses the same coin. To get better probability that all honest parties get the same coin, one need to do something smarter where the parties interact. One would then run the coin-flip protocol right after receiving $n-t$ RVOTES and use the outcome to set the coin. All that is needed for stabilization is that often enough the coin is random and identical at all parties. The ideal functionality of a coin-flip protocol would be something along the lines of the following.

Definition 10.2 (Coin-Flip (informal)) Coin-Flip is given by an ideal functionality Flip. Party P_i has a port Flip_i . Each coin-flip will have a unique flip ID fid . Honest parties input $(\text{FLIP}, \text{fid})$ to start the coin-flip. At some point Flip outputs $(\text{COIN}, \text{fid}, c_i \in \{0, 1\})$ to P_i to inform it that the outcome was c_i . It is allowed that $c_i \neq c_j$ and that the c_i 's were picked by the adversary. But it must always again eventually happen that a coin-flip is good. A good coin-flip is one in which after the first honest party input $(\text{FLIP}, \text{fid})$ the ideal functionality generated a uniformly random $c \in \{0, 1\}$ and set $c_i = c$ for all honest P_i .

Our approach with local coin-flips implement the above, but for large n it becomes very rare that all coins are identical. There are other protocol where it always happens that the coins are identical. One such protocol based in the DDH assumption, discussed in Chapter 9, is presented in [13].

10.11 Impossibility of Deterministic Asynchronous Byzantine Agreement*

The SVUP protocol used random coins to ensure termination. In this section we argue that we had to do this. We argue that all ABA protocols which has Agreement and Validity have infinite executions. Even worse, there are infinite executions even if the protocol tolerates just a single crashing party (no Byzantine corruptions allowed) and these infinite executions occur even when all parties are honest, so it is just the threat of a single crash which provoke them. As will be clear later, it is the Godot problem all over again.

Having some infinite executions does not mean that no execution ever stops, but for any length L there is a way to make the protocol run for L steps. This, in particular, show that the protocol does not have Termination: it is not *always* guaranteed to terminate. The best we can hope for is therefore Probabilistic Termination: the protocol terminates with probability 1.

The two central concepts in the proof are:

Unstable state: Recall that a state of a protocol is a state for each server plus the messages in transit. An example is given by the left timeline in Fig. 10.23. Parties P_1, P_2, P_3 had input 0 and P_4 and P_5 had inputs 1. Some messages were sent and delivered, so are in transit. A state is called **unstable** if there is one way to do schedule message deliveries such that the parties end up with outputs 0 and another way to do schedule message deliveries such that the parties end up with outputs 1.

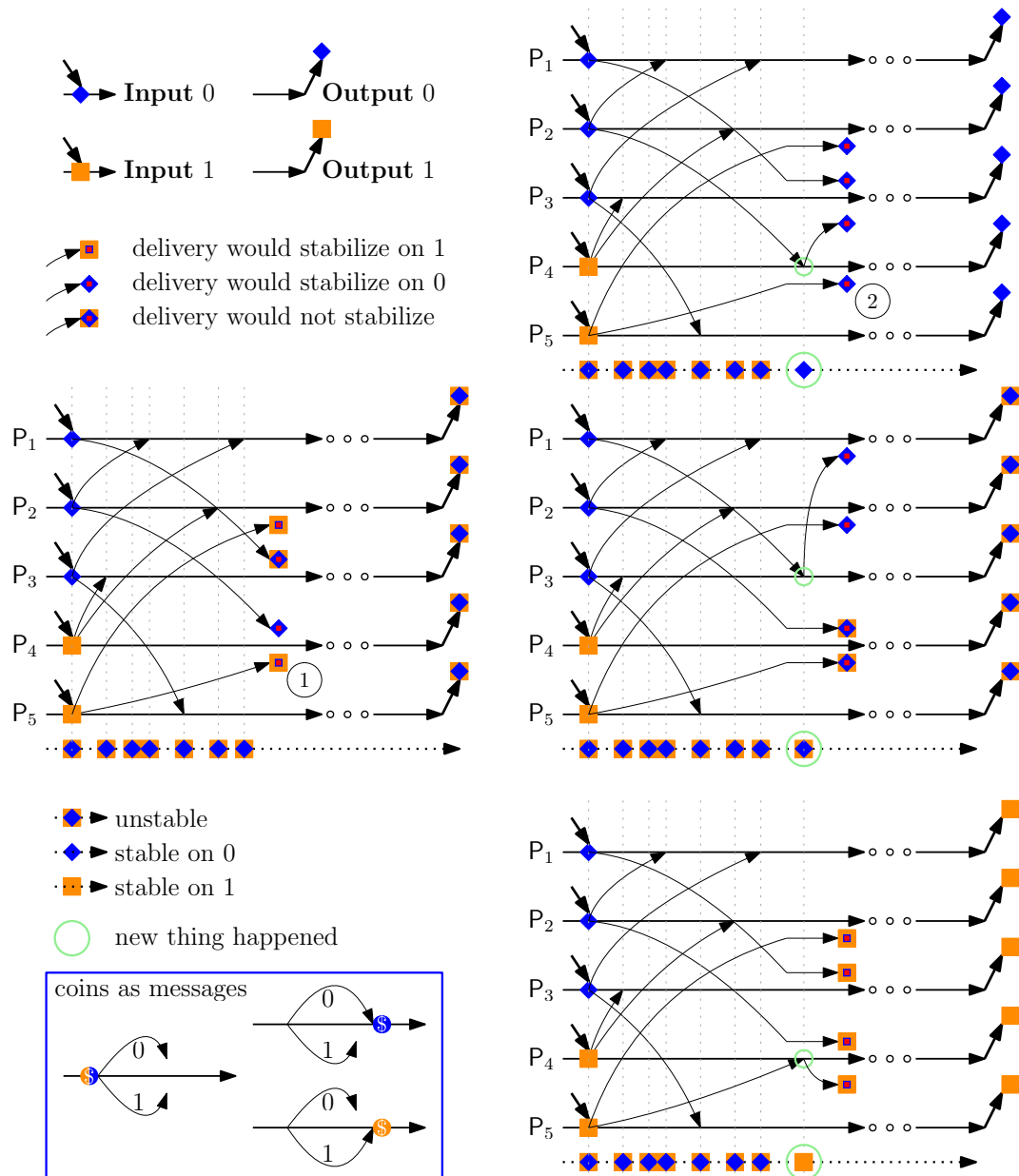


Figure 10.23: NoDetABA

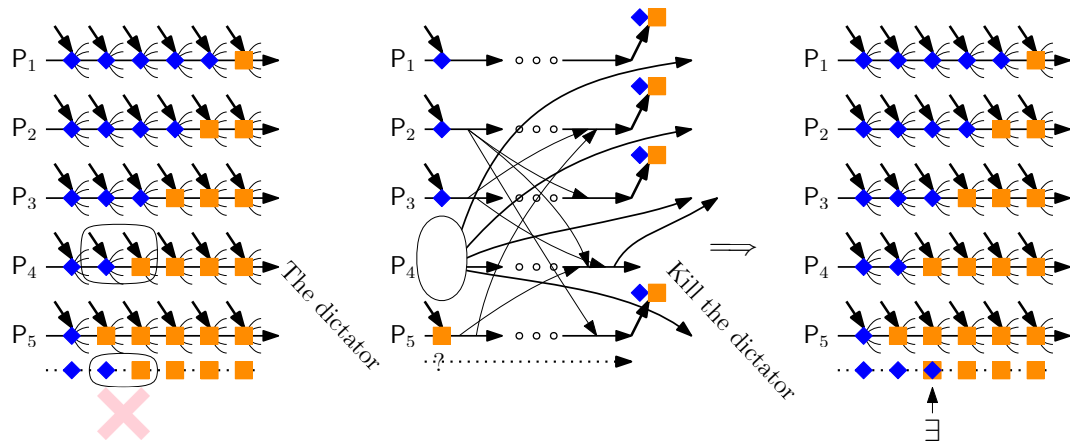


Figure 10.24: If the all settings of the inputs are stable, then one can go from stable on 0 to stable on 1 by changing a single input, here of P_4 . But then P_4 is a Dictator, a party which has an input determining the output. But all messages from the Dictator can be delayed forever, such that the other parties terminate without hearing from the Dictator. But then the input of the Dictator cannot influence the outputs of the other parties. Ergo, there cannot be a Dictator. Ergo, there must be some unstable setting of the inputs.

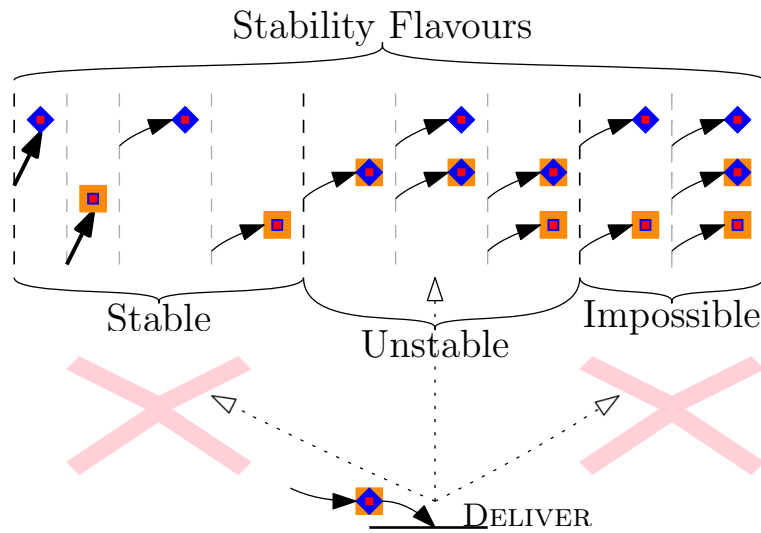


Figure 10.25: Above, the nine flavours of stability an ABA protocol can have. From left to right: 1) The protocol already terminated with output 0. 2) The protocol already terminated with output 1. Then seven flavours where the protocol did not terminate and there are messages in transit. 3) All messages in transit are stable on 0. 4) All messages in transit are stable on 1. 5) All messages in transit are unstable. 6) Some messages stable on 0 and some unstable. 7) Some messages stable on 1 and some unstable. 8) Some messages stable on 0, some stable on 1 and none unstable. 9) Some messages stable on 0, some stable on 1, and some unstable. In Section 10.11.4 we argue that the two right most flavours are impossible. No protocol can be in a state of this flavour. The delivery of an unstabilizing message by definition cannot bring us to one of the four left-most states, as they are stable. Namely, an unstabilizing message is defined to be one which brings us to an unstable state. Obvious the delivery of an unstabilizing message cannot bring us to an impossible state. Ergo, the delivery of an unstabilizing message brings us in a state with one of the three flavours marked *Unstable*. Note that they are contain an unstabilizing message.

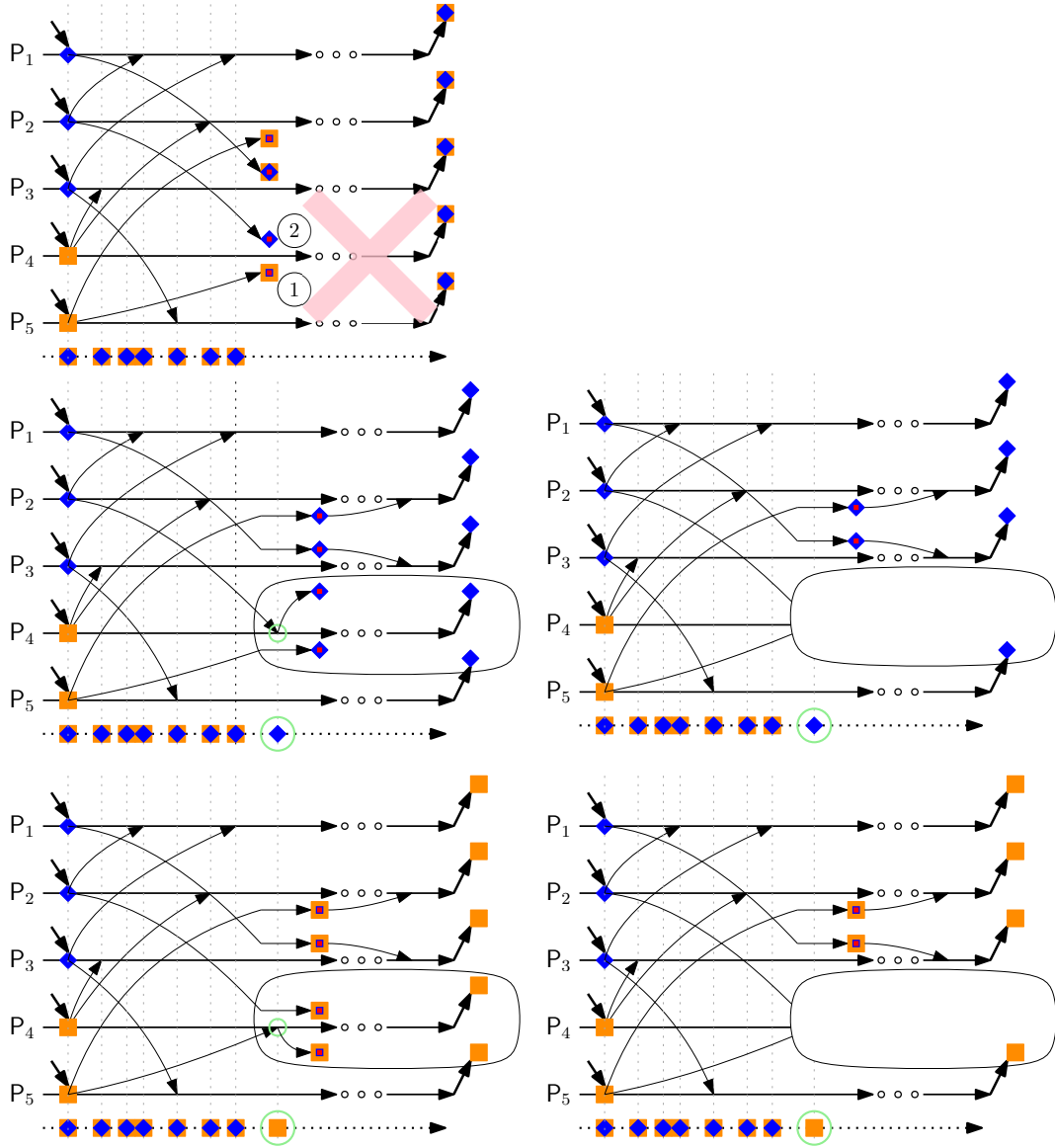


Figure 10.26: A 1-stabilizing message is delivered to P_4 at (1) or a 0-stabilizing message is delivered to P_4 at (2). In the middle, left timeline we deliver the 0-stabilizing message to P_4 . Since we delivered a 0-stabilizing message the network is now, by definition, biased on 0. In the bottom, left timeline we deliver the 1-stabilizing message to P_4 . Since we delivered a 1-stabilizing message the network is now biased on 1. However, if we “Kill the Dictator” and delay the messages from P_4 until the other parties gave output, they cannot know which message we delivered to P_4 . So they output the same in the middle, right timeline as in the bottom, right timeline, as the same messages are delivered at the same times. We conclude that $0 = 1$. So, we must have started from an impossibility.

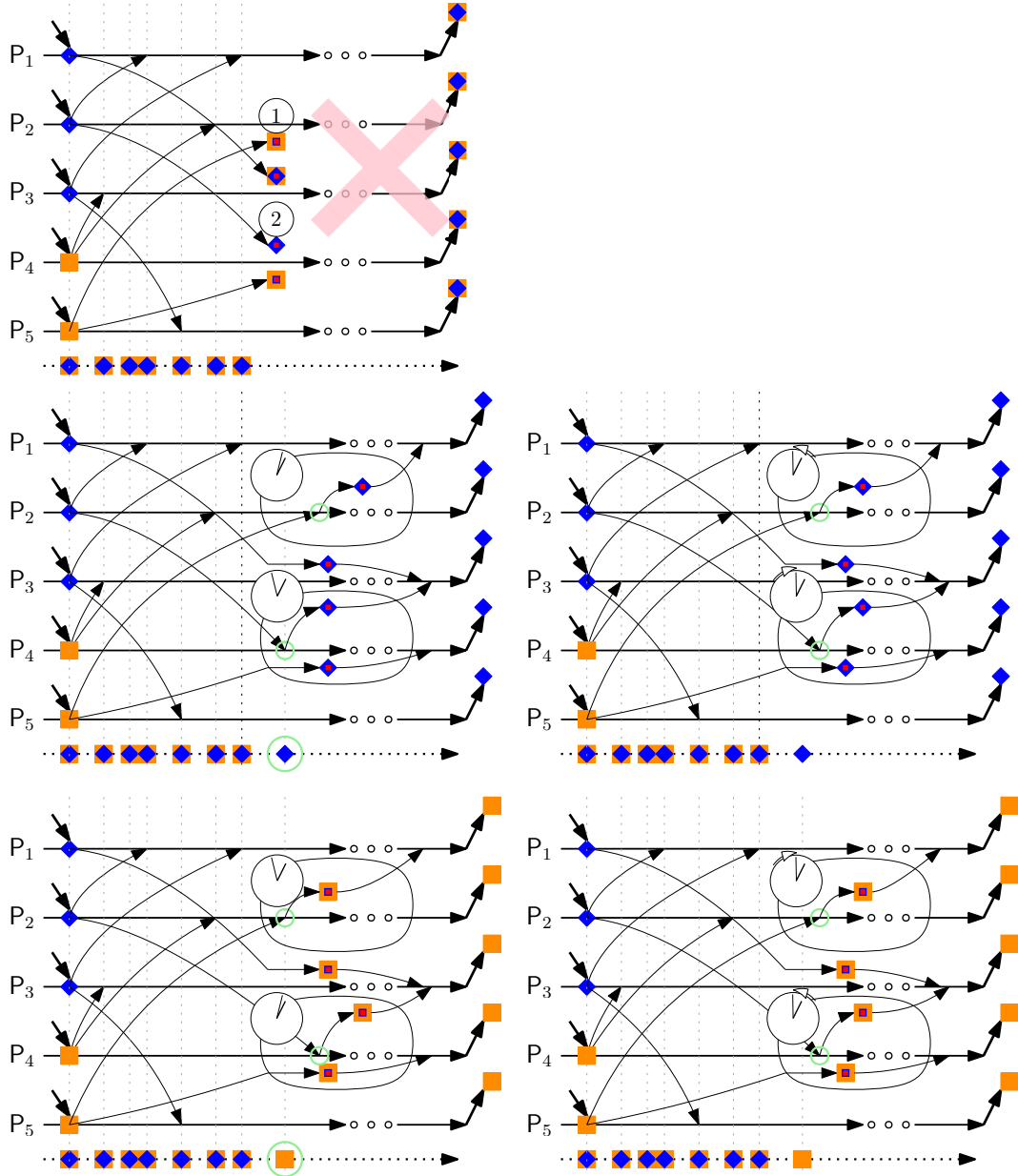


Figure 10.27: A 1-stabilizing message is delivered to P_2 at (1) and a 0-stabilizing message is delivered to P_4 at (2). In the middle, left time diagram first we deliver the 0-stabilizing message to P_4 first and then to P_2 . Since we delivered the 0-stabilizing message first the network is now, by definition, biased on 0. However, by slightly adjusting local delivering times without changing the causal order of messages we can see the two messages as being delivered at the same times, as in the middle, right timeline. Since we did not change the order of messages, parties will have the same outputs, i.e., 0. In the bottom, left time diagram first we deliver the 1-stabilizing message to P_2 first and then to P_4 . Since we delivered the 1-stabilizing message first the network is now biased on 1. However, by slightly adjusting local delivering times without changing the causal order of messages we can see the two messages as being delivered at the same times, as in the bottom, right timeline. Since we did not change the order of messages, parties will have the same outputs, i.e., 1. But in the middle, right diagram and the bottom, right diagram the same messages are delivered in the same order, so the parties have the same outputs in the two runs. Ergo, $0 = 1$. We must therefore have started from an impossibility.

Unstabilizing message: An unstabilizing message of a state is a message in transit which has the property that if this message is delivered next, then the state will still be unstable. I.e., delivering this message will not fix what the final outputs will be.

The two central observations in the proof are:

There is an unstable initial state: It is possible for the adversary to pick initial inputs such that after all parties were given these inputs and sent their initial message, then protocol is in an unstable state.

Unstable states have unstabilizing messages: Whenever the protocol is in an unstable state, it has an unstabilizing message.

In the rest of the section we will focus on arguing that the two central observations are true. Once this is done the proof that there are infinite execution is simple. Namely, an adversary can first give inputs such that the protocol starts in an unstable initial state. This unstable state then has an unstabilizing message. Deliver that unstabilizing message. Then the protocol is again in an unstable state, as an unstabilizing message is one which leads to an unstable state. This unstable state then has an unstabilizing message. Deliver that unstabilizing message, and the protocol is again in an unstable state. This unstable state has an unstabilizing message. Deliver that unstabilizing message, *et cetera in infinitum*.

10.11.1 Dealing with Random Coins

Before we proceed we need to be more precise about how we deal with random coins.

As we noted above, the presence of infinite executions prevents Termination, but we can still hope to get terminate with probability 1 if we use randomness.

This might seem odd, is probability 1 not the same as *always*? No! Consider the experiment of flipping a coin until it comes out heads. The probability that this takes F flips is 2^{-F} . There is an infinite run, namely that we flip *heads, heads, heads* and so on forever. However, the probability is 0. So the experiment stops with probability 1, but it does not *always* stop. For any length F there is a probability 2^{-F} that it takes F tries.

We are interested in whether there are infinite execution, not how likely they are. So, we take the randomness out of the coins. As shown in the left bottom box in Fig. 10.23 we will replace all coin-flips by a party P by P sending a 0 and a 1 to itself. The result of the coin-flip will be the message delivered first. Notice that this now lets the adversary decide the coin-flip. That is of course bad for Probabilistic Termination, but it does not affect whether a protocol has normal Termination. In fact, this will make our proof much simpler. We are only interest in whether there *exist* infinite executions, not how likely they are. If there are infinite executions, now the adversary can demonstrate this by showing use the sequence of bad coins which would produce them.

After his hack, the protocol is now deterministic, which will make our argument easier. What we want to argue now is then that any deterministic ABA has infinite executions.

10.11.2 Stable/Unstable States/Messages

We then add a bit more detail on the notions of stability. Consider the timeline to the left in Fig. 10.23. We say that a state like that is stable on b if for any possible future scheduling the protocol will always terminate and the outputs will always be b . If the protocol is not stable on 0 nor stable on 1, then we call it unstable. Whenever a message is delivered we mark under the timeline the status of the new state.

For messages in transit we mark them stable on 0, stable on 1 or unstable depending on what would happen if they are delivered next. So, for instance, a message is unstable if when we deliver it next, then resulting state of the protocol is unstable. We mark the status on a message on the message.

Note that the status of a message is not the bit sent, a blue message is not the sending of the bit 0. A blue message is a message where is delivered next the only possible final outputs of the protocol become 0.

Note also that whenever a message is delivered the status of all other messages might change. For instance, if a blue message is delivered, then the only possible final outputs of the protocol become 0. Therefore all future states are stable on 0 and thus all messages in transit are stable on 0 too: delivering them results in a state stable on 0.

To the right in Fig. 10.23 is shown the effect of delivering a blue message to P_4 , an unstabilizing message to message to P_3 , respectively an orange message to P_4 .

10.11.3 There are Unstable Initial States

We first argue that there are unstable initial states. We give the argument for $n = 5$, but it works in general. To the left in Fig. 10.24 the columns correspond to some of the possible inputs of 5 parties. The first is when they all have input 0. The next is when P_5 has input 1 and the rest 0, and so on. Below the columns we mark whether the initial state, right after all parties sent their initial messages and none were delivered is stable or unstable. We know that the left most column must be stable on 0, as an ABA must always output 0 when all inputs are 0. This is Validity. Similarly, we know that the right most column must be stable on 1. Therefore, as we move from left to right on the marks at the bottom, there must be a first time where the mark is not blue. We are interested in what happens here.

If it orange or unstable? If it is unstable we are done, as we wanted to find an arrangement of inputs leading to an unstable initial state. So, let us just argue that it cannot be orange.

The reason why the first non-blue mark cannot be orange is that then there is a Dictator. In the present example P_4 would dictate the result. Parties P_1, P_2, P_3 have input 0 in both of column 2 and 3. Party P_5 has input 1 in both columns. The only difference is that P_4 has input 0 in columns 2 and P_4 has input 1 in columns 3. And column 2 is stable on 0 and columns 3 is stable on 1. So the protocol *always* outputs the input bit of P_4 .

But this cannot be. Recall that the protocol tolerates a single corruption. This means that the other parties in particular will terminate the protocol without hearing from P_4 . If we delay the message of P_4 for long enough and deliver all other messages, then the other parties must terminate because of the Godot problem. Party P_4 may be corrupted and have sent no messages at all. This is the middle timeline in Fig. 10.24.

But this is now absurd. Obviously the other parties cannot output the input of P_4 without hearing from P_4 , as all other inputs and messages are the same. Short of telepathy there is no way the other parties can learn about the input of P_4 before they must give output.

This allows us to conclude that as we move left to right the first non-blue mark is not orange. Ergo, it is unstable as sketched in the right hand diagram in Fig. 10.24. We take the corresponding input $(0, 0, 0, 1, 1)$ and we have our initial unstable state the parties these inputs and active them to make them send their initial messages.

10.11.4 The Forbidden States

We then have our initial unstable state. On our way to showing that it contains an unstabilizing message, we need to take a closer look at the different flavour of stability a state can have. They are listed in Fig. 10.25. We want to show that the two right-most flavours are impossible. In

particular, no protocol can have a messages stable on 0 and a message stable on 1. The reason is that delivering them as the next two messages would have impossible consequences, so creating the state with those flavours has to be impossible in the first place.

To see that these flavours are impossible consider what happens if we deliver the message biased on 0 and then right after deliver the message biased on 1. Either this takes place at the same party or at two different parties.

If it happens at the same party P , then we have a new dictator. Depending on which message was delivered first the output of the protocol has to be 0 respectively 1. But we can delay the messages from P such that it cannot inform the other parties about the order of delivery. It is impossible that the parties learn the order without P communicating the information to them. This is illustrated in Fig. 10.26.

If it happens at two different parties P and Q , then the delivery events are concurrent. And since the parties have no clocks there is no way to determine which message was delivered first. Delivering first the 0-stabilizing message to P and then the 1-stabilizing message to Q brings the network in the same state as if delivering first the 1-stabilizing message to Q and then the 0-stabilizing message to P . This is illustrated in Fig. 10.27

10.11.5 Unstable States have Unstabilizing messages

We can then argue that an unstable state has an unstabilizing message. The argument is sketched in Fig. 10.25. An unstabilizing message is one which when delivered produces an unstable state. So delivering it can, by definition, not lead to any of the four stable flavours to the left. Neither can the delivering of a message lead to an impossible state. Therefore, delivering an unstabilizing messages produces a state of the flavours labelled “Unstable”. We see that all three contain an unstabilizing message. The adversary can deliver this unstabilizing message next, get a new unstable state with an unstabilizing message, and so on forever.

10.12 Exercises

Exercise 10.1 (Breaking the Bracha Rules) Consider the state of the Bracha protocol in Fig. 10.28. Where were wrong messages sent? Where we messages not sent which should have been sent? ▼

Exercise 10.2 (Concrete runs of Bracha) Consider the state of the Bracha protocol in Fig. 10.29. Party P_1 is Byzantine corrupted and has made a mess. Turns out the Bracha software of P_1 had a bug. It has now been updated and rebooted, so from now on P_1 will follow the Bracha Protocol. So will P_2, P_3 and P_4 . Can you do an exhaustive scheduling of deliveries and activations such that:

1. P_2, P_3, P_4 all output FOO?
2. P_2, P_3, P_4 all output BAR?
3. P_2 outputs FOO and P_4 outputs BAR?
4. One of P_2, P_3, P_4 did no give output?

An exhaustive scheduling is one where no messages are in transit and where no honest party would send a new message if activated. ▼

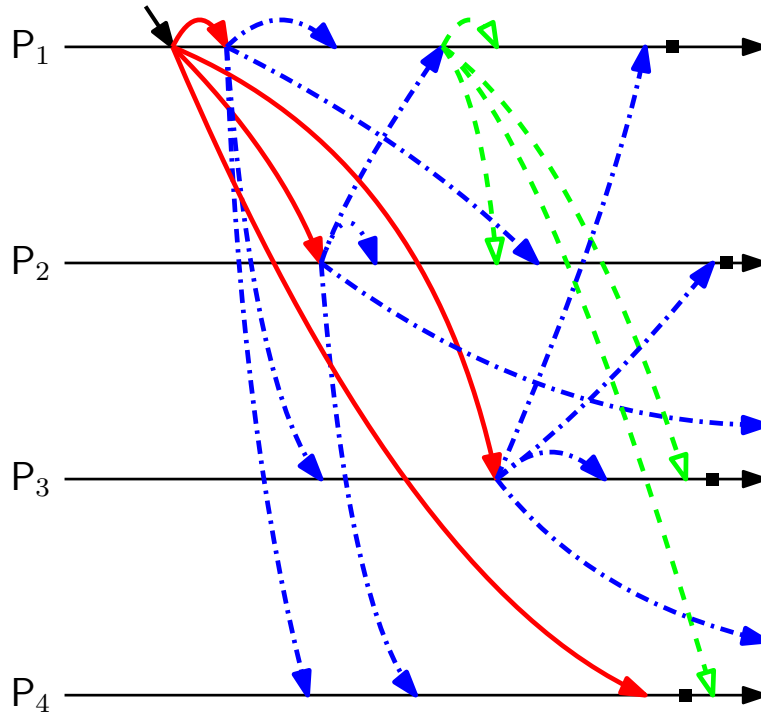


Figure 10.28: The parties installed a new version of their Bracha software, but some fool released a buggy version. Find places where wrong messages were sent or where required messages were not sent. The last time a party was activated is marked by a black square. When activated last time a party saw the messages to the left of the square. The messages to the right of the square arrived after the party was activated last time, so they have not been processed yet.

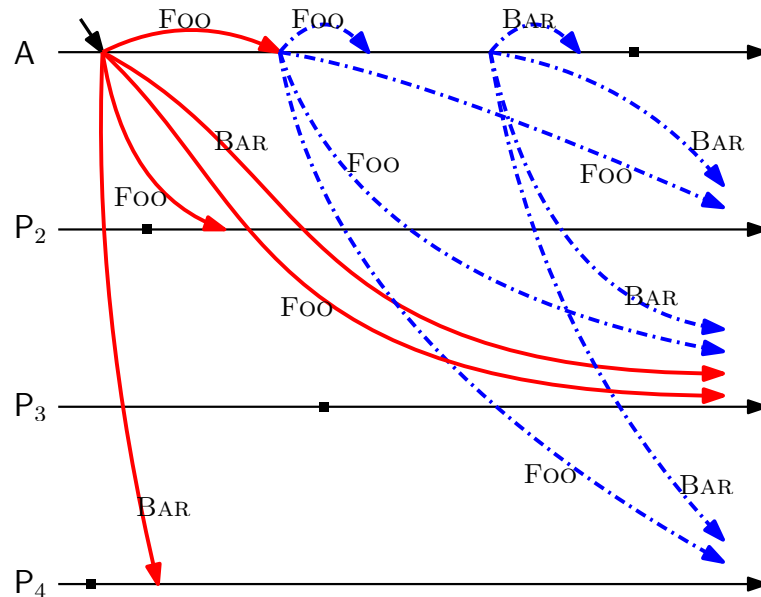


Figure 10.29: P_1 is Byzantine corrupted and made a mess. Here $m = \text{Foo}$ and $m' = \text{Bar}$ are different messages. It is indicated by a black square when a party was last activated, i.e., parties P_2 and P_4 have received messages, but were not activated yet to process them. If you do an exhaustive scheduling and all parties (including P_1) follow the rules of Bracha from now on, what will happen? What will the outputs be? Will there even be outputs?

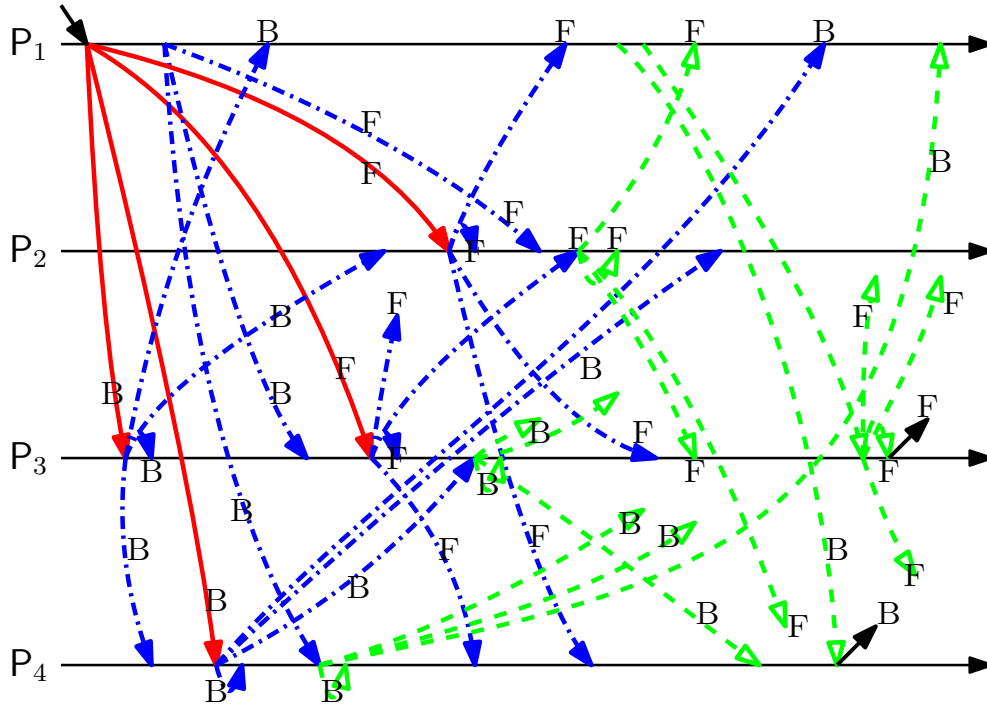


Figure 10.30: Two seemingly honest parties gave different outputs. It turns out that only P_1 “choose” to corrupted. One of the other parties ran a version of Bracha where one of the activation rules was implemented with a small bug. Which party had a buggy version and what was the bug?

Exercise 10.3 (Bracha Bug) See Fig. 10.30. Explain the figure by P_1 being corrupted and the implementation of one of the other parties having a bug. Try to identify the simplest bug which can explain the behavior. ▼

Exercise 10.4 (Concrete run of SVUP) Make a run of SVUP with the same inputs and coins as in Fig. 10.20, but make the output be 1. See Fig. 10.31. ▼

Exercise 10.5 (Breaking the SVUP Rules) See Fig. 10.32. Find all violations of the SVUP activation rules. A violation is a message sent at a time, where the necessary messages to trigger an activation rule has not arrived or the state of the party does not allow to execute the activation rule. For the places where there are no violations, specify which activation rule was used. If in doubt about an activation rule, use Fig. 10.22 as the authoritative source. ▼

Exercise 10.6 (π -SVUP) Some poor protocol designer has taken a course on distributed systems and learned that the parties in SVUP need to do random choices. They also heard that the digits of π “like behave like random”, so they take the bits of π written in binary, 11.001001000011111..., and use these bits as the coins. I.e., in the first attempt all parties use the coin 1, in the second attempt 1, in the third 0, then 0, 1, 0, *et cetera*. You are the adversary. Describe a way to schedule SVUP for four parties such that it runs forever if implemented using π for coins. ▼

Exercise 10.7 (SVUP Bug) See Fig. 10.33. Which party is corrupted? Which of the activation rules was implemented wrongly? Argue for your answers. ▼

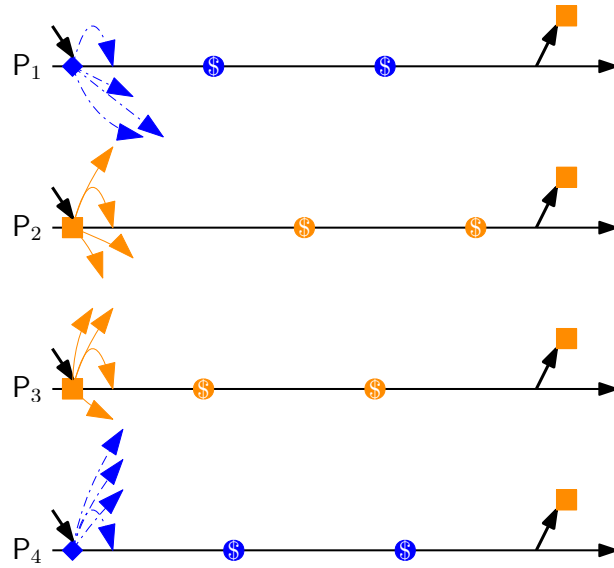


Figure 10.31: You are the adversary and controls scheduling. Make a run of SVUP giving outputs 1 using the same inputs and coins as in Fig. 10.20.

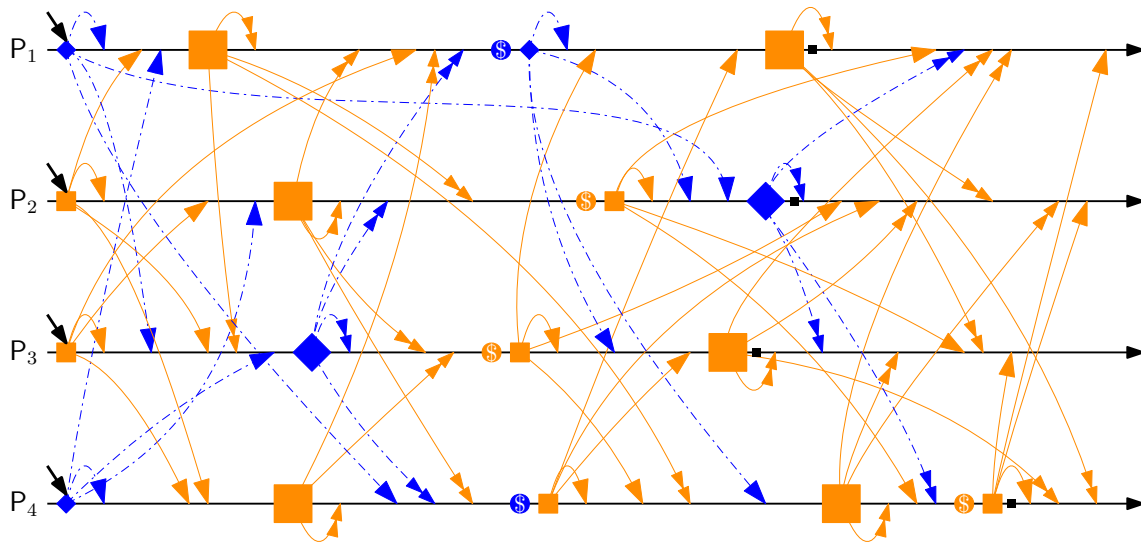


Figure 10.32: A run of SVUP, but apparently with a poor implementation. Eight mistakes were made, can you find them all?

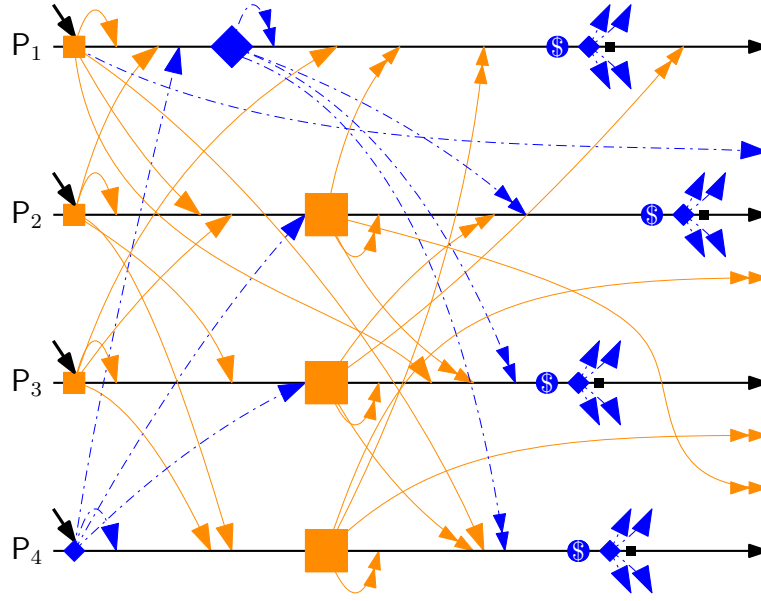


Figure 10.33: A run of SVUP just went from stable to super-stable, but on the opposite bit. This should be impossible. It is due to one corrupted party and one implementation error.

Exercise 10.8 (Stability Preservation of SVUP) Argue that in SVUP for general $n = 3t + 1$, if attempt a is stable on b , then attempt $a + 1$ is also stable on b . Mimic the proof for the case with $n = 4$, but adopt the arguments to the quorums of $t + 1$ and $n - t$. ▼

Exercise 10.9 (TVA of SVUP) Argue Probabilistic Termination, Validity and Agreement of SVUP for general $n = 3t + 1$. You can use that each unstable attempt has a non-zero probability of becoming stable. You can also assume Exercise 10.8 was solved, so if attempt a is stable on b , then so are all subsequent attempts. Mimic the proofs for the case with $n = 4$, but adopt the arguments to the quorums of $t + 1$ and $n - t$. ▼

Exercise 10.10 (Ready?) In Bracha the parties only trigger **Echo** once. This is important for Agreement. Only one message should be echoed by each honest party. But the parties are willing to trigger **Ready 1** and **Ready 2** any number of times. What would happen if we changed the protocol such that honest parties were only willing to trigger **Ready 1** and **Ready 2** once? I.e., they only trigger the rule for the first message m (for each bid) for which they hear $n - t$ ECHO messages respectively $t + 1$ READY messages. Does the protocol still have all the properties required to be a reliable broadcast, i.e., Agreement, Validity, Termination 1 and Termination 2? ▼

Exercise 10.11 (Optimality of the Corruption Bound, ARB) Assume a fully connected model with authenticated channels and asynchronous communication with eventual delivery. Assume that there are Byzantine errors. Show that there is no protocol for asynchronous reliable broadcast in this model tolerating $t = n/3$ errors. We already argued this for the case $n = 3$ and $t = 1$ in the CAP theorem in Section 10.3. The main idea of the proof for the general case is to group the $3t$ parties into three groups of size t , called S (containing the sender), R_1 and R_2 . Then mimic the proof of the CAP theorem for $n = 3$. ▼

Exercise 10.12 (Optimality of the Corruption Bound, ABA) Assume a fully connected model with authenticated channels and asynchronous communication with eventual delivery. Assume that there are Byzantine errors. Argue that there is no protocol for ABA in this model

tolerating $t = n/3$ errors. Argue it by arguing that if you can do ABA with corruption threshold t , then you can also do ARB with corruption threshold t . How do you use one run of ABA to ARB one bit? ▼

In the below exercises you are asked to solve an agreement problem. It could be that the problem is possible to solve deterministically. The parties are allowed to use a PKI and signatures. It could also be that it is impossible to solve deterministically but is possible to solve using randomness. It might also be that it is impossible to solve all together. If it is impossible to solve the problem you should argue why this is the case. If it is impossible to solve it deterministically but it can be done with randomness, then you should argue why this is the case and then give a probabilistic solution, for instance by using ABA. If it *is* possible to solve it deterministically, then you should show this by giving a solution which does not use randomness—maybe by only using ARB. Note that using ABA counts as using randomness. The best approach to proving that a deterministic solution does not exist is to show that if one could solve the problem at hand then one could solve ABA. The best approach to proving that not even a probabilistic solution exists is to show that if one could solve the problem at hand then one could solve something impossible, like ABA for $n = 3t$ parties and t corruptions.

All the exercises are for the asynchronous model where messages can have arbitrary long delay, even between honest parties.

If you come up with a solution using ABA, try to use as few calls to ABA as possible.

Exercise 10.13 (Possible or Impossible I) Read the general instructions at the beginning of the section. The problem at hand is as follows. There are $n = 3$ servers. Each has an earthquake censor. Servers are either correct or Byzantine. Sensors are either correct or crashed. A correct sensor inputs EARTHQUAKE to its server if and only if there is an earthquake. A correct server does nothing until it hears a detection of an earthquake. When there is a detection it will talk to the other servers and reach a common decision. A crashed censor never inputs anything to its servers. So it does false negatives but never false positives. We want to tolerate one Byzantine server: Maybe we anticipate that the server at the epicenter will be corrupted. We also want to tolerate that one honest server has a corrupted sensor. We assume authenticated channels with eventual delivery between servers. There is no delay between censors and their server. Your solution should be able to tolerate that *either* there is a corrupted server *or* a crashed sensor at an honest server. It need not tolerate that these two faults happen at the same time. We can model the problem as follows.

Inputs Parties could get inputs of the form EARTHQUAKE.

Validity If an honest server outputs EARTHQUAKE, then an honest server got the input EARTHQUAKE.

Detection If two honest servers get input EARTHQUAKE, then all honest servers should output EARTHQUAKE.

Propagation If any honest server outputs EARTHQUAKE, then all honest servers eventually output EARTHQUAKE. ▼

Exercise 10.14 (Possible or Impossible II) Read the general instructions at the beginning of the section. The problem at hand is as follows. There are $n = 3$ servers. Each has an earthquake censor. Each hour or so the servers look at their censor and reads off if there is an earthquake or not. Then they communicate and reach a joint decision. The servers are

geographically separated and might not have synchronised clocks so it might happen that one honest server detects no earthquake and some other honest server detects an earthquake a bit later. Servers are either correct or Byzantine. Sensors are either correct or crashed. A correct sensor inputs EARTHQUAKE to its server iff there is an earthquake. A crashed sensor never inputs anything to its servers. So it does false negatives but never false positives. We want to tolerate one Byzantine server. We also want to tolerate that one honest server has a corrupted sensor. We assume authenticated channels with eventual delivery. Your solution should be able to tolerate that *either* there is a corrupted server *or* a crashed sensor at an honest server. It need not tolerate that these two faults happen at the same time. We can model a single execution of the problem as follows.

Inputs Each party eventually gets input EARTHQUAKE or NEQ.

Termination Eventually each party gives output EARTHQUAKE or NEQ.

Validity If an honest server outputs O , then an honest server got the input O .

Agreement All honest servers output the same.



Exercise 10.15 (Possible or Impossible III) Same a II with $n = 4$.



Exercise 10.16 (When did it Happen) Read the general instructions at the beginning of the section. The problem at hand is as follows. Try to answer the question for both $n = 3$ servers and $n = 4$ servers. Each has an earthquake sensor. Servers are either correct or Byzantine. Sensors are either correct or crashed. A correct sensor inputs EARTHQUAKE to its server iff there is an earthquake. A crashed sensor never inputs anything to its servers. So it does false negatives but never false positives. We want to tolerate one Byzantine server. We also want to tolerate that one honest server has a corrupted sensor. We assume authenticated channels with eventual delivery. Your solution should be able to tolerate that *either* there is a corrupted server *or* a crashed sensor at an honest server. It need not tolerate that these two faults happen at the same time. Assume that the servers already agreed that an earth quake happened. Now they what to know *when* it happened. Since an earth quake takes time to propagate the detected time of the servers might differ by one time unit. We want them to agree on one of the possible two detection times. We can model the problem as follows.

Inputs Each party eventually gets input $t_i \in \mathbb{N}$. If honest P_i and P_j get input t_i and t_j then $|t_i - t_j| \leq 1$.

Termination Eventually each honest party P_i gives output w_i .

Validity If an honest server outputs w_i , then an honest server got the input w_i .

Agreement All honest servers output the same.



Exercise 10.17 (Multi-Valued ABA I) Read the general instructions at the beginning of the section. The problem at hand is as follows. There are $n = 3t + 1$ servers of which t might be Byzantine corrupted.

Inputs Each party eventually gets input $V_i \in \{0, 1\}^*$.

Termination Eventually each party gives output $W_i \in \{0, 1\}^*$.

Strong Validity If an honest server outputs W_i , then an honest servers got the input V_i .

Agreement All honest servers output the same.



Exercise 10.18 (Multi-Valued ABA II) Like *Multi-Valued ABA I* above but with:

Weak Validity If all honest servers have the same input V then no honest server outputs $W_j \neq V$.



Chapter 11

Privacy-Enhancing Technologies (2025)

Reading Guide *This chapter is structured as a collection of different techniques.*

Contents

11.1 Anonymous Communication	306
11.1.1 Onion Routing	306
11.1.2 Mix Networks	308
11.2 Deniability and Zero-Knowledge	309
11.3 Secure Multiparty Computation	312
11.3.1 Secret-Sharing	313
11.3.2 Homomorphic Encryption	314
11.4 Data Anonymization and Differential Privacy	316
11.4.1 k -anonymity and Limitations	317
11.4.2 Differential Privacy	318
11.5 Exercises	320

In the chapters on Confidentiality, Authenticity, and Key Exchange, you have been introduced to fundamental cryptographic tools such as encryption, digital signatures, and key exchange protocols. In particular, these chapters have shown how two mutually trusting parties can establish a secure communication channel to protect their communications from a malicious third party aiming to eavesdrop or tamper with their messages.

However, the situation is often more complicated than that. Over the internet, we do not necessarily trust those we are interacting with, so the threat model is more complex than in the basic scenario for secure communication. It is not clear that the “enemy” is necessarily a third party; it could be the very entity we are interacting with. At the same time, more and more data is produced, collected, and stored. It has been said that “data is the new oil,” and this is true in more than one way. The availability of large amounts of data is an invaluable resource for enabling complex data analytics and machine learning. These techniques can be used for good, helping individuals, companies, and institutions make smarter data-driven decisions. However, data is like oil in the sense that if it is not managed correctly, it can leak, with huge repercussions for the privacy of individuals and other sensitive information. Like oil, once data has leaked, it is very difficult—if not impossible—to put it back in the confidential domain.

Fortunately, research in privacy and cryptography can now offer a variety of advanced solutions to several cases where there is a tension between privacy and utility, by allowing many

advanced tasks (beyond simple communication) to be performed in a “privacy-preserving” way. These techniques come with many names, but a common one that has recently gained popularity is **privacy-enhancing technologies (PETs)**. PETs are slowly being integrated into everyday products and are, in some cases, being recommended in upcoming legislation. This chapter reviews a few of these techniques, focusing on those that can be explained given the cryptographic background you should have developed at this point in the book.

11.1 Anonymous Communication

We start by reviewing techniques for **anonymous communication**, which aim to ensure that not only the content of the communication is hidden, but also the identities of the sender and the receiver. In the Confidentiality chapter, we have seen how (symmetric-key and public-key) encryption can hide the content of communication. We now ask whether it is possible to hide *who is communicating with whom*. That is, we want to protect the identities of the parties involved, not just the message content.

The term for all information related to communication that does not include the content itself is **metadata**. Many have argued that metadata can be just as sensitive as the message content, yet metadata often lacks the same legal protections. For instance, when using (physical) mail, a postcard offers the lowest level of protection: anyone handling it can see the content. Using encryption turns your postcard into a sealed letter inside an envelope. The mailman cannot read the message, but they can still see the names and addresses of the sender and recipient on the envelope, as well as when the letter was sent.

To understand why metadata can be as sensitive as the content of communication, we can look at examples brought forward by privacy advocates following Snowden’s revelations. (Trigger warning: This example discusses sensitive topics, including suicide). The Electronic Frontier Foundation illustrates it as follows: *“They know you rang a phone sex service at 2:24 am and spoke for 18 minutes. But they don’t know what you talked about. They know you called the suicide prevention hotline from the Golden Gate Bridge. But the topic of the call remains a secret. They know you spoke with an HIV testing service, then your doctor, then your health insurance company in the same hour. But they don’t know what was discussed...”*

These examples show how revealing metadata alone can uncover deeply private details without ever accessing the message content. In fact, the former head of the NSA famously remarked, *“[...] we kill people based on metadata.”*

Returning to the mail analogy, you might think it is possible to hide the sender’s identity by not writing it on the envelope. However, you still need to drop the letter in a mailbox, which could allow it to be traced back to your general location. Hiding the recipient’s address, while still ensuring the letter is delivered, seems nearly impossible.

At this point, it should be clear that protecting metadata is a complex task.

11.1.1 Onion Routing

The first technique for anonymous communication we review is **onion routing**, which is used in practice by e.g., *The Onion Routing project*, commonly known as Tor. The “onion” part of the name refers to the fact that this technique uses several layers of encryption, which are peeled off one-by-one, in a way reminiscent of how an onion is made of many layers. The routing part refers to the fact that this “onion” is routed from the sender to the receiver through a network of servers (usually called **relays**) which offer their services to those interested in communicating anonymously. A relay is essentially just a server that receives encrypted messages (the onion),

decrypts them by peeling one layer off, and forwards them to the next relay as instructed, without knowing the full route or content.

From a very high-level point of view, and omitting many details, onion routing works as follows:

1. A selects some relays to route the communication. In practice often three relays are used, say S_1, S_2, S_3 each with a public-key pair (pk_i, sk_i) . Their public keys are available via a certificate infrastructure.
2. A prepares an onion of encryptions, where each layer of the onion contains the next recipient and a ciphertext encrypted using their public key:

$$C_1 = \text{Enc}(pk_1; S_2, \text{Enc}(pk_2; S_3, \text{Enc}(pk_3; B, \text{Enc}(pk_B; m))))$$

and sends it to the first relay S_1 .

3. S_1 receives C_1 , decrypts it with their secret key sk_1 as

$$(S_2, C_2) = \text{Dec}(sk_1; C_1)$$

and then forwards C_2 to S_2 .

4. The same process is repeated for the other relays: S_2 decrypts C_2 , obtaining (S_3, C_3) , and forwards C_3 to S_3 , who decrypts it and sends (B, C_B) to B .
5. B retrieves the message by computing $m = \text{Dec}(sk_B, C_B)$.

Looking at the protocol, you should notice that each relay only knows the previous hop and the next. S_1 knows that A sent a message via S_2 , but does not know the final recipient. Similarly, S_2 knows that S_1 and S_3 are involved, but is unaware of the sender and recipient. S_3 knows the message is destined for B , but has no information about A . As long as one relay remains honest, the link between A and B is concealed. For each new message, A can choose different relays.

The main weakness of onion routing is traffic analysis. For instance, if an adversary can observe both the incoming traffic at S_1 and the outgoing traffic at S_3 , and if only a single onion is being routed at that time, they can correlate the incoming traffic at S_1 with the outgoing traffic at S_3 , deducing that A is communicating with B .

Thus, onion routing is most effective when a large number of users are sending messages continuously. To further obscure traffic patterns, relays are instructed to delay and reorder onions, making it harder for attackers to link messages to specific users. These techniques, among others, deter traffic correlation attacks. For example, imagine that many onions are being continuously routed through the system at a high frequency. The adversary controlling S_1 might observe onions entering from several users, while other onions exit the system at S_3 . However, they would have no way to definitively link the onions entering S_1 with those exiting S_3 .

The description above simplifies several aspects, such as the exact details of how onion encryptions should be performed.

Moreover, the description assumes that B has a publicly available identity, such as a URL or IP address, since S_3 needs to know how to reach B . However, this does not account for situations where B wishes to remain anonymous as well. In such cases, B can register a hidden address in a public directory maintained by the relays, allowing A to initiate communication without knowing B 's real identity or location. This mechanism allows for both parties to stay anonymous during the entire exchange.

11.1.2 Mix Networks

The second technique we review are **mix-networks** (mix-nets). While onion routing is an asynchronous protocol that continuously processes messages as they are delivered, mix-nets operate synchronously. Messages from many users are processed and mixed together simultaneously, hiding the identity of the sender among all users participating in that round.

In more detail, suppose users $U_1, \dots, U_n$ want to anonymously publish messages $m_1, \dots, m_n$ respectively. A mix-net is a system composed of several servers. For concreteness, we will again use three servers, S_1, S_2, S_3 with public-key pairs (pk_i, sk_i) respectively. Then the process is as follows:

1. Each user U_i with message m_i submits an onion-encrypted message

$$C_{1,i} = \text{Enc}(pk_1; \text{Enc}(pk_2; \text{Enc}(pk_3; m_i)))$$

to S_1 .

2. S_1 decrypts the outer layer of all received messages:

$$C_{2,i} = \text{Dec}(sk_1; C_{1,i})$$

It then permutes the ciphertexts in a random order using a random permutation P and sends the resulting ciphertexts

$$C_{2,P(1)}, C_{2,P(2)}, \dots, C_{2,P(n)}$$

to S_2 .

3. The same process is repeated at S_2 and S_3 : S_2 decrypts all ciphertexts, randomly shuffles them, and forwards them to S_3 , which decrypts the final layer and retrieves the messages, publishing them in random order.

Let P, Q, R be the permutations applied by S_1, S_2, S_3 respectively. Then the results published by S_3 are¹

$$m_{P(Q(R(1)))}, m_{P(Q(R(2)))}, \dots, m_{P(Q(R(n)))}$$

As in onion routing, as long as at least one server is honest (and keeps its permutation secret), it is impossible to link the outgoing messages to the original users. For example, S_1 knows which user submitted which ciphertext, S_2 knows how the ciphertexts from S_1 are related to those it sends to S_3 , and S_3 knows which decrypted messages map to its inputs. However, no single server has enough information to correlate the inputs with the outputs across the system.

Mix-nets provide different, and in some sense stronger, anonymity guarantees than onion routing since they are less vulnerable to traffic analysis. However, they are often less practical because all users must send messages at the same time or, in practice, a user would need to wait for their message to be processed until enough other messages are available. For meaningful anonymity, a sufficiently large number of users needs to participate, which can lead to significant delays in communication. Despite these limitations, mix-nets are ideal for certain applications, as we describe now.

¹As described above, the protocol allows a set of users to publish messages anonymously. This design matches the use case in the next paragraph. However, if the application instead requires that the messages be delivered privately to a set of receivers, similar to onion routing, the protocol can be modified. To do so, replace each incoming message m_i with a pair consisting of the intended receiver's identity and a message encrypted under the public key of the receiver. In this case, S_3 would decrypt the final layer and deliver each message to its intended recipient.

Electronic Voting. Electronic voting is gradually gaining popularity. Although it is unclear whether electronic voting should replace traditional secret voting in all contexts, there are certainly cases where it provides a meaningful and effective alternative to paper ballots.

Electronic voting is a very complex topic. Defining appropriate security requirements, threat model, and solutions for an e-voting system could easily fill a book. At a minimum, an e-voting system must ensure **coercion resistance**, meaning it should not be possible for someone to force voters to vote a certain way. This implies **receipt-freeness**, meaning a user should not be able to prove to a third party how they voted, which could be used by a coercer. Another important property of e-voting is **verifiability**, which ensures voters can audit the system to confirm that their vote was counted and that the final tally is correct.

Mix-nets are well suited for e-voting systems because voting is inherently a synchronous process: all users cast their votes at the same time (e.g., on election day). Additionally, it is acceptable – in fact desirable! – to wait until everyone has voted before publicly announcing the result. Using mix-nets, each voter can cast their vote by encrypting their choice using an onion structure, as described above. Once all ballots are cast, the mix-net outputs the votes in a random order, allowing for an anonymous count of the votes.

However, mix-nets alone do not satisfy all e-voting requirements. For instance, a corrupted server in a mix-net could replace input ciphertexts with votes of its own choosing and manipulate the outcome of the election. This can be mitigated using tools like **zero-knowledge proofs**, which we introduce next.

11.2 Deniability and Zero-Knowledge

In the Authentication chapter, we discussed digital signatures and their important property of **non-repudiation**. This means that once a user signs a document or message, they cannot later deny having done so. Non-repudiation is essential in many applications. For instance, when you sign a contract, it ensures that you cannot later escape your obligations by repudiating your signature.

However, some applications have the opposite security requirement: users must be able to plausibly deny having participated in a specific communication or action, even if they were involved. This property is known as **deniability**. Deniability is especially important in contexts where users fear retaliation from powerful entities, which may prevent them from taking actions that would otherwise be necessary.

Examples include whistleblowing, which is the practice of reporting illegal or unethical activities within an organization (such as those committed by superiors), freedom of speech under oppressive regimes, protection of journalistic sources, and electronic voting, where deniability is crucial to prevent coercion.

Additionally, note that whistleblowing systems are legally required for many large companies in Europe under whistleblower protection legislation.

Signature-Based Identification. Consider a user logging into a system (for instance, a whistleblowing platform) and needing to do so in a deniable way. First, let's examine the shortcomings of a solution based on a challenge-response mechanism, similar to the one described when addressing countermeasures against replay attacks:

1. A user U with public-key pair (pk, sk) wishes to log in to a server S , which knows the user's public key pk .
2. The user sends a login request to S containing their username U .

3. The server responds with a random, sufficiently long challenge e .
4. The user signs the tuple (U, S, e) using their secret key as $s = \text{Sig}(sk, (U, S, e))$.
5. The server verifies the signature using the public key, checking whether

$$\text{Ver}(pk, s, (U, S, e)) \stackrel{?}{=} 1$$

This protocol provides several useful security properties. The signature binds the user's identity, the server, and the random challenge together, offering strong protection against impersonation, man-in-the-middle, and replay attacks. However, the protocol also leaves a “trace,” specifically the user's signature s . As a result, the server can prove to a third party that the user logged into the system, which may expose the user to retaliation.

Deniable Identification. We now describe a deniable identification protocol using techniques from discrete-logarithm-based cryptography, introduced in the Key Exchange section of this book.

In this system all users will use the same discrete-logarithm parameters (g, p) where, p is large enough prime number, and g is a “generator of order $p - 1$ ”. This means simply that you can get any number between 1 and $p - 1$ by computing the powers of g e.g., the following two sets are equal (meaning they have the same elements, not necessarily in the same order):

$$\{1, \dots, p - 1\} = \{g^1 \bmod p, \dots, g^{p-1} \bmod p\}$$

In particular note that it will always be the case that $g^{p-1} \bmod p = 1$.

Each user in the system has their own public-key pair of the form $(pk, sk) = (h, x)$, where $h = g^x \bmod p$, and the value x is a random integer between 1 and $p - 1$. The deniable identification protocol proceeds as follows:

1. The user U picks a random value r between 1 and $p - 1$, computes $a = g^r \bmod p$ (called the **commitment**), and sends it to the server.
2. The server responds with a random **challenge** e , selected uniformly between 1 and $p - 1$.
3. The user replies with the **response** $z = e \cdot x + r \bmod p - 1$.
4. The server verifies the identification attempt by checking whether

$$h^e \cdot a \bmod p = g^z \bmod p$$

To see why the server accepts the correct response, observe that:

$$g^z \bmod p = g^{e \cdot x + r \bmod (p-1)} \bmod p = (g^x)^e \cdot g^r \bmod p = h^e \cdot a \bmod p$$

The protocol is secure and prevents impersonation attacks because only the owner of the secret key can consistently generate valid responses. The intuition behind this is as follows: A user who does not know the secret key can only successfully log in with a very low probability, namely $1/(p - 1)$. For instance, an adversary can try to impersonate a user by sending a commitment $a = g^r$, just as the honest user would do, and then hope that the server chooses the challenge $e = 0$. In this case, the correct response z is $z = 0 \cdot x + r = r$, allowing the adversary to impersonate the user. This is a trivial case where the response does not depend on the secret key, but this approach can be generalized to any value of e .

In general, an adversary hoping for a specific challenge e can prepare a fake random response z , and choose the corresponding commitment a as:

$$a = g^z \cdot h^{-e} \bmod p$$

Plugging this value into the verification equation will make the server accept. It can be shown that this is the best an attacker can do. In particular, it can be proven that an attacker who can prepare themselves to respond to even *two different* challenges e, e' must necessarily know the secret key.

To conclude, the protocol is secure against impersonation attacks for the following reason: even though, as argued above, anyone who can predict the challenge e can produce an accepting transcript, the server will only accept the identification of an dishonest user with very low probability. The key point is that the server selects e uniformly at random *after* it has received the commitment a . This means that even if an adversary had prepared responses to some challenges e' , the actual challenge e chosen by the server will match e' only with very low probability $1/(p-1)$ (remember that p is thousands of bits long in discrete-logarithm based systems).

Thus, unless the adversary can guess the challenge correctly in advance, the server will not be persuaded by their attempt to impersonate an honest user.

The protocol also ensures deniability, which is the key feature we aimed to introduce. This deniability holds in situations where the server behaves honestly during the protocol execution but is later coerced by an adversary to reveal the transcripts of the interaction between the user and the server. As discussed earlier, anyone who can predict the challenge e can generate an accepting transcript of the protocol *without knowing the secret key*. This implies that simply observing the transcript of a login between a user and a server does not constitute proof that the user actually participated in the protocol. Since anyone could have generated an accepting transcript, it is just as plausible that the one accusing the user of participation might have created the transcript themselves.

(See the exercises for an analysis of how a corrupt server could potentially break the deniability of this protocol.)

From Deniable Authentication to Zero-Knowledge Proofs. The key idea behind the protocol described above is that the interaction between the user and the server reveals **zero-knowledge** about the user's secret key. Formally, this means that the server could have **simulated** the same protocol transcript by itself, without knowing the secret key. Therefore, an accepting protocol transcript reveals no information about the secret key.

By contrast, the challenge-response protocol based on digital signatures does not have this property. In the case of signatures, a server that knows only the user's public key cannot produce an accepting transcript, even by guessing the challenge in advance, because the transcript contains an unforgeable signature created with the user's secret key.

This idea of a protocol being zero-knowledge—because anyone could simulate the transcript without access to the secret—is incredibly powerful. It forms the basis of many advanced cryptographic protocols, especially those called **zero-knowledge proofs**. A zero-knowledge proof is a cryptographic proof that demonstrates knowledge of some secret without revealing any information about the secret itself.

In fact, the identification protocol described earlier can be viewed as a zero-knowledge proof that the user knows their secret key. This concept can be generalized to prove knowledge of any secret while maintaining zero-knowledge properties.

Zero-knowledge proofs have a wide range of applications in cryptography. For instance, in the e-voting scheme based on mix-nets (described in the previous section), zero-knowledge proofs

can be used to allow each server to prove that its output ciphertexts are simply a permutation of the decryptions of its input ciphertexts. This enables public verifiability of the election results without compromising privacy.

Another important application of zero-knowledge proofs is in **anonymous credential systems**, where users can prove certain attributes about themselves without revealing unnecessary personal information. Consider, for example, a user trying to purchase an age-restricted item, such as a bottle of wine, online. The law requires the webshop to verify the user's age, but a straightforward solution like uploading a photo of their passport would expose unnecessary private details (e.g., date of birth, place of birth).

A more privacy-preserving approach involves using anonymous credentials, where users store a signed and encrypted copy of their personal data, such as their date of birth, under the authority of a trusted public institution (e.g., the government). In this system, the user's data is digitally signed by the government using its secret key, verifying the authenticity of the information. When a user needs to prove their age (for example, to purchase an age-restricted item), they can present a zero-knowledge proof showing that they are above 18, without revealing any additional information about themselves, such as their exact birthdate or any other personal details.

A notable example of zero-knowledge proofs are **SNARKs** (Succinct Non-interactive Arguments of Knowledge), which are highly efficient cryptographic tools. SNARKs allow users to prove arbitrarily complex statements about secret data with proofs that remain fixed in size—sometimes as small as a few hundred bytes—regardless of the complexity of the statement being proven.

11.3 Secure Multiparty Computation

As we have seen, zero-knowledge proofs are a powerful tool that allows a user to prove that some secret value known only to them satisfies a certain property (for example, a ciphertext contains their age, and their age is above 18).

But what if several users, each with some secret data, want to jointly compute a function on their combined data without revealing their individual inputs to one another? This is the fascinating topic of **secure multiparty computation (MPC)**.

More specifically, MPC allows a set of mutually distrusting participants $P_1, \dots, P_n$, each with secret inputs $x_1, \dots, x_n$, to jointly compute the output of some function $z = f(x_1, \dots, x_n)$ without having to disclose their data to each other or to any third party.

MPC is an incredibly powerful technology that enables joint computations in scenarios where users may benefit from sharing data but are prevented from doing so due to privacy concerns, intellectual property protection, or privacy regulations.

The first practical use case of MPC was a pilot project in Denmark in the early 2000s. MPC was used by Danish farmers to discover the market value of certain farming commodities (in this case, the right to produce and sell sugar beets) without revealing their private data to any third party. Traditionally, this price discovery process involved all participants sharing their buying and selling preferences (e.g., “I am willing to buy X at price Y”) with a central third party, which would then calculate the market price based on the participants' inputs. Disclosing such preferences, however, is sensitive, as it can reveal internal financial details and provide a competitive advantage to others who gain access to this information. By using MPC, the same price discovery could be conducted without any private data being disclosed.

There are many other examples of MPC being used in privacy-preserving real-world applications. For instance, a project in Boston used MPC to collect salary data from various companies to quantify the gender wage gap in the region. Companies were unwilling to share

salary information due to its sensitive nature, but MPC ensured that no individual salary data was exposed, while still allowing the computation of meaningful statistics. Thus, MPC enabled public benefits from computations on sensitive data while maintaining the privacy of participants.

At first glance, it may seem impossible for two or more parties to jointly compute a function on their inputs without revealing any information about their data to each other. When the concept of MPC was introduced in the late 1980s, it was illustrated with a classic toy example called the *millionaire's problem*: Suppose two millionaires meet and want to determine who is wealthier without disclosing their exact wealth. How can they compare their wealth in a privacy-preserving way?

In the following, we will review some simple techniques for MPC to provide intuition about how it is possible to compute joint functions while preserving the privacy of individual inputs.

11.3.1 Secret-Sharing

A central tool in MPC is **secret-sharing**, which involves splitting a secret into parts in such a way that only when enough parts are brought back together can the secret be reconstructed. Importantly, with an insufficient number of parts, no information about the secret is revealed. The simplest form of secret-sharing is **additive secret sharing**, which can be seen as a generalization of the one-time pad encryption scheme from the Confidentiality chapter.

To illustrate, suppose a party P_1 wants to share their secret input x among a group of participants $P_1, \dots, P_n$. For simplicity, assume that x is a number between 0 and $B - 1$, where B is a known upper bound. The secret-sharing process works as follows: P_1 generates random numbers $x_2, x_3, \dots, x_n$, each uniformly chosen between 0 and $B - 1$, and distributes these shares to the corresponding parties $P_2, \dots, P_n$. To maintain consistency, P_1 computes their own share as:

$$x_1 = x - x_2 - x_3 - \dots - x_n \bmod B$$

At this point, the individual shares $x_1, x_2, \dots, x_n$ contain no useful information about the original secret x . They appear to be random numbers when viewed individually. However, when combined, they allow the participants to reconstruct the original secret:

$$x = x_1 + x_2 + \dots + x_n \bmod B$$

This method ensures that no single participant or any subset of participants can discover the secret unless they collectively bring together all the shares.

Secure Computation Using Secret-Sharing. Now, suppose that another party, P_2 , also has an input y and the parties jointly want to learn the sum $z = x + y \bmod B$ without either party revealing their secret input. Here's how they can achieve this using secret-sharing: P_2 can secret-share y in the same way as P_1 shared x , sending each party a random share y_i such that $\sum_i y_i \bmod B = y$. Then when each party has their own share of x and y , they can compute a new share for the sum z by simply adding their individual shares:

$$z_i = x_i + y_i \bmod B$$

After computing this, each party can publish their shares of the sum. Then, the parties combine all the shares to reconstruct the final result as $z = \sum_i z_i$. This produces the correct result since:

$$z = \sum_i z_i \bmod B = \sum_i (x_i + y_i) \bmod B = \sum_i x_i + \sum_i y_i \bmod B = x + y \bmod B$$

Thus, the parties jointly compute the sum of their inputs without learning anything about each other's private values. The protocol is secure because the published shares are just random numbers that add up to the correct result, and no individual share leaks information about the inputs.

Thanks to the commutative property of modular addition, this protocol can be easily extended to allow more than two parties to compute the sum of their inputs securely. This approach can, for example, be used to implement a privacy-preserving polling system. Suppose a municipality wants to ask a group of citizens whether they are in favor or against a proposal. Each citizen can secret share a vote of 1 if they are in favor or 0 if they are against. Using the protocol for summing secret shares, the tally can be computed by summing the votes, revealing only the total number of votes in favor, while keeping each citizen's individual preference private.

However, while this protocol preserves privacy, it does not guarantee the integrity of the results. A malicious participant could, for instance, secret share a value other than 1 or 0 (e.g., 10 or -10), thereby biasing the poll in their preferred direction. Solutions to this problem exist but they are beyond the scope of this course.

Extending the Protocol. While the protocol described above can be useful for some applications, addition alone is not sufficient to compute all functions securely. For instance, one cannot solve the millionaire's problem (that is, deciding whether x is greater than y or not) using linear combinations only. In order to compute more complex and expressive functions, we also need to be able to perform secure multiplications. Protocols for secure multiplications exist but require interaction between the parties and additional assumptions, and will not be presented here.

Once both addition and multiplication can be performed securely, we can compute any function by transforming it into an arithmetic circuit. An arithmetic circuit is a way of expressing a computation as a sequence of addition and multiplication operations. Each gate in the circuit represents either an addition or a multiplication, and the inputs and outputs of the gates correspond to the values held by the parties.

To securely evaluate an arithmetic circuit using MPC, the parties perform addition and multiplication on their shares for each gate in the circuit. Once all the gates are evaluated, the final output can be reconstructed from the shares.

Most protocols for MPC require to transform the function that one want to evaluate into an arithmetic (or Boolean) circuit. Note that when transforming a high-level program (such as a Go program) into an arithmetic circuit for secure computation one has to be careful. Note in particular that it is not straightforward to use common control structures like if-statements or for-loops with variable lengths. This is because the flow of the computation, such as the number of iterations in a loop, might reveal information about the inputs. In particular, revealing that one branch of an if-statement was taken or that a loop terminated early could leak valuable information about private data.

To prevent information leakage through the computation flow, most MPC protocols require that loops be executed for a fixed number of iterations, regardless of the input, and that all branches of if-statements be executed. While this might seem inefficient, it is necessary to hide the flow of the computation and ensure that no intermediate step leaks information about the private inputs.

11.3.2 Homomorphic Encryption

A very powerful tool for privacy-preserving computation is **homomorphic encryption**. Homomorphic encryption is a public-key encryption scheme that allows not only the encryption and

decryption of data but also manipulation of the underlying plaintext while it remains encrypted. This property enables meaningful computations on encrypted data without revealing the data itself.

More precisely, given an encryption $c = \text{Enc}(pk, x)$ and some function f , a homomorphic encryption scheme allows the computation of a new ciphertext $c' = \text{Eval}(f, c)$, which encodes the result $f(x)$ of applying f to the plaintext x , without the need to decrypt c .

A useful analogy for homomorphic encryption is that it works like a *glove box*. A glove box is a box with gloves attached, often used to handle hazardous materials (like chemicals or radioactive elements) or elements that must be manipulated in a sterile environment. Using a glove box you can manipulate the object inside the box by using the gloves, but without ever taking the object out. Homomorphic encryption is like a glove box which has been painted completely in black: you can manipulate the object inside the box (the encrypted data), but you cannot see the content of the box.

A Basic Example: RSA’s Homomorphic Properties. The insecure version of the RSA encryption scheme from the Confidentiality chapter already offers a basic form of homomorphism. Given two ciphertexts $c_1 = x_1^e \bmod n$ and $c_2 = x_2^e \bmod n$, it is possible to compute the product of the plaintexts by simply multiplying the ciphertexts:

$$\text{Enc}(pk, x_1) \cdot \text{Enc}(pk, x_2) \bmod n = c_1 \cdot c_2 \bmod n = (x_1 \cdot x_2)^e \bmod n = \text{Enc}(pk, x_1 \cdot x_2 \bmod n)$$

which results in an encryption of the product of the plaintexts, $x_1 \cdot x_2 \bmod n$. However, this is a very limited form of homomorphic encryption since it only supports multiplication, and most importantly, it is completely insecure for general use (as this basic version of RSA is not randomized).

Fully Homomorphic Encryption. Modern homomorphic encryption schemes go far beyond this limited example: it is possible to build encryption schemes that can evaluate more complex functions on encrypted data, including both addition and multiplication. Some schemes can evaluate any function (and are known as **fully homomorphic encryption scheme**) but are not very efficient in practice. However, if one is only interested in handle functions of bounded complexity, there exist homomorphic encryption schemes which offer reasonable efficiency.

Secure Outsourcing. One important application of homomorphic encryption is in **secure delegation of computation**, also known as **secure outsourcing**. Today, it is common to outsource computationally intensive tasks to external cloud computing platforms. With homomorphic encryption, a user can encrypt their data before uploading it to the cloud. The cloud can then perform computations on the encrypted data and return the result to the user in an encrypted format, without ever seeing the original data. This allows a computationally weak device to outsource a heavy computation to an external server without compromising the privacy of the inputs.

Private Function Evaluation. Homomorphic encryption can also be used in MPC tasks where several parties want to jointly compute some functions of their inputs in a privacy-preserving way. A special example of this is **private function evaluation**, which has applications in privacy-preserving machine learning. Consider a user who has private data x and wants to run it through a machine-learning model M (e.g., a neural network) owned by a company. For instance, x could represent sensitive health data, such as the user’s DNA sequence, and the company might have developed a method to predict certain diseases based on this data.

The challenge lies in balancing the user’s need to keep their sensitive data private and the company’s desire to protect its machine-learning model M , which is likely an intellectual property. Using homomorphic encryption, the user can send an encrypted version of their data $c = \text{Enc}(pk, x)$ to the company. Thanks to the properties of homomorphic encryption, the company can compute $c' = \text{Eval}(M, c)$ —an encryption of the result of applying the model M to x —without ever decrypting c or learning x . The company sends back the encrypted result c' to the user, who can then decrypt it and obtain $M(x)$.

With the right homomorphic encryption schemes, the privacy of the company’s model M can also be guaranteed. In such a case, the user only learns the result $M(x)$ and gains no additional information about the model itself. This preserves both the privacy of the data and the intellectual property of the company.

11.4 Data Anonymization and Differential Privacy

So far we have described several techniques that allow multiple parties to jointly compute some functions of their inputs while keeping their data private, but we have not really defined what we mean by “private”. Intuitively, the guarantee we have achieved so far is that only the result of the computation is revealed, and nothing else about the inputs.

However, there are situations in which this is not enough. Consider again the example of securely evaluating how many users vote “yes” to a certain proposal. As seen, this could be securely implemented using, e.g., mix-nets, MPC protocols based on secret-sharing, or more.

Now, consider the setting in which a lecturer uses a privacy-preserving polling system like the ones described earlier to ask a sensitive question to their students, such as how many students have ever cheated on an exam. The lecturer learns that x students said “yes”, but learns nothing about who voted “yes” thanks to the properties of the privacy-preserving protocol. After a couple of weeks, a new student joins the class, and the lecturer performs a new privacy-preserving poll to include the new student. This time, the lecturer learns that $x + 1$ students voted “yes”. Despite the use of a privacy-preserving protocol, the lecturer learns that the new student voted “yes”.

In this section, we discuss several methods that have been proposed for preserving the privacy and anonymity of individuals whose data is used for data analytics and statistical purposes.

How not to do it – re-identification attacks. Companies and institutions often collect large amounts of sensitive data about individuals. There is no doubt that this data can be extremely valuable for research and decision-making. For instance, learning that a certain disease is disproportionately common in a specific geographical area might help detect an otherwise unknown source of pollution. Thus, there is widespread interest in making this data available to researchers. However, this must be done in a way that preserves the privacy of individuals.

A tempting solution is to think that one can simply anonymize the data by removing fields that clearly identify individuals. For example, a hospital might assume that releasing a database of patient records, after removing names, social security numbers, and addresses, would be sufficient to ensure that the data can be used for societal benefit while protecting the privacy of individuals.

This is exactly what happened in 1997 in Massachusetts, USA, where the government released a database of anonymized hospital records. However, an MIT researcher, Latanya Sweeney, quickly demonstrated that using other publicly available information about citizens (in this case, the public voter register), it was possible to de-anonymize 87% of the individuals in the database. In particular, Sweeney was able to identify the governor of Massachusetts’

hospital records by cross-referencing information such as date of birth, gender, and postal code with the voter registry.

The “attack” is simple yet powerful. Even though the hospital had removed the most sensitive information, the combination of seemingly non-sensitive data (such as postcode and year of birth) was enough to uniquely re-identify a large fraction of the users. By combining this data with the public voter registry, it became possible to connect individuals’ names to their supposedly anonymized records.

This is only one example; other notable examples include the AOL search data scandal in 2006, where search queries released for research purposes were traced back to individuals. In one case, reporters from *The New York Times* were able to identify Thelma Arnold, a 62-year-old woman from Georgia, by examining her search queries, which included specific queries about her local area, medical conditions, and even searches related to her dog’s health. Similarly, in the Netflix Prize dataset released in 2006, researchers were able to re-identify users by cross-referencing their movie ratings with data from other publicly available sources (e.g., ratings on IMDB). These cases highlight how even seemingly anonymized datasets can still compromise privacy when combined with other information.

11.4.1 k -anonymity and Limitations

A first attempt to prevent this kind of re-identification attack is to ensure that no records in the database are “unique enough.” This leads to the notion of k -anonymity. In a nutshell, a database satisfies k -anonymity if the information of each record cannot be distinguished from the information in at least $k - 1$ other records. There are many algorithms to achieve k -anonymity, and some of the basic techniques include **generalization** and **suppression**.

Here is a small example to make this more concrete. The original database could be:

Name	Age	ZIP Code	Job Title	Favorite Activity
Alice	24	8200	Software Engineer	Biking
Bob	21	8210	Teacher	Gaming
Charlie	23	8240	Graphic Designer	Painting
David	29	2100	Software Engineer	Biking
Eve	28	2300	Manager	Gaming

The dataset contains *key attributes*, such as the name, which identify individuals directly. Clearly, this information needs to be removed to prevent direct identification. The dataset also contains *quasi-identifiers*, such as age, zip code, and job title. These do not uniquely identify individuals by themselves, but when combined, they can be used to re-identify individuals, especially when cross-referenced with external databases. Finally, the dataset contains a *sensitive attribute*, in this case the favorite activity. This is the sensitive attribute of interest to the researchers who want to use the data for statistical analysis and is therefore left untouched.

Then, the k -anonymized version of the dataset (with $k = 2$) could be:

ID	Age	ZIP Code	Job Title	Favorite Activity
1	20-25	82**	***	Biking
2	20-25	82**	***	Gaming
3	20-25	82**	***	Painting
4	25-30	2***	***	Biking
5	25-30	2***	***	Gaming

Here, we used generalization to reduce the specificity of age and zip code by replacing exact values with ranges, and suppression to remove the job title, which could have led to direct re-identification. Even if you already know David, someone in the age range 25-30 who lives in an area whose postcode starts with 2, you still have no information on whether David likes biking or gaming in his free time.

Limitations of k -Anonymity and Generalizations. While k -anonymity offers a level of protection by ensuring that each individual is indistinguishable from at least $k - 1$ others, it does not provide full privacy protection. One major issue is that k -anonymity does not protect against attacks where sensitive values within an equivalence class lack diversity. For example, assume David was also into gaming. Then the anonymized records for both David and Eve would be equivalent, so the attacker can still learn that attribute even though the group appears anonymized. Additionally, k -anonymity is vulnerable to other attacks when the attacker possesses background knowledge about the target, allowing them to narrow down the possibilities despite the anonymization. There are several generalizations of k -anonymity, such as ℓ -diversity and t -closeness, which attempt to address some of these limitations. However, all of these approaches suffer from similar issues: they focus on syntactic properties of the output dataset, rather than on the capabilities of the attacker.

Of course, the question of anonymity always goes hand in hand with utility. It is always possible to guarantee privacy by outputting *no* dataset at all. Thus, there is a balance between utility and privacy that must be carefully navigated.

11.4.2 Differential Privacy

Differential privacy is becoming the de facto standard for defining and achieving privacy in data collections and privacy-preserving data analytics. In a nutshell, a differentially private system injects statistical noise into published data to obscure individual contributions while preserving the utility of aggregate statistics.

Differential privacy has been used in several practical applications. Notably, it was implemented in the *2020 U.S. Census* to protect individual privacy while still providing valuable population statistics. Differential privacy has also been applied by major tech companies like *Apple* and *Google*. Apple uses differential privacy in its operating systems to collect anonymized usage statistics such as frequently used emojis, ensuring that data about individual users remains private. Similarly, Google uses differential privacy in services such as *Google Chrome* to gather browsing statistics while protecting user identities, and in *Google Maps* to anonymize shared location data while preserving its utility for generating traffic insights and other analytics, such as how popular certain locations are at different times.

In essence, differential privacy guarantees that whatever conclusion an adversary might draw about any individual, based on the released dataset, should only depend on that individual with a very small probability. More formally, let D_1 be a dataset that includes the data of some user U , and let D_2 be the same dataset where U 's data has been replaced with random fake data. Let $M(D_i)$ be the output of some mechanism on the dataset D_i , and let $A(M(D_i)) = 1$ represent the event in which the adversary A achieves their goal (for instance, learning some information about U). Then, ε -differential privacy dictates that:

$$Pr[A(M(D_1)) = 1] \leq (1 + \varepsilon) \cdot Pr[A(M(D_2)) = 1]$$

In words, this means that the probability that the adversary achieves their goal when given the outcome of the mechanism on dataset D_1 (containing U 's true data) is essentially the same

as when the adversary is given the outcome of the mechanism on dataset D_2 (where U 's data has been replaced with random fake information).

For example, if $\varepsilon = 0.01$, the probability that the adversary learns some information about any specific user will differ by at most 1% depending on whether the user's data is in the dataset or not. This makes users more willing to allow their data to be used for statistical purposes, as “it almost does not matter” whether they participate in the study.²

This guarantee holds in a strong sense, even against an adversary who knows all other entries in the dataset. For example, differential privacy would prevent the type of privacy breach discussed earlier, where a lecturer performs a poll among students.

Randomization and Noise. The definition of differential privacy implies that the mechanism M used for data anonymization must be randomized and that the output must include some form of noise. As an example, a differentially private mechanism for a yes/no poll would collect all individual votes, count the number of “yes” votes, and then add or subtract a random number of votes. The noise introduced should be selected carefully using a distribution that corresponds to the required privacy parameter ε . The goal here is to add as little noise as possible to ensure that the mechanism returns a result which is as close as possible to the true one, while adding enough noise to ensure that privacy is preserved. (We do not discuss further the kind of noise distributions used in differentially private mechanisms.)

Central vs. Local Differential Privacy. In the example above, all votes are collected by a central party that needs to be trusted to keep the votes secret and to add the right amount of noise. If no such central party is available, one can distribute the computation using MPC techniques, as described earlier, or use a mechanism satisfying **local differential privacy**. In local differential privacy, it is the users themselves who add noise to their inputs before releasing them for analysis. Local differential privacy has the advantage that no central entity is required, but it typically requires more noise (and thus less accuracy) compared to central differential privacy.

Using local differential privacy, the lecturer could conduct a privacy-preserving poll among students using a mechanism known as **randomized response**. Each student has a yes/no vote. With probability p , they flip their vote and then announce the result. For example, the mechanism might run with $p = 1/6$, meaning the student rolls a dice (without letting anyone else see the result). If the outcome is 1-5, they announce their real vote. If it is 6, they announce the opposite vote.

At first, this mechanism might seem counterintuitive since the students publicly announce whether they cheated or not, yet it is still called private. The key idea is that students report a noisy version of their input, providing **plausible deniability**. Even if a student reports “yes”, there is uncertainty about whether they actually voted “yes” and did not flip, or voted “no” but flipped due to the mechanism.

Now, let us consider the accuracy of this mechanism. Suppose $y\%$ have “yes” as their true input. In expectation, the outcome of the mechanism will be

$$(1 - p)y + p(1 - y) = p + (1 - 2p)y$$

In the case where $p = 1/6$, the fraction of “yes” votes will, in expectation, be $\frac{1}{6} + \frac{2}{3}y$, providing a reasonable approximation of y .

²For formal reasons, the actual definition of differential privacy uses the term e^ε instead of $(1 + \varepsilon)$ in the expression above. The expression e^ε approximates $(1 + \varepsilon)$ for small ε and offers several advantages when analyzing the composition of differentially private mechanisms and proving privacy guarantees.

11.5 Exercises

Exercise 11.1 (Private Set Intersection) A popular secure multiparty computation task is private set-intersection (PSI). In a nutshell, in PSI, two users each have a set as input, say sets X and Y respectively. At the end of the protocol, both parties want to learn the intersection of their sets, $Z = X \cap Y$, and nothing else. PSI has many practical applications. One example is private contact discovery: when you download a new messaging app, the app might request access to your contact list to inform you which of your contacts are already using the app. Typically, the app may send your entire contact list to their server, where the server checks which phone numbers are registered users. This presents a privacy risk: why should the server learn about individuals in your contact list who are not using the service?

Private contact discovery addresses this problem by allowing users to discover which of their contacts are already registered on a service without disclosing information about other contacts. Clearly, PSI is a valuable tool for building privacy-preserving contact discovery systems.

Here's a simple attempt to compute set intersection in a privacy-preserving way:

1. The user has as input a set $X = \{x_1, \dots, x_n\}$. Using a cryptographic hash function H , the user computes the set $C = \{c_1, \dots, c_n\}$ where $c_i = H(x_i)$, and sends C to the server.
2. For each element $y \in Y$, the server concludes that $y \in X \cap Y$ (i.e., is in the intersection) if $H(y) \in C$.

Questions:

1. Explain why the protocol above is not a secure PSI protocol. How can the server learn more information than just the intersection of the sets?
2. For concreteness, assume that all elements in X are 8-digit Danish phone numbers. How long would it take for the server to completely recover the user's input X ? (You are free to choose the cryptographic hash function used in the protocol.)
3. (Optional:) Can you describe a secure PSI protocol using homomorphic encryption?



Exercise 11.2 (Number of Servers in Anonymous Communication) In the chapter, for concreteness, we used three servers S_1, S_2, S_3 for both onion routing and mix-nets. However, different numbers of servers could have been chosen as well.

Questions:

1. **Fewer servers:** Can you use fewer servers (e.g., one or two servers)? What would happen to the anonymity guarantees if the number of servers is reduced? Could a single server be enough to ensure anonymity in onion routing or mix-nets?
2. **More servers:** What would be the impact of using more than three servers? Consider the trade-off between anonymity and performance.



Exercise 11.3 (Security of e-Voting Systems) Many practical e-voting schemes allow users to cast their vote multiple times, ensuring that only the latest vote cast by any user is counted in the final tally. At first, this might seem strange, as multiple voting is not typically allowed in

physical paper-based voting. Moreover, this approach usually increases the complexity of the cryptographic protocols used in e-voting.

Questions:

1. What could be the reason for this design choice in e-voting systems?



Exercise 11.4 (Groups (Algebra)) Groups are very useful algebraic abstractions when working with cryptography, especially in the case of RSA or discrete-logarithm-based systems. A group is simply a set G of elements connected by some operation $\cdot$, satisfying some properties such as:

- *Closure*: The result of the group operation on any pair of group elements is still a group element. That is, if $a, b \in G$, then $a \cdot b \in G$.
- *Identity*: There is some special neutral element (usually called $1 \in G$) such that combining 1 with any element of the group results in the same element. That is, $a \cdot 1 = a$.
- *Inverse*: For each group element a , there is a group element a^{-1} such that $a \cdot a^{-1} = 1$.

It turns out that the set $\{1, \dots, p-1\}$ for prime p is a group with respect to multiplication modulo p . For instance, the set of numbers $\{1, \dots, 6\}$ forms a group under multiplication modulo 7.

When describing the deniable protocol for identification, we introduced the concept of a generator g of the group, a special element of the group such that if you take consecutive powers of the generator, you obtain every element in the group.

Questions:

1. Verify that 3 is a generator of the group, that is, that you can get all values $\{1, \dots, 6\}$ by computing iteratively the powers of 3 modulo 7.
2. Verify that 2 is not a generator of the group.



Exercise 11.5 (Attacks on Deniability) As described in the chapter, the identification protocol based on discrete logarithms is only deniable under the following threat model: *The server S is honest at the time of login with the user U and runs the protocol honestly. In particular, the server S picks the challenge e uniformly at random between $1, \dots, p-1$. Later on, the server leaks the transcripts to a third party, a verifier V .* As argued in the chapter, the verifier V has no way to determine whether the transcripts they receive are from a real interaction between the user U and the server S , or from fake transcripts simulated by the server itself.

Questions:

1. Consider the following strategy for the server: instead of choosing e at random, the server S chooses e as a function of a , the first message sent by the user (for instance, choosing $e = a$ or $e = H(a)$ for some cryptographic hash function H). Can the server now show these transcripts to the verifier to persuade the verifier that the user really participated in the protocol? In other words, could the server have simulated a transcript of this form? Why or why not?

(Note that the attack on deniability described in this section is also a common technique for turning interactive challenge-response protocols into non-interactive protocols. This technique is known as the *Fiat-Shamir* transformation.) ▼

Exercise 11.6 (Zero-Knowledge Proofs) Consider a user U with a public-key pair (pk, sk) wanting to prove knowledge of their secret key to a server S knowing the public key pk . Consider the following protocol:

- The server encrypts a random message r as $C = Enc(pk, r)$ and sends C to the user U .
- The user decrypts $r = Dec(sk, C)$ and returns r to the server.

Questions:

1. Does the protocol prevent impersonation attacks? Can the server be persuaded that the user really knows sk ? Why or why not?
2. Is this protocol deniable? Can the server prove to a third party that U participated in the protocol?
3. Do you think this protocol is zero-knowledge? Does the server learn anything they are not supposed to during the interaction? Could the server have simulated the protocol transcripts?

Consider both the case where the server behaves honestly or maliciously during the protocol. ▼

Exercise 11.7 (Deniability in Off-The-Record Messaging) Off-the-record messaging (OTR) is a popular cryptographic protocol that enables deniable communication. In a nutshell, and omitting many technical details, OTR starts with an authenticated Diffie-Hellman key exchange (DHKE), as described in previous chapters. After the key exchange, both A and B can derive two keys from the protocol: K_{enc} , a key for encryption, and K_{mac} , a key for message authentication (MAC). These keys are then used to ensure both confidentiality and authenticity during their communication.

Once the session is complete, both parties are instructed to delete the encryption key K_{enc} to protect the confidentiality of the conversation. Surprisingly, they are also instructed to *publish* the MAC key K_{mac} (for example, to a public repository on the web).

Questions:

- How does publishing the MAC key help with deniability?
 - Why is it safe to publish the MAC key but not the encryption key?
- ▼

Exercise 11.8 (Threshold Secret-Sharing) The simple additive secret-sharing described in the chapter offers very high confidentiality. Since $n - 1$ shares are chosen independently at random, the secret x can only be reconstructed when all n shares are brought together. In particular, any set of $n - 1$ shares contains no information about the secret. This holds in a strong, unconditional sense, similar to the security of the one-time pad.

Secret-sharing is often used in practice to store sensitive information, such as secret keys, passwords, etc. Using additive secret-sharing ensures that unless the adversary manages to steal

information from all n locations where the shares are stored, they will learn nothing about the secret.

However, additive secret-sharing is not a great solution if one is concerned with availability. Since all n shares are *necessary* to reconstruct the secret, if even a single share is lost or destroyed (either due to error or because an adversary destroys it), the secret x is lost forever. This motivates the need for secret-sharing schemes that can tolerate the loss of some shares. For example, consider a secret-sharing scheme that splits a secret into three parts, such that any two parts are sufficient to reconstruct the secret. This scheme still offers meaningful confidentiality (one share contains no information about the secret), but allows reconstruction even if one share becomes unavailable.

Questions:

1. Can you design a simple secret-sharing scheme that splits a secret into three parts and allows reconstruction given any two shares, and where each individual share contains no information about the secret?

Hint: A “share” does not need to be a single number; it can contain multiple components.

2. The scheme above splits the secret into $n = 3$ parts and allows reconstruction with any $t = 2$ parts. Does your solution generalize to arbitrary values of n, t ? Why or why not?



Exercise 11.9 (Homomorphic Encryption) Fully-homomorphic encryption allows for the evaluation of arbitrary functions on encrypted data. In this exercise, you are asked to design an “encrypted spam filter” system using fully homomorphic encryption. Multiple senders will send encrypted messages to a receiver. Before the messages reach the receiver, a service will process the encrypted messages and use a method M to classify whether the email should be marked as spam or not.

Questions:

1. Describe, at a high level, how the system would work.
2. Can the service move messages classified as spam into the user’s spam folder, while preserving the encryption?
3. How much work does the user need to do for spam messages in your system? What about for non-spam messages?



Exercise 11.10 (Differential Privacy of Randomized Responses) In this exercise, you will prove that the randomized response mechanism satisfies ϵ -differential privacy. As a reminder, the randomized response mechanism works as follows:

- Let $x \in \{0, 1\}$ be a binary response (e.g., yes/no).
- The true response x is flipped with probability p .
- The user reports $x' = x$ (the true response) with probability $1 - p$, and reports $x' = 1 - x$ (the opposite response) with probability p .

You are tasked with showing that this mechanism satisfies ε -differential privacy. Begin by recalling the definition of ε -differential privacy. You can choose whether to use the definition based on $(1 + \varepsilon)$ or e^ε . For simplicity, assume that the mechanism is applied to a single user, and the output is a single bit. Thus, you can assume the adversary's output is directly based on the mechanism's output. In this case, the simplified notion of differential privacy becomes:

$$\Pr[M(x) = 1] \leq e^\varepsilon \cdot \Pr[M(1 - x) = 1]$$

Write down the probabilities of the adversary observing $x' = 1$ and $x' = 0$ in both cases (i.e., when the true response is $x = 1$ and when the true response is $x = 0$). Use these probabilities to find the relationship between the probability of flipping p and the parameter for differential privacy ε .



Exercise 11.11 (Composition of Differential-Privacy) In this exercise, you will explore the concept of *composition* in differential privacy and how it affects the privacy guarantees when running differentially private mechanisms multiple times. Consider the scenario of a teacher running a privacy-preserving poll among their students to ask a sensitive question, such as: “Have you ever cheated on an exam?” The poll is conducted using the *randomized response* mechanism, where each student flips their vote with probability p to protect their privacy, as described in the chapter. A week later, a new student joins the class, and the teacher decides to run the poll again with the same class, including the new student. The students from the original class now become concerned that by releasing two noisy versions of their input, the teacher may be able to infer their true answer by combining the two results.

Questions:

1. Why might the students feel that their privacy is at greater risk after participating in two polls instead of just one? How could the teacher infer more information combining the two results?
2. The definition of differential privacy says that the probability that the teacher learns anything about any individual is at most e^ε larger than in the case where their record is replaced with fake information. You can now think of the two polls from the example above as a single differentially-private mechanism that takes as input the two datasets used in each poll. Thus, the new dataset contains two records for each student who participated in both polls (perhaps containing the same information). Can you use the definition of differential privacy to quantify the privacy of running both mechanisms? *Hint: Let D_0 be the joint dataset containing the real data. And let D_2 be the dataset where both records concerning user U are replaced with fake data. What if, for the purpose of the analysis, you introduce a dataset D_1 in which a single of the two entries has been replaced?*



Chapter 12

System Security Mechanisms (2025)

Reading Guide *This chapter introduces and reframes many concepts and terminology that were already implicitly defined before. The best way to navigate is to assimilate the earlier sections and treat the systems security mechanisms as components in a TCB spanning the whole system. The taxonomies later help qualifying and enumerating types of attack(er)s, and they make much more sense when applied to the attacks introduced in the last section. In other words, the best way to process the chapter is in reverse: when attacks are introduced in the end, try to understand how they can be classified with the taxonomies and how they violate assumptions and threat models, or how augmenting the TCB with security mechanisms can protect against them.*

Contents

12.1 Introduction	326
12.2 The Trusted Computing Base	327
12.3 Protecting communication	329
12.3.1 Packet Filtering	330
12.3.2 Proxy Firewalls	331
12.3.3 Stateful Firewalls	331
12.4 Protecting software	332
12.4.1 Malicious Software	332
12.4.2 Virus Scanners	333
12.5 Intrusion Detection	334
12.5.1 A possible classification	334
12.5.2 Limitations	334
12.6 Access Control	335
12.7 Categorizing attacks	336
12.7.1 What is the attacker's goal: STRIDE	336
12.7.2 What means are used in the attack: X.800	336
12.7.3 Where are we attacked, and by whom: EINO	337
12.7.4 Where did we make the mistake: TPM	338
12.8 Attacks and Vulnerabilities	338
12.8.1 Malicious Inputs and Parsing Errors	339
12.8.2 Microarchitectural Attacks	343
12.8.3 Insecure Library Interfaces	345
12.9 Exercises	346

12.1 Introduction

As mentioned in the first chapter of this book, we usually organize the way we think about security into the concepts of

Security Policy A specification of the system in question, a description of the security objectives for the system, and perhaps a high-level strategy for achieving the objectives.

Threat Model A description of the attacks we want the system to be protected against.

Security Mechanisms The technical solutions we use to reach our objectives.

Whereas the previous chapters have been about Security Mechanisms, and to some extent about Threat Models, we now turn to Security Policies. Here is a more detailed but still informal definition:

Definition 12.1 A security policy contains a description of the system that the policy applies to. It then describes the security objectives you want to achieve. This can take different forms: it can be a specification of what different entities are allowed to do and not allowed to do. It can also be a specification of which events we want to tolerate when the system runs. The system must then ensure, either that no entity can do what is not allowed, or that no event occurs that we do not want to tolerate. Finally, the policy may contain a high level (non technical) description of the methods that will be used to enforce the security objectives.

Note that it is of course important to say which system we want to protect, i.e., it makes no sense to define what security means for a system unless we also describe the system itself, at least to some level of detail. This also has the purpose of limiting the problem: we do not have to protect something that we have explicitly defined as being outside of our system. At the same time, *multiple* threat models can be defined for the same system, depending on what attacks are deemed more relevant. A system secure under one threat model is not necessarily secure under another one, which makes threat modelling one of the most important activities in the design phase of a secure system.

When designing secure systems, it is also common to document security aspects of a system from an *adversarial* point of view, in which the following concepts are useful:

Attack Surface A description of the parts of the system accessible to an attacker.

Attack Model A description of the conditions under which an attack would work or how it has worked in the past, in the case of a successful attack. This can also be seen as a subset of a threat model to elaborate a particular attack.

Documenting systems from an adversarial point of view is fundamental to exercise a *security mindset*.

An example: a smart safe. We will start with a concrete example of how to build and communicate an attack/threat model. Attack-defense trees (or graphs more generally) are a classical tool for documenting attack surfaces, threat models and adversarial goals. They are structured as a collection of nodes and directed edges representing characteristics of the system that influence how an attacker can achieve its objective. There are many different representation formats documented in the literature, but here we will stick with some conventions:

- The source node is *reality* itself. The sink node is the adversarial *goal*, i.e., when a successful attack is performed.

- The first layer is composed of facts derived from reality that the adversary can use in its favor. These include the attack surface of a system, common operational failures or physical limitations of system components.
- The intermediate nodes from the source to sink can be either attacks that can be mounted or security mechanisms that can be deployed as countermeasures.
- The edges indicate a logical progression between an attack and the corresponding countermeasure. Deploying a countermeasure can lead to another attack of lower complexity, which is sometimes represented as a dashed (back)-edge.

Now we will apply these ideas to a hypothetical example. Imagine a *smart* safe, in the sense that the safe is built from some sturdy physical material but operated by a combination of software and hardware – a classic embedded system. Opening the safe requires inputting the correct combination through one of its interfaces, for example a touch screen or provided remotely by an application running on a mobile device. We can assume that such an application communicates with the embedded system through a wireless network mechanism. The adversarial goal is to open the safe, reaching the sink node using one of the possible attack venues formalized in the graph. Notice that some of the deployed countermeasure push the attacker to attempt attacks with lower complexity but higher cost, and ultimately there are a few attacks that cannot be fully protected against, other than mitigated. The figure below, drawn using the Deciduous tool (<https://www.deciduous.app/>), represents the idea.

Security Policies are closely connected to some of the elements of system security. In the next section, we introduce the trusted computing base (TCB), the “incorruptible” part of a system that is required to protect against even privileged attackers. The security policy can be thought of as the place where we formulate the rules that the TCB should enforce.

12.2 The Trusted Computing Base

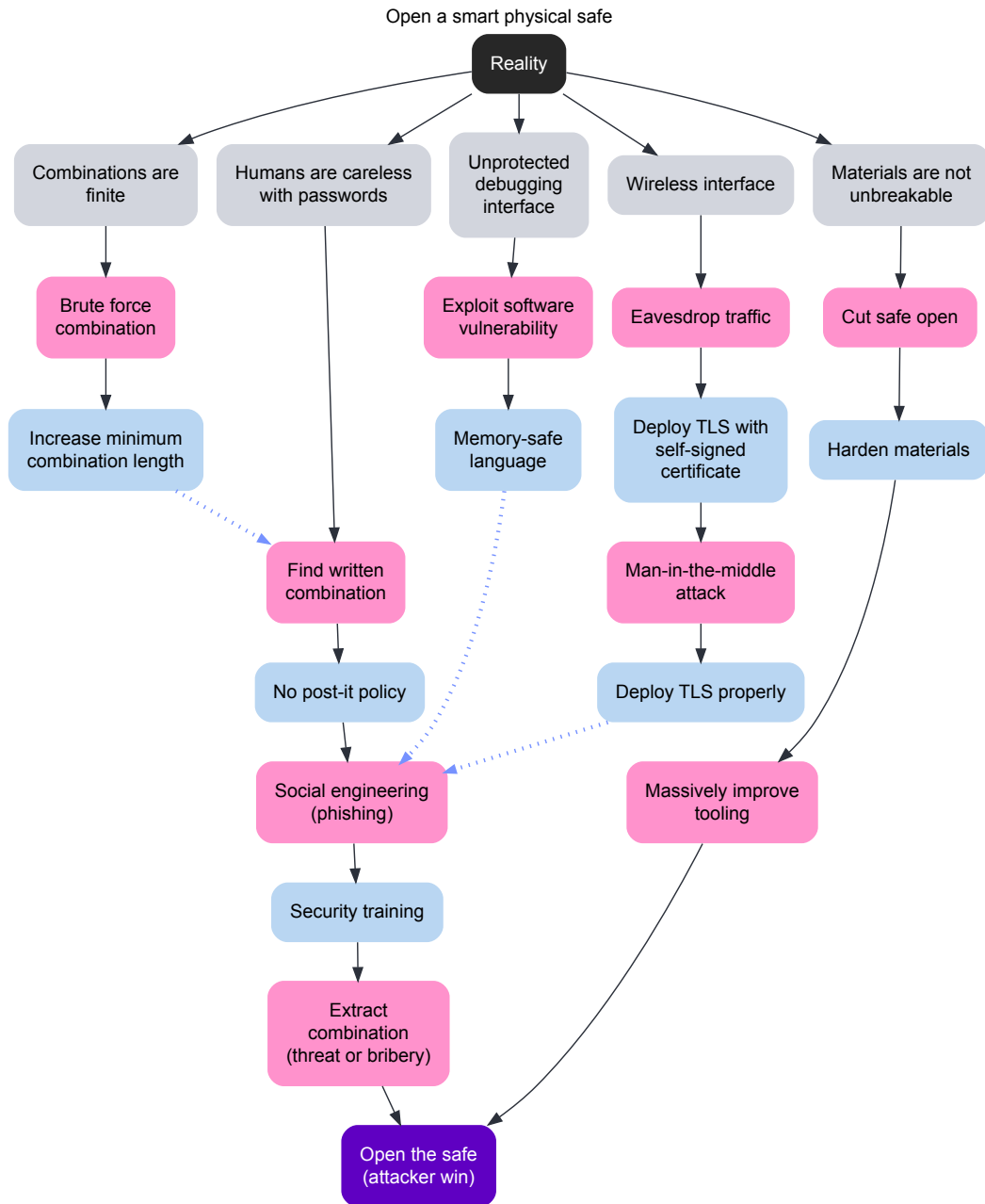
Whereas network security is about securing communication from one system to the other, system security is about defensive mechanisms that try to prevent external attackers from getting into the system, and also about mechanisms for preventing attacks from parties who are already users of the system.

So the typical scenario here is that some party wants access to some part of a system, and we want to ensure that access is only given in cases where this is acceptable with regards to the security policy. Here access should be understood broadly: getting *access* to a machine does not necessarily mean *physical* access, it could also just mean that I am allowed to communicate with it or execute some code on it.

Clearly, one needs to be able to identify the party who wants access, and we already discussed several issues that are relevant for identification. For instance, password security, hardware tokens and biometrics, but also authenticated key exchange is a way to ensure that we know who we are talking to.

However, identification is not enough. A secure system also needs to have some part that takes the decision on whether some action should be allowed according to the security policy, and also enforces the decision: if access is denied it should not be possible to circumvent the system and get access anyway.

It is obvious that we need to assume that this part of the system cannot be modified by attackers or malicious users. This part of the system is therefore called the *trusted computing base* (TCB), i.e., it is some part or parts of the system that is trusted to perform according to specification, even under attack.



A TCB can be established, for instance, using hardware security and/or software security, depending on the threat model you are in. Concretely, the TCB will often be part of the operating system's kernel, but it may also consist of one or more hardware units. So one should not expect that the TCB is physically located in one place.

Analysis of a system very often starts by trying to identify what part(s) of the system act as TCB, and estimating whether they can be assumed reliable enough under the assumed threat model.

In the following, we look first at how to protect a system from attacks coming from the outside and then turn to security policies for controlling the system's behavior towards users with privilege (*insiders*). All this can be thought of as descriptions of how different parts of the TCB can be designed.

An example: TEEs. An example of a hardware-based TCB is a Trusted Execution Environment (TEE), a hardware-protected area on the main processor that is capable of executing instructions and securely storing a small amount of data. TEEs guarantee that data and code loaded inside are protected with respect to confidentiality and integrity, by making it infeasible to read internal data from the outside (through hardware-based memory encryption); and preventing unauthorized parties to replace or modify its code, ensuring that it behaves exactly how it was specified. In the case that a TEE needs to store data outside its secure memory, it will then encrypt and authenticate it before it is sent out.

A TEE has an ambitious threat model, being designed to be secure even against high-privilege processes running in the same machine, effectively protecting it against the machine's owner; and to produce evidence that this is indeed the case. TEEs are sometimes called *secure enclaves*, and there are several implementations of TEEs deployed in practice, with Intel's SGX (Software Guard Extensions) being the most famous. Other notable examples are ARM TrustZone, from which many other TEEs are built from; and Apple's secure enclaves in iOS.

There are many possible applications: for instance, an enclave can be set up to only execute software that has been signed by certain parties. It also holds private signature and encryption keys so it can execute a program P and produce a signed version of the output. This means other parties would get evidence that the data they see came from a correct execution of P , even if they do not trust the owner of the involved machine. The same type idea can also be used for what is known as *remote attestation*: a software vendor could send code to a machine it does not fully trust, perhaps with essential parts of it encrypted. The code would then run inside an enclave which would be able to check that the software is genuine (against a signature from the vendor) and could also provide an anonymized proof that a genuine enclave has executed the program. One can think of this as a signature that only proves that some enclave authenticated the data, it does not show which enclave did it, as to protect user privacy.

All this holds great promise in many ways, but one has to remember that the idea also has certain inherent weaknesses: First of all, the integrity of the whole system hinges on the integrity of the hardware vendor. For instance, they are the only ones who can install the initial signature key in an enclave and distribute the corresponding public key. Second, when an enclave uses external memory which is often necessary due to the small size of internal memory, the access pattern can be observed, even if the data itself is encrypted. Being able to see which parts of its memory a program accesses and when can often reveal a lot of information. Finally, security also depends on the specific implementation details. For example, in 2018 a serious attack against SGX was published, the so called Foreshadow attack. It is closely related to the Spectre and Meltdown attacks that we will take a closer look at later. It shows that current implementations of SGX are not secure and it is possible to read secret keys from an enclave - which of course completely breaks the idea of remote attestation. This does not mean that the *idea* of SGX is fundamentally flawed, but is a symptom of the fact that Intel seems to have put too much priority on speed over security in the current design. SGX is listed as "deprecated" in recent generations of Intel processors, and therefore is not supported in latest non-enterprise processors.

In the following sections, we will examine several examples of TCB components designed to protect software or communication in various ways.

12.3 Protecting communication

We have seen many ways to set up secure channels over insecure networks, but this actually does not necessarily help the overall security of a system. Of course, by setting up a secure channel between A and B , we can make sure that no third party can break into the communication. But

even if A will only set up secure communication with parties it knows and trusts, the Internet is an open place where anyone can at least try to communicate with anyone else. At the start of the communication establishment, A of course does not know who he is talking to, in fact the purpose of an authenticated key exchange is to make sure who the other party is. Conclusion: even a computer who insists on doing authenticated key exchange with anyone who tries to connect will have to physically do some amount of communication with potentially untrusted parties.

This opens the possibility that an adversary can hack into A 's computer, using for instance an overflow attack (which we discuss in detail later in the chapter). With current state of the art, it is very difficult to make sure that such hacks are *never* possible. This sets the stage for the simple idea behind the *firewall* concept: the adversary cannot break into a computer he cannot communicate with.

12.3.1 Packet Filtering

So from a high level point of view, a firewall is simply a machine in the network, or a piece of software that stops and removes part of the traffic on a network. It is typically placed on the connection between a company's local network and the Internet, so that network traffic must pass through the firewall to go from the Internet to the local network, or vice versa. The simplest use of a firewall is to make it filter out all communication trying to connect to machines for which you do not want any communication with the outside world.

To understand how this is possible, recall that the connection oriented TCP protocol requires that particular types of packet must be sent and acknowledged in order to open a connection, and such packets can easily be recognized because a certain flag in the packet is set. Later packages can be recognized as belonging to a connection established earlier. Another protocol that is used in some cases instead of TCP is the UDP protocol. This is not connection oriented, one basically just sends a packet and hopes for the best. Among other things, UDP is used for communicating with DNS servers, which are servers that know how to translate (or *resolve*) human-readable names of Internet servers to actual IP addresses. Both UDP and TCP packets can always be recognised.

The simplest tool we can use in a firewall is called *packet filtering*. The definition of packet filtering is as follows: the firewall looks at each packet and decides whether the packet should be allowed through, based only on the content of the packet and some fixed set of rules.

Thus the packet-filter firewall can, for instance, just remove all packets trying to open a TCP connection to a local machine and stop all UDP traffic.

Although this may seem like a simple way to get rid of most risks, it is usually a bad idea, however. Typically, a company has machines for which you definitely want communication with anyone else, for instance the machine running the web server. Clearly the firewall should not stop such communication, so we would have to make the firewall allow for incoming connections to the web server. Moreover, "inside" machines that want to use an external web service for perfectly legal reasons, and therefore need to communicate with DNS servers using UDP, should be able to communicate. This can be done by packet filtering, because we can determine which incoming connections is for the web-server, and which UDP packets are meant for or are coming from a known DNS server.

However, this also opens us to attacks: if the attacker can break into the web server, he can move from there to other machines. More precisely, attacking other machines now only requires sending messages from the web server on the local network, and such local traffic is not stopped by the firewall. More advanced solutions use two firewalls, where the web-server is behind a not so strict firewall, in a region typically named *demilitarized zone* (DMZ), while the rest of the system is protected by a more strict filter. Still, this does not completely remove the problem.

Summarizing on packet filters: if simple packet filtering is to be secure, it needs to be so restrictive that it is hard to live with. If we make it less restrictive, we open a possibility that an adversary can try to communicate with and attack machines on the local network. This is typically done by scanning through a large interval of IP addresses and port numbers until one finds a machine that is willing to respond. This is also sometimes known as “port scanning”.

The conclusion is that a firewall needs to do something more intelligent than just a simple packet filter, it needs to examine packets in more detail and either take action that depends on the purpose of a packet, or even modify the packets.

12.3.2 Proxy Firewalls

One radical approach to “intelligent filtering” is a *proxy firewall*. Such a firewall will not allow any connection at all from a local machine to the outside or vice versa. Instead, it will handle any connection needed *on behalf of* the local machines. That is, if a local machine has some client software that wants to use a service on the net, the client must be configured to connect to the firewall instead, and then the firewall will connect to the service. This means that from the outside, the local network is completely hidden, it seems like only the firewall is there. Thus scanning through a large number of IP addresses until some machine out there reacts will not work here. You will never get response except from the firewall, so you are forced to hack the firewall before attempting anything else. Making sure that a particular machine cannot be hacked is hard enough, but at least easier than having to worry about dozens of local machines.

Therefore, proxy firewalls are regarded as highly secure. But it is of course not a very flexible solution. For instance, suppose we want to use client software for some web service on a local machine, but this software does not know how to use a proxy. The result will be that the client insists on connecting to the site where the service is placed, but this connection will be refused by the proxy firewall, who on its side insists that all local machines must connect to the firewall and no one else. Of course, we may be able to make the client believe that the service can be obtained from our firewall. Then the client will connect as it should, but now the firewall (or some front end that we have to build) has to know to where this particular type of connection should be routed. In other words, we will be forced to do some special implementation or configuration for every type of connection we want to allow.

12.3.3 Stateful Firewalls

It can be a better solution to use a *stateful firewall*. Such a device keeps track of every connection going through it. Whenever someone tries to open a connection it checks whether this type of connection is allowed and blocks it if not. All other packets are checked to see if they belong to an existing connection and are blocked if not (where, however, some flexibility is of course needed for UDP packets). This of course requires the firewall to remember which connections are active at the moment, and this is why such firewalls are called stateful.

With such a firewall, it is even possible to translate the addresses used by local machines to something different in the outside traffic, thus hiding the local addresses from an outside attacker, this is sometimes known as a *masquerading firewall*, and provides another way to avoid the “IP address scanning” mentioned earlier, or at least make it harder. A final step further is to have the firewall inspect the behavior of a connection to see if it does something suspicious, and either stop it or drop packets that have suspicious content. For instance, in some attacks on web servers, the attacker would need to send an unusually long input string to a particular routine running on the server. The firewall can be configured to catch such traffic and remove it.

The final word on firewalls is a word of warning: one advantage of firewalls is that they provide a way to do filtering of network traffic by just administrating one machine, instead of having to do so on every single machine in the local network. Of course it would be wonderful if firewalls provided an out-of-the-box and maintenance-free solution to this type of problem. While this may be the case to some extent for personal firewalls protecting only one PC, it is *never* the case when protecting larger corporate networks. The optimal configuration of the firewall of course depends on the configuration of the local network, and hence it has to be updated as the structure of the local network changes. In other words the statement “we installed a firewall, therefore we are secure” is wishful thinking if no one is maintaining the firewall.

12.4 Protecting software

There are other security mechanisms that can help enforcing security policies, besides a rigorously-designed TCB. In practice, many such security mechanisms deal with resource access and sharing between different pieces of code, so they are typically implemented at the operating system level, as means to protect the software executed on top. Depending on the specific implementation and deployment considerations, they may be part or not of the TCB.

Secure booting: only allow authenticated software to be loaded on the device, starting from the operating system *loader* and potentially including all the follow-up binaries, with authentication usually coming as digital signatures from the device manufacturer. Because this can be quite restrictive in general-purpose computers, unless a user has the ability to disable it or have the ability to authenticate the software themselves, it is more commonly deployed in embedded systems.

Package management: only allow legitimate (or perhaps reasonably up to date) software to be installed and executed in the device. Keeping software updated is critical to deploy security fixes in already installed software, a process called *software patching*. Modern package managers verify the integrity of software by checking digital signatures from the software vendor and/or the maintainers responsible for distributing updates.

Process isolation: only allow a process to access its own set of resources, preventing its internal state from being accessed by unauthorized entities.

Modern systems significantly rely on process isolation for security, and there are many ways of implementing it, with varying costs. An expensive way is duplicating the physical components where process state is stored, such that unauthorized access needs to overcome some physical barrier. One could imagine physically partitioning memory in regions that can only be accessed by privileged or user-level software, for example. A cheaper way is temporal/logical separation, where state from different processes does not overlap in time/space, and unauthorized access needs to overcome only some logical barrier. Virtual memory is a cheap example of process isolation, in which address translation dictates what requests to read/write memory are allowed for each process. Historically, *virtualization* has been one of the preferred mechanisms of isolating different pieces of running software, and it is largely deployed in modern multi-tenant cloud platforms. The TEEs that we discussed previously are an example of cryptographically enforcing process isolation.

12.4.1 Malicious Software

One of the things attackers would want to do when they bypass operating system defenses is to install malicious software on the attacked machines. Such software comes in many forms:

- *Trojan horses*: programs that appear to be useful (or go completely unnoticed by the user), but have some hidden malicious purpose, such as granting unauthorized access, reveal private data, or destroy data. A well-known special case is Spyware: programs that monitor peoples' Internet usage and send information on this to the creator of the program – who can use this information himself, or sell it to a third party.
- *Viruses*: infect programs on your computer, spread themselves from one program to another, and eventually does something harmful to the machine.
- *Worms*: similar to viruses, but exist as stand-alone programs that actively try to spread themselves from one machine to the other.
- *Ransomware*: this is a program that initially acts as a trojan, but at some point, it will encrypt all your files under a public key for which the attacker holds the secret key. It will then tell the user that the files will only be unlocked against payment, usually in cryptocurrency, so the attacker can get the payment anonymously (although most cryptocurrencies are not really anonymous).

12.4.2 Virus Scanners

If malicious software makes it into your computer, you need a tool that can recognize such programs and remove them. This is what the well known virus scanners do, they basically scan all potentially dangerous files and locations on your hard disk, looking for pieces of code that are known to be suspicious. This is possible given that a database has already been gathered, containing information on a large number of known viruses, typically in the form of *virus signatures*, particular bit patterns characteristic for particular viruses. Virus scanners are only useful if their databases are kept up to date. The largest producers of virus scanners are typically remarkably fast in updating their data files when a new virus hits the net. Unfortunately, many users do not update their own copy of the scanner, or do so too slowly. And hence many infections happen because of outdated virus scanner files and not because of new viruses. In recent years, various forms of malicious software have started trying to fool virus scanners by encrypting themselves in order to not look like what is in the scanner's databases. This can work if the virus contains a small piece of cleartext code that will decrypt the real virus and load it in memory just before execution.

An additional precaution that is very simple but also crucial is back-up: many of the attacks, ransomware in particular, lead to loss of data, and the back-up may be your only hope of getting your data back. Note that convenient solutions where your machine has access all the time to a back-up disk and backs up automatically will not help against randomware: the attack can encrypt all files it has access to, including the back-up!

How does malicious code make it into your machine in the first place? Even if an attacker can talk directly to a machine, he can usually not immediately install any software he wants, in particular if he is not a registered user of the targeted machine. There are several ways around this problem, however: the attacker can try to exploit weaknesses in the operating system or server software, as we will look in detail later. This can lead to attacks that will work if a user just visits a particular web page, provided his operating system has some known and exploitable bug. It is important to understand here that a web page is not a passive object you just look at, a web page may contain code that is executed as soon as the page is shown, for instance JavaScript and the like, leading to potential problems if the code has been maliciously designed.

In many cases, however, it is just as easy to fool the user into installing the malicious code himself! This is a special case of social engineering that we discussed earlier. For instance, there are millions of web pages with pop-up windows that claim to have identified a problem on the

user's machine and offer to fix problem for free, if the user clicks OK to installing some piece of software. Surprisingly many users fall for this, and it is important to understand that filtering at the network level and the like cannot help much here, the only hope is that the anti-virus or anti-spyware software will recognize what you are about to install.

12.5 Intrusion Detection

A different way to spot malicious software or users is to try to detect malicious *behavior*, rather than the malicious code itself. That is, we monitor processes and/or users of a system and try to detect if their behavior is suspicious. The big potential advantage is that this method does not have to depend on information on already known viruses and/or attacks.

We already saw one example of this: a stateful firewall can keep track of the connections going through it, and can therefore detect if the client sends data of suspicious form or size to a server. For instance, if a server is expecting input data coming from web forms that users are supposed to fill in, then data strings are usually supposed to be of at most a certain length and/or have a certain format. If an input string is much longer, this may be caused by an attack where a malformed input string is being sent in the hope that this will make the server behave in some unexpected way that can be exploited by the attacker. Other form of this look at specific aspects of the behavior of users or programs, such as login and session activity, command or program execution activity, file access activity or communication patterns across the network.

12.5.1 A possible classification

There are two approaches to deciding whether something suspicious is going on, known as *misuse-based* and *anomaly-based* detection. In the first case, the matching is done based on a model of the likely behavior of an intruder, for example networking patterns caused by known attacks (also called *signature-based* intrusion detection); with the impossibility of detecting new attacks as the main drawback. In the absence of an accurate model for the intruder, matching can also be performed by looking for anomalies deviating from the legitimate behavior. The main limitation in the latter is the number of *false positives*.

Intrusion detection is typically implemented through *pattern matching*. The pattern matching process can either be *rule-based* or *statistical*. We describe these in terms of anomaly-based detection modeling benign behavior, but the converse naturally applies to misuse detection. In rule-based matching, we set up in advance a set of rules describing what normal behavior is, and anything that is significantly different is considered illegal. In statistical matching, the idea is to first gather statistics on actual honest behavior and then compare to the statistics of the observed user or program.

12.5.2 Limitations

In essence, the basic problem with intrusion detection is similar to basic problem with biometrics: the entity monitoring the system must be tolerant enough to allow legal behavior, but restrictive enough to catch suspicious behavior. At the time of writing, this problem cannot be said to be completely understood or solved, and there are many obstacles for effective detection. For example, encrypted traffic is hard to analyze, attacks might be too specific to particular software versions, and network noise can trigger false alarms. This fundamental obstacle has motivated many startups to attempt black-box approaches for pattern matching and experiment with machine learning, but this remains at the research stage. With this approach, the training process builds a model of likely (ab)normal behavior, and inference is used to decide if an event is benign or malicious. Another issue is that the entire idea is not effective, if the system or

operator does not react when an attack is detected. Thus, intrusion detection tends to be as good as the *incident response* efforts that follow. Having too many false positives lead to *incident fatigue*, or the inability to properly assess the real risk due to excessive alarms.

A final very useful trick in the intrusion detection area is the so called *honeypot* approach. The idea is to create a resource on your system that will look interesting to an attacker but is actually fake, that is, a resource that no honest user would want to use. One example from networks is deploying a fake service in the DMZ part of the network, with the sole purpose of attracting the attacker. As a result, if activity involving the honey-pot is detected, we can assume this is adversarial activity. By logging such traffic one can hope, for instance, to trace it back to the attacker before it becomes aware that it is being traced, or simply to collect more information about current attacks that can inform *threat intelligence* efforts. One known example of this could take place at a hospital where it was suspected that someone was illegally accessing confidential files on patients. Following the honeypot approach, a number of files for non-existing patients were created, and accesses to these files were used to trace the attacker and better understand its methods.

12.6 Access Control

The models we have seen so far allow us to describe on a high level what flow of information or control between different persons or parts of systems we want to allow. It is implicitly assumed that when a user wants to execute an operation on some object, the TCB is able to verify the identity of the user and figure out which rights and privileges he has. Checking the identity of someone is a technical problem that has to do with password and hardware security or even biometrics, and has been treated earlier. Figuring out which rights a given user has at a given time is a different problem. In principle, this can be computed from the user's identity and the security policy. However, doing so on the fly —while a user is waiting for access, for instance— requires that the information about this is organized in a appropriate way. So the way we treat access control here can be thought of as a treatment of how we can represent the rules implied by the security policy in a convenient way so it can be accessed and used by the TCB.

The most generic way to represent the information is known as the *Access control matrix*. This is a matrix A with a row for each subject s and a column for each object o . Entry $A[s, o]$ contains a list of all operations s is allowed to execute on o . On most realistic systems, storing the entire matrix in a central location is not practical. So in practice, two basically different approaches are taken:

Access Control Lists Here we store, together with each object, the column assigned to the object, and this is called an access control list (ACL). Such lists make it easy to decide for a given object who has access, but harder to say what a given subject can do. For instance, this is basically the approach of the UNIX system.

User Capabilities Here we store, for each subject, the row assigned to the subject, this is typically called a user capability. They make it easy to compute what a given user can do, but harder to decide who can access a given object. For instance, Windows uses this basic approach.

For large systems, ACL's get much too long if they are specified per subject (user), and so one typically splits subjects in predefined groups and the ACL says only what subjects from each group can do. For instance, UNIX splits subjects from the point of view of a single user, into: the user himself, the user group he is in, and anyone else. Groups also make it possible for several subjects to share the same list of user capabilities.

A variant of the user group idea is known as *role-based access control*. Here, a number of roles are defined, such as “ordinary user”, “administrator”, etc. and capabilities are specified for each role. Users are assigned a certain role when they log in and are granted access according to the role. This is slightly different from the user group approach in that the same physical user may play different roles at different times.

12.7 Categorizing attacks

In this session, we look at various ways to categorize threats against systems, and we give some examples of common attacks and the mistakes that make them possible.

As discussed earlier, any reasonable approach to design of secure IT systems must contain a *threat model*, specifying which attacks we are going to protect against - but also (perhaps implicitly) those we will not address. In order to have some assurance that we have thought about everything that is relevant, it can be very useful to have a number of ways to categorize attacks into different types. We start by looking at a number of such taxonomies.

12.7.1 What is the attacker’s goal: STRIDE

This is a way to categorize attacks according to their effect, or in other words, what the attacker is trying achieve:

- S**poofing Identity: results in the attacker being able to impersonate another person (user).
- T**ampering: results in attacker being able to manipulate data without this being detected.
- R**epudiation: results in the attacker being able to deny having done something he actually did.
- I**nformation Disclosure: results in the attacker being able to access data he should not see.
- D**enial of service: results in the attacker being able to deny others access to a system.
- E**levation of privilege: results in the attacker being able to have more rights in a system than he was supposed to have.

The same attack may have several different effects at the same time, depending on the context. A man-in-the-middle attack, such as the one on the Needham-Schroeder protocol, usually results in spoofing of identity, but as a result, you may get information disclosure as well.

12.7.2 What means are used in the attack: X.800

One may also categorize attacks according to the *means* utilized by the attacker. The classification below is adapted from the X.800 standard published by the CCITT organization who is also behind the X.509 standard for certificates. The X.800 standard operates with two main types of attacks on network transmission, namely passive and active attacks. These can be further divided as follows:

Passive attacks:

- Eavesdropping:** The attacker listens in and reads the information sent.
- Traffic Analysis:** The attacker only looks at who is sending to who and how much is sent.

Active Attacks:

Replay: The attacker resends an old message.

Blocking: The attacker stops a message from arriving.

Modification: The attacker changes the data sent or injects a new message of his own.

Clearly, passive attacks are very difficult to detect, so the obvious defense is to use encryption against eavesdropping. This cannot in itself stop traffic analysis. For this, a number of tricks are available, such as sending the same message to many parties to hide the intended receiver, or to deliberately randomize the length of messages to hide the size of the actual message. A final method is known as *steganography*, which roughly speaking means information hiding, i.e., methods to hide information inside a message that otherwise seems to have a different content. One example of this is to make slight changes to a digital picture, such that some of the bits representing the picture encode the message you want to send. However, if one chooses carefully where to place these message bits, the change is not easily visible. In this way, one can hide even the fact that a message is sent.

Active attacks are easier to detect, but harder to prevent. So focus is often on detection of attacks, using message authentication codes, signatures, sequence numbering of messages, etc. X.800 focuses on data transmission over networks, but the types of attacks also make sense for storage of data. Nevertheless, X.800 is somewhat limited in scope, and one should not think of it as a complete list of tricks an attacker can use.

There is a certain overlap between X.800 and STRIDE, which is not surprising as there is often a direct correspondence between the goal of an attack and the means used.

12.7.3 Where are we attacked, and by whom: EINOO

When analyzing the possible consequences of attacks, and deciding which kinds of attacks we want to protect against, it is often useful to think about who is attacking, and where exactly the attack takes place. First, we look at *who* attacks us:

E xternal attackers, who are not legal users of our system; or

I nsiders, who are registered as users of our system and so have some amount of access to start with.

Likewise, one can distinguish according to *where* the attacker is able to hit us. One talks about

N etwork attacks, where the adversary can only listen to and perhaps modify network traffic, that is, he is limited to using the means described in X.800. It is usually impossible to prevent that network attacks are attempted, but most of them can be defended against against using cryptography (beware of traffic analysis, however!).

O ff-line attacks, where the adversary gets unauthorized access to information that is stored permanently in our system, on disk or on other media. He can then steal and/or modify this information. Examples include stealing the password database file, or copying from a PC the file containing the owners private key for Internet banking. Off-line attacks are harder to carry out than network attacks because they require breaking the system's access control to some extent, but on the other hand they can be attempted any time that is convenient for the attacker, and once access is obtained he can usually use standard search tools to find what he is after. Cryptographic protection of data is a obvious defense against off-line attacks, but one must realize that the keys used for this have to be stored somewhere, and if the off-line attacks gives access to the keys, this protection is useless.

On-line attacks, where the adversary breaks into the system and observes it while programs handling sensitive information are running. He may now be able to read secret keys from the RAM, or modify what is shown on displays to users, e.g., to make them believe they are signing a transaction that is different from the one actually signed. On-line attacks are harder to mount than off-line: the adversary needs to completely break the system's access control and he must be there at the right time when the process he targets is actually running. He must also have sufficient knowledge of how the memory is organized to find what he is after. On-line attacks can have very severe consequences and can only be defended against by having different parts of the system protected in different ways so that one can reasonably assume that not all parts of the system are subject to the attack.

The EINO classification can be useful when designing threat models for concrete systems. An example: suppose we are designing a system where a web server should collect sensitive data from some users and store them permanently. Which attacks should we worry about? if the server is hosted by a reliable organization using a well maintained machine and operating system, we might choose to say that we will worry only about attacks from outsiders, and only take care of network and off-line attacks. The rationale here is that since we trust the organization that hosts our server, and have sufficient confidence in the protection they have put up (firewall, intrusion detection etc.), we make the assumption that outsiders will not be able to do an on-line attack. There is of course no guarantee that such an assumption is actually true. But since it is never practical to protect against ALL attacks, some assumption always has to be made, and being clear and conscious about the assumptions you make is clearly much better than not knowing exactly what kind of security guarantees you have.

12.7.4 Where did we make the mistake: TPM

A final way to categorize attacks is according to what makes the attack possible, put another way, according to the way in which attacks exploit mistakes in our framework:

Threat model: the attack is possible because the threat model was not complete: an attack we did not think of turned out to be relevant, or an attack we dismissed as being too unlikely turned out to be relevant after all.

Policy: the attack is possible because the specification of the security policy expresses something different from what we intended. For instance because the policy was too complicated and we did not realize what it actually allowed.

Mechanism: the attack is possible because the security mechanisms can be circumvented

The hope is that by being conscious about the TPM classification while studying attacks that happened to existing systems, one will know better where to focus when designing new systems.

12.8 Attacks and Vulnerabilities

In the following a number of very different attacks are shown. This hopefully illustrates that identifying all relevant attacks is often extremely difficult. Hopefully, the list can serve as inspiration to specify a good threat model, but it is also intended to warn the reader to always be skeptical about the completeness of any threat model – perhaps particularly those you have designed yourself!

Apart from the attacks explicitly mentioned here, it is of course useful to recall all the different attacks that were already mentioned in the notes and exercises on cryptography, network protocols, etc.

12.8.1 Malicious Inputs and Parsing Errors

Perhaps the most important type of attack in practice is the type that circumvents security mechanisms by exploiting mistakes in the software that supposedly implements the mechanisms. This takes an amazing number of different forms, but the basic idea is always the same: The attacker gives some input to a program, where the input has a form that the programmer did not expect. This can cause the program to behave in some inappropriate way that the attacker may be able to exploit. This usually happens because the data is interpreted as something else than just data, say as a pointer or as code that can be executed.

Overflow Attacks

In an overflow attack, the adversary gives an input to a program that is longer than expected. If the programmer did not check for this properly, a multitude of bad things can happen. Programs written in C may have this problem, because C does not have any built-in protection against many kinds of run-time errors such as writing outside an array. On the other hand, C is fast and closer to the underlying machine than other languages, and this is why lots of low-level software, such as operating system kernels and loaders, are written in C.

Consider the following simple procedure:

```
void foo(char* input){
    char buf[3];
    strcpy(buf, input);
}
```

The procedure `foo` declares a local variable, an array `buf` with room for 3 characters. It then copies the string pointed to by `input` into `buf`. In C, strings are 0-terminated and therefore the standard function `strcpy` will continue to copy until it sees a 0-character in the array it copies from.

This of course means that if the input string is longer than what we have room for in `buf`, the remaining characters will be written outside the memory that was allocated for the array. Namely, the bytes actually allocated for `buf` will be at addresses `buf`, `buf + 1`, `buf + 2`, and the “overflow” characters from `input` will be written at addresses `buf + 3`, `buf + 4`,

This can be bad enough in that the program may crash if this happens, but the real problem occurs if the input string can be chosen by an adversary. This could well be the case if the procedure `foo` was part of a web server, and is called to handle a form that a user has to fill in. An adversarial user might notice that the web server seems to have problems if he types a very long string into a field and may then start to wonder if this bug can be exploited.

To see why an attack may be possible, we have to understand what happens on the machine when the procedure `foo` is executed. Usually, the machine works with a *stack*, which on its top contains a *stack frame* which is a piece of data that is assigned to the procedure that is currently being executed. So when we call `foo`, this instance of `foo` is assigned a stack frame that is put on top of the stack. The stack frame has space inside it for all local variables of the procedure, 3 bytes for `buf` in this case. But it also contains the *return address*: if the program counter was at address `addr` when we call `foo`, then we should return to address `addr + 1` when `foo` is done. So the return address `addr + 1` is put into a fixed position in the stack frame. This means we have a stack frame containing 3 bytes for `buf` and the return address, and they will usually be placed next to, or close to each other.

This in turn implies that if an adversarially chosen string that is too long is copied into `buf`, then not only will the return address be overwritten, *it will be replaced by a value that the*

adversary can choose. So this means that after `foo` is done, the machine will go to a place in memory decided by the adversary and start to execute whatever it finds there.

It should be obvious that this is a very dangerous scenario, in particular because a clever adversary might even be able to make the return address point to the very string he just supplied as input, which means that it gets to write the code that will be executed after `foo`. This code will execute with the same privileges as the attacked program so the adversary may now have access to all sorts of interesting stuff. This type of attack exists in an endless number of variants and even “innocent” looking bugs can be exploitable in surprising ways. So,

it is safe to say that no bug is ever safe!

An obvious solution seems to be to use a language safer than C with protection against overflow bugs, but this usually comes with a performance penalty. The Rust programming language has been introduced with efficiency and memory-safety in mind, and has a type system specifically designed to prevent memory corruption bugs without relying on garbage collection. This is achieved by *ownership* and *borrowing* mechanisms that keep close track at compile-time of which portions of memory are being passed as arguments. The ownership/borrowing mechanisms enforce that each individual value has at most one code path as owner at a single time, in a way that writing code with (intended or not) side-effects is much harder.

When adopting another programming language is prohibitive because of legacy or efficiency reasons, then programmers must become better at writing secure code. For instance, libraries exist containing safe procedures for string handling in C, and static code analyzers can detect many memory corruption bugs. Compilers can also be told to include *canaries* to detect overflow bugs in real time, and memory layout can be randomized to make targeted addresses much harder to predict, in a mechanism called *Address Space Layout Randomization* (ASLR). Sadly, the main obstacle is often that they first need to understand the problem and care about it.

Command Injection

Memory-safety vulnerabilities give the possibility of an attacker to exploit the execution context of a running program to manipulate the control flow such that an attacker-chosen sequence of unintended instructions is executed under the privileges of the victim. What is challenging about memory-safety vulnerabilities is that they can arise in overlooked parts of the software stack, such as runtime libraries that the program depends on, or even be introduced by accident by a defective compiler.

Naturally, this is not the only way possible for an adversary to take control over a running process. Complex systems commonly interact with backend services, such as databases or authentication systems, by issuing commands on behalf of the user. If an adversary can find ways to abuse this interface and directly inject unintended arbitrary commands, it can achieve the capability of executing arbitrary commands remotely. Typically, this is achieved by exploiting *parsing* vulnerabilities, which allow an adversary to bypass filters and encode commands as data. *Remote code execution* vulnerabilities is among the most serious found in computer systems, and typically carry heavy compensation in bug bounty programs.

The Unicode Exploit. A concrete old example comes from parsing character encoding, when Unicode support was still being rolled out. Older versions of Microsoft’s web server application, Internet Information Services (IIS) 4 and 5, were affected by the so-called Unicode Exploit. The idea is that the IIS is supposed to check whether the requests it gets from the outside try to access directories that should not be accessed. At the same time, requests are allowed to contain Unicode characters to encode special characters like `ø`, `å`. This allows directory names to contain

all kinds of international characters. These characters must be decoded before taking action, but unfortunately IIS performed the security check of the requests *before* the decoding, or at least before all decoding steps were completed, which allowed an attacker to mask an offending request by encoding it in Unicode in a special way so it takes a form that the security check would not recognize. The attack therefore consisted of sending such a carefully masked request to the IIS, a request that would execute, e.g., a command shell. This will (or at least previously it would) run with the same access rights as the IIS, so the attacker could now upload and run various interesting software to the target computer. For details of how a request was actually masked to fool the security check, read **McClure *et al.*, p.175–184.**

SQL injection. A classical example of command injection in Web systems is SQL Injection, where the attacker includes a database query command on a web request and finds a way to trigger its execution on the database server. A common way to write SQL Injection exploits is to find a database query that can be manipulated by providing malicious input through the URL. The exploit then consists of a garbage string that closes the original query, and additional commands injected by the attacker- While SQL Injection is properly mitigated in a modern web servers by *escaping* engines that invalidate any SQL command embedded into user inputs, similar vulnerabilities are still popular against other backend systems. Another modern defense against SQL Injection is to use *prepared statements* that create templates for database queries and limit what the adversary can change.

Log4Shell. A more recent example was a command injection vulnerability in Log4j, a popular Java logging framework, involving arbitrary code execution. It was found in 2021, but it had existed unnoticed since 2013 and is was estimated to affect hundreds of millions of devices. Exploiting the vulnerability was simple and involved abusing the Java Naming and Directory Interface (JNDI) subsystem, taking advantage of Log4j’s capability to allow requests to arbitrary JNDI servers.

The JNDI subsystem allows the lookup of Java objects at runtime, given a path to their data. JNDI can connect to several directory interfaces, for example the Lightweight Directory Access Protocol (LDAP), a standard protocol used to lookup information or devices within a network. By encoding JNDI lookups (with form `${jndi:ldap://example.com/file}`) into logging strings processed by Log4j, the library would replace the lookup variable with its results, triggering the download and execution the malicious code hosted in a public URL. This allows attackers to execute arbitrary Java code on a server or another computer, or leak sensitive information.

Cross-Site Scripting (XSS)

In cross-site scripting, the attacker exploits poor design of various web pages. It is a form of command injection affecting web programming in particular, so we discuss it separately.

We describe here a very simple form of the attack which actually is automatically stopped by newer versions of most browsers. However, many other variants exist that are not so easily caught. A web site might be vulnerable to cross-site scripting if it has a page accepting input from a client and that is returned back to the user in a new page. As a typical example consider web site *naive.com*, which has a search form on one of its pages. The expected use is that a user fills in the form, say with a string “cross-site scripting” to search for, and presses GO.

Next, the site shows a page where it says:

You searched for: cross-site scripting
Search results:

xx
yy
..

While this may seem innocent, let's have a closer look at how this is actually implemented: typically, when the search request comes in, a new page is dynamically created. The page contains the search results, and also the string that was searched for, put in just after "You searched for:". In some cases, this is done by simply copying directly what was in the search box.

This turns out to be a very bad idea. The first problem is that it is possible to fill into the search box a string which is executable JavaScript code. When this string comes back to the browser in the result page, the browser will usually execute this code. Of course, this was not the intention, but the browser has no way to know what the intention was!

One could argue that this is the client's own problem: he should know better than to search for an executable string. But it gets much worse: when *naive.com* returns a result page, there is no guarantee whatsoever that the content of the "You searched for"-field actually comes from the client it is sent to!

Suppose the site *evil.org* has a page where it says "search naive.com" and then a link you can click – plus of course some plausible explanation why searching *naive.com* would be a great idea. If unfortunate user John Doe clicks the link, the URL which is sent to *naive.com* contains a search request, but as (part of) the string to search for, it contains a piece of JavaScript code designed by *evil.org*. When *naive.com* returns the search result, this code is executed by John Doe's browser!

Note that from John's browser's point of view, the JavaScript code comes from *naive.com*, a site John may know well and has said he trusts. So there seems to be no reason not to execute the code. Worse, since the code seems to come from *naive.com* it has access to this site's data. For instance, if John has an account at *naive.com*, his browser may store a cookie containing John's username and password to the site. So now, the code from *evil.com* is free to read this cookie and send the data somewhere else. This can be done, for instance by the code sending a http request to a URL controlled by *evil.org*, and include the data in this request.

Cross-site scripting is a particularly nasty threat because the place where the vulnerability is, is not the place that suffers the consequences. As a user, there is very little you can do, except perhaps turning off JavaScript when you are browsing suspicious places. Browsers can be designed, and are being designed to partially solve the problem. For instance, the simplest form of the attack that we described here (where code is embedded directly in a URL) is often stopped by modern browsers, they simply filter out code that appears in a URL. But the browser can never completely solve the problem. After all, it is supposed to receive (JavaScript) code and execute it and we cannot expect it to filter out all adversarial code. To do this, the browser would have to understand that the intention is with each web site you visit, and this is of course not realistic.

The right solution is to stop the problem at the place it is coming from, namely web site designers should verify data they get as input more carefully, to make sure that code from other sites does not make into pages they send to their users.

An example: Heartbleed

OpenSSL is an open-source software package that implements SSL/TLS. It is used by a very large number of web servers across the world, including some of the big players such as Facebook and Google. Despite this, OpenSSL was maintained by a rather small group of volunteers, at

least up to 2014, where one person was permanently employed. It is therefore maybe not surprising that bugs occur, sometimes with serious consequences.

The *heartbeat extension* in SSL/TLS transmits a keep-alive message between client and server to prevent low-activity connections from timing out. This is done by sending a nonce (in this case a character string) to the server and have the sever return the same nonce. The client sends both the nonce and its length. As a result the server has at some point stored in RAM both the nonce (in an array) and the length ℓ . It will then produce the message to send the client by reading ℓ characters from the array.

The Heartbleed bug was a simple case of missing input validation: in an update of the software, it was forgotten to verify whether the length sent by the client matches the actual length of the nonce. Now, if a malicious client sends a length that is much too large, the server will continue to read beyond the end of the array and will return some piece of its internal memory to the client. This is potentially a serious problem because the memory may contain confidential data, such as the server's private decryption or signing key.

The bug was discovered in the spring of 2014, and had existed for about 2 years. It potentially affected a large fraction of the servers worldwide, but there are no reports confirming that any criminals exploited it. It may well have been known to various secret agencies such as NSA, but this is of course not confirmed either.

12.8.2 Microarchitectural Attacks

In 2018, a very important new type of attacks emerged that in a very fundamental way challenges the way modern CPUs have been optimized for speed. In this section we will discuss the basic problem and sketch a simple example of the attack. It is not possible to give a complete overview: in particular SPECTRE is actually a family of related attacks, and the number of forms in which it exists is growing even at the time of writing.

To understand what happens, we need to first discuss the physical memory organization of a modern computer: as CPUs get faster, the process of fetching data from the main memory (RAM) relatively gets slower and slower. Therefore all modern CPUs have *caches*. A cache is a fast on-chip memory of limited size, and the basic idea is that when data are fetched from RAM, the CPU also puts it into the cache (and may also fetch other data that are stored close to the item in question). The hope is that, at least for a while, the CPU will only need data that are already in cache and hence are available fast. Of course, this will sometimes fail, in which case we have a so called *cache miss*.

In case of a cache miss, the CPU will have to wait until the data it asked for is available. Or does it? Enter the main issue we have to discuss: *speculative execution*. What most CPUs do is to make educated guesses at which instructions will have to be done after the data is available from RAM, and then execute them (if possible). For instance, the data we are waiting for may be controlling whether a conditional statement should be executed, but the CPU may guess that the statement should be done and just execute it. The CPU will store a copy of its registers before it starts the speculative execution so that if the guess was wrong and what it did was not supposed to happen, it can roll back to the previous state. This is no worse than if it had been idle while waiting for data. On the other hand, if the guess was correct, the CPU has saved time.

It is not uncommon to see speculative execution of several hundred instructions, and speculations on what will happen can be correct more than 80-90% of the time. This idea therefore has a major effect on performance.

So what's not to like about this? The basic problem is that even if the CPU rolls back to a previous state if it guessed wrong, some side effects of the speculative execution may not be erased by this action. In particular, if data has been fetched to cache as a result of the

speculative actions, this is not reversed by the CPU registers being rolled back. It may seem that this should not be a problem: the data in cache is not wrong. Nevertheless, this may have some very unfortunate effects, as we now explain by an example:

Suppose an adversary wants to steal data from some program while it is running. For instance, the adversary could be a user in a multi-user system and he is now trying to steal data from another user. The attack could also happen in a browser, where a malicious web-page is trying to steal data from another webpage that is running inside the same browser.

Suppose the target program contains the following small piece of code:

```
if (x < array1_size)
    y= array2[array1[x]*4096];
```

We assume the attacker knows the source code of the target program and that he can arrange for the following conditions to be true when the code above is about to be executed:

- The attacker controls the value of `x`, perhaps because `x` is an external input to the program.
- The CPU branch prediction believes that the check in the if-statement will evaluate to True. This could be the case if the adversary can make the target program run the critical code several times with values for `x` that are in range, before doing the actual attack.
- `x` and `array1` are in cache, while `array2` and `array1_size` are not in cache (note that `array1_size` is a variable and not a constant). Again, if the adversary can give input to the target program, he may create this situation because the choice of input indirectly affects which data will be used more and hence which data are in cache.

If indeed we are in this situation, the attacker will get the code to execute with a value of `x` that is too large and hence `array1[x]` can be almost any byte in the memory that the target program has access to.

When we get to the critical piece of code, the CPU will start to fetch `array1_size` from the main memory, but it also will start executing the code inside the if-statement since it guesses that the condition will be true. `array1[x] * 4096` can be quickly evaluated since everything is in cache, so then the CPU will need to start fetching `array2[array1[x] * 4096]` to cache¹. Now, while `array2[array1[x] * 4096]` is on its way to cache, `array1_size` arrives and the CPU realizes its mistake. The state of the registers is rolled back, but still:

*array2[array1[x] * 4096] is in cache, while no other element from array2 is in cache.*

Note that if the attacker can find out which element from `array2` is in cache, this will immediately reveal the value of `array1[x]` which is the data the attacker wants. Of course, if the attacker could get the target program to execute any code he wants, this would be easy: just load every element from `array2` into a register, one after the other, and measure the time taken for each element. One of these will load much more quickly than the others, namely the one that was already in cache.

Of course, the target program containing exactly such a piece of code is too much to hope for for the adversary. However, as we just demonstrated, it is still true that the timing of various operations done by the program depends on the state of the cache. Moreover, the cache is a single piece of hardware that is used by all processes running on the CPU and so the timings of accesses to memory and cache done by different processes are not always independent. Based on this, several existing methods have been found for figuring out which data from a target program are in cache even if the adversary can only observe from a different process. We will

¹Scaling the address by the factor 4096 has (essentially) the effect that only the relevant element from `array2` is fetched. This has to do with something called hardware pre-fetching which we will not discuss here. The adversary will have to hope that such a scaling is in the target program for other legitimate reasons.

not discuss the details further here but just give one simple alternative method: if the adversary knows the content of `array1`, then immediately after the attack we described, he can provoke execution of the same code, now with a value x' that is in range, and time the execution. If `array1[x] = array1[x']`, this will be fast and otherwise it will be slow. Thus the adversary learns the value of `array1[x]` if the execution is fast. If the involved values are bytes, the adversary can hope to learn the correct value with probability $1/256$ in one go. This is not great, but certainly also not a negligible success probability.

It is important to realize that, while the SPECTRE family of attacks are not easy to exploit in practice, proof of concept code has been developed that demonstrate the attacks on many different CPUs, both in C and in Javascript running in browsers. Thus this line of attacks cannot be ignored although we do not yet know any such attack happening “in the wild”.

The fundamental problem that SPECTRE points out, is that the speculative optimizations break the contract between software and hardware developers – in the sense that even if the software is safe in itself, there can still be serious security problems, because what the hardware does is not always exactly what was written in the program.

Even if the problem is serious, solving it is very hard. Simply turning off speculation is possible, but unacceptable in practice because of the serious performance penalty. There are some attempts to have compilers help solve the problem: the compiler will try to recognize critical code and turn off speculation temporarily while that code is running. This will certainly help, but is clearly only a partial solution. It would seem that the only way to really make SPECTRE go away is to develop fundamentally new types of microarchitecture for CPUs.

12.8.3 Insecure Library Interfaces

Modern software cannot perform its functionalities without relying on other software components, such as software libraries and support routines provided by the operating system. All this hidden infrastructure adds complexity to the design and to management of risk, since your program’s dependencies on increase the attack surface and have independent development workflows.

There are many ways in which dependencies can affect security of your own software:

- *Unsafe library routines:* some library routines are known for decades to be inherently unsafe, but they are still supported for backwards compatibility. In most cases the problems are well-documented, and sometimes the programmer may even receive a warning to move to safer alternatives, but this is not always possible. Classical examples of unsafe library routines are string handling functions in the C programming language, such as `strcpy()` and `gets()`, that do not take buffer sizes as arguments and thus can be exploited as part of a buffer overflow exploit. Another instance is insecure generation of random bits, where the default random generator in most programming languages is simply not good enough for cryptographic applications, because they do not collect enough entropy or produce biased outputs. The `rand()/srand()` functions in the C programming language, and the Mersenne Twister used in the `random` package in Python are clear examples of unsafe functions still supported.
- *Legacy code:* The implementation of the same runtime library routine across different operating systems can be radically different. Even if your program uses a secure routine implemented in a modern system, it may be compiled and linked against an old implementation that is not secure.
- *Low-level interfaces:* Programmers sometimes find undocumented internal library interfaces, and prefer to use those directly instead of the long-term application programming

interface (API). However, internal interfaces might lack proper sanitization and house-keeping that protect the internal library state.

- *Error-handling*: Library routines and system calls can return error conditions, and those should be checked and acted upon. Not addressing errors in security-critical functions can also create vulnerabilities, such as when dynamic memory allocation calls fail and the program ends up with invalid pointers.

While the issues above are generic to software development, in the next chapter we will examine some specific cases that affect some of the most critical software components – cryptographic software. In particular, we will take a deeper look into how inadequate security policies and incomplete threat models may compromise the security of a system, illustrated with examples from cryptography.

12.9 Exercises

Exercise 12.1 Write an explicit threat model and security model for the smart safe example. There are at least three different attackers: a social engineering attacker, a logical attacker who is able to exploit software/networks, and a physical attacker. What is the attack surface of the system? What additional security mechanisms would you deploy? ▼

Exercise 12.2 Suppose you are commissioned to construct a system preventing cheating at written exams, where people are allowed to bring their own laptops. Via a local area network, all machines get access to the same printer, which is to be used by everyone participating in the exam.

- Which threats would you include in the threat model?
- Sketch a concrete security policy, i.e., what are the basic security objectives you think the system should satisfy.
- Mention some security mechanisms you would use to implement your policy. Note that they do not all have to be “electronic” mechanisms.
- What is the TCB for your system?

Exercise 12.3 Categorize Heartbleed, speculative execution and XSS in this chapter using STRIDE, X.800, EINO and TPM. Argue why you made the choices you did. ▼

Exercise 12.4 How would you redesign the heartbeat feature in TLS to minimize the likelihood of implementation errors? If you think of your redesigned heartbeat mechanism as a security *feature*, what is the security goal being achieved? ▼

Exercise 12.5 All residents in Denmark must eventually use MitID to access banks and government websites. A few years ago, MitID introduced QR codes as a security mechanism to strengthen the system against attacks.² What vulnerability does this mechanism address? Use the STRIDE, EINO and X.800 taxonomies to categorize a potential attack.³ ▼

²Blog post from 2023 at <https://www.mitid.dk/en-gb/about-mitid/news/an-update-of-the-mitid-app-aims-to-reduce-fraud-and-make-it-easier-to-use-mitid-on-mobile-devices/>

³This security analysis may give you a hint: <https://www.sciencedirect.com/science/article/pii/S0167404823002869>

Exercise 12.6 (Electronic Election 1) *Danmarksdebatten* was a web site set up by the Ministry of Science. Here, public institutions could put various matters up for public debate. The site worked as most debate sites, but with a somewhat more complicated administration, because different users can start and moderate their own debates, but not those of others.

In 2005 it was decided to expand the functionality so it becomes possible to have non-binding voting via the site. The idea is that public institutions can define their own subjects to vote on. To vote, one has to be a user of the public digital signature system and one may have to satisfy certain extra demands defined by the party who sets up the vote. For instance, a city council can set up a vote on a local subject and decide that only citizens of the city can vote. All votings are non-binding, so the city council is not obliged to follow the result.

Voting takes place over the net, and the result is published on the site as soon as the deadline for the voting is over.

A company was commissioned by the ministry to implement the system. The company then presented the following set of security objectives to one of the authors of this book for evaluation:

- All voters who are qualified to vote can vote, but only those, and you can only vote once.
- After you have voted, you will be able to see the tally of votes cast so far.
- Only the final result must become publicly known, in particular it must not be known how a single voter voted. However, it is part of the purpose of the system to gather various demographic and statistical data that can be given to others. For instance, it can become known how many women voted yes. This has been verified to be in accordance with current law, as long as every group for which separate information is given, consists of at least 25 people.

Your task: it is quite clear that this piece of text is not a security policy as defined in the course. For instance, there is no specification of which system we are protecting (presumably the system to protect is not the entire Internet!). Furthermore one or more natural and important security objectives is/are missing. Rewrite the text so that it can be called a security policy and such that the set of security objectives is more complete than before. ▼

Chapter 13

Attacks and Pitfalls in Cryptography (2025)

Reading Guide *This chapter introduces several new attacks and should be read with the taxonomies from the last chapter in mind. It starts by introducing the concept of entropy, which is very important in cryptography and analyzing real-world security of systems managing cryptographic keys. Exploiting malleability is also a recurrent theme in real-world systems, and padding oracle attacks come back every other year in some new shape and form. The latter sections are more verbose, but it is critical to go over the full technicalities of at least one of the attacks to properly explain it.*

Contents

13.1 Introduction	349
13.2 Randomness Generation	350
13.2.1 Insufficient Entropy	351
13.2.2 Backdoored Designs	353
13.3 Choice of Algorithms and Parameters	354
13.4 Inadequate Trust Models	358
13.4.1 IBM 4758	358
13.4.2 GSM	359
13.5 Certificates and Key Management	361
13.6 Covert and Side Channels	362
13.7 Exercises	364

13.1 Introduction

Deployments of cryptography are not inherently special, in the sense that they can also be badly designed and implemented, or suffer from vulnerabilities in the software, network infrastructure and runtime libraries. What is really special and challenging about cryptography is the concentration of *risk*: as soon as some form of cryptography is deployed in a system, user and designers will take *decisions* based on the fact that the cryptographic mechanisms fulfill their purpose. Like most system security mechanisms, cryptographic techniques are frequently included as part of the Trusted Computing Base, and successful attacks against the TCB can be disastrous.

In order to be secure in the real world, a cryptographic technique must be soundly designed, implemented and deployed. A sound design means that the technique has been properly studied from a theoretical perspective: it relies on computational problems that are indeed hard, and the best known attack compromising its security goal forces the attacker to solve such a problem. If the technique is part of a *protocol* involving multiple parties, this means that no specific party behaving maliciously should be able to abuse its position and obtain disproportionate advantage against the honest ones. A sound implementation means that the techniques themselves have been correctly implemented as a combination of software and hardware, and with high *assurance* (there are no vulnerabilities). A sound deployment means that shortcomings of the target platform(s) have been taken in consideration, and a proper key management process was conducted to generate and distribute cryptographic secrets securely. On top of all of these, one also wishes for strong cryptography to remain *legal* in a certain jurisdiction during the deployment lifetime.

When a deployment goes well, cryptography frequently represents the most secure component of the TCB, and attackers have no option other than to avoid the part of the attack surface protected by cryptographic techniques. Renowned cryptographer Adi Shamir famously stated this fact in his Turing award lecture: “*Cryptography is typically bypassed, not penetrated*”. But not everything goes well all the time, and there are numerous examples of bad deployments of cryptography that were insecure enough to be among the *weakest* spots in a system, giving disproportionate advantage to attackers. In fact, some pitfalls in the design and implementation of cryptographic mechanisms are so recurrent that they deserve further discussion. In this chapter, we will discuss pitfalls affecting the design and implementation of cryptography in a top-down approach. We start with problems in randomness generation, that can affect the deployment of multiple cryptographic techniques at once; and finish with security issues affecting specific implementations running in specific platforms.

13.2 Randomness Generation

A first example of pitfall in cryptography consists of vulnerabilities in pseudorandom number generation. Good randomness is an essential tool of any system using cryptography. When a user generates any secret in a cryptographic protocol, the process must be unpredictable from an outsider point of view. Consider for instance a silly example in which a user chooses their secret key to be equal to the time of the day. Then an adversary can easily estimate what the value of the key is, and therefore would be able to break the security of the encryption scheme. Biased keys that are easier to brute-force are not the only problem, and repeated/predictable IVs for symmetric encryption also come to mind.

Generating random bits is such a strong security requirement for most cryptographic techniques, that it motivated state agencies to standardize backdoored designs in an attempt to convince the industry to use them. That makes complete sense from an adversarial perspective, since targeting generation of randomness would potentially affect multiple primitives simultaneously. In all fairness, generating random bits securely is challenging for developers even when backdoors are not involved. Insecure algorithms are widely implemented in the standard libraries of major programming languages, hiding behind misleading APIs with names such as **random**. Frequently, they implement the Mersenne Twister algorithm, which does not satisfy cryptographic strength. Many other things can go wrong in practice, for example not having enough unpredictable data to start with. Unfortunately, good randomness is hard to come by and expensive. In most systems, the generation of random bits is performed by means of a *pseudo-random generator*, or PRG for short.

Pseudorandom Generators. While *true randomness* generators exist (for instance, devices that measure unpredictable and complex physical phenomena), such devices are quite expensive and therefore most PCs and other consumer grade hardware are not equipped with them. Regular computers collect all the data they can from unpredictable sources and, after some processing, give it back to applications as randomness. We call this data *entropy*, and the sources include you typing on the keyboard, interrupt timings, the movement of your mouse, or some other circuit implemented in hardware. This generates bits of “true randomness” which unfortunately do not come in bandwidth high enough to satisfy all needs of a modern computer system.

Therefore, most computer systems use PRGs to “expand” the few bits of true randomness they have available to strings of unbounded length which look random, in a way similar to what has been described previously when talking about stream ciphers (in fact, one could use a good stream cipher, such as a block cipher “in counter mode” to construct a good PRG).

We will introduce a little notation for working with PRGs. A PRG is an algorithm G that on input the current state s_i outputs some random looking string r_i and the next state s_{i+1} in the following way:

$$(r_i, s_{i+1}) = G(s_i)$$

The PRG is initialized with some truly random string s_0 known as the “seed”. Note that there is nothing random about a PRG after the seed has been fixed, in other words, given a seed s_0 then the entire output of the PRG is fully determined. Moreover, given any state s_i it is possible to completely predict the future outputs of the PRG and therefore the state should stay secret. On the other hand, given access only to the output strings r_i , it should not be possible for any (computationally bounded) adversary to predict the next PRG outputs (or in fact, distinguish the r_i ’s from truly random strings).

13.2.1 Insufficient Entropy

A secure PRG can only perform its duty if given a *truly random* seed as input. Failing to satisfy this requirement has been an *Achilles’ Heel* of cryptographic deployments, especially in resource-constrained embedded systems where entropy sources are not readily available. The term entropy comes from the general scientific concept that is commonly associated with a state of disorder or uncertainty. In information theory, entropy of a random variable is the measure of the amount of uncertainty (or missing information) inherent to the variable’s possible outcomes. In other words, entropy measures the amount of information conveyed by identifying the outcome of a random trial.

Given a discrete random variable X , which assumes values in the set $\mathcal{X}$ according to a probability distribution p , the entropy $H(X)$ is defined as

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log_2(p(x)),$$

where the summation denotes the sum over the variable’s possible values. Entropy is measured in *bits* and it assumes its maximum level when X is distributed uniformly. A variable X that is uniformly distributed across only 2 possible values will then have a single bit of entropy.

Translating entropy measurements to PRG security then becomes straightforward. A secure PRG satisfying a security level of n bits will require seed and internal state that are at least n bits long. This means that, given access to the output strings r_i , the adversary would need to attempt at least 2^n candidates in order to recover the seed/state, and evaluate the PRG to predict the next outputs. Hence, the security of the PRG does not depend only in the quality of the expansion process, but fundamentally in its initial conditions. In other words, how the seed

was generated and how much entropy it contains. Putting it differently, Given a predictable seed with less than n bits of entropy, a secure PRG will still generate outputs that will look indistinguishable from random (given access to the outputs only). However, the adversary now has a much faster shortcut to break the PRG, which consists in brute-forcing the space of possible seeds instead.

A surprising number of critical systems fails the simple test of generating seeds from a large enough space. One of the earlier examples was using the infamous combination `srand(time(0))` in the C programming language, which seeds a *linear congruential generator* (LCG) with a 32-bit value. In this particular case, the seed is a timestamp in seconds since 01/01/1970, thus far from unpredictable if the adversary has some estimate of when the randomness generation happened. But the problem is not only from the common implementation flaw with the insecure choice of seed, the generator design is also deeply flawed. Given constant integers a, b, m with m prime, an LCG is defined by the linear function

$$s_{i+1} = G(s_i) = as_i + b \mod m.$$

Earlier implementations returned the whole state s_{i+1} as output r_i , which gives the adversary everything it needs if a single value for the output is disclosed. Because the entire behavior of the LCG is linear, it is also possible to recover a, b from three consecutive outputs r_i, r_{i+1}, r_{i+2} in case those are not constant and made secret. Notice that updating the generator with a better underlying algorithm (such as past attempts to fix `rand/srand` based on the insecure RC4 stream cipher) and using a truly random seed still does not solve the problem: there are only 2^{32} possible seeds, which can be brute-forced quickly.

Examples. Small seed spaces affecting randomness generation were discovered in 1995 by David Wagner and Ian Goldberg in one of the first implementations of SSL, developed by Netscape. In this implementation, the randomness used to derive session keys was generated by hashing the time of the day and process identifiers (PIDs) in the Unix operating system¹. This would allow an adversary to recover the key material and decrypt traffic on behalf of the communicating parties, given only a timestamp corresponding to when the connection started and a small cost to brute-force the 2^{15} PID combinations. Another high-profile case was the Debian/OpenSSL fiasco in 2008, when a Debian maintainer detected the use of uninitialized data in OpenSSL's internal PRG and forced it to be zero. It turns out that embedded systems (such as routers) running OpenSSL without proper entropy sources relied on uninitialized data from memory to seed the PRNG, which now depended only on low-entropy PIDs. In 2012, security researchers were able to scan and detect that 0.75% of TLS certificates available on web servers shared RSA keys due to insufficient entropy during key generation, many of which were generated by this faulty version of OpenSSL.

More recently, insufficient entropy has been found in implementations of Bitcoin wallets, voting machine software (using the timestamp or the screen contents as the seed), disk encryption firmware and many other embedded systems. The same vulnerability appeared again in 2022 (twice) in a cryptographic library by Intel implementing the Paillier cryptosystem!²

Modern computer systems provide all the tools to finally eliminate this vulnerability class. We have secure PRG algorithms and a good understanding of how to collect entropy in both embedded and complex systems. Current embedded systems are either equipped with simple entropy sources in hardware, or can collect entropy from temperature sensors or network noise measurements. Complex systems implement PRGs at the operating system level, such as `/dev/random` in GNU/Linux or the Windows equivalent `CryptGenRandom()`. In 2012, Intel

¹<https://people.eecs.berkeley.edu/~daw/papers/ddj-netscape.html>

²<https://github.com/intel/pailliercryptolib/issues/2>

introduced the now widely supported `RDRAND` instruction combining a hardware circuit for raw entropy with postprocessing using AES, which served as an example for many other vendors to follow. The bigger challenge is actually to raise awareness among developers, such that they properly understand the platforms they are developing for, and research what runtime library APIs are secure to generate cryptographic randomness.

13.2.2 Backdoored Designs

Historically, there are a few symmetric algorithms that are suspected of being backdoored by the government agencies proposing them. However, none of them comes close in terms of supporting evidence to one particular attempt of backdooring a standardized PRG. This is the story of how, allegedly, an American three-letter agency has interfered with the standardization process of a cryptographic algorithm by NIST. As a result of this, the agency might have been able to bypass the encryption layer of the SSL/TLS protocol, thus facilitating surveillance of Internet traffic. Thus this is an example of how, sometimes, even a very paranoid threat model might not be paranoid enough.

Dual EC DBRG. In 2007 the US standardization body NIST standardized a PRG called “Dual EC DBRG”. This PRG uses elliptic curve cryptography, which is beyond what you have learned in this book. However, it is possible (by forgetting many details and oversimplifying a bit) to explain the main ideas using only the math which you have seen when describing the Diffie-Hellman Key Exchange.

In a nutshell, this PRG uses two (hardwired) “points on an elliptic curve” P and Q , which in our simplified explanation will just be two large numbers between 1 and some prime number n . The PRG is specified as following:

1. Take as input the current state s_i (which is a number between 1 and n);
2. Compute the next state as $s_{i+1} = P^{s_i} \bmod n$;
3. Compute the output as $r_i = Q^{s_i} \bmod n$;

It is possible to give a mathematical proof that, if P and Q have been chosen randomly and are large enough (2000 – 3000 bits), without any influence by the adversary, then this is a secure PRG. However it is also very easy to see that, if the adversary gets to choose P and Q , then it is very easy to break the PRG. This was no secret, and was immediately pointed out by a Microsoft researcher at the CRYPTO conference in 2007, shortly after the NIST announcement that they were going to standardize this PRG. Note that at the time no one had the imagination to assume that this was being done with malicious intent, and it was just assumed that NIST had made an honest mistake. However the standardization process continued, and eventually the PRG was standardized. Crucially, the standard also contained two predetermined values of P and Q that users where invited to adopt. No discussion on how P and Q where chosen was provided in the standard.

Let’s now see how it is possible to break the PRG for an adversary that has chosen P and Q in a malicious way.

1. First, the adversary chooses a random number Q , and another random number x . Then, the adversary computes $P = Q^x \bmod n$. The adversary gives P, Q to the unsuspecting user, and keeps x for himself.
2. Then, we assume that the adversary observes some output of the PRG r_i . We will return later on whether this is a realistic assumption. For the moment, imagine that the PRG is

used in an electronic casino, as the source for determining the next number which should be output by the roulette. Therefore, the output of the PRG can be observed by anyone.

3. Finally, the adversary computes the next state $s_{i+1} = (r_i)^x \bmod n$ and can therefore predict all future outputs of the PRG (and, in the online casino example, become filthy rich!)

To see why the attack works we are going to use the commutative property of exponentiations.

Remember that the rule for updating the state of the PRG is $s_{i+1} = P^{s_i} \bmod n$. But the adversary chose P as $Q^x \bmod n$ so we can write $s_{i+1} = (Q^x \bmod n)^{s_i} \bmod n$. Using the commutativity of the exponentiation we can then write $s_{i+1} = (Q^{s_i} \bmod n)^x \bmod n$. If we now remember that the rule to compute the output of the PRG is $r_i = Q^{s_i} \bmod n$, we can conclude that $s_{i+1} = (r_i)^x \bmod n$ and therefore the adversary has successfully managed to compute the next state of the PRG.

Breaking TLS with Dual EC DBRG As discussed above, the attack only works if the adversary gets to see the “raw” output of the PRG. This is not always the case: For instance, if you use a PRG to generate a secret key, you are not going to publish the secret key!

To see how one can use the backdoor of Dual EC DBRG to break cryptography, remember how TLS works.

1. In Step 1 of the protocol, the client generates a random nonce n_C (using the PRG); This value is sent unencrypted to the server, so anyone eavesdropping on the channel will learn n_C ;
2. The client will also use the PRG to generate its secret exponent r_c for the Diffie-Hellman key exchange.
3. It is very important that the r_c stays secret: if an adversary knows it, he can use it and the servers (public) Diffie-Hellman message $g^{r_s} \bmod p$ to compute the shared secret $s = (g^{r_s})^{r_c} \bmod p$. Now the adversary knows enough to be able to decrypt the entire following traffic.

At this point it is possible to see how the attack works: the adversary observes $n_C = r_i$, one of the outputs of the PRG. Using the backdoor x the adversary can compute the next state s_{i+1} , and therefore the adversary can predict the value of the next output by evaluating the PRG e.g., $(pms = r_{i+1}, s_{i+1}) = G(s_{i+1})$.

13.3 Choice of Algorithms and Parameters

A second source of problems is the imprecise definition of security properties and inadequate selection of algorithms and parameter sizes to satisfy them. Before authenticated encryption became commonplace, implementations typically attempted to obtain integrity guarantees by checking the message format after decryption, under the assumption that manipulation of the ciphertext would necessarily randomize parts of the obtained plaintext in a detectable manner. However, this conflates integrity and confidentiality: it is not generally true and several modes of operation for symmetric encryption can be manipulated in much more subtle ways.

Another example of inadequate choice of algorithms is using a high-speed hash function to hash passwords. This speeds up the verifier, but also speeds up an attacker executing a dictionary attack against a data breach involving credentials, or a brute-force attack. That is

why best practices recommend to use a *password hash function* instead, such as Argon2.³ These functions are designed to be *memory-hard* and not only to be slower, but also to consume a flexible amount of memory for hashing a password, slowing down attacks using GPUs or dedicated hardware that are harder to scale in terms of fast memory access.

Before we discuss specific examples, we can list many ways in which algorithms can be inappropriately selected and deployed:

- *Homebrew cryptography*: Deploying algorithms or implementations of cryptography that were not peer-reviewed by experienced cryptographers is inherently dangerous. They can be flawed in subtle ways and create a false sense of security. This piece of advice is frequently summarized as “Don’t roll your own cryptography”. The focus should be instead on deploying standardized cryptography, and as closest as possible to what is specified in the standards. If the standard specifies nonces to be a certain length and non-repeating, that is exactly what the implementation should do. Many algorithms break horribly if keys, IVs or nonces are reused, so make sure to respect the operating conditions of any deployed algorithm.
- *Obsolete algorithms*: Knowledge in the field of cryptanalysis advances every year, which means that a few cryptographic algorithms, even standardized ones, get eventually broken. Or perhaps their operating conditions change, requiring increases in key length or restricting in which applications they can be used. For instance, hash functions that are known to *not* be collision-resistant may not be broken inside a MAC construction such as HMAC. Even though it is tempting to take the risky decision of deploying such algorithms for a performance boost, it is better to behave conservatively and simply avoid algorithms with known structural flaws, such as the MD5 and SHA1 hash functions. Moreover, avoid deprecated ciphers that cannot offer long-term security, such as the RC4 stream cipher and 3DES.
- *Insufficient key lengths*: Cryptographic security can be a moving target, and attacks sometimes force updates in key lengths to address algorithmic advances in cryptanalysis or continuous improvements in computational power. Make sure to regularly check recommendations for key sizes from a reputable source collecting analysis by different bodies, such as <https://www.keylength.com>
- *Low-level libraries*: A recent trend in cryptography is to design libraries such that the programmer does not have full control over the choice of algorithm and key size. What the programmer actually needs from a library is secure encryption or a secure digital signature, and not ECB encryption using a custom algorithm. Restricting what choices of algorithms are exposed in the library interface reduces the likelihood of a programmer picking the wrong choice due to lack of training or experience.
- *Algorithm negotiation*: Many modern cryptographic protocols have a mechanism to negotiate algorithms during the establishment of a connection. This mechanism is useful to discontinue insecure algorithms by disabling it in one of the endpoints, an ability that is called *cryptographic agility*. However, if the agility mechanism is itself vulnerable, it might allow an attacker to perform a *downgrade attack* and force the parties to selected insecure algorithms or key sizes. When in doubt, keep it simple: assign a version to the overall protocol instead and limit what algorithms can be used in that particular version. That is precisely what happened in TLS 1.3, after multiple downgrade attacks affected the agility mechanisms implemented in previous versions.

³<https://www.password-hashing.net/>

Now that the generic issues are out of the way, we can take a look at how algorithms that are secure in a particular context can become insecure in another context. We will examine issues with malleability, padding oracles and length extension attacks.

Malleability. Encryption algorithms that only guarantee confidentiality still leave room for the attacker to tamper with ciphertexts. One such manner is to exploit the malleability properties of encryption. A first example comes from encryption using a stream cipher, such as using a block cipher under the counter mode of operation. In such a mode, encryption c of a message m with a keystream k simply amounts to computing $c = m \oplus k$. This means that an adversary can flip bits at selected positions of the message after decryption by just flipping the same bits in the ciphertext. For instance, by intercepting the ciphertext c and computing $c' = c \oplus 1 = (m \oplus k \oplus 1) = (m \oplus 1) \oplus k$ and transmitting c' instead, the adversary is able to change the plaintext message underlying the ciphertext to $m' = m \oplus 1$. Thus, if the adversary has auxiliary information about the underlying message format to know what bits to flip (think of a financial transaction with a field for the amount), and this is certainly the case in many settings, it can tamper with the encryption process without detection.

A mode of operation does not have to be fully malleable for it to be dangerous. Block ciphers in CBC mode, for example, are partially malleable: flipping a bit in a ciphertext block will completely mangle the plaintext block it decrypts to, but will result in the same bit flipped in the next plaintext block. This allows an attacker to “pay the price” of corrupting one block in order to manipulate data in the next one, possibly manipulating the message in the exact way that was needed. This property will be used a bit later to mount a padding oracle attack on CBC. A natural solution to the malleability problem is to compute a MAC over the ciphertext that limits what the adversary can do, in a so-called *authenticated encryption with associated data mode*, or AEAD for short. These modes provide confidentiality and integrity simultaneously, as noted in Chapter 6, and also allow to authenticate auxiliary data that is not encrypted, such as metadata about the sender/destination of the message.

The problem does not affect only symmetric encryption, and textbook versions of public-key encryption are also affected. For instance, in the case of the textbook RSA cryptosystem in which encryption is computed as $c = m^e \bmod n$, we know that the product of encryptions is the encryption of the product. In other words, an adversary can multiply a carefully-chosen encryption $t^e \bmod n$ of a message t to a ciphertext $c = m^e \bmod n$, and transform it into the encryption of $(m \cdot t) \bmod n$, again without detection. Padding schemes, such as OAEP, remove this malleability, which is incompatible with chosen-ciphertext security. Notice that the simplistic RSA digital signature algorithm without hashing discussed in Chapter 6 would be malleable in the same way but with respect to signatures. In the context of public-key cryptography, malleability should again be addressed by implementing integrity checking mechanisms using digital signatures.

Padding Oracles. When using CBC encryption, the final block must be padded in some way, so you have an integral number of blocks. Certain protocols instruct the receiver to check after decryption if the padding has the right form and send a message to the sender saying whether an error was detected. Now, consider an attacker who has seen some encrypted text c . The attacker is now free to fabricate from c some manipulated ciphertext c' , send it to the receiver, and observe whether he sends back “OK” or not. The important point is that with certain padding schemes, if c' was sufficiently cleverly constructed, the attacker will be able to find information on the original plaintext after a few attempts.

A possible way to construct c' is by exploiting the partial malleability of CBC. For a ciphertext with j blocks, an attacker can flip bits in block with index $j - 1$ to manipulate the last

plaintext block. If the attacker is able to achieve this in a way that the padding will still be correct if and only if a specific plaintext byte will have a certain value y , then the absence of error from the receiver reveals one byte of the original plaintext. It has been demonstrated that in this way, a short encrypted message can be recovered efficiently from real transmissions for some types of standardized padding schemes.

A similar attack is also possible against public-key encryption. Let's consider the PKCS#1.5 version of RSA encryption, that specifies that messages are padded by adding random non-zero bytes to the left such that the total length after padding is equal to the length of the modulus n . A properly padded message m then has the format: `0x00 0x02 [padding bytes] 0x00 [m]`. It follows that a random string of bytes will be correctly padded according to this scheme with probability roughly between 2^{-15} and 2^{-17} , which is far from negligible.

Notice that we use the zero bytes to remove ambiguity and separate the message from the padding, and that is why the padding bytes cannot contain zeroes. Upon decryption, the server will check the first two bytes, and require them to be `0x00 0x02`, in that order. Then it will look for the next byte with value `0x00` to know when padding ends and the message starts. Assume that an attacker has intercepted some message encrypted under the server's public key (n, e) for which it wants to obtain the decryption without access to the private key, and suppose that the server is similarly flawed as in the CBC case and notifies the client when padding errors occur.

The attacker will send many requests to the server. For each request, the attacker generates a value s and sends $c' = cs^e \bmod n$ to the server to decrypt. After decryption, the server will obtain $m' = (cs^e)^d \bmod n$, which will not be correctly padded most of the time. However, once every 2^{15} to 2^{17} attempts, the padding will be correct by sheer luck. If that is the case, then the attacker learns that $(m \cdot s) \bmod n$ lies in the specific range of integers starting with `0x00 0x02`. The rest of the attack consists in trying again, with carefully chosen values s that give some information that helps narrowing the value of m , until the attacker is able to derive exactly the value of m . Naturally, this attack would not scale against very long messages, but this may not be needed depending on how RSA encryption is deployed within protocols. The attack was first presented in 1998 by Daniel Bleichenbacher against SSL 3.0, when it was able to recover the premaster secret from which session keys are derived. Despite it being fixed in TLS and by moving to the OAEP standard for RSA encryption, variants of the attack have been re-discovered against TLS and in other settings, such as in the POODLE and ROBOT attacks⁴.

Length extension attacks. This attack can be seen as a version of malleability with respect to integrity and message authentication. Suppose we build a simple MAC algorithm from a hash function H that computes an authentication tag $H(k||m)$ for message m and symmetric key k . The attacker can use the MAC computed for a message m_1 as $H(k||m_1)$ and the length of m_1 to calculate the MAC $H(k||m_1||m_2)$ for an attacker-controlled message m_2 , but without necessarily needing to know m_1 .

The attack allows the forgery of a MAC for the concatenation of m_1 and m_2 without knowledge of the key k , and particularly affects hash functions based on the Merkle-Damgård construction, such as SHA-1 and the SHA-2 family (for reference, see Exercise 6.4). The vulnerable hash functions process the input message $(k||m_1)$ by splitting it into blocks which are used to transform an internal state through a *compression function*. After all blocks have been processed, the hash digest is generated by outputting the internal state of the function. An attacker can then use the output to reconstruct the internal state and continue processing blocks of message m_2 , producing a valid forgery at the end. Truncating the hash function output, as to not

⁴<https://robotattack.org/>

expose the entire internal state, solves the problem. In any case, it is better to use a dedicated MAC algorithm instead of a custom construction, and HMAC and SHA-3 are not vulnerable to this attack because they are designed differently.

13.4 Inadequate Trust Models

Another source of vulnerabilities in cryptography is when security policies are incorrectly specified, or threat models are simply incomplete. Attacks can also be mounted by insiders in order to foil the security policy, by circumventing the techniques that were supposed to protect the system from its users.

We discuss one respective historical example of each below.

13.4.1 IBM 4758

This is an interesting example of a case where the security policy was incorrectly specified. In the case of this hardware unit, the security mechanism is formed by the hardware protection of the box, which prevents an attacker from reading data in the box and from modifying the code it runs. No one knows how to circumvent this mechanism.

The security policy is formed by the API offered by the box, as this specifies which operations an outsider is allowed to do to the data held inside the box. Via an attack found by some PhD students from Cambridge (Mike Bond and Richard Clayton), it turns out to be possible to (mis)use the API and efficiently extract cryptographic keys from the box without having permission to do so (or at least it used to be possible — IBM has changed the API since then). This was done by simply using the functions in the API as they were defined, although in a different sequence than the designers had expected. So this is an example of an attack exploiting that the security policy was not specified as intended.

The main reason why the attack was possible is that the 4758 does both single DES encryption with a 56 bit key, and two-key triple DES where two DES keys are concatenated to form a 112 bit key. Single DES is not considered secure anymore and is mainly there for backwards compatibility, but this actually constitutes a security risk, as we shall see. Skipping many technical details, the essence is as follows:

A first observation is that the attacker can have the box choose a 56 bit DES key K_0 and encrypt some given data under K_0 . There is no way to have the box give you K_0 directly, but given the matching plain and ciphertext, one can find K_0 quite efficiently by exhaustive search, a 56 bit key is simply not long enough these days.

So the real question is if we can steal some of the *double length* keys the box holds?

To this end, we need to know the way in which one can transport a (double length) key from one 4758 to another. Let K be such a key, held in one box. One can then ask it to split the key in two *key parts*, K_1, K_2 where $K = K_1 \oplus K_2$. The idea is now that the key parts are sent to the other side via two different channels, say, transported by two different persons, who then manually key in their respective key parts. This allows the receiving box to compute the key, one can ask it to do so by calling a function called *Combine Key Parts*. Further, once the key K has been computed, and if it has been defined to be a so called key encryption key, one thing you can ask for is to use K to do an *Export Key* operation which will make the box output $E_K(K')$ where K' can be virtually any other key stored in the box. Hence, if a user of the box can get into a situation where he knows both K_1 and K_2 , he can steal all keys from the box very easily.

Of course, this could happen during a normal transport of a key, if the two users who know the key parts would collude. But there is nothing new in this, this is an already recognized risk.

The interesting thing is that a user can do an attack without colluding with anyone, assuming only that he has permission to do the Combine Key Parts operation.

To this end, one can exploit the unfortunate fact that there is (was) no complete separation between the use of single and double length keys. For instance, it is possible to call a function that forms a double length key by concatenating a single length key with itself, a so called *replicated key*. This function is there for backwards compatibility, since using a double length key where both halves equal single key K is equivalent to single DES encryption under K . Of course, the designers knew that such a key is not very secure, so you can only export single length keys using such a key. However, the attacker can:

1. Have the box create a single length key K_0 internally and compute K_0 using exhaustive search, that is, ask the box to encrypt some known data M under K_0 and use the resulting ciphertext C to test all possible keys until the correct key that encrypts M to C is found.
2. Ask the box to form a random double length key encryption key K_2 with *replicated* halves. Ask the box to export K_0 protected under K_2 . This is legal since K_0 is a single length data key so it is not stronger than K_2 .
3. The above step produced $E_{K_2}(K_0)$. We can now find K_2 by exhaustive search as above. Note that we have to define K_2 as a key encryption key that can be used for exporting other keys, for the purposes of the rest of the attack. This also means that it is illegal to use K_2 for encrypting data. Hence the detour where we first make a known single length key.
4. Manually type in a *non-replicated* key part K_1 , pretending you are doing a transport of a key.
5. Call Combine Key Parts to produce $K = K_1 \oplus K_2$
6. Because K_1 is not replicated, neither is K , so it will be classified as full strength key encryption key. Moreover, it is not a key part known by a single user. You can therefore export under K all the keys you want from the box. Nevertheless, the attacker knows K .

The attack is also an illustration of the dangers of having different levels of security side by side in the same system, here represented by single and double length DES keys. If you can make the system mix the two levels, you can often do a successful attack.

13.4.2 GSM

The Global System for Mobile Communications (GSM) is a standard developed to describe the protocols for second-generation (2G) digital cellular networks used by mobile devices such as mobile phones and tablets. The story of security in the GSM network is an interesting story about poor design that end up violating security, but also about a system that was nevertheless very successful commercially. To understand IT security in practice it is very instructive to look at GSM and see how both stories can be true at the same time.

Before GSM was introduced, an older system called NMT was in use, which had virtually no security. A phone connecting to the wireless network would simply send a unique ID number and would then be allowed to connect. The obvious attack, of eavesdropping an ID number and then using it to make calls that someone else pays for, was becoming a serious problem for the telephone companies. Therefore a new system was introduced – GSM. In this system, the goal was to prevent attacks on the wireless part of the network. The intended security policy was to prevent spoofing of identities of phones, and to prevent information disclosure from the wireless transmissions of calls.

The system that was put in place works as follows: it involves a unique key for each SIM card. The i 'th SIM card stores its K_i , and this key is also stored in a central database of the phone company. When a phone connects to a base station and needs to authenticate itself, it sends a so-called IMSI (International Mobile Subscriber Identity) to the base station which is a unique ID number in the SIM card. This IMSI is sent to the central database, which uses it to look up the key K_i . It will then send a triple (K_c, S_{RES}, R) to the base station. Here, R is a random bitstring and $K_c || S_{RES} = H(R, K_i)$ where $||$ is concatenation and H is a function designed for GSM, which one can think of as a one-way hash function. The idea is that the base station can now send R to the phone which, if it knows K_i , can recompute K_c and S_{RES} and send S_{RES} back to the base station. This response serves as proof that the phone knows the correct key. At the same time, both phone and base station know K_c if all goes well, so this key can be used as a session key, to encrypt future calls.

The GSM system included several different encryption algorithms on different security levels that would be used for encrypting calls, using the session key. A primary problem with this system was that the designers insisted on keeping all algorithms secret, thus violating the so-called Kerkhoff's principle that one has to assume that the encryption algorithm you use will always become known at some point. It remains a mystery that anyone could believe that the algorithms would remain secret when they were distributed inside millions of SIM cards.

But sure enough, the algorithms were reversed engineered and published before long. It turns out that they had several security problems that would surely have been found if the algorithms had been published in advance. In particular, the weakest of the encryption algorithms can be broken given only ciphertext, and an adversary would be able to listen to a call with only a few minutes delay.

Another problem was a poor design of the threat model: it was explicitly decided that the goal was to protect the wireless part of the network, and hence it seems that no one had considered attacks on the communication between base station and central database – probably because this happens on a wired connection. As a result, that communication was in cleartext. So if you eavesdrop that communication, you can get the entire triple (K_c, S_{RES}, R) and so connect a phone without knowing K_i .

The lessons to take away from this are: first, do not let anyone design a cryptosystem while working in isolation. Mistakes are bound to happen that will hurt you once the algorithm becomes known - which eventually it will. Instead one should use a standard algorithm that has been tried and tested in public. Second, think carefully about your threat model!

From this story one would be tempted to conclude that GSM was a failure. But one should note that this depends dramatically on who you ask. While security experts would say GSM failed, the phone companies would say it was a huge success - which indeed it was from a commercial point of view. The point is that GSM stopped the one attack the phone companies really worried about, namely it was way too easy to spoof phone identities. Even though GSM failed to make such spoofing infeasible, it did become significantly harder. This seems to have implied that adversaries turned to other attacks such as stealing prepaid SIM cards. This, of course, was a less efficient attack and also not a direct cost for the phone companies. Furthermore, while the security objective of keeping calls confidential was not really reached, this did not stop users from using the system, and so never became a commercial problem.

To be fair, it should be said that in more recent times, privacy seems to have gotten a more prominent place on many people's agenda. And so later systems, such as 3G and 4G use much better crypto and designs, in particular they use standard algorithms as the basis.

13.5 Certificates and Key Management

Issues with key management are the next problem in the list. This includes key storage and derivation mechanisms, but also validation of public keys. A surprising example of bad key management is not having key management *at all*, and instead hardcoding the same symmetric across all devices. This has affected some widely deployed systems, like videogame consoles (see below for a detailed discussion), voting machines in many countries and resource-constrained sensors and other Internet of Things devices.

Key management presents an interesting contradiction in cryptographic research. It is certainly not of the most exciting topics of research, since it involves a lot of process and organizational issues, but it certainly represents one of the most important in practice. After all, cryptographic solutions to security problems end up turning them into key management problems. A core technique in key management is to employ a *key derivation function* (KDF), a cryptographic algorithm that derives one or more secret keys from a secret value such as a master key or long passphrase.

A list of recurrent security problems when managing cryptographic keys can be found below:

- *Deriving keys from passwords:* Using a KDF to derive a key from a weak password will always generate a weak key, in the statistical sense. Just like in the discussion about PRGs, a KDF cannot just produce uniformly distributed outputs from low-entropy inputs. Some techniques may help with mitigating brute-force attacks, such as employing *key stretching* to take in additional randomness or to make the derivation process deliberately slow, like in password hashing.
- *Lack of domain separation:* Encryption systems using bidirectional communication should employ one key to encrypt/authenticate each direction of communication, like in TLS and SSH. This can be achieved by using a key agreement protocol to generate a master secret, and then deriving a different key for each usage and direction by using a string that specifies it. This prevents a ciphertext encrypted in one context of the communication to be still valid (encrypted under the right key) in another context. It also prevents an attacker who is able to obtain access to one key to read the two directions of communication, providing some measure of *defense in depth*. Many KDF algorithms have a specific input for domain separation strings.
- *Same keys for encrypting and authenticating:* Cryptographic algorithms may have weird interactions when the same key is used for encryption and authentication, especially public-key cryptosystems that have more structure. Following the previous advice, different keys should be used for different usages, even though they can be derived from the same shared key material.
- *Insufficient public key validation:* Public-key cryptography operates under the assumption that public keys are authentic. A big advantage of public-key cryptography in terms of key management is to not need to share secrets, but the problem becomes validating the authenticity of public keys, which is frequently solved through certificates. From an abstract point of view, verifying certificates looks simple, after all it is just verifying a digital signature from the certificate authority. In practice, it is much more complicated than that and one needs to deal with expiry dates, parsing issues, complicated standards with many extensions (X.509), private key compromises, revocation processes, non-trivial certificate chains, among other difficulties. In a now celebrated paper from 2012, Dan Boneh and co-authors referred to certificate validation logic as “The most dangerous code in the world”, especially when the task is performed by non-browser software. Hence,

validation of public key certificates must be rigorous: not only the whole chain of certificates must have its signatures validated, but also a leaf certificate should be checked for expiration or compromise. Client-side code should not even fully trust user-installed root certificates, and instead accept only system-level root certificates satisfying some criteria (so-called *certificate pinning*), at the risk of facilitating man-in-the-middle attacks. The *Certificate Transparency* project maintains a ledger of legitimately issued certificates that can be used to detect rogue for the certificates seen in the field.

The Xbox 360. The Microsoft game console Xbox was physically constructed very much like a normal PC. But of course, the manufacturers do not want users to use the console as a standard machine, since this opens up possibilities of playing copied CD's and DVD's and all sorts of other unwanted ways to use the machine. In the first version of the Xbox 360, some measures have been taken to try to ensure that the box always boots up in a preset state (from where it will only do things it is allowed to do). There is of course a boot record in the system that would normally contain the very first code that is executed.

However, in 2005, a PhD student from MIT (Andrew “bunnie” Huang) found out that this visible boot record is a decoy. In fact, the box contains a secured hardware unit which stores the real boot record. At boot time, the real thing is copied to the right location via a high-speed bus. This real boot record contains a secret key and code that is used to decrypt and verify some further code that is stored in external flash memory. This code, once decrypted and verified, forms the operating system kernel.

The real boot record is transferred to the CPU in clear text, and apparently this was assumed to be secure because of the high speed of the bus. However, Huang showed that it is possible to read the key from the bus using fairly standard equipment. Of course, once you know the key, it is possible to replace the code for the system kernel by something else. An important remark here is that the secret key was the same in *all* Xboxes! This may be a good choice in terms of making the manufacturing process easier and cheaper, but is of course a serious security problem.

There were more layers of security in the Xbox, so this hack alone did not allow to run arbitrary applications on the Xbox. However, the hacking community found ways around all of the security measures and it became possible to run custom software in the console.

13.6 Covert and Side Channels

There are also attacks which are typically done by insiders in order to foil the security policy, by circumventing the trusted computing base that was supposed to protect the system from its users. Suppose a malicious user obtains access to a high-security file which contents it wants to *exfiltrate* to the outside world. However, the system is *air-gapped* and inside some physical security perimeter, so the user cannot simply copy the file to an USB drive and carry it out of the building. One way to do this would be transmit the file through some communication channel that can be accessed out of the security perimeter. For example, the user could write an application that blinks a particular LED in the air-gapped system according to an encoding of the file contents. Another user with a camera can capture the signal and decode the final contents to obtain the exfiltrated data. A variant of the same attack could happen between a high-privilege user attempting to leak sensitive information to a low-privilege user without leaving noticeable traces. This phenomenon is called a *covert* channel and can be very difficult to prevent if different parts of a system share some hardware component.

A very large number of such channels have been found and documented, and some of them may not be intentional. An attack sometimes known as a *side-channel attack* arises in the

scenario in which an external adversary is trying to steal information from the system using a “channel” that the system designers were not aware of. This leads to an almost endless series of scenarios where the threat model turns out to be incomplete. One example of this is the attack we discussed in the chapter on Key Management and Infrastructures where you could steal a secret key from a chip card by measuring its power consumption while it was using the key. Other instances of side-channel attack would use different side channels, for example sound and electromagnetic emanations correlated with secret information. *Cold boot attacks* are a type of side channel attack in which an attacker with physical access to a computer performs a hard reset of the target machine to obtain a pre-boot dump of its RAM memory. The attack relies on the data remanence property of DRAM and SRAM to retrieve memory contents that remain readable in the seconds to minutes following a power switch-off, sometimes using a cooling substance to slow down the degradation. These attacks are typically used for recovering encryption keys from a running operating system for malicious reasons or as part of criminal investigations.

The padding oracle attack we observed earlier can also be seen as a side-channel attack, if you assume that the error-handling condition was leaking information about the validity of the padding, which in turn leaks information about bytes of plaintext. However, a padding oracle attack does not work for an external adversary, even even if CBC and the dangerous padding rule is used. The reason is that *everything* sent is encrypted, including error messages, so the attacker cannot get the information he needs. So everything should be OK?

However, Canvel *et al.* at the CRYPTO’03 conference found that there are still problems. The attacker can instead measure the time it takes the receiver to answer. If a padding error is found, we can expect an error message to be sent almost immediately. But if there is no padding error, the server has to go on according to the protocol and check a MAC before the final OK can be sent. This will take more time, and if the time difference can be observed from the outside, the attacker has essentially the same information as before, and the attack still works. Canvel *et al.* tested their attack on real networks and found that if they listen from a point sufficiently close to the server (in terms of the number of nodes between them and the server), the attack indeed works. The attack was found to work with two switches and a firewall between server and attacker. As one might expect, the information needed disappears in random networks delays if they move too far away.

The best known defense against side-channel attacks is to implement cryptographic algorithms in the most regular way possible. For *timing attacks*, this means that algorithms are implemented in *constant-time*, such that they take the same time to execute independently of the inputs. As a motivating example, think about the comparison of two byte strings, needed for example to check if an authentication tag computed from a MAC is correct. A naive implementation would compare the two strings and abort as soon as the first mismatched bytes would be found, since at this point it is already known that the strings are different. However, this actually leaks to an external adversary how many iterations of the comparison were executed, which in turn leaks how many bytes of a candidate tag were correct. Now an adversary can use the side-channel information to forge a tag without knowing the key. It starts by iterating all possibilities for the first byte for a few times, while measuring the latency for the server response. We know that the server will respond just slightly later for the correct byte value. With this information in mind, the adversary can discover the first byte and repeat the attack for all the remaining bytes, one at a time.

For an authentication tag comprised of n bytes, the adversary effort is now linear in n , for example by measuring the average latency of 100 attempts at each of the 2^8 potential candidates for each of the n bytes. This is actually much less than the expected complexity of 2^{8n} attempts. Other side-channel attacks, such as *differential power analysis* will instead collect many power

consumption *traces* for some computation involving the same secret key. From this information, the key may be recovered with overwhelming probability. Such an attack requires physical access to the device storing the secret to monitor the power consumption at the source, thus it is less dangerous and requires more powerful threat models than the timing attack example described above.

13.7 Exercises

Exercise 13.1 Look at the vulnerability discovered in Intel’s Paillier library and try to enumerate all the problems you can see there. There are at least three security issues. ▼

Exercise 13.2 Look at the different studies proposing recommendations at <https://www.keylength.com> and make a suggestion for the length of an RSA key that will encrypt data that needs to remain secret until at least 2050. ▼

Exercise 13.3 Show how to recover the LCG parameters (a, b) from three consecutive outputs r_i, r_{i+1}, r_{i+2} , assuming that the generator outputs the $r_i = s_i$ as in the earlier deployments of LCGs. ▼

Exercise 13.4 Think of a simple implementation of RSA decryption using square-and-multiply exponentiation that computes an exponentiation with the private key d . How one can mount a timing attack against such an implementation? State what information you obtain and under which assumptions. Can you think of a way to prevent leakage of information about the private key through the timing channel?

Hint: You are free to use microarchitectural attacks from the last chapter, as long as the attack exploits timing only. ▼

Exercise 13.5 Try to come up with a covert channel type of attack on your solution to exercise 2 from last week. ▼

Exercise 13.6 In the Elgamal cryptosystem instantiated over integers modulo a prime p , a plaintext m is encrypted under a public key $y = g^x$ (for private key x) as $E(m) = (g^r \bmod p, m \cdot y^r \bmod p)$ for random nonce r . Derive the decryption function, and answer how can one exploit the malleability of Elgamal encryption. **Hint:** The exponential version of Elgamal, where messages are encoded as g^m , is additively homomorphic and may help you solve the optional exercise in Chapter 11. ▼

Exercise 13.7 Select a subset of attacks against cryptography described in this chapter and classify them using the STRIDE and EINO taxonomies from the last chapter. ▼

Exercise 13.8 (Electronic Election 2) Consider the voting system for the web site “Danmarks Debatten”, for which you studied a security policy last week. The system was actually been built as a prototype, and below follows a description of the solution that was proposed. The system consists of 3 main parts:

Server - the central hard- and software that collects and counts the votes cast.

Client - the hard- and software used by a user to cast his vote.

Database - the persistent memory used on the serverside to store information on the votes cast so far.

When a vote is cast, the communication flow is as follows:

1. At the entrance to the election, a secure SSL connection is set up with only server authentication, so the client authenticates the server, but the server so does not know which client it talks to.
2. To check that the client has a valid digital signature, a new temporary SSL connection *Temp* is set up with both server and client authentication. This will fail unless the client has the secret key corresponding to its certificate. It is checked whether the user has voted before, and a lookup is done in the central CPR register (via a new SSL connection), where data on the voter is collected. This can be used to check whether the user is eligible to vote. For instance it may be required that you live in a certain area to be able to vote. If everything is OK, a token is handed to the client via *Temp*, namely an MD5 hash of session ID and the server's system time.
3. If the temporary session was successful, the original connection proceeds and gets the vote from the client. The client must also produce the token from the temporary session. This is to check that we here speak to the same client as in the temporary session.

In the database, the data stored for each vote cast is organized as follows: data is stored in two tables, *T1* and *T2*. In *T1* we have an entry for every vote cast in every election defined in the system. This entry contains a SHA1-hash of the CPR-number of the voter, concatenated with a secret key *K*. If you know *K*, this table allows to check whether a user with a given CPR number has already voted.

T2 also has an entry for every vote cast in every election. An entry contains the vote cast, and some data on the voter, namely age, sex and the area where the voter lives.

The system has a number of back-end users, who administrate elections. Of course, there are also security issues here, but we leave this issue out of scope in this exercise.

The system is planned to be hosted by a company who is obliged by contract to keep the operating system up to date with security updates. The host machine runs behind the company firewall. In addition, extra security measures are taken as follows: an additional firewall is running on the machine itself. An intrusion detection system called Tripwire is used. All systems files are checked regularly for integrity, and installed software is monitored daily for known security related bugs.

Now, the exercise is the following:

1. Put together a threat model for this system. Use, for instance, the EINO0 classification. Which attacks might occur? Try to form an opinion on how likely the attacks are and how bad the consequences would be, should the attack succeed. For instance, is breaking into the database as serious as breaking into the server? does it matter *when* you break in? As a result, try to conclude which attacks it would be reasonable to protect against.
2. Study the solution sketched above, and compare to the security policy from last week and the attacks you just listed. Does the solution live up to the policy given the attacks you listed? if not, what would it take to repair the solution? Could the solution be simplified without affecting security?



Chapter 14

State Machine Replication (2026)

Contents

14.1 Introduction	368
14.2 Core-Set Selection (CSS)	369
14.2.1 Ben-Or-Canetti-Goldreich Core-Set Selection	370
14.3 State Machines	371
14.4 State-Machine Replication (SMR)	374
14.5 SMR from CSS	376
14.6 Avoiding Total-Order	379
14.6.1 Tentative States	380
14.6.2 Eventual Consistency and Bounded Divergence	382
14.6.3 Commutative Updates	382
14.6.4 Serializability	382
14.6.5 Locks	383
14.6.6 Operational Transformation	383
14.6.7 Sequencers a.k.a. Leaders	384
14.6.8 Sharding	386
14.7 Reading from just one Server	386
14.7.1 Authenticated Data-Structures	386
14.7.2 Succinct Proofs of Knowledge, SNARKs	388
14.8 Client-Centric Consistency	388
14.9 Exercises	390
14.10 Eventually Synchronous Implementation of Total-Order Broadcast*	397
14.10.1 Discussion of the Eventually Synchronous Model	398
14.10.2 Optimistic Protocol	398
14.10.3 Shutting Down	400
14.10.4 Cleaning Up	402
14.10.5 Repeat	405
14.10.6 Which ABA to use	405

Attack Model *In this chapter we assume that*

1. *There are n parties.*
2. *Up to $t < n/3$ parties might be Byzantine corrupted.*

3. *We assume authenticated communication between all parties.*
4. *We consider the asynchronous model, with no clocks and no upper bound on message delay.*
5. *We consider eventual message delivery, so if an honest party sends a message to another honest party, then it will eventually arrive.*

14.1 Introduction

In this chapter we cover state-machine replication. When a service in a distributed system is important, its continued operation can be secured by replicating it. This could, for instance, be an important database or a file system, which we would like to keep working even if one or more physical machines crash.

Replication just means that you run the service on several computers and keep them consistent. To give input to the system, you send it to at least one correct server that will then forward it to the other servers. Then they each run their copy of the service on the given input. That way they all end up with an up-to-date copy of the service. If, for instance, you want to make a change to a file, the file is changed on all replicas. It is clear that some mechanism must be in place to ensure that updates are applied in the same order at all replicas. We discuss this issue later. If one of the replicas crashes, you can use any of the other ones. If you want to tolerate malicious behavior, you need at least three servers and must trust that at most one is corrupted. In that case, to receive an output from the system you need to receive the same output from two different servers. We want to solve the problem in the asynchronous model, where we can tolerate at most $t < n/3$ Byzantine corruptions when solving Byzantine agreement. Since state-machine replication can be used to solve Byzantine agreement, it follows that for SMR we cannot hope to tolerate more than $t < n/3$ Byzantine corruptions either.

The main problem in state machine replication is to ensure that all servers apply the same updates in the same order. We will see how to solve this in the asynchronous model. We basically proceed in rounds, where in each round each server collects a set of updates that have not yet been applied, and then they broadcast their sets. To prevent them from reporting different sets to different servers, they will use a reliable broadcast protocol that ensures that all servers receive the same output, for instance Bracha broadcast. With four servers they will then all wait for three of the sets to arrive. They cannot wait for all four sets to arrive because of the Godot problem, but they can wait for three because at most one server is malicious. Unfortunately, they cannot just take the union of the three sets, because they might have heard from different sets of servers. Server S_1 might hear from S_1, S_2, S_3 and server S_2 might hear from S_2, S_3, S_4 . This could happen even if all servers are honest. To solve this problem, they will run Byzantine agreement to agree on which three sets to use.

The three servers selected by the Byzantine agreement protocol are called the **core**. For each server S_i the servers run an asynchronous Byzantine agreement protocol ABA_i to decide whether S_i is in the core or not. Since the servers use ABA they end up agreeing on the same core. This is the main idea of the protocol, which is called CSS (Core-Set Selection). We will give the details of the protocol later. Then they take the union of the sets from the servers in the core and apply those updates in some deterministic order. If a server received a broadcast from a server S_i that is not in the core, then it ignores the broadcast. This process ensures that all servers apply the same updates in the same order, so they all end up in the same state.

One subtlety is that we need to ensure that applying the same updates in the same order results in the same state. This is not always the case. For instance, if the service uses randomness, then it might be that applying the same updates in the same order results in different states because of the randomness. To solve this problem we need to ensure that the service is

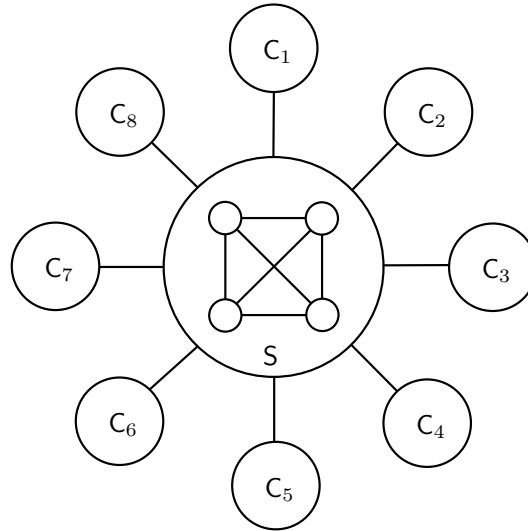


Figure 14.1: Illustration of a replicated server. Four copies of a database are being run. Clients can connect to any of the servers to update and read the system. It should be transparent to them which machine they connect to. There are two consistency problems to consider. The four copies in the big circle should be kept consistent. This is called **server consistency**. Also, if a client disconnects from a server and then reconnects to a new one, it should see consistent views. This is called **client consistency**.

deterministic. We will abstract the service as a so-called **state machine** and require that it is deterministic.

This, all in all, guarantees that the servers are consistent with each other, called **server consistency**. But what about the clients? If a client disconnects from one server and then reconnects to another server, it should see a consistent view. Since the system is asynchronous, one server could be behind another server. The servers apply updates in the same order, but they might be applying them at different speeds. So a client might reconnect to a server that is behind and see an old state. To solve this problem we need to ensure that the servers eventually catch up with each other. This is called **eventual consistency**. It will be guaranteed by our messages having eventual delivery and the servers applying updates as soon as they receive them. We then also need a mechanism for the clients to detect that they are seeing an old state. This can be done by having the servers include a clock in their responses indicating how many updates they have applied. The client can then compare the clock values from different servers and detect if it is seeing a state which is older than a previously seen state. This ensures that a client is consistent with itself over time, called **client consistency**.

We now dive into the details.

14.2 Core-Set Selection (CSS)

We start by looking at core-set selection.

Definition 14.1 (Core-Set Selection) A protocol for n servers $S_1, \dots, S_n$, where each server S_i has a port CSS_i for getting input and giving output.

Each server has as input a message V_i and the servers want to agree on a set

$$S \subseteq \{S_1, \dots, S_n\}$$

and a message W_j for $S_j \in S$. We write

$$\text{Core} = (S, \{(S_j, W_j)\}_{S_j \in S}) .$$

We let $|\text{Core}| = |S|$. We write $(S_i, W_i) \in \text{Core}$ to mean that $(S_i, W_i) \in \{(S_j, W_j)\}_{S_j \in S}$. The protocol should have the below trace properties.

Agreement: If two honest servers S_i and S_j give outputs Core_i and Core_j , then $\text{Core}_i = \text{Core}_j$.

Validity: If an honest server S_i gives an output Core and $(S_j, W_j) \in \text{Core}$ and S_j is honest, then at some earlier point S_j had the input $V_j = W_j$.

Large Support: If an honest server S_i gives an output Core then $|\text{Core}| \geq n - t$.

Liveness: If all honest servers receive an input, then eventually all honest servers give an output.



14.2.1 Ben-Or-Canetti-Goldreich Core-Set Selection

We present a core-set selection protocol called the BCG core-set selection, as it was introduced by Ben-Or, Canetti and Goldreich in their seminal paper on asynchronous secure computation [2]. It uses an asynchronous reliable broadcast protocol ARB as a subroutine, for instance Bracha. It also uses an asynchronous Byzantine agreement protocol ABA as a subroutine, for instance SVUP. The protocol is given in Fig. 14.2.

The BCG core-set selection protocol runs as follows. We give the code for S_i .

Input: On input V_i on CSS_i server S_i will use an instance of ARB, called ARB_i , to broadcast the value V_i .

n **ABAs:** For each server S_j the servers will run one instance of ABA, called ABA_j , to decide whether S_j is in the core or not.

Positive Vote: If S_i receives V_j from S_j via ARB_j , then S_i votes 1 in ABA_j , unless it has already voted 0 in ABA_j .

Negative Vote: If S_j sees that $n - t$ of the ABAs $\text{ABA}_1, \dots, \text{ABA}_n$ have terminated with decision 1, then S_j will input 0 to every ABA_j to which it did not already give an input.

Output: Once all n ABAs have output a decision at S_i , it can compute the core as follows. Let $S = \{S_j \mid \text{ABA}_j \text{ output } 1\}$. Wait for the ARB ARB_j to terminate with some value W_j for all $S_j \in S$ and let $\text{Core} = (S, \{(S_j, W_j)\}_{S_j \in S})$. Then output Core on CSS_i .

Figure 14.2: BCG core-set selection CSS.

Note that due to the Godot problem we cannot say that if they do not receive V_j then they vote 0. The message might be arbitrarily slow and the parties do not even have a clock to do a timeout. So, we needed a smarter way to know when to give a negative vote. The servers do it after they saw $n - t$ of the ABAs decide on 1. This way to do negative votes solves the problem of detecting when a server waited long enough. We cannot use time, but once $n - t$ of the ABAs gave output 1 we know the core will be big enough, so we can safely

give input 0 to the remaining ABAs. This also ensures that the servers will not wait forever for the remaining ABAs to terminate. All honest parties will end up giving input to all ABAs even if some corrupted servers did not ARB their V_k . So, this way to do negative votes solves Liveness without violating Large Support. Note that our way to do negative votes might even have a server give input 0 to one of the ABAs that terminated with output 1, as an ABA might produce output before getting input from all servers. This is not a problem.

A run is illustrated in Fig. 14.3.

We now argue that the above protocol implements CSS.

As for Agreement, note that the set S is computed from the decisions of the n instances of ABA. Since ABA has agreement, the servers will agree on S . Then, for each $S_j \in S$ the servers wait for ARB_j to terminate with some value W_j . Since ARB has agreement, the servers will agree on W_j . Therefore they will agree on Core.

As for Validity, if S_j is honest and some honest S_i outputs Core with $(S_j, W_j) \in \text{Core}$, then the instance ARB_j output W_j . So by Validity of ARB, at some point the honest S_j gave input W_j to ARB_j . Hence $W_j = V_j$.

As for Liveness, assume that all honest servers receive an input. Then all $n - t$ honest servers S_j will eventually give input V_j to ARB_j . Since ARB_j has liveness, all honest servers will eventually receive V_j from S_j via ARB_j .

We first argue that this implies that eventually $n - t$ of the ABAs will output 1 at some honest server. Namely, as long as it did not happen that $n - t$ of the ABAs output 1 at some honest server, it holds that when an honest server receives V_j from S_j via ARB_j , it will input 1 to ABA_j , as it did not already input 0. When all honest servers input 1 to ABA_j , then it will, by Validity and Termination, eventually output 1 as its decision. This holds for all $n - t$ of the ABA_j 's for honest S_j . Therefore, the $n - t$ ABAs for honest servers will all eventually output 1 at some honest server. So, we have argued that as long as it did not happen that $n - t$ of the ABAs output 1 at some honest server, then it will eventually happen that $n - t$ of the ABAs output 1 at some honest server. This is of course the same as saying that eventually $n - t$ of the ABAs output 1 at some honest server.

We concluded that eventually $n - t$ of the ABAs output 1 at some honest server. After this, by Termination of ABA, these $n - t$ ABAs will eventually output 1 at all honest servers. When this happened, all honest servers will eventually input 0 to every ABA_j to which they did not already give input 1. Therefore, all honest servers will eventually give input to all n ABAs. Therefore, by Liveness of ABA, eventually all n ABAs will terminate with some output. Therefore, all honest servers will eventually compute S . Then they wait for ARB_j for $S_j \in S$. This cannot deadlock. Namely, if $S_j \in S$ then ABA_j output 1. Therefore, by Validity of ABA, some honest S_i inputted 1 to ARB_j . Therefore some honest S_i saw ARB_j give output W_j . Therefore, by Termination 2 of ARB all honest servers will eventually receive W_j from ARB_j . Then they compute Core and output it.

14.3 State Machines

Recall that the plan is to broadcast all inputs and then have all servers run the service on the same sequence of inputs. For this to work it is important that if you have two identical copies of the service and you implement the same change on both, then they end up being identical again. We say that the service needs to be deterministic. That might sound trivial, but it, for instance, excludes services using randomness. Also, some data structures in some programming languages have underspecified behaviour. If for instance you have a Set data structure and you iterate over the elements, then the order in which you see the elements is not specified. Different versions of the software (maybe for different operating systems) might have different

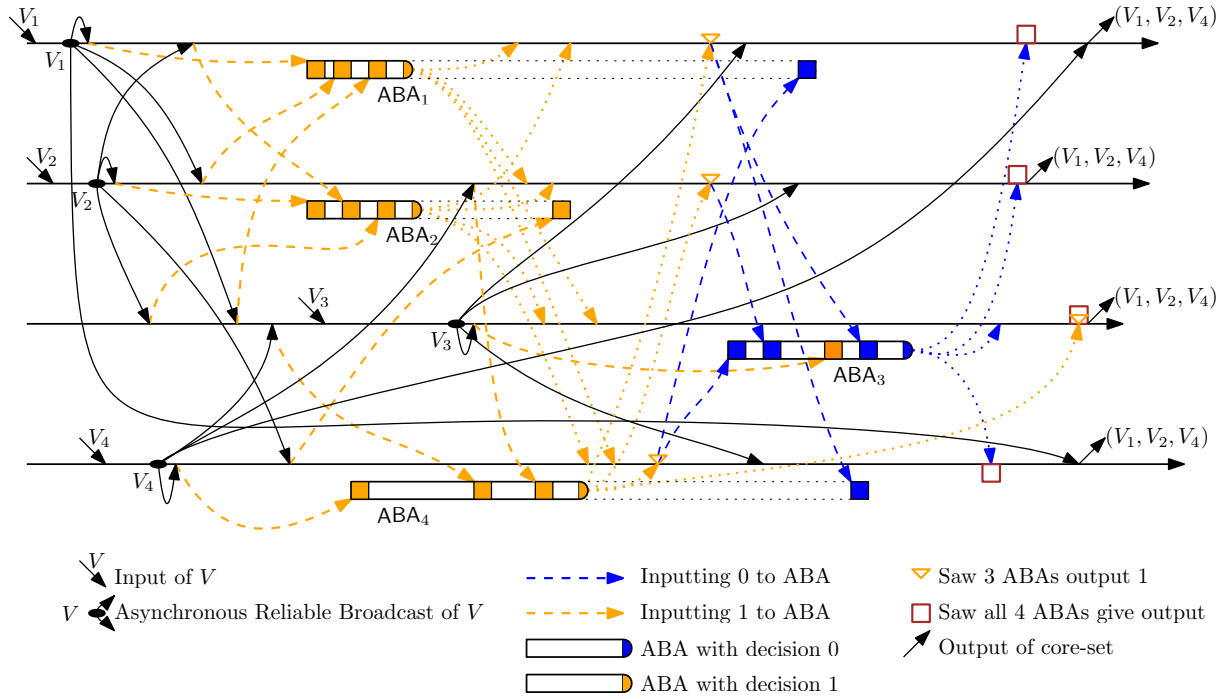


Figure 14.3: Illustration of BCG Core-Set Selection. First S_1 gets input V_1 and ARBs this input. First V_1 arrives at S_1 and it votes 1 in ABA_1 . When V_1 arrives at S_2 this server votes 1 in ABA_1 , and when V_1 arrives at S_3 this server votes 1 in ABA_1 . As these three 1-votes enter ABA_1 it eventually decides 1, and this decision eventually arrives at all four servers. When V_1 arrives at S_4 this server does *not* vote 1 in ABA_1 , as it already voted 0 in ABA_1 , as discussed later. A bit later S_2 gets input V_2 and ARBs this input. It arrives at S_2 , which votes 1 in ABA_2 . When it arrives at S_3 this server votes 1 in ABA_2 , and when V_2 arrives at S_1 this server too votes 1 in ABA_2 . As these three 1-votes enter ABA_2 it eventually decides 1, and this decision eventually arrives at all four servers. When V_2 arrives at S_4 it votes 1 in ABA_2 , but the vote enters the ABA protocol after ABA_2 already decided, so it has no effect. A bit later S_4 gets input V_4 and ARBs this input. It arrives at S_4 which votes 1 in ABA_4 . When it arrives at S_3 this server votes 1 in ABA_4 , and when V_4 arrives at S_2 this server too votes 1 in ABA_4 . As these three 1-votes enter ABA_4 it eventually decides 1, and this decision eventually arrives at all four servers. When V_4 arrives at S_1 it does nothing as it already voted 0 in ABA_4 . Later, S_3 gets input V_3 and ARBs this input. First V_3 arrives at S_3 and it votes 1 in ABA_3 . When V_3 arrives at S_1 , S_2 , and S_4 they do nothing as they already voted 0 in ABA_3 . Server S_4 is the first to hear three ABAs output 1. It hears from ABA_2 , ABA_1 , and then ABA_4 . It therefore votes 0 in the ABAs in which it did not already vote 1, here ABA_1 and ABA_3 . Then S_1 hears 1 from ABA_1 , ABA_2 and ABA_4 and votes 0 in ABA_3 and ABA_4 . Then S_2 hears ABA_1 , ABA_2 and ABA_4 output 1 and votes 0 in ABA_3 as it did not give input to it yet. As ABA_3 now got three 0-inputs it eventually decides 0, and this decision eventually arrives at all servers. Server S_4 is the first to hear all four ABAs make a decision. It hears $(ABA_1, ABA_2, ABA_3, ABA_4) = (1, 1, 0, 1)$. It already received V_2 and V_4 , but did not yet receive V_1 . When V_1 eventually arrives it outputs (V_1, V_2, V_4) . Then S_2 hears $(ABA_1, ABA_2, ABA_3, ABA_4) = (1, 1, 0, 1)$. It already received all three needed messages and immediately outputs (V_1, V_2, V_4) . Then S_1 hears $(ABA_1, ABA_2, ABA_3, ABA_4) = (1, 1, 0, 1)$. It waits for V_4 to eventually arrive and then outputs (V_1, V_2, V_4) . Finally S_3 hears $(ABA_1, ABA_2, ABA_3, ABA_4) = (1, 1, 0, 1)$. It already received all three needed messages and immediately outputs (V_1, V_2, V_4) .

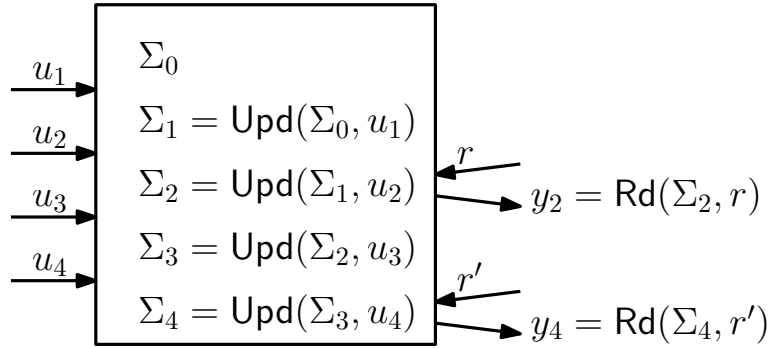


Figure 14.4: Illustration of a state-machine. It starts in state Σ_0 . On update u_1 it enters the state $\Sigma_1 = \text{Upd}(\Sigma_0, u_1)$. On update u_2 it enters the state $\Sigma_2 = \text{Upd}(\Sigma_1, u_2)$. On update u_3 it enters the state $\Sigma_3 = \text{Upd}(\Sigma_2, u_3)$. On update u_4 it enters the state $\Sigma_4 = \text{Upd}(\Sigma_3, u_4)$. When a read operation r is performed the state does not change. Instead some function $\text{Upd}(\Sigma_2, r)$ depending on the current state is returned. We sometimes denote the returned value as y_2 when computed from state Σ_2 .

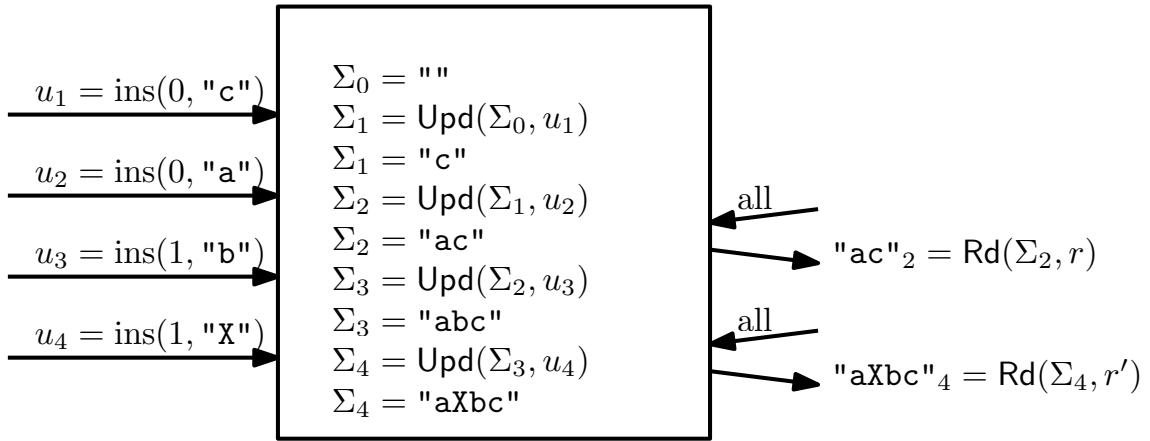


Figure 14.5: State machine where the state is a string, updates are insertion and the only read operation is to read all of the string.

implementations. This might result in two servers running the same code but having different behaviours. One therefore has to be very careful when implementing deterministic behaviour.

We abstract the deterministic service to be replicated by a so-called state machine M .

Definition 14.2 (State Machine) A state machine M consists of:

- A set of states States .
- A start state $\text{State}_0 \in \text{States}$
- A set of update operations Updates .
- A set of read operations Reads .
- A set of outputs Outputs .
- An update function $\text{Upd} : \text{States} \times \text{Updates} \rightarrow \text{States}$.
- A read function $\text{Rd} : \text{States} \times \text{Reads} \rightarrow \text{Outputs}$.

A state machine starts in State_0 . For a sequence of updates $u_1, \dots, u_m$ the state machine will go through a sequence of states $\text{State}_0, \text{State}_1, \dots, \text{State}_m$ where $\text{State}_i = \text{Upd}(\text{State}_{i-1}, u_i)$ for $i = 1, \dots, m$. We write $\text{State}_m = M(u_1, \dots, u_m)$ to mean that if we start M in the initial state

and run it on input u_1 , then input u_2 and so forth, then it ends up in state State_m . When the state machine is in state State_i and receives a read operation $r \in \text{Reads}$ then it outputs $y = \text{Rd}(\text{State}_i, r)$. Note that reads do not change the state.

The above definition is quite general. It captures a lot of services that we might want to run.

For instance, a database can be captured as a state machine where the state is the content of the database, the updates are the write operations and the reads are the read operations.

A file system can be captured as a state machine where the state is the content of the file system, the updates are the write and delete operations and the reads are the read operations.

A cryptocurrency can be captured as a state machine where the state is the ledger, the updates are transactions and the reads are queries on the ledger.

At a finer level of granularity, a data structure like a set can be captured as a state machine where the state is the content of the set, the updates are the add/remove operations and the reads are the contains operations, so $\text{Rd}(\text{State}, e)$ would return 1 if e is in the set State and 0 otherwise.

At a trivial level the state could just be the sequence of updates and reads could return this list. So $\text{State}_0 = ()$ is the empty sequence, $\text{Upd}(\text{State}, u) = \text{State} \| u$ is the sequence State with u appended at the end and $\text{Rd}(\text{State}, r) = \text{State}$ just returns the sequence of updates. This is a very trivial state machine, but it is still a state machine. Replicating this state machine is essentially the same as implementing total-order broadcast.

At a very high level of granularity, the state could be the entire state of the service, for instance the state of Amazon Market Place, the updates could be the user interactions and the reads could be the queries on the system, who bought what, what is in stock and so forth. Having one formalism capturing all these different levels of granularity is very convenient.

14.4 State-Machine Replication (SMR)

We denote a service running a replicated state machine where all servers run state machine M by SMR_M . What we want is illustrated in Fig. 14.6. Clients can do updates at different servers, yet the system should look like they are updating one coherent system as in the top left corner. We want to specify how the big circle in the upper left corner is intended to behave. It is convenient to formalise it as an ideal functionality rather than via trace-properties. The reason is that trace properties would only talk about the relationship between updates and reads. Saying that these correspond to a state machine is a bit cumbersome without just saying that they are consistent with having run a state machine. The ideal functionality allows us to say this by making the state machine explicit in the definition.

When we specify the ideal functionality we should be careful to not ask for too much. As illustrated in Fig. 14.7, in an implementation updates cannot be applied immediately everywhere, so it is possible that a read at one server does not reflect an update already seen at another server. To reflect this in the ideal functionality, updates are put in a tentative set. This corresponds to the fact that in the implementation the updates first have to be ordered, so the servers cannot do immediate local update. The ideal functionality also has a port **Progress** for indicating progress of a server. The adversary controls the progress. This corresponds to the fact that in the implementation the servers might be running at different speeds, and the adversary has control of whom is faster, as we are in the asynchronous model, where it is the adversary delivering messages.

So the ideal functionality basically captures that:

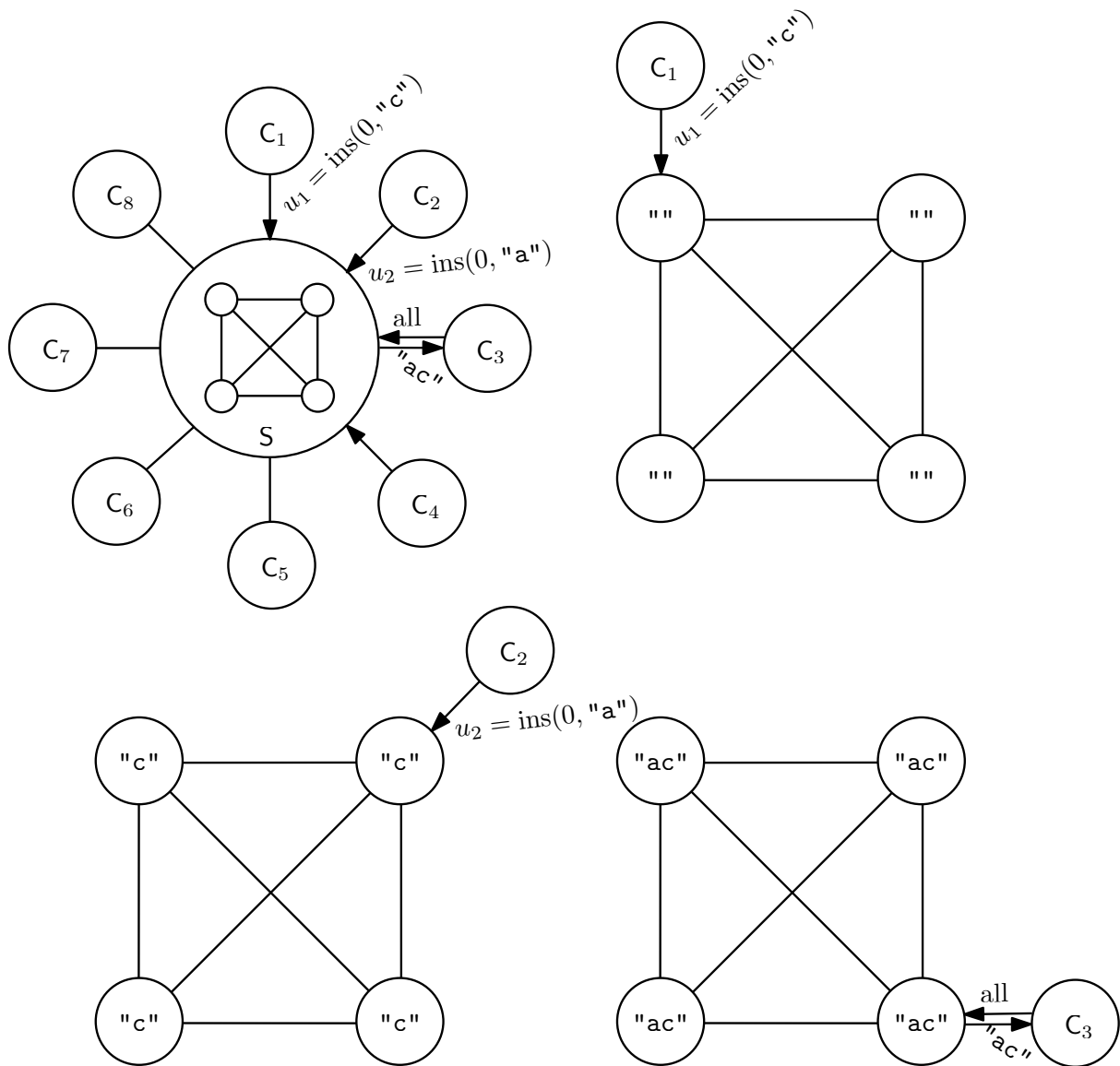


Figure 14.6: State-machine replication of a state machine allowing to update and read strings. Updates can be at different servers, yet all servers should hold the same current state.

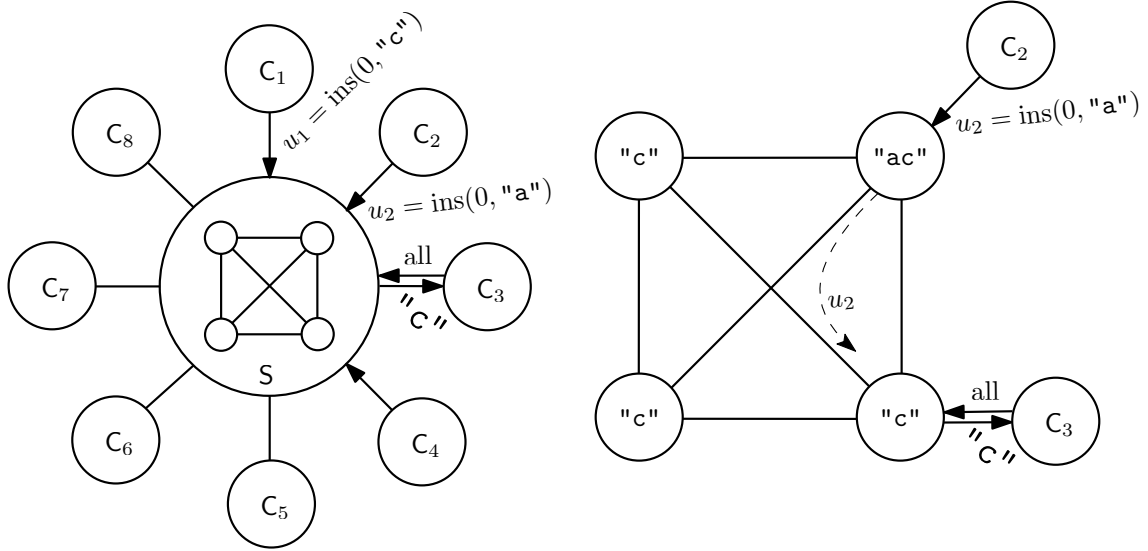


Figure 14.7: Updates cannot propagate immediately, so the ideal functionality has to reflect this.

The ideal functionality is for n servers $S_1, \dots, S_n$, where each server S_i has a port SMR_i for receiving updates and reads and giving outputs. It has a special port **Apply** for receiving updates to be applied from the adversary. It also has a port **Progress** for indicating progress. For each server S_i it also has a local variable c_i for keeping track of how many updates it has applied.

Init: Let $c = 0$ and $State_c = State_0$. Let $c_i = 0$ for all S_i . Let $Tentative = \emptyset$. Let $Log = ()$, i.e., the empty list.

Update: On input u on SMR_i , if $u \in Updates$, then add u to $Tentative$.

Apply: On input u on **Apply**, if $u \in Tentative$, then let $Log = Log || u$, let $State_{c+1} = Upd(State_c, u)$, let $c \leftarrow c + 1$, and remove u from $Tentative$.

Progress: On input S_i on **Progress**, if $c_i < c$ then let $c_i \leftarrow c_i + 1$.

Read: On input r on SMR_i , if $r \in Reads$, then output $y = Rd(State_{c_i}, r)$ on SMR_i .

Figure 14.8: Ideal Functionality SMR_M for State Machine Replication of state machine M

All servers apply the same sequence of updates. They therefore go through the same sequence of states, but are not necessarily in the same state at the same point in time.

The details are given in Fig. 14.8.

14.5 SMR from CSS

We now look at how to get state-machine replication from core-set selection. The main idea is to have the servers run in rounds. In each round they will collect a set of updates that have not yet been applied, and then they will run a core-set selection protocol to agree on which updates to apply. They will then apply those updates in some deterministic order. This ensures that all servers apply the same updates in the same order, so they all end up in the same state.

If an honest S_i updates $State_{c_i}$ with u_j , then it must have received **Core** from CSS^{c_i} and

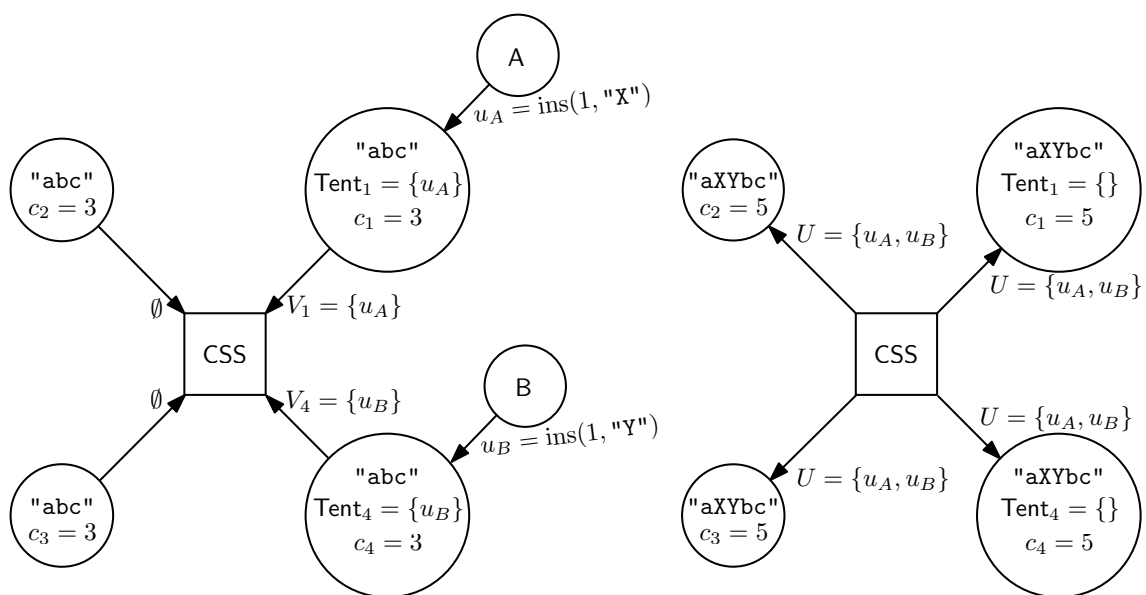


Figure 14.9: An illustration of SMR from CSS. Four servers are in the same state "abc" after applying 3 updates. At server 1 a client insert X in position 1. Around the same time, at server 4 a client insert Y in position 1. These updates enter the tentative sets and are not yet applied. In the next round of CSS the tentative sets are given as input—the V_i values in the left figure. Then a core is computed by the CSS service, and here servers 1 and 4 are in the core. The figure shows just the union of the updates of the core, called U in Fig. 14.10. Then all servers apply the updates from U in some deterministic order, here $u_B; u_A$. Now they all applied 5 updates and are in the same state "aXYbc" again. The servers remove the applied updates from the tentative set.

The protocol has n servers S_i and uses CSS.

- Each S_i has a port SMR_i for receiving inputs and giving outputs for the replicated state machine service.
- Each S_i has a port connected to CSS_i so it can give CSS inputs and get outputs.

The protocol runs as follows. We give the code for S_i .

Init: Let $c_i = 0$ and $State_{c_i} = State_0$. Let $Tentative_i = \emptyset$. Let $CSSisRunning_i = \perp$. Let $Log = ()$ be the empty list.

Start Next CSS: If $CSSisRunning_i = \perp$, then let $CSSisRunning_i \leftarrow \top$, let $V_i = Tentative_i$, and input V_i on CSS_i . Use a session identifier containing c_i to distinguish the different runs of CSS, and call the new one CSS^{c_i} .

Update: On input u on SMR_i , if $u \in Updates$, then add u to $Tentative_i$.

Apply: If $CSSisRunning_i = \top$ and CSS^{c_i} outputs $Core$ on CSS_i , then proceed as follows:

1. $U = \left(\bigcup_{(S_j, W_j) \in Core} W_j \right) \cap Updates$.
2. Sort U in some deterministic way into $\vec{u} = (u_1, \dots, u_m)$.
3. Let $Log = Log \parallel \vec{u}$.
4. For $j = 1, \dots, m$, let $State_{c_i+1} = Upd(State_{c_i}, u_j)$ and $c_i \leftarrow c_i + 1$.
5. $Tentative_i \leftarrow Tentative_i \setminus U$.
6. $CSSisRunning_i \leftarrow \perp$.

Read: On input r on SMR_i , if $r \in Reads$, then output $y = Rd(State_{c_i}, r)$ on SMR_i .

Figure 14.10: RSM for $M = (States, State_0, Updates, Reads, Outputs)$ from CSS

applied the updates in $U = \bigcup_{(S_j, W_j) \in Core} W_j$ (which are legal updates from $Updates$). Since $Core$ has agreement, all honest parties receive the same $Core$. Therefore, all honest parties apply the same updates in the same order. Since M is deterministic, they all end up in the same state after c updates. They might not have seen the same number of CSS terminate, so they might have different c_i , but this is allowed by the ideal functionality. When the updates from U have been applied S_i removes them from $Tentative_i$. The ideal functionality does the same. So the implementation exhibits a legal behavior of the ideal functionality.

Note that a corrupt server S_j can add any updates it wants to CSS, but this just corresponds to the adversary giving these same updates as input to S_j in the ideal functionality, so this is also allowed.

When it comes to liveness, the situation gets more complicated. We have so far not specified any liveness properties. The adversary may never apply any updates or progress any server. There are many choices to be made and we will discuss just a few here. Consider the following two properties:

Strong Eventual Application: If an honest server gets an update u as input, then eventually u will be applied at all honest servers.

Weak Eventual Application: If $t+1$ honest servers get an update u as input, then eventually u will be applied at all honest servers.

It is clear that strong EA implies weak EA. But does weak EA imply strong EA? No, it does

not, and our protocol is a separating example: it has weak EA but not strong EA. The reason is that if only one honest server gets u as input, then it might be the case that this honest server is never in the core, so it never gets to contribute u to the core and therefore u is never applied at any honest server. More concretely, assume that all updates are given to S_1 and S_1 is never in the core. Then all $W_j = \emptyset$ for all W_j in all cores forever.

On the other hand, the protocol *does* satisfy weak EA, because if $t + 1$ honest servers get u as input, then they all add u to their Tentative_i . And u will stay in Tentative_i until it is applied. So, if $t + 1$ honest servers get u as input, then they all have u in their Tentative_i until u is applied. Since the $t + 1$ honest servers use $V_i = \text{Tentative}_i$ in CSS, at least one of their sets Tentative_i will be in the core. Namely, the core has size at least $n - t$, so one of the $t + 1$ honest servers must be in the core. Therefore, u will be in U and therefore it is applied at all honest servers.

It is also clear that we can implement strong EA from weak EA. We can just have the servers exchange their tentative updates with each other, so if one honest server has u in its tentative set, then all honest servers will have u in their tentative set. After this weak EA ensures that u will be applied at all honest servers.

In the exercises we explore another way to implement strong EA from weak EA, which does not require the servers to exchange their tentative updates with each other.¹

14.6 Avoiding Total-Order

In the above we have looked at a very simple state machine replication system. It is not very efficient, but it is simple and it is a good starting point for understanding the basic principles of state machine replication. In practice, there are many optimizations that can be done to make the system more efficient. We will not go into these optimizations in detail, but we will briefly discuss some of them here.

The main source of inefficiency in the above system is that it requires a total order on all updates. This means that all updates have to be ordered in the same way at all servers. This is a very strong requirement and it can lead to high latency, especially in wide-area deployments. As we discussed in Chapter 4 the problem is that when we require total order, then updates cannot be applied immediately at a server, what we called immediate local delivery in Chapter 4. Instead, they have to be ordered first and then applied in that order. This is what we called delayed local delivery in Chapter 4. Since Bracha uses at least two rounds of all servers talking to each other, and SVUP uses at least two rounds of Bracha, our CSS will use at least four rounds of all servers talking to each other before an update can be applied. In a wide-area deployment, latency is dominated by network round-trip time. With a typical RTT of 20–100 ms, a four-round total-order protocol incurs roughly 100–500 ms latency accounting for processing time. This is a very high latency for many applications, especially interactive ones. Let us be kind and use 100 ms.

Consider for instance a database where we write a table and then read from it, say some aggregate query. Then we write and read again. We do this 1000 times. If more clients might access the table then we need total order, so we have to wait for the write to be ordered and applied before we can do the read. This gives us a latency of 100 ms per write-read pair, which means that it will take 100 seconds to do the 1000 write-read pairs. This is a very high latency for a database, and a user typically loses attention after a few seconds and go make a cup of coffee. On the other hand, on modern NVMe storage, a dependent write-read round takes on

¹The curious reader may already consider how to do this. The problem is that S_1 sends a new V_1 in each round, so it might continuously be ignored. All these V_1 's will eventually arrive, but maybe always arrive after the corresponding CSS instance has terminated. What is the solution? Beware of the Godot problem!

the order of 0.1ms in a conservative estimate. This means that we can do the 1000 write-read pairs in 100 ms, which according to the Jakob Nielsen rule of thumb[12] is the maximum latency for an interaction to feel instantaneous.

We would therefore like to avoid the total order requirement. The main trick is to ensure that updates do not need to be ordered in the same way at all servers to end up in the same state or to ensure that there is some simpler way to ensure they are already ordered. We look at some of the techniques to do this below.

14.6.1 Tentative States

The main idea is to have a so-called tentative state at each server. The server will try to apply the updates in Tentative_i in some order and hope that the resulting state is consistent with how the updates later occur in the total-ordered transaction Log , which will also contain tentative Tentative_j from other servers.

In a bit more detail this means that S_i keeps a so-called tentative state

$$\Sigma_{c_i}^{\text{TENT-}i} = \text{Upd}(\Sigma_{c_i}, \text{Tentative}_i) ,$$

where it applies the updates in Tentative_i to its currently finalized state Σ_{c_i} . A client at S_i can then do a so-called tentative read and get back

$$y = \text{Rd}(\Sigma_{c_i}^{\text{TENT-}i}, r) .$$

This is fast, as tentative updates and reads are now local operations. This can be combined with servers forwarding local updates to other servers, which also add them to their set Tentative_j . This way all servers apply all tentative updates, but maybe in different orders. This is illustrated in Fig. 14.11.

However, later when Tentative_i becomes part of the total-ordered transaction Log , then the updates in Tentative_i may be applied in another order in

$$\Sigma_{c_i+m} = \text{Upd}(\Sigma_{c_i}, \vec{u})$$

and may be mixed with updates from Tentative_j from other servers.

A bit of terminology. It is common to call Σ_{c_i+m} the **committed state** and Log the **committed updates** when distinguishing them from the tentative state and tentative updates. They are sometimes also called the **finalized state** and **finalized updates**. Sometimes the tentative state is also called the **speculative state** and applying Tentative_i locally is called **speculative execution**. A tentative read is also sometimes called a **dirty read**.

When a dirty read $y = \text{Rd}(\Sigma_{c_i}^{\text{TENT-}i}, u)$ was done then y might not be consistent with the finalized state $\Sigma_{c_i+m} = \text{Upd}(\Sigma_{c_i}, \vec{u})$. It could for instance be that $u_2 \in \text{Tentative}_i$ was a transaction of 100kr away from an account A with balance 150kr, but in the final log Log another transaction u_1 which is also a transaction of 100kr away from A is injected such that Log contains the sub-sequence $u_1; u_2$. After applying u_1 the balance of A is 50kr, so u_2 will get rejected. In the tentative state of S_i the transaction u_2 was accepted as we assume u_1 happened tentatively at another server. Therefore the dirty read y will have reported the payment as completed to its user. Hopefully the users was also shown a ● to warn the user that the payment is still tentative and awaiting finalization. Later when u_2 finds its place in the finalized Log the status can then go ● or ● depending on whether such u_1 appeared.

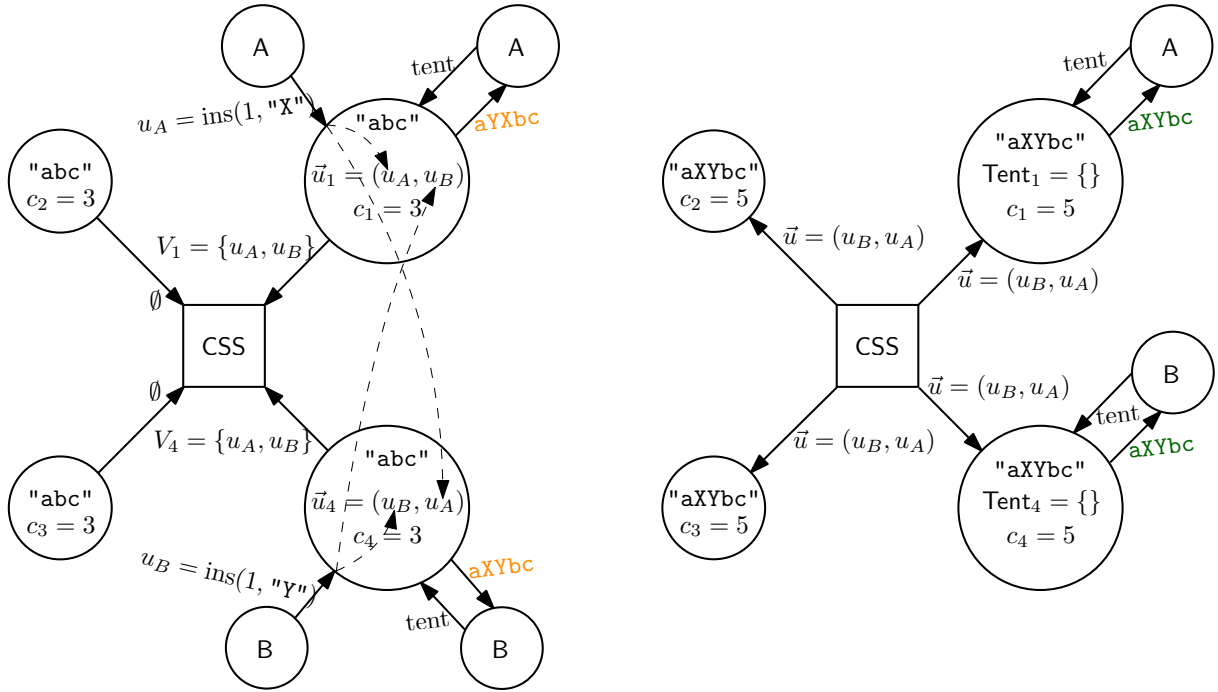


Figure 14.11: Illustration of a tentative read. When doing a tentative read the server apply the updates in their tentative set before returning the result, called speculative execution. When doing speculative execution it can make sense to exchange updates as they arrive, i.e., out-of-band of the CSS mechanism. Here S_1 gets u_A and sends it to all other servers—we only show communication with S_4 . At the same time S_4 gets u_B and sends it to all other servers—we only show communication with S_1 . They apply the updates immediately when seeing them, so they will apply them in different orders. So, a tentative read will yield different results. In the final state after core-set selection, the updates in the core U will be sorted into some order $\vec{u}$. Here they are applied in the order $u_B; u_A$, i.e., the serialized state will evolve as $abc \mapsto aXbc \mapsto aYXbc$. At S_1 the state evolved as $abc \mapsto aXbc \mapsto aYXbc$, and then when $\vec{u}$ arrived it jumped to $aXYbc$. This means that when doing its tentative read A saw a state which the system did not officially move through. In particular, the state machine will never be in the state $aYXbc$ at S_4 , nor will it be in the state $aXbc$. We say that the views of S_1 and S_4 are not serializable. Yet, the clients ended up seeing consistent states in the end. This is called eventual consistency. It might look weird to a user that the state jumps from $aYXbc$ to $aXYbc$. Sometimes it is desirable that the system gets there via a sequence of legal updates like $aYXbc \mapsto aXbc \mapsto aXYbc$, sometimes called operational transformation.

14.6.2 Eventual Consistency and Bounded Divergence

In other applications like collaborative editing seeing inconsistent dirty states might not be a problem. In Fig. 14.11 party B will never be able to see the state **aYXbc** which A saw, but once the parties stop editing they end up in the same state **aXYbc**. This is called **eventual consistency**: if updates stop arriving then the servers end up in the same state. Note that eventual consistency is a funny safety condition. If in Fig. 14.11 two new updates

$$u'_A = \text{ins}(1, "I"), \quad u'_B = \text{ins}(1, "J")$$

would be made the pattern could repeat and on the next reads the clients would see **aJIXYbc** and **aIJXYbc**, where again A sees a state that B can never see. So, in principle A could forever see inconsistent states, yet, if only the parties would stop editing then they eventually see the same. We could in principle implement this by cheating: we just let A and B edit different documents and then at the very end we let the one B wrote be the final version and throw away the work of A. This is eventually consistent, but something seems off. It is therefore common to augment eventual consistency with guarantees about how far the server might drift apart. In our example the servers are never too far away from the last final state $\text{Upd}(\Sigma_0, \text{Log})$. By putting an upper bound on how large Tentative_i can become before we block editing at the clients, like 100 updates, then the tentative states are never more than 100 updates away from the previous final state. And if any two clients do tentative reads at servers with the same $c_i = c_j$ then they see results consistent with states not more than 200 edits apart. This is called **bounded divergence**.

14.6.3 Commutative Updates

One approach to eventual consistency is called **commutative updates**. If two updates commute, then it does not matter in which order they are executed. This could for instance be a write to two different tables in a database. If the updates commute, then we can execute them in any order and end up in the same state. But along the way we might go through different states. This means that we do not necessarily have to agree on the order of these updates. We can just execute them in the order they arrive at each server. This can give a significant performance boost as it allows for more parallelism.

14.6.4 Serializability

If S_1 applies two commutative updates as $u_1; u_2$ and S_2 applies them as $u_2; u_1$, then they will end up in the same state. We can say that S_2 is in a state as if it had applied $u_1; u_2$.

However, at a point where S_1 applied only u_1 and S_2 applied only u_2 , reading the entire state at S_2 does not look like S_2 applied a prefix of the updates $u_1; u_2$.

This is illustrated in Fig. 14.11. This relates to the notion of **serializability**. It means that all observations of a system, like the reads, can be explained as if they were of one system which went through one global sequence of updates.

In this terminology we can say that if one only does reads of finalized states in our system, then the reads are serializable, but if one does dirty reads, then the reads might not be serializable. Serializability is of course very close to the notion of total order broadcast. It is also exactly what our ideal functionality in Fig. 14.8 requires of an implementation. Being an implementation of an ideal functionality means that your input-output behavior is contained in that of the ideal functionality. And observations of the ideal functionality are serializable as they by definition are of a state evolution resulting from applying the one sequence Log .

14.6.5 Locks

It can be hard to know if two updates can safely be applied in any order. If server S_1 sees an update u_1 to table 1 and an update u_2 to table 2 it looks safe to just apply them. But what if server S_2 saw another update u_3 to table 1, which overwrites the same cell as u_1 . Now u_1 and u_2 commute, but u_1 and u_3 do not commute. If this happens often then S_1 will often see dirty reads give results not consistent with the later finalized states, upsetting the efficiency gains we had hoped for.

It comes the concept of a lock. If for instance a client locks a table in a database, then only that client can update that table until it releases the lock. A lock is just another type of update to the state machine. After a client has locked a table, it can then give a sequence of numbered updates $u_1, u_2, \dots$, the rule being that they should be applied FIFO. Now servers can apply these updates in the order they arrive. Updates from other clients to the locked table are ignored. There is no reason to wait for a global order, as only one client can send updates, and this client is doing the ordering. The huge advantage of this is that it allows to apply the updates as soon as they arrive at the servers, without waiting for all of the messages of ARB, ABA and CSS to be sent. At the end, the client gives a special update operation which releases the lock, at which point other clients can start sending updates or locking the table.

The locks themselves are sent via total order such that if two clients try to lock the same table at the same time, then only one of them will succeed. The other one will have to wait until the lock is released. In other words, you only have a lock when it says so in the finalized state. But we have still replaced a lot of slow updates with one slow locking operation and then a lot of fast updates.

Clients can of course proceed optimistically and tentatively apply updates and then throw the work away if they do not get the lock in the finalized state.

14.6.6 Operational Transformation

When tentative updates occur in another order in the finalized Log it is called **reorganisation**. The new finalized state is computed from the finalized order $\vec{u}$. If the local client is continuously viewing the tentative state, as would be the case in collaborative editing like Google Docs, then switching from the tentative state to the finalized state would look like some big jump where a lot of text changes. It can be dizzying for the users if large parts of text suddenly appear or disappear. It is also easier to implement systems which only move forward and never has to roll back to a previous state. In such a system **operational transformation** is often used, which is a technique to add new updates after the already executed tentative order to make the total effect the same as having run the finalized sequence. Now the user would see some sequence of edits happen instead of a big jump.

Consider a document represented as a string, initially

$$s = \text{"abc"} .$$

Two users concurrently issue updates:

$$u_A = \text{ins}(1, \text{"X"}), \quad u_B = \text{ins}(1, \text{"Y"}) .$$

They both see their own update first and then that of the other. When applied tentatively, we obtain:

$$\text{Upd}(\text{Upd}(s, u_A), u_B) = \text{"aYXbc"} , \quad \text{Upd}(\text{Upd}(s, u_B), u_A) = \text{"aXYbc"} ,$$

and the replicas diverge.

Say the finalized order becomes $u_B; u_A$. Replica A first applies

$$\text{"aXbc"} = \text{Upd}(\text{"aXYbc"}, \text{del}(2))$$

and then

$$\text{"aXYbc"} = \text{Upd}(\text{"aXYbc"}, \text{ins}(2, \text{"Y"})) .$$

To A this just looks like B moving Y.

Thus the replicas apply different sequences of updates yet converge to the same state.

Note that B goes through the state evolution

$$\text{abc} \mapsto \text{aYbc} \mapsto \text{aXYbc} ,$$

whereas A goes through the state evolution

$$\text{abc} \mapsto \text{aXbc} \mapsto \text{aYXbc} \mapsto \text{aXbc} \mapsto \text{aXYbc} .$$

Note that this has the same effect as in Fig. 14.11, but we just ensure that B gets to the eventually consistent state via a meaningful sequence of allowed updates instead of jumping to the final state or jumping back to a previous state like `abc` and running forward again with U .

14.6.7 Sequencers a.k.a. Leaders

Another way to ensure that updates are already ordered is to use a **sequencer** or **leader**. This is a special server that is responsible for ordering all updates. All updates go via the leader to get a sequence number. Updates still have to be sent, but now only back and forth, and a sequence of updates can be sent to the sequencer and back as a stream, i.e., we do not have to wait to send u_2 after getting back u_1 , we send u_2 right after sending u_1 . Therefore the delay from a server getting an update from a client to tentatively applying it is the round-trip time of the network, typically well below 100 ms if the leader and client are on the same continent. This is fast enough for collaborative editing to appear instantaneous. The idea is illustrated in Fig. 14.12. A related concept is that of a **primary**. A primary is a special server that is responsible for executing the updates of a particular table. We can think of this as the table being locked to the server and not a client. If a client locked a table it makes sense to make the server that the client is connected to the primary such that all operations are local.

Both ideas introduce a single point of failure. This can be handled by an **error detector** component which will detect cheating leaders. Typically we ask the leader to sign the ordered sequence and we also send it through CSS. If the signed tentative sequence sent to a server does not match the final one, then the server can prove this to the other servers by showing them the signature on its own divergent tentative sequence. When a server detects that the leader is corrupted in inputs **The leader is cheating!** to the CSS. If at some point $t + 1$ servers did this, then we know that at least one honest server did it. We then replace the leader. The new leader can be chosen in a round-robin fashion or by some other **leader election protocol**. It must ensure that all servers agree on who the next leader is.

The above ensures that the leader reports the same sequence to all—or gets replaced—but this does not solve the problem of a crashed or deliberately silent leader. This again leads to the concept of an **error detector** protocol, now for discovering when the leader is down. The other servers could just use a conservative timeout to detect that the leader is down and report their accusation via CSS. When $t + 1$ servers sent **The leader is slow!** in CSS the leader is again replaced. The timeout has to be such that it does not happen too often that an honest server is replaced. A server could also accuse a leader which seems to not include the updates that the server sends to the leader, so called censoring.

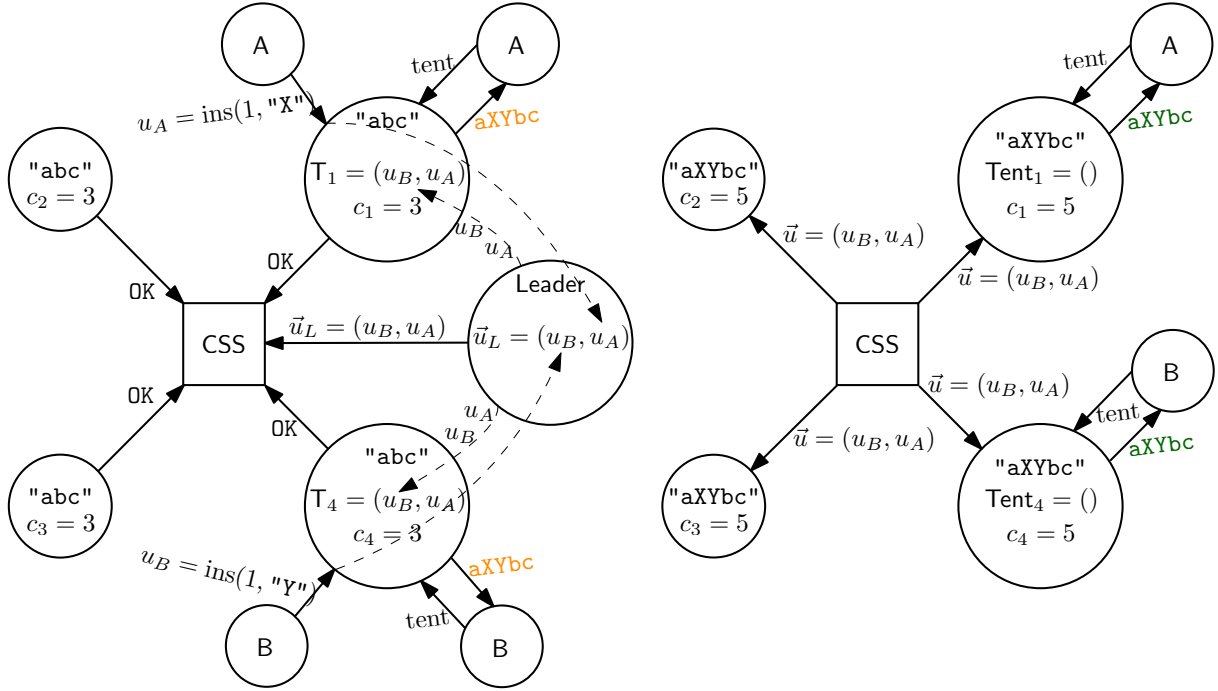


Figure 14.12: Illustration of how to order tentative updates using a leader, a.k.a. sequencer. The updates are sent directly to the leader which orders them and sends them back in a streaming manner. The servers apply them as they arrive. The updates are also added to CSS, as an ordered list of updates. The leader must use the same order as when streaming. The other servers just input an OK signal when all is well. The only way the final state can be different from the tentative one is that the leader is cheating. By requiring the leader to sign the stream a server can prove this cheating to the other servers. Then they all input **The leader is cheating!** to CSS in the next round, and if it ever happens that there are $t + 1$ **The leader is cheating!** in the core (i.e., at least one honest server accused the leader), then the leader is replaced by a new leader. When for a long time the leader does not contribute to the core or add the updates that a server sent to the leader, then that server will input **The leader is slow!** to the CSS. If it happens that $t + 1$ parties do this (i.e., at least one honest server did so), then the leader is also replaced. This works as in a democracy: we cannot ensure to elect good leaders, but we have a mechanism for getting rid of the bad ones in an orderly manner.

The above uses time in a soft way: we guess a timeout of how long it takes a honest server to add our updates to the core. If it is slower we replace it. If we set the timeout too low, the worst that can happen is that we keep replacing honest leaders. If the system keeps changing leaders we can just increase the timeout. However, using time is never for free. In the above solution a corrupted leader could choose to deliberately be as slow as possible without triggering the leader election. This allows it to slow down the system, maybe as a denial-of-service attack. The fully asynchronous system where all parties contribute to the core cannot be slowed down like this. It is as fast that the $t + 1$ fastest honest parties.

14.6.8 Sharding

Another way to make systems more efficient is to use **sharding**. In sharding, the state machine is partitioned into different shards. Each shard is replicated using a separate instance of the state machine replication protocol. Each update is then sent to the shard that is responsible for the part of the state that it updates. This allows for more parallelism as updates to different shards can be executed in parallel without waiting for a global order. However, it also introduces some complexity as updates that span multiple shards need to be coordinated. It also introduces some security issues as an adversary could try to attack one shard and not the others, so it is important to ensure that each shard has enough servers to tolerate the expected number of corruptions. There should also be a mechanism in place to agree on what servers are on what shard and replace them if the shard deadlocks.

This can all be done via a global SMR. Basically, phrase the solution to the coordination problem as a single state machine M , and apply SMR to M among all servers. Then they read M to learn who is on what shard right now and to see what shards have been reported as deadlocked. Such a global component is sometimes called a **configuration service** or **supervisor**. Since it only handles metadata and not the actual updates, it can be implemented across all servers for increased fault tolerance. See for instance [5].

14.7 Reading from just one Server

A final optimisation is that we would like that a client can read from just one server. The problem is that t servers could be corrupted, so the client might be talking to a corrupted server, which might give back a wrong $y \neq \text{Rd}(\Sigma_c, r)$. We can solve this as follows. Whenever a server computed Σ_c , the state after c updates, then compute a short hash

$$h_c = \text{Hash}(\Sigma_c) ,$$

sign h_c and send the signature to all other servers. When a client reads, the server will return Σ_c along with at least $t + 1$ such signatures and the client will check that they sign $\text{Hash}(\Sigma_c)$ and if so, compute $\text{Rd}(\Sigma_c, r)$ itself. This works as $t + 1$ signatures from distinct servers guarantees that at least one honest server signed h_c and then Σ_c must be correct, as the honest server only signs $\text{Hash}(\Sigma_c)$ for the correct Σ_c and it is hard to find collisions for Hash .

14.7.1 Authenticated Data-Structures

It is of course not desirable to return the entire state Σ_c to the client. In practice one instead uses **authenticated data-structures**, where essentially from $h_c = \text{Hash}(\Sigma_c)$ and $y = \text{Rd}(\Sigma_c, r)$ and some short auxiliary proof π the client can be sure that y is computed correctly from the Σ_c which matches h_c . This works even though h_c, y , and π are much shorter than Σ . Using for instance a cryptographic tool called Merkle hash trees it can be done efficiently for reads which look up in an array, i.e., Σ is an array and $\text{Rd}(\Sigma, i) = \Sigma[i]$.

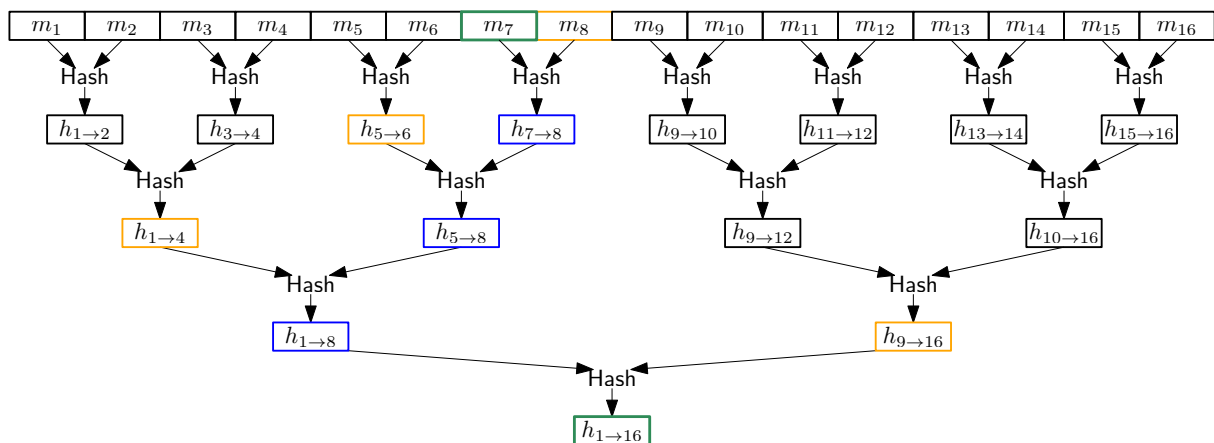


Figure 14.13: Illustration of a Merkle tree. The Merkle hash of a large message is recursively computed as the hash of the Merkle hash of the left half and the Merkle hash of the right half. The hash value is $h = h_{1 \rightarrow 16}$. It authenticates the entire $m = (m_1, \dots, m_{16})$. To reveal just m_7 one sends m_7 and sends along $\pi = (m_8, h_{5 \rightarrow 6}, h_{1 \rightarrow 4}, h_{9 \rightarrow 16})$, i.e., the orange elements. The receiver can then check all instances of **Hash** on the path from m_7 to the root $h = h_{1 \rightarrow 16}$ were computed corrected. The blue elements are compute from m_7 and the proof. Here we send a proof with 4 elements instead of sending the entire m . In general one sends a proof of $\log n$ elements when m is n elements long.

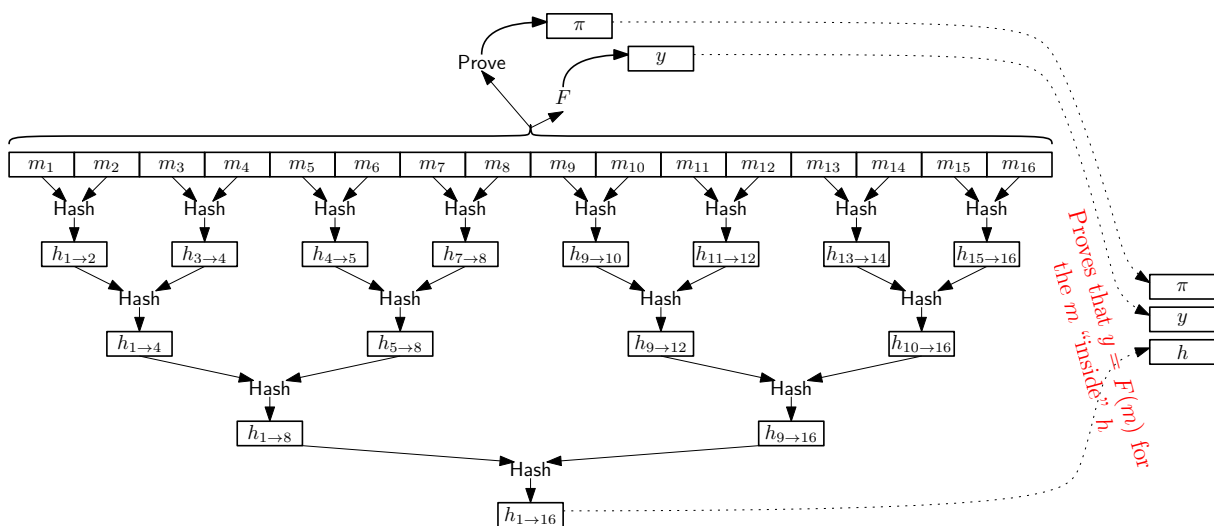


Figure 14.14: Illustration of a SNARK. Now the value y revealed about m can be any function F of m , i.e., $y = F(m)$. One again sends h to define m , i.e., m is the value such that $h = \text{MerkleHash}(m)$. Note, however, that h is too short to define the much longer m . The sender therefore sends along a short proof π which proves that it *knows* m such that $h = \text{MerkleHash}(m)$. Since Merkle hashing is collision resistant this defines m uniquely. The proof also proves that for this m it is true that $y = F(m)$.

14.7.2 Succinct Proofs of Knowledge, SNARKs

Using a cryptographic tool called succinct proof systems it can even be done for any efficiently computable read function, i.e., Σ is a bit string and $\text{Rd}(\Sigma, F) = F(\Sigma)$. The server basically proves that “I know Σ_c such that $h_c = \text{Hash}(\Sigma_c)$ and such that $y = F(\Sigma_c)$ ”. The proof π can be a few kilobytes in size even if the database Σ is Terabytes large and F takes a month to compute. The applied technology is also often called a **SNARK**—Succinct Non-interactive ARguments of Knowledge.

14.8 Client-Centric Consistency

By now we have a good understanding of how to implement a replicated state machine. We have seen that if we have n servers and at most $t < n/3$ of them are corrupted, then we can implement a total-order broadcast channel, which allows us to implement a replicated state machine. This gives us a very strong consistency guarantee: all correct servers will execute the same commands in the same order and therefore end up in the same state. This is what we called **server-centric consistency**. However, this does not guarantee that the servers are in the same states at the same time. It could be that one server is lagging behind and has only executed 8 commands, while another server has executed 9 commands. If a client reads from one server and then from another—maybe the client is mobile and changes servers over time or the underlying system changes to the faster server automatically—it might see a state that is as if it had traveled back in time. There is an illustration of this in Fig. 14.15. Avoiding such problems is what we called **client-centric consistency**.

The main problem to solve is that commands by a single server are consistent over time, even if it interacts with different servers. there are four basic properties that we would like to have. Assuming a system based on a total order of updates they can be phrased as follows. Assume that the servers have executed c commands so far, and the state after executing these c commands is Σ_c . We say that a client who reads from a server with counter c gets a value y_c based on the state Σ_c , i.e., we use c to indicate the counter of the state from which the read was computed.

Monotonic Reads: If a client reads from a server and gets a value y_c and later (in local physical time of the client) reads from another server and gets a value $y_{c'}$, then $c' \geq c$.

Monotonic Writes: If a client makes an update u and later (in local physical time of the client) makes another update u' , then the counter of u' in the total order **Log** becomes larger than the counter of u .

Reads-follows-writes: If a client makes an update u and later (in local physical time of the client) reads from a possibly different server and gets a value $y_{c'}$, then $c' \geq c$, where c is the counter of the update u in the total order **Log**.

Writes-follows-reads: If a client reads from a server and gets a value y_c and later (in local physical time of the client) makes an update u , then the counter of u in the total order becomes larger than c .

We briefly discuss how to achieve these properties. Monotonic reads can be achieved by having the servers keep track of the highest counter they have executed so far and report this to the client along with the value of the read. The client can then keep track of the highest counter it has seen so far and compare it to the value of the next read. If the next read has a lower counter, then it can wait until it has seen a read with a higher counter.

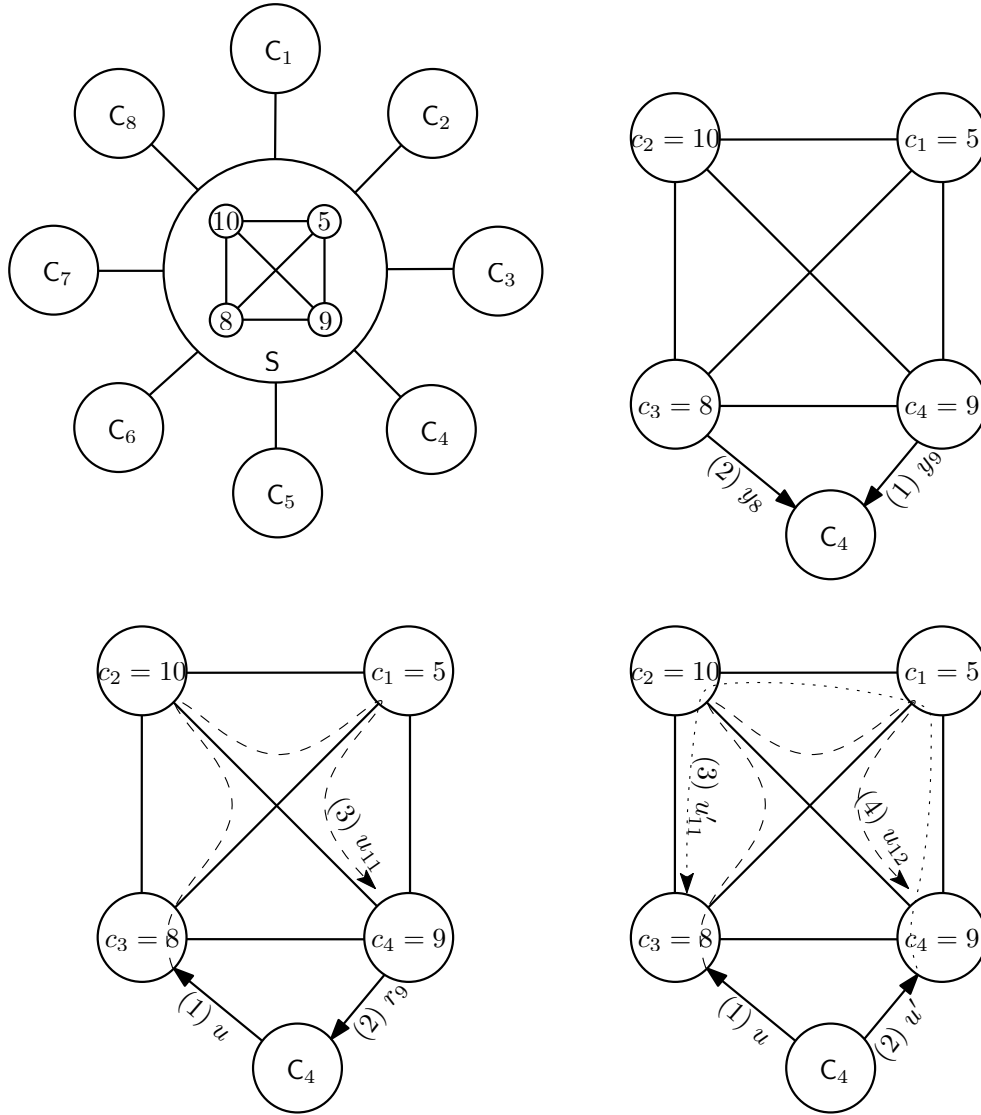


Figure 14.15: Server S_1 has executed 5 commands, server S_2 has executed 10 commands, server S_3 has executed 8 commands, and server S_4 has executed 9 commands. This allows several consistency issues. For instance, if a client reads from S_4 and gets a read y_9 based on the state Σ_9 after executing 9 commands, and then reads from S_3 and gets a read y_8 based on the state Σ_8 after executing 8 commands, then it will see a state that is as if it had traveled back in time. This is called the **monotonic-read problem**. It could also be that it makes an update u on S_3 and then it reads from S_4 and gets a read y_9 , but update u takes time to propagate in the systems—because of all the ARBs and ABAs—and ends up being update number 11. Then the client will see a state that does not include the update u even though it made the update *before* reading. This is what we called the **read-your-writes problem**. It could also be that the client makes an update u at S_3 and then an update u' at S_4 but because we are in an asynchronous model the update u' might arrive before the update u and become update number 11 and u becomes update number 12. This is called the **monotonic-write problem**.

Monotonic writes can be achieved by having the client C keeping a counter of the number of updates it has made and include them in the updates. The replicated state machine can then apply the updates from C in a FIFO manner, which ensures that the updates of C are ordered in the same way at all servers.

As for reads-follows-writes, if the updates of C are ordered in a FIFO manner, as discussed above, then the servers can keep track of the highest counter of C that they have executed so far and report this to the client along with the value of the read. The client can then compare this to its own counter to ensure that all its updates are applied at the next server before reading.

As for writes-follows-reads, this is trivially ensured when the SMR is based on a total order. If the client reads from a server and gets a value y_c , then c updates were already executed at that server, so they are included in the global total order. Any update u that the client makes after reading y_c will be ordered after these c updates, so it will have a counter larger than c .

However, as discussed in the previous section, we sometimes let a client read a local tentative state at a server before the local updates are ordered in the total order. In this case, it could be that the client reads a value y_c based on a tentative state $\Sigma_c^{\text{TENT-3}}$ at S_3 and based on this writes u at S_4 . Now if there is a lock-conflict between S_3 and S_4 then S_3 is made to roll back and the state of S_4 becomes the one in the global total order, then u might end up being ordered before the updates that led to y_c .

This problem can be solved by using sessions, i.e., the reads and writes are part of a session. Some session token t is included in the session identifier of the update u . When S_3 does a roll back, the entire session t is rolled back, which means that u is also rolled back, even if it is sent via another server. So it always gets moved to be with the other operations of the session t , and will probably be applied FIFO within that session. One can look at this as entire sessions being the unit of “update”.

14.9 Exercises

Exercise 14.1 (Total Order by Sequencer (DEPRECATED)) *This exercise is one in a series of practical exercises (Exercise ??, Exercise ??, Exercise ??, Exercise 14.1, Exercise ??) having you implement total-order broadcast using a Nakamoto style blockchain. It could be an advantage to take a glance at the next exercise in the series before you start structuring your software architecture.*

Start from your solution in Exercise ?? and make it into a system using the following idea:

1. Your system runs in two phases. In phase 1 the peers connect to the network. In phase 2 they can send signed transactions.
2. The peer that started the network is a designated **sequencer**.
3. The sequencer creates a special RSA key pair called the sequencer key pair.
4. When connecting to a network the new client is informed who is the sequencer.
5. It is the order in which the sequencer received the transactions that counts. This is communicated to the other peers as follows: Every 10 seconds the sequencer will take the transactions that it saw, but which have so far not been sequenced. Then it puts the IDs of those transactions into a block. A block has a block number and an ordered list of IDs, `[]string`. It numbers the blocks $0, 1, \dots$ in the order they are sent. The sequencer signs the block and sends the block on the network.
6. A client will accept a block if and only if it has the next block number it has not seen yet and the block is signed by the sequencer.

7. All clients process the transactions they receive in the order chosen by the sequencer.
8. A transaction is ignored if it would make the sending account negative.

Your solution should describe

- How your system was designed and why.
- How the TA can run your system.
- Your test must try to send transaction at the same time at different peers to see if the system handles concurrent transactions correctly. A suggestion for one test could be: Have an account with 1,000 coins on it. Have a program P_1 which at replica 1 repeatedly executes a transaction moving 1 coin from account A to B . It should send the transaction 1000 times. Have a program P_2 which at replica 2 repeatedly executes a transaction moving 1 coin from account A to C . It should send the transaction 1000 times. Run the two programs at the same time. Make sure that all replicas see the same number of coins end up on all accounts. check that account A is 0 when it is all done. If your system is too slow to do 2000 transactions, pick a lower number. But it is important that the test runs long enough that both programs are running at the time when account A hits 0.
- How the system was tested and how to run your test if you did an automatic test.

You do not need to:

- Handle errors, neither Byzantine nor crash errors. In particular, if the sequencer crashes, then the system is allowed to die.
- You do not have to make your test automatic, but it is highly recommended.



Exercise 14.2 (Client-Centric Consistency I) Assume that all servers are honest and server-consistent, but might drift apart in how many commands they executed. Consider the fully asynchronous model. Consider a client that wants to move from S_1 to S_2 . Which of the following methods give monotonic reads:

1. Log off S_1 , wait for an hour and then start reading from S_2 .
2. Before moving ask S_1 how many commands it executed. Call this number c . When logging onto S_2 , ask it how many commands it executed. Wait until it executed at least c commands, then start reading from S_2 .
3. Before moving send a big random number N into the state machine M : say it has a part of its state where everyone can write what they want. When logging onto S_2 , wait until N arrives in M at S_2 , and then start reading from S_2 .



It is often desirable that a client does not need to keep state between sessions. The next exercise asks you to think about a system that achieves that even when there are corrupted servers.

Exercise 14.3 (Client-Centric Consistency II) Assume that there are n servers and that t can be maliciously corrupted and $n = 4t + 1$. The honest servers are server-consistent, but might drift apart in how many commands they executed. When a server is contacted it replies with (c, S) where c is how many commands it executed and S the state it claims to be in after executing c commands. Consider the asynchronous model where a client might ask for a report, but only is guaranteed that the honest servers eventually report it. You can assume that the system keeps getting inputs, so for any number c , eventually all servers will have executed c commands. Which of the following methods allow clients to read with monotonic read and eventual liveness:

1. Whenever the client C wants to read the state of the system it reads from any $t + 1$ servers and takes the earliest state reported, i.e., the state resulting from executing fewest commands. In a bit more detail, when we say that it reads from $t + 1$ servers we mean that it contacts all servers, but progresses already when hearing from the first $t + 1$. We do not mean that it contacts $t + 1$ servers and waits to hear from all of them, as this would violate liveness: some of them might not respond.
2. Whenever the client C wants to read the state of the system it reads from any $t + 1$ servers and takes the latest state reported, i.e., the state resulting from executing most commands.
3. Whenever C wants to read the state of the system it reads from any $2t + 1$ servers and records the largest number of commands executed by any server. Call this number c . It asks all servers to report their state after c commands and the servers report this state if and when they executed c commands. Then the client waits until $2t + 1$ servers report having executed c commands and records their reported state after executing c commands. Then it takes the state which is reported by at least $t + 1$ of the servers having executed c commands.
4. Whenever C wants to read the state of the system it reads from any $3t + 1$ servers and records the largest number of commands executed by any server. Call this number c . Then it waits until $2t + 1$ servers report having executed c commands and records their reported state after executing c commands. Then it takes the state which is reported by at least $t + 1$ of the servers having executed c commands.
5. Whenever C wants to read the state of the system it reads from any $3t + 1$ servers and records the largest number of commands executed by any server. Call this number c . Then it waits until $3t + 1$ servers report having executed c commands and records their reported state after executing c commands. Then it takes the state which is reported by at least $t + 1$ of the servers having executed c commands.
6. Whenever C wants to read the state of the system it reads from any $4t + 1$ servers and records the largest number of commands executed by any server. Call this number c . Then it waits until $4t + 1$ servers report having executed c commands and records their reported state after executing c commands. Then it takes the state which is reported by at least $t + 1$ of the servers having executed c commands.



The main building block to implement state-machine replication is **total-order broadcast**. We already defined total-order broadcast in Chapter 4 so we will not go into details here. Total-order broadcast is a protocol for n servers $S_1, \dots, S_n$, where S_i has a port TOB_i for receiving

inputs and giving outputs. During the protocol a servers S_i will receive a sequence of input $x_{i,1}, x_{i,2}, \dots$. When read the system outputs a sequence of outputs $y_{i,1}, y_{i,2}, \dots$. The protocol should have the below trace properties.

The following property says that honest inputs are eventually delivered.

Definition 14.3 (Eventual Delivery) If an input x is input on $S_i.TOB_i$ for an honest S_i then eventually x will appear as output on $S_j.TOB_j$ at all honest S_j .

The following property says that if all honest servers have output ℓ or more values, then they agree on the first ℓ values delivered. Some servers might have delivered more elements than others, but then the others will eventually catch up.

Definition 14.4 (Eventual Consistency) If an honest server S_i has output $x_{i,1}, \dots, x_{i,\ell_i}$ and an honest server S_j has output $x_{j,1}, \dots, x_{j,\ell_j}$ and $\ell_i \leq \ell_j$ then $x_{i,1}, \dots, x_{i,\ell_i} = x_{j,1}, \dots, x_{j,\ell_i}$. Furthermore, if an honest S_i has output ℓ elements on TOB_i , then eventually all honest parties S_j will output ℓ elements on TOB_j .

Exercise 14.4 (TOB from SMR) Consider the state machine M which has $\Sigma_0 = ()$, the empty list, and where $\text{Upd}(\Sigma, u) = \Sigma \| u$, i.e., u is appended to Σ , and where $\text{Rd}(\Sigma, \cdot) = \Sigma$. Argue that SMR of M implements TOB. Technically we need to make the transformation that when a server sees a message x it sends it to all other servers, who all input as an update, and the protocol constantly reads $\text{SMR}_M.\text{SMR}_i$ and when the returned list Σ grew it outputs the new elements on TOB_i in the order they appear in Σ . Notice that the argument should not go into the implementation of SMR, you must argue that the desired properties of TOB follows from the guaranteed properties of SMR. ▼

Exercise 14.5 (Synchronous total-order Broadcast (I)) Consider the following protocol for the synchronous round-based model trying to implement total-order broadcast. We consider $t < n/3$ Byzantine corruptions and assume the used sub-protocols are secure against that number of corruptions.

1. When a new message x arrives at a server the server floods the message. Assume the flooding network could output at different parties in different rounds. So, it is for instance done by signing messages and relaying signed messages.
2. In each round each server takes the messages that it received from all the other parties, orders them lexicographically and outputs them in that order.

Does it work? Justify you answer. ▼

Exercise 14.6 (Synchronous total-order Broadcast (II)) Consider the following protocol for the synchronous round-based model trying to implement total-order broadcast. We consider $t < n/3$ Byzantine corruptions and assume the used sub-protocols are secure against that number of corruptions.

1. When a new message x arrives at a server the server floods the message. This rule is always active, i.e., it goes on in the background independently of the below rules.
2. In each so-called epoch, in some pre-agreed round each server takes the messages that it received from all the other parties during this epoch and broadcasts them using RB. Say that RB terminates at all parties in the same round to avoid that parties desynchronise. A bit more precisely: The first round of RBs is run in round 1. The next round of RBs is run in the round after the first round of RBs all terminated and so on. One epoch is the rounds it takes to run one round of RBs.

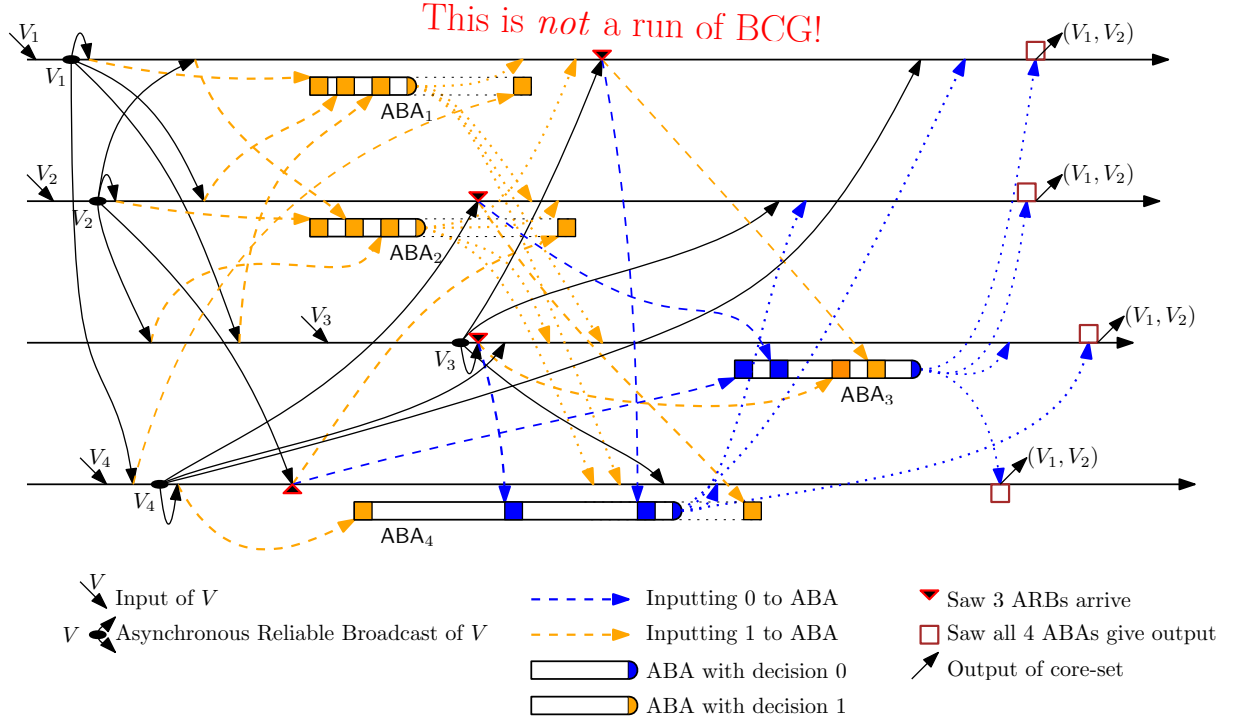


Figure 14.16: Illustration of notBCG used in the exercises. Each server will wait until it hears 3 of the ARBs arrive and vote 0 in the missing one. Since all servers vote 0 in at most 1 ABA, there are at most 4 votes for 0. If distributed on two of the ARBs, this is enough to make 2 of the ARBs output 0, but not more. So, the core will only have size 2. But it cannot get below size 2 with only 4 votes for 0.

3. At the end of an epoch, when all the RBs terminated, all servers takes the union of the sets output by the RBs, orders them lexicographically and outputs them in that order. Then a new epoch starts, i.e., they go back to Step 2.

Does it work? Justify your answer. ▼

Exercise 14.7 (Asynchronous total-order Broadcast (I)) Consider the following protocol for the asynchronous model trying to implement total-order broadcast. We consider $t < n/3$ Byzantine corruptions and assume the used sub-protocols are secure against that number of corruptions.

1. When a new message x arrives at a server the server floods the message.
2. In each “epoch” each server waits to receive a message from each of the other servers. Then it orders them lexicographically and outputs them in that order. Then a new epoch starts, i.e., they go back to Step 1.

Does it work? Justify your answer. ▼

Exercise 14.8 (Asynchronous total-order Broadcast (II)) Consider the following protocol for the asynchronous model trying to implement total-order broadcast. We consider $t < n/3$ Byzantine corruptions and assume the used sub-protocols are secure against that number of corruptions.

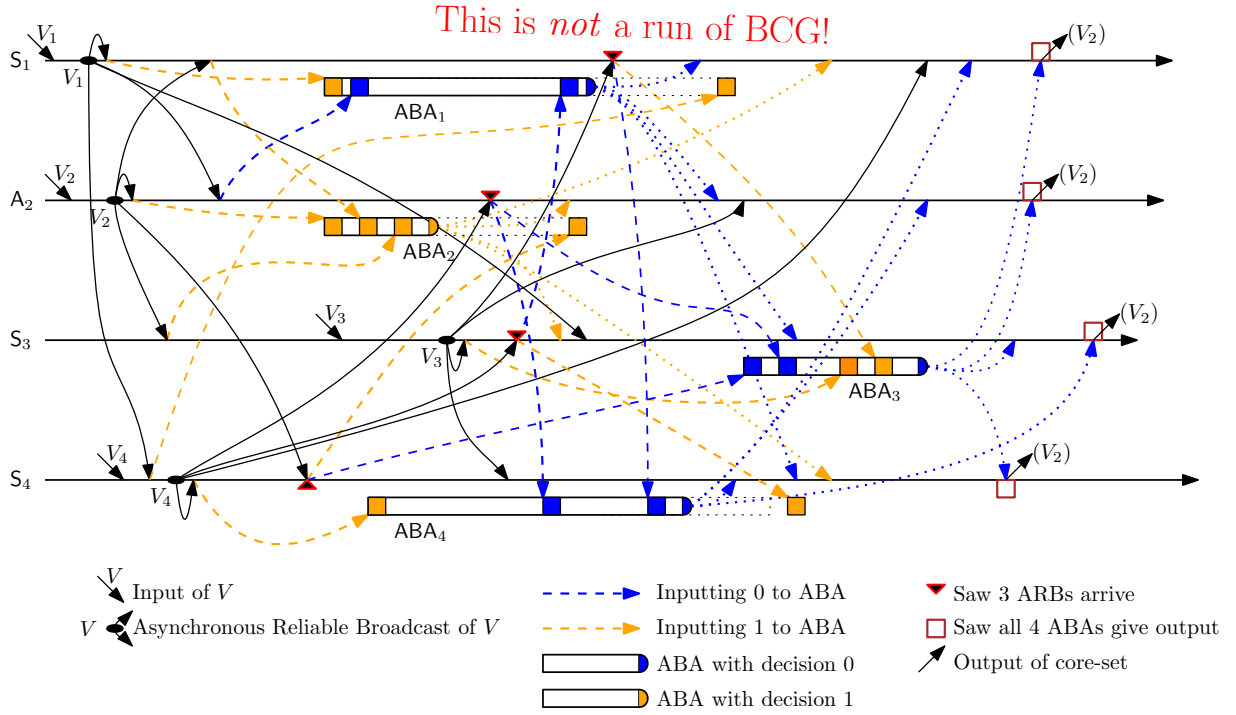


Figure 14.17: Illustration of notBCG used in the exercises. Each server will wait until it hears 3 of the ARBs arrive and vote 0 in the missing one. If there are three honest servers this gives 3 votes on 0 from honest servers. If there is a corrupted server which votes 0 in the ABA of all the other servers, then that is 3 additional votes for 0. If distributed equally among 3 ABAs this is enough to make three ABAs output 0. On the other hand, even if the adversary votes 0 in all ABAs, this is still only 7 votes for 0. So it cannot be that all 4 ABAs get 2 votes for 0, as that would be 8 votes for 0. So one ABA will have to get 3 votes for 1, and this one will output 1. The adversary can make sure it is the ABA for the corrupted party, so the core consist only of values controlled by the adversary.

1. When a new message x arrives at a server the server broadcasts the message using ARB.
2. Each server then waits to receive a message from $n - t$ of the other servers. Then it broadcasts this set using ARB.
3. Each server then waits to receive these sets of message from $n - t$ of the other servers. Then it picks the set from the server with the lowest name (order the names lexicographically say). Then it orders and outputs the transactions sent by that server.
4. Then go to Step 1.

Does it work? Justify your answer. ▼

Exercise 14.9 (Not BCG I) Consider the following variant of BCG core-set selection, which we will call notBCG. Instead of waiting for $n - t$ ABAs to output 1 and then inputting 0 to the remaining t ABAs, the parties will wait until they saw $n - t$ ARBs arrive. Then they vote 1 in the corresponding ABAs and 0 in the ABAs for the t servers from which they did not receive an ARB. The protocol is illustrated in Fig. 14.16 for $n = 4$ and $t = 1$. We can assume that an ABA for $n = 4$ will not output 0 unless it gets 2 votes for 0, as a single vote for 0 could be from a corrupted server and the ABA is only allowed to decide 0 if some honest server voted 0. On the other hand, if an ABA gets 2 votes for 0 then it may output 0 as $2 = t + 1$, so some honest server must have voted 0. So, let us assume for the sake of this exercise that when an ABA gets 2 votes for 0 it always ends up with decision 0. When all four servers are honest, then there are at most 4 votes for 0, so at most 2 ARBs can get 2 votes for 0. So, the core will have size at least 2 even under the worst possible scheduling by the adversary. Your job is to consider what happens for $n = 7$ and $n = 10$.

1. For the case $n = 7$ and $t = 2$ how small a core can the adversary create by nasty scheduling when all servers are honest. Recall that each server now inputs 0 to $t = 2$ ABAs and an ABA needs $t + 1 = 3$ votes for 0 to decide 0. Give a concrete scheduling producing the size of core you claim is the smallest and give an argument why a smaller core cannot be produced.
2. Is it guaranteed that there is an honest V_i in the core when $n = 7$ and $t = 2$?
3. For general $n = 10, 13, 16, \dots$ can you see how small the core can be made?
4. Is it guaranteed that there is a honest V_i in the core when $n = 10, 13, \dots$ and $t = 3, 4, \dots$? ▼

Exercise 14.10 (Not BCG II) Consider the variant of BCG which we called notBCG above. We can assume that an ABA for $n = 4$ outputs 0 if it gets 2 votes for 0. Consider now the case where a server is corrupted and inputs 0 to some ABAs where it should have inputted 1. In Fig. 14.17 there is a run where the adversary schedules such that the core only has size 1. Even if the corrupted party inputs 0 to all 4 of the ABAs, then there will be at most 7 votes for 0 along with the 3 honest votes for 0. So, not all 4 ABAs can get 2 votes for 0. So at least one ABA will output 1. So the core will have size at least 1. Argue that for $n = 7$ and $t = 2$, if two corrupted servers vote 0 to all ABAs, then there is a scheduling such that the core will end up with size 0. ▼

Exercise 14.11 (SMR—Empty Runs) Notice that in our implementation of SMR S_i starts running the next CSS^{c_i} once the previous one stopped. Even if $Tentative_i$ is empty. This means

that if no updates arrive anywhere, then the system will be running idle, producing a lot of traffic to handle 0 updates. This might not be desirable. Consider the following two attempts of only running when there are updates to handle.

1. CSS1: Server S_i will not start the next CSS^{c_i+1} until $Tentative_i$ is non-empty.
2. CSS2: Server S_i will start the next CSS^{c_i+1} if one of the the following happen:
 - (a) $Tentative_i \neq \emptyset$.
 - (b) Inside the run of CSS^{c_i+1} some ARB_j returned a W_j such that $W_j \cap \text{Updates} \neq \emptyset$ and ABA_j decided 1. I.e., some S_j already sent a non-empty $Tentative_j$ for the next round.

Argue that CSS1 is a CSS but without even weak EA, i.e., it can happen that $t+1$ honest servers get an update u , but yet u never gets applied. Then argue that CSS2 is a CSS and that it does have weak EA. [Hint: the addition to CSS2 which is not in CSS1 should tell you why CSS1 does not have weak EA.]

▼

Exercise 14.12 (SMR—Strong EA) Notice that in our implementation of SMR the adversary could choose to not schedule $S_1, \dots, S_t$ until CSS is done. This is allowed, as the only restriction on the scheduling is that each V_j is eventually delivered. And they *are* eventually delivered, just always too late to be in the core. Therefore $S_1, \dots, S_t$ are never in Core. So, if only these servers get updates, then no updates ever enter the core. Therefore the protocol does not have strong EA. We want to fix this.

Consider the following modification. When running CSS^{c+1} , if it is the case that P_j was not in the core of the previous run CSS^c , then in CSS^{c+1} party P_j will not ARB a new V_j . Instead, in CSS^{c+1} the parties wait for the ARB of V_j from CSS^c instead. This is done recursively, so if P_j was not in the core of any of the previous runs $CSS^1, \dots, CSS^c$ then in CSS^{c+1} the parties wait for the ARB of V_j from CSS^1 .

Argue that this modified protocol has strong EA.

▼

14.10 Eventually Synchronous Implementation of Total-Order Broadcast*

In this section we present a total-order broadcast algorithm for the eventually synchronous model. In this model we assume that there is some bound Δ_{GUESS} on message delivery that is known to the protocol. However, initially the adversary is not required to actually deliver messages in time Δ_{GUESS} . However, the ideal functionality for communication has a special port **EventualsNow**. If the adversary inputs a message on **EventualsNow**

The ideal functionality has a special port **EventualBound**. As the first thing the adversary has to input a integer Δ_{GUESS} on this port. This integer is output to all parties. The ideal functionality for communication has a special port **EventualsNow**. If the adversary inputs any message on **EventualsNow**, then, from that time on all messages which have ever been sent and which will ever be sent must be delivered to all parties before Δ_{GUESS} time passed. When an input is given on **EventualsNow** we say that the network has become synchronous. We do not tell the parties of the protocol that an input has been given on **EventualsNow**, so they do not know when the network has become synchronous.

When we prove liveness of a protocol in the eventually synchronous model, then we require that if the network becomes synchronous, then the protocol will eventually start having all its liveness properties forever after.

14.10.1 Discussion of the Eventually Synchronous Model

It looks like a very strong model to assume that there is a point in time after which the network is synchronous forever. A seemingly strong model would be to have a network which could become synchronous on and off and then require that the protocol is live whenever the network is synchronous for long enough. However, this would be exactly the same model. To see this, consider a protocol for the eventually synchronous mode. Consider two different settings. In setting A the adversary inputs a message on **EventualsNow** and starts delivering all messages in time Δ_{GUESS} . In setting B the adversary does not input a message on **EventualsNow**, but still starts delivering all messages in time Δ_{GUESS} . In setting A the protocol will eventually become live. Since the protocol is not told if a message was input on **EventualsNow** the protocol cannot behave differently in the two settings. Therefore, in setting B the protocol will eventually become live. Later in setting B, if the adversary delivers messages slow again, the protocol might loose liveness again, but it cannot delete the liveness already produced as liveness properties like having delivered a message are persistent. Furthermore, if in setting B the adversary later actually input a message in **EventualsNow** and again starts delivering messages fast, then the protocol must become live again. This argument can be repeated and it shows that a protocol for the simple model in fact has the strong property that each time the network become synchronous for long enough, the protocol must become live. But importantly, we have a simple model forcing the protocol to have this property without us having to go through the pain of defining synchrony that can turn on and off and defining what *long enough* means.

14.10.2 Optimistic Protocol

We will use the eventually synchronous protocol to illustrate an important design principle for the eventually synchronous model. We will make a protocol which is very efficient when the network synchronous and which will then go into an expensive cleanup mode when the network is not synchronous. The protocol will never loose safety properties, not even in asynchronous periods.

As usual the protocol will have a subsystem which uses **Flood** to flood all messages that arrive on **TOB**. TOB_i and an honest P_i and add them to a local variable UnQueued_i when delivered by **Flood**. We will therefore just assume that the parties have access to the local variables UnQueued_i which eventually contain the messages to be broadcast on **TOB**.

Agreement If for two honest parties P_i and P_j it holds that at some point in time $\text{Unique}_e^{P_i} = U_i \neq \perp$ and at some point in time $\text{Unique}_e^{P_j} = U_j \neq \perp$, then $U_i = U_j$.

This is easy to see. For an honest P_i to set $\text{Unique}_e^{P_i} = U_i \neq \perp$ it saw $n - t$ unique signatures on (ECHO, e, U_i) . For an honest P_j to set $\text{Unique}_e^{P_j} = U_j \neq \perp$ it saw $n - t$ unique signatures on (ECHO, e, U_j) . This is at least $2(n - t) > 4t + 2 = n + t + 1$ unique signatures. So at least $t + 1$ parties signed two different values. This does not happen as there are at most t corrupt parties and honest parties only sign one value.

Validity If an honest P_i sets $\text{Unique}_e^{P_i} = U \neq \perp$, then P_L sent a signed (epoch, U) .

Honest Liveness If P_L is honest, then for all P_i and all e it eventually happens that $\text{Unique}_e^{P_i} \neq \perp$.

Since the leader adds heart beats if no other transactions arrived, the leader will eventually execute Leader Signature for all e . Since it signs only one U per e , the $n - t$ honest parties will flood $\text{Sig}_{\text{sk}_i}(\text{ECHO}, e, U)$ for a unique (e, U) for e . These messages will eventually arrive at all honest parties making them execute Unique.

The FastTrack system.eventobi

Initialise Each P_i gets input $(\Delta_{\text{GUESS}}, \text{Queued}, L)$. The honest parties agree on Δ_{GUESS} , Queued , and L . Initially, let $\text{epoch}_i = 0$, $\text{Queued}_i = \text{Queued}$, $\text{Signed}_e^{P_i}(U) = \perp$ and $\text{Agreed}_e^{P_i} = \perp$ for all epochs $e \geq \text{epoch}_i$ and blocks U . Initially the leader sets $\text{Sent} = \text{Queued}_L$.

Leader Signature Leader P_L : If $\text{UnQueued}_L \neq \text{Sent}_L$, then let $U = \text{UnQueued}_L \setminus \text{Queued}_L$, let $\text{Sent} = \text{Sent} \cup U$, flood signed (epoch, U) , and let $\text{epoch} = \text{epoch} + 1$. We assume that all signed messages contains a session identifier to prevent replay attacks across different executions of FastTrack.

Timing Stuff In *Leader Signature* a delay should be put in to make sure that each new message does not become its own block. The correct block size could be determined experimentally for the given network and traffic. We will ignore this here for simplicity.

Similarly, if $\text{UnQueued} = \text{Sent}$ for too long (no new transactions) the leader could add HEARTBEAT to UnQueued to trigger *Leader Signature*. We assume P_L will add a dummy message to UnQueued every Δ_{GUESS} seconds if no other transaction arrived.

Receiving Signed Blocks P_i : On a message (e, U) signed by P_L , set $\text{Signed}_e^{P_i}(U) = \top$.

Echo First Signed Block P_i : When it happens for the first time for e that $\text{Signed}_e^{P_i}(U) = \top$ for some $U \neq \perp$ and $\text{Unique}_{e-1}^{P_i} = \top$, flood $\text{Sig}_{\text{sk}_i}(\text{ECHO}, e, U)$.

Unique P_i : On $n-t$ values $\text{Sig}_{\text{sk}_i}(\text{ECHO}, e, U)$ for the same (e, U) , where $\text{Signed}_e^{P_i}(U) = \top$, set $\text{Unique}_e^{P_i} = U$.

Unique Prefix P_i : At all times let prefix_i be the number for which $\text{Unique}_{\text{prefix}_i+1}^{P_i} = \perp$ and $\text{Unique}_e^{P_i} \neq \perp$ for $e = 1, \dots, \text{prefix}_i$.

Finalize P_i : At all times let $\text{final}_i = \text{prefix}_i - 1$. If for some $e < \text{final}_i$ and some U it holds that $\text{Unique}_e^{P_i} = U$, then let $\text{Agreed}_e^{P_i} = U$.

Output P_i : At all times output on TOB_i the missing blocks to ensure that the output on TOB_i is $(0, \text{Agreed}_0^{P_i}, \dots, (\text{final}_i, \text{Agreed}_{\text{final}_i}^{P_i}))$.

Figure 14.18: Eventually Synchronous total-order Broadcast (I)

Propagation If for some e and some honest P_i it happens that $\text{Unique}_e^{P_i} \neq \perp$ then it eventually happens for all honest P_j that $\text{Unique}_e^{P_j} \neq \perp$. This holds even if the activation rule Echo First Signed Block was deleted at some honest parties (a fact we use later).

This is easy to see. Namely, if $\text{Unique}_e^{P_i} \neq \perp$ then P_i received $n-t$ $\text{Sig}_{\text{sk}_i}(\text{ECHO}, e, U)$. These were sent using flooding and therefore eventually will arrive at all honest parties P_j making them execute Unique.

14.10.3 Shutting Down

We now look at a method to gracefully shut down the TOB. Parties can get input SHUTDOWN. In response to this, all honest parties should eventually output some $(\text{SHUTDOWN}, Q_i)$, where Q_i is the epoch in which they stopped. We want the following properties.

Almost Agreement If two honest parties output Q_i and Q_j then $Q_i \leq Q_j + 1$.

At most One Rollback If an honest party output $(\text{SHUTDOWN}, Q_i)$, then $\text{Unique}_{Q_i+2}^{P_i} = \perp$.

No Missing Blocks If an honest party outputs $(\text{SHUTDOWN}, Q_i)$, then $\text{Unique}_{Q_i}^{P_i} \neq \perp$.

Liveness If all honest parties get input SHUTDOWN, then eventually all honest parties output some $(\text{SHUTDOWN}, \cdot)$.

The properties *No Rollback* and *No Missing Blocks* just says that parties quit after the last block they output. The reason why we only require agreement within one epoch on when to quit is that if we assume exactly agreement, we are solving an agreement problem, and we would be forced to use randomness. The above simpler property can be achieved using a deterministic protocol.

Shutdown On input SHUTDOWN delete the activation rule *Echo First Signed Block*.

Report Prefix When deleting activation rule *Echo First Signed Block*, flood $\text{Sig}_{sk_i}(\text{SHUTDOWN}, \text{prefix}_i)$ for the value of prefix_i when the activation rule was deleted.

Report Prefix Update After deleting *Echo First Signed Block*, flood $\text{Sig}_{sk_i}(\text{SHUTDOWN}, \text{prefix}_i)$ for the value of prefix_i .

Furthermore, whenever hereafter prefix_i becomes larger flood $\text{Sig}_{sk_i}(\text{SHUTDOWN}, \text{prefix}_i)$ again. It is OK to put in a heuristic delay so one does not send out $\text{Sig}_{sk_i}(\text{SHUTDOWN}, \text{prefix}_i)$ each time prefix_i increases. It should just hold that if prefix_i increases to p then P_i eventually sends $\text{Sig}_{sk_i}(\text{SHUTDOWN}, p')$ for $p' \geq p$. A good heuristic could be to wait $2\Delta_{\text{GUESS}}$ before sending the first update

Shutdown When $n - t$ values $\text{Sig}_{sk_j}(\text{SHUTDOWN}, Q_i)$ arrived for the same Q_i for the first time wait until $\text{Unique}_{Q_i}^{P_i} \neq \perp$. Then output $(\text{SHUTDOWN}, Q_i)$ and terminate. Keep running Flood to help processes still alive to receive their messages.

Figure 14.19: Eventually Synchronous total-order Broadcast (II). The Shutdown sub-system.

No Missing Blocks The property *No Missing Blocks* follows from the activation rule *Shutdown*: by construction an honest party does not output $(\text{SHUTDOWN}, Q_i)$ until it has $\text{Unique}_{Q_i}^{P_i} \neq \perp$.

At most One Rollback We prove the property *At most One Rollback*. Assume for the sake of contradiction that an honest P_i outputs $(\text{SHUTDOWN}, Q_i)$ and has $\text{Unique}_{Q_i+2}^{P_i} = \perp$. From

Almost Agreement We prove the property *Almost Agreement*.

We say P_i has reached epoch S if $\text{Unique}_{S^{P_i}} = \top$. Let R be the highest slot reached by an honest party before all honest parties terminated by outputting $(\text{SHUTDOWN}, Q_i)$ or were deadlocked because all messages had been delivered and there were no more rules that could be triggered to produce new messages.

Let T_{P_i} be the slot that P_i reached when P_i got input SHUTDOWN and deleted the activation rule *Echo First Signed Block*.

Let T_S be the number of honest P_i with $T_{P_i} = S$.

Let T be the highest slot such that $\leq t$ parties timed out while having reached T , i.e., $\sum_{S \leq T} T_S \leq t$ and $\sum_{S \leq T+1} T_S > t$.

Assume that P_i output $(\text{SHUTDOWN}, Q_i)$. Then we say that P_i had quorum at Q_i . This implies that Q_i received $n - t$ values $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, Q_i)$ and did not receive $n - t$ values $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, Q_i - 1)$ before that. We say epoch Q had quorum of some honest party had quorum at Q . Let $\mathbb{Q}$ be the set of epochs that had quorum.

An honest party only send $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, S)$ if they reach S , so for all $Q > R$ no honest party sends $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, S)$. This implies that:

Fact 14.5 *If $Q \in \mathbb{Q}$ then $Q \leq R$.*

An honest party only ever sends $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, S)$ if they were in S or smaller when they got SHUTDOWN , so for all $Q \leq T$ at most t honest party sends $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, S)$. The t corrupted parties might also send $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, S)$, but if $Q \leq T$ then at most $2t < n - t$ signatures $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, S)$ are received by any honest party. This implies that:

Fact 14.6 *If $Q \in \mathbb{Q}$ then $Q \geq T + 1$.*

Finally we have that

Fact 14.7 $T \geq R - 2$.

To see this note that some P_i reached R and hence had $\text{Unique}_R^{P_i} = \top$. This implies that P_i received $\text{Sig}_{\text{sk}_j}(\text{ECHO}, R, U)$ from $n - t$ parties. Of these $n - t$ parties at least $t + 1$ are honest. Honest parties P_j do not send $\text{Sig}_{\text{sk}_j}(\text{ECHO}, R, U)$ if they got input SHUTDOWN before $R - 1$, as they delete the activation rule *Echo First Signed Block* on input SHUTDOWN . So there are $t + 1$ honest parties which at least reached $R - 1$ before they got input SHUTDOWN . We prove the property for the case with exactly t corrupted parties and $n = 3t + 1$. It trivially also holds for less than t corrupted parties then. So if $t + 1$ parties reached $R - 1$ then at most t honest parties did not reach $R - 1$. So there are at most t honest parties which only reached epoch $\leq R - 2$ before they timed out. This by definition means that $T \geq R - 2$.

Combining the last two facts above we get that if $Q \in \mathbb{Q}$ then

$$Q \geq R - 1 .$$

Combined with the first fact above, we get that:

$$\mathbb{Q} \subset \{R - 1, R\} .$$

This proves *Almost Agreement* as the set $\mathbb{Q}$ is the set of Q_i for which an honest party output $(\text{SHUTDOWN}, S)$.

Liveness Finally we prove liveness. If all honest parties get inout SHUTDOWN, then they all eventually delete *Echo First Signed Block* and stop sending new signatures $\text{Sig}_{\text{sk}_i}(\text{ECHO}, e, \cdot)$. Therefore there is a highest e which will ever get $t + 1$ signatures $\text{Sig}_{\text{sk}_i}(\text{ECHO}, e, \cdot)$ from honest parties. Therefore there is a highest e that will ever get $n - t$ signatures $\text{Sig}_{\text{sk}_i}(\text{ECHO}, e, \cdot)$. Therefore there is a highest e for which an honest party will ever have $\text{Unique}_{Q_i}^{P_i} \neq \perp$. So let H be the highest number such that $\text{Unique}_H^{P_i} \neq \perp$ for an honest P_i .

We now break the analysis into two. First we assume that no honest party terminates, and then we assume that at least one honest party terminates. These cases cover all possibilities and we show that in both of them there is liveness.

So assume no honest party terminates. If an honest P_j does not terminate then it will eventually receive the same signatures as P_i and set $\text{Unique}_H^{P_j} \neq \perp$. So since no honest party terminates all honest parties eventually have $\text{Unique}_H^{P_k} \neq \perp$. At this point it will never happen that any of them get $\text{Unique}_{H'}^{P_j} \neq \perp$ for $H' > H$ (by definition of H). This means that all honest parties eventually send $\text{Sig}_{\text{sk}_k}(\text{SHUTDOWN}, H)$. This implies that all honest parties eventually receive $n - t$ such values. At this point in time they all have $\text{Unique}_H^{P_j} \neq \perp$ and therefore will trigger *Shutdown* and output $(\text{SHUTDOWN}, H)$, which proves *Liveness* for this case.

Assume then that at least one honest P_i terminated. This means P_i triggered *Shutdown*. This means that $n - t$ values $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, Q_i)$ arrived for the same Q_i and $\text{Unique}_{Q_i}^{P_i} \neq \perp$. Since a party keeps running *Flood* after terminating, the signatures $\text{Sig}_{\text{sk}_j}(\text{SHUTDOWN}, Q_i)$ and the signatures that made P_i set $\text{Unique}_{Q_i}^{P_i} \neq \perp$ will arrive at all other parties. At this time they will trigger *Shutdown* if they did not already do this. Hence all parties eventually output some $(\text{SHUTDOWN}, \cdot)$.

14.10.4 Cleaning Up

We now look at the clean up procedure. It follows from *Agreement* above that the parties always will agree on the sequence $\text{Unique}_0^{P_i}, \dots, \text{Unique}_{\text{prefix}_i}^{P_i}$, though they may have different lengths. And, it follows from *Honest Liveness* that if the leader is honest then the sequence will keep growing. So our only problem is that if the leader is corrupted, then liveness might die. The sequence stops growing. We will now explore how to detect this and how fix it. The general idea is to figure out how fast the sequence should be growing if the leader is honest and then do a shutdown of the optimistic protocol if we do not see the proper growth. After that we then elect a new leader and start again.

The main complication will be that when the protocol shuts down, the honest parties might have different prefix_i , so it is not clear in which epoch we should restart. But by *Almost Agreement*, at least the values of prefix_i have a difference of at most 1. We just need to agree which of the two values to use. If we adopt the smallest value, then we agree to stop in epoch e , yet some honest party has $\text{prefix}_i = e + 1$, so the last block reached will be dropped. This is OK. This is why we only output blocks when at least one new block was seen (recall that $\text{final}_i = \text{prefix}_i - 1$). To determine which epoch to restart in, we will use Byzantine agreement.

Expected Progress

We now analyse how fast *FastTrack* should produce new blocks. We analyse the protocol under the following assumptions.

All Present: All honest parties are running.

Honest Leader: The party P_L is honest.

Synchronous Network: All messages on Flood are delivered within time Δ_{GUESS} .

Enough Workload: A new transaction arrives on TOB_L at least every Δ_{GUESS} seconds.

Lemma 14.8 *Under the above assumptions it holds for all honest P_i and all e that the time from $\text{Unique}_{e+1}^{P_i} \neq \top$ until $\text{Unique}_{e+1}^{P_i} = \top$ is at most $4\Delta_{\text{GUESS}}$.*

PROOF Under the above assumptions P_L sends out a new signed block at least every Δ_{GUESS} seconds. These message might arrive instantaneously or at at most Δ_{GUESS} seconds. This means that the time from $\text{Signed}_e^{P_i}(U) = \top$ for some U until $\text{Signed}_{e+1}^{P_i}(U') = \top$ for some U' is at most $2\Delta_{\text{GUESS}}$ seconds. By construction of the activation rule *Unique* it does not happen that $\text{Unique}_e^{P_i} \neq \perp$ until $\text{Signed}_e^{P_i}(U) = \top$. Therefore, the time from $\text{Unique}_e^{P_i} \neq \perp$ until $\text{Signed}_{e+1}^{P_i}(U') = \top$ for some U' is at most $2\Delta_{\text{GUESS}}$ seconds. By assumption on Flood propagating faster than Δ_{GUESS} it will take at most Δ_{GUESS} seconds from $\text{Signed}_{e+1}^{P_i}(U') = \top$ for some honest P_i until $\text{Signed}_{e+1}^{P_j}(U') = \top$ for all honest P_j . Put the above together we see that from $\text{Unique}_e^{P_i} \neq \perp$ for the first honest P_i until $\text{Signed}_{e+1}^{P_j}(U') = \top$ for some U' for all honest P_j is at most $3\Delta_{\text{GUESS}}$. Then time from $\text{Signed}_{e+1}^{P_j}(U') = \top$ for all honest P_j until all honest parties received $n - t$ values $\text{Sig}_{\text{sk}_j}(\text{ECHO}, e + 1, U')$ is at most an additional Δ_{GUESS} . At this time $\text{Unique}_{e+1}^{P_i} \neq \top$ for all honest parties. $\square$

We now prove an invariant when *Synchronised Start* is executed. It is broken into safety and liveness conditions. The liveness ones are only required to hold if the network delay is less than Δ_{GUESS} and P_L is honest. The other conditions are safety conditions and will always hold.

Synchrony (L): All honest parties call *Synchronised Start* within a time difference of Δ_{GUESS} .

Agreement (L): For an honest party P_i let O_i be its output from fast track. Then $O_i = O_j$.
This follows from $\text{final}_i = \text{final}_j$ and agreement of

At the point in time where an honest party terminates *Determine Cut-Off*

In the base case we assume that the parties call *Initialize* within Δ_{NET} and liveness should only hold when $\Delta_{\text{NET}} \leq \Delta_{\text{GUESS}}$. In the restart case we by assumption that the parties terminate the ABAs within time Δ_{NET} and again we can assume that $\Delta_{\text{NET}} \leq \Delta_{\text{GUESS}}$.

Let $\mathbb{Q}$ be the set of epochs e for which some honest party output $(\text{SHUTDOWN}, Q_i)$.

Lemma 14.9 *The parties agree on epoch and epoch $\in \mathbb{Q}$ and epoch + 1*

Let $S = \max(\mathbb{Q})$ such that $Q \in \mathbb{Q}$ implies that $Q = S$ or $Q = S - 1$.

If $Q_i = S$ then P_i inputs 1 to exactly $\text{BA}_{S \bmod 4}$ and $\text{BA}_{S-1 \bmod 4}$. If $Q_i = S - 1$ then P_i inputs 1 to exactly $\text{BA}_{S-1 \bmod 4}$ and $\text{BA}_{S-2 \bmod 4}$. This means all honest parties input 1 to $\text{BA}_{S-1 \bmod 4}$ and no honest party input 1 to $\text{BA}_{S+1 \bmod 4} = \text{BA}_{S-3 \bmod 4}$. From validity we get

Fact 14.10 $d_{S-1 \bmod 4} = 1$ and $d_{S+1 \bmod 4} = 0$.

Lemma 14.11 *For all x and all honest P_i, P_j it holds that $D_x^{P_i} = D_x^{P_j}$.*

Initialize Let the initial leader be $L = 1$. Start the flooding subsystem which updates the set Received_i . Let $\text{Queued}_i = \emptyset$. Let $\text{epoch} = 1$. Trigger *Synchronised Start*.

Synchronised Start This activation rule is triggered only from other activation rules. The purpose of the rule is to ensure that if all parties trigger the rule within time Δ_{GUESS} , then the leader will not start running FastTrack until all honest parties started running fast track. It goes as follows. $P_i \neq P_L$: Input $(\text{epoch}, \text{Queued}_i, L)$ to FastTrack. P_L : Wait Δ_{GUESS} seconds and then input $(\text{epoch}, \text{Queued}_L, L)$ to FastTrack.

Monitor Liveness If it takes more that $4\Delta_{\text{GUESS}}$ seconds between $\text{Unique}_e^{P_i} \neq \top$ and $\text{Unique}_{e+1}^{P_i} \neq \top$, then input SHUTDOWN on TOB_i and wait for output $(\text{SHUTDOWN}, Q_i)$

Vote Set $s_i = Q_i \bmod 4$ and let $p_i = Q_i - 1 \bmod 4$. Run 4 ABAs $\text{ABA}_0, \text{ABA}_1, \text{ABA}_2, \text{ABA}_3$.^a Vote 1 in ABA_{s_i} and ABA_{p_i} . Vote 0 in the other two.

Determine Cut-Off When all ABA_i outputted some decision d_i , then Set $D_{Q_i-2} = d_{Q_i-2 \bmod 4}$, $D_{Q_i-1} = d_{Q_i-1 \bmod 4}$, $D_{Q_i} = d_{Q_i \bmod 4}$, $D_{Q_i+1} = d_{Q_i+1 \bmod 4}$. Set $D_x = 0$ for all other x .

Let e be the largest value such that $D_e = 1$. Wait until $\text{final}_i = e$.

Let $\text{epoch} = 0$.

Update $\Delta_{\text{GUESS}} = 2\Delta_{\text{GUESS}}$.

Update the leader to be $L = L + 1$. If $L = n + 1$, then let $L = 1$.

Trigger *Synchronised Start*.

^aThese ABAs should have the property that all honest parties terminate within the network delay of each other. This is commonly the case for asynchronous protocols, as there are no timeout. One can use a time capsule like message to ensure this if it is not already the case.

Figure 14.20: Eventually Synchronous total-order Broadcast (III)

PROOF We know that $Q_i = S$ or $Q_i = S - 1$. If $Q_i = S$ then:

$$\begin{aligned}
 D_{S-3}^{P_i} &= D_{Q_i-3 \bmod 4} = 0 \\
 D_{S-2}^{P_i} &= D_{Q_i-2 \bmod 4} = d_{Q_i-2 \bmod 4} = d_{S-2 \bmod 4} \\
 D_{S-1}^{P_i} &= D_{Q_i-1 \bmod 4} = d_{Q_i-1 \bmod 4} = d_{S-1 \bmod 4} = 1 \\
 D_S^{P_i} &= D_{Q_i \bmod 4} = d_{Q_i \bmod 4} = d_{S \bmod 4} \\
 D_{S+1}^{P_i} &= D_{Q_i+1 \bmod 4} = d_{Q_i+1 \bmod 4} = d_{S+1 \bmod 5} = 0
 \end{aligned}$$

and all other values are 0. If $Q_i = S - 1$ then:

$$\begin{aligned}
D_{S-3}^{P_i} &= D_{Q_i+1-3 \bmod 4} = d_{Q_i-2 \bmod 4} = d_{S-3 \bmod 4} = d_{S+1 \bmod 4} = 0 \\
D_{S-2}^{P_i} &= D_{Q_i+1-2 \bmod 4} = d_{Q_i-1 \bmod 4} = d_{S-2 \bmod 4} \\
D_{S-1}^{P_i} &= D_{Q_i+1-1 \bmod 4} = d_{Q_i \bmod 4} = d_{S-1 \bmod 4} = 1 \\
D_S^{P_i} &= D_{Q_i+1 \bmod 4} = d_{Q_i+1 \bmod 4} = d_{S \bmod 4} \\
DP_{iS+1} &= D_{Q_i+1+1 \bmod 4} = d_{Q_i+2 \bmod 4} = 0
\end{aligned}$$

and all other values are 0. □

Now, since the parties agree on all D_x values, they will also agree on the largest value e such that $D_S = 1$. Furthermore, we know that $D_{S-1} = 1$ and $D_f = 0$ for $f > S$, so $e = S - 1$ or $e = S$, which implies that $e \in \mathbb{Q}$.

We prove that when the protocol terminated all honest parties output the same sequence of S values without any DROP-messages. In addition it might be that some parties output (DANGLING, $S + 1, \cdot$) followed by (DROP, $S + 1$).

Assume that S be such that $D_S = 1$ and $D_{S+1} = 0$. Then some honest party input 1 to $ABA_{S \bmod 4}$ so (DANGLING, $S, \cdot$) will reach all parties. If a party also gets (DANGLING, $S + 1, \cdot$) then it will be dropped. We therefore just have to argue that no honest parties output (DANGLING, $S + 2, \cdot$) before the ABAs terminated. But if an honest party outputs (DANGLING, $S + 2, \cdot$) then $R = S + 2$ and therefore $\mathbb{Q} \subset \{S + 1, S + 2\}$, which implies that $D_S = 0$, a contradiction.

14.10.5 Repeat

When the above protocol terminates then the parties agree in which epoch it happened. Then they can try to adjust parameters to ensure pre-agreement in the next run. When they are ready to run again they will synchronize to within network delay as follows.

1. Send $\text{Sig}_{\text{sk}_i}(\text{READY}, S)$ to all parties.
2. On $n - t$ signatures $\text{Sig}_{\text{sk}_j}(\text{READY}, S)$ call them a proof of ready. Send it to all parties.
3. On a proof of ready send it to all parties and start running $\pi(S)$.

14.10.6 Which ABA to use

As for the ABA to use, a good choice is the one in Chapter 10. Recall that there was the need for a good coin-flip which was random and on which the honest parties often agree. In the partially-synchronous model it can be implemented as follows.

The input is a coin-flip identifier cfi which is unique and agreed upon by the parties. This is just the “name” of the coin-flip, like “coin-flip number 7 in the 11'th run of SVUP in the broadcast with broadcast identifier bid .” The protocol also uses a guess Δ_{GUESS} at the network delay. Each party P_i has a secret key sk_i for a signature scheme and all other parties know the public key so that they can verify signatures from P_i . The signature scheme should have unique signatures, i.e., for a fixed m the value $\text{Sig}_{\text{sk}_i}(m)$ is fixed. The protocol also uses a hash function H modelled as a random oracle.

The protocol runs as follows.

1. When told to run the coin-flip, send FLIPCOIN to all parties if this was not already done.

2. At any time, if seeing $t + 1$ values FLIPCOIN, send FLIPCOIN to all parties.
3. At any time, if seeing $n - t$ values FLIPCOIN, send $\text{Sig}_{\text{sk}_i}(\text{FLIPCOIN}, \text{cfi})$ to all parties. Then wait for $2\Delta_{\text{GUESS}}$ seconds and go to the next step.
4. Collect all values $\text{Sig}_{\text{sk}_j}(\text{FLIPCOIN}, \text{cfi})$ which arrived already and wait to receive at least $n - t$ such values. Then let the winner be the P_j with maximal $H(\text{LOTTERY}, \text{Sig}_{\text{sk}_j}(\text{FLIPCOIN}, \text{cfi}))$. Take the coin to be the parity of this $H(\text{FLIPCOIN}, \text{Sig}_{\text{sk}_j}(\text{FLIPCOIN}, \text{cfi}))$.

The protocol has the following properties. With probability $2/3$ the maximal $H(\text{Sig}_{\text{sk}_j}(\text{FLIPCOIN}, \text{cfi}))$ will belong to an honest P_j . This is because the signature are unique, so there are at most n different values $(\text{Sig}_{\text{sk}_j}(\text{FLIPCOIN}, \text{cfi}))$, and $n - t > 2t + 1$ of these values are honest and at most t are corrupt. Furthermore, when the timeout is long enough and the winner is honest, then all honest agree on the winner. To see this note that if some honest see $n - t$ values FLIPCOIN then all honest will see $n - t$ values FLIPCOIN within time Δ_{GUESS} . Therefore all honest will send $\text{Sig}_{\text{sk}_i}(\text{FLIPCOIN}, \text{cfi})$ within Δ_{GUESS} off each other. So the first honest to send $\text{Sig}_{\text{sk}_i}(\text{FLIPCOIN}, \text{cfi})$ will see all honest $\text{Sig}_{\text{sk}_j}(\text{FLIPCOIN}, \text{cfi})$ within $2\Delta_{\text{GUESS}}$ seconds of sending its own. Since the honest parties wait $2\Delta_{\text{GUESS}}$ before taking maximum it follows that all honest parties have their $H(\text{FLIPCOIN}, \text{Sig}_{\text{sk}_j}(\text{FLIPCOIN}, \text{cfi}))$ be part of the maximum, so when the maximal winner is honest all honest parties agree on the maximum.

Furthermore, when the winner is honest then the coin-flip is unpredictable for the adversary before the first honest party is told to start the coin-flip. This is because when P_i is honest then the value $\text{Sig}_{\text{sk}_i}(\text{FLIPCOIN}, \text{cfi})$ until P_i sends it, or the signature could be forged by the adversary. But then the value $H(\text{FLIPCOIN}, \text{Sig}_{\text{sk}_i}(\text{FLIPCOIN}, \text{cfi}))$ is unpredictable, as you cannot predict the outcome of a random oracle on an input you do not know. So all we have to show that no honest party sends $\text{Sig}_{\text{sk}_i}(\text{FLIPCOIN}, \text{cfi})$ until the first honest party was told to start the protocol. This is easy to see. An honest party only sends its signature after seeing $n - t$ values FLIPCOIN which clearly implies that an honest party must have been told to start the protocol: the corrupted parties together can only create t FLIPCOIN messages.

Chapter 15

Crypto Currencies* (2025)

Contents

15.1 Crypto Currencies*	407
15.2 Smart Contracts	408
15.3 Oracles	409
15.4 Energy	409
15.5 Building Trust from Distrust	410

In this chapter we cover cryptocurrencies with smart contracts.

15.1 Crypto Currencies*

A currently very hyped use of state machine replication is known as **cryptocurrencies**. This basically just means that we replicate a machine which keeps track of the holding of some accounts which hold some money, e.g., Bitcoins. Or rather, the accounts just hold some integers, like 42, and it is up to the users of the system to interpret them as money.

The idea of cryptocurrency started with the Bitcoin project. The idea of the Bitcoin project was to do keep a replicated state machine keeping track of an account system in a world-wide peer-to-peer network, where everybody can run a replica. That way one would have a peer-to-peer currency without a single point of attack and which was not controlled by any state. It is imagined that eventually the system will be replicated over maybe millions of machines across the planet. If you solved Exercise ??, then you already saw a toy example of doing this in a peer-to-peer network.

Anyone can potentially send a command to move money between accounts. One way to prevent stealing of money is to let accounts be named by a verification key vk . The account owner has the corresponding signing key sk . To spend from the account, the transaction needs to be signed by the corresponding sk . This is to ensure that only the account holder can spend the holdings on the account. If you solved Exercise ??, then you already saw an example of this.

The state of a cryptocurrency is then a map **Account** which maps verification keys vk to their amounts. We write **Account**[vk] for the amount on account vk . One would typically have one or more special genesis accounts which hold all the initial coins. In addition new coins can be minted during operation of the system, for instance by giving them to the server producing the

next block in the underlying total-ordered broadcast.¹ Sometimes there is a maximal number of coins which will ever be created. In Bitcoin this number is 21 million. The thought is that creating a limited resource can have market mechanisms drive up the price of the coins. In yet other systems there is no limit, and more coins are created all the time. The thought typically is that slight inflation is good for monetary systems. These are considerations of economy and not distributed systems, so we do not cover them further here.

It is important to use total-ordered broadcast to broadcast transactions, to be able to agree on the order in which transactions are made: if there is an attempt to transfer more money from an account than it holds, the latest withdrawal has to be cancelled. To be able to do that, all parties need to agree which transaction was the last one. If some replicas cancel one transaction and other replicas cancel the other transaction, they will no longer hold identical copies of **Account**.

A cryptocurrency as described above is a layer which can in principle be put on top of any total-ordered broadcast. How the standings on the accounts attain value is an economic problem outside the scope of this book. One way could be that there is a law which guarantees that the numbers can be exchanged for real money, as with your bank account. In the case of Bitcoin, the value simply emerged. People started to exchange Bitcoins for real money and real world objects like pizzas, and once this happened an expectation arose that the holders of Bitcoins could sell them for money in the future or use them to buy things, and that the price therefore might go up. Then they became a speculation object and the price sky rocketed. On May 22, 2010 the first documented purchase using bitcoins was made. A pizza was bought for 10,000 Bitcoins. At the market price as of writing this book, those 10,000 Bitcoins are worth 345,720,000 USD. One can only hope it was a good pizza.

15.2 Smart Contracts

Some cryptocurrencies also have a notion of **smart contracts**. Treating the topic of smart contracts fairly belongs to a programming language course and is outside the scope of this book.

A smart contract is simply a (small) state machine M that can be “uploaded” to the replicated machine. Programs have to be associated to an account and can send money from the account, so to prevent abuse an upload would have to be signed by the secret key of the account it is uploaded to.

Technically, uploading a program would simply mean that you broadcast something like $(\text{CREATE}, M, vk, \sigma)$ on the TOB. If the signature is valid, the contract is associated to account vk . The smart contract will now be run on the sequence of messages on the TOB and can be programmed to change state in reaction to some of them. It might in addition to changing state also send messages to other smart contracts (these messages can be thought of as being added to the TOB from where other smart contracts can see them). Finally a smart contract can choose to send money from its owner’s account to other smart contracts or accounts.

Note that the replicated state machine being run by a cryptocurrency with smart contracts is the totality of all the accounts and smart contracts. So if there is a million users each with an account and smart contract, then each of the servers running the system will constantly be running all these smart contracts. So in a planet-wide peer-to-peer system with a millions servers, we would have a million servers all running a million small programs.² On one hand,

¹In the language of cryptocurrencies, the total-ordered broadcast is often called a blockchain, as it produces a chain of blocks (of transactions). We return to this in more detail in the next chapter.

²In very large systems with millions of servers, an optimization called sharding can be applied where each contract is only run by a small subset of the servers. The subset is picked large enough to have good replication and small enough to have efficiency. Typically the subset is picked at random to make it harder for an attacker

this is an enormous computational overhead. On the other hand it gives high security. No single server, or maybe not even up to one third of the servers in malicious collusion, can steal the money on an account or make a smart contract compute something wrong.

Because a smart contract is replicated across many servers, once a smart contract is uploaded it cannot be deleted again. So if it was programmed to send money to some other account in face of some event, then it will do so. There is no stopping it. This is what makes it a “contract”, it binds the owner of the account to pay money in response to certain events.

15.3 Oracles

Some cryptocurrencies also have a notion of oracles. These are just special smart contracts which report some values from the real world, like the stock prices on some stock exchange. Oracles is a way to allow the smart contract to “see” values in the real world. To avoid that these oracles can lie, one can have for instance 13 companies or institutions run each their sub-oracle. They all report the same value, and then the users of the TOB take the true value to be the one reported by most sub-oracles. This would give security against up to 6 malicious and colluding organisations. Once smart contracts can see values in the real world, they can be programmed to exchange money in response to real world events. You might for instance pay some other smart contract money if the EUR to USD price reaches some particular value on a particular day. You might also pay some person money if a flight or train is delayed by some amount of time.

15.4 Energy

A common pattern in smart contracts is to have the caller pay for having the called contract run. Among other things, this helps to prevent misuse of the system where parties just run a lot of calls to smart contracts to overload the system. One can think of this mechanism as paying the energy that goes into running the contract. And sometimes the unit in which one pays is called energy, and sometimes gas. We will sketch a very simplified approach here, where a call to the Replicated Smart Contracts system is of the form (vk, x, E) , where vk is the name of the contract, x is the input to the smart contract, and E is the amount of energy sent along with the call. Here E is a non-negative integer. We will take a very simplistic view of this and have each step in the computation just pay 1 unit of energy. Therefore, for each step in the computation we decrease E by one. If E hits 0, then the execution stops.

The contract making a call like (vk, x, E) has to pay for the amount of energy from the account making the call. Therefore the call (vk, x, E) has to be signed by the calling account, or be generated by the smart contract associated to the calling account. The calling account of course also has to have enough money to pay E .

When a unit of energy is consumed it can be thought of as being added to a special system account from where it can be used to pay for instance the servers running the total ordered broadcast used to implement the cryptocurrency. For instance it could be paid to the server who made the block in which the command was added.

Often there is some exchange rate between the cryptocurrency and the energy unit. This exchange rate might vary to keep the value of the energy pecked for instance to the electricity price, or some other resource which is central for running the servers implementing the underlying total ordered broadcast.

to have only its own machines running some target contract they it wants to attack. We will not dive further into sharding for now.

15.5 Building Trust from Distrust

The ability to make contracts is a huge driver of modern society. There is hardly an economic transaction going on without an underlying contract of sorts. For contracts to work it is important that the parties making the contract can be sure it is enforced. Traditionally this enforcement has been accomplished by judicial systems. If your peer in the contract does not deliver the promised service you can take them to court. One disadvantage of this approach is that it is expensive to take someone to court. This makes it hard to use contracts to protect very small and one-time interactions. Another disadvantage is that the peers in a contract might not trust the same judicial system, so contracts involving peers from different countries can be harder to make and enforce. Finally, making a good contract often involves a lawyer or notary, which are costly resources.

Smart contracts on blockchains are fundamentally different from the old type of contract. The trust that the contract will be enforced is not rested in one judicial system. Instead it comes from the belief that having the contract replicated across many servers will ensure that at least a majority will execute the code correctly and thereby automatically enforce the contract. In some sense you trust that the servers running the system distrust each other enough that they will not all collude against you. Another difference is that you do not have to take someone to court to enforce your contract. Instead the contract is a piece of code which will enforce itself. When the conditions are fulfilled it executes and moves the money needed. Finally, making a smart contract is done simply by uploading a piece of code.

How to design the incentive systems of the blockchains, the programming languages for programming smart contracts, semantics of executing smart contracts, *et cetera*, is a rapidly evolving research area these years involving aspects of programming languages, game theory, distributed systems, and cryptography.

The reason for the surging interest in cryptocurrencies is that a vision is touted that this could create a future where macroscopic and microscopic aspects of society are driven by smart contracts, where establishing trust between two peers who never met is as easy as sending a packet on the internet today. Making contracts could be turned into a sub-branch of programming, and one could imagine a large open-source library of standard contracts which could be reused and combined. There are for sure many caveats to this story. However, having trust built by distribution and not centralisation is certainly appealing from the distributed systems point of view.

This is a book on distributed systems, so we will ignore most interesting aspects of cryptocurrencies. However, in the next chapter we will dive into the details of how to implement a total-ordered broadcast channel in a world-wide peer-to-peer system.

Chapter 16

Nakamoto-Style Consensus* (2025)

Contents

16.1 Blockchains: Nakamoto Style Consensus	412
16.1.1 Blockchains versus Cryptocurrencies	414
16.2 Distributed Lotteries: Proof-of-Work versus Proof-of-Stake	415
16.2.1 Proof-of-Work	415
16.2.2 Proof-of-Stake	416
16.2.3 Proof-of-Resource	416
16.3 A Note on Digital Identity: Parties versus Public Keys	417
16.4 Synchronous Clocks	417
16.5 Flooding System	417
16.6 Lottery System	418
16.7 Growing a Tree	420
16.7.1 The Main Idea in a Nutshell	421
16.7.2 How does the Tree Grow	422
16.7.3 Rollback and Finalisation	424
16.7.4 Ghost Growth	424
16.8 Understanding Tree Growth	425
16.8.1 The Standard Tree Scenario	427
16.8.2 Tree Growth	427
16.8.3 Chain Quality	429
16.8.4 Limited Rollback	429
16.9 Making it a Total-Order Broadcast (TODO)	434
16.10 Exercises	434

Attack Model *In this chapter we assume that*

1. There are n parties, each having some amount of resource like computing power or money.
2. Some parties might be Byzantine corrupted. We will assume that the corrupted parties hold some limited fraction of the overall amount of the resource, e.g., 49% or 33%.
3. The adversary cannot drop messages.
4. The adversary can inject messages, but:

5. *The parties can use signatures to authenticate messages.*
6. *Protocols are in a variant of the eventually synchronous round-based model. Specifically:*
 - (a) *There is a bound on network delay and eventually all messages are delivered within the bound often enough. Details below.*
 - (b) *The parties have access to synchronised clocks.*

16.1 Blockchains: Nakamoto Style Consensus

In this section we give another synchronous implementation of **total-order broadcast**. It will be using lottery-based blockchains. It will be using time in a much less critical way than the round-based Dolev-Strong protocol and will therefore be much more efficient. In particular, we will only assume that we can set an upper bound on message delivery that holds most of the time. Hence we design for the case where the messages sometimes are slower than expected. The protocol is therefore an example of a partially synchronous protocol. The design we present in this chapter is sometime called *Nakamoto Style Consensus*. This style of total-order broadcast was first proposed for the cryptocurrency Bitcoin attributed by the *nom de plume* Satoshi Nakamoto. NSC TOBs were designed to run on top of peer-to-peer networks where there are many participants which can come and go. It might also be that no party at any point in time has an overview of who is participating, and at least there might not be agreement on who is currently participating due to the dynamic participation. The main design criterions for NSC TOBs are as follows:

Autonomy: The system should not be controllable by a single person or organisation. We say that the system should not have a **single point of attack**. An autonomous system of course cannot run on a single physical machine. Otherwise, the owner of the machine could take over the system, or just cut the power cord. Neither can an autonomous system run inside a single organisation. Nor should the system run solely inside a single country, which might shut it down using legislation. To ensure that no entity could take over the system, it is necessary to set up a system where anyone can join and help run the system and where there is a large number of participants from all over the planet. NSC is a design optimised for making it easy for servers to join the system. In particular, you can join the system without having a permission from anyone. Such TOBs are also called **unpermissioned**.

Self-Incentivization: The TOB is being used to implement a cryptocurrency. This cryptocurrency is in turn being used to pay the servers that is running the TOB. Notice that this gives a self contained system, where 1) the cryptocurrency is secure (and therefore valuable) because a lot of servers help running it, and 2) a lot of servers help run the system because the cryptocurrency is valuable. After all, they would not be interested in being paid in the cryptocurrency if it was not valuable. This is circular and therefore hard to establish in the beginning. But once it has been established, it helps autonomy, as there is no way to cut the funding used to pay the servers. Contrast this to a situation where all servers were being paid by a single organisation. In that situation this organization would be a single point of attack.

Peer-to-Peer Friendly: To implement an unpermissioned TOB it is an obvious choice to use a peer-to-peer network where servers can join the network at their own choice. Such networks were after all designed exactly to be autonomous and unpermissioned. This means that in the implementation of the TOB all communication is done by flooding

messages. One cannot send a message to some designated server. Typically in the peer-to-peer network you only know the IP address of your neighbours. To some extent this is a feature as it makes it harder to do a so-called denial-of-service attack on the TOB.

Denial-of-Service (DoS) Attack Resistant: Peer-to-peer networks are very open networks. To allow anyone to join, the existing peers cannot hide behind a firewall or edge servers. And since anyone can join it is also hard to control what configurations other peers are running, so one cannot enforce that peers use a firewall. Therefore peers in a peer-to-peer network will tend to be vulnerable to denial-of-service attacks where for example an adversary will flood them with useless messages. For this reason NSC protocols do not necessarily make public the IP address of any peer. The design only assumes that messages can be flooded in a way that all other peers eventually receive them. This allows a machine which was attacked to disconnect and reconnect to the peer-to-peer network using a new IP and still be part of the protocol.

Nakamoto Style Consensus has three main components.

Distributed Lottery: A crucial part is a distributed lottery or distributed leader election. Recall that in the synchronous TOB we saw in Chapter 14 there was a leader in each round sending out a block. In Chapter 14 the leader was elected round-robin. This unfortunately does not work in a peer-to-peer setting as there is no agreement across the whole system which parties are part of running the system at any point in time. Without agreement on who is part of the system, round robin would lead to different parties electing different leaders. It is not feasible to first reach agreement on which parties are in the system and then elect a leader after that. Reaching agreement on which parties are in the system is an agreement problem in itself, and as we are implementing a TOB, the problem we are try to solve is also an agreement problem. The idea of NSC is to say that we should attack the problem at hand directly: we need to elect a new leader without knowing who is on election! Solving this problem is the claim-to-fame of Bitcoin. We will see below how it is done. At a feature level the lottery has the following properties:

Local Computation Only: To participate in the lottery all you need is to know the current state of the TOB. The rest of the computations are local. You do not have to register to participate in the lottery. In fact, you do not need to communicate with other parties to participate in the lottery. This allows anyone to participate without anyone having an overview of who is taking part. As a consequence of this, the lottery will sometime have no winners and sometimes several winners, exactly as real lotteries. This is inevitable. Imagine a distributed lottery which could ensure there is precisely one winner. Maybe it is party P_7 . But recall that we consider a setting where parties can come and go. So it might be that party P_7 is not right now taking part in running the TOB. In that case there would be 0 winners. And the other parties cannot know if P_7 is right now taking part as parties do not register for the lottery, they only perform local computations. Hence distributed lotteries with Local Computation Only inevitably must have an element of randomness to them, and then sometimes there are several winners and sometimes none. This will give us a lot of ~~problems~~ fun later!

Non-predictability: No one can predict who will win the lottery. This is important for DoS resilience. If the corrupted parties could predict who would win the lottery, they could DoS attack the leader before it sends its block.

Public Verifiability: Once you won the lottery, you can prove this to the rest of the peers. This is important, as the other peers need to know who the leader is.

One Message: To prove that you are the winner you have to send only a single message. Along with this message you can send the next block. It should be possible to associate the block with the proof that you won the lottery such that the adversary cannot misuse your proof that you won to send a different block. This is typically done by your verification key being integrated into the proof in a non-tamperable way. This allows the winner in a given round to ever only send a single message. This is important for DoS resilience: once a leader sent its first message the adversary might learn its IP address and launch a DoS attack against the leader. If the leader is never to send a second message this does not matter. The leader just has to survive long enough to send a single message.

Sybil Attack Resistant: It should not be possible to enter the lottery for free. If this was possible an attacker wanting to attack a system with 1,000 honest servers could just add 11,000 corrupted serves to the lottery. Then it wins more than 90% of the time and can easily attack the system, The overall design requires that the honest parties win at least 51% of the time.

Tree of Blocks: Once a leader is chosen it sends out a new block. When it does this, it includes the proofs that it won the lottery and a pointer to what it considers the previous block. This pointer is just the hash of the previous block. The used hash function is collision resistant, so one can use the hash to uniquely name the block. Since blocks point to previous blocks, they form a chain. This is the basis for the name **blockchain**. The sequence of the transactions is the order in which they appear in the chain of blocks.

However, since there are sometimes several winners it can happen that two new blocks B and C are made at the same time and both point to the same previous block A . Now we are no longer building a chain, but a tree. This is the source of a lot of the fun we will have later.

Longest-Chain Rule: Since the basic protocol is building a tree and not a chain it is not clear what *the* blockchain is anymore, is it the one ending in B or the one ending in C ? Therefore it is also not clear what is the sequence of the TOB. Such branching could also happen because a winner is malicious and extends on a previous block and not the most recent one. Since the network might not agree on what is the most recent block, we cannot force parties to extend on the most recent block. To fix accidental and malicious branching, there is a rule that when you make a new block you should build on the longest chain you know. If there are two of the same length, apply some deterministic tie breaking rule. If 51% of the winners follow this rule of extending the longest chain, then a very long chain will tend to grow, with just a few short side branches. It is this longest chain which is the truth. All the transactions in the side branches are ignored. If less than 50% of the nodes follow the rule, chaos can emerge as two equally long chains keep growing. This is why we need the lottery to be Sybil attack resistant.

16.1.1 Blockchains versus Cryptocurrencies

A NSC TOB is often called a **blockchain** because of the implementation detail of using a chain of blocks. Blockchains are currently a popular way to implement total-order broadcast. Cryptocurrencies can be added on top of any total-order broadcast. However, the first popular cryptocurrency was Bitcoin and it was implemented on top of a NSC total-order broadcast, so the terms blockchain and cryptocurrency have become somewhat married in popular writing. They are, however, very different creatures. A blockchain is an implementation of total-order

broadcast and can be used for anything where a total-order broadcast is useful. A cryptocurrency is a layer that can be put on top of any total-order broadcast, not just the NSC ones. It is just a replicated account system, possibly with associated smart contracts.

16.2 Distributed Lotteries: Proof-of-Work versus Proof-of-Stake

We first informally discuss several types of distributed lotteries.

16.2.1 Proof-of-Work

The idea behind Bitcoin's lottery was fairly simple and we sketch it here. Let A be the latest block. Let H be a hash function. Let $a = H(A)$. We use a as a short name for the block we want to extend. We call a a pointer to A . To win the right to extend the chain at A you will compute $y_c = H(a, \text{vk}, c)$ for your verification key vk and where c is a counter. Let us assume it is $c = 1$ for now. Using SHA256 this will give a 256-bit output y_1 . It is believed that the outputs of good cryptographic hash functions behave like random values for many practical purposes. We can therefore use y_1 as some random value determining whether vk won the lottery. The way Bitcoin does it is via a hardness level h . This is a positive integer. You win if the first h bits of y_1 are all 0. If y_c is random, the probability of this happening is 2^{-h} . So for hardness $h = 10$ you win with probability $2^{-10} = 1/1024$. Now, you cannot just let each account vk get one try y_1 to win. This would not be Sybil attack secure. The corrupted parties could just make a lot of accounts and then get an unfair advantage in the lottery. This would not be secure. We need that the honest parties win most of the time.

Instead the hardness is set much higher. If you set $h = 30$ then each vk wins with probability about 10^{-9} . And then we allow peers to try as many times as they want. You are allowed to compute $y_1 = H(a, \text{vk}, 1)$, $y_2 = H(a, \text{vk}, 2)$, $y_3 = H(a, \text{vk}, 3)$, and so on. If any of these y_c have h initial 0s, then vk won the lottery. Any vk to find a winning y_c is a winner of the lottery.

If you have a computer which can compute C of the values y_c per second, then within one second you win with probability about $C2^{-h}$.¹ This prevents Sybil attacks. To see this consider the following example. Say we have an attacker which can do C computations. Say instead of having a single account it makes two accounts. If it uses C_1 computations to try to win with vk_1 and C_2 computations to win with account vk_2 it still needs to have $C_1 + C_2 \leq C$ as it can only do C computations in total. But then it might as well just use all C on the same account. So in Bitcoin Sybil attacks are prevented using a rule we might call *one hash-computation = one ticket*. To buy a ticket in the lottery you have to compute one y_c . Then your number of tickets is limited by your computation power.

For the Bitcoin lottery to be sure we need that the nodes which will follow the Longest-Chain Rule has 51% of the computing power.

Environmental Considerations

There is an unfortunate design choice in Bitcoin. To reward peers for taking part in the lottery and sending blocks, Bitcoin rewards the winner of the lottery with some Bitcoins. Bitcoins are currently worth a lot of money. Therefore it is worth a lot of money to win the lottery. This means that it pays off to do a lot of computations of y_c -values to win the lottery. Basically, if you burn less energy on computations than you get back when you win the lottery, you still make a profit. So node operators are buying a lot of machines to just do hash computations.

¹Actually it is $1 - (1 - 2^{-h})^C$, but $1 - (1 - 2^{-h})^C$ and $C2^{-h}$ are very close to each other for relatively small C and relatively large h .

As of writing the Bitcoin network does 10^{20} hashes per second. This burns a lot of energy. Since all this burned energy is burned just to determine who won a lottery, it is a very silly and wasteful design.

16.2.2 Proof-of-Stake

Another, much more energy efficient design for distributed lotteries is called **proof-of-stake**. There are a couple of variants of these lotteries. Here we sketch the one we use in this book. In a PoS lottery each account vk has an amount of stake a , this could for instance just be the number of coins in the cryptocurrency that are on the account vk . Now a lottery will be run for each so-called slot $slot = 1, 2, \dots$. These slots are spaced out in time with a regular pattern, for instance one each second. All parties agree when lottery number $slot$ will be run.

To check if account vk won the lottery it computes a draw

$$\text{Draw} = \text{Sig}_{sk}(\text{LOTTERY}, \text{slot}) ,$$

where LOTTERY is just the text string “lottery”. Then a value of the draw is computed as $\text{Val} = a \cdot H(\text{Draw})$. Here $H(\text{Draw})$ is a hash, which we think of as a random value. Finally there is a hardness **Hardness**. If $\text{Val} \geq \text{Hardness}$ then you won the lottery. Notice that the value is scaled by the amount a . So an account with more money on it is more likely to win the lottery.

To prove that you won the lottery you send $\text{Draw} = \text{Sig}_{sk}(\text{LOTTERY}, \text{slot})$. The receiver checks the signature with vk and can check that $a \cdot H(\text{Draw}) \geq \text{Hardness}$. This proves that your verification key vk won. To send along the new block B you send along $\text{Sig}_{sk}(B)$. This binds the block to the draw.

The reason why we scale the value $\text{Val} = a \cdot H(\text{Draw})$ with the amount of cryptocurrency a that the account has is to protect against Sybil attacks. An attacker could try to open a lot of accounts to become more likely to win the lottery. When the winning probability is scaled by the amount of cryptocurrency on the account, this attack does not work unless it puts some cryptocurrency on the accounts. But this cryptocurrency will then be taken from some other account of the attacker. And, the probability of a given account with stake a winning the lottery can be set such that the total probability of winning does not depend on which accounts your cryptocurrency are stored, only how much you have in total. This mitigates Sybil attacks.

In this case the assumption that most of the winners are honest translates to the assumption that the owners of most of the cryptocurrency in the system are honest. This makes sense in that it is the total-order broadcast that protects the integrity of the system. Breaking the total-order broadcast would break the cryptocurrency, which would have the cryptocurrency held in the system lose most of its value. Hence the holders of the cryptocurrency in the system have an interest in keeping the system healthy.

Environmental Considerations

Notice the if you have 10,000 peers in your peer to peer network, then a proof-of-stake lottery would do 10,000 hashes per second. That is 10^4 hashes. Bitcoin does 10^{20} . Therefore PoS is about $10^{16} = 10,000,000,000,000,000$ times more energy efficient than PoW. It also means that the notion that blockchains are energy inefficient is largely a notion of the past. You should just use PoS instead of PoW.

16.2.3 Proof-of-Resource

Distributed lotteries in general have to depend on some resource which is costly and which cannot be copied. The fact that the resource cannot be copied protects against Sybil attacks.

Then you make the winning probability proportional to the amount of the resource a peer has, and you have to assume most of the resource is held by good nodes. For proof-of-work the resource is computation power. For Proof-of-stake it is cryptocurrency. There are also other types of lotteries using resources like memory and disk storage. One can in principle try to base the lotteries on any finite resource. Proof-of-resource cannot be based on human stupidity as this resource is infinite.

16.3 A Note on Digital Identity: Parties versus Public Keys

We have in previous chapters had a loose notion of a *party*. It is not always clear what is meant as the physical entity behind this notion. Sometimes it is the physical machine that can be found at the other end of an authenticated channel. Sometimes it is a secret key, or the holder of that secret key. In this section it is important to be a bit more precise. We will identify parties P_i by their verification keys vk_i for a signature scheme. We assume that the secret key sk_i corresponding to the public key pk_i is stored on a single machine. We sometime call this machine **party** P_i . However, if the machine holding sk_i crashes or is slowed down due to a denial of service attack we might move sk_i to another machine. We still think of it as the same party, it just uses different hardware to run itself. When it is using this hardware to run the correct algorithm we call it honest. When it is crashed and not currently running on a new machine we call it crashed. If it is running another algorithm than it should we call it corrupted. If sk_i is currently not on a running machine because the owner does not want to take part in running the TOB at the moment we call party P_i offline. If sk_i leaks to the adversary, then the adversary can send alternative messages in the name of P_i . Therefore P_i is corrupted in this case: it might send messages it should not send.

16.4 Synchronous Clocks

We now proceed to give a more formal treatment of a Nakamoto Style Consensus protocol for TOB. We assume that there is a notion of global physical time t . We assume that each party vk_i has a local clock $Clock_i$. We assume that there is a bound $MaxDrift$ on clock drift. Specifically we assume that $|t - Clock_i| \leq MaxDrift/2$ for all correct P_i . This in particular means that the clock of any two correct P_i and P_j will have the relation $|Clock_i - Clock_j| \leq MaxDrift$. This could for instance be done by each client synchronising against a server.

16.5 Flooding System

As usual we assume that there is a flooding system. This models the peer-to-peer network. We consider a model where parties can come and go. If a party is awake when a message is sent and stays awake for long enough, then it is guaranteed to get the message.

We assume that the network might have a varying network delivery time. At time t we use $DelTime_t$ to denote the current network delivery time. This means that any party which is alive and honest at time t and stays alive and honest will receive all messages sent at time t before time $t + DelTime_t$. We assume the parties agree on a guess $DelGuess$ at the delivery time and that $DelTime_t$ is below this bound “often enough”. More details about this later.

The ideal functionality works as follows:

- For each party P_i there is a protocol port $Flood.Flood_i$ for sending and receiving messages. Each P_i also has a port $WAKEUP_i$ and a port $GoToSleep_i$.

There is a set **Awake**, which is initially empty.

For each P_j there is a set **InTransit_j** of messages that are in transit to P_j but were not delivered yet. It is initially empty.

- The ideal functionality keeps a set **Awake**. On any input on **WAKEUP_i** it adds i to **Awake**. On any input on **GoToSleep_i** it removes i from **Awake**.
- On input x on **Flood_i** where $i \in \text{Awake}$, add x to **InTransit_j** for all P_j .
- On input x on **Deliver_i** where x is in **InTransit_i**, remove x from **InTransit_i**. If $i \in \text{Awake}$, then output x on **Flood.Flood_i**.

For a given message x , let I_x be the time t when x was input to a correct and alive P_i the first time. Let O_x be the time t when x was delivered at the last correct P_i that was in **Awake** since time I_x . We call $O_x - I_x$ the delivery time of the message and assume that most of the time it is less than **DelGuess**. For the case where x was input to an incorrect P_i the guarantee is as follows: If any correct P_i outputs x at time I_x and P_j is correct and alive when this happens and P_i and P_j remain correct and alive for **DelGuess** seconds, then P_j will also output x no later than **DelGuess** seconds after P_i . Again, we only assume this holds most of the time. We return to this below in more detail.

16.6 Lottery System

We also assume a lottery system **LOTTERY**. It breaks time up into so-called **slots** of length **SlotLength**. We will in this book assume that **SlotLength** = **DelGuess**. Other choices are possible but setting **SlotLength** = **DelGuess** gives an easier description and analysis.

A party P_i is in slot **slot** if

$$\text{slot} - 1 \leq \frac{\text{Clock}_i}{\text{SlotLength}} < \text{slot} .$$

There is a lottery system which allows a party P_i to get a **Draw_{i,slot}** in each slot. Each such draw has an associated value **Val(Draw_{i,slot})**. The winners of the slot are the parties with **Val(Draw_{i,slot})** higher than some threshold. In a bit more details, the lottery proceeds as follows:

- In the first round each party generates a key pair (vk_i, sk_i) for a signature scheme and vk_i is broadcast to all other parties. This is done by putting it in a so-called genesis block, see below. This signature scheme should have unique signatures such that for each vk_i and each message m there is at most one value σ such that $\text{Ver}_{vk_i}(\sigma, m) = \top$. This holds for instance for the RSA signature scheme, if the keys were generated honestly.
- Each party vk_i has an associated value **Tickets_i**. In a PoS it is, e.g., the amount of cryptocurrency held by vk_i . Think of this as how many tickets party vk_i has bought. These numbers are made public in the genesis block too.
- Finally a random number **Seed** is made public. It should be unpredictable by all parties before they pick their verification key vk_i . This is to prevent that parties pick keys which are particularly good for the lottery. We will discuss later how such a random number could be picked. The seed is made public in the genesis block.
- For each $\text{slot} = 0, 1, \dots$, party vk_i can compute the draw

$$\text{Draw} = \text{Sig}_{sk_i}(\text{LOTTERY}, \text{Seed}, \text{slot}) .$$

We use the short hand

$$\text{Draw}_{\text{slot},i} \leftarrow \text{Draw}(\text{sk}_i, \text{slot})$$

for P_i computing its draw in slot slot . Note that because of *Seed* being picked *after* the keys were published, these draw values were unknown to the parties when they picked sk_i .

- Other parties can verify the draw by checking that $\text{Ver}_{\text{vk}_i}(\text{Draw}, (\text{LOTTERY}, \text{Seed}, \text{slot})) = \top$. We write this as $\text{VerDraw}_{\text{vk}_i}(\text{Draw}, \text{slot}) = \top$ below.
- The value of a draw is defined as follows: If $\text{VerDraw}_{\text{vk}_i}(\text{Draw}, \text{slot}) = \perp$, then $\text{Val}(\text{vk}_i, \text{slot}, \text{Draw}) = -\infty$. Otherwise,

$$\text{Val}(\text{vk}_i, \text{slot}, \text{Draw}) = \text{Tickets}_i \cdot H(\text{LOTTERY}, \text{Seed}, \text{slot}, \text{vk}_i, \text{Draw}) .$$

The idea here is that when you apply a hash function H to an unknown value, you get a completely random looking value. And then we multiply by the amount of tickets to make the winning probability proportional to the number of tickets.

Who Wins?

There is a hardness level **Hardness**. Any party vk_i with $\text{Val}(\text{vk}_i, \text{slot}, \text{Draw}) \geq \text{Hardness}$ won the lottery at slot slot .

The function H will be a cryptographic hash function with 256-bit output, for instance SHA256. For the analysis we will assume that it is a **random oracle**, that is, it outputs uniformly random values (except that on the same input it always outputs the same value). Therefore the value of a draw is random and but can at the same time be verified by the other parties.

What do you Win?

Recall that when we looked at state-machine replication and synchronous total-order broadcast the blueprint for the total-order broadcast was to use a flooding system to disseminate the messages and then in each round have a leader send a block which determined the order of some of the messages that had not so far been ordered. We used a simple round-robin schedule to elect the leader. Blockchain-based total-order broadcast is meant for a peer-to-peer system with sporadic participation (nodes come and go). In such a system we cannot use round-robin as we don't know which nodes are present in an hour from now and might not agree on who is alive at any given point in time. What we will do instead is use the above lottery system to elect the next leader at random. Since the leader is to produce the next block, the leader is sometimes also called the *block producer* for the slot.

Winners Private until Announced

Notice that only the party P_i who knows sk_i can compute

$$\text{Draw}_{\text{slot},i} = \text{Sig}_{\text{sk}_i}(\text{LOTTERY}, \text{Seed}, \text{slot}) ,$$

otherwise the signature scheme would not be secure. Therefore $\text{Val}(\text{vk}_i, \text{slot}, \text{Draw}_{\text{slot},i})$ is completely unknown to all other parties until vk_i makes $\text{Draw}_{\text{slot},i}$ public. Namely, you cannot predict the output $y = H(x)$ of a random oracle H without knowing the input x . This means that a possible network attacker does not know who the next round leader is until the leader makes $\text{Draw}_{\text{slot},i}$ public. This is an important design principle to withstand so-called **deniable of service** (DoS) attacks, where an attacker tries to attack and crash important servers in the system. Resilience to DoS attacks is particularly important in an open peer-to-peer system.

Since all the peers can send messages to each other, so can the attacker. Often sending a lot of messages to a machine is all it takes to crash it or make it disconnected from the network. So, if an attacker knew all future round leaders well ahead of time it could focus on crashing these before it is their time to lead. This would kill the liveness of the system. And as we will see later, this also kills safety in the blockchain.

Winners can be Recognised

Notice, however, that all parties know all the values Tickets_i and vk_i , so all parties can compute $\text{Val}(\text{vk}_i, \text{slot}, \text{Draw}) = \text{Tickets}_i \cdot H(\text{LOTTERY}, \text{Seed}, \text{slot}, \text{vk}_i, \text{Draw})$ once they are given $(\text{vk}_i, \text{slot}, \text{Draw})$. Hence the winner can be recognised once it makes itself known.

Why Unique Signatures?

The reason why we need unique signatures is that if P_i could compute several valid Draw 's for which $\text{Ver}_{\text{vk}_i}(\text{Draw}, (\text{LOTTERY}, \text{Seed}, \text{slot})) = \top$ then it could compute $\text{Val}(\text{vk}_i, \text{slot}, \text{Draw}) = \text{Tickets}_i \cdot H(\text{LOTTERY}, \text{Seed}, \text{slot}, \text{vk}_i, \text{Draw})$ for all of them and use the Draw giving the highest value. So a malicious server would get multiple attempts at winning the lottery, and we would like that corrupt parties lose the lottery as often as possible.

What is the Seed About?

Assume that Seed was not there. Then we would have that

$$\text{Draw}_{\text{slot}, i} = \text{Sig}_{\text{sk}_i}(\text{LOTTERY}, \text{slot}) ,$$

so the future draws would essentially depend only on slot . Then when a corrupted party P_i is picking its keys $(\text{vk}_i, \text{sk}_i)$ it could try to pick them as to make $\text{Val}(\text{vk}_i, \text{slot}, \text{Sig}_{\text{sk}_i}(\text{LOTTERY}, \text{slot}))$ as large as possible for a lot of future values of slot , to get an advantage in the lottery. The use of Seed thwarts this attack as vk_i is picked *before* Seed is made public.

16.7 Growing a Tree

The ideal situation in NSC is that there is a sequence of honest winners with time enough between the winning events for the previous block to have reached the next winner. In this ideal situation the system will build a single chain of blocks, just as in the synchronous total order broadcast protocol we saw in Chapter 14. However, the world is not that kind to us. We have two types of problems to solve in NSC. Sometimes a winning block might get delivered only to some correct parties—either because the block is sent late with malice or because the delivery time is too high in the slot—and sometimes there is no slot winner. Finally, there might also be multiple winners in a slot. It turns out that the solution to all of these problems is to grow a tree instead of a chain. Each party is then supposed to take the longest path in the tree as its chain. As we will see, this will tend to converge on agreement.

Definition 16.1 (block tree) A tree node is of the form $(\text{BLOCK}, \text{vk}, \text{slot}, \text{Draw}, U, h, \sigma)$, where BLOCK is just the text “block” that specifies the type of the tuple, vk to verification key of the winner, $\text{slot} \in \mathbb{N}$ is a block number, Draw is the draw that was used to win the lottery, U is the block data, h is a block hash (of some previous block), and σ is a signature on the whole thing, which is supposed to be valid under vk . We define the value of a block N to be $\text{Val}(N.\text{vk}, N.\text{slot}, N.\text{Draw})$. There is a special block $(\text{BLOCK}, \perp, 0, \perp, U_0, \perp)$ with value ∞ called the genesis block, where U_0 is the genesis data. The genesis block contains **Hardness** and

more. We sometimes also just call G **genesis**. A set S of blocks which contains G and where all blocks are valid defines a **tree** as follows. The nodes of the tree is a sub-set of the nodes in **Tree**. The root of the block tree is the genesis block G . Edges are directed and points towards the root. There is an edge to $N_1 \in \text{Tree}$ from $N_2 \in \text{Tree}$ if and only if $N_2.h = H(N_1)$ and $N_2.\text{slot} > N_1.\text{slot}$, i.e., N_2 points to N_1 by containing a hash of N_1 and we require that nodes point to nodes from an earlier slot. For a given node N in the tree we let $\text{PathTo}(N)$ be the list of nodes from G to N , including G and N and indexed from 0.

▼

The fact that nodes point to nodes with a smaller **slot** ensures that the graph has no cycles. The fact that the tree actually becomes a tree uses that H is collision resistant. Each node can point to at most one other node because $H(N_1)$ identifies N_1 .

An important component in blockchains is the notion of some paths in the tree being better than another. In general we just like the longest path the best. This is the one we want to build on. However, when two paths have the same length we might want to use a tie breaker, and when we discuss finalization in Chapter ??, we will add more bells and whistles.

Definition 16.2 (path weight) A blockchain is a block tree where each block has at most one parent. A path weight is a function PathWeight mapping blockchains into a total-order set. It should have the property that if P' is a proper prefix of P , then $\text{PathWeight}(P') < \text{PathWeight}(P)$. It should also have the property that if $P \neq P'$ then $\text{PathWeight}(P) \neq \text{PathWeight}(P')$. Finally it should have the property that it cannot happen that $\text{PathWeight}(P') < \text{PathWeight}(P)$ and $\text{Len}(\text{PathWeight}(P')) > \text{Len}(\text{PathWeight}(P))$. The default path weight is just the order which sorts first on length of P and then $\text{Val}(\text{Leaf}(P))$. So $\text{PathWeight}(P_1) < \text{PathWeight}(P_2)$ if and only if $\text{Len}(P_1) < \text{Len}(P_2)$ or $\text{Len}(P_1) = \text{Len}(P_2)$ and $\text{Val}(P_1) < \text{Val}(P_2)$. Finally, if $\text{Len}(P_1) = \text{Len}(P_2)$ and $\text{Val}(P_1) = \text{Val}(P_2)$ then one can sort the paths lexicographically when represented as bit-string.

▼

Definition 16.3 (best path, best leaf) Let $\text{Tree} = \text{Tree}(S)$ be a block tree. The best path in Tree is the path from G to a leaf that maximizes PathWeight . We write $P = \text{BestPath}(\text{Tree})$. The best leaf of Tree is

$$\text{BestLeaf}(\text{Tree}) = \text{Leaf}(\text{BestPath}(\text{Tree})) .$$

▼

The protocol uses two auxiliary functions GetMetaData and ValidMetaData . The first one will ask the local system if there is extra data to include in the block data. The latter will check whether some meta data is valid. For now just assume that $\text{GetMetaData}() = \emptyset$ and $\text{ValidMetaData}(\cdot) = \top$. We do not use meta data until later, at which point we discuss it again.

16.7.1 The Main Idea in a Nutshell

The main idea behind the protocol is that if a party wins the right to produce a new block it must add it to the tree by making it point to the leaf of the currently best path in its own tree. Therefore the path which is already best will remain best, and get even better, when it is honest parties which extend the tree. A corrupted party might be naughty and extend for instance the next-best path. But if there are more honest winners than corrupted winners, then the best chain will grow faster than the next-best chain and will stay best. We can therefore define the blockchain via the blocks in the longest path in the tree. There are a lot more details to make that argument work, which we return to below. The protocol proceeds as in Fig. 16.1.

- The protocol uses a lottery system LOTTERY.
- The protocol uses system Flood for flooding messages. When P_i receives a message on TOB_i it sends it on $Flood.Flood_i$. If a message arrives on $Flood.Flood_i$ then P_i adds it to $Received_i$.
- Each P_i has a clock $Clock_i$ and $|Clock_i - Clock_j| \leq \text{MaxDrift}$ for all pairs of honest parties.
- The protocol breaks time up into so-called slots of length SlotLength . A party P_i is in slot slot if $\text{slot} - 1 \leq \frac{Clock_i}{\text{SlotLength}} < \text{slot}$. Throughout the protocol let slot_i denote the current slot number of P_i . Initially $\text{slot}_i = 0$.
- P_i : Initially, let $Received_i = \emptyset$. Let G be the genesis block and initially let $S_i = \{G\}$. The set S_i is just the set of all received blocks, even invalid ones which do not fit into the tree. Throughout the protocol let $\text{Tree}_i = \text{Tree}(S_i)$. This is the tree made out of the legal blocks.
- For a path P in Tree_i let $\text{Delivered}(P)$ be the set of messages delivered in P , i.e., the messages appearing in a block in P .
- Each P_i runs the below activation rules. All values are sent via Flood.
 1. Immediately when slot slot_i is entered, let $P = \text{BestPath}(\text{Tree}_i)$ and get the draw Draw_i . If $\text{Val}(\text{vk}_j, \text{slot}, \text{Draw}) \geq \text{Hardness}$, then proceed as follows. Let $M_i = \text{GetMetaData}$, let $U_i = \text{Received}_i \setminus \text{Delivered}(P)$, let $B_i = \text{Leaf}(P)$, let $h_i = H(B_i)$, let $\sigma_i = \text{Sig}_{\text{sk}_i}(\text{BLOCK}, \text{slot}_i, (U_i, M_i), h_i)$, and send $(\text{BLOCK}, \text{vk}_i, \text{slot}_i, \text{Draw}_i, (U_i, M_i), h_i, \sigma_i)$.
 2. On input $(\text{BLOCK}, \text{vk}_j, \text{slot}, \text{Draw}, (U, M), h, \sigma)$ with $\text{Ver}_{\text{vk}_j}(\sigma_j, (\text{BLOCK}, \text{slot}, (U, M), h)) = \top$, $\text{VerDraw}_{\text{vk}_i}(\text{Draw}, \text{slot}) = \top$ and $\text{Val}(\text{vk}_j, \text{slot}, \text{Draw}) \geq \text{Hardness}$, store it and wait until it happens that $\text{slot}_i > \text{slot}$ and $\text{ValidMetaData}(M) = \top$, then add it to S_i .

Figure 16.1: Building a Block Tree

16.7.2 How does the Tree Grow

We now want to explore how the block tree behaves over time. Recall that we use the word winner to denote any party which has a value which is higher than Hardness . This means that there can be several winners and sometimes there might be no winner. An important concept will be the honest tree.

Definition 16.4 (honest tree) Given two valid trees Tree_i and Tree_j the union $\text{Tree}_i \cup \text{Tree}_j$ is simply the tree with root G that contains all the paths of both Tree_i and Tree_j . This is again a valid tree. Let H be the set of all correct P_i at time t and let Tree_i^t be the tree of P_i at this time. Let

$$\text{HonestTree}_{\text{slot}}^t = \bigcup_{i \in H} \text{Tree}_i^t.$$

▼

Notice that by definition all correct parties at any time sees a tree which is a subset of

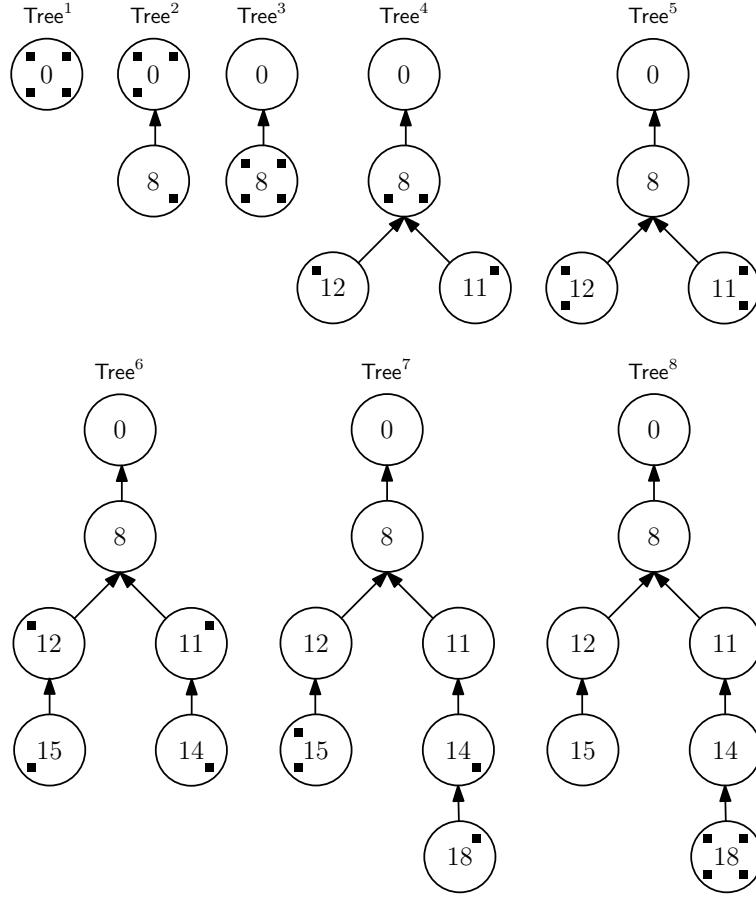


Figure 16.2: We depict four parties as small black squares. A circle around a number means that some party won that slot number and created a block. We show the whole honest tree. If we put a square in a circle, it means that that party considers that block to be the currently best leaf. At the beginning (Tree¹) all four parties know only the genesis block. Then in slot 8 some party P_i wins the slot and sends a new block. At the time P_i adds the block to S_i the honest tree grows by that block. But so far only P_i saw it. This creates Tree². After some time all the other processes see the new block and consider it the currently best leaf. Then some P_j wins slot 11 and right after P_k wins slot 12 before seeing block 11. This creates the situation in Tree⁴. The two new blocks reach different parties at different times, creating the Tree⁵. Now we get even more unlucky and one of the parties in block 11 wins slot 14 and about the same time a party in block 12 wins slot 15. They don't have time to see each other's blocks and therefore extend different paths, as shown in Tree⁶. Then one of the other parties sees block 15 first and the other sees block 14 first. It goes to 14 and right after wins slot 18, as shown in Tree⁷. Now there is a sufficiently long time where there is no new slot winner, so block 18 reaches all parties before the honest tree grows. Therefore all four parties consider block 18 the best one and go to that block. Next time there is an honest block winner it will extend at 18 and the short so-called fork ending in block 15 will soon be forgotten.

the honest tree. Namely, for all correct P_i , clearly $\text{HonestTree}^t = \text{HonestTree}^t \cup \text{Tree}_i^t$. Notice furthermore that if DelTime_t is the network delivery time at time t and P_i is a party which is alive at time t until time $t + \text{DelTime}_t$, then

$$\text{HonestTree}^t \subseteq \text{Tree}_i^{t+\text{DelTime}_t},$$

as all nodes seen by any honest party at time t will reach P_i by time $t + \text{DelTime}_t$. So in some vague sense, if we squeeze our eyes enough that we cannot see what happens in intervals of time DelTime_t , all honest parties basically see HonestTree . In particular, if for long enough the honest tree does not grow, then all honest parties will in fact see the honest tree and will therefore agree on the best leaf. So if there is long enough between slot winners, all honest parties tend to build on the best path in their common honest tree, which will therefore get better and better (longer and longer) than all small unlucky forks.

To start familiarising ourselves with how the tree grows, consider the example in Fig. 16.2. This example shows how the tree protocol tends to get rid of unlucky disagreements: at some point one of the branches of a fork will get longer than the other and reach all parties in time that they all switch to it. Notice that for this to be true, it is important that there is often enough breaks of length DelTime_t where no new block is made. If we produce block winners much faster than the blocks can reach all parties, two branches as in tree 6 in Fig. 16.2 could both keep growing.

16.7.3 Rollback and Finalisation

A big problem with NSC protocols is that it is not easy to know when one can safely deliver a transaction. Notice that if a party P_i delivered a transaction T that was in block 12 on TOB_i , then it cannot take it back when it changes to the path $(0, 8, 11, 14, 18)$, and in that path maybe T had to be delivered at a different point in the order. Therefore agreement would be violated. When a party changes from the chain $(0, 8, 12, 15)$ to $(0, 8, 11, 14, 18)$ it is called a **rollback**. All the transactions performed in blocks 12 and 15 are rolled back and might now have to be performed in a different order. If we want to avoid rollbacks, parties cannot deliver on TOB_i any transaction that is in a branch that might later be rolled back. A block that cannot ever be rolled back is called a **final block**.

There are two different approaches to deciding when a block is final. One of them is to just wait “long enough” and only consider a block safe when it is long enough up in the tree. Notice for instance that when the parties were divided between blocks 15 and 14, if they looked two blocks up in the tree, they would all see block 8, which will never be rolled back. At least it never rolled back in the small example in the figure. Another approach is to run a separate process which detects which blocks are final, called a **finalization layer**. We will explore both approaches. We explore the first approach in this chapter and explore finalization in Chapter ??.

16.7.4 Ghost Growth

To be able to understand how far we need to look up in the tree to find a block that will never be rolled back, we need to much better understand how the tree grows. The first step is to first look into another attack possibility by corrupted parties. Consider Tree^8 in Fig. 16.2. Recall that long break from slot 0 to slot 8, where no slot was won. It turns out that indeed slot 2 and slot 6 were won by corrupted parties. They just did not send their blocks. The corrupted parties also won slots 10, 13, and 16, and did not send those blocks either. They were collecting ammunition for a nasty attack.

Notice that winning a slot just means that $\text{Val}(\text{vk}_i, \text{slot}, \text{Draw}) \geq \text{Hardness}$. So if a corrupt vk_i wins slot slot it can for all possible block data U and all previous blocks B with a smaller

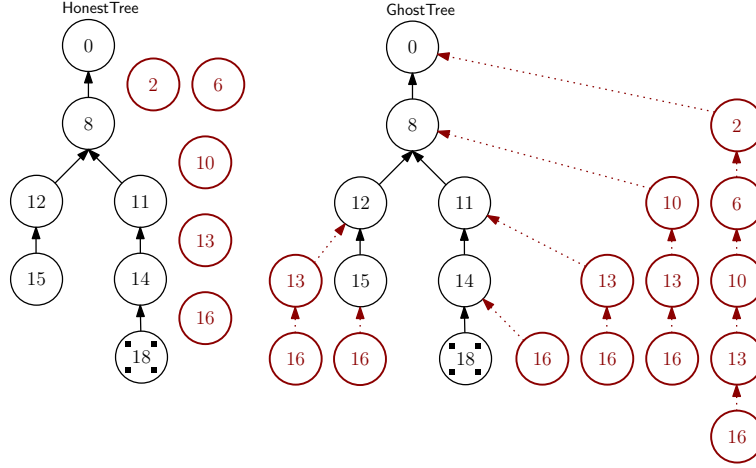


Figure 16.3: To the left: an honest tree along with five slots won by corrupted processes which did not send their blocks yet. To the right: some of the ghost tree. The ghost tree is just the largest tree that the corrupted parties might build with their withheld blocks. They can also choose to build some subset of the ghost tree. We only show some of the ghost tree. We only show the longest possible forks that the corrupted parties could add. They can add any sub-sequence of these. The corrupted parties could for example also choose to add (2, 6, 13, 16) at 0.

slot number produce a block $(\text{BLOCK}, vk_i, \text{slot}, \text{Draw}, U, H(B), \sigma)$ which will be valid. The ghost tree is the tree obtained by starting with the honest tree and for each withheld winning draw, add the corresponding blocks at all possible places. In particular, the corrupted parties can add a new block at any node in the ghost tree with a smaller slot number. Iteratively extending the ghost tree by adding all corrupted blocks at all possible places gives what is known as the ghost tree. We denote the `GhostTree` at time t by `GhostTree` ^{t} . In Fig. 16.3 some of the ghost tree is shown to the right.

If the corrupted parties produce the block corresponding to a ghost branch in the tree and send them on the network, we say that they reveal the ghost branch. Notice that if the corrupted parties reveal the ghost branch (2, 6, 10, 13, 16) in the ghost tree in Fig. 16.3, then the honest parties would be forced to make a long rollback, as they will all adopt (0, 2, 6, 10, 13, 16) as the best path. Notice that this happened even though there were six honest winners and only five corrupt winners. However, the honest tree did not grow by six nodes as there was some branching which wasted two honest winners. We can therefore see that if we want to avoid long rollbacks, it will be a balance between how many honest parties are in the network, how many corrupted parties are in the network, and how much wasteful branching there is due to late arrival of blocks.

One solution which often comes to mind for limiting the problem is to just say that we do not allow too long rollbacks. See Fig. 16.4 to see why this is a bad idea.

16.8 Understanding Tree Growth

In understanding tree growth, three properties have crystallised out of the current scientific literature called **tree growth**, **chain quality**, and **limited rollback**.

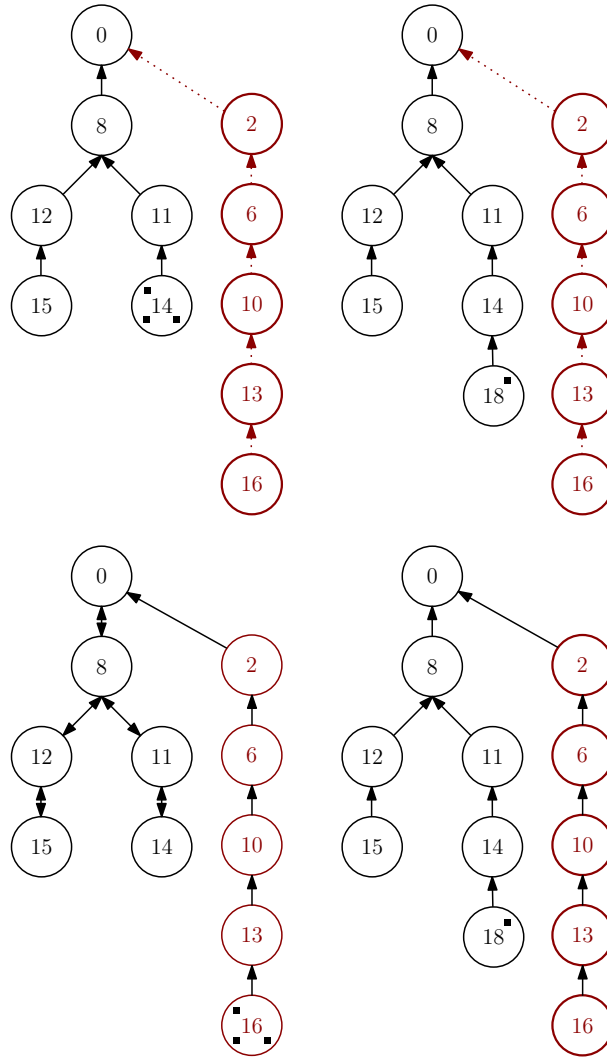


Figure 16.4: An example why it is dangerous to use time in distributed systems. Imagine we enforce the rule that you are never allowed to do a roll back longer than three. Consider a situation as in the top row. Three parties are in block 14 still, but the slot winner already made it to block 18. Now the corrupted parties quickly reveal a ghost branch. The parties in block 14 are willing to do the roll back. The party in 18 is not willing (or allowed to) as the rollback is too long, so it stays on the path containing 18 forever. Now agreement will never be reached, and the network is broken. This shows why using time is either dangerous or useless. If you ban rollbacks of length L and you might have branches of length L then the above example might happen and agreement is broken forever. On the other hand, if you ban rollbacks of length L and you never can have branches of length L , then why ban rollbacks of length L : they never happen anyway. So do not use time to deal with rollbacks.

16.8.1 The Standard Tree Scenario

Which guarantees you get from a blockchain depends a lot on the level of corruption and the block time relative to the network propagation time. Instead of trying to derive a general formula we will for clarity instead study some concrete settings of the parameters and try to understand these concrete settings. We consider an example scenario which we revisit below, so let us call it our **standard tree scenario**. We assume that:

- At least 90% of the alive tickets are honest, i.e., among the parties which will run the system in the period we analyse the honest parties hold 90% of the tickets.
- $\text{DelGuess} = \text{SlotLength}$ is set high enough that 95% of all slots are timely. We call a slot timely if the network delivery time is smaller than DelGuess , see below.
- The hardness is set such that 90% of all honest winners are unique, i.e., they win a slot in which there are no other honest winners.

Of course, assuming 90% honesty is a strong assumption. In some of the other chapters we only assume 51% and 67%. In fact, the NSC we present in this chapter can be proven secure for 51% honesty, but the analysis is very complicated. For 90% honesty the proof is simple, and still illustrates the main ideas.

The assumption that 95% of all slots are timely can be ensured by setting the slot time large enough. Notice that we only need the slot time to be longer than network propagation time 95% of the time. This is much weaker than in the fully synchronous model where we assumed that *all* message be delivered faster than DelGuess .

The assumption that 90% of all honest slots are unique can be ensured by setting the hardness such that the probability that a given slot is won is at most 10%. Consider now any slot won by one honest party already. The probability of another one will win it is at most 10%. Hence an honest party winning a slot has 90% chance of being unique.

16.8.2 Tree Growth

We first discuss tree growth. It basically says that the tree gets higher all the time as there are more and more slot winners. It is parametrised by a fraction TreeGrowth which says that about a fraction TreeGrowth of the slot winners make the tree get higher.

Definition 16.5 (tree growth) Let $\text{TreeGrowth} \in [0, 1]$ be a real number. We say that a protocol has a tree growth of TreeGrowth if after n slot winners the height of HonestTree is very close to $\text{TreeGrowth} \cdot n$ except with small probability.



We show that the above protocol has a good tree growth under the assumptions that there are a lot of honest tickets, that slot winners are not produced too fast and that messages are delivered reasonably timely.

Definition 16.6 (timely slot) We define what it means for a slot slot to be timely. Let t be the time the slot starts. We say slot slot is timely if all messages sent before time $t - \text{DelGuess}$ are delivered before time t .



Note that $t - \text{DelGuess}$ is the beginning time of the previous slot. So, in particular all honest blocks from the previous slot would have time to reach all honest parties before the beginning of a timely slot. More generally, if slot is timely, then any block seen by an honest party at the beginning of slot $\text{slot} - 1$ would reach all honest parties before the beginning of the timely slot. Keep that thought, we use it below.

We call slot slot an **honest slot** if there is at least one honest winner in slot slot . An honest slot might have corrupted winners too and several honest winners. We call slot slot a **lucky slot** if it is a timely slot *and* an honest slot.

Lemma 16.7 (lucky slot) *Consider a timely slot slot and let HonestTree^t be the honest tree when the slots begins. Assume that there is an honest winner in slot slot . Let $\text{HonestTree}^{t-\text{DelGuess}}$ be the honest tree DelGuess seconds before. Then*

$$\text{Len}(\text{BestPath}(\text{HonestTree}^t)) \geq \text{Len}(\text{BestPath}(\text{HonestTree}^{t-\text{DelGuess}})) + 1 .$$

The *lucky slot* lemma say that if a slot is both timely and is being won by an honest party, then the honest tree will grow. This is easy to see. Consider the tree $\text{HonestTree}^{t-\text{DelGuess}}$. Let P_i be the slot winner. By time t all messages reached P_i so $\text{HonestTree}^{t-\text{DelGuess}} \subseteq \text{Tree}_i^t$. Hence

$$\text{Len}(\text{BestPath}(\text{Tree}_i^t)) \geq \text{Len}(\text{BestPath}(\text{HonestTree}^{t-\text{DelGuess}})) .$$

When P_i then extends $\text{BestPath}(\text{Tree}_i^t)$ it gets a new tree with

$$\text{Len}(\text{BestPath}(\text{Tree}_i')) = \text{Len}(\text{BestPath}(\text{Tree}_i^t)) + 1 .$$

Since Tree_i' is part of HonestTree^t it follows that

$$\text{Len}(\text{BestPath}(\text{HonestTree}^t)) \geq \text{Len}(\text{BestPath}(\text{HonestTree}^{t-\text{DelGuess}})) + 1 .$$

Notice that the lemma does *not* say that the specific path

$$P = \text{BestPath}(\text{HonestTree}^{t-\text{DelGuess}})$$

will grow, as

$$P' = \text{BestPath}(\text{HonestTree}^t)$$

might actually not be an extension of

$$P = \text{BestPath}(\text{HonestTree}^{t-\text{DelGuess}}) .$$

For instance, the adversary might reveal a longer ghost chain to move people away from P . But that ghost chain will have at least the same length as P to make parties swap to it, so the lemma still holds if P_i extends the revealed ghost chain.

By the lucky slot lemma we see that whenever there is a new honest winner the honest tree grows in height by 1 unless one of these happened:

1. The winner is in a slot which was already honest—in which case both winners might grow the tree at the same leaf.
2. The slot was not timely, in which case the winner might not build on the best path in the honest tree because it did not have time to reach the honest winner.

Definition 16.8 (wasted honest winners) We call an honest winner wasted if it wins a slot which is not timely or it wins a timely slot which had another honest winner with a value of its ticket that is higher. We let $\text{WastedHonestWinners}^t$ be the number of wasted honest winners at time t .



This immediately gives us this result, which can be used to establish tree growth.

Theorem 16.9 (tree growth) *Let HonestWinners^t be the number of honest winners at time t . Then*

$$\text{Len}(\text{BestPath}(\text{HonestTree}^t)) \geq \text{HonestWinners}^t - \text{WastedHonestWinners}^t.$$

Consider the standard tree scenario. As for wasted honest winners, note that out of 100 winners on average 90 are honest. Of these at most 10% are not unique. Since 10% of 90 is 9 it follows that at least 81 of the honest winning events are honest and unique. Of these at most 5% hit slots that are not timely. Since 5% of 81 is 4.05 which is less than 5 the number of wasted honest winners is less than $9 + 5 = 14$. Hence $\text{TreeGrowth} > 90\% - 14\% = 76\%$.

16.8.3 Chain Quality

Chain quality asks that a lot of the nodes on the best path were produced by honest parties. In the extreme case where all nodes are produced by corrupted parties, they might censor some transactions by not adding them to their blocks. If only few nodes are produced by honest parties, the throughput of censored transactions can be lowered by the corrupted parties which can be bad enough.

Definition 16.10 (chain quality) Let $\text{ChainQuality} \in [0, 1]$ be a real number. We say that a protocol has a chain quality of ChainQuality if the best path in HonestTree has length L then very close to $\text{ChainQuality} \cdot L$ nodes on the chain were produced by parties which were honest when they produced the node, except with negligible probability.



We can now do a very simple analysis of chain quality in the standard tree scenario. There is tree growth 76%, so after 100 winning events the tree grows by at least 76 blocks. So at most 24 blocks sit in side branches to the longest path. Worst case these 24 are all honest nodes. Of the 100 nodes on average 90 are honest. At most 24 of these were lost to side branches. So 66 of them must be sitting on the longest path. So worst case 66 of the 76 blocks on the longest chain are honest. Since $66/76 > 87\%$ it follows that $\text{ChainQuality} > 87\%$.

Exercise 16.1 (worse chain quality) Consider the standard tree scenario but consider lower threshold for honest tickets like 85% and 80%. Try to calculate which chain quality you get for these lower thresholds. What is the lowest threshold for which you still get chain quality at least 51%?



16.8.4 Limited Rollback

The limited rollback property asks that there is a limit RollbackLimit such that the probability that there will ever be a rollback of length longer than RollbackLimit is some negligibly small probability.

To analyse the limited rollback property we introduce the notion of a super slot. We define a **super slot** to be one which was won by a honest party and where the slot was timely and where no other honest party won the slot. A **super block** is the block produced by the honest party winning a super slot. Notice that for each slot number slot there is at most one super block with slot number slot . This is because a super slot has exactly one honest winner and honest

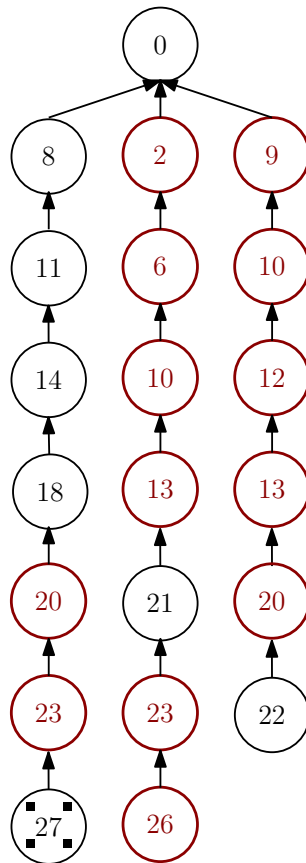


Figure 16.5: An example of paths with super slots and filler slots. The super slots are the black ones. The filler slots are the red ones. Notice that the super slots sit at different heights. This will always be the case.

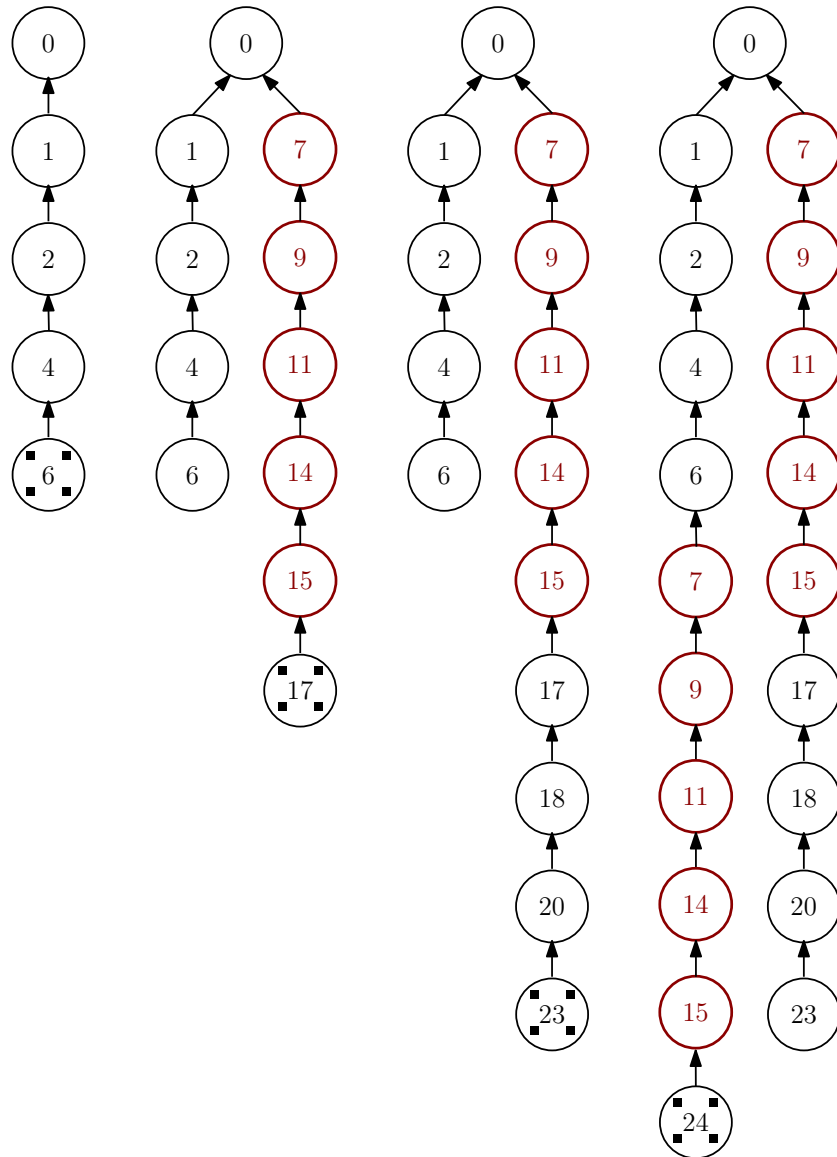


Figure 16.6: Another example of paths with super slots and filler slots. Here a rollback of length 9 is caused using only 5 distinct fillers.

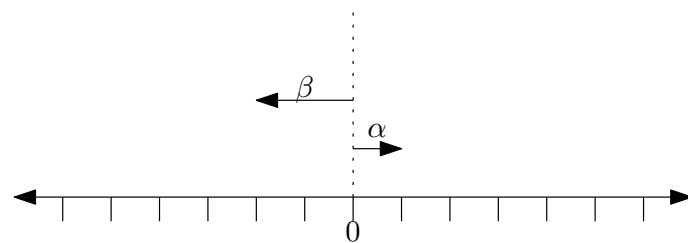


Figure 16.7: Random walk. You start from 0 and takes steps to the left with probability β and steps of some other length to the right with probability $\alpha = 1 - \beta$. Where do you tend to end up after n steps?

winner produce a single block. There might be a block with the same slot number, but then it will be produced by a corrupted party and hence will not be a super block.

Consider our standard tree scenario. Out of every 100 winners, there will tend to be about 90 honest ones. Less than 10% of these will hit a slot with another winner, so there will tend to be 81 that have their own slot. Of these at most 5% will not be timely, so at least $81 - 5 = 76$ will be super slots.

We now prove an important lemma:

Lemma 16.11 (super slots) *All super blocks sit at different heights in the honest tree.*

To see this consider the time when super block number $i + 1$ was added. At this time the honest winner saw super block number i in its tree already, as super blocks are won in different slots and in timely slots. So super block number $i + 1$ was added at a higher point than super block number i .

Call all the other blocks which are not super blocks the **filler blocks**. Here is another important lemma:

Lemma 16.12 (filler slots) *Assume that a rollback is possible from branch B_1 to branch B_2 , both rooted at node N . If since N was created there were created s super blocks, then at least $s/2$ distinct filler blocks were created.*

To get the intuition for the above theorem inspect Fig. 16.6 which shows a maximally unlucky sequence for the honest parties. In general we know that the super slots are at distinct heights. So if there are two branches involved in the rollback, then there are at least as many filler slots as super slots. Each filler block can sit on both branches, but can not sit on the same branch twice. The reason for this is that the slot number is strictly increasing along a chain. The same logic applies even when more branches are involved, but we will not dive into the details.

In our standard tree scenario out of every 100 slot winners about 76 will be super slots. Therefore there will tend to be only 24 filler slots. Since $2 \cdot 24 < 76$, there will tend to not be long rollbacks—there are simply not enough filler blocks to cause them. To get an even better bound on how long rollbacks we should expect consider the following game illustrated in Fig. 16.7. You start at position 0. In each round you flip a coin that comes out heads with probability 76%. If it comes out heads you take one step to the right (+1). Otherwise you take two steps to the left (−2). In one step your expected position is $.76 - 2 \cdot .24 = 0.28$. After n rounds your expected position is therefore $0.28n$, by linearity of expectation.

Flipping a heads corresponds to getting a super block. Tails corresponds to getting a filler block. Note that if there are less than twice as many super blocks as filler blocks, then the position is negative. This is also the case where we cannot exclude a rollback. The question then is: how large must n be that there isn't a big chance that your position is negative?

Let ℓ be the number of steps to the left and let r be the number of steps to the right. The position is negative if and only if $r \leq 2\ell$. Since $n = \ell + r$, this is the same as $r \leq \frac{2}{3}n$. So we can consider a different game. Throw a coin as before. If it is heads go right (+1) otherwise stay put (+0). After n rounds what is the probability that your position is less than $\frac{2}{3}n$. It is well known how to analyse this using a so-called Chernoff bound. Let $X_i = 1$ if coin number i came out heads. Let $X_i = 0$ otherwise. Let

$$X = \sum_{i=1}^n X_i .$$

This is your position after n steps. We have that the expected value of X_i is 0.76, i.e.,

$$\mathbb{E}[X_i] = 0.76 ,$$

and

$$\mathbb{E}[X] = 0.76n .$$

Since we cannot have a rollback if there are 67% super blocks, we are interested in $X \geq 0.67n$. Since the expected value of X is $0.76n$ and we only need it to be $0.67n$, there seems to be hope. Since X is the sum of n independent random variables taking on values in $\{0, 1\}$ and they are all 1 with probability 76%, the so-called Chernoff lower bound tell us that it holds for all δ with $0 < \delta < 1$ that

$$\Pr[X \leq (1 - \delta)\mathbb{E}[X]] \leq e^{-\mathbb{E}[X]\delta^2/2} .$$

In our case $\mathbb{E}[X] = .76n$ and we are looking at the probability that $\mathbb{E}[X] \leq \frac{2}{3}n$. So we have to solve $(1 - \delta)\frac{76}{100} = \frac{2}{3}$, which gives us that $\delta \geq 0.12$. We have that $e^{-.76 \cdot (.12)^2/2} < 0.995$. So plugging it all into the Chernoff bound we get that the probability of a rollback of length n is no more than 0.995^n . We can then solve for n in $0.995^n = 2^{-80}$. Taking $\log_2$ we get that $n \log_2(0.995) = -80$ or $n = -80/\log_2(0.995) \approx 11062$.

So if we want security except with probability 2^{-80} we should behave in a way where we tolerate that up to $n = 11062$ distinct blocks are involved in a rollback. Note that when this rollback happens, then the analysis was set up such that $2n/3$ of the blocks are super blocks and $n/3$ are filler blocks. The filler blocks might appear on both chains in the rollback, the old one and the new one. That means that when we look at the number of blocks with repetition, there might be up to $4n/3$ blocks involved. The new chain must by definition be longer than the old chain (the one being rolled back), so the old chain is less than $2n/3$ long. With $n = 11062$ this gives ≤ 7375 .

This all in all means that to consider a block final, it should be on the longest chain and be 7375 blocks up into the tree. In other words, if you consider the longest chain, and count from the leaf, you should go 7375 blocks back in the chain. The block that you find there you can consider to be final. We call this the **lookback length**. If we use the rule that we do not output on TOB_i the contents of a block until it is 7375 blocks deep on the best chain, then we are guaranteed (in the standard tree scenario) except with probability 2^{-80} that you are implementing a total-order broadcast.

The above bound sounds very pessimistic, but it can also be improved significantly. Notice in particular that we gave the adversary a lot of power for free. Inspect Fig. 16.6. To make the attack it first needs the honest parties to get ahead and then catch up with its own blocks. Otherwise all of the blocks cannot fit into both chains. In particular, to fit in both chains the blocks it uses to catch up must all have higher slot numbers than the super blocks the honest parties used to get ahead. Therefore in practice most filler blocks fit in only one chain. So we do not need twice as many super blocks as filler blocks. Another thing we gave for free is the row in the right most tree with nodes 7 and 15. Here there is no super block, so in the right most tree there are 8 super blocks but 10 slots that need to be filled with filler blocks, so the adversary need 5 filler blocks to compete with the 8 super blocks. In the analysis we assumed that he needed only 4. This makes no big difference when we assume a long lucky sequence, but as the expected length of lucky streaks of the adversary will be very small, these extra fillers will weigh him down significantly in practice. Also, in the analysis we took the honest slots that did not happen to be super slots and just gave them to the adversary as if they were corrupted filler blocks which might appear twice. This is also too pessimistic: an honest filler block will not appear twice in the tree. Finally, it will also happen for the adversary that it wins a slot that was already won by the adversary. This will give him no more power than winning the slot once,

Background. Start from your solution to Exercise 6.16. At this point, your system is a distributed ledger with authenticated transactions, but without total order. As a consequence, rejecting overdrafts would risk inconsistencies between peers due to different transaction orders.

Your task is to add total-order broadcast using a Nakamoto-style, tree-based blockchain based on proof-of-stake, as described in Section 16.7 and Section 16.2.2.

System model and currency. As in Exercise 6.16, accounts hold balances, which we interpret as a cryptocurrency. We call the currency unit AU . All transactions and balances are expressed in integral AUs.

Genesis block. The genesis block contains ten special public keys. Each of these accounts has an initial balance of 10^6 AUs. All other accounts start with balance 0.

When a peer starts, it knows the genesis block, extracts the ten initial accounts, and initializes their balances accordingly. You generate all ten key pairs and know the corresponding secret keys. In a real system, these keys would belong to different parties; here this simplification is intentional.

The genesis block also contains an initial seed **Seed**, chosen by you and hardcoded. In practice, this seed should be chosen securely after the initial keys are fixed, but a simple approach suffices here.

Transactions. Balances and transaction amounts are integers. A transaction must transfer at least 1 AU to be valid.

When a transaction is executed, the receiver receives 1 AU less than the amount sent. This difference is a transaction fee.

Maintaining ledger state. Your blockchain forms a tree of blocks, where each block contains a list of transactions. The ledger state depends on the longest path in this tree. To avoid mixing state and updates in complex ways, we recommend implementing the following abstraction:

- **GetCurrentTransactions(*lb* int):** returns the sequence of transactions on the current longest path, but omits the latest *lb* blocks to allow for small rollbacks.
- **LedgerFromTransactions(transactions):** given a sequence of transactions starting from genesis, recompute the corresponding ledger by executing them in order.

By combining these two operations, a peer can always compute the ledger corresponding to the stable part of the chain (e.g. using $lb = 2$ or another small constant). Peers should execute only transactions that are valid and correctly signed.

Blocks and timing. Blocks have a maximum capacity of **BlockSize** transactions. You choose this parameter. It should be small enough that blocks can be transmitted within one block interval, but large enough to process transactions efficiently.

Blocks are produced according to the proof-of-stake lottery. A block may contain any number of transactions up to **BlockSize**, and may be empty if no transactions are available.

The slot length **SlotLength** is 1 second. You may increase it if signature generation is slow. Each peer must compute at least one signature per slot.

Proof-of-stake lottery. To participate in the lottery and produce blocks, a peer must control an account with a positive balance. The number of tickets assigned to a peer is the balance of its account *in the genesis block*. This number remains fixed throughout the execution.

As a result, only the ten initial accounts participate in block production. This is a simplification; in real systems the stake would evolve over time. See Exercise 16.4 for a discussion.

The same signature keys are used both for signing transactions and for participating in the lottery. In practice, separate keys would be used to reduce key exposure.

Block validity and rewards. A block is added to the tree only if all included transactions:

- are correctly signed, and
- are valid, meaning that no account balance becomes negative at any point.

When a block is created, the block producer receives 10 AU, plus 1 AU for each transaction included in the block. The fixed reward represents inflation and incentivizes participation even when transaction volume is low.

Execution parameters. Run the system with 10 peers, each controlling one initial account. Set the lottery hardness so that a new block is created approximately every 10 seconds. If this is too frequent for your system to maintain a stable longest chain, increase the interval.

Report. Your report should include:

1. A description of your system design. If parts of the specification were unclear or under-specified, explain which choices you made. In particular, explain how you chose `BlockSize` and why.
2. Instructions for your TA explaining how to run your code.
3. A description of how you tested your system. You should perform the same testing as in Exercise 4.6 and Exercise 6.16. In addition, include invalid transactions (e.g., negative amounts, zero amounts, invalid signatures, or transactions causing overdrafts) and verify that they are rejected. Provide a program that your TA can run to perform these tests.
4. During testing, reduce the block interval enough to provoke rollbacks and verify that your system tolerates them. If you simulate the network, you may need to introduce artificial delays.
5. Measure the transaction throughput (transactions per second) when the system is not under attack. If you vary `BlockSize`, report which value yields the best throughput. A transaction counts as completed only once it has been ordered and applied to account balances.

Optional extensions. If you want to explore further:

1. Support transfers to and from accounts beyond the initial ones.
2. Run experiments with a fraction ϕ of malicious peers that attempt to disrupt the system, for example by extending non-longest branches. Measure rollback lengths for different values of ϕ .
3. Handle dynamic stake securely (see Exercise 16.4).



Exercise 16.3 (better lookback bound*) In the above analysis the blocks which are not super block are called filler blocks. There are two kinds of filler blocks, the honest filler blocks produced by an honest winner and the corrupted filler blocks produced by a corrupted winner. The corrupted filler blocks can appear twice. However, the honest filler blocks only appear once, as honest winners only add one block per won slot. This means that if we let S be the number of super blocks, let H be the number of honest filler blocks, and let C the number of corrupted filler blocks, then we only need $S > H + 2C$ to avoid a rollback. Argue that this is true. Then use it to do an analysis as the one above determining in the standard tree scenario how long lookback is needed to get security 2^{-30} .



Exercise 16.4 (dynamic stake) The proof-of-stake blockchain we described in this chapter is still missing a lot of bells and whistles which are in real systems. We mention one aspect here which is related to security. In a proof-of-stake blockchain the power in the lottery is related to the amount of cryptocurrency one has stored in the system, to ensure that those which run the system has a lot of stake in the system. However, cryptocurrency moves around. Maybe some party P who has a lot of cryptocurrency in the system sells it to some other party such that P now has no cryptocurrency at all. Before P had a lot of lottery power because it had a lot of cryptocurrency, but now it no longer has any cryptocurrency. So if we do not update its lottery power we have a party with a lot of lottery power but no stake. This is a bad idea. We can end up with the system being run solely by parties with no stake in it.

It seems easy to fix the problem. Simply let the lottery power of a party be proportional to its *current* amount of cryptocurrency in the system. There are, however, several subtle security problems with implementing this securely. We describe one of them here.

Consider an adversary which opens a lot of accounts vk_i , say 100 accounts. It can then mount the following attack. For each account vk_i the adversary can for all future slots $Slot$ determine the value $H(LOTTERY, Seed, slot, vk_i, Sig_{sk_i}(LOTTERY, Seed, slot))$ and find the account $vk_{max} = vk_i$ where $H(LOTTERY, Seed, slot, vk_i, Sig_{sk_i}(LOTTERY, Seed, slot))$ is maximal. After that it can then move all its cryptocurrency to this account to make $Tickets'_{max} = \sum_i Tickets_i$. Let us say that it has time to do so before the system reaches $Slot$. If it uses a lot of accounts this can make $Tickets'_{max} \cdot H(LOTTERY, Seed, slot, vk_{max}, Sig_{sk_{max}}(LOTTERY, Seed, slot))$ much larger than if it had had to put all the money on a single account before learning which account was lucky. So now the adversary wins more often than it should. This could lead to the adversary winning more often than the honest parties and the demise of the system.

1. Consider an adversary which has so much cryptocurrency that if it left it on a single account this account would win with probability 1%. Say it mounts the above attack using 100 accounts and the same amount of stake. What is the probability that the adversary wins the lottery with at least one of these accounts?
2. How could you combat the above attack while still allowing to update the stake? Hint: It was important for the attack that the adversary could compute the luck of an account ahead of time and have time to move its cryptocurrency when it learned which account was lucky. What if you could ensure that one cannot compute the luck too far into the future and then at that point in the future used a cryptocurrency distribution from before the luck could be predicted.



Chapter 17

Back Material

Bibliography

- [1] Michael Ben-Or. Another advantage of free choice: Completely asynchronous agreement protocols (extended abstract). In Robert L. Probert, Nancy A. Lynch, and Nicola Santoro, editors, *Proceedings of the Second Annual ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing, Montreal, Quebec, Canada, August 17-19, 1983*, pages 27–30. ACM, 1983.
- [2] Michael Ben-Or, Ran Canetti, and Oded Goldreich. Asynchronous secure computation. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing (STOC)*, pages 52–61. ACM, 1993.
- [3] Gabriel Bracha. An asynchronous $[(n-1)/3]$ -resilient consensus protocol. In Tiko Kameda, Jayadev Misra, Joseph G. Peters, and Nicola Santoro, editors, *Proceedings of the Third Annual ACM Symposium on Principles of Distributed Computing, Vancouver, B. C., Canada, August 27-29, 1984*, pages 154–162. ACM, 1984.
- [4] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA*, pages 136–145. IEEE Computer Society, 2001.
- [5] Bernardo David, Bernardo Magri, Christian Matt, Jesper Buus Nielsen, and Daniel Tschudi. Gearbox: Optimal-size shard committees by leveraging the safety-liveness dichotomy. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 683–696. ACM, 2022.
- [6] Danny Dolev and H. Raymond Strong. Authenticated algorithms for byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.
- [7] Cynthia Dwork, Nancy A. Lynch, and Larry J. Stockmeyer. Consensus in the presence of partial synchrony. *J. ACM*, 35(2):288–323, 1988.
- [8] Paul Feldman and Silvio Micali. Optimal algorithms for byzantine agreement. In Janos Simon, editor, *Proceedings of the 20th Annual ACM Symposium on Theory of Computing, May 2-4, 1988, Chicago, Illinois, USA*, pages 148–161. ACM, 1988.
- [9] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.
- [10] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice-Hall, 1985.
- [11] Ueli Maurer. Constructive cryptography - A new paradigm for security definitions and proofs. In Sebastian Mödersheim and Catuscia Palamidessi, editors, *Theory of Security and Applications - Joint Workshop, TOSCA 2011, Saarbrücken, Germany, March 31 -*

April 1, 2011, Revised Selected Papers, volume 6993 of *Lecture Notes in Computer Science*, pages 33–56. Springer, 2011.

- [12] Jakob Nielsen. Response times: The 3 important limits. Nielsen Norman Group, 1993 (updated later). <https://www.nngroup.com/articles/response-times-3-important-limits/>.
- [13] Jesper Buus Nielsen. A threshold pseudorandom function construction and its applications. In Moti Yung, editor, *Advances in Cryptology - CRYPTO 2002, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 2002, Proceedings*, volume 2442 of *Lecture Notes in Computer Science*, pages 401–416. Springer, 2002.

17.1 List of Exercise

readingguide.1theorem.1.1theorem.1.2	
Exercise 1.1 Generalising Replication to More Parties	17
exers.1.1theorem.1.3	
Exercise 1.2 Crash recovery	20
exers.1.2	
Exercise 1.3 trace properties I	23
exers.1.3	
Exercise 1.4 trace properties II	23
exers.1.4	
Exercise 1.5 detecting fail-silent crashes	25
exers.1.5readingguide.2theorem.2.1	
Exercise 2.1 probabilistic solution 1	46
exers.2.1	
Exercise 2.2 probabilistic solution 2	46
exers.2.2	
Exercise 2.3 more crashes	46
exers.2.3	
Exercise 2.4 cleaning up	46
exers.2.4theorem.2.2theorem.2.3theorem.2.4theorem.2.5theorem.2.6theorem.2.7theorem.2.8theorem.2.9read	
Exercise 3.1 Shared Variable Space from Message Passing	53
exers.3.1	
Exercise 3.2 putting the pieces together	70
exers.3.2	
Exercise 3.3 diameter, connectivity	80
exers.3.3theorem.3.1	
Exercise 3.4 Warm-up: TCP Connections and Message Passing in Go	83
exers.3.4readingguide.4attackmodel.1theorem.4.1theorem.4.2	
Exercise 4.1 properties of causality sets	105
exers.4.1	
Exercise 4.2 properties of causal-past relation	106
exers.4.2theorem.4.3theorem.4.4theorem.4.5theorem.4.6	
Exercise 4.3 Causal Past	109
exers.4.3	
Exercise 4.4 Safety and liveness of Vector Clocks	110
exers.4.4theorem.4.7theorem.4.8	
Exercise 4.5 Lamport Clocks	125
exers.4.5	
Exercise 4.6 Peer-to-Peer Flooding and a Simple Ledger	126
exers.4.6	
Exercise 4.7 better total-order	130
exers.4.7	
Exercise 4.8 Causal Regions	131
exers.4.8	
Exercise 4.9 Causal Present I	131
exers.4.9	
Exercise 4.10 Causal Present II	131
exers.4.10	

Exercise 4.11 Causal Order	131
exers.4.11	
Exercise 4.12 Implementing Chandy-Lamport Snapshotting	132
exers.4.12	
Exercise 4.13 Helping Einstein	133
exers.4.13	
Exercise 4.14 Causal Communication	133
exers.4.14theorem.4.9readingguide.5theorem.5.1theorem.5.2theorem.5.3	
Exercise 5.1 One-Time Pad Theory	157
exers.5.1	
Exercise 5.2 When to use Encryption?	158
exers.5.2	
Exercise 5.3 Insecurity of Deterministic Encryption	158
exers.5.3	
Exercise 5.4 CBC Decryption	158
exers.5.4	
Exercise 5.5 CBC-mode with predictable IV	158
exers.5.5	
Exercise 5.6 On The Need For Padding in RSA	158
exers.5.6	
Exercise 5.7 DES vs. AES Security	159
exers.5.7	
Exercise 5.8 Meet-in-The Middle Attack	159
exers.5.8	
Exercise 5.9 Permutation Ciphers	159
exers.5.9	
Exercise 5.10 Understanding Modular Arithmetic	159
exers.5.10	
Exercise 5.11 RSA and AES Encryption	159
exers.5.11	
Exercise 5.12 Implementing a Secure Channel I*	161
exers.5.12	
Exercise 5.13 Implementing a Secure Channel II*	161
exers.5.13readingguide.6theorem.6.1theorem.6.2theorem.6.3theorem.6.4	
Exercise 6.1 Enc-then-sign or Sign-then-encrypt	179
exers.6.1	
Exercise 6.2 CBC-MAC Padding	180
exers.6.2	
Exercise 6.3 Merkle-Damgård Construction	180
exers.6.3	
Exercise 6.4 HMAC and Length-Extension Attack	181
exers.6.4	
Exercise 6.5 Hash For Authentication	181
exers.6.5	
Exercise 6.6 Replay Attacks	181
exers.6.6	
Exercise 6.7 Not a Signature	182
exers.6.7	
Exercise 6.8 RSA Malleability	182

exers.6.8	
Exercise 6.9 Hash-and-Sign	182
exers.6.9	
Exercise 6.10 Combining Encryption and MACs	182
exers.6.10	
Exercise 6.11 CRC in WEP	183
exers.6.11	
Exercise 6.12 Certificates Authorities	183
exers.6.12	
Exercise 6.13 Hash Trees	183
exers.6.13	
Exercise 6.14 CR $\Rightarrow$ OWF	184
exers.6.14	
Exercise 6.15 RSA Signatures	184
exers.6.15	
Exercise 6.16 Authenticated Transactions in a Peer-to-Peer Ledger	186
exers.6.16readingguide.7	
Exercise 7.1 precise guarantee	199
exers.7.1	
Exercise 7.2 different up and down latencies	199
exers.7.2	
Exercise 7.3 quorum clock synchronisation I	199
exers.7.3	
Exercise 7.4 quorum clock synchronisation II	199
exers.7.4readingguide.8attackmodel.2theorem.8.1theorem.8.2theorem.8.3theorem.8.4	
Exercise 8.1 Dolev-Strong I	223
exers.8.1	
Exercise 8.2 Dolev-Strong II	223
exers.8.2	
Exercise 8.3 Dolev-Strong III	226
exers.8.3	
Exercise 8.4 Dolev-Strong IV	226
exers.8.4	
Exercise 8.5 Dolev-Strong V	226
exers.8.5	
Exercise 8.6 Impossibility for more parties	226
exers.8.6	
Exercise 8.7 Message complexity of Dolev-Strong	226
exers.8.7	
Exercise 8.8 Better Reliable Broadcast from BA	226
exers.8.8readingguide.9	
Exercise 9.1 Yet Another AKE Protocol	245
exers.9.1	
Exercise 9.2 Needham-Schroeder	246
exers.9.2	
Exercise 9.3 Liveness Check	246
exers.9.3	
Exercise 9.4 Weak Parameters for Diffie-Hellman	247
exers.9.4	

Exercise 9.5	Variants of DH-AKE	247
exers.9.5		
Exercise 9.6	Password vs. Passkeys	247
exers.9.6		
Exercise 9.7	Salt	248
exers.9.7		
Exercise 9.8	Using Secure Hardware	248
exers.9.8		
Exercise 9.9	Multiuser Security	249
exers.9.9		
Exercise 9.10	Rainbow Tables	249
exers.9.10		
Exercise 9.11	Software Wallet	250
exers.9.11	attackmodel.3theorem.10.1theorem.10.2	
Exercise 10.1	Breaking the Bracha Rules	297
exers.10.1		
Exercise 10.2	Concrete runs of Bracha	297
exers.10.2		
Exercise 10.3	Bracha Bug	299
exers.10.3		
Exercise 10.4	Concrete run of SVUP	299
exers.10.4		
Exercise 10.5	Breaking the SVUP Rules	299
exers.10.5		
Exercise 10.6	π-SVUP	299
exers.10.6		
Exercise 10.7	SVUP Bug	299
exers.10.7		
Exercise 10.8	Stability Preservation of SVUP	301
exers.10.8		
Exercise 10.9	TVA of SVUP	301
exers.10.9		
Exercise 10.10	Ready?	301
exers.10.10		
Exercise 10.11	Optimality of the Corruption Bound, ARB	301
exers.10.11		
Exercise 10.12	Optimality of the Corruption Bound, ABA	301
exers.10.12		
Exercise 10.13	Possible or Impossible I	302
exers.10.13		
Exercise 10.14	Possible or Impossible II	302
exers.10.14		
Exercise 10.15	Possible or Impossible III	303
exers.10.15		
Exercise 10.16	When did it Happen	303
exers.10.16		
Exercise 10.17	Multi-Valued ABA I	303
exers.10.17		
Exercise 10.18	Multi-Valued ABA II	304

exers.10.18readingguide.10	
Exercise 11.1 Private Set Intersection	320
exers.11.1	
Exercise 11.2 Number of Servers in Anonymous Communication	320
exers.11.2	
Exercise 11.3 Security of e-Voting Systems	320
exers.11.3	
Exercise 11.4 Groups (Algebra)	321
exers.11.4	
Exercise 11.5 Attacks on Deniability	321
exers.11.5	
Exercise 11.6 Zero-Knowledge Proofs	322
exers.11.6	
Exercise 11.7 Deniability in Off-The-Record Messaging	322
exers.11.7	
Exercise 11.8 Threshold Secret-Sharing	322
exers.11.8	
Exercise 11.9 Homomorphic Encryption	323
exers.11.9	
Exercise 11.10 Differential Privacy of Randomized Responses	323
exers.11.10	
Exercise 11.11 Composition of Differential-Privacy	324
exers.11.11readingguide.11theorem.12.1	
Exercise 12.1	346
exers.12.1	
Exercise 12.2	346
exers.12.2	
Exercise 12.3	346
exers.12.3	
Exercise 12.4	346
exers.12.4	
Exercise 12.5	346
exers.12.5	
Exercise 12.6 Electronic Election 1	347
exers.12.6readingguide.12	
Exercise 13.1	364
exers.13.1	
Exercise 13.2	364
exers.13.2	
Exercise 13.3	364
exers.13.3	
Exercise 13.4	364
exers.13.4	
Exercise 13.5	364
exers.13.5	
Exercise 13.6	364
exers.13.6	
Exercise 13.7	364
exers.13.7	

Exercise 13.8 Electronic Election 2	364
exers.13.8attackmodel.4theorem.14.1theorem.14.2	
Exercise 14.1 Total Order by Sequencer (DEPRECATED)	390
exers.14.1	
Exercise 14.2 Client-Centric Consistency I	391
exers.14.2	
Exercise 14.3 Client-Centric Consistency II	392
exers.14.3theorem.14.3theorem.14.4	
Exercise 14.4 TOB from SMR	393
exers.14.4	
Exercise 14.5 Synchronous total-order Broadcast (I)	393
exers.14.5	
Exercise 14.6 Synchronous total-order Broadcast (II)	393
exers.14.6	
Exercise 14.7 Asynchronous total-order Broadcast (I)	394
exers.14.7	
Exercise 14.8 Asynchronous total-order Broadcast (II)	394
exers.14.8	
Exercise 14.9 Not BCG I	396
exers.14.9	
Exercise 14.10 Not BCG II	396
exers.14.10	
Exercise 14.11 SMR—Empty Runs	396
exers.14.11	
Exercise 14.12 SMR—Strong EA	397
exers.14.12theorem.14.5theorem.14.6theorem.14.7theorem.14.8theorem.14.9theorem.14.10theorem.14.11att	
Exercise 16.1 worse chain quality	429
exers.16.1theorem.16.11theorem.16.12	
Exercise 16.2 Static Proof-of-Stake	434
exers.16.2	
Exercise 16.3 better lookback bound*	437
exers.16.3	
Exercise 16.4 dynamic stake	437
exers.16.4	

